CHƯƠNG 1: MATLAB CƠ BẢN

§1. CÁC TOÁN TỬ CƠ BẢN CỦA MATLAB

1. Các toán tử cơ bản: Matlab là một phần mềm cao cấp dùng để giải các bài toán. Để khởi động MATLAB ta bấm đúp vào icon của nó. Các file MATLAB có dạng *.m và chỉ chạy trong môi trường MATLAB. MATLAB xử lí số liệu như là ma trận. Khi ta đánh lệnh vào cửa sổ lệnh, nó sẽ được thi hành ngay và kết quả hiện lên màn hình. Nếu ta không muốn cho kết quả hiện lên màn hình thì sau lệnh ta đặt thêm dấu ";". Nếu lệnh quá dài, không vừa một dòng dòng có thể đánh lệnh trên nhiều dòng và cuối mỗi dòng đặt thêm dấu ... rồi xuống dòng. Khi soạn thảo lệnh ta có thể dùng các phím tắt :

↑	Ctrl-P	gọi lại lệnh trước đó
↓	Ctrl-N	gọi lệnh sau
←	Ctrl-B	lùi lại một kí tự
→	Ctrl-F	tiến lên một kí tự
Ctrl-→	Ctrl-R	sang phải một từ
Ctrl-←	Crtl-L	sang phải một từ
home	Ctrl-A	về đầu dòng
end	Ctrl-E	về cuối dòng
esc	Ctrl-U	xoá dòng
del	Ctrl-D	xoá kí tự tại chỗ con nháy đứng
backspace	Ctrl-H	xoá kí tự trước chỗ con nháy đứng

☞ Các phép toán cơ bản của MATLAB gồm:

+	cộng
-	trừ
*	nhân
/	chia phải
\	chia trái
^	luỹ thừa
‘	chuyển vị ma trận hay số phức liên hợp

☞ Các toán tử quan hệ :

<	nhỏ hơn
<=	nhỏ hơn hay bằng
>	lớn hơn
>=	lớn hơn hoặc bằng
==	bằng
~=	không bằng

☞ Các toán tử logic :

&	và
|	or
~	not

☞ Các hằng :

pi	3.14159265
i	số ảo
j	tương tự i
eps	sai số 2^{-52}
realmin	số thực nhỏ nhất 2^{-1022}
realmax	số thực lớn nhất 2^{1023}
inf	vô cùng lớn
NaN	Not a number

2. Nhập xuất dữ liệu từ dòng lệnh: MATLAB không đòi hỏi phải khai báo biến trước khi dùng. MATLAB phân biệt chữ hoa và chữ thường. Các số liệu đưa vào môi trường làm việc của MATLAB được lưu lại suốt phiên làm việc cho đến khi gặp lệnh *clear all*. MATLAB cho phép ta nhập số liệu từ dòng lệnh. Khi nhập ma trận từ bàn phím ta phải tuân theo các quy định sau :

- ngăn cách các phần tử của ma trận bằng dấu "," hay dấu trống
- dùng dấu ";" để kết thúc một hàng
- bao các phần tử của ma trận bằng cặp dấu ngoặc vuông [ ]

Để nhập các ma trận sau:

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 3 & -2 & 5 \\ 1 & 5 & 3 \end{bmatrix} \qquad B = \begin{bmatrix} 1 & 4 & -2 & 1 \end{bmatrix} \qquad C = \begin{bmatrix} 1 \\ 4 \\ 7 \end{bmatrix}$$

ta dùng các lệnh:

A = [*1 2 3; 3 -2 4; 1 5 3*]
B = [*1 4 2 1*]
C = [*1; 4; 7*]

3. Nhập xuất dữ liệu từ file: MATLAB có thể xử lí hai kiểu file dữ liệu: file nhị phân *.mat và file ASCII *.dat. Để lưu các ma trận A, B, C dưới dạng file nhị phân ta dùng lệnh:

save ABC A B C

và nạp lại các ma trận A, B bằng lệnh:

load ABC A B

Nếu muốn lưu số liệu của ma trận B dưới dạng file ASCII ta viết:

save b.dat B /ascii

Ta viết chương trình ***ct1_1.m*** như sau:

```
clear
A = [1 2 3; 4 5 6]
B = [3; -2; 1];
C(2) = 2; C(4) = 4
disp('Nhan phim bat ky de xem nhap/xuat du lieu tu file')
save ABC A B C %luu A,B & C duoi dang MAT-file co ten 'ABC.mat'
clear('A', 'C') %xoa  A va C khoi bo nho
load ABC A C %doc MAT - file de nhap A va C vao bo nho
save b.dat B /ascii %luu B duoi dang file ASCII co ten 'b.dat'
clear B
load b.dat %doc ASCII
b
x = input('Nhap x:')
format short e
x
format rat, x
format long, x
format short, x
```

4. Nhập xuất dữ liệu từ bàn phím: Lệnh ***input*** cho phép ta nhập số liệu từ bàn phím. Ví dụ:

```
x = input('Nhap x:  ')
```

Lệnh ***format*** cho phép xác định dạng thức của dữ liệu. Ví dụ:

```
format rat % so huu ti
format long % so sẽ có 14 chu so sau dau phay
format long e % so dang mu
format hex % so dang hex
format short e %so dang mu ngan
format short %tro ve so dang ngan (default)
```

Một cách khác để hiển thị giá trị của biến và chuỗi là đánh tên biến vào cửa sổ lệnh MATLAB. Ta cũng có thể dùng ***disp*** và ***fprintf*** để hiển thị các biến. Ví dụ:

```
disp('Tri so cua  x = '), disp(x)
```

Ta viết chương trình ***ct1_2.m*** như sau:

```
clc
f = input('Nhap nhiet do Fahrenheit[F]:');
c = 5/9*(f - 32);
fprintf('%5.2f(do Fahrenheit) la %5.2f(do C).\n', f, c)
fid = fopen('ct1_2.dat', 'w');
fprintf(fid, '%5.2f(do Fahrenheit) la %5.2f(do C).\n', f, c);
fclose(fid);
```

Trong trường hợp ta muốn nhập một chuỗi từ bàn phím, ta cần phải thêm kí tự s vào đối số. Ví dụ:

```
ans = input('Ban tra loi  <co> hoac  <khong>: ','s')
```

5. Các hàm toán học:

a. Các hàm toán học cơ bản:

exp(x) hàm e^x
sqrt(x) căn bậc hai của x
log(x) logarit tự nhiên
log10(x) logarit cơ số 10
abs(x) modun của số phức x
angle(x) argument của số phức a
conj(x) số phức liên hợp của x
imag(x) phần ảo của x
real(x) phần thực của x
sign(x) dấu của x
cos(x)
sin(x)
tan(x)
acos(x)
asin(x)
atan(x)
cosh(x)
coth(x)
sinh(x)
tanh(x)
acosh(x)
acoth(x)
asinh(x)
atanh(x)

b. Các hàm toán học tự tạo: MATLAB cho phép ta tạo hàm toán học và lưu nó vào một file để dùng như là hàm có sẵn của MATLAB. Ví dụ ta cần tạo hàm:

$$f_1(x) = \frac{1}{1+8x^2}$$

và hàm:

$$f_2(x) = \begin{bmatrix} f_1(x_1, x_2) \\ f_2(x_1, x_2) \end{bmatrix} = \begin{bmatrix} x_1^2 + 4x_2^2 - 5 \\ 2x_1^2 - 2x_1 - 3x_2 - 2.5 \end{bmatrix}$$

Muốn thế ta tạo ra file ***f1.m*** như sau:

```
function y = f1(x)
y = 1./(1+8*x.^2);
```

và file ***f2.m***:

```
function y = f2(x)
y(1) = x(1)*x(1)+4*x(2)*x(2) -5;
y(2) = 2*x(1)*x(1)-2*x(1)-3*x(2) -2.5;
```

Khi nhập lệnh ***f1(2)*** ta có giá trị của hàm f1 tại x = 2. Khi nhập lệnh ***f2([2 4])*** ta có giá trị của hàm f2 tại $x_1 = 2$ và $x_2 = 4$. Lệnh ***feval***(‘***f1***’, ***2***) và ***feval***(‘***f2***’, [***2 4***]) cũng cho kết quả tương tự.

Cách thứ hai để biểu diễn một hàm toán học một biến trên dòng lệnh là tạo ra một đối tượng inline từ một biểu thức chuỗi. Ví dụ ta có thể nhập từ dòng lệnh hàm như sau:

```
f1 = inline('1./(1 + 8*x.^2)','x');
f1([0 1]), feval(f1, [0 1])
```

Ta cũng có thể viết:

```
f1 = '1./(1 + 8*x.^2)';
x = [0 1];
eval(f1)
```

Nếu hàm là đa thức ta chỉ cần nhập ma trận các hệ số từ số mũ cao nhất. Ví dụ với đa thức $P_4(x) = x^4 + 4x^3 + 2x + 1$ ta viết:

```
P = [1  4  0  2  1]
```

Để nhân hai đa thức ta dùng lệnh ***conv***; để chia 2 đa thức ta dùng lệnh ***deconv***. Muốn tính trị số của đa thức ta dùng lệnh ***polyval*** và lệnh ***polyvalm*** dùng khi đa thức là ma trận.

c. Các lệnh xử lí hàm: Lệnh ***fplot*** vẽ đồ thị hàm toán học giữa các giá trị đã cho. Ví dụ:

```
fplot('f1', [-5  5 ])
```

```
grid on
```

Cho một hàm toán học một biến, ta có thể dùng lệnh ***fminbnd*** của MATLAB để tìm cực tiểu địa phương của hàm trong khoảng đã cho. Ví dụ:

```
f = inline('1./((x - 0.3).^2+0.01) + 1./((x - 0.9).^2 + 0.04) - 6 ');
x = fminbnd(f, 0.3, 1)
```

Lệnh ***fminsearch*** tương tự hàm ***fminbnd*** dùng để tìm cực tiểu địa phương của hàm nhiều biến. Ta có hàm 3 biến lưu trong file ***three_var.m*** như sau:

```
function b = three_var(v)
    x = v(1);
    y = v(2);
    z = v(3);
    b = x.^2 + 2.5*sin(y) - z^2*x^2*y^2;
```

Bây giờ tìm cực tiểu đối với hàm này bắt đầu từ x = -0.6 , y = -1.2 và z = 0.135 bằng các lệnh:

```
v = [-0.6 -1.2  0.135];
a = fminsearch('three_var', v)
```

Lệnh ***fzero*** dùng để tìm điểm zero của hàm một biến. Ví dụ để tìm giá trị không của hàm lân cận giá trị -0.2 ta viết:

```
f = inline('1./((x - 0.3).^2 + 0.01) + 1./((x - 0.9).^2 + 0.04) - 6');
a = fzero(f, -0.2)

Zero found in the interval: [-0.10949, -0.264].
a =
     -0.1316
```

6. Các phép toán trên ma trận và vec tơ:

a. Khái niệm chung: Giả sử ta tạo ra các ma trận a và b bằng các lệnh:

```
a = [1  2  3; 4  5  6];
b = [3  -2  1];
```

Ta có thể sửa đổi chúng:

```
A = [a; 7 8 9]
B = [b; [1 0 -1]]'
```

Toán tử ‘ dùng để chuyển vị một ma trận thực và chuyển vị liên hợp một ma trận phức. Nếu chỉ muốn chuyển vị ma trận phức, ta dùng thêm toán tử “.” nghĩa là phải viết “.’”. Ví dụ:

C = [*1 + 2*i 2 - 4*i; 3 + i 2 - 2*j*];
X = *C′*
Y = *C.’*

b. Chỉ số: Phần tử ở hàng i cột j của ma trận m×n có kí hiệu là A(i, j). Tuy nhiên ta cũng có thể tham chiếu tới phần tử của mảng nhờ một chỉ số, ví dụ A(k) với k = i + (j - 1)m. Cách này thường dùng để tham chiếu vec tơ hàng hay cột. Trong trường hợp ma trận đầy đủ thì nó được xem là ma trận một cột dài tạo từ các cột của ma trận ban đầu. Như vậy viết A(5) có nghĩa là tham chiếu phần tử A(2, 2).

Để xác định kích thước của một ma trận ta dùng lệnh ***length***(trả về kích thước lớn nhất) hay ***size***(số hàng và cột). Ví dụ:

c = [*1 2 3 4; 5 6 7 8*];
length(c)
[*m, n*] = *size(c)*

c. Toán tử “:” : Toán tử “:” là một toán tử quan trọng của MATLAB. Nó xuất hiện ở nhiều dạng khác nhau. Ví dụ:

1:10

tạo một vec tơ hàng chứa 10 số nguyên từ 1 đến 10. Lệnh:

100: -7: 50

tạo một dãy số từ 100 đến 51, giảm 7 mỗi lần. Lệnh:

0: pi/4: pi

tạo một dãy số từ 0 đến pi, cách đều nhau pi/4

Các biểu thức chỉ số tham chiếu tới một phần của ma trận. Viết A(1:k, j) là tham chiếu đến k phần tử đầu tiên của cột j. Ngoài ra toán tử “:” tham chiếu tới tất cả các phần tử của một hàng hay một cột. Ví dụ:

B = *A*(:, [*1 3 2*])

tạo ra ma trận B từ ma trận A bằng cách đổi thứ tự các cột từ [1 2 3] thành [1 3 2]

d. Tạo ma trận bằng hàm có sẵn: MATLAB cung cấp một số hàm để tạo các ma trận cơ bản:

zeros tạo ra ma trận mà các phần tử đều là zeros

z = zeros(2, 4)

ones tạo ra ma trận mà các phần tử đều là 1

x = ones(2, 3)
*y = 5*ones(2, 2)*

rand tạo ra ma trận mà các phần tử ngẫu nhiên phân bố đều

d = rand(4, 4)

randn tạo ra ma trận mà các phần tử ngẫu nhiên phân bố trực giao

e = randn(3, 3)

magic(n) tạo ra ma trận cấp n gồm các số nguyên từ 1 đến n^2 với tổng các hàng bằng tổng các cột n phải lớn hơn hay bằng 3.

pascal(n) tạo ra ma trận xác định dương mà các phần tử lấy từ tam giác Pascal.

pascal(4)

eye(n) tạo ma trận đơn vị

eye(3)

eye(m, n) tạo ma trận đơn vị mở rộng

eye(3, 4)

e. Lắp ghép: Ta có thể lắp ghép(concatenation) các ma trận có sẵn thành một ma trận mới. Ví dụ:

a = ones(*3, 3*)
*b = 5*ones*(*3, 3*)
c = [*a + 2*; *b*]

f. Xoá hàng và cột : Ta có thể xoá hàng và cột từ ma trận bằng dùng dấu []. Để xoá cột thứ 2 của ma trận b ta viết:

b(*:, 2*) = []

Viết *x*(*1: 2: 5*) = [] nghĩa là ta xoá các phần tử bắt đầu từ đến phần tử thứ 5 và cách 2 rồi sắp xếp lại ma trận.

g. Các lệnh xử lí ma trận:

Cộng : X= A + B

Trừ : X= A - B

Nhân : X= A * B

: X.*A nhân các phần tử tương ứng với nhau

Chia : X = A/B lúc đó X*B = A

: X = A\B lúc đó A*X = B

: X=A./B chia các phần tử tương ứng với nhau

Luỹ thừa : X = A^2

: X = A.^2

Nghịch đảo : X = inv(A)

Định thức : d = det(A)

7. Tạo số ngẫu nhiên: MATLAB có các lệnh tạo số ngẫu nhiên là ***rand*** và ***randn*** tạo ra các số ngẫu nhiên theo phân bố Gauss.

rand(m, n) tạo ra ma trận các số ngẫu nhiên phân bố đồng nhất.

randn(m, n) tạo ra ma trận các số ngẫu nhiên theo phân bố chuẩn Gauss.

rand(3, 3)
randn(3, 3)

8. Các lệnh dùng lập trình:

a. Các phát biểu điều kiện if, else, elseif:

Cú pháp của if:

```
if <biểu thức điều kiện>
        <phát biểu>
end
```

Nếu <biểu thức điều kiện> cho kết quả đúng thì phần lệnh trong thân của if được thực hiện.

Các phát biểu else và leseif cũng tương tự.

Ví dụ: Ta xét chương trình) ***ct1_4. m*** để đoán tuổi như sau:

```
clc
disp('Xin chao! Han hanh duoc lam quen');
x = fix(30*rand);
disp('Tuoi toi trong khoang 0 - 30');
gu = input('Xin nhap tuoi cua ban:  ');
if gu < x
    disp('Ban tre hon toi');
 elseif gu > x
```

```
    disp('Ban lon hon toi');
else
    disp('Ban bang tuoi toi');
end
```

b. switch: Cú pháp của ***switch*** như sau :

```
switch <biểu thức>
        case n1 : <lệnh 1>
        case n2 : <lệnh 2>
        . . . . . . . . . . . . . .
        case nn : <lệnh n>
        otherwise : <lệnh n+1>
end
```

c. while: vòng lặp while dùng khi không biết trước số lần lặp. Cú pháp của nó như sau:

```
while <biểu thức>
        <phát biểu>
end
```

Xét chương trình in ra chuoi "Xin chao" lên mà hình với số lần nhập từ bàn phím ***ct1_5.m*** như sau:

```
clc
disp('xin chao');
gu = input('Nhap so lan in: ');
i = 0;
while i ~= gu
    disp(['Xin chao' i]);
    i = i + 1
end
```

d. for: vòng lặp for dùng khi biết trước số lần lặp. Cú pháp như sau:

```
for <chỉ số> = <giá trị đầu> : <mức tăng> : <giá trị cuối>
```

Ta xây dựng chương trình đoán số ***ct1_6.m***:

```
clc
x = fix(100*rand);
n = 7;
t = 1;
for k = 1:7
  num = int2str(n);
  disp(['Ban co quyen du doan ', num, '  lan']);
  disp('So can doan nam trong khoang 0 - 100');
  gu = input('Nhap so ma ban doan: ');
```

```
  if gu < x
    disp('Ban doan nho hon');
  elseif gu > x
    disp('So ban doan lon hon');
  else
    disp('Ban da doan dung. Xin chuc mung');
    t = 0;
    break;
  end
  n = n - 1;
end
if t > 0
  disp('Ban khong doan ra roi');
  numx = int2str(x);
  disp(['Do la so: ', numx]);
end
```

e. break: phát biểu ***break*** để kết thúc vòng lặp ***for*** hay ***while*** mà không quan tâm đến điều kiện kết thúc vòng lặp đã thoả mãn hay chưa.

§2. ĐỒ HOẠ TRONG MATLAB

1. Các lệnh vẽ: MATLAB cung cấp một loạt hàm để vẽ biểu diễn các vec tơ số liệu cũng như giải thích và in các đường cong này.

plot đồ họa 2-D với số liệu 2 trục vô hướng và tuyến tính
plot3 đồ họa 3-D với số liệu 2 trục vô hướng và tuyến tính
polar đồ hoạ trong hệ toạ độ cực
loglog đồ hoạ với các trục logarit
semilogx đồ hoạ với trục x logarit và trục y tuyến tính
semilogy đồ hoạ với trục y logarit và trục x tuyến tính
plotyy đồ hoạ với trục y có nhãn ở bên trái và bên phải

2. Tạo hình vẽ: Hàm ***plot*** có các dạng khác nhau phụ thuộc vào các đối số đưa vào. Ví dụ nếu y là một vec tơ thì ***plot(y)*** tạo ra một đường thẳng quan hệ giữa các giá trị của y và chỉ số của nó. Nếu ta có 2 vec tơ x và y thì ***plot(x, y)*** tạo ra đồ thị quan hệ giữa x và y.

```
t = [0: pi/100: 2*pi]
y = sin(t);
plot(t, y)
grid on
polar(t, y)
```

3. Đặc tả kiểu đường vẽ: Ta có thể dùng các kiểu đường vẽ khác nhau khi vẽ hình. Muốn thế ta chuyển kiểu đường vẽ cho hàm ***plot***. Ta viết chương trình ***ct1_7.m*** tạo ra đồ thị hàm hình sin:

```
t = [0: pi/100: 2*pi];
y = sin(t);
plot(t, y, '. ') % vẽ bằng đường chấm chấm
grid on
```

4. Đặc tả màu và kích thước đường vẽ: Để đặc tả màu và kích thước đường vẽ ta dùng các tham số sau:

LineWidth độ rộng đường thẳng,tính bằng số điểm
MarkerEdgeColor màu của các cạnh của khối đánh dấu
MarkerFaceColor màu của khối đánh dấu
MarkerSize kích thước của khối đánh dấu

Màu được xác định bằng các tham số:

Mã	Màu	Mã	Màu
r	red	m	magenta
g	green	y	yellow
b	blue	k	black
c	cyan	w	white

Các dạng điểm đánh dấu xác định bằng:

Mã	Kiểu đánh dấu	Mã	Kiểu đánh dấu
+	dấu cộng	.	điểm
o	vòng tròn	x	chữ thập
*	dấu sao	s	hình vuông
d	hạt kim cương	v	điểm tam giác hướng xuống
^	điểm tam giác hướng lên	<	tam giác sang trái
>	tam giác sang phải	h	lục giác
p	ngũ giác		

Các dạng đường thẳng xác định bằng:

Mã	Kiểu đường	Mã	Kiểu đường
-	đường liền	:	đường chấm chấm
--	đường đứt nét	-.	đường chấm gạch

Ta xét chương trình ***ct1_8.m*** như sau:

```
x = -pi : pi/10 : pi;
```

```
y = tan(sin(x)) - sin(tan(x));
plot(x, y, '--rs', 'LineWidth', 2, 'MarkerEdgeColor', 'k',...
      'MarkerFaceColor', 'g', 'MarkerSize', 10)
```

Chương trình này sẽ vẽ đường cong y = f(x) có các đặc tả sau :

- đường vẽ là đường đứt nét(--)
- khối đánh dấu hình vuông (s), đường vẽ màu đỏ(r)
- đường vẽ rộng 2 point
- các cạnh của khối đánh màu đen
- khối đánh dấu màu green
- kích thước khối đánh dấu 10 point

5. Thêm đường vẽ vào đồ thị đã có: Để làm điều này ta dùng lệnh ***hold***. Khi ta đánh lệnh ***hold on*** thì MATLAB không xoá đồ thị đang có. Nó thêm số liệu vào đồ thị mới này. Nếu phạm vi giá trị của đồ thị mới vượt quá các giá trị của trục toạ độ cũ thì nó sẽ định lại tỉ lệ xích.

6. Chỉ vẽ các điểm số liệu: Để vẽ các điểm đánh dấu mà không nối chúng lại với nhau ta dùng đặc tả nói rằng không có các đường nối giữa các điểm, nghĩa là ta gọi hàm ***plot*** chỉ với đặc tả màu và điểm đánh dấu. Ta xét chương trình ***ct1_9.m*** như sau:

```
x = -pi : pi/10 : pi;
y = tan(sin(x)) - sin(tan(x));
plot(x, y, 's', 'MarkerEdgeColor', 'k')
```

7. Vẽ các điểm và đường: Để vẽ cả các điểm đánh dấu và đường nối giữa chúng ta cần mô tả kiểu đường và kiểu điểm. Ta xét chương trình ***ct1_10.m***:

```
x = 0:pi/15:4*pi;
y = exp(2*sin(x));
plot(x, y, '-r', x, y, 'ok')
```

dùng vẽ đường cong y = f(x) có đường nối liền, màu đỏ. Điểm đánh dấu là chữ o có màu đen.

8. Vẽ với hai trục y: Lệnh ***plotyy*** cho phép tạo một đồ thị có hai trục y. Ta cũng có thể dùng ***plotyy*** để cho giá trị trên hai trục y có kiểu khác nhau nhằm tiện so sánh. Ta xét chương trình ***ct1_11.m***:

```
t = 0:900;
A = 1000;
b = 0.005;
a = 0.005;
```

```
z2 = sin(b*t);
z1 = A*exp(-a*t);
[haxes, hline1, hline2] = plotyy(t, z1, t, z2,'semilogy', 'plot');
```

9. Vẽ đường cong với số liệu 3 - D: Nếu x, y, z là 3 vec tơ có cùng độ dài thì ***plot3*** sẽ vẽ đường cong 3D. Ta viết chương trình ***ct1_12.m***:

```
t = 0:pi/50:10*pi;
plot3(sin(t),cos(t),t)
axis square;
grid on
```

10. Đặt các thông số cho trục: Khi ta tạo một hình vẽ, MATLAB tự động chọn các giới hạn trên trục toạ độ và khoảng cách đánh dấu dựa trên số liệu dùng để vẽ. Tuy nhiên ta có thể mô tả lại phạm vi giá trị trên trục và khoảng cách đánh dấu theo ý riêng. Ta có thể dùng các lệnh sau:

axis	đặt lại các giá trị trên trục toạ độ
axes	tạo một trục toạ độ mới với các đặc tính được mô tả
get và set	cho phép xác định và đặt các thuộc tính của trục toạ độ đang có
gca	trở về trục toạ độ cũ

MATLAB chọn các giới hạn trên trục toạ độ và khoảng cách đánh dấu dựa trên số liệu dùng để vẽ. Dùng lệnh ***axis*** có thể đặt lại giới hạn này. Cú pháp của lệnh:

axis[xmin , xmax , ymin , ymax]

Ta xét chương trình ***ct1_13.m*** như sau:

```
x = 0:0.025:pi/2;
plot(x, tan(x), '-ro')
axis([0 pi/2 0 5])
```

MATLAB chia vạch trên trục dựa trên phạm vi dữ liệu và chia đều. Ta có thể mô tả cách chia nhờ thông số ***xtick*** và ***ytick*** bằng một vec tơ tăng dần. Ví dụ xét chương trình ***ct1_14.m***:

```
x = -pi: .1: pi;
y = sin(x);
plot(x, y)
set(gca, 'xtick', -pi :pi/2:p);
set(gca, 'xticklabel', {'-pi', '-pi/2', '0', 'pi/2', 'pi'})
```

11. Ghi nhãn lên các trục toạ độ: MATLAB cung cấp các lệnh ghi nhãn lên đồ hoạ gồm :

title	thêm nhãn vào đồ hoạ

xlabel thêm nhãn vào trục x
ylabel thêm nhãn vào trục y
zlabel thêm nhãn vào trục z
legend thêm chú giải vào đồ thị
text hiển thị chuỗi văn bản ở vị trí nhất định
gtext đặt văn bản lên đồ hoạ nhờ chuột
\bf bold font
\it italics font
\sl oblique font (chữ nghiêng)
\rm normal font

Các kí tự đặc biệt xem trong ***String properties*** của Help.

Ta dùng các lệnh ***xlabel , ylabel , zlabel*** để thêm nhãn vào các trục toạ độ. Ta có thể thêm văn bản vào bất kì chỗ nào trên hình vẽ nhờ hàm text. Ta có chương trình ***ct1_15.m***:

```
x = -pi: .1: pi;
y = sin(x);
plot(x, y)
xlabel('t = 0 to 2\pi', 'Fontsize', 16)
ylabel('sin(t)', 'Fontsize', 16)
title('\it{Gia tri cua sin tu zero đến 2 pi}', 'Fontsize', 16)
text(3*pi/4, sin(3*pi/4),'\leftarrowsin(t ) = 0.707', 'FontSize', 12)
```

12. Định vị văn bản trên hình vẽ: Ta có thể sử dụng đối tượng văn bản để ghi chú các trục ở vị trí bất kì. MATLAB định vị văn bản theo đơn vị dữ liệu trên trục. Ví dụ để vẽ hàm $y = Ae^{\alpha t}$ với A = 0.25 , t = 0 đến 900 và α = 0.005 ta viết chương trình ***ct1_16.m***:

```
t = 0: 900;
plot(t, 0.25*exp(-0.005*t))
plot(t, y)
text(300, .25*exp(-.005*300),...
'\bullet\leftarrow\fontname{times}0.25{\ite}^{-   0.005{\itt}} tai,...
 {\itt} = 300', 'FontSize', 14)%ghi chu tai t = 300
```

Tham số ***HorizontalAlignment*** và ***VerticalAlignment*** định vị văn bản so với các toạ độ x, y, z đã cho.

13. Đồ hoạ đặc biệt:

a. Khối và vùng: Đồ hoạ khối và vùng biểu diễn số liệu là vec tơ hay ma trận. MATLAB cung cấp các hàm đồ hoạ khối và vùng :

bar hiển thị các cột của ma trận m*n như là m nhóm, mỗi nhóm có n bar

barh hiển thị các cột của ma trận m*n như là m nhóm, mỗi nhóm có n bar nằm ngang

bar3 hiển thị các cột của ma trận m*n như là m nhóm, mỗi nhóm có n bar dạng 3D

bar3h hiển thị các cột của ma trận m*n như là m nhóm, mỗi nhóm có n bar dạng 3D nằm ngang

Mặc định, mỗi phần tử của ma trận được biểu diễn bằng một bar. Ta xét chương trình ***ct1_17.m***:

```
y =   [5  2  1
       6  7  3
       8  6  3
       5  5  5
       1  5  8];
bar(y)
```

b. Mô tả dữ liệu trên trục: Ta dùng các hàm ***xlabel*** và ***ylabel*** để mô tả các dữ liệu trên trục. Ta xét chương trình ***ct1_18.m***:

```
nhdo = [29 23 27 25 20 23 23 27];
ngay = 0: 5: 35;
bar(ngay, nhdo)
xlabel('Ngay')
ylabel('Nhiet do (^{o}C)')
set(gca,'YLim',[15 30],'Layer','top')
grid on
set(gca,'YLim',[15 30])
```

Mặc định,phạm vi giá trị của trục y là từ 0 đến 30. Để xem nhiệt độ trong khoảng từ 15 đến 30 ta thay đổi phạm vi giá trị của trục y:

```
set(gca,'YLim',[15 30],'Layer','top')
```

và trên đồ thị, phạm vi giá trị của trục y đã thay đổi.

c. Xếp chồng đồ thị: Ta có thể xếp chồng số liệu trên đồ thị thanh bằng cách tạo ra một trục khác trên cùng một vị trí và như vậy ta có một trục y độc lập với bộ số liệu khác.

```
TCE = [515 420 370 250 135 120 60 20];
nhdo = [29 23 27 25 20 23 23 27];
ngay = 0:5:35;
bar(ngay, nhdo)
xlabel('Ngay')
```

ylabel('Nhiet do (^{o}C)')

Để xếp chồng một số liệu lên một đồ thị thanh ở trên, có trục thứ 2 ở cùng vị trí như trục thứ nhất ta viết:

h1 = gca;

và tạo trục thứ 2 ở vị trí trục thứ nhất trước nhất vẽ bộ số liệu thứ 2:

h2 = axes('Position',get(h1,'Position'));
plot(days,TCE,'LineWidth',3)

Để trục thứ 2 không gây trở ngại cho trục thứ nhất ta viết:

set(h2,'YAxisLocation','right','Color','none','XTickLabel',[])
set(h2,'XLim',get(h1,'XLim'),'Layer','top')

Để ghi chú lên đồ thị ta viết:

text(11,380,'Mat do','Rotation',--55,'FontSize',16)
ylabel('TCE Mat do (PPM)')
title('Xep chong do thi','FontSize',16)

(lưu trong ***ct1_19.m***)

d. ***Đồ hoạ vùng***: Hàm ***area*** hiển thị đường cong tạo từ một vec tơ hay từ một cột của ma trận. Nó vẽ các giá trị của một cột của ma trận thành một đường cong riêng và tô đầy vùng không gian giữa các đường cong và trục x. ta xét chương trình ***ct1_20.m***:

```
Y =  [5 1 2
      8 3 7
      9 6 8
      5 5 5
      4 2 3];
area(Y)
```

hiển thị đồ thị có 3 vùng, mỗi vùng một cột. Độ cao của mỗi đồ thị vùng là tổng các phần tử trong một hàng. Mỗi đường cong sau sử dụng đường cong trước làm cơ sở. Để hiển thị đường chia lưới ta dùng lệnh:

```
set(gca,'Layer','top')
set(gca,'XTick',1:5)
grid on
```

f. Đồ thị pie: Đồ thị pie hiển thị theo tỉ lệ phần trăm của một phần tử của một vec tơ hay một ma trận so với tổng các phần tử. Các lệnh ***pie*** và ***pie3*** tạo ra đồ thị 2D và 3D. ta xét chương trình ***ct1_21.m***:

```
X =  [19.3   22.1   51.6;
```

```
        34.2   70.3   82.4;
        61.4   82.9   90.8;
        50.5   54.9   59.1;
        29.4   36.3   47.0];
x = sum(X);
explode = zeros(size(x));
[c,offset] = max(x);
explode(offset) = 1;
h = pie(x,explode)
%A = [ 1 3 6];
%pie3(A)
```

Khi tổng các phần tử trong đối số thứ nhất bằng hay lớn hơn 1, ***pie*** và ***pie3*** chuẩn hoá các giá trị. Như vậy cho vec tơ x, mỗi phần có diện tích $x_i / \mathrm{sum}(x_i)$ với x_i là một phần tử của x. Giá trị được chuẩn hoá mô tả phần nguyên của mỗi vùng. Khi tổng các phần tử trong đối số thứ nhất nhỏ hơn 1, ***pie*** và ***pie3*** không chuẩn hoá các phần tử của vec tơ x. Chúng vẽ một phần pie.

```
x = [.19 .22 .41];
pie(x)
```

g. Làm hình chuyển động: Ta có thể tạo ra hình chuyển động bằng 2 cách

- tạo và lưu nhiều hình khác nhau và lần lượt hiển thị chúng
- vẽ và xoá liên tục một đối tượng trên màn hình,mỗi lần vẽ lại có sự thay đổi.

Với cách thứ nhất ta thực hiện hình chuyển động qua 3 bước:

- dùng hàm ***moviein*** để dành bộ nhớ cho một ma trận đủ lớn nhằm lưu các khung hình.
- dùng hàm ***getframes*** để tạo các khung hình.
- dùng hàm ***movie*** để hiển thị các khung hình.

Sau đây là ví dụ sử dụng ***movie*** để quan sát hàm ***fft(eye(n))***.Ta tạo chương trình ***ct1_22.m*** như sau :

```
axis equal
M = moviein(16, gcf);
set(gca, 'NextPlot', 'replacechildren')
h = uicontrol('style', 'slider', 'position',[100 10 500 20], 'Min', 1, 'Max', 16)
for j = 1:16
        plot(fft(eye(j + 16)))
        set(h, 'Value', j)
M(:, j) = getframe(gcf);
end
clf;
axes('Position', [0 0 1 1]);
```

movie(M, 30)

Bước đầu tiên để tạo hình ảnh chuyển động là khởi gán ma trận. Tuy nhiên trước khi gọi hàm ***moviein***, ta cần tạo ra các trục toạ độ có cùng kích thước với kích thước mà ta muốn hiển thị hình. Do trong ví dụ này ta hiển thị các số liệu cách đều trên vòng tròn đơn vị nên ta dùng lệnh ***axis equal*** để xác định tỉ lệ các trục. Hàm ***moviein*** tạo ra ma trận đủ lớn để chứa 16 khung hình. Phát biểu:

set(gca, 'NextPlot', 'replacechildren')

ngăn hàm ***plot*** đưa tỉ lệ các trục về ***axis normal*** mỗi khi nó được gọi. Hàm ***getframe*** không đối số trả lại các điểm ảnh của trục hiện hành ở hình hiện có. Mỗi khung hình gồm các số liệu trong một vec tơ cột. Hàm ***getframe(gcf)*** chụp toàn bộ phần trong của một cửa sổ hiện hành. Sau khi tạo ra hình ảnh ta có thể chạy chúng một số lần nhất định ví dụ 30 lần nhờ hàm ***movie(M, 30)*** .

Một phương pháp nữa để tạo hình chuyển động là vẽ và xoá, nghĩa là vẽ một đối tượng đồ hoạ rồi thay đổi vị trí của nó bằng cách thay đổi toạ độ x, y và z một lượng nhỏ nhờ một vòng lặp. Ta có thể tạo ra các hiệu ứng khác nhau nhờ các cách xoá hình khác nhau. Chúng gồm:

- none MATLAB không xoá đối tượng khi nó di chuyển
- background MATLAB xoá đối tượng bằng cách vẽ nó có màu nền
- xor MATLAB chỉ xoá đối tượng

Ta tạo ra M-file có tên là ***ct1_23.m*** như sau:

```
A = [ -8/3 0 0; 0 -10 10; 0 28 -1 ];
y = [35 -10 -7]';
h = 0.01;
p = plot3(y(1), y(2), y(3),'.', ...
'EraseMode', 'none', 'MarkerSize', 5);
axis([0 50 -25 25 -25 25])
hold on
for i = 1:4000
        A(1,3) = y(2);
        A(3,1) = -y(2);
        ydot = A*y;
        y = y + h*ydot;
        set(p, 'XData', y(1), 'YData', y(2), 'ZData', y(3)) % thay doi toa do
        drawnow
        i = i + 1;
end
```

13. Đồ hoạ 3D:

a.Các lệnh cơ bản: Lệnh ***mesh*** và ***surf*** tạo ra lưới và mặt 3D từ ma trận số liệu. Gọi ma trận số liệu là z mà mỗi phần tử của nó z(i, j) xác định tung độ của mặt thì ***mesh(z)*** tạo ra một lưới có màu thể hiện mặt z còn ***surf(z)*** tạo ra một mặt có màu z.

b. Đồ thị các hàm hai biến: Bước thứ nhất để thể hiện hàm 2 biến z=f(x,y) là tạo ma trận x và y chứa các toạ độ trong miền xác định của hàm. Hàm meshgrid sẽ biến đổi vùng xác định bởi 2 vec tơ x và y thành ma trận x và y. Sau đó ta dùng ma trận này để đánh giá hàm.

Ta khảo sát hàm ***sin(r)/r***. Để tính hàm trong khoảng -8 và 8 theo x và y ta chỉ cần chuyển một vec tơ đối số cho ***meshgrid***:

```
[x,y] = meshgrid(-8:.5:8);
r = sqrt(x.^2 + y.^2) + 0.005;
```

ma trận r chứa khoảng cách từ tâm của ma trận. Tiếp theo ta dùng hàm ***mesh*** để vẽ hàm.

```
z = sin(r)./r;
mesh(z)
```

c. Đồ thị đường đẳng mức: Các hàm contour tạo, hiển thị và ghi chú các đường đẳng mức của một hay nhiều ma trận. Chúng gồm:

clabel	tạo các nhãn sử dụng ma trận contour và hiển thị nhãn
contour	hiển thị các đường đẳng mức tạo bởi một giá trị cho trước của ma trận Z.
contour3	hiển thị các mặt đẳng mức tạo bởi một giá trị cho trước của ma trận Z.
contourf	hiển thị đồ thị contour 2D và tô màu vùng giữa 2 các đường
contourc	hàm cấp thấp để tính ma trận contour

Hàm ***meshc*** hiển thị contour và lưới và ***surfc*** hiển thị mặt contour.

```
[X,Y,Z] = peaks;
contour(X,Y,Z,20)
```

Mỗi contour có một giá trị gắn với nó. Hàm ***clabel*** dùng giá trị này để hiển thị nhãn đường đồng mức 2D. Ma trận contour chứa giá trị clabel dùng cho các đường contour 2D. Ma trận này được xác định bởi ***contour***, ***contour3*** và ***contourf***.

Để hiển thị 10 đường đẳng mức của hàm ***peak*** ta viết:

```
Z = peaks;
[C,h] = contour(Z,10);
clabel(C,h)
title({'Cac contour co nhan','clabel(C,h)'})
```

Hàm ***contourf*** hiển thị đồ thị đường đẳng mức trên một mặt phẳng và tô màu vùng còn lại giữa các đường đẳng mức. Để kiểm soát màu tô ta dùng hàm ***caxis*** và ***colormap***. Ta viết chương trình ***ct1_26.m***:

```
Z = peaks;
[C, h] = contourf(Z, 10);
caxis([-20 20])
colormap autumn;
title({'Contour co to mau', 'contourf(Z, 10)'})
```

Các hàm ***contour(z, n)*** và ***contour(z, v)*** cho phép ta chỉ rõ số lượng mức contour hay một mức contour cần vẽ nào đó với z là ma trận số liệu, n là số đường contour và v là vec tơ các mức contour. MATLAB không phân biệt giữa vec tơ một phần tử hay đại lượng vô hướng. Như vậy nếu v là vec tơ một phần tử mô tả một contour đơn ở một mức hàm ***contour*** sẽ coi nó là số lượng đường contour chứ không phải là mức contour. Nghĩa là, ***contour(z, v)*** cũng như ***contour(z, n)***. Để hiển thị một đường đẳng mức ta cần cho v là một vec tơ có 2 phần tử với cả hai phần tử bằng mức mong muốn. Ví dụ để tạo ra một đường đẳng mức 3D của hàm peaks ta viết chương trình ***ct1_27.m***:

```
xrange = -3: .125: 3;
yrange = xrange;
[X,Y] = meshgrid(xrange, yrange);
Z = peaks(X, Y);
contour3(X, Y, Z)
```

Để hiển thị một mức ở Z = 1, ta cho v là [1 1]

```
v = [1 1]
contour3(X, Y, Z, v)
```

Hàm ***ginput*** cho phép ta dùng chuột hay các phím mũi tên để chọn các điểm vẽ. Nó trả về toạ độ của vị trí con trỏ. Ví dụ sau sẽ minh hoạ các dùng hàm ***ginput*** và hàm spline để tạo ra đường cong nội suy hai biến.

Ta tạo một M-file có tên ***ct1_28.m*** như sau:

```
disp('Chuot phai tro cac diem tren duong ve')
disp('Chuot trai tro diem cuoi cua duong ve')
axis([0 10 0 10])
hold on
x = [];
y  = [];
n = 0;
```

```
but = 1;
while but = =1
        [xi,yi,but] = ginput(1);
        plot(xi, yi, 'go')
        n = n  + 1;
        x(n, 1) = xi;
        y(n,1) = yi;
end
t = 1:n;
ts = 1: 0.1: n;
xs = spline(t, x, ts);
```

ys = spline(t, y, ts);
plot(xs, ys, 'c-');
hold off

14. Vẽ các vectơ: Có nhiều hàm MATLAB dùng hiển thị các vec tơ có hướng và vec tơ vận tốc. Ta định nghĩa một vec tơ bàng cách dùng một hay 2 đối số. Các đối số mô tả thành phần x và thành phần y của vec tơ. Nếu ta dùng 2 đối số thì đối số thứ nhất sẽ mô tả thành phần x và đối số thứ ha mô tả thành phần y. Nếu ta chỉ dùng một đối số thì MATLAB xử lí nó như một số phức, phần thực là thành phần x và phần ảo là thành phần y.

Các hàm vẽ vec tơ gồm:

compass vẽ các véc tơ bắt đầu từ gốc toạ độ của hệ toạ độ cực
feather vẽ các vec tơ bắt đầu từ một đường thẳng
quiver vẽ các vec tơ 2D có các thành phần (u, v)
quiver3 vẽ các vec tơ 3D có các thành phần (u, v, w)

a. Hàm compass: Ta xét ví dụ vẽ hướng và tốc độ gió. Các vec tơ xác định hướng (góc tính bằng độ) và tốc độ gió (km/h) là:

hg = [45 90 90 45 360 335 360 270 335 270 335 335];
td = [6 6 8 6 3 9 6 8 9 10 14 12];

Ta biến đổi hướng gió thành radian trước khi biến đổi nó thành toạ độ vuông góc.

*hg1 = hg * pi/180;*
[x, y] = pol2cart(hg1, td);
compass(x, y)

và tạo ra ghi chú trên đồ thị:

gc = {'Huong gio và suc gio tai san bay Da Nang')
text(–28, 15, gc)

b. Hàm feather: Hàm feather hiển thị các vec từ bắt đầu từ một đường thẳng song song với trục x. Ví dụ để tạo ra các vec tơ có góc từ 90^0 đến 0^0 và cùng độ dài ta viết chương trình ***ct1_30.m***:

theta = 90: –10: 0;
r = ones(size(theta));

trước khi vẽ, chuyển các số liệu sang toạ độ vuông góc và tăng độ lớn thành r để dễ nhìn:

```
[u, v] = pol2cart(theta*pi/180, r*10);
feather(u, v)
axis equal
```

Nếu đối số là số phức z thì ***feather*** coi phần thực là x và phần ảo là y. Ta xét chương trình ***ct1_31.m***:

```
t = 0: 0.3: 10;
s = 0.05 + i;
Z = exp(–s*t);
feather(Z)
```

c. Hàm quiver: Hàm quiver hiển thị các vec tơ ở các điểm đã cho trong mặt phẳng. Các vec tơ này được xác định bằng các thành phần x và y.
Ví dụ để tạo ra 10 contour của hàm peaks ta dùng chương trình ***ct1_32.m***:

```
n = –2.0: .2: 2.0;
[X,Y,Z] = peaks(n);
contour(X, Y, Z, 10)
```

Bây giờ dùng hàm gradient để tạo các thành phần của vec tơ dùng làm đối số cho ***quiver***:

```
[U, V] = gradient(Z, .2);
```

Đặt ***hold on*** để thêm đường contour:

```
hold on
quiver(X,Y,U,V)
hold off
```

d. Hàm quiver3: Hàm ***quiver3*** hiển thị các vec tơ có các thành phần (u,v,w) tại điểm (x, y, z). Ví dụ ta biểu diễn quỹ đạo của một vật được ném đi theo t. Phương trình của chuyển động là:

$$z(t) = v_0 t + \frac{at^2}{2}$$

Ta viết chương trình ***ct1_33.m***. Trước hết ta gán vận tốc ban đầu và gia tốc a:

```
v0 = 10; % Van toc ban dau
a = –32; % gia toc
```

Tiếp theo tính z tại các thời điểm:

```
t = 0:.1:1;
z = vz*t + 1/2*a*t.^2;
```

Tính vị trí theo hướng x và y:

```
vx = 2;
x = vx*t;
vy = 3;
y = vy*t;
```

Tính các thành phần của vec tơ vận tốc và hiển thị bằng các dùng ***quiver3***:

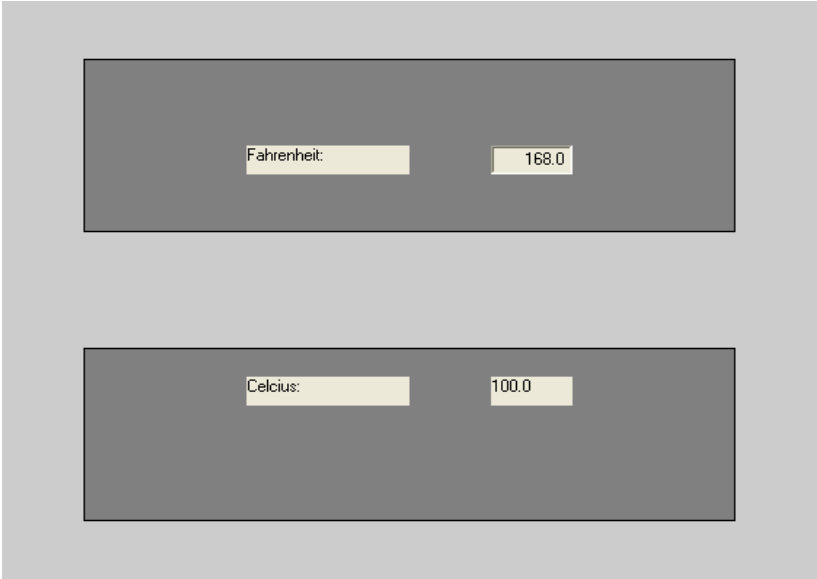
```
u = gradient(x);
v = gradient(y);
w = gradient(z);
scale = 0;
quiver3(x, y, z, u, v, w, scale)
axis square
```

§3. GIAO DIỆN ĐỒ HOẠ

1. Khái niệm chung: Để tiện dụng ta có thể tạo nên giao diện đồ hoạ(GUI - Graphic User Interface) giữa người dùng và MATLAB. Trong giao diện này ta có thể xuất dữ liệu dưới 2 dạng: văn bản và đồ hoạ. Mỗi một GUI có một hay nhiều layout(diện mạo). Việc tạo GUI tạo nên một công cụ đồ hoạ phục vụ nhập xuất dữ liệu một cách trực giác, rất thuận tiện. Ngoài ra có thể dùng GUI để giám sát các quá trình, hiển thị các đối tượng.

2. Nhập xuất kí tự, số liệu ra GUI:

a. Tạo khung hình: Ta xét các lệnh sau(***ct1_35.m***):



```
 f = input('Nhap nhiet do(do K): ');
c = (f - 32)*5/9;
fprintf(1,'nhiet do(do C) la: %g\n', c)
```

Ba dòng lệnh trên thực hiện các công việc sau:

- nhập giá trị đầu vào
- thực hiện phép tính quy đổi nhiệt độ
- xuất kết quả ra màn hình

Bây giờ ta tìm cách cài các dòng lệnh trên sao cho chúng thực hiện trên khuôn khổ một khung đồ hoạ có dạng như trên

Các lệnh sau(***ct1_36.m***) thực hiện công việc trên:

```
set(gcf,'DefaultUicontrolUnit', 'Normalized')
frame_1 = uicontrol(gcf, 'Style', 'Frame',...
                    'Position', [0.1 0.1  0.8 0.3]);
frame_2 = uicontrol(gcf, 'Style', 'Frame',...
                    'Position', [0.1 0.6  0.8 0.3]);
set(frame_1, 'BackgroundColor',  [0.5 0.5 0.5]);
set(frame_2, 'BackgroundColor',  [0.5 0.5 0.5]);
text_f = uicontrol(gcf, 'Style',  'Text',...
                   'String',      'Fahrenheit: ',...
                   'Position',   [0.3 0.7 0.2 0.05],'HorizontalAlignment','Left');
edit_f = uicontrol(gcf, 'Style',   'Edit',...
                   'String',   '168.0',...
                   'Position',  [0.6 0.7 0.1 0.05 ],...
                   'HorizontalAlignment',  'Right',...
                   'Callback',   'ct1_38');
text_c1 = uicontrol(gcf,'Style',   'Text',...
                    'String',   'Celcius: ',...
                    'Position',   [0.3 0.3 0.2 0.05],...
                    'HorizontalAlignment',   'Left');
text_c2 = uicontrol(gcf,'Style',   'Text',...
                    'String',   '100.0',...
                    'Position',   [0.6 0.3 0.1 0.05],...
                    'HorizontalAlignment',   'Left');
```

Bây giờ ta sẽ xem các lệnh trên hoạt động như thế nào. Các lệnh sau:

```
set(gcf,'DefaultUicontrolUnit',   'Normalized')
frame1 = uicontrol(gcf,'Style',   'Frame',...
                   'Position',   [0.1 0.1 0.8 0.3]);
```

```
frame2 = uicontrol(gcf,'Style',  'Frame',...
                        'Position',   [0.1 0.6 0.8 0.3]);
set(frame1,'BackgroundColor',   [0.5 0.5 0.5]);
set(frame2,'BackgroundColor',   [0.5 0.5 0.5]);
```

tạo hai khung hình chữ nhật trong cửa sổ Figure hiện hành với nền màu xám. Hai khung (Frames) có toạ độ các góc dưới trái là (0.1, 0.1) và (0.1, 0.6), cùng chiều cao 0.3 đơn vị và bề rộng 0.8 đơn vị. Đơn vị được tính bằng % của kích cỡ ngoài của Figure. Vậy ta có thể diễn giải như sau:

- Khung thứ nhất có góc trái dưới tại điểm có toạ độ 10% chiều ngang và 10% chiều cao của khung ngoài Figure.
- Khung thứ 2 có góc trái phía dưới tại điểm có toạ độ ứng với 10% chiều ngang và 60% chiều cao của khung ngoài Figure.
- Cả hai khung có chiều cao bằng 30% chiều cao và bề ngang bằng 80% bề ngang của khung ngoài Figure.

b. Dùng lệnh edit và text để nhập xuất kí tự và số liệu: Trên đây ta đã dùng lệnh uicontrol để tạo và xác định vị trí hai khung hình. Đoạn lệnh sau sử dụng uicontrol để viết chuỗi kí tự "Fahrenheit" lên khung bên trên:

```
text_ f = uicontrol(gcf,'Style','Text','String','Fahrenheit: ',...
                        'Position',[0.3 0.7 0.2 0.05],'HorizontalAlignment','Left');
```

Chuỗi kí tự "Fahrenhaeit" được đặt vào đúng vị trí dồn trái của ô có Position ghi trong đoạn chương trình trên. Đoạn lệnh sau dùng Edit để viết chuỗi kí tự "68.0" vào vị trí bên cạnh của "Fahrenheit". Chuỗi kí tự có vị trí dồn phải trong ô (Position Box).

```
edit_f = uicontrol(gcf,'Style',   'Edit',...
                    'String',   '168.0',...
                    'Position',  [0.6 0.7 0.1 0.05 ],...
                    'HorizontalAlignment',  'Right',...
                    'Callback',   'ct1_38');
```

Do sử dụng edit, chuỗi kí tự "68.0" là chuỗi có thể viết lại được trực tiếp trên GUI. Sau khi nhấn nút trên, giá trị mới viết lại được tiếp nhận và MATLAB sẽ gọi lệnh viết trong phần callback ***ct1_38.m***.

Cuối cùng ta còn phải dùng uicontrol để tạo ta chuỗi text, hiển thị chuỗi "Celcius" và "20.0" trong khung bên dưới.

```
text_c1 =  uicontrol(gcf,'Style','Text','String','Celcius: ',...
                        'Position',[0.3 0.3 0.2 0.05],'HorizontalAlignment','Left');
text_c2 =  uicontrol(gcf,'Style','Text','String','20.0','Position',...
```

[0.6 0.3 0.1 0.05],'HorizontalAlignment','Left');

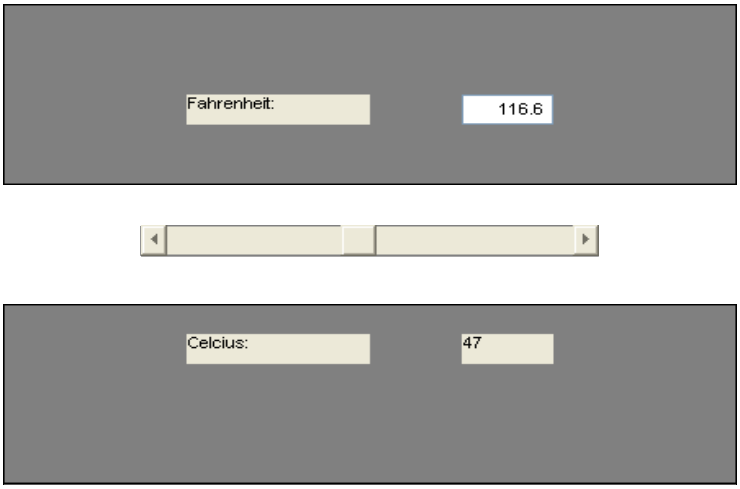
c. Tự động cập nhật giá trị lên GUI: Để hoàn thiện ví dụ GUI ta thực hiện chương trình với nhiệm vụ tính quy đổi từ độ K sang độ C và tự động điền kết quả vào các ô bên cạnh chuỗi Celcius. Đoạn mã sau phục vụ mục đích callback (hoàn trả giá trị) được lưu vào file ***ct1_38.m*** và có nội dung như sau:

```
f = get(edit_f, 'String');
f = str2num(f);
c = ( f - 32)*5/9;
c = num2str(c);
set(text_c2, 'String',c);
```

Đoạn mã trên nhận giá trị do lệnh uicontrol "edit" đọc vào dưới dạng chuỗi (string) và sau đó:

- biến đổi từ dạng string sang dạng số
- tính quy đổi từ nhiệt độ fahrenheit sang nhiệt độ celcius
- biến đổi từ số sang string
- xuất kết quả dưới dạng string ra GUI nhờ text_c2

3. Nhập số liệu từ thanh trượt: Ngoài cách nhập số liệu từ bàn phím, ta có thể nhập số liệu từ thanh trượt. Ta muốn tạo ra một giao diện như sau:



Trong giao diện này, con trượt sẽ làm thay đổi giá trị nhiệt độ đua vào và nhiệt độ quy đổi tính theo độ C cũng sẽ thay đổi tương ứng. Các lệnh tạo ra giao diện này (***ct1_37.m***) là:

```
set(gcf, 'DefaultUicontrolUnit', 'Normalized')
frame_1 = uicontrol(gcf, 'Style', 'Frame', 'Position', [0.1 0.1  0.8 0.3]);
frame_2 = uicontrol(gcf, 'Style', 'Frame', 'Position', [0.1 0.6  0.8 0.3]);
```

```
set(frame_1, 'BackgroundColor' ,[0.5 0.5 0.5]);
set(frame_2, 'BackgroundColor', [0.5 0.5 0.5]);
text_ f = uicontrol(gcf, 'Style', 'Text', 'String', 'Fahrenheit: ','Position',...
            [0.3 0.7 0.2 0.05], 'HorizontalAlignment', 'Left');
edit_f = uicontrol(gcf, 'Style',  'Edit',...
                    'String',   '168.0'.,,,
                    'Position',  [0.6 0.7 0.1 0.05 ],...
                    'HorizontalAlignment',   'Right',...
                    'Callback',   'ct1_38');
text_c1 =  uicontrol(gcf,'Style',   'Text',...
                     'String',   'Celcius: ',...
                     'Position',   [0.3 0.3 0.2 0.05],...
                     'HorizontalAlignment',   'Left');
text_c2 =  uicontrol(gcf,'Style',   'Text',...
                     'String',   '100.0',...
                     'Position',   [0.6 0.3 0.1 0.05],...
                      'HorizontalAlignment',   'Left');
slider_f  =  uicontrol(gcf,'Style',   'Slider',...
                      'Min',  32.0, 'Max',   212.0,...
                      'Value',   68.0,...
                      'Position',   [0.6 0.8 0.2 0.05],...
                     'Callback',   'ct1_39; ct1_38');
```

Để tạo thanh trượt ta dùng lệnh:

```
slider_f = uicontrol(gcf,'Style','Slider','Min',32.0,'Max',212.0,...
                     'Value',68.0,'Position',[0.6 0.8 0.2 0.05],...
                     'Callback','ct1_39; ct1_38');
```

Như vậy Callback có thể gọi một chuỗi các lệnh MATLAB, phân cách nhau bằng dấu chấm than hay dấu phẩy. Chuỗi callback gọi ***ct1_39.m***:

```
f = get(slider_f,'Value');
f = num2str(f);
set(edit_f,'String',f,'CallBack','ct1_40; ct1_38');
```

với tác dụng nhập nhiệt độ giữ tại 'Value' của slider_f vào vị trí bên cạnh ô chứa chuỗi "Fahrenheit". Sau đó Callback gọi tiếp ***ct1_38.m*** để tính quy đổi giá trị nhiệt độ và gán vào ô cạnh chuỗi "Celcius". File ***ct1_40.m*** như sau:

```
f = get(edit_f,'String');
f = str2num(f);
```

```
set(slider_f,'Value',f);
```

có nhiệm vụ cập nhật giá trị giữ tại ‘Value’ của slider_f để rồi sau đó ***ct1_38.m*** làm nốt phần việc còn lại: tính đổi nhiệt độ và gán vào vị trí cạnh ô chứa chuỗi “Celcius”.

4. Chọn lựa khi xuất số liệu:

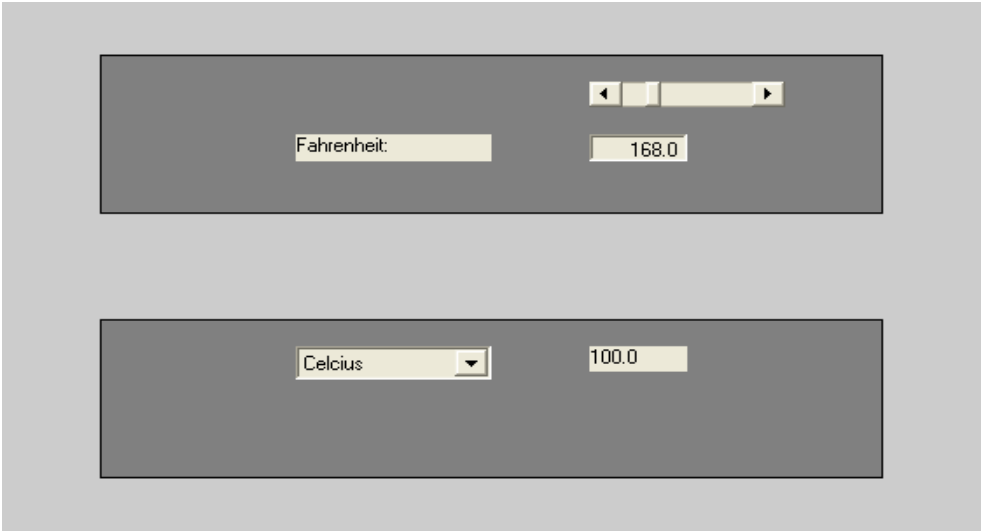
a. Khái niệm chung: Ngoài khả năng xuất dữ liệu cố định theo kiểu string hay kiểu số, ta có thể xuất dữ liệu theo một danh mục nào đó. Để minh hoạ, ta tạo file ***ct1_41.m*** như sau:

```
f = input('Nhap nhiet do: ');
r = f + 459.7;
c = (f - 32)*5/9;
k = c + 273.15;
choice = input(['Nhap 1 cho Rankie', '2 cho Celcius', '3 cho Kelvin: ']);
if choice = = 1
    fprintf(1, 'Nhiet do (do R) la: %g\n', r);
elseif choice = = 2
    fprintf(2, 'Nhiet do (do C) la: %g\n', c);
elseif choice = = 3
    fprintf(2, 'Nhiet do (do C) la: %g\n', c);
end
```

Từ cửa sổ lệnh, nhập lệnh ***ct1_41*** thì MATLAB sẽ hỏi nhiệt độ và đích quy đổi rồi hiển thị kết quả. Tuy nhiên công cụ GUI của MATLAB cho phép ta thực hiện việc lựa chọn thuận lợi hơn. Ta có thể chọn một trong 4 phương xuất dữ liệu sau đây:

- dùng popupmenu
- dùng list box
- dùng radio button
- dùng check box

b. Dùng popupmenu: Ta tạo ra giao diện như sau:



Các lệnh thực hiện công việc trên (***ct1_42.m***) là:

```
set(gcf, 'DefaultUicontrolUnit',  'Normalized')
frame_1 = uicontrol(gcf, 'Style',  'Frame',...
                        'Position',   [0.1 0.1  0.8 0.3]);
frame_2 = uicontrol(gcf, 'Style',   'Frame',...
                        'Position',   [0.1 0.6  0.8 0.3]);
set(frame_1, 'BackgroundColor',  [0.5 0.5 0.5]);
set(frame_2, 'BackgroundColor'  ,[0.5 0.5 0.5]);
text_f = uicontrol(gcf,'Style',  'Text',...
                       'String',   'Fahrenheit: ',...
                       'Position',   [0.3 0.7 0.2 0.05],...
                       'HorizontalAlignment',  'Left');
edit_f = uicontrol(gcf,'Style',   'Edit',...
                       'String',...'168.0',...
                        'Position',  [0.6 0.7 0.1 0.05 ],...
                        'HorizontalAlignment',  'Right',...
                        'Callback',  'ct1_38');
popup_c = uicontrol(gcf,...
                        'Style','Popupmenu',...
                        'String','Rankine|Celcius|Kelvin',...
                        'Value',2,...
                        'Position',[0.3 0.3 0.2 0.05],...
                        'Callback','ct1_43; ct1_45');
text_c2 =  uicontrol(gcf, 'Style',   'Text',...
                        'String',   '100.0',...
                        'Position',   [0.6 0.3 0.1 0.05],...
                        'HorizontalAlignment',   'Left');
slider_f = uicontrol(gcf, 'Style',   'Slider',...
```

```
'Min',  32.0, 'Max',  212.0,...
 'Value',   68.0,...
'Position',   [0.6 0.8 0.2 0.05],...
'Callback', 'ct1_39; ct1_45');
```

Khi kích chuột vào Popupmenu , có ba khả năng chọn lựa sẽ xuất hiện. Tiếp tục nháy chuột vào một trong 3 khả năng đó , ***Popupmenu*** biến mất chỉ còn lại đơn vị được chọn. Khi dùng chuột kéo thanh trượt ở frame phía trên, ta có được giá trị quy đổi sang đơn vị được chọn hiển thị ở phía dưới. Trong đoạn mã trên, giá trị 'Value' đặt sẵn là 2. Khi Callback gọi ***ct1_43.m***:

```
choice = get(popup_c,'Value');
```

thì giá trị của biến choice được đưa tới 'Value'. Sau đó Callback gọi tiếp ***ct1_45.m*** để xem kết quả giữ trong choice. File ***ct1_45.m*** như sau:

```
 f = get(edit_f, 'String');
 f = str2num(f);
r = f  +  459.7;
 c = (f - 32)*5/9;
 k = c + 273.15;
choice = input(['Nhap 1 cho Rankie', '2 cho Celcius', '3 cho Kelvin: ']);
if choice = = 1
  t = r;
elseif choice = = 2
  t = c;
elseif choice = = 3
   t = k
end
t = num2str(t);
set(text_c2, 'String',t);
```

Bằng cách thay '***Popupmenu***' bằng '***Radiobutton***' uicontrol ta có phương án ***Radiobutton***. Giao diện sẽ có dạng:

Các lệnh thực hiện công việc này (***ct1_46.m***) là:

```
set(gcf, 'DefaultUicontrolUnit',   'Normalized')
frame_1 = uicontrol(gcf, 'Style', 'Frame', 'Position',  [0.1 0.1  0.8 0.3]);
frame_2 = uicontrol(gcf, 'Style', 'Frame', 'Position',  [0.1 0.6  0.8 0.3]);
set(frame_1,'BackgroundColor',  [0.5 0.5 0.5]);
set(frame_2,'BackgroundColor',  [0.5 0.5 0.5]);
text_f = uicontrol(gcf, 'Style',  'Text', 'String',  'Fahrenheit: ','Position',...
                        [0.3 0.7 0.2 0.05], 'HorizontalAlignment', 'Left');
edit_f = uicontrol(gcf, 'Style', 'Edit', 'String','168.0', 'Position',...
                       [0.6 0.7 0.1 0.05 ], 'HorizontalAlignment',...
                      'Right', 'Callback','ct1_41');
strings = ['Rankine'; 'Celcius'; 'Kelvine'];
show   = [    0;        1;         0];
ys     = [    3;        2;         1]*0.075 + 0.075;
for i = 1:3
   radio_c(i) = uicontrol(gcf,...
                              'Style',   'Radiobutton',...
                              'String', strings(i),...
                              'Value',  show(i),...
                              'Position',  [0.3 ys(i)  0.2 0.05],...
                              'Callback',       'ct1_47; ct1_45');
end
text_c2= uicontrol(gcf, 'Style', 'Text', 'String','100.0', 'Position',...
```

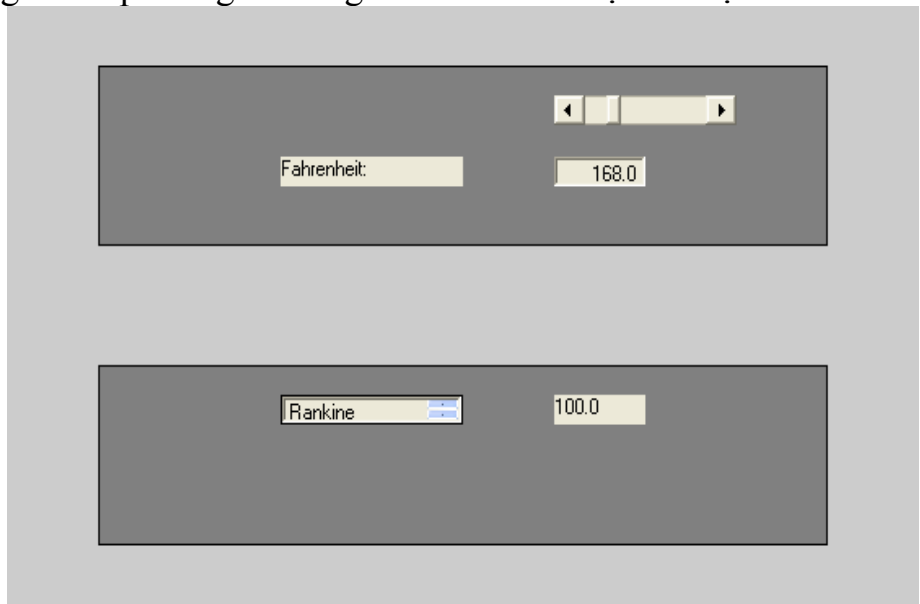
```
                [0.6 0.3 0.1 0.05], 'HorizontalAlignment', 'Left');
slider_f = uicontrol(gcf, 'Style', 'Slider', 'Min',32.0, 'Max', 212.0,...
                'Value', 68.0, 'Position', [0.6 0.8 0.2 0.05],...
                                'Callback', 'ct1_39; ct1_45');
```

File ***ct1_47.m***:

```
for i = 1:3
   if gcbo = = radio_c(i)
                choice = i;
      set(radio_c(i), 'Value', 1);
  elseif
      set(radio_c(i), 'Value', 0);
        end;
end;
```

Đoạn lệnh trên là một vòng lặp, so sánh số (handle) Callback thu được (giá trị do hàm gcbo trả về) với handle của mỗi nút. Nút nào có số trùng sẽ được đóng (turn on, 'Value' = 1) và nút nào khác số sẽ bị ngắt (turn off,'Value' = 0). Cuối cùng Callback gọi ***ct1_45.m*** để thực hiện việc tính quy đổi được chọn và hiển thị kết quả. Điểm khác duy nhất là khi chọn, Popupmenu chỉ chứa một phần tử thì radiobutton có thể đồng thời chứa nhiều phần tử.

Cuối cùng ta xét phương án dùng ***listbox***. Giao diện cần tạo như sau:



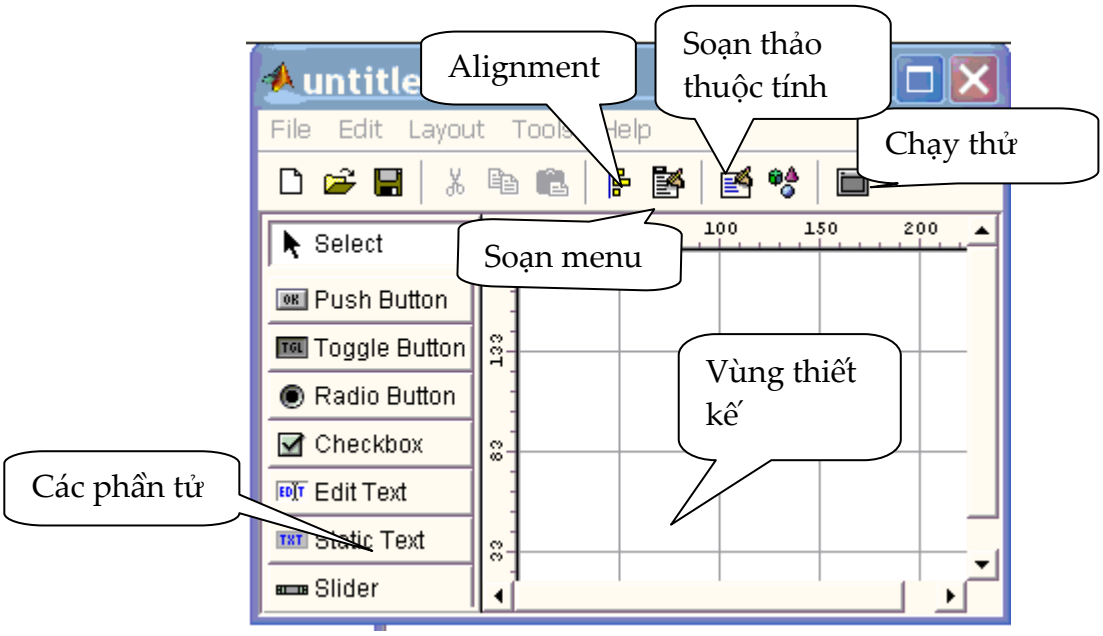
Các mã tạo ra giao diện trên (***ct1_48.m***) là:

```
set(gcf, 'DefaultUicontrolUnit', 'Normalized')
frame_1 = uicontrol(gcf, 'Style', 'Frame', 'Position', [0.1 0.1  0.8 0.3]);
```

```
frame_2 = uicontrol(gcf, 'Style', 'Frame', 'Position', [0.1 0.6  0.8 0.3]);
set(frame_1, 'BackgroundColor', [0.5 0.5 0.5]);
set(frame_2, 'BackgroundColor', [0.5 0.5 0.5]);
text_f = uicontrol(gcf, 'Style', 'Text', 'String', 'Fahrenheit: ', 'Position',...
            [0.3 0.7 0.2 0.05], 'HorizontalAlignment', 'Left');
edit_f = uicontrol(gcf, 'Style', 'Edit', 'String', '168.0', 'Position',...
            [0.6 0.7 0.1 0.05 ], 'HorizontalAlignment',...
            'Right', 'Callback', 'ct1_38');
listbox_c = uicontrol(gcf,...
             'Style', 'Listbox',...
             'String', 'Rankine|Celcius|Kelvin',...
             'Value', 2,...
             'Position', [0.3 0.3 0.2 0.05],...
             'Callback', 'ct1_49;ct1_45');
text_c2 =  uicontrol(gcf, 'Style', 'Text', 'String', '100.0', 'Position',...
            [0.6 0.3 0.1 0.05], 'HorizontalAlignment', 'Left');
slider_f = uicontrol(gcf, 'Style', 'Slider', 'Min',32.0, 'Max', 212.0,...
              'Value', 68.0, 'Position', [0.6 0.8 0.2 0.05],...
                              'Callback', 'ct1_39; ct1_45');
```

5. Công cụ đồ hoạ tạo GUI

a. Tạo GUI bằng công cụ đồ hoạ: Trên đây ta đã xem xét cách tạo GUI bằng phương pháp thủ công. Ta có thể tạo GUI bằng công cụ đồ hoạ. Khi nhập lệnh guide ta gọi trình đồ hoạ (Graphics User Interface Development Environment) để soạn thảo layout. Kết quả đầu tiên là ta có một layout rỗng như sau:

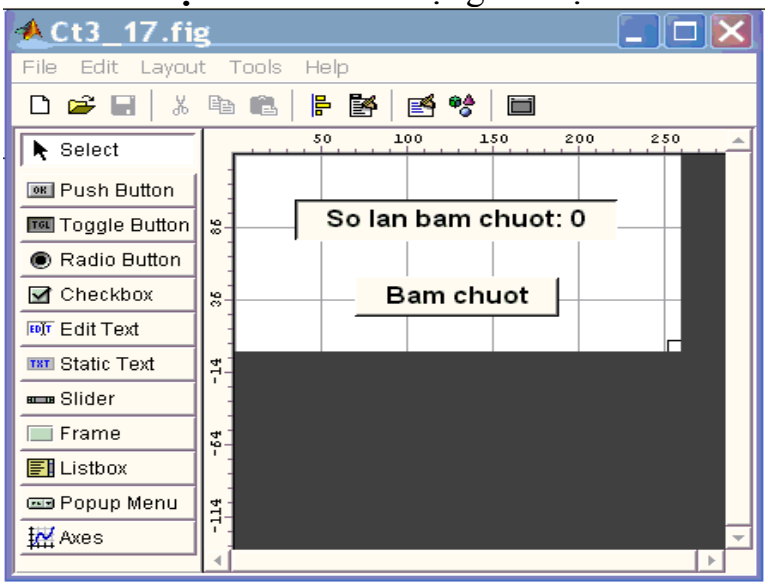


Việc đầu tiên là ta thiết kế giao diện mong muốn. Ta sẽ dùng chuột kéo các phần tử cần dùng từ bên trái và thả vào layout rỗng bên phải. Ta có thể dịch chuyển các phần tử này đế các vị trí mong muốn và cân chỉnh bằng công cụ Alignment. Với mỗi phần tử ta cấn xác định thuộc tính cho nó bằng cách bấm đúp vào phần tử hay bấm vào công cụ soạn thảo thộc tính

Sau khi thiết kế xong ta lưu nó lại. Lúc này MATLAB tự động tạo ra file *.fig dùng lưu giao diện vừa tạo và file *.m chưa các mã lệnh cần thực hiện. Việc cuối cùng là viết các mã lệnh vào file *.m. Trong quá trình thiết kế ta có thể chạy thử xem sau mỗi bước thiết kế đã đạt yêu cầu chưa bằng cách bấm vào ô chạy thử

b. Một số ví dụ tạo GUI:

☞ ***Đếm số lần bấm chuột***: Ta thiết kế một giao diện như sau:



Ta muốn là khi bấm chuột, số lần bấm sẽ được đếm và ghi lại. Trước hết ta gọi guide và có được một layout rỗng. Vào Property Inspector (ô soạn thảo thuộc tính) và ghi vào Name chuỗi "ct1_52" và chấp nhận thuộc tích Tag mặc định của nó là figure1; dùng Font chữ mặc định, cỡ chữ 12, bold. Ta dùng ô Edit Text để ghi lại số lần bấm. Ta vào Property Inspector rồi chọn String. Ta nhập vào ô này chuỗi "So lan bam chuot: 0". Ta ghi vào ô Tag chuỗi "editmot" và cũng dùng Font chữ mắc định, cỡ chữ 12 và bold. Tiếp theo kéo Pushbutton vào layout và soạn thảo thuộc tính cho nó với Font chữ mặc định, cỡ chứ 12, bold. Trong thuôc tính String ghi chuỗi " Bam chuot"; ghi và Tag chuỗi "pushbuttonmot". Như vậy là ta đã thiết kế xong. Bây giờ ta lưu lại với tên là ***ct1_52.fig*** và ***ct1_52.m***.

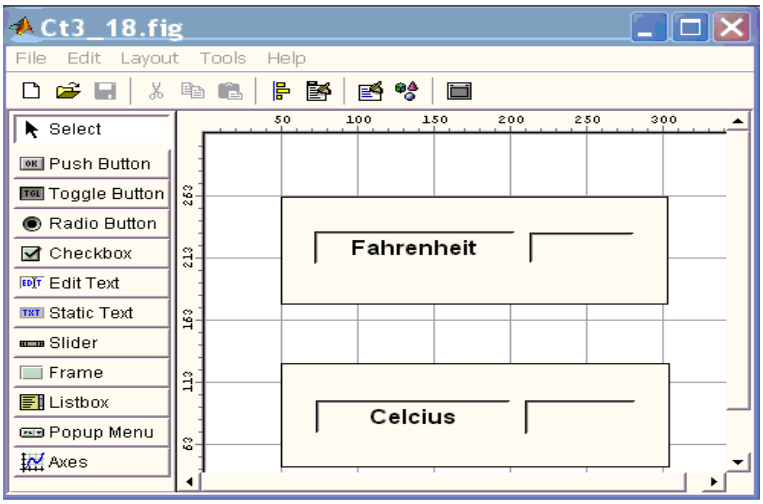
Nhiệm vụ tiếp theo là ghi các lệnh cần thiết vào file ***ct1_52.m***. File này đã được MATLAB tự động tạo ra. Ta phải thêm vào đó các mã lệnh để khi bấm chuột thì số lần bấm được thể hiện trên ô Edit Text. Ta sẽ ghi các mã lệnh này vào phần:

function varargout = pushbuttonmot_Callback(h, eventdata, handles, varargin)

do lệnh cần được thực hiện khi gọi pushbutton. Nội dung của ***ct1_52.m*** là:

```
function varargout = Ct1_52(varargin)
if nargin = = 0
        fig = openfig(mfilename,'reuse');
        set(fig, 'Color', get(0, 'defaultUicontrolBackgroundColor'));
        handles = guihandles(fig);
        guidata(fig, handles);
        if nargout > 0
                varargout{1} = fig;
        end
elseif
     ischar(varargin{1})
        try
                [varargout{1:nargout}] = feval(varargin{:});
        catch
                disp(lasterr);
        end
end
function varargout = pushbuttonmot_Callback(h, eventdata, handles, varargin)
persistent dem;%bien dem la persistent de no ton tai giua lan goi ham
if isempty(dem)
   dem = 0;
end
dem = dem  + 1;
str = sprintf('So lan bam chuot: %d',dem);
set(handles.editmot,'String',str);
```

☞ ***Chuyển đổi từ độ Fahrenheit sang độ Celcius***: Ta thiết kế một GUI để chuyển đổi nhiệt độ. Giao diện có dạng như sau:



Thuộc tính của Layout được ghi Name: ***ct1_53*** còn các thuộc tính khác là mặc định.

Ta dùng hai Frame với các Tag là frmmot và frame2. Các thuộc tính khác chấp nhận giá trị mặc định.

Edit Text thứ nhất có các thuộc tính FontName: Arial, FontSize: demi, FontWeight: demi, String: Fahrenheit, Tag: editmot còn các thuộc tính khác là mặc định.

Edit Text thứ hai có các thuộc tính FontName: Arial, FontSize: demi, FontWeight: demi, String: để trống, Tag: edithai còn các thuộc tính khác là mặc định.

Edit Text thứ ba có các thuộc tính FontName: Arial, FontSize: demi, FontWeight: demi, String: Celcius, Tag: editba còn các thuộc tính khác là mặc định.

Edit Text thứ tư có các thuộc tính FontName: Arial, FontSize: demi, FontWeight: demi, String: để trống, Tag: editbon còn các thuộc tính khác là mặc định.

Sau khi thiết kế xong, lưu nó với tên ***ct3_18.fig***. MATLAB tạo thêm ***ct1_53.m***. Bây giờ ta cần viết mã cho nó. Nhiệm vụ của đoạn mã là khi ta nhập nhiệt độ Fahrenheit vào ô Edit text thứ hai thì trong ô Edit Text thứ 4 phải xuất hiện giá trị nhiệt độ Celcius tương ứng. Do vậy nội dung của ***ct1_53.m*** là:

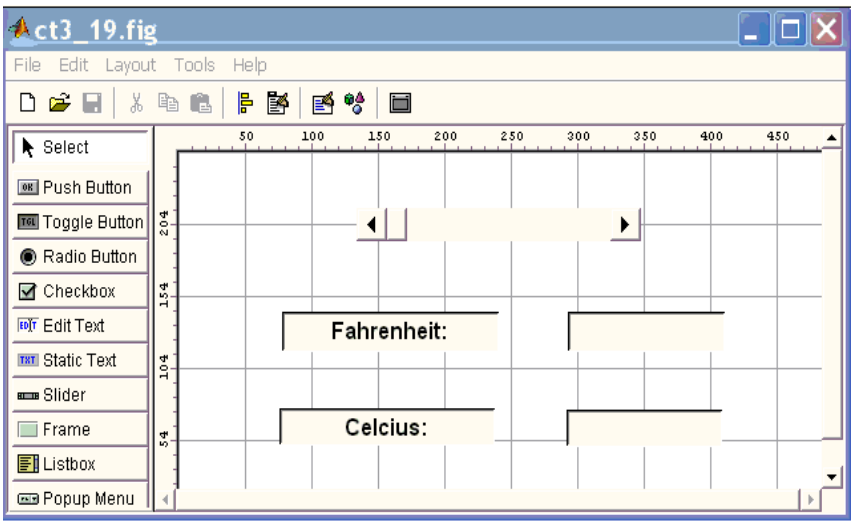
```
function varargout = Ct1_53(varargin)
if nargin == 0  % LAUNCH GUI
        fig = openfig(mfilename,'reuse');
        set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
        handles = guihandles(fig);
        guidata(fig, handles);
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1})
        try
                [varargout{1:nargout}]    =    feval(varargin{:});    %    FEVAL
switchyard
        catch
                disp(lasterr);
        end
end
function varargout = edithai_Callback(h, eventdata, handles, varargin)
f = get(handles.edithai,'String');
f = str2num(f);
c = (f - 32)*5/9;
c = num2str(c);
set(handles.editbon,'String',c);
```

Trong đó đoạn mã cần viết nằm trong đoạn:

function varargout = edithai_Callback(h, evendata, handles, varargin)

Các lệnh khác là do MATLAB tự động tạo ra.

☞ ***Dùng slider để nhập số liệu***: Ta dùng ví dụ chuyển đổi nhiệt độ trên nhưng bây giờ sẽ thêm slider để thay đổi nhiệt độ đầu vào. Giao diện sẽ có dạng:



Như vậy ta cần 5 phần tử, trong đó có một phần tử là slider và 4 phần tử Edit Text.

Layout có thuộc tính Name: ***ct1_54***, còn các thuộc tính khác ta chấp nhận giá trị mặc định.

Slider có thuộc tính Max: 1.0 và Min: 0.0.

Edit Text thứ nhất có thuộc tính FontSize: 12, FontWeight: bold, String: Fahrenheit còn các thuộc tính khác chấp nhận giá trị mặc định.

Edit Text thứ 2 có thuộc tính FontSize: 12, FontWeight: bold, String: để trống.

Edit Text thứ 3 có thuộc tính FontSize: 12, FontWeight: bold, String: Celcius.

Edit Text thứ 4 có thuộc tính FontSize: 12, FontWeight: bold, String: để trống. (Các thuộc tính mà ta không nhắc đến có nghĩa là chấp nhận giá trị mặc định).

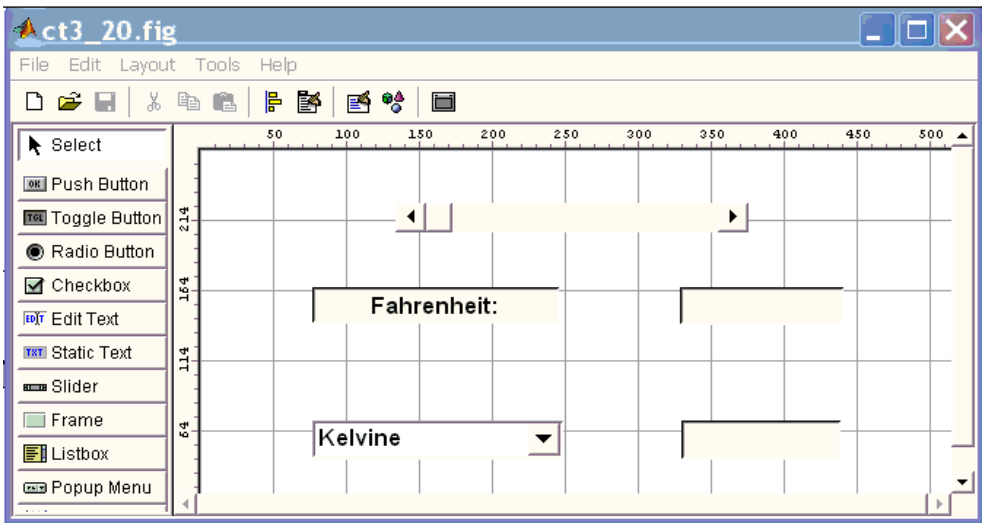
Layout được lưu với tên ***ct1_54.fig***.

Bây giờ ta viết mã cho phần ***ct1_54.m*** mà MATLAB đã tự động tạo ra. Nhiệm vụ của nó là nhận giá trị thay đổi từ con trượt, cập nhật cho Edit Text 2 và Edit Text 4. Ta có nội dung của ***ct1_54.m***:

```
function varargout = ct1_54(varargin)
if nargin = = 0
        fig = openfig(mfilename,'reuse');
        handles = guihandles(fig);
        guidata(fig, handles);
```

```
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1})
        try
                [varargout{1:nargout}]    =    feval(varargin{:});    %    FEVAL
switchyard
        catch
                disp(lasterr);
        end
end

function varargout = slider1_Callback(h, eventdata, handles, varargin)
f = get(handles.slider1,'Value');%nhan gia tri tu con truot
f = f*180 + 32;%tinh ra do Fahrenheit
a = num2str(f);%bien lai thanh chuoi
set(handles.edit2,'String',a);%ghi vao o do Fahrenheit
b = (f-32)*5/9;%doi thanh do Celcius
b = num2str(b);%doi lai thanh chuoi
set(handles.edit4,'String',b);%ghi vao o do Celcius
```

☞ ***Xuất số liệu có lựa chọn***: Ta vẫn dùng ví dụ trên nhưng bây giờ nhiệt độ quy đổi có thể được tính theo thang nhiệt độ Kenvine, Celcius hay Rankine. Để có thể chọn lựa ta dùng một trong các phương án: Popupmenu, Rdiobutton, Listbox hay Checkbox. Giao diện khi dùng Popupmenu như sau:



Như vậy là ta cần một Slider, ba Edit Text và một Popupmenu. Layout có thuộc tính Name: ***ct13_55***.

Slider có thuộc tính Max: 1 và Min: 0

Edit Text thứ nhất có thuộc tính FontSize: 12, FơntWeight: bold và String: Fahrenheit.

Edit Text thứ hai có thuộc tính FontSize: 12, FơntWeight: bold và String để trống.

Edit Text thứ 3 có thuộc tính FontSize: 12, FơntWeight: bold và String để trống.

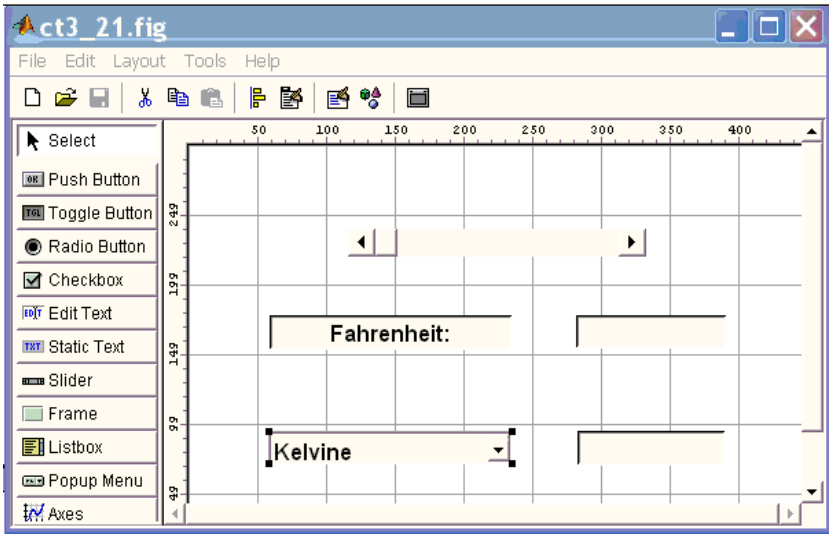
Popupmenu có thuộc tính FontSize: 12, FontWeight: bold. Để ghi vào thuộc tính String ta bấm đúp chuột vào icon của nó và viết 3 dòng: Kelvine, Celcius và Rankine.

File được lưu với tên ***ct1_55.fig***. Vấn đề còn lại là viết mã trong file ***ct1_55.m***. Mã cần thực hiện nhận giá trị từ Slider, xem Popupmenu nào được chọn để hiển thị nhiệt độ tương ứng. File ***ct1_55.m*** như sau:

```
function varargout = ct1_55(varargin)
if nargin == 0  % LAUNCH GUI
        fig = openfig(mfilename,'reuse');
        set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
        handles = guihandles(fig);
        guidata(fig, handles);
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1})
        try
                [varargout{1:nargout}] = feval(varargin{:});
        catch
                disp(lasterr);
        end
end
function varargout = slider1_Callback(h, eventdata, handles, varargin)
f = get(handles.slider1,'Value');
f = f*180 + 32;
a = num2str(f);
set(handles.edit2,'String',a);
r = f + 495.7;
c = (f - 32)*5/9;
k = c + 273.15;
chon = get(handles.popupmenu1,'Value');
if chon = = 1
   t = k;
elseif chon = = 2
   t = c;
```

```
elseif chon = = 3
    t = r;
end
t = num2str(t);
set(handles.edit3,'String',t);
```

Tiếp theo ta xét trường hợp dùng listbox. Thay vì dùng Popupmenu ta dùng Listbox. Các phần tử khác và thuộc tính của nó không thay đổi. Thuộc tính Name của Layout là ***ct1_56***. Ta vào ô String của Listbox và ghi vào đó 3 dòng Kelvine, Celcius và Rankine. Giao diện như sau:

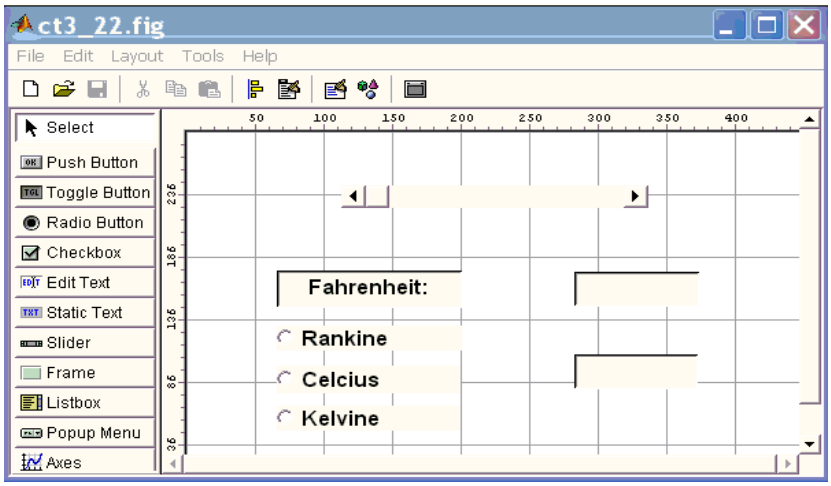


File được lưu với tên ***ct1_56.fig***. Tiếp theo viết lệnh cho ***ct1_56.m***. Ta có file này như sau:

```
function varargout = ct1_56(varargin)
if nargin = =
        fig = openfig(mfilename,'reuse');
        set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
        handles = guihandles(fig);
        guidata(fig, handles);
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1})
        try
                [varargout{1:nargout}] = feval(varargin{:});
        catch
                disp(lasterr);
        end
```

```
end

function varargout = slider1_Callback(h, eventdata, handles, varargin)
f = get(handles.slider1,'Value');
f = f*180 + 32;
a = num2str(f);
set(handles.edit2,'String',a);
r = f + 495.7;
c = (f - 32)*5/9;
k = c + 273.15;
chon = get(handles.listbox1,'Value');
if chon = = 1
   t = k;
elseif chon = = 2
   t = c;
elseif chon = = 3
   t = r;
end
t = num2str(t);
set(handles.edit3,'String',t);
```

Ta tiếp tục xét phương án dùng Radiobutton. Giao diện có dạng:



Ta dùng ba Radiobutton thay cho Listbox. Radiobutton thứ nhất có thuộc tính FontSize: 12, FontWeight: bold và String: Rankine. Radiobutton thứ 2 có thuộc tính FontSize: 12, FontWeight: bold và String: Celcius. Radibutton thứ 3 có thuộc tính FontSize: 12, FontWeight: bold và String: Kelvine. Các phần tử khác và thuộc tính của chúng vẫn như cũ. Layout có thuộc tính Name: ***ct1_57***. Lưu GUI với tên ***ct1_57.fig***.

Tiếp theo ta viết các mã lệnh trong ***ct1_57.m***:

```
function varargout = ct1_57(varargin)
if nargin = = 0
      fig = openfig(mfilename,'reuse');
      set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
      handles = guihandles(fig);
      guidata(fig, handles);
      if nargout > 0
        varargout{1} = fig;
      end
elseif ischar(varargin{1})
      try
                [varargout{1:nargout}] = feval(varargin{:});        catch
                disp(lasterr);
      end
end
function mutual_exclude(off)
set(off,'Value',0);
function varargout = slider1_Callback(h, eventdata, handles, varargin)
global chon
f = get(handles.slider1,'Value');
f = f*180 + 32;
a = num2str(f);
set(handles.edit2,'String',a);
r = f + 495.7;
c = (f - 32)*5/9;
k = c + 273.15;
if chon = = 1
    t = r;
elseif chon = = 2
    t = c;
elseif chon == 3
    t = k;
end
t = num2str(t);
set(handles.edit3,'String',t);
function varargout = radiobutton1_Callback(h, eventdata, handles, varargin)
```

```
global chon;
off = [handles.radiobutton2, handles.radiobutton3];
mutual_exclude(off);
chon = 1;
function varargout = radiobutton2_Callback(h, eventdata, handles, varargin)
global chon;
off = [handles.radiobutton1, handles.radiobutton3];
mutual_exclude(off);
chon = 2;
function varargout = radiobutton3_Callback(h, eventdata, handles, varargin)
global chon;
off = [handles.radiobutton1, handles.radiobutton2];
mutual_exclude(off);
chon = 3;
```

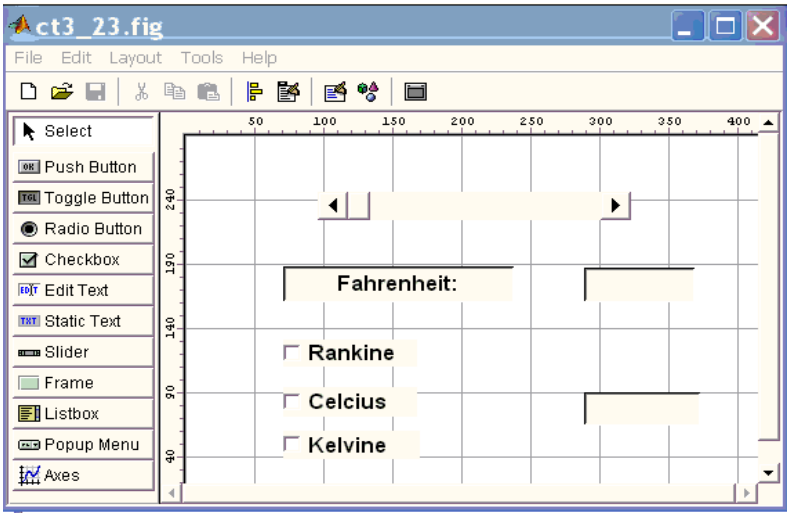
Đoạn lệnh:

```
function mutual_exclude(off)
set(off,'Value',0);
```

làm cho 3 nút lệnh trở thành một nhóm. Các câu lệnh:

```
off = [handles.radiobutton1, handles.radiobutton2];
mutual_exclude(off);
```

làm cho khi chọn một nút Radiobutton này thì không chọn được nút khác nữa. Cuối cùng ta xét phương án dùng Checkbox. Giao diện như sau:



Cũng như phương án dùng Radiobutton, ta dùng ba Checkbox.

Checkbox thứ nhất có thuộc tính FontSize: 12, FontWeight: bold và String: Rankine.

Checkbox thứ hai có thuộc tính FontSize: 12, FơntWeight: bold và String: Celcius.

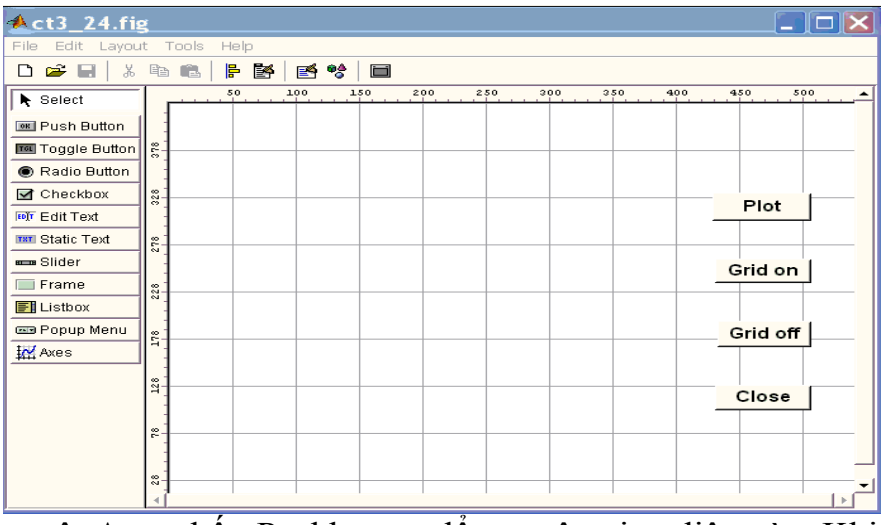
Checkbox thứ ba có thuộc tính FontSize: 12, FơntWeight: bold và String: Kelvine.

Các phần tử khác không co gì thay đổi. Ta lưu file với tên ***ct1_58.fig***. Tiếp theo ta viết mã lệnh cho ***ct1_58.m***:

```
function varargout = ct1_58(varargin)
if nargin = = 0
        fig = openfig(mfilename,'reuse');
         set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
        handles = guihandles(fig);
        guidata(fig, handles);
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1})
        try
                [varargout{1:nargout}] = feval(varargin{:});        catch
                disp(lasterr);
        end
end
function mutual_exclude(off)
set(off,'Value',0);
function varargout = slider1_Callback(h, eventdata, handles, varargin)
global chon
f = get(handles.slider1,'Value');
f = f*180 + 32;
a = num2str(f);
set(handles.edit2,'String',a);
r = f + 495.7;
c = (f - 32)*5/9;
k = c + 273.15;
if chon = = 1
   t = r;
elseif chon = = 2
   t = c;
elseif chon = = 3
   t = k;
end
t = num2str(t);
set(handles.edit3,'String',t);
function varargout = checkbox1_Callback(h, eventdata, handles, varargin)
```

```
global chon;
off = [handles.checkbox2, handles.checkbox3];
mutual_exclude(off);
chon = 1;
function varargout = checkbox2_Callback(h, eventdata, handles, varargin)
global chon;
off = [handles.checkbox1, handles.checkbox3];
mutual_exclude(off);
chon = 2;
function varargout = checkbox3_Callback(h, eventdata, handles, varargin)
global chon;
off = [handles.checkbox2, handles.checkbox1];
mutual_exclude(off);
chon = 3;
```

☞ ***GUI có dùng đồ hoạ***: Ta xây dựng một GUI dùng để vẽ đồ thị hàm y=tsin(t). Giao diện như sau:



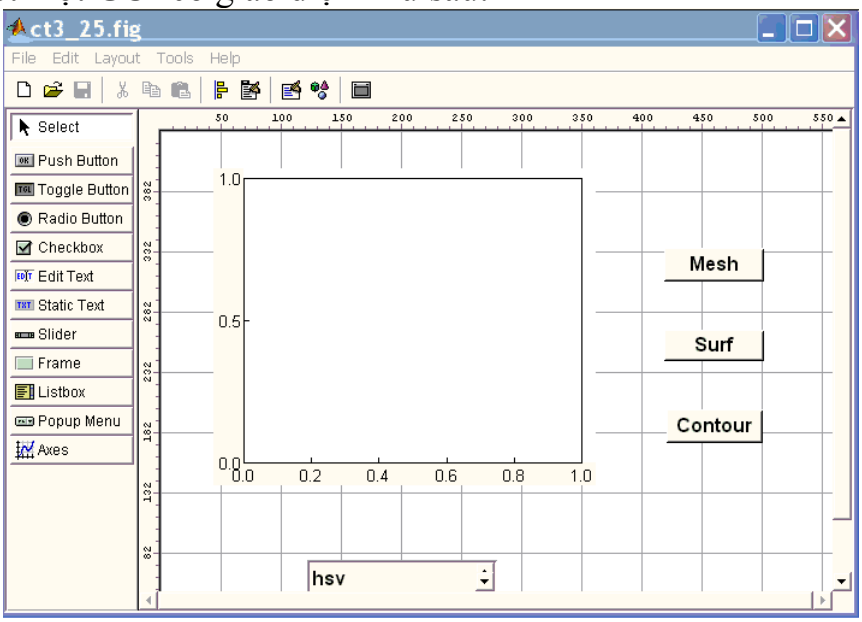
Ta dùng một Axes, bốn Pushbutton để tạo nên giao diện này. Khi nhấn Plot, đồ thị của hàm y = tsin(t) được vẽ. Khi nhấn Grid on, đồ thị đươc chia lưới. Khi nhấn Grod off, lưới bị xoá. Nhấn Close để đóng đồ thị.

Layout có thuộc tính Name: ***ct1_59***, HandleVisibility: callback.
Các Pushbutton đều có thuộc tính FontSize: 12, FontWeight: bold và các String là các tên lệnh. GUI được lưu với tên file là ***ct1_59.fig***. Tiếp theo ta soạn thảo lệnh cho ***ct1_59.m***:

```
function varargout = ct1_59(varargin)
if nargin = = 0
        fig = openfig(mfilename,'reuse');
        handles = guihandles(fig);
        guidata(fig, handles);
```

```
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1})
        try
                [varargout{1:nargout}]    =    feval(varargin{:});    %    FEVAL
switchyard
        catch
                disp(lasterr);
        end
end
function varargout = pushbutton1_Callback(h, eventdata, handles, varargin)
grid on
function varargout = pushbutton2_Callback(h, eventdata, handles, varargin)
grid off
function varargout = pushbutton3_Callback(h, eventdata, handles, varargin)
close
function varargout = pushbutton4_Callback(h, eventdata, handles, varargin)
t = 0:0.01:20;
y = t.*sin(t);
plot(t,y);
```

Tiếp theo ta xét một GUI có giao diện như sau:



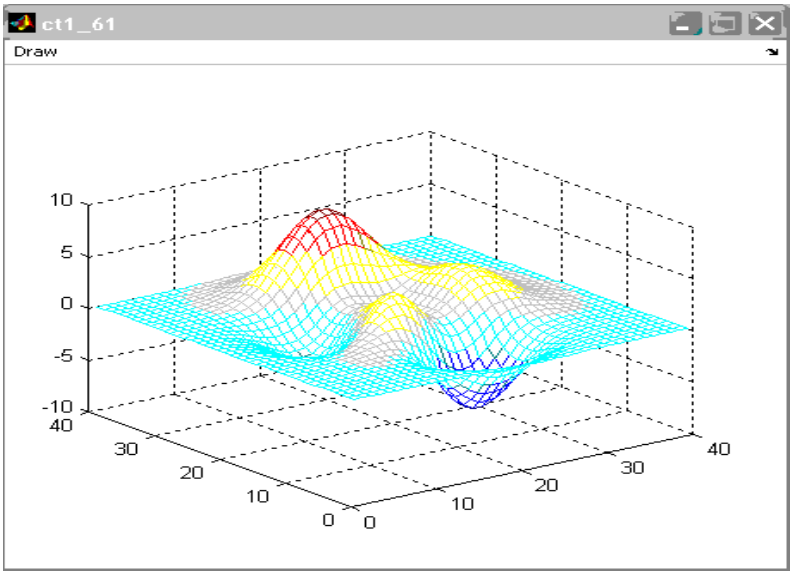
Nhiệm vụ của GUI là vẽ đồ thị của hàm peaks theo các dạng khác nhau(mesh, surf và contour) với các Colormap khác nhau(hsv, hot, gray, prism, cool, winter và summer). Việc vẽ các dạng đồ thị thực hiện nhờ các Pushbutton. Việc chọn Colormap thực hiện nhờ Listbox.

Layout có thuộc tính Name: ***ct1_60*** và thuộc tính HandleVisbility: on. Các Pushbutton đều có thuộc tính FontSize: 12 và FơntWeight: bold. Ta lưu GUI với tên ***ct1_60.fig***. Mã trong ***ct1_60.m*** gồm:

```
function varargout = ct1_60(varargin)
if nargin = = 0
        fig = openfig(mfilename,'reuse');
        set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
        handles = guihandles(fig);
        guidata(fig, handles);
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1})
        try
                [varargout{1:nargout}] = feval(varargin{:});
        catch
                disp(lasterr);
        end
end
function varargout = pushbutton1_Callback(h, eventdata, handles, varargin)
z = peaks(40);
chon = get(handles.listbox1,'Value');
if chon = =1
   colormap(hsv(256));
elseif chon = =2
   colormap(hot(256));
elseif chon = =3
   colormap(gray(256));
elseif chon = =4
   colormap(prism(256));
elseif chon = =5
   colormap(cool(256));
elseif chon = =6
   colormap(winter(256));
elseif chon = =7
   colormap(summer(256));
end
mesh(z);
function varargout = pushbutton2_Callback(h, eventdata, handles, varargin)
z = peaks(40);
chon = get(handles.listbox1,'Value');
if chon = =1
```

```
    colormap(hsv(256));
elseif chon = =2
    colormap(hot(256));
elseif chon = =3
    colormap(gray(256));
elseif chon = =4
    colormap(prism(256));
elseif chon = =5
    colormap(cool(256));
elseif chon = =6
    colormap(winter(256));
elseif chon = =7
    colormap(summer(256));
end
surf(z);
function varargout = pushbutton3_Callback(h, eventdata, handles, varargin)
z = peaks(40);
chon = get(handles.listbox1,'Value');
if chon = =1
    colormap(hsv(256));
elseif chon = =2
    colormap(hot(256));
elseif chon = =3
    colormap(gray(256));
elseif chon = = 4
    colormap(prism(256));
elseif chon = = 5
    colormap(cool(256));
elseif chon = = 6
    colormap(winter(256));
elseif chon = = 7
    colormap(summer(256));
end
contour(z);
```

☞ ***GUI có dùng đồ hoạ***: Ta xây dựng một GUI dùng menu. Giao diện của GUI như sau:



Menu Draw gồm các menu con Mesh, Contour và Close. GUI được lưu trong file ***ct1_61.fig*** và chương trình được lưu trong file ***ct1_61.m***:

```
function varargout = ct1_61(varargin)
gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
                   'gui_Singleton',  gui_Singleton, ...
                   'gui_OpeningFcn', @ct1_61_OpeningFcn, ...
                   'gui_OutputFcn',  @ct1_61_OutputFcn, ...
                   'gui_LayoutFcn',  [] , ...
                   'gui_Callback',   []);
if nargin && ischar(varargin{1})
   gui_State.gui_Callback = str2func(varargin{1});
end
if nargout
   [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
   gui_mainfcn(gui_State, varargin{:});
end
handles.output = hObject;
function varargout = ct1_61_OutputFcn(hObject, eventdata, handles)
varargout{1} = handles.output;
function mnumesh_Callback(hObject, eventdata, handles)
z = peaks(40);
mesh(z);
function Untitled_3_Callback(hObject, eventdata, handles)
z = peaks(40);
contour(z);
function mnuclose_Callback(hObject, eventdata, handles)
```

```
clf
close
function mnudraw_Callback(hObject, eventdata, handles)
```

CHƯƠNG 2: MA TRẬN

§1. MỘT SỐ KHÁI NIỆM

☛ Ma trận [A] gọi là ***đối xứng*** nếu $[A]^T = [A]$

☛ Cho một ma trận vuông [A], cấp n. Ta nói ma trận [A] ***không suy biến*** (non singular) nếu ma trận có thể nghịch đảo được hay nói cách khác, định thức của ma trận khác không.

☛ ***Ma trận Hermite*** là một ma trận vuông có các phần tử là số phức bằng chuyển vị liên hợp của nó, nghĩa là phần tử ở hàng i cột j bằng số phức liên hợp của phân tử ở hàng j cột i $\left[A^*\right]^T = \left[A\right]$. Ví dụ ma trận $[A] = \begin{bmatrix} 3 & 2+j \\ 2-j & 1 \end{bmatrix}$ là ma trận Hermite.

☛ ***Ma trận Householder*** là một ma trận vuông dạng:

$$[H] = [E] - \frac{2}{[U]^T[U]}[U][U]^T$$

Trong đó v là vec tơ cột khác zero

☛ Ma trận [A] gọi là ***trực giao*** nếu $[A]^T[A] = [E]$

☛ Ma trận phức [U] gọi là ***ma trận unita*** nếu $\left[U\right]\left[U^*\right]^T = \left[E\right]$. Ví dụ ma trận

$[U] = \begin{bmatrix} \frac{1+j}{2} & \frac{-1+j}{2} \\ \frac{1+j}{2} & \frac{1-j}{2} \end{bmatrix}$ là ma trận unita

☛ Một ma trận chỉ có một cột gọi là một ***vec tơ***

☛ ***Chuẩn*** của một vec tơ X, kí hiệu là $\|X\|$, là một số thực thoả mãn:

- $\|X\| > 0$
- $\|cX\| = |c|\|X\|$
- $\|X+Y\| \le \|X\| + \|Y\|$

Giả thiết $X = [x_1, x_2, \ldots, x_n]^T$, ta thường dùng một trong 3 chuẩn sau đây:

- $\|X\|_1 = \max_j |x_j|$
- $\|X\|_2 = \sum_{j=1}^{n} |x_j|$
- $\|X\|_3 = \sqrt{\sum_{j=1}^{n} |x_j|^2}$

☛ ***Chuẩn*** của một ma trận [A], kí hiệu là $\|A\|$, là một số thực thoả mãn:

- $\|A\| > 0$

- $\|cA\| = |c|\|A\|$
- $\|A + B\| \le \|A\| + \|B\|$
- $\|AB\| \le \|A\|\|B\|$

Ta thường dùng một trong 3 chuẩn sau đây:

- $\|A\|_1 = \max_i \sum_{j=1}^{n} |a_{i,j}|$
- $\|A\|_1 = \max_j \sum_{i=1}^{n} |a_{i,j}|$
- $\|A\|_3 = \sqrt{\sum_{i,j=1}^{n} |a_{i,j}|^2}$

☛ Ma trận [A] gọi là ***xác định dương*** nếu với vec tơ [x] bất kì ta có:

$$[x]^T[A][x] > 0$$

☛ Ma trận [A] gọi là ***nửa xác định dương*** nếu với vec tơ [x] bất kì ta có:

$$[x]^T[A][x] \ge 0$$

Ta định nghĩa ma trận xác định âm và nửa xác định âm một cách tương tự.

☛ ***Hạng của ma trận*** là cấp của ma trận con của ma trận ấy có định thức khác không còn mọi ma trận con cấp cao hơn đều có định thưc bằng không(ma trận con là ma trận có được bằng cách xoá một số hàng và cột của ma trận ban đầu).

§2. BIẾN ĐỔI HOUSEHOLDER

1. Ma trận Householder: Ta biến đổi ma trận [A] về dạng có các phần tử thuộc đường chéo chính, các phần tử phía trên và phía dưới đường chéo chính khác zero, còn các phần tử còn lại bằng zero(ma trận ba đường chéo) bằng cách dùng phép biến đổi Householder.

Phép biến đổi Householder dùng ma trận Householder.

$$[H] = [E] - \frac{[U][U]^T}{Q} \qquad (1)$$

Trong đó:

$$Q = \frac{1}{2}[U]^T[U] = \frac{1}{2}\|[U]\|^2 \qquad (2)$$

Do [H] đối xứng nên:

$$[H]^T[H] = [H][H] = \left([E] - \frac{[U][U]^T}{Q}\right)\left([E] - \frac{[U][U]^T}{Q}\right)$$

$$= [E] - 2\frac{[U][U]^T}{Q} + \frac{[U]\left([U][U]^T\right)[U]^T}{Q^2}$$

$$=[E]-2\frac{[U][U]^T}{Q}+\frac{[U](2Q)[U]^T}{Q^2}=[E]$$

Từ đây ta thấy [H] cũng là ma trận trực giao.

Cho [X] là vec tơ bất kỳ và khảo sát phép biến đổi [H][X]. Chọn:

$$[U] = [X] + k[I_1] \qquad (3)$$

Trong đó:

$$k=\pm|[X]| \qquad [I_1]=\begin{bmatrix}1 & 0 & \cdots & 0\end{bmatrix}^T$$

Ta có:

$$[H][X]=\left([E]-\frac{[U][U]^T}{Q}\right)[X]=\left\{[E]-\frac{[U]([X]+k[I_1])^T}{Q}\right\}[X]$$

$$=[X]-\frac{[U]\left([X]^T[X]+k[I_1]^T[X]\right)}{Q}=[X]-\frac{[U]\left(k^2+k[X_1]\right)}{Q}$$

Nhưng:

$$2Q=([X]+k[I_1])^T([X]+k[I_1])=|[X]|^2+k\left([X]^T[I_1]+[I_1]^T[X]\right)+k^2[I_1]^T[I_1]$$

$$=k^2+2kx_1+k^2=2(k^2+kx_1)$$

Như vậy:

$$[H][X]=[X]-[U]=-k[I_1]=\begin{bmatrix}-k & 0 & 0 & \cdots & 0\end{bmatrix}^T \qquad (4)$$

nghĩa là phép biến đổi loại trừ tất cả các phần tử của [X] trừ phần tử đầu tiên.

2. Biến đổi Householder một ma trận đối xứng: Bây giờ ta áp dụng phép biến đổi cho ma trận [A] đối xứng:

$$[P_1][A]=\begin{bmatrix}1 & [0]^T\\ [0] & [H]\end{bmatrix}\begin{bmatrix}a_{11} & [X]^T\\ [X] & [A']\end{bmatrix}=\begin{bmatrix}a_{11} & [X]^T\\ [H][X] & [H][A']\end{bmatrix} \qquad (5)$$

Trong đó [X] là cột đầu tiên của [A] với phần tử đầu tiên bị bỏ đi. [A’] có được từ [A] bằng cách bỏ đi cột và hàng đầu tiên. Ma trận [H] cấp (n -1) được xây dựng theo các công thức (1) ÷ (3). Do (4) ta thấy phép biến đổi này làm cột đầu tiên của [A] trở thành:

$$\begin{bmatrix}a_{11}\\ [H][H]\end{bmatrix}=\begin{bmatrix}a_{11}\\ -k\\ 0\\ \vdots\\ 0\end{bmatrix}$$

Phép biến đổi:

$$[P_1][A][P_1]=\begin{bmatrix} a_{11} & ([H][X])^T \\ [H][X] & [H][A'][H] \end{bmatrix} \rightarrow [A] \qquad (6)$$

sẽ đường chéo hoá hàng đầu tiên và cột đầu tiên của ma trận [A]. Sơ đồ biến đổi của ma trận 4×4 là:

$$\begin{array}{|c|c|}\hline 1 & 0\ \ 0\ \ 0 \\ \hline \begin{matrix}0\\0\\0\end{matrix} & [Q] \\ \hline \end{array} \times \begin{array}{|c|c|}\hline a_{11} & a_{12}\ a_{13}\ a_{14} \\ \hline \begin{matrix}a_{21}\\a_{31}\\a_{41}\end{matrix} & [A'] \\ \hline \end{array} \times \begin{array}{|c|c|}\hline 1 & 0\ \ 0\ \ 0 \\ \hline \begin{matrix}0\\0\\0\end{matrix} & [Q] \\ \hline \end{array} = \begin{array}{|c|c|}\hline a_{11} & -k\ \ 0\ \ 0 \\ \hline \begin{matrix}-k\\0\\0\end{matrix} & \begin{matrix}[Q][A']\\[Q]\end{matrix} \\ \hline \end{array}$$

Hàng và cột thứ 2 của ma trận [A] được biến đổi tiếp bằng cách dùng phép biến đổi đối với phần bên phải, phía dưới của ma trận. Phép biến đổi này có thể biểu diễn bằng $[P_2][A][P_2] \rightarrow [A]$, trong đó:

$$[P_2]=\begin{bmatrix} [E_2] & [0]^T \\ [0] & [H] \end{bmatrix} \qquad (7)$$

với $[E_2]$ là ma trận đơn vị 2×2 và [H] là ma trận (n - 2)×(n - 2) có được bằng cách chọn [X] từ (n - 2) phần tử phía dưới của cột thứ 2 của ma trận [A]. Thực hiện (n - 2) phép biến đổi:

$$[P_i]=\begin{bmatrix} [E_i] & [0]^T \\ [0] & [H] \end{bmatrix} \qquad i = 1, 2,..., n - 2$$

để có được ma trận ba đường chéo(tridiagonal). Ta có:

$$[A'][H]=[A']\left([E]-\frac{[U][U]^T}{Q}\right)=[A']-\frac{[A'][U]}{Q}[U]^T=[A']-[V][U]^T$$

Trong đó:

$$[V]=\frac{[A'][U]}{Q} \qquad (8)$$

Do vậy:

$$\begin{aligned}[H][A'][H]&=\left([E]-\frac{[U][U]^T}{Q}\right)\left([A']-[V][U]^T\right)\\ &=[A']-[V][U]^T-\frac{[U][U]^T}{Q}\left([A']-[V][U]^T\right)\\ &=[A']-[V][U]^T-\frac{[U]\left([U]^T[A']\right)}{Q}+\frac{[U]\left([U]^T[V]\right)[U]^T}{Q}\\ &=[A']-[V][U]^T-[U][V]^T+2g[U][U]^T\end{aligned}$$

Trong đó:

$$g = \frac{[U]^T[V]}{2Q} \quad (9)$$

Đặt: [W] = [V] - g[U] (10)

Ta thấy ngay phép biến đổi có dạng:

$$[H][A'][H] = [A'] - [W][U]^T - [U][W]^T \quad (11)$$

Thuật toán có thể tóm lại như sau:

- Cho [A'] là ma trận vuông cấp (n - i) có được từ phần dưới bên phải của ma trận [A]
- Đặt $[X] = \begin{bmatrix} a_{i+1,i} & a_{i+2,i} & \cdots & a_{n,i} \end{bmatrix}^T$
- Tính $\|[X]\|$. Cho k = $\|[X]\|$ nếu $x_1 > 0$ và k = -$\|[X]\|$ nếu $x_1 < 0$
- Cho $[U] = \begin{bmatrix} k + x_1 & x_2 & \cdots & x_{n-i} \end{bmatrix}^T$
- Tính $Q = \frac{\|[U]\|^2}{2}$
- Tính $[V] = \frac{[A'][U]}{Q}$
- Tính $g = \frac{[U]^T[V]}{2Q}$
- Tính [W] = [V] - g[U]
- Tính $[A] = [A'] - [W][U]^T - [U][W]^T$
- Đặt $a_{i,i+1} = a_{i+1,i} = -k$

Ta xây dựng hàm ***housetrans()*** để thực hiện thuật toán trên:

```
function A = housetrans(A)
% Bien doi Householder ma tran A thanh ma tran
% ba đường chéo dang[c\d\c].
% De co c va d dung d = diag(A), c = diag(A,1).
n = size(A, 1);
for k = 1:n-2
   u = A(k+1:n, k);
   uMag = sqrt(dot(u, u));
   if u(1) < 0;
      uMag = -uMag;
   end
   u(1) = u(1) + uMag;
   A(k+1:n, k) = u; % Luu u vao phan duoi cua A.
```

```
    H = dot(u, u)/2;
    v = A(k+1:n,k+1:n)*u/H;
    g = dot(u, v)/(2*H);
    v = v - g*u;
    A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - v*u' - u*v';
    A(k, k+1) = -uMag;
end
k = zeros(n);
for i = 1:n
    k(i, i) = A(i, i);
end
for i = 1:n-1
    k(i, i+1) = A(i, i+1);
    k(i+1, i) = A(i, i+1);
end
A = k;
```

Để tính ma trận ba đường chéo theo phép biến đổi Householder ta dùng chương trình ***cthousetrans.m***:

```
clear all, clc
a = [ 1  2  3  4;  2  9  3  5;  3  3  3  7;  4  5  7  6];
b = householder(a)
d = diag(b)
c = diag(b, 1)
```

§3. BIẾN ĐỔI THÀNH MA TRẬN HESSENBERG

Nếu ma trận [A] là ma trận đối xứng, phương pháp Householder có thể được sử dụng để biến đổi nó thành ma trận đồng dạng đối xứng ba đường chéo. Nếu ma trận [A] không đối xứng, phương pháp Householder biến đổi ma trận [A] thành ma trận đồng dạng Hessenberg.

Ma trận Hessenberg là ma trận có dạng:

$$[H] = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1,n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ 0 & a_{32} & a_{33} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn} \end{bmatrix}$$

Ta thực hiện phép biến đổi Householder trên ma trận [A] và có được:

[Q][H][Q’] = [A]

trong đó [Q] là ma trận trực giao (ta gọi đây là phân tích Hessenberg ma trận [A]) .

Thuật toán có thể tóm lại như sau:

- Cho [Q] là ma trận đơn vị cấp n
- Đặt $\left[X\right]=\left[0 \quad a_{i+2,i} \quad \cdots \quad a_{n,i}\right]^T$
- Tính $\|[X]\|$. Cho $\alpha = \|[X]\|$ nếu $a_{i+2,i} > 0$ và $\alpha = -\|[X]\|$ nếu $a_{i+2,i} < 0$
- Cho $\left[U\right]=\left[0 \quad \alpha + x_2 \quad \cdots \quad x_{n-i}\right]^T$
- Tính $\beta = \dfrac{\|[U]\|^2}{2}$
- Tính $\left[P\right]=\left[E\right]-\dfrac{[U][U']}{\beta}$
- Tính $\left[Q\right]=\left[Q\right]\left[P\right]$
- Tính $\left[A\right]=\left[P\right]\left[A\right]\left[P\right]$

Ta xây dựng hàm ***hessenberg()*** để thực hiện phép phân tích trên:

```
function [H, Q] = hessenberg(a)
[n, n] = size(a);
q = eye(n);
for k = 1:n - 2
   alfa = 0;
   for j = k+1:n
      alfa = alfa + a(j, k)^2;
   end
   alfa = sign(a(k+1, k))*sqrt(alfa);
   u = zeros(1, n);
   u(k+1:n) = a(k+1:n, k);
   u(k+1) = u(k+1) + alfa;
   beta = .5*u*u';
   p = eye(n);
   for i = 1:n
      p(i, 1:n) = p(i, 1:n) - (u(i)*u(1:n))/beta;
   end
   q = q*p;
   a = p*a*p;
end
H = a;
Q = q;
```

Để phân tích ma trận ta dùng chương trình ***cthessenberg.m***:

```
clear all, clc
a = [ 1  2  3  4; 5  6  7  4; 6  4  8  9; 3  5  7  9];
[H, Q] = hessenberg(a)
```

§4. PHÂN TÍCH MA TRẬN THEO PHƯƠNG PHÁP DOOLITTLE

Một ma trận không suy biến [A] gọi là phân tích được thành tích hai ma trận [L] và [R] nếu:

[A] = [L] [R]

Việc phân tích này, nếu tồn tại, là không duy nhất.

Nếu ma trận [L] có các phần tử nằm trên đường chéo chính bằng 1, ta có phép phân tích Doolittle.

Nếu ma trận [R] có các phần tử nằm trên đường chéo chính bằng 1, ta có phép phân tích Crout.

Nếu $[R] = [L]^T$ (hay $[L] = [R]^T$) ta có phép phân tích Choleski.

Với ma trận bậc 3, [L] và [R] có dạng:

$$[L]=\begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \qquad [R]=\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

Để tìm l_{ij} và r_{ij} ta thực hiện phép nhân. Sau khi nhân ta có:

$$[A]=\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{11}l_{21} & r_{12}l_{21}+r_{22} & r_{13}l_{21}+r_{23} \\ r_{11}l_{31} & r_{12}l_{31}+r_{22}l_{32} & r_{13}l_{31}+r_{23}l_{32}+r_{33} \end{bmatrix}$$

Bây giờ ta thực hiện phép khử Gauss đối với phương trình trên. Đầu tiên ta chọn hàng thứ nhất làm trụ và thực hiên phép biến đổi:

hàng 2 - $l_{21} \times$ hàng 1 (khử a_{21}) $\rightarrow$ hàng 2

hàng 3 - $l_{31} \times$ hàng 1 (khử a_{31}) $\rightarrow$ hàng 3

kết quả ta có:

$$[A_1]=\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & r_{22}l_{32} & r_{23}l_{32}+r_{33} \end{bmatrix}$$

Sau đó ta lấy hàng thứ hai làm trụ và thực hiện biến đổi:

hàng 3 - l32 $\times$ hàng 2 (khử a_{32}) $\rightarrow$ hàng 3

và có:

$$[A_2]=\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

Như vậy ta thấy ngay rằng ma trận [R] là ma trận có được khi thực hiện loại trừ Gauss tiến ma trận [A] và các phần tử của [L] là các nhân tử dùng khi loại trừ a_{ij}. Điều đó có nghĩa là để tìm ma trận [L] và [R] ta dùng phép khử Gauss tiến. Ta xây dựng hàm ***doolittle()*** để thực hiện loại phân tích Doolittle.

```
function [l,r] = doolittle(A)
%Phan tich ma tran A thanh A = L*U
n = size(A, 1);
u = zeros(n);
for k = 1:n-1
   for i = k+1:n
      if A(i, k)~= 0.0
         lambda = A(i, k)/A(k, k);
         A(i, k+1:n) = A(i, k+1:n) - lambda*A(k, k+1:n);
         A(i, k) = lambda;
      end
   end
end
l = tril(A);
for i = 1:n
   l(i, i) = 1;
end
l = triu(A);
for i = 1:n
  l(i,i) = A(i, i);
end
```

§5. PHÂN TÍCH MA TRẬN THEO PHƯƠNG PHÁP CROUT

Tương tự như thuật toán Doolittle, ta có thể phân tích ma trận [A] theo thuật toán Crout thành tích của ma trận [L] và [R]. Các ma trận bậc 3 theo Crout có dạng:

$$[L]=\begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \qquad [R]=\begin{bmatrix} 1 & r_{12} & r_{13} \\ 0 & 1 & r_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

Để tìm l_{ij} và r_{ij} ta thực hiện phép nhân. Sau khi nhân ta có:

$$[A]=\begin{bmatrix} l_{11} & l_{11}r_{12} & l_{11}r_{13} \\ l_{21} & l_{21}r_{12}+l_{22} & l_{21}r_{13}+l_{22}r_{23} \\ l_{31} & l_{31}r_{12}+l_{32} & l_{31}r_{13}+l_{32}r_{23}+l_{33} \end{bmatrix}$$

Như vậy:

$a_{11} = 1.\ r_{11} + 0.0 + 0.0 = r_{11}$;

$a_{12} = r_{12}$; $a_{13} = r_{13}$
$a_{21} = l_{21}r_{11}$;
$a_{22} = l_{21}r_{12} + r_{22}$; $a_{23} = l_{31}r_{11}$
$a_{31} = l_{31}r_{11}$; $a_{32} = l_{31}r_{12}$;
$a_{33} = l_{31}r_{13} + l_{32}r_{23} + r_{33}$

Một cách tổng quát ta có :

với j > i : $l_{ij} = r_{ji} = 0$

với i = 1 : $r_{1j} = a_{1j}$ (j = 1 tới n)

$l_{j1} = aj_1/r_{11}$ (j = 1 tới n)

với i = 2 tới n

$$r_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} r_{kj} \quad (\text{ j = i tới n})$$

$$l_{ji} = \frac{a_{ji} - \sum_{k=1}^{i-1} l_{jk} r_{ki}}{r_{ii}} \quad (\text{j = i tới n})$$

Ta xây dựng hàm ***crout()*** để phân tích ma trận theo thuật toán Crout:

```
function [l, r] = crout(a)
n = size(a, 1);
l = zeros(n);
r = zeros(n);
for i = 1:n
   r(1, i) = a(1, i);
   l(i, i) = 1.;
   l(i, 1) = a(i, 1)/a(1, 1);
end
for k = 2:n
   r(k, k:n) = a(k, k:n) - l(k, 1:k)*r(1:k, k:n);
   if k~= n
      for i = 1:n
         l(i, k) = (a(i, k)- l(i, 1:k-1)*r(1:k-1, k))/r(k, k);
      end
   end
end
```

§6. PHÂN TÍCH MA TRẬN THEO PHƯƠNG PHÁP CHOLESKI

Thuật toán Choleski cho phép phân tích ma trận [A] thành tích hai ma trận: $[A] = [L][L]^T$.

Thuật toán này đòi hỏi:

- [A] là ma trận thực, đối xứng

- [A] là ma trận xác định dương

Ta vuông [A] cấp 3 theo thuật toán Choleski:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

Sau khi thực hiện phép nhân ta có:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11}^2 & l_{11}l_{21} & l_{11}l_{31} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} \\ l_{11}l_{31} & l_{21}l_{31} + l_{22}l_{32} & l_{31}^2 + l_{32}^2 + l_{33}^2 \end{bmatrix}$$

Vế phải là ma trận đối xứng. Cân bằng các phần tử của hai ma trận ta có:

$$l_{11} = \sqrt{a_{11}} \qquad l_{21} = a_{21}/l_{11} \qquad l_{31} = a_{31}/l_{11}$$

$$l_{22} = \sqrt{a_{22} - l_{21}^2} \qquad l_{32} = (a_{32} - l_{21}l_{31})/l_{22} \qquad l_{33} = \sqrt{a_{33} - l_{31}^2 - l_{32}^2}$$

Tổng quát, với ma trận cấp n, ta có:

$$\left([L][L]^T\right)_{ij} = l_{i1}l_{j1} + l_{i2}l_{j2} + \cdots + = \sum_{k=1}^{j} l_{ik}l_{jk} \quad i \ge j$$

Cân bằng với phần tử của ma trận [A] ta có:

$$a_{ij} = \sum_{k=1}^{j} l_{ik}l_{jk} \quad i = j, j+1, \ldots, n \quad j = 1, 2, \ldots, n$$

Do ma trận [L] là ma trận tam giác trái nên đối với cột thứ nhất ta có:

$$l_{11} = \sqrt{a_{11}} \qquad l_{i1} = a_{i1}/l_{11}$$

Đối với cột khác, rút l_{ij} ra khỏi tổng ta có:

$$a_{ij} = \sum_{k=1}^{j-1} l_{ik}l_{jk} + l_{ij}l_{jj}$$

Nếu i = j (phần tử trên đường chéo) thì:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2} \qquad j = 2, 3, \ldots, n$$

và phần tử nằm ngoài đường chéo:

$$l_{ij} = \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik}l_{jk}\right)\frac{1}{l_{jj}} \qquad j = 2, 3, \ldots, n \qquad i = j+2, j+3, \ldots, n$$

Dựa vào thuật toán trên ta xây dựng hàm ***choleski()***

```
function L = choleski(A)
% Phan tich ma tran a thanh A = LL'.
% Cu phap: L = choleski(A)
f = posdef(A);
```

```
if f == 0
   error('Ma tran khong xac dinh duong!');
   return
end
n = size(A, 1);
for j = 1:n
   temp = A(j, j) - dot(A(j, 1:j-1),A(j, 1:j-1));
   if temp < 0.0
      error('Ma tran khong xac dinh duong')
   end
   A(j, j) = sqrt(temp);
   for i = j+1:n
      A(i, j)=(A(i, j) - dot(A(i, 1:j-1),A(j, 1:j-1)))/A(j, j);
   end
end
L = tril(A);

function f = posdef(M)
%Kiem tra lieu ma tran M co xac dinh duong hay kong
isposdef = true;
for i=1:length(M)
  if ( det( M(1:i, 1:i) ) <= 0 )
    isposdef = false;
    break;
  end
end
f = isposdef;% 0 neu sai, 1 neu dung
```

§7. PHÂN TÍCH QR BẰNG THUẬT TOÁN HOUSEHOLDER

Cho ma trận [A], phân tích QR của nó cho ta:

[A] = [Q]*[R]

Trong đó [Q] là ma trận trực giao và [R] là ma trận tam giác phải.

Ta dùng biến đổi Householder để tìm các ma trận [Q] và [R].

$$[H_{n-1}][H_{n-2}]\cdots[H_1][A]=[R] \tag{1}$$

Như vậy:

$$\begin{aligned}[A]&=\left([H_{n-1}][H_{n-2}]\cdots[H_1]\right)^{-1}[R]=[H_1]^{-1}\cdots[H_{n-2}]^{-1}[H_{n-1}][R]\\&=[H_1]\cdots[H_{n-2}][H_{n-1}][R]=[Q][R]\end{aligned} \tag{2}$$

Tích của tất cả các ma trận Householder:

$$[Q]=[H_1]\cdots[H_{n-2}][H_{n-1}] \tag{3}$$

không những đối xứng mà còn trực giao như mỗi ma trận $[H_k]$:

$$[Q]^T[Q] = \left([H_1]\cdots[H_{n-2}][H_{n-1}]\right)^T [H_1]\cdots[H_{n-2}][H_{n-1}]$$
$$= [H_{n-1}]^T[H_{n-2}]^T \cdots [H_1]^T[H_1]\cdots[H_{n-2}][H_{n-1}] = [E]$$

Ta xây dựng hàm ***qrdecom()*** để phân tích ma trận:

```
function [Q, R] = qrdecom(A)
%Phan tich QR
n = size(A, 1);
R = A;
Q = eye(n);
for k = 1:n - 1
   H = householder(R(:, k), k);
   R = H*R; %Pt.(1)
   Q = Q*H; %Pt.(3)
end
```

Hàm ***householder()*** dùng để tạo ra ma trận Householder:

```
function H = householder(x, k)
% Tao ma tran Householder
n = length(x);
tmp = sum(x(k+1:n).^2);
g = sqrt(x(k)^2 + tmp);
c = sqrt((x(k) + g)^2 + tmp);
u = zeros(n, 1);
u(k) = (x(k) + g)/c;
u(k + 1:n) = x(k + 1:n)/c;
H = eye(n) - 2*u*u'; %ma tran Householder
```

Để phân tích ma trận ta dùng chương trình ***ctqrdecom.m***:

```
clear all, clc
a = [4 1 3 -2; 1 -2 4 1; 3 4 1 2; -2 1 2 3];
[q, r] = qrdecom(a)
```

§8. PHÂN TÍCH QR BẰNG THUẬT TOÁN QUAY GIVENS

Kỹ thuật quay Givens là một phương pháp để phân tích ma trận [A] thành tích của ma trận [Q] và ma trận [R] bằng cách làm cho các phần tử lần lượt bằng zero cho đến khi có được ma trận tam giác phải. Ý tưởng là dùng một ma trận quay đơn giản 2 × 2 đặt dọc theo đường chéo chính của một ma trận đơn vị và làm cho một phần tử

của ma trận bằng zero. Các phần tử của ma trận quay để quay một vec tơ ngược chiều kim đồng hồ một góc θ là:

$$[Q_\theta] = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Nếu ta muốn quay vec tơ $[x_1\ x_2]^T$ và muốn làm cho x_2 bằng zero rồi quaytheo chiều kim đồng hồ một góc θ(hay ngược chiều kim đồng hồ một góc -θ) trong đó:

$$\theta = \text{arctg}\frac{x_2}{x_1}$$

thì ma trận quay để thực hiện phép quay này theo chiều kim đồng hồ một góc θ là:

$$[Q_\theta] = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Trong đó:

$$\cos\theta = c = \frac{x_1}{\sqrt{x_1^2 + x_2^2}} \qquad \sin\theta = s = \frac{x_2}{\sqrt{x_1^2 + x_2^2}}$$

Do đó:

$$[Q_\theta] = \frac{1}{\sqrt{x_1^2 + x_2^2}} \begin{bmatrix} x_1 & x_2 \\ -x_2 & x_1 \end{bmatrix} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$$

Chú ý là như mong muốn:

$$[Q_\theta]\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} cx_1 + sx_2 \\ -sx_1 + cx_2 \end{bmatrix} = \begin{bmatrix} \dfrac{x_1^2 + x_2^2}{\sqrt{x_1^2 + x_2^2}} \\ 0 \end{bmatrix} = \begin{bmatrix} \sqrt{x_1^2 + x_2^2} \\ 0 \end{bmatrix}$$

Nếu A là ma trận m × n, ta sẽ xem điều gì xảy ra khi ta thay các phần tử của [Q] vào ma trận con xác định bằng các cột và hàng thứ i, các cột và hàng thứ j. Nói cách khác ta thay ma trận 2 × 2 này dọc theo đường chéo chính tại một số điểm:

$$[G_{kl}] = \begin{cases} \delta_{kl} & k \neq i, l \neq j \\ c & k, l = i; k, l = j \\ s & k = i; l = j \\ -s & k = j; l = i \end{cases} = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & c & \cdots & s & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & \cdots & -s & \cdots & c & \cdots & 0 \\ \vdots & \vdots & 0 & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix}$$

Như vậy [G] là ma trận đơn vị m × m ngoại trừ các giá trị đã bị thay thế:

$g_{ii} = g_{jj} = c$

$g_{ij} = -g_{ij} = s$

Điều này sẽ tạo ra ma trận unita:

$$[G]^T[G] = [E]$$

nghĩa là:

$$\sum_l g_{lk} g_{lp} = \delta_{kp}$$

và đòi hỏi:

$$c^2 + s^2 = 1$$

Điều này đúng vì $\cos^2\theta + \sin^2\theta = 1\ \forall\theta$. Khi ma trận này được áp dụng cho ma trận m $\times$ n ta có:

$$b_{kp} = \sum_l g_{kl} a_{lp} = \begin{cases} \sum_l \delta_{kl} a_{lp} = a_{kp} & k \neq i, j \\ \sum_l g_{il} a_{lp} = c a_{ip} + s a_{jp} & k = i \\ \sum_l g_{jl} a_{lp} = -s a_{ip} + c a_{jp} & k = j \end{cases}$$

Như vậy ma trận mới chỉ bị thay đổi ở hàng i và cột j. Ta chọn s và c sao cho các phần tử ở cột r và hàng j bằng zero:

$$s = \frac{a_{jr}}{\sqrt{a_{jr}^2 + a_{ir}^2}} \qquad c = \frac{a_{ir}}{\sqrt{a_{jr}^2 + a_{ir}^2}}$$

Như vậy ta sẽ có:

$$b_{jr} = \frac{-a_{jr} a_{ir} + a_{ir} b_{jr}}{\sqrt{a_{jr}^2 + a_{ir}^2}} = 0$$

Ta xây dựng hàm ***givens()*** để thực hiện thuật toán trên:

```
function [Q, R] = givens(A);
% Phan tich QR bang thuat toan quay Givens
n = size(A, 1);
Q = eye(n);
for j = 1:n-1
  for i = n:-1:j+1
    z = 1/sqrt(A(i-1, j)^2 + A(i, j)^2);
    c = A(i-1, j)*z;
    s = A(i, j)*z;
    A(i-1:i,:) = [c s; -s c]*A(i-1:i,:);
    Q(i-1:i,:) = [c s; -s c]*Q(i-1: i,:);
  end
end
R = A;
```

```
Q = Q';
```

Để phân tích một ma trận ta dùng chương trình ***ctgivens.m***:

```
clear all, clc
A = [17 24 30 17; 8 13 20 7; 2 10 8 6; -23 -43 -54 -26];
[Q, R] = givens(A)
```

§9. PHÂN TÍCH QR BẰNG THUẬT TOÁN GRAM - SCHMIDT

Ta có thể thực hiện việc phân tích ma trận [A] thành tích các ma trận [Q] và [R] bằng cách trực giao hoá các cột của ma trận [A]. Ta gọi các cột của ma trận [A] là $a_1,...,a_n$. Từ các vec tơ này ta muốn có n vec tơ trực giao $v_1,...,v_n$. Vec tơ trực giao đầu tiên được chọn là:

$$v_1 = a_1$$

Để có vec tơ thứ hai, ta dùng y_2 nhưng trừ bớt đi phần y_2 cùng chiều với v_2. Như vậy ta có:

$$v_2 = y_1 - ba_1$$

với b được chọn sao cho v_1 trực giao với v_2:

$$v_1 v_2 = v_1(a_2 - bv_1) = v_1 a_2 - bv_1 v_1 = 0$$

hay:

$$b = \frac{v_1 a_2}{v_1 v_1}$$

Tiếp tục quá trình đến bước thứ k ta có:

$$v_k = a_k - \sum_{i=1}^{k-1} \frac{v_i a_k}{v_i v_i} v_i$$

Như vậy thuật toán gồm các bước:

- $r_{11} = \|a_1\|,\ q_1 = \dfrac{a_1}{r_{11}}$
- lặp từ k = 2 đến n

$$q_k = \left(a_k - \sum_{i=1}^{k-1} r_{ik} q_i \right) \Big/ r_{kk}$$

với

$$r_{ik} = q_i^T a_k$$

và r_{kk} được chọn sao cho $\|q_k\| = 1$, nghĩa là:

$$z = a_k - q_k r_{ik}$$

$$r_{kk} = \|z\|$$

Ta xây dựng hàm ***qrgramschmidt()*** để thực hiện thuật toán trên:

```
function [Q, R] = qrgramschmidt(A);
% Phan tich mt bang thuat toan Gram - Schmidt
[m,n] = size(A);
R(1,1) = norm(A(:, 1));
Q(:,1) =A(:, 1)/R(1, 1);
for k = 2:n
        R(1:k-1, k) = Q(1:m, 1:k-1)'*A(1:m, k);
        z = A(1:m, k) - Q(1:m, 1:k-1)*R(1:k-1, k);
        R(k,k) = norm(z);
        Q(1:m,k) = z/R(k, k);
end
```

Để phân tích một ma trận ta dùng chương trình chương trình ***ctqrgamschmidt.m***:

```
clear all, clc
a = [ 1 2 3 4 5; 6 7 8 9 0; 3 4 5 6 7; 8 9 0 1 2; 2 4 6 8 1];
[q, r] = qrgramschmidt(a)
```

§10. PHÂN TÍCH MA TRẬN THEO GIÁ TRỊ RIÊNG

Cho ma trận [A], ta có:

$[A][X] = \lambda[X]$

Nếu ta đặt [U] là một ma trận mà các cột của nó là các vec tơ riêng của ma trận [A] và ma trận [Λ] là ma trận đường chéo có các phần tử trên đường chéo chính là λ_i thì:

$[A][U] = [\Lambda][U]$

hay:

$[A] = [U][\Lambda][U]^{-1}$

Dạng này của ma trận được gọi là dạng phân tích theo giá trị riêng và vec tơ riêng. Ta dùng chương trình ***cteigdecom.m*** để phân tích ma trận:

```
clear all, clc
a = [ 1  3  5; 3  4  9;  5  9  6];
[L, U] = eigjacobi(a)
```

§11. PHÂN TÍCH LQ

Cho ma trận $[A]^T$, ta có thể phân tích QR ma trận này thành:

$[A]^T = [Q_1][R_1]$

Do $([Q][R])^T = [R_1]^T[Q_1]^T$ nên:

$([A]^T)^T = [A] = [L][Q]$

và ta nhận được phân tích LQ của ma trận [A]. Ta xây dựng hàm ***lqdecom()*** để thực hiện thuật toán này:

```
function [Q, L] = lqdecom(A)
A = A';
[Q, L] = qrdecom(A);
L = L';
Q = Q';
```

Để phân tích một ma trận ta dùng chương trình ***ctlqdecom.m***:

```
clear all, clc
a = [ 1  3  5; 2  4  6; 7  8  9];
[Q, L] = lqdecom(a)
```

§12. PHÂN TÍCH JORDAN

1. Ma trận có thể đường chéo hoá: Ma trận [A] gọi là có thể đường chéo hoá nếu và chỉ nếu tồn tại phép biến đổi đồng dạng [V] sao cho $[A] = [V][\Lambda][V]^{-1}$ trong đó [Λ] là ma trận đường chéo $[\Lambda] = \mathrm{diag}(\lambda_1, \lambda_2,..., \lambda_n)$. Điều kiện cần để [A] có thể đường chéo hoá là [A] có n vec tơ riêng độc lập tuyến tính. Điều kiện đủ để [A] có thể đường chéo hoá là [A] có n giá trị riêng phân biệt vì khi [A] có n giá trị riêng phân biệt thì các vec tơ riêng tương ứng là độc lập tuyến tính. Số lần lặp lại m_i của giá trị riêng λ_i gọi là vô số đại số (algebraic multiplicity) của λ_i, kí hiệu là $AM(\lambda_i)$. Số vec tơ riêng độc lập tuyến tính tương ứng với giá trị riêng λ_i gọi là vô số hình học (geometric multiplicity) của λ_i, kí hiệu là $GM(\lambda_i)$.

2. **Dạng Jordan**: Khi không thể tìm được n giá trị riêng phân biệt, nghĩa là ma trận [A] không có n vec tơ riêng độc lập tuyến tính thì ma trận [A] không thể đường chéo hoá. Tuy nhiên, nếu có phép biến đổi đồng dạng [M] biến đổi [A] thành [J]:

$$[A] = [M][J][M]^{-1}$$

Trong đó [J] là ma trận gần đường chéo:

$$[J] = \mathrm{diag}(J_1,..., J_n)$$

$$[J_i] = \begin{bmatrix} \lambda_i & 1 & 0 & \cdots & 0 \\ 0 & \lambda_i & 1 & \ddots & \vdots \\ \vdots & \ddots & \lambda_i & \ddots & \vdots \\ \vdots & \ddots & \ddots & \lambda_i & 1 \\ \vdots & \cdots & \cdots & 0 & \lambda_i \end{bmatrix}$$

là khối Jordan và ma trận [J] được gọi là dạng Jordan kinh điển của ma trận [A]. Số khối Jordan bằng số vec tơ riêng độc lập tuyến tính của ma trận [A], nghĩa là bằng $GM(\lambda_i)$. Cụ thể, mỗi vec tơ riêng độc lập tuyến tính tương ứng với mỗi khối. Do vậy nếu ma trận [A] có các vec tơ riêng độc lập tuyến tính thì dạng Jordan trùng với dạng

đường chéo của ma trận $[S]^{-1}[A][S] = [\Lambda]$ trong đó $[\Lambda] = \text{diag}(\lambda_1,..., \lambda_n)$ và [S] có các cột là các vec tơ riêng của [A]

3. Xây dựng dạng Jordan của ma trận [A]: Khi [A] không có n vec tơ riêng độc lập tuyến tính để tạo ra các cột của ma trận [M] thì ta có thể thêm các vec tơ độc lập tuyến tính vào các vec tơ riêng để tạo ra ma trận này.

Trước hết ta khảo sát một giá trị riêng λ_i có $GM(\lambda_i) < AM(\lambda_i)$. Nếu $GM(\lambda_i) = p_i$, $AM(\lambda_i) = m_i$ thì ta cần tìm $m_i - p_i$ vec tơ độc lập tuyến tính để kết hợp với giá trị riêng này. Các vec tơ này được tạo từ các vec tơ riêng và được gọi là vec tơ riêng tổng quát hoá của [A]. Gọi λ là giá trị riêng và [x] là vec tơ riêng tương ứng. k - 1 vec tơ riêng tổng quát hoá $\{[x_1],..., [x_k]\}$ được tạo ra như sau:

$$[A][x_1] = \lambda[x_1]$$
$$[A][x_2] = \lambda[x_2] + [x_1]$$
$$\vdots$$
$$[A][x_k] = \lambda[x_k] + [x_{k-1}]$$

$\{[x_1],..., [x_k]\}$ tạo thành chuỗi các vec tơ có vec tơ $[x_1]$ đứng đầu. Chuỗi này tương ứng với khối Jordan đơn.

$$[A][x_1,\ldots,x_n] = [x_1,\ldots,x_n]\begin{bmatrix} \lambda & 1 & 0 & \cdots & 0 \\ 0 & \lambda & 1 & 0 & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & \cdots & \lambda & 1 \\ 0 & \cdots & \cdots & 0 & \lambda \end{bmatrix}$$

Xuất phát từ vec tơ tổng quát hoá bậc k của [A] ứng với λ, kí hiệu là $[x_k]$ ta có:

$$[x_k]$$
$$[x_{k-1}] = ([A] - \lambda[E])[x_k]$$
$$\vdots$$
$$[x_i] = ([A] - \lambda[E])^{k-i}[x_k]$$
$$\vdots$$
$$[x_1] = ([A] - \lambda[E])^{k-1}[x_k]$$

Chú ý là $[x_1]$ là một vec tơ riêng của [A] vì:

$$([A] - \lambda[x_1]) = ([A] - \lambda[E])([A] - \lambda[E])^{k-1}[x_k]$$

Để phân tích ma trận [A] ta dùng thuật toán Filipov gồm các bước sau:

- Giả sử rằng kích thước không gian cột của ma trận [A] là r < n. Phải có r vec tơ độc lập tuyến tính $[x_i]$ trong không gian cột mà nó là các vec tơ riêng hay vec tơ riêng tổng quát hoá, nghĩa là $[A][x_i] = \lambda[x_i]$ hay $[A][x_i] = \lambda[x_i] + [x_{i-1}]$

- Giả sử rằng không gian không và không gian cột của ma trận [A] có phần chung với kích thước p. Mỗi vec tơ $[x_i]$ trong jg guan không của [A] là một vec tơ

riêng tương ứng với $\lambda = 0$, nhưvậy $[A][x_i] = 0$. Bây giờ nếu $[x_i]$ cũng là không gian cột của [A] thì $[x_i] = [A][y_i]$ với mọi $[y_i]$

- Cuối cùng do kích thước của không gian không của [A] là n - r và p của các vec tơ là trong cả không gian không lẫn không gian cột nên có n - r - p vec tơ $[z_i]$ ở trong không gian không mà không ở trong không gian cột.

Các vec tơ $[x_i]$, $[y_i]$ và $[z_i]$ tìm được là độc lập tuyến tính. Chúng tạo nên các cột của [M] và $[J] = [M][A][M]^{-1}$ là dạng Jordan.

Ta xây dựng hàm ***jordandecom()*** thực hiện thuật toán trên:

```
function [M, J] = jordandecom(a)
%Tinh phan tich Jordan cua ma tran A
% sao cho A*M = M*J
small = 2*sqrt(eps);
[r, c] = size(a);
if r ~= c
  error('Ma tran A phai la ma tran vuong!')
end
n = r;
if n == 1
  J = a;
  M = 1;
  return
end
if n<1
  J = [];
  M = [];
  return
end
[m, d] = eig(hess(a));
d = sort(diag(d));
tiny = norm(a)*eps;
%lam cac gia tri bang zero
p = find(abs(d)<=tiny);
if ~isempty(p)
  d(p) = 0;
end
%A*M=M*J
[M, J] = jord(a, d, small);

function [M, D] = jord(a, d, small)
%Tinh phan tich Jordan cua ma tran A
```

```
norma = sqrt(mean(mean(abs(a))));
tiny = norma*eps;
if nargin<3
   small = (1 + norma)*sqrt(eps);
end
[r, c] = size(a);
if r~=c
   error('A phai la ma tran vuong!')
end
n = r;
I = eye(n);
if r == 1
   D = a;
   M = 1;
   return
end
if r<1
   D = [];
   M = [];
   return
end
condofa = cond(a);
if condofa>1e6
   Condition_number_of_A = condofa
   warning('So dieu kien cua A qua lon!');
end
d = d(:);
if size(d,1) ~= n
   d = d
   error('Gia tri rieng khong dung!')
end
da = det(a);
dp = prod(d);
e = abs(abs(da) - abs(dp));
if e>sqrt(eps)
   disp(' ')
   warning('Cac gia tri rieng co the khong chinh xac!')
end
ds = flipud(sort(d));
sds = size(ds,1);
du = flipud(unique(ds));
```

```
sdu = size(du, 1);
if sdu == sds
   [M, D] = eig(a);
  return
end
M = [];
for kk = 1:sdu
      e = du(kk);
      ameig = sum(ismember(ds, e));
      a1 = a - e*I;
   if ameig == 1
      [u, s, v] = svd(a1);
      M =[M v(:, end)];
   else
      pp = 0;
      ns = [];
      pp = pp + 1;
      aa = I;
      for k = 1:ameig
          aa = a1*aa;
          nn = size(nulld(aa, small), 2);
          ns(k) = nn;
      end
      nsaa = [0; ns(:)]';
      dns = diff(nsaa);
      if max(dns) ~= dns(1)
         Cond_of_A = cond(a)
         save jord
         M = I;
         D = I;
         error('Kich thuoc khong gian khong sai')
      end
      clear ec;
      ec(1:dns(1)) = 1;
      for k = 2:length(dns)
          ec(1: dns(k)) = ec(1:dns(k)) + 1;
      end
      if sum(ec) ~=a meig
         Cond_of_A = cond(a)
         save jord
         M = I;
```

```
        D = I;
        error('Kich thuoc khong gian khong sai')
      end
      k = 1;
     clear jv;
     while k<= dns(1)
        p = find(ec == ec(k));
        if isempty(p)
            Cond_of_A = cond(a);
            save jord
           M = I;
           D = I;
           error('Kich thuoc khong gian khong sai');
        end
        aa = I;
        for m = 1:ec(k)
            aa = aa*a1;
        end
        pp = max(size(p));
        v = nulld(aa, small);
        jv(:,p) = v*(rand(size(v, 2), pp) - 0.5)*16;
        k = k + pp;
     end
     clear v;
     for k = 1:dns(1)
        v(:,1) = jv(:, k);
        for p = 2:ec(k)
            v(:, p) = a1*v(:, p-1);
        end
        vv = fliplr(v(:, 1:ec(k)));
        M = [M vv];
     end
   end
end
k = abs(det(M))^(-1/n);
M = k*M;
Mi = inv(M);
D = Mi*a*M;
d0 = diag(D);
d1 = diag(D, 1);
D = diag(d0) + diag(d1, 1);
```

```
function Z = nulld(A, small)
norma = sqrt(mean(mean(abs(A))));
tiny = norma*eps;
if nargin<2
   small = (1 + norma)*sqrt(eps);
end
[m, n] = size(A);
if  m~= n
   error('Ma tran phai vuong!')
end
p = find(abs(A)<tiny);
if ~isempty(p)
   A(p) = 0;
end
[U,S,V] = svd(A, 0);
S = diag(S);
s = S;
norma = max(s);
smax = max(s);
if smax == 0;
    smax = 1;
end
s = s/smax;
snorm = s;
t = find(s>0);
if isempty(t);
    Z = V;
    return;
end
p = find(s<tiny);
if ~isempty(p)
   s(p) = tiny;
end
p = find(s == 0);
if ~isempty(p)
   s(p) = small*min(s(t));
end
logs = log10(s);
sdifflog = -diff(logs);
smax = max(sdifflog);
```

```
r = find(sdifflog == smax);
if min(size(r))>0
  r = r(end);
else
  r = 0;
end
Z = V(:,r+1:n);
if snorm == ones(n, 1);
   Z = zeros(n, 0);
end
if max(S) <= small;
   Z = V;
end
```

§13. PHÂN TÍCH MA TRẬN THEO CÁC GIÁ TRỊ KÌ DỊ

Phân tích ma trận theo các giá trị kì dị (kì dị value) được thực hiện trên các ma trận vuông hay chữ nhật. Ta có:

$[A_{np}] = [U_{nn}][S_{np}][V_{pp}]$

Trong đó:

$[U]^T[U] = [E_{nn}]$

$[V]^T[V] = [E_{pp}]$

nghĩa là các ma trận [U] và [V] là trực giao.

Các cột của [U] là các vec tơ kì dị trái, [S] có các giá trị kì dị và là ma trận đường chéo và $[V]^T$ có các hàng là các vec tơ kì dị phải. Để tính các ma trận [U], [S] và [V] ta tìm các giá trị riêng của $[A][A]^T$ và $[A]^T[A]$. Các vec tơ riêng của $[A]^T[A]$ tạo nên các cột của [V]. Các vec tơ riêng của $[A][A]^T$ tạo nên các cột của [U]. Các giá trị kì dị của [S] là căn bậc hai của các giá trị riêng của $[A][A]^T$ hay $[A]^T[A]$. Các giá trị riêng trên đường chéo của [S] được sắp xếp theo thứ tự giảm dần. Để hiểu được thuật toán này ta xét ví dụ sau:

Ta xây dựng hàm ***svddecomp()*** để thực hiện thuật toán này:

```
function [U, S , V] = svddecomp(A)
[m, n] = size(A);
if (m > n)
   % ta can timcac vec to kì dị phai truoc
   %[V D] = eigs(A'*A)
   [V, D] = eigs(A'*A);
   [dummy, perm] = sort(-diag(D));
   S = diag(sqrt(diag(D(perm, perm))));
   V = V(:, perm);
   sinv = diag(1./sqrt(diag(D)));
```

```
    U = (A*V)*sinv;
else
    % ta can tim cac vec to kì dị trai truoc
    % [U D] = eigs(A*A')
    [U, D] = eigs(A*A');
    [dummy, perm] = sort(-diag(D));
    S = diag(sqrt(diag(D(perm, perm))));
    U = U(:, perm);
    sinv = diag(1./sqrt(diag(D)));
    V = sinv*(U'*A);
    V = V';
end
```

Để phân tích một ma trận ta dùng chương trình ***ctsvddecomp.m***:

```
clear all, clc
%a = [ 1 2 3 4 5; 6 7 8 9 0; 3 4 5 6 7; 8 9 0 1 2; 2 4 6 8 1];
a = [ 1 1; 0 1; 1 0];
[u, s, v] = svddecomp(a)
```

§14. PHÂN TÍCH SCHUR

Cho ma trận vuông [A], cấp n ta phân tích nó thành:

[A] = [T][U][T]*

Trong đó:

[T] - ma trận unita và $[T]^*$ là ma trận chuyển vị liên hợp của [T](lấy chuyển vị của [T] rồi lấy liên hợp của các phần tử).

[U] = [Λ] + [N]

[Λ] - là ma trận đường chéo có các phần tử là các giá trị riêng của [A]

[N] - ma trận tam giác phải, có các phần tử thuộc đường chéo chính bằng zero.

Mọi ma trận vuông đều có thể phân tích Schur. Tuy nhiên phân tích này không duy nhất. Nếu [A] là ma trận trực giao thì [U] là ma trận đường chéo và các cột của [T] là các vec tơ riêng của [A]. Phân tích Schur khi này được gọi là phân tích phổ. Nếu [A] xác định dương, phân tích Schur chính là phân tích SVD.

Để phân tích ma trận theo thuật toán Schur ta thực hiện các bước sau:

- Tìm giá trị riêng λ_1 của [A] và vec tơ riêng tương ứng $[v_1]$
- Chọn n- 1 vec tơ $[w_1],...,[w_{n-1}]$ độc lập tuyến tính và trực giao với $[v_1]$
- Tạo $[V_1]$ là ma trận có các cột là $[v_1]$, $[w_1],...,[w_{n-1}]$ và tính:

$$[V_1]^*[A][V_1] = \begin{bmatrix} \lambda_1 & * \\ 0 & [A_1] \end{bmatrix}$$

Trong đó $[A_1]$ là ma trận (n-1)×(n-1)

- Lặp lại quá trình với ma trận $[A_1]$ ta có:

$$[V_2]^*[A_1][V_2] = \begin{bmatrix} \lambda_2 & * \\ 0 & [A_2] \end{bmatrix}$$

Trong đó $[A_2]$ là ma trận (n-2)×(n-2)

- Do $[T_2]^*[A]^*[T_2] = \begin{bmatrix} \lambda_1 & * & * \\ 0 & \lambda_1 & * \\ 0 & 0 & [A_2] \end{bmatrix}$

Trong đó $[T_2] = [V_1][\hat{V}_2]$ với $[\hat{V}_2] = \begin{bmatrix} 1 & 0 \\ 0 & [V_2] \end{bmatrix}$

- Tiếp tục quá trình ta tìm được $[V_1],...,[V_n]$. Cuối cùng $[U] = [T]^*[A][T]$

$$[T_2] = [V_1][\hat{V}_2]\cdots[\hat{V}_n]$$

Ta xây dựng hàm ***schurdecom()*** thực hiện thuật toán trên:

```
function [T, U] = schurdecom(a)
% Phan tich Schur ma tran A
n = size(a, 1);
v = zeros(n, 1);
v(1) = 1;
b = zeros(n, n);
b(:, n) = v;
for k = 2:n
    v = a*v;
    b(:, n-k+1) = v;
end
c = a*v;
rho = -b\c;
rho = [1 rho'];
lambda = roots(rho);
n = size(lambda, 1);
evec = zeros(n);
c = evec;
e = eye(n);
for i = 1:n
    b = a - lambda(i)*e;
    c = nulld(b);
    evec(:, i) = c(:,1);
```

```
end
p = grams(evec);
T = conj(transpose(p))*a*p;
U = p;
```

Để phân tích ma trận ta dùng chương trình ***ctschur.m***:

```
clear all, clc
a = [ 1 2 3 5; 4 5 6 2; 4 6 8 9; 9 3 6 7];
[t, u] = schurdecom(a)
```

§15. ĐỊNH THỨC CỦA MA TRẬN

Cho một ma trận vuông cấp n. Ta cần tìm định thức của nó. Trước hết chúng ta nhắc lại một số tính chất quan trọng của định thức:

- nếu nhân tất cả các phần tử của một hàng (hay cột) với k thì định thức được nhân với k
- định thức không đổi nếu ta cộng thêm vào một hàng tổ hợp tuyến tính của các hàng còn lại.
- nếu đổi chỗ hai hàng cho nhau thì định thức đổi dấu

Trước khi đi đến định nghĩa về định thức ta tìm hiểu khái niệm về hoán vị và phép thế.

Cho một dãy số, nếu ta đổi chỗ các số trong dãy cho nhau thì ta đã thực hiện một phép hoán vị. Ví dụ 123, 132,.. là các hoán vị của dãy số {1, 2, 3}. Trong hoán vị $\alpha_1\alpha_2\ldots\alpha_i\ldots\alpha_j\ldots\alpha_n$ ta nói α_i làm một nghịch thế với α_j nếu $i < j$ mà $\alpha_i > \alpha_j$. Ví dụ trong hoán vị 1432 số 4 làm với số 3 một nghịch thế , số 4 làm với số 2 một nghịch thế, số 3 làm với số 2 một nghịch thế. Một hoán vị gọi là chẵn nếu tổng số nghịch thế trong hoán vị đó là một số chẵn; một hoán vị gọi là lẻ trong trường hợp ngược lại. Như vậy 1432 là một hoán vị lẻ.

Cho một dãy số, nếu ta tạo ra một dãy số mới bằng cách đổi chỗ các phần tử cho nhau thì ta đã thực hiện một phép thế.

Ví dụ $p = \begin{pmatrix} 2 & 1 & 4 & 3 \\ 1 & 4 & 2 & 3 \end{pmatrix}$ là phép thế biến 2 thành 1, 1 thành 4, 4 thành 2 và 3 thành 3.

Một phép thế gọi là chẵn nếu tính chẵn lẻ của dòng trên và dòng dưới như nhau và lẻ trong trường hợp ngược lại. Phép thế trên là phép thể lẻ.

Cho ma trận vuông [A] cấp n. Các phần tử của hàng thứ i là $a_{i,1}, a_{i,2},\ldots,a_{i,n}$. Các phần tử của cột thứ j là $a_{1,j}, a_{2,j},\ldots, a_{n,j}$. Ta xem hàng thứ i là một vec tơ, kí hiệu là A_{i*} và cột thứ j cũng là một vec tơ, kí hiệu là A_{*j}. Với mỗi phép thế:

$$p = \begin{pmatrix} i_1 & i_2 & \cdots & i_n \\ j_1 & j_1 & \cdots & j_n \end{pmatrix} \qquad (1)$$

ta lập tích:

$$a_{i_1j_1}a_{i_2j_2}\ldots a_{i_nj_n} \qquad (2)$$

Trước mỗi tích (2) ta đặt dấu + nếu và dấu - nếu phép thế (1) lẻ. Sau đó ta lập tổng của n! tích có dấu như vậy, nghĩa là tổng:

$$\sum_p (-1)^{t(p)} a_{i_1j_1}a_{i_2j_2}\ldots a_{i_nj_n} \qquad (3)$$

trong đó:

t(p) = 1 nếu phép thế p lẻ

t(p) = 0 nếu phép thế p chẵn

Tổng (4) được gọi là định thức của ma trận vuông [A], cấp n.

Ta xây dựng hàm ***determinant()*** để tính định thức của ma trận theo định nghĩa:

```
function d = determinant(A)
% DETERMINANT tinh dinh thuc theo dinh nghia.
[m, n] = size(A);
if ( m ~= n )
   fprintf ( '\n' );
   fprintf ( ' Chi ma tran vuong moi co dinh thuc!\n' );
   return
end
p = zeros(1, n);
nf = prod([1:n]);
d = 0.0;
for i = 1:nf
   p = nextperm(p);
   s = permsign(p);
   x = diag(A([1:n],p));
   d = d + s*prod(x);
end

function psign = permsign(p)
% PERMSIGN tra ve dau phep the   .
%    +1, neu phep the chan,
%    -1, neu phep the le.
n = length ( p );
psign = 1;
for i = 1:n-1
   j = i;
   while (p(j) ~= i)
      j = j + 1;
```

```
    end
    if ( j ~= i )
        temp = p(i);
        p(i) = p(j);
        p(j) = temp;
        psign = - psign;
    end
end

function q = nextperm(p)
n = length(p);
q = p;
if(n == 1)
    q = 1;
elseif (q == 0)
    q = [1:n];
else
    i = n - 1;
    while (q(i) > q(i+1))
        i = i - 1;
        if (i == 0)
            break;
        end
    end
    if (i == 0)
        q = [1:n];
    else
        j = n;
        while (q(j) < q(i))
            j = j - 1;
        end
        t = q(j);
        q(j) = q(i);
        q(i) = t;
        q(i+1:n) = q(n:-1:i+1);
    end
end
```

Để tính định thức ta dùng chương trình ***ctdeterminant.m***:

```
clear all, clc
```

```
%a = [1  2; 3  5];
a = [1  3  5; 3  4  6; 4  6  3];
d = determinant(a)
```

§16. TÍNH ĐỊNH THỨC BẰNG CÁCH PHÂN TÍCH MA TRẬN

Cho ma trận [A]. Nếu

[A] = [B]×[C]

thì

det[A] = det[B]×det[C]

Mặt khác với một ma trận tam giác, ví dụ:

$$[B]=\begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ 0 & b_{22} & b_{23} & b_{24} \\ 0 & 0 & b_{33} & b_{34} \\ 0 & 0 & 0 & b_{44} \end{bmatrix}$$

thì

$$\det[B]=b_{11}b_{22}b_{33}b_{44}$$

nghĩa là đối với ma trận tam giác, định thức bằng tích các phần tử trên đường chéo chính.

Khi phân tích ma trận [A] theo thuật toán Doolitte, ta dùng chương trình ***ctdoodecmp.m*** để tính định thức của nó:

```
clear all, clc
a = [1 2 3 4; 3 4 5 7; 2 3 8 5; 4 9 1 4];
[l, r] = doolittle(a);
d = prod(diag(l))*prod(diag(r))
```

Khi phân tích ma trận [A] theo thuật toán Crout, ta dùng chương trình ***ctcrotdecmp.m*** để tính định thức của nó:

```
clear all, clc
a = [1 2 3 4; 3 4 5 7; 2 3 8 5; 4 9 1 4];
[l, r] = crout(a);
d = prod(diag(l))*prod(diag(r))
```

Khi phân tích ma trận [A] theo thuật toán Choleski, ta dùng chương trình ***ctcholdecmp.m*** để tính định thức của nó:

```
clear all, clc
a = [4 -2 2;-2 2 -4;2 -4  11];
```

```
a = pascal(5);
l = choleski(a);
d = prod(diag(l))*prod(diag(l'))
```

§17. THUẬT TOÁN TRỤ CHIÓ

Cho ma trận [A] có $a_{1,1} \neq 0$. Ta xây dựng ma trận [B] có các phần tử

$$b_{i,j} = a_{1,1}a_{i,j} - a_{i,n}a_{n,j}$$

thì:

$$\|[A]\| = a_{1,1}^{2-n}\|[B]\|$$

nghĩa là:

$$\det[A] = a_{11}^{n-2}\det\begin{bmatrix} \det\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} & \det\begin{bmatrix} a_{11} & a_{13} \\ a_{21} & a_{23} \end{bmatrix} & \cdots & \det\begin{bmatrix} a_{11} & a_{1n} \\ a_{21} & a_{2n} \end{bmatrix} \\ \det\begin{bmatrix} a_{11} & a_{12} \\ a_{31} & a_{32} \end{bmatrix} & \det\begin{bmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{bmatrix} & \cdots & \det\begin{bmatrix} a_{11} & a_{1n} \\ a_{31} & a_{3n} \end{bmatrix} \\ \vdots & \vdots & \cdots & \vdots \\ \det\begin{bmatrix} a_{11} & a_{12} \\ a_{n1} & a_{n2} \end{bmatrix} & \det\begin{bmatrix} a_{11} & a_{13} \\ a_{n1} & a_{n3} \end{bmatrix} & \cdots & \det\begin{bmatrix} a_{11} & a_{1n} \\ a_{n1} & a_{nn} \end{bmatrix} \end{bmatrix}$$

Ta xây dựng hàm ***chiopivot()*** để thực hiện thuật toán trên:

```
function d = chiopivot(a)
%tinh dinh thuc bang thuat toan Chio pivotal condensation
if a(1, 1) == 0
   error('Khong dung phuong phap  nay  de tinh dinh thuc duoc !');
   return;
end
c = a(1, 1);
n = size(a, 1);
if (n <= 2)
   d = a(1, 1)*a(2, 2) - a(2, 1)*a(1, 2);
   return
end
m = n - 1;
while (m >= 1)
   for i = 1:m%hang
      b(i, 1:m) = a(1, 1)*a(i+1, 2:m+1) - a(i+1, 1)*a(1, 2:m+1);
   end
   if (m > 2)
      a = b;
      c = c*a(1,1);
      clear b;
   end
   m = m - 1;
end
d = b(1, 1)*b(2, 2) - b(2, 1)*b(1, 2);
d = d/c;
```

Để tính định thức ta dùng chương trình ***ctchiopivot.m***:

```
clear all, clc
a = [1 2 3 4; 3 4 5 7; 2 3 8 5; 4 9 1 4];
d = chiopivot(a)
```

§18. THUẬT TOÁN LAPLACE

Để tính định thức theo thuật toán Laplace, ta khai triển định thức theo hàng hay cột. Cho ma trận vuông [A] cấp n. Nếu bỏ đi hàng i và cột j (tức xoá hàng và cột chứa phần tử a_{ij}) thì ta có một ma trận cấp (n - 1), định thức của nó gọi là định thức con cấp (n - 1) ứng với phần tử a_{ij} (minor) , ký hiệu là M_{ij}. Ta chú ý đến hàng thứ i và a_{ij} là một phần tử của hàng đó. Trong det[A] ta gộp những số hạng chứa a_{ij} lại và đặt a_{ij} làm thừa số chung, hệ số của nó kí hiệu là A_{ij} và gọi là phần bù đại số (cofactor) của phần tử aij. Cofactor A_{ij} của ma trận [A] là:

$$A_{ij} = (-1)^{i+j} M_{ij}$$

Định thức của [A] khi khai triển theo hàng là:

$$\det[A] = \sum_{i=1}^{n} a_{ij} A_{ij}$$

Ta xây dựng hàm ***cofactor()*** để tính các phần bù đại số:

```
function c = cofactor(A, i, j)
% cac phan bu dai so cua ma tran
% dung de nghich dao mt
% C = cofactor(A, i, j) tra ve phan bu dai so cua
%ng i, cot j cua A.
if nargin == 3
   M = A;
   M(i,:) = [];
   M(:,j) = [];
   c = (-1)^(i+j) * det(M);
else
   [n, n] = size(A);
   for i = 1:n
      for j = 1:n
         c(i,j) = cofactor(A, i, j);
      end
   end
end
```

Sau khi phần bù đại số, ta xây dựng hàm ***cofactordet()*** để tính định thức của [A]

```
function d = cofactordet(A)
```

```
d = 0;
for i = 1:size(A, 1)
   c = cofactor(A, i, 1);
   d = d + A(i, 1)*c;
end
```

Để tính định thức ta dùng chương trình ***ctcofactordet.m***:

```
clear all, clc
a = [1 2 3 4; 3 4 5 7; 2 3 8 5; 4 9 1 4];
det = cofactordet(a)
```

§19. THUẬT TOÁN DODGSON

Thuật toán cô đặc Dodgson (Dodgson condensation) dùng để tính định thức của ma trận vuông. Nó được Charles Ludwidge Dodgson đưa ra. Để tính định thức của ma trận cấp n × n, ta xây dựng các ma trận cấp (n - 1) × (n - 1) cho đến ma trận cấp 1 × 1 là định thức của ma trận cần tìm.

Bước đầu tiên ta xây dựng ma trận cấp (n - 1)×(n - 1) từ các định thức của các ma trận con 2×2. Ví dụ với ma trận

$$\begin{bmatrix} 5 & 1 & 0 \\ 2 & 8 & 5 \\ 0 & 6 & 7 \end{bmatrix}$$

ta có các ma trận con là:

$$\begin{bmatrix} 5 & 1 \\ 2 & 8 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 \\ 8 & 5 \end{bmatrix} \quad \begin{bmatrix} 2 & 8 \\ 0 & 6 \end{bmatrix} \quad \begin{bmatrix} 8 & 5 \\ 6 & 7 \end{bmatrix}$$

Các ma trận này sẽ tạo ra các phân tử của ma trận 2×2. Phần tử ử hàng r, cột c là định thức của ma trận con 2×2 của ma trận ban đầu với hàng r và cột c ở góc trên trái. Như vậy ma trận mới là:

$$\begin{bmatrix} 38 & 5 \\ 12 & 26 \end{bmatrix}$$

Ma trận k×k được tạo ra bằng cách lấy định thức của ma trận con 2×2 của ma trận (k+1)×(k+1) như ở trên và chia nó cho phần tử tương ứng của ma trận trung tâm, nghĩa là ma trận đã bỏ đi hàng trên cùng, hàng dưới cùng, cột bên phải và cột bên trái của ma trận (k+2)×(k+2)

Ta xây dựng hàm ***dodgson()*** để thực hiện thuật toán trên:

```
function dt = dodgson(a)
if size(a, 1) ~= size(a, 2)
```

```
    error('Ma tran A phai la mt tran vuong');
end;
n = size(a, 1);
if n == 2
    dt = a(1, 1)*a(2, 2) - a(2, 1)*a(1, 2);
    return
end;
if n == 3;
    for i = 1:n-1
        b(i, 1:n-1) = a(i, 1:n-1).*a(i+1, 2:n) - a(i+1, 1:n-1).*a(i, 2:n);
    end
    dt = (b(1, 1)*b(2, 2) - b(2, 1)*b(1, 2))/a(2,2);
    return
end
c = a;
c(:, 1) = [];
c(:, n-1) = [];
c(1, :) = [];
c(n - 1, :) = [];
for i = 1:n-1
    b(i, 1:n-1) = a(i, 1:n-1).*a(i+1, 2:n) - a(i+1, 1:n-1).*a(i, 2:n);
end
m = size(b, 1);
while m >= 2
    for i = 1:m-1
        for j = 1:m-1
            d(i, j) = (b(i, j)*b(i+1, j+1) - b(i+1, j)*b(i, j+1))/c(i, j);
        end
    end
    if m > 3
        c = b;
        c(:, 1) = [];
        c(:, m-1) = [];
        c(1, :) = [];
        c(m - 1, :) = [];
        b = d;
    end
    m = m - 1;
end
dt = (d(1, 1)*d(2, 2) - d(2, 1)*d(1, 2))/b(2,2);
```

Để tính định thức ta dùng chương trình ***ctdodgson.m***:

```
clear all, clc;
a = [1 2 3 4; 5 1 3 2; 4 9 2 2; 6 3 4 1];
dt = dodgson(a)
```

§20. NGHỊCH ĐẢO MA TRẬN BẰNG CÁCH DÙNG MINOR

Cho ma trận [A], ta có:

$$\left(a^{-1}\right)_{i,j} = \frac{A_{j,i}}{\det[A]}$$

Trong đó:

$\left(a^{-1}\right)_{i,j}$ là phần tử ở hàng i, cột j của ma trận $[A]^{-1}$

$A_{i,j}$ là phần bù đại số của phần tử $a_{i,j}$ của ma trận [A]

Ta xây dựng hàm ***minorinv()*** để thực hiện thuật toán trên:

```
function c = minorinv(a)
% Tim ma tran nghich dao bang thuat toan minor
n = size(a, 1);
ms = det(a);
for i = 1:n
   for k = 1:n
      b = cofactor(a, i, k);
      c(i, k) = b/ms;
   end
end
c = transpose(c);
```

Để tìm ma trận nghịch đảo ta dùng chương trình ***ctminorinv.m***:

```
clear all, clc;
a = [1  3  5;  3  4  9; 5  9  6];
b = minorinv(a)
```

§21. NGHỊCH ĐẢO BẰNG CÁCH PHÂN TÍCH MA TRẬN

Cho ma trận [A[, ta có thể phân tích nó thành ma trận tam giác phải [R] và tam giác trái [L]:

[A] = [L][R]

Do định nghĩa ma trận nghịch đảo:

$[L]^{-1}[L] = [L][L]^{-1} = [E]$

nên:

$[R] = [L]^{-1}[L][R] = [L]^{-1}[A]$

và:

$[L] = [L][R][R]^{-1} = [A][R]^{-1}$

Do vậy:

$[A] = [L][R] = ([A][R]^{-1})([L]^{-1}[A]) = [A][R]^{-1}[L]^{-1}[A]$

$[A][R]^{-1}[L]^{-1} = [E]$

Kết quả là:

$[R]^{-1}[L]^{-1} = [R]^{-1}$

Với ma trận tam giác phải [R], các hàng khi nghịch đảo là $l_1,..,l_n$ được tính theo cách sau:

- Cho $[e_1],.., [e_i],..,[e_n]$ là các cột của ma trận đơn vị [E] cấp n
- $l_n = \dfrac{[e_n]}{a_{n,n}}$
- $l_i = \left([e_i] - a_{i,i+1}l_{i+1} - \cdots a_{i,n}l_n\right)\dfrac{1}{a_{n,n}}$

Ta xây dựng hàm ***luinverse()*** để thực hiên thuật toán tính định thức của ma trận [R]:

```
function r = luinverse(a)
% Nghich dao ma tran tam giac phai
n = size(a, 1);
e = zeros(n, n);
c = zeros(n, 1);
l = e;
b = e;
for i = 1:n
   c(i) = 1;
   e(:, i) = c;
   c(i) = 0;
end
l(:, n) = e(:, n)/a(n, n);
for i = n-1:-1:1
   c = zeros(n, 1);
   for k = i+1:n
      c = c + a(i, k)*l(:, k);
   end
   l(:, i) = (e(:, i) - c)/a(i, i);
end
r = l';
```

Để nghich đảo ma trận tam giác ta dùng chương trình ***ctluinv.m***:

```
clear all, clc
a = [1 2 3; 0 4 5;  0 0 2];
r = luinverse(a)
```

Để nghịch đảo ma trận tam giác trái ta dùng các quan hệ:

$$[L]^{-1} = ([L^T]^{-1})^T$$

vì $[L]^T$ sẽ là một ma trận tam giác phải. Ta dùng chương trình ***ctluinverse.m*** để nghịch đảo ma trận thông thường:

```
clear all, clc
a = [ 1  3  5; 3  4  9;  5  9  6];
[l, r] = doolittle(a);
b = luinverse(r)*(luinverse(l'))'
```

§22. NGHỊCH ĐẢO MA TRẬN BẰNG THUẬT TOÁN GAUSS - JORDAN

Cho ma trận [A] và ma trận đơn vị [E] tương ứng. Dạng của ma trận [E] cấp 4, là:

$$E = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Như vậy, vấn đề là ta cần tìm ma trận $[A]^{-1}$. Phương pháp loại trừ để nhận được ma trận nghịch đảo $[A]^{-1}$ được thực hiện qua n giai đoạn, mỗi một giai đoạn gồm hai bước. Đối với giai đoạn thứ k:

- chuẩn hoá phần tử a_{kk} bằng cách nhân hàng với nghịch đảo của nó
- làm cho bằng không các phần tử phía trên và phía dưới đường chéo cho đến cột thứ k. Khi k = n thì $[A]^{(k)}$ sẽ trở thành ma trận đơn vị và [E] trở thành $[A]^{-1}$

Ta xây dựng một hàm nghịch đảo ***invmat()***:

```
function x = invmat(a)
% Nghich dao ma tran a
%Cu phap: x = invmat(a)
k = size(a, 1);
n = k;
b = eye(n);
a = [a, b];
i = 1;
while i<=n
   if a(i, i) ~= 0
      c = a(i, i);
```

```
        a(i, i:2*n) = a(i, i:2*n)/c;
      end
          for k = 1:n
              if k~=i
         c = a(k, i);
         a(k, i:2*n) = a(k, i:2*n)- a(i, i:2*n)*c;
        end
      end
          i = i+1;
    end
    x(:, 1:k) = a(:, (1+k):(2*k));
```

Để nghịch đảo ma trận

$$[A] = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

ta dùng chương trình ***ctinvmat.m***

```
clear all, clc
a = [ 2  1  1;  1  2  1;  1  1  2];
b = invmat(a)
```

§23. NGHỊCH ĐẢO BẰNG THUẬT TOÁN MOORE - PENROSE

Cho ma trận [A] cấp m×n. Ma trận nghịch đảo tổng quát hoá Moore – Pensore là ma trận n×m được Moore đưa ra năm 1920 và sau đó Pensore đưa ra năm 1955. Ma trận Moore – Pensore [B] thoả mãn các điều kiện:

[A][B][A] = [A]

[B][A][B] = [B]

Từ các điều kiên này ta có:

$[B] = [A]^T([M][M])^{-1}$

Để tính ma trận [B] ta dùng chương trình ***ctpseudoinv.m***:

```
clear all, clc
a = [ 3 4 5 6 7; 3 6 7 8 9; 2 3 4 5 4];
b = a'*invmat(a*a')
```

§24. GIÁ TRỊ RIÊNG VÀ VEC TƠ RIÊNG CỦA MA TRẬN

Cho ma trận [A] vuông, cấp n. Ta gọi số vô hướng λ là giá trị riêng của ma trận [A] và [V] là vec tơ riêng phải của [A] tương ứng với λ nếu:

$[A][V] = \lambda[V]$ (1)

[V] gọi là vec tơ riêng trái của [A] tương ứng với λ nếu:

$$[V]^T[A] = \lambda[V]^T$$

Ta có thể viết lại (1) dưới dạng:

$$([A] - \lambda[E])[V] = 0$$

Từ đó ta có hệ n phương trình thuần nhất. Nghiệm tầm thường của hệ chính là [X] = 0. Hệ sẽ có nghiệm không tầm thường nếu:

$$\det([A] - \lambda[E]) = 0 \qquad (2)$$

Khai triển (2) ta nhận được phương trình đặc tính:

$$a_1\lambda^n + a_2\lambda^{n-1} + \cdots + a_n\lambda + a_{n+1} = 0$$

Các nghiệm λ_i của phương trình đặc tính là các giá trị riêng của [A]. Nghiệm $[V_i]$ của:

$$([A] - \lambda_i[E])[V] = 0$$

là các vec tơ riêng của [A]. Ta xây dựng hàm ***nulld()*** để giải hệ phương trình tuyến tính thuần nhất này:

```
function Z = nulld(A, small)
norma = sqrt(mean(mean(abs(A))));
tiny = norma*eps;
if nargin<2
  small = (1 + norma)*sqrt(eps);
end
[m, n] = size(A);
if  m~= n
  error('Ma tran phai vuong!')
end
p = find(abs(A)<tiny);
if ~isempty(p)
  A(p) = 0;
end
[U,S,V] = svd(A,0);
S = diag(S);
s = S;
norma = max(s);
smax = max(s);
if smax == 0;
   smax = 1;
end
s = s/smax;
snorm = s;
t = find(s>0);
```

```
if isempty(t);
   Z = V;
   return;
end
p = find(s<tiny);
if ~isempty(p)
  s(p) = tiny;
end
p = find(s == 0);
if ~isempty(p)
  s(p) = small*min(s(t));
end
logs = log10(s);
sdifflog = -diff(logs);
smax = max(sdifflog);
r = find(sdifflog == smax);
if min(size(r))>0
  r = r(end);
else
  r = 0;
end
Z = V(:,r+1:n);
if snorm == ones(n,1);
   Z = zeros(n,0);
end
if max(S) <= small;
   Z = V;
end
```

§25. BIẾN ĐỔI ĐỒNG DẠNG

Ta khảo sát bài toán về giá trị riêng của ma trận:

$$[A][X] = \lambda[X] \tag{1}$$

với [A] là ma trận đối xứng. Bây giờ ta áp dụng phép biến đổi:

$$[X] = [P][X]^* \tag{2}$$

với [P] là ma trận không suy biến.

Thay (2) vào (1) và nhân hai vế với $[P]^{-1}$ ta có:

$$[P]^{-1}[A][P][X] = \lambda[P]^{-1}[P][X]^*$$

hay: $$[A]^*[X]^* = \lambda[X]^* \tag{3}$$

Trong đó

$$[A]^*[P][X] = [P]^{-1}[A][P]$$

Do λ không đổi khi thực hiện phép biến đổi nên giá trị riêng của ma trận [A] cũng chính là giá trị riêng của ma trận $[A]^*$. Các ma trận có cùng giá trị riêng được gọi là ma trận đồng dạng và phép biến đổi giữa chúng gọi là phép biến đổi đồng dạng.

Phép biến đổi đồng dạng thường dùng để đưa bài toán tìm giá trị riêng về dạng dễ giải hơn. Giả sử ta có cách nào đó để tìm ma trận [P] mà nó đường chéo hoá ma trận [A], lúc đó (3) có dạng:

$$\begin{bmatrix} A_{11}^* - \lambda & 0 & \cdots & 0 \\ 0 & A_{22}^* - \lambda & \cdots & 0 \\ \vdots & \vdots & \cdots & 0 \\ 0 & 0 & \cdots & A_{nn}^* - \lambda \end{bmatrix} \begin{bmatrix} x_1^* \\ x_1^* \\ \vdots \\ x_1^* \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Nghiệm của phương trình trên là:

$$\lambda_1 = A_{11}^* \quad \lambda_2 = A_{22}^* \quad \cdots \quad \lambda_n = A_{nn}^* \tag{4}$$

$$[X_1]^* = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad [X_2]^* = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad \cdots \quad [X_n]^* = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

hay: $[X]^* = \begin{bmatrix} X_1^* & X_2^* & \cdots & X_n^* \end{bmatrix} = [E]$

Theo (2), vec tơ riêng của [A] là:

$$[X] = [P][X]^* = [P][E] = [P] \tag{5}$$

Như vậy ma trận biến đổi [P] là ma trận các vec tơ riêng của [A] và các giá trị riêng của [A] là các số hạng trên đường chéo của $[A]^*$.

§26. TÌM GIÁ TRỊ RIÊNG BẰNG CÁC PHƯƠNG PHÁP LUỸ THỪA

1. Phương pháp luỹ thừa vô hướng: Giả sử ma trận vuông [A] cấp n có n giá trị riêng phân biệt:

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n| \tag{1}$$

và các vec tơ riêng tương ứng $[V_1], [V_2],...,[V_n]$

Ta chọn một vec tơ [X] bất kì có các thành phần khác zero. Khi đó [X] sẽ được biểu diễn bằng:

$$[X] = \alpha_1[V_1] + \alpha_2[V_2] + \cdots + \alpha_n[V_n] \tag{2}$$

Do $[A][V_n] = \lambda_n[V_n]$ nên:

$$
\begin{aligned}
[A][X] &= [A]\alpha_1[V_1] + [A]\alpha_2[V_2] + \cdots + [A]\alpha_n[V_n] \\
&= \alpha_1[A][V_1] + \alpha_2[A][V_2] + \cdots + \alpha_n[A][V_n] \\
&= \alpha_1\lambda_1[V_1] + \alpha_2\lambda_2[V_2] + \cdots + \alpha_n\lambda_n[V_n] \\
&= \lambda_1\left(\alpha_1[V_1] + \alpha_2\frac{\lambda_2}{\lambda_1}[V_2] + \cdots + \alpha_n\frac{\lambda_n}{\lambda_1}[V_2]\right)
\end{aligned}
$$

Lặp lại lần thứ k ta có:

$$
[X_k] = [A]^k[X_0] = \lambda_1^k\left\{\alpha_1[V_1] + \alpha_2\left(\frac{\lambda_2}{\lambda_1}\right)^k[V_2] + \cdots + \alpha_n\left(\frac{\lambda_n}{\lambda_1}\right)^k[V_2]\right\} \quad (3)
$$

Khi $k \to \infty$, do (1) nên (3) hội tụ về $[V_1]$ khi $\alpha_1 \neq 0$. Vậy là giá trị riêng λ_1, có trị tuyệt đối lớn nhất, và vec tơ riêng tương ứng $[V_1]$, có thể tính bằng cách cho trước một vec tơ $[X_0]$ có các thành phần khác zero theo hướng $[V_1]$ và lặp theo thủ tục sau:

$$
[X_{k+1}] = [A]\frac{[X_k]}{\|[X_k]\|_\infty} \to \lambda_1[V_1] \quad (4)
$$

Trong đó:

$$
\|[X]\|_\infty = \max\{|[X_n]|\}
$$

Tốc độ hội tụ phụ thuộc vào độ lớn của $|\lambda_2|/|\lambda_1|$.

Thuật toán tóm lại gồm các bước:

- cho vec tơ ban đầu $[X_0]$
- tính $[X_1] = [A][X_0]$
- tính $\|[X]\|$
- cho $[X] = [X] = [X]/\|[X]\|$ và lặp lại các bước 2 - 4 cho đến khi hội tụ

Ta xây dựng hàm ***eigpower()*** để thực hiện thuật toán trên:

```
function [lambda, x, iter] = eigpower(A, tol, maxiter, x0)
% Tim gia tri rieng lon nhat
% bang thuat loan lap luy thua
[n,m] = size(A);
if n ~= m
   error('Chi dung cho cac ma tran vuong');
end
if nargin == 1
  tol = 1e-8;
  x0 = ones(n,1);
  nmax = 200;
end
x0 = x0/norm(x0);
```

```
pro = A*x0;
lambda = x0'*pro;
err = tol + 1;
iter = 0;
while err > tol*abs(lambda) & abs(lambda) ~= 0 & iter <= nmax
  x = pro;
  x = x/norm(x);
  pro = A*x;
  lambdanew = x'*pro;
  err = abs(lambdanew - lambda);
  lambda = lambdanew;
  iter = iter + 1;
end
```

Để tính giá trị riêng và vec tơ riêng ta dùng chương trình ***cteigpower.m***:

```
clear all, clc
A = [ 1 2 3 4 5; 6 7 8 9 0; 3 4 5 6 7; 8 9 0 1 2; 2 4 6 8 1];
[lambdamax, v, iter] = eigpower(A)
```

2. Phương pháp luỹ thừa nghịch đảo: Phương pháp này dùng để tìm giá trị riêng có trị tuyệt đối bé nhất và vec tơ riêng tương ứng bằng cách áp dụng thuật toán trong mục 1 cho ma trận $[A]^{-1}$. Ý tưởng của phương pháp phán này dựa trên phương trình:

$$[A][V] = \lambda[V] \rightarrow [A]^{-1}[V] = \lambda^{-1}[V] \qquad (5)$$

Ta xây dựng hàm ***inveigpower()*** thực hiện phương trình này:

```
function [lambda, v] = inveigpower(A, tol, maxiter)
B = A^-1;
[lambda, v] = eigpower(B, tol, maxiter);
lambda = 1/lambda;
```

Để tính giá trị riêng có trị tuyệt đối bé nhất ta dùng chương trình ***ctinveigpower.m***:

```
clear all, clc
A = [2 0 1; 0 -2 0; 1 0 2];
tol = 1e-8;
maxiter = 100;
[lambdamin, v] = inveigpower(A , tol, maxiter)
```

3. Phương pháp dịch chuyển: Cho ma trận [A] đối xứng. Giả sử cần tìm giá trị riêng gần với một số s cho trước. Do:

$$[A][X] = \lambda[X]$$

nên trừ hai vế cho c[E][X] ta có:

$$([A] - s[E])[X] = (\lambda - s)[X]$$

hay: $[A^*][X^*] = \lambda^*[X]$

$$\frac{1}{\lambda^*}\left[X\right] = \left[A^*\right]^{-1}\left[X\right] \qquad (6)$$

Dùng phương pháp lặp luỹ thừa cho (6) ta nhận được $\frac{1}{\lambda^*}$ hay

$$\lambda^*_{min} = \varepsilon$$

$$\lambda - s = \lambda^* = \varepsilon$$

$$\lambda = s + \varepsilon = s + \lambda^*$$

Ta xây dựng hàm ***shiftpower()*** để thực hiện thuật toán trên:

```
function [lambda, v] = shiftpower(A, s)
A = (A - s*eye(size(A)))^-1;
maxiter = 150;
tol = 1e-8;
[lambda, v] = eigpower(A, tol, maxiter);
lambda = 1/lambda + s;
```

Để tính giá trị riêng của ma trận gần với s ta dùng chương trình ***ctshiftpower.m***:

```
clear all, clc
A = [2 0 1; 0 -2 0; 1 0 2];
tol = 1e-8;
maxiter = 100;
s = 0.5;
[lambda, v] = shiftpower(A, s)
```

§27. TÌM GIÁ TRỊ RIÊNG CỦA MA TRẬN BA ĐƯỜNG CHÉO ĐỐI XỨNG

1. Dãy Sturm: Sau khi thực hiện biến đổi đồng dạng bằng phương pháp dùng ma trận Householder ta nhận được ma trận tridiagonal đối xứng. Để tim các giá trị riêng và vec tơ riêng tương ứng ta có thể dùng thuật toán lặp QR. Tuy nhiên phương pháp chia đôi(bisection) cũng rất hiệu quả.

Đa thức đặc tính của ma trận có dạng:

$$P_n(\lambda) = \det([A] - \lambda[E]) = \begin{bmatrix} d_1 - \lambda & c_1 & 0 & 0 & \cdots & 0 \\ c_1 & d_2 - \lambda & c_2 & 0 & \cdots & 0 \\ 0 & c_2 & d_3 - \lambda & c_3 & \cdots & 0 \\ 0 & 0 & c_3 & d_4 - \lambda & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & c_{n-1} & d_n - \lambda \end{bmatrix}$$

có thể tính với 3(n - 1) phép nhân dùng dãy sau:

$$\begin{aligned} &P_0(\lambda) = 1 \\ &P_1(\lambda) = d_1 - \lambda \qquad (1) \\ &P_i(\lambda) = (d_i - \lambda)P_{i-1}(\lambda) - c_{i-1}^2 P_{i-2}(\lambda) \qquad i = 2, 3, ..., n \end{aligned}$$

Các đa thức $P_0(\lambda), P_1(\lambda), ..., P_n(\lambda)$ tạo nên dãy Sturm có các tính chất sau:

- Số lần đổi dấu trong dãy $P_0(a), P_1(a), ..., P_n(a)$ bằng số nghiệm của $P_n(\lambda)$ nhỏ hơn λ

- Nếu một phần tử $P_i(a)$ của dãy là zero, dấu của nó ngược với dấu của $P_{i-1}(a)$.

Ta xây dựng hàm ***sturmseq()*** để tạo ra dãy Sturm:

```
function p = sturmseq(c, d, lambda)
% Cho day Sturm p gan voi ma tran
% tridiagonal  A = [c\d\c] va lambda.
% Chu y |A - lambda*I| = p(n).
n = length(d) + 1;
p = ones(n, 1);
p(2) = d(1) - lambda;
for i = 2:n-1
   p(i+1) = (d(i) - lambda)*p(i) - (c(i-1)^2 )*p(i-1);
end
```

Tiếp theo ta xây dựng hàm ***countevals()*** để đếm số lần đổi dấu của dãy Sturm:

```
function num = countevals(c, d, lambda)
% Tinh so gia tri rieng nho hon lambda cua ma tran
% A = [c\d\c]. Dung day Sturm
p = sturmseq(c, d, lambda);
n = length(p);
oldsign = 1;
num = 0;
for i = 2:n
   psign = sign(p(i));
```

```
    if psign == 0;
        psign = -oldsign;
    end
    if psign*oldsign < 0
        num = num + 1;
    end
    oldsign = psign;
end
```

2. Định lý Gerschgorin: Định lý Gerschgorin rất có ích khi xác định biên toàn cục trên các giá trị riêng của ma trận vuông [A] cấp n. Biên toàn cục là biên bao lấy toàn bộ các giá trị riêng. Đối với ma trận đối xứng thì:

$$a_i - r_i \le \lambda \le a_i + r_i \quad i = 1, 2, ..., n$$

Trong đó:

$$a_i = a_{ii} \qquad r_i = \sum_{\substack{j=1 \\ j \ne i}}^{n} \left| a_{ij} \right| \tag{2}$$

Vậy biên toàn cục của các giá trị riêng là:

$$\lambda_{min} \ge \min_i (a_i - r_i) \qquad \lambda_{max} \le \max_i (a_i + r_i) \tag{3}$$

Ta xây dựng hàm ***gerschgorin()*** để xác định biên trên và dưới của các giá trị riêng của ma trận đối xứng:

```
function [evalmin, evalmax] = gerschgorin(c, d)
% Danh gia cac  bien cua gia tri rieng
n = length(d);
evalmin = d(1) - abs(c(1));
evalmax = d(1) + abs(c(1));
for i = 2:n-1
    eval = d(i) - abs(c(i)) - abs(c(i-1));
    if eval < evalmin;
        evalmin = eval;
    end
    eval = d(i) + abs(c(i)) + abs(c(i-1));
    if eval > evalmax
        evalmax = eval;
    end
end
eval = d(n) - abs(c(n-1));
if eval < evalmin
    evalmin = eval;
```

```
end
eval = d(n) + abs(c(n-1));
if eval > evalmax
   evalmax = eval;
end
```

Ta xây dựng hàm ***evalbrackets()*** để vây các giá trị riêng nhỏ nhất của ma trận ba đường chéo. Nó cho dãy $r_1, r_2,...,r_{m+1}$ trong đó mỗi đoạn $[r_i, r_{i+1}]$ chứa một giá trị riêng. Thuật toán trước hết tìm biên toàn cục của các giá trị riêng theo định lí Gerschorin. Phương pháp chia đôi kết hợp với dãy đặc tính của dãy Sturm được dùng để xác định biên trên $\lambda_m, \lambda_{m-1},..., \lambda_1$

```
function r = evalbrackets(c, d, m)
% Nhom m gia tri rieng min cua ma tran  A = [c\d\c]
```

```
[evalmin, evalmax] = gerschgorin(c, d);
r = ones(m+1, 1);
r(1) = evalmin;
% Tim cac gia tri rieng theo thu tu giam dan
for k = m:-1:1
   % chia doi doan (evalmin, evalmax)
   eval = (evalmax + evalmin)/2;
   h = (evalmax - evalmin)/2;
   for i = 1:100
      % Tim so gia tri rieng nho hon eval
      numevals = countevals(c, d, eval);
      % lai chia doi doan chua eval
      h = h/2;
      if numevals < k ;
         eval = eval + h;
      elseif numevals > k ;
         eval = eval - h;
      else;
         break
      end
   end
   valmax = eval;
   r(k+1) = eval;
end
```

3. Tính các giá trị riêng: Một khi đã tìm được khoảng chứa các giá trị riêng ta có thể xác định nghiệm của đa thức $P_n(\lambda)$ bằng phương pháp Brent. Ta xây dựng hàm ***eigenvals()*** để tìm các giá trị riêng nhỏ nhất của ma trận ba đường chéo đối xứng bằng phương pháp Brent.

```
function evals = eigenvals(c, d, m)
% Tinh m gia tri rieng be nhat cua a = [c\d\c].
% c va d phai khai bao 'global' trong chuong trinh goi
evals = zeros(m, 1);
r = evalbrackets(c, d, m); % nhom cac gia tri rieng
for i = 1:m
   evals(i) = brent(@func, r(i), r(i+1));
end
```

Khi đã biết các giá trị riêng, cách tốt nhất để tìm các vec tơ riêng tương ứng là phương pháp luỹ thừa nghịch đảo. Ta dùng hàm ***invpower()*** để thực hiện công việc này:

```
function [eval, evec] = invpower(c, d, s, maxiter, tol)
% Tinh cac gia tri rieng cua A = [c\d\c] gan voi s va
% va vec to rieng tuong ung bang phuong phap lap nghich dao.
if nargin < 5;
   tol = 1.0e-6;
end
if nargin < 4;
   maxiter = 50;
end
n = length(d);
e = c;
d = d - s;
[c, d, e] = ludec3(c, d, e); % phan tich A* = A - sI
x = rand(n, 1);
xmag = sqrt(dot(x, x));
x = x/xmag; % chuan hoa x
for i = 1:maxiter
   xold = x;
   x = lusol3(c, d, e, x); % giai he A*x = xOld
   xmag = sqrt(dot(x, x));
   x = x/xmag; % chuan hoa x
   xsign = sign(dot(xold, x)); %t hien su doi dau cua x
   x = x*xsign;
   % kiem tr tinh hoi tu
   if sqrt(dot(xold - x, xold - x)) < tol
      eval = s + xsign/xmag;
      evec = x;
      return
   end
end
error('Qua nhieu lan lap')

function [c,d,e] = ludec3(c, d, e)
% Phan tich ma tran A thanh A = [c\d\e].
n = length(d);
for k = 2:n
   lambda = c(k-1)/d(k-1);
   d(k) = d(k) - lambda*e(k-1);
   c(k-1) = lambda;
end
```

```
function x = lusol3(c, d, e, b)
% Giai he A*x = b voi A = [c\d\e]
n = length(d);
for k = 2:n % thay the tien
   b(k) = b(k) - c(k-1)*b(k-1);
end
b(n) = b(n)/d(n); % thay the lui
for k = n-1:-1:1
   b(k) = (b(k) -e(k)*b(k+1))/d(k);
end
x = b;
```

Ta xây dựng hàm ***symmateig()*** để tìm các giá trị riêng và vec tơ riêng:

```
function [ eigenvalues, eigvec] = symmateig(a)
global c d
m = size(a, 1);
b = housetrans(a);
d = diag(b);
c = diag(b, 1);
p = householderp(a);
evals = eigenvals(c, d, m);
for i = 1:m
   s = evals(i)*1.0000001;
   eval = invpower(c, d, s);
end
eigenvalues = diag(evals');
n = size(eigenvalues, 1);
eigvec = zeros(n);
c = eigvec;
e = eye(n);
for i = 1:n
   b = a - eigenvalues(i, i)*e;
   c = nulld(b);
   eigvec(:, i) = c(:,1);
end
```

Để tìm giá trị riêng của một ma trận đối xứng bằng cách biến đổi Householder ta dùng chương trình ***cthouseholdereig.m***:

```
clear all, clc
```

```
global c d
a = [ 1  2  3  4; 2  7  6  5; 3  6  9  0;4  5  0  3];
[eval, evec] = symmateig(a)
n = size(a, 1);
```

§28. TÌM GIÁ TRỊ RIÊNG BẰNG PHƯƠNG PHÁP QUÉT

Để tìm các giá trị riêng khác của ma trận ta dùng phương pháp quét. Cho phương trình thuần nhất dưới dạng ma trận:

$$[A][X] = \lambda[X] \tag{1}$$

Bằng phương pháp lặp luỹ thừa ta nhận được giá trị riêng lớn nhất. Để tìm giá trị riêng lớn thứ hai λ_2 ta dùng phương pháp quét. Muốn vậy vec tơ ban đầu $[X_0]$ phải chọn sao cho không cùng phương với vec tơ riêng ứng với giá trị riêng lớn nhất λ_1. Vì các vec tơ riêng tạo thành cơ sở nên:

$$[X_0] = c_1[X_1] + c_2[X_2] + \cdots + c_n[X_n] \tag{2}$$

Nhân cả hai vế của (2) với chuyển vị $[X_1]^T$ ta có:

$$[X_1]^T[X_0] = c_1[X_1]^T[X_1]$$

hay:

$$c_1 = \frac{[X_1]^T[X_0]}{[X_1]^T[X_1]} \tag{3}$$

Bây giờ ta chọn vec tơ ban đầu là:

$$[Y_0] = [X_0] - c_1[X_1] = [X_0] - \frac{[X_1]^T[X_0][X_1]}{[X_1]^T[X_1]} \tag{4}$$

Như vậy vec tơ $[Y_0]$ không trùng phương với $[X_1]$ và nếu ta dùng phép lặp công suất với vec tơ ban đầu $[Y_0]$ nó sẽ hội tụ tới giá trị riêng lớn thứ 2.

Vec tơ $[Y_0]$ trong (4) có thể viết dưới dạng:

$$[Y_0] = \left([E] - \frac{[X_1][X_1]^T}{[X_1]^T[X_1]}\right)[X_0] \tag{5}$$

và:

$$[A][Y_0] = \left([A] - \frac{[A][X_1][X_1]^T}{[X_1]^T[X_1]}\right)[X_0] \tag{6}$$

$$= [D_1][X_0] \tag{7}$$

Trong đó:

$$[D_1] = [A] - \frac{\lambda_1[X_1][X_1]^T}{[X_1]^T[X_1]} \tag{8}$$

là ma trận dùng cho phép lặp để tính giá trị riêng lớn thứ 2 của [A]. Để tính giá trị riêng lớn thứ 3, thứ 4,... ta dùng các ma trận cho phép lặp luỹ thừa:

$$[D_2]=[D_1]-\frac{\lambda_2[X_2][X_2]^T}{[X_2]^T[X_2]}$$

$$[D_3]=[D_2]-\frac{\lambda_3[X_3][X_3]^T}{[X_3]^T[X_3]}$$

Tiếp tục quá trình trên n lần ta thu được các giá trị riêng của ma trận [A]. Ta xây dựng hàm ***sweeping()*** để thực hiện thuật toán trên:

```
function [t, s, iter] = sweeping(A)
%function t = sweeping(A)
%Tinh tat ca cac gia tri rieng va vec to rieng cua ma tran A doi xung
n = size(A, 1);
t = eye(n);
s = zeros(n);
k = zeros(n, 1);
[l, v, iter] = eigpower(A);
t(1, 1) = l;
s(:, 1) = v;
k(1, 1) = iter;
for i = 2:n
   A = A - (l*v*v')/(v'*v);
   [l, v, iter] = eigpower(A);
   t(i,i) = l;
   s(:, i) = v;
   k(i, 1) = iter;
end
iter = max(k);
```

Để tính giá trị riêng và vec tơ riêng của ma trận ta dùng chương trình ***ctsweeping.m***:

```
clear all, clc
A = [1  3  5;  3  4  9; 5  9  6];
%A = [17 24 30 17; 8 13 20 7; 2 10 8 6; -23 -43 -54 -26];
[t, s, iter] = sweeping(A)
```

§29. TÌM GIÁ TRỊ RIÊNG BẰNG PHƯƠNG PHÁP JACOBI

Phương pháp này cho phép ta tìm tất các giá trị riêng của ma trận [A] đối xứng. Ta tạo ra ma trận trực giao chuẩn [V] gồm các các vec tơ riêng sao cho $[V]^T[V] = [E]$;

$[V]^{-1} = [V]^T$ và dùng nó để biến đổi đồng dạng ma trận [A]. Khi đó ta sẽ nhận được ma trận đường chéo có các giá trị riêng nằm trên đường chéo chính:

$$[V]^T[A][V] = [V]^{-1}[A][V] = [\lambda] \tag{1}$$

Để hiểu phương pháp Jacobi, trước hết ta định nghĩa ma trận quay pq:

$$\left[R_{pq}(\theta)\right] = \begin{array}{c} \begin{array}{cccccccc} & & & p & & & q & \end{array} \\ \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 & \cdots & 0 & \cdots \\ 0 & 1 & \vdots & 0 & 0 & \cdots & 0 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & \cdots & \cos\theta & \cdots & \cdots & -\sin\theta & \cdots \\ 0 & 0 & \cdots & 0 & \cdots & \cdots & 0 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & \sin\theta & \cdots & \cdots & \cos\theta & \cdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \end{array} \begin{array}{c} \\ \\ \\ p \\ \\ \\ q \\ \\ \end{array} \tag{2}$$

Do ma trận [R] trực giao nên các vec tơ hàng và cột của nó cũng trực giao và chuẩn hoá:

$$\left[R_{pq}\right]^T\left[R_{pq}\right] = \left[E\right] \qquad \left[R_{pq}\right]^{-1} = \left[R_{pq}\right] \tag{3}$$

Nhân trước và sau ma trận [A] với $\left[R_{pq}\right]^T$ và $\left[R_{pq}\right]$ tạo nên biến đổi đồng dạng:

$$\left[A_{(1)}\right] = \left[R_{pq}\right]^T\left[A\right]\left[R_{pq}\right] \tag{4}$$

Chú ý là phép biến đổi đồng dạng không làm thay đổi giá trị riêng nên bất kì ma trận nào có được từ các lần lặp:

$$\begin{aligned} \left[A_{(k+1)}\right] &= \left[R_{(k)}\right]^T\left[A_{(k)}\right]\left[R_{(k)}\right] \\ &= \left[R_{(k)}\right]^T\left[R_{(k-1)}\right]^T\cdots\left[R\right]^T\left[A\right]\left[R\right]\cdots\left[R_{(k-1)}\right]\left[R_{(k)}\right] \end{aligned} \tag{5}$$

có cùng giá trị riêng. Hơn nữa, nếu nó là ma trận đường chéo, các giá trị riêng sẽ nằm trên đường chéo chính và ma trận được nhân vào bên phải của ma trận [A] là ma trận [V]:

$$\left[V\right] = \left[R\right]\cdots\left[R_{(k-1)}\right]\left[R_{(k)}\right] \tag{6}$$

Vấn đề còn lại là làm sao biến ma trận (5) thành ma trận đường chéo. Cần nhớ rằng phép biến đổi (4) chỉ làm thay đổi hàng và cột p và q.

$$\begin{aligned} v_{pq} = v_{qp} &= a_{qp}(c^2 - s^2) + (a_{qq} - a_{pp})sc \\ &= a_{qp}\cos 2\theta + \frac{1}{2}(a_{qq} - a_{pp})\sin 2\theta \end{aligned} \tag{7a}$$

$$v_{pn} = v_{np} = a_{pn}c + a_{qn}s \quad n \neq p, q \tag{7b}$$

$$v_{qn} = v_{nq} = -a_{pn}s + a_{qn}c \quad n \neq p, q \tag{7c}$$

$$v_{pp} = a_{pp}c^2 + a_{qq}s^2 + 2a_{pq}sc = a_{pp}c^2 + a_{qq}s^2 + a_{pq}\sin 2\theta \tag{7d}$$

$$v_{qq} = a_{pp}s^2 + a_{qq}c^2 - 2a_{pq}sc = a_{pp}s^2 + a_{qq}c^2 - a_{pq}\sin 2\theta \tag{7e}$$

$c = \cos\theta$, $s = \sin\theta$

Ta làm cho

$$v_{pq} = v_{qp} = 0 \tag{8}$$

bằng cách chọn θ của ma trận quay $[R_{pq}(\theta)]$ sao cho:

$$tg2\theta = \frac{\sin 2\theta}{\cos 2\theta} = -\frac{2a_{pq}}{a_{pp} - a_{qq}} \qquad \cos 2\theta = \frac{1}{\sqrt{1 + tg^2 2\theta}}$$

$$\sin 2\theta = tg2\theta\cos 2\theta \tag{9}$$

$$\cos\theta = \sqrt{\cos^2\theta} = \sqrt{\frac{1 + \cos 2\theta}{2}} \qquad \sin\theta = \frac{\sin 2\theta}{2\cos\theta}$$

Để cho ma trận gần ma trận đường chéo sau mỗi lần lặp, ta coi chỉ số hàng và cột của phần tử lớn nhất nằm ngoài đường chéo là p và q và làm cho nó bằng zero.

Ta xây dựng hàm ***eigjacobi()*** để tính các giá trị riêng λ_i và các vec tơ riêng của một ma trận đối xứng cấp n bằng phương pháp Jacobi.

```
function [lambda, v] = eigjacobi(A, tol, maxiter)
%Phuong phap Jacobi cho ma tran A doi xung
if nargin < 3
   maxiter = 100;
end
if nargin < 2
   tol = 1e-8;
end
n = size(A, 2);
lambda =[];
v = [];
for m = 1:n
   if norm(A(m:n, m) - A(m, m:n)') > eps
      error(' Ma tran khong doi xung !');
   end
end
v = eye(n);
for k = 1: maxiter
   for m = 1:n - 1
      [Am(m), Q(m)] = max(abs(A(m, m + 1:n)));
```

```
        end
        [Amm, p] = max(Am);
        q = p + Q(p);
        if Amm < eps*sum(abs(diag(lambda)))
           break;
        end
        if abs(A(p, p) - A(q, q))<eps
           s2 = 1;
           s = 1/sqrt(2);
           c = s;
           cc = c*c;
           ss = s*s;
        else
           t2 = 2*A(p, q)/(A(p, p) - A(q, q)); %Pt.(9a) tg2(theta)
           c2 = 1/sqrt(1 + t2*t2); %Pt.(9b) cos2(theta)
           s2 = t2*c2; %Pt.(9c) sin2(theta)
           c = sqrt((1 + c2)/2); %Pt.(9d) cos(theta)
           s = s2/2/c; %Pt.(9e) sin(theta)
           cc = c*c;
           ss = s*s;
        end
        lambda = A;
        lambda(p, :) = A(p,:)*c + A(q,:)*s; %Pt.(7b)
        lambda(:, p) = lambda(p,:)';
        lambda(q, :) = -A(p, :)*s + A(q, :)*c; %Pt.(7c)
        lambda(:, q) = lambda(q, :)';
        lambda(p, q) = 0;
        lambda(q, p) = 0; %Pt.(7a)
        lambda(p, p) = A(p, p)*cc +A(q, q)*ss + A(p, q)*s2; %Pt.(7d)
        lambda(q, q) = A(p, p)*ss +A(q, q)*cc - A(p, q)*s2; %Pt.(7e)
        A = lambda;
        v(:, [p q]) = v(:, [p q])*[c -s;s c];
     end
     lambda = diag(diag(lambda));
```

Để tính các giá trị riêng ta dùng chương trình chương trình ***cteigjacobi.m***:

```
     clear all, clc ;
     a = [ 1  3  5; 3  4  9;  5  9  6];
     [eigval, eigvec] = eigjacobi(a)
```

Các bài toán vật lí thường đưa đến bài toán giá trị riêng dưới dạng:

$$[A][X] = \lambda[B][X] \quad (10)$$

Trong đó [A] và [B] là các ma trận đối xứng cấp n. Ta giả sử [B] xác định dương. Bài toán như vậy phải đưa về dạng chuẩn trước khi dùng thuật toán đường chéo hoá Jacobi. Do [B] là ma trận đối xướng, xác định dương nên theo thuật toán Choleski ta phân tích nó thành tích 2 ma trận $[B] = [L][L]^T$. Bây giờ ta đưa vào phép biến đổi:

$$[X] = \left([L]^{-1}\right)^T [Z] \quad (11)$$

Thay (11) vào (10) ta có:

$$[A]\left([L]^{-1}\right)^T [Z] = \lambda[L][L]^T \left([L]^{-1}\right)^T [Z]$$

Nhân trước hai vế với $[L]^{-1}$ ta được:

$$[L]^{-1}[A]\left([L]^{-1}\right)^T [Z] = [L]^{-1}\lambda[L][L]^T \left([L]^{-1}\right)^T [Z]$$

Do $[L]^{-1}[L] = [L]^T([L]^{-1})^T = [E]$ nên:

$$[H][Z] = \lambda[Z] \quad (12)$$

là dạng chuẩn của bài toán tìm giá trị riêng và vec tơ riêng với:

$$[H] = [L]^{-1}[A]([L]^{-1})^T \quad (13)$$

Tóm lại để giải bài toán (10) ta theo các bước sau:

- dùng thuật toán Choleski tính [L]
- tính $[L]^{-1}$
- tính [H] theo (13)
- giải bài toán (12) theo thuật toán đường chéo hoá Jacobi
- tìm vec tơ riêng theo (11), giá trị riêng không đổi khi biến đổi

Ta xây dựng hàm ***stdform()*** để tạo ra ma trận [H] và $[T] = ([L]^{-1})^T$

```
function [H, T] = stdform(A, B)
% Bien doi A*x = lambda*B*x thanh H*z = lambda*z
n = size(A, 1);
L = choleski(B);
Linv = invert(L);
H = Linv*(A*Linv');
T = Linv';

function Linv = invert(L)
% nghich dao ma tran tam giac trai L.
n = size(L, 1);
for j = 1:n-1
   L(j, j) = 1/L(j, j);
   for i = j+1:n
      L(i, j) = -dot(L(i, j:i-1), L(j:i-1, j)/L(i, i));
```

```
    end
end
L(n, n) = 1/L(n, n);
Linv = L;
```

§30. THUẬT TOÁN QR

Cho ma trận [A] ta có thể tìm được ma trận [Q] và [R] sao cho:

[A] = [Q][R]

Trong đó [Q] là ma trận trực giao và [R] là ma trận tam giác phải.

Để tìm các gía trị riêng của ma trận [A], ta thực hiện phân tích QR cho [A] bằng thuật toán Householder hay thuật toán Givens. Sau đó ta xây dựng ma trận $[A_1] = [R_1][Q_1]$. Tiếp tục phân tích QR ma trận $[A_1] = [Q_1][R_1]$ và lại xây dựng ma trận $[A_2]= [R_2][Q_2]$ cho đến khi hội tụ. Thuật toán này có thể áp dụng cho ma trận bất kỳ. Tuy nhiên nó sẽ hiệu quả hơn nếu ma trận [A] là ma trận ba đường chéo hay ma trận Hessenberg. Ta xây dựng hàm ***qreig()*** để thực hiện thuật toán trên:

```
function [lambda, evec]  = qreig(a)
% Tinh cac gia tri rieng bang thuat toan phan tich QR
for i = 1: 500
   s1 = diag(a);
   [q, r] = qrdecom(a);
   a = r*q;
   s2 = diag(a);
   if abs(max(s2 - s1)) < 1e-8
      break;
  end
end
lambda = diag(a);
n = size(lambda, 1);
evec = zeros(n);
c = evec;
e = eye(n);
for i = 1:n
   b = a - lambda(i)*e;
   c = nulld(b);
   evec(:, i) = c(:, 1);
end
lambda = diag(lambda);
```

Để tìm giá trị riêng của một ma trận đối xứng bằng cách biến đổi Householder ta dùng chương trình ***cthouseholder.m***:

```
clear all, clc
global c d
a = [ 1  2  3  4; 2  7  6  5; 3  6  9  0;4  5  0  3];
[eval, evec] = symmateig(a)
n = size(a, 1);
```

§31. THUẬT TOÁN RUTISHAUSER

Cho ma trận [A] đối xứng, xác định dương, ta dùng thuật toán Crout để phân tích nó thành:

$$[A] = [L][R] \quad (1)$$

Sau đó ta xây dựng ma trận:

$$[A_1] = [R][L] \quad (2)$$

Ma trận $[A_1]$ sẽ có cùng giá trị riêng với ma trận [A]. Ta lại tiếp tục phân tích ma trận $[A_1]$ theo thuật toán Crout:

$$[A_1] = [L_1][R_1] \quad (3)$$

Và sau đó ta xây dựng ma trận:

$$[A_2] = [R_1][L_1]$$

Tiếp tục quá trình ta nhận được ma trận:

$$[A_n] = [L_n][R_n]$$

$$[A_{n+1}] = [R_n][L_n]$$

Khi $n \to \infty$, $[A_n]$ trở thành ma trận tam giác trên có các phần tử trên đường chéo chính là các giá trị riêng của ma trận [A]. Phép lặp này hội tụ khi [A] là ma trận đố xứng, xác định dương. Khi [A] không xác định dương, phép lặp không ổn định. Phân tích Crout cũng không thực hiện được khi có một phần tử $r_{ii} = 0$. Ta xây dựng hàm ***rutishauser()*** để thực hiện thuật toán trên:

```
function [lambda, evec] = rutishauser(a)
% Tinh cac gia tri rieng bang thuat toan phan tich LR
for i = 1: 500
   s1 = diag(a);
   [l, r] = crout(a);
   a = r*l;
   s2 = diag(a);
   if abs(max(s2 - s1)) < 1e-8
      break;
  end
end
lambda = diag(a);
n = size(lambda, 1);
evec = zeros(n);
```

```
c = evec;
e = eye(n);
for i = 1:n
   b = a - lambda(i)*e;
   c = nulld(b);
   evec(:, i) = c(:, 1);
end
lambda = diag(lambda);
```

Để tính các giá trị riêng của ma trận ta dùng chương trình ***ctrutishauser.m***:

```
clear all, clc
a = [ 1 2 3 4; 5 6 7 8; 9 0 1 2; 3 4 5 6];
[lambda, evec] = rutishauser(a)
```

§32. THUẬT TOÁN FADDEEV – LEVERIER - SOURIAU

Cho ma trận [A]:

$$[A]=\begin{bmatrix} a_{11} & a_{11} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \cdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$$

Các giá trị riêng của ma trận [A] là nghiệm của đa thức đặc trưng:

$$\det([A]-\lambda[E])=0 \qquad (1)$$

Khi khai triển định thức trên, ta được đa thức cấp n:

$$P_n(\lambda)=\lambda^n-p_1\lambda^{n-1}-p_1\lambda^{n-1}-\cdots-p_n=0 \qquad (2)$$

Gọi vết của ma trận là tổng các phần tử trên đường chéo chính:

$$\mathrm{trace}([A]) = a_{11}+a_{22}+\cdots+a_{nn} \qquad (3)$$

Các hệ số của đa thức (2) được xác định theo:

$$p_1=\mathrm{trace}([B_1]) \qquad [B_1]=[A]$$

$$p_2=\frac{1}{2}\mathrm{trace}([B_1]) \qquad [B_2]=[A]([B_1]-p_1[E])$$

$$p_3=\frac{1}{3}\mathrm{trace}([B_3]) \qquad [B_3]=[A]([B_2]-p_2[E])$$

$$\cdots$$

$$p_n=\frac{1}{n}\mathrm{trace}([B_n]) \qquad [B_n]=[A]([B_{n-1}]-p_{n-1}[E])$$

Giải hệ (3) ta có các giá trị riêng của [A].

Ta xây dựng hàm ***fadlev()*** để thực hiện thuật toán trên:

```
function [lambda, evec] = fadlev(a)
%Tim cac gia tri rieng va vec to rieng bang phuong phap Faddeev - Leverrier
n = size(a, 1);
b = a;
for i = 1:n
   s = 0;
   for k = 1:n
      s = s + b(k, k);
   end
   p = s/i;
   b = a*(b - p*eye(n));
   r(i+1) = -p;
end
r(1) = 1;
lambda = roots(r);
n = size(lambda, 1);
evec = zeros(n);
c = evec;
e = eye(n);
for i = 1:n
   b = a - lambda(i)*e;
   c = nulld(b);
   evec(:, i) = c(:, 1);
end
lambda = diag(lambda);
```

Để tìm các giá trị riêng của ma trận ta dùng chương trình ***ctfaclev.m***:

```
clear all, clc
a = [11 2 3 1 4; 2 9 3 5 2; 4 5 6 7 8; 3 2 1 6 7; 4 9 6 8 2];
[lambda, evec] = fadlev(a)
```

§33. THUẬT TOÁN KRYLOV

Đa thức đặc trưng của ma trận [A] có dạng:

$$P_n(\lambda) = \lambda^n - p_1\lambda^{n-1} - p_1\lambda^{n-2} - \cdots - p_n = 0 \qquad (1)$$

Ta viết lại (1) dưới dạng:

$$P_n(\lambda) = \lambda^n + \sum_{i=0}^{n-1} p_i\lambda^i \qquad (2)$$

Theo định lý Hamilton - Kelly ta có P([A]) = 0. Xét dãy lặp:

$$v^{(i+1)} = [A]v^{(i)} \qquad i = 0, 1,..., n-1 \qquad (3)$$

Do P([A]) = 0 nên:

$$\rho([A])v^{(0)} = v^{(n)} + \sum_{i=0}^{n-1}\rho_i v^{(i)} = 0 \qquad (4)$$

Đặt $v^{(i)} = \left[v_1^{(i)},...,v_n^{(i)}\right]^T$ ta có:

$$v_j^{(n)} + \sum_{i=0}^{n-1}\rho_i v^{(i)} = 0 \qquad (5)$$

hay:

$$\begin{bmatrix} v_1^{(n-1)} & \cdots & v_1^{(0)} \\ \vdots & \cdots & \vdots \\ v_n^{(n-1)} & \cdots & v_n^{(0)} \end{bmatrix}\begin{bmatrix} \rho_{n-1} \\ \vdots \\ \rho_0 \end{bmatrix} = -\begin{bmatrix} v_1^{(n)} \\ \vdots \\ v_n^{(n)} \end{bmatrix} \qquad (6)$$

Vì $v^{(k+1)} = [A]v^{(k)}$

nên:

$$v_i^{(k+1)} = \left([A]v^{(k)}\right)_i = \sum_{j=1}^{n} a_{i,j}v_j^{(k)} \quad i = 1, 2,..., n \quad k = 0, 1,..., n-1 \qquad (7)$$

Tóm lại quá trình tính toán theo thuật toán Krylov như sau:

- Chọn $v^{(0)}$ tuỳ ý, tính $v_i^{(k)}$ theo (7)
- Giải hệ (6) để tìm ρ

Khi đã tìm được ρ ta có đa thức đặc trưng.

Ta xây dựng hàm ***krylov()*** để thực hiện thuật toán trên:

```
function [lambda, evec] = krylov(a)
% Tim gia tri rieng bang thuat toan Krylov
n = size(a, 1);
v = zeros(n, 1);
v(1) = 1;
b = zeros(n, n);
b(:, n) = v;
for k = 2:n
   v = a*v;
   b(:, n-k+1) = v;
end
c = a*v;
rho = -b\c;
rho = [1 rho'];
lambda = roots(rho);
n = size(lambda, 1);
evec = zeros(n);
```

```
c = evec;
e = eye(n);
for i = 1:n
   b = a - lambda(i)*e;
   c = nulld(b);
   evec(:, i) = c(:, 1);
end
lambda = diag(lambda);
```

Để tìm giá trị riêng của ma trận ta dùng chương trình ***ctkrylov.m***:

```
clear all, clc
a = [17 24 30 17; 8 13 20 7; 2 10 8 6; -23 -43 -54 -26];
[lambda, evec] = krylov(a)
```

§34. THUẬT TOÁN HYMAN

Cho ma trận Hessenberg [B] và λ là một giá trị riêng của [B]. Ta tìm:

$\alpha(\lambda)$, $x_1(\lambda)$, $x_2(\lambda)$,...,$x_{n-1}(\lambda)$

sao cho $[X] = [x_1(\lambda), x_2(\lambda),...,x_{n-1}(\lambda), x_n]^T$ với $x_n = 1$ là nghiệm của bài toán:

$$([B]-\lambda[E])[X]=\alpha[e_1]$$

nghĩa là:

$$\begin{cases}(b_{1,1}-\lambda)x_1+b_{1,2}x_2+\cdots+b_{1,n}1=\alpha\\ b_{2,1}x_1+(b_{2,2}-\lambda)x_2+\cdots+b_{2,n}1=0\\ \qquad\vdots\\ \qquad b_{n,n-1}x_{n-1}+(b_{n,n}-\lambda)1=0\end{cases}$$

Thay thế ngược ta có được các nghiệm x_{n-1}, x_{n-2},..., x_1 và $\alpha(\lambda)$.

Theo quy tắc Cramer ta có:

$$1=x_n=\frac{\det\begin{bmatrix}b_{1,1}-\lambda & * & \alpha\\ b_{2,1} & * & 0\\ 0 & b_{n-1,n-1}-\lambda & 0\\ 0 & b_{n,n-1} & 0\end{bmatrix}}{\det([B]-\lambda[E])}$$

$$=\frac{\alpha(\lambda)(-1)^{n-1}b_{2,1}\cdots b_{n,n-1}}{\det([B]-\lambda[E])}$$

Do đó:

$$\alpha(\lambda) = \frac{(-1)^{n-1}}{b_{2,1}\cdots b_{n,n-1}} \det([B] - \lambda[E])$$

Đa thức $\alpha(\lambda)$ chỉ sai khác đa thức đặc trưng $p(\lambda)$ một nhân tử hằng. Do vậy ta có thể dùng $\alpha(\lambda)$ để tìm các giá trị riêng thay cho $p(\lambda)$. Hàm ***alffa()*** thực hiện công việc này:

```
function r = alfa(a);
n = size(a, 1);
b = a;
l = 1;
for i = 2:n
   l = l*a(i, i-1);
end
sign = (-1)^(n-1);
for i = 1:n
   s = 0;
   for k = 1:n
      s = s + b(k, k);
   end
   p = s/i;
   b = a*(b - p*eye(n));
   r(i+1) = -p;
end
r(1) = 1;
r = sign*r/l;
```

Nếu cho một ma trận bất kì [A], ta có thể biến đổi Householder nó thành ma trận Hessenberg [B] đồng dạng với [A]. Sau khi có các giá trị riêng, ta tìm các vec tơ riêng tương ứng. Hàm ***hyman()*** thực hiện thuật toán này:

```
function [lambda, evec] = hyman(a)
%Tim cac gia tri rieng va vec to rieng bang phuong phap Hyman
b = hessenberg(a);
r = alfa(b);
lambda = roots(r);
n = size(lambda, 1);
evec = zeros(n);
c = evec;
e = eye(n);
for i = 1:n
   b = a - lambda(i)*e;
```

```
    c = nulld(b);
    evec(:, i) = c(:,1);
end
lambda = diag(lambda);
```

Để tìm các giá trị riêng và vec tơ riêng tương ứng của một ma trận ta dùng chương trình ***cthyman.m***:

```
clear all, clc
a = [ 1 2 3 4; 2 5 2 3;7 1 4 1; 3 2 3 7];
[lambda, evec] = hyman(a)
```

§35. TRỰC GIAO HOÁ ARNOLDI

Trong đại số tuyến tính, phương pháp Arnoldi được W. Arnoldi đưa ra năm 1951. Phương pháp lặp Arnoldi sử dụng trực giao hoá Gram – Schmidt để tạo ra dãy các vec tơ trực giao $[q_1]$, .., $[q_n]$ gọi là các vec tơ Arnoldi. Thuật toán cụ thể gồm các bước:

- Cho vec tơ bất kì $[q_1]$ có $\|[q_1]\| = 1$
- Lặp từ k = 2,3,...
  - $[A][q_{k-1}] = [q_k]$
  - for j = 1:k-1
    - $* \left[q_j\right]^T \left[q_k\right] = h_{j,k-1}$
  - $* \left[q_k\right] = h_{j,k-1}\left[q_j\right]$
  - $\|[q_k]\| = h_{k,k-1}$
  - $\left[q_k\right] = \dfrac{q_k}{h_{k,k-1}}$

Ta gọi $[Q_n]$ là ma trận tạo từ n vec tơ Arnoldi đầu tiên $[q_1]$,..,$[q_n]$ đầu tiên và $[H_n]$ là ma trận (Hessenberg trên) tạo bởi n hàng đầu tiên của [H] và có:

$[H_n] = [Q]^T[A][Q]$

Ta xây dựng hàm ***arnoldi()*** thực hiện thuật toán trên:

```
function [Hn, Vn] = arnoldi(A);
k = 50;
m = size(A, 1);
v = rand(m, 1);
n = size(A, 1);
H = zeros(m+1, m);
V = zeros(n, m+1);
```

```
V(:,1) = v/norm(v);
for j = 1:m
   w = A*V(:, j);
   for i = 1:j
      H(i, j) = V(:, i)'*w;
      w = w - H(i, j)*V(:, i);
   end
   H(j+1, j) = norm(w);
   if H(j+1, j) <= eps,
     break;
   end
   V(:, j+1) = w/H(j+1, j);
end
Hn = H(1:m, :);
Vn = V(:, 1:m);
```

Để phân tích ma trận ta dùng chương trình ***ctarnoldi.m***:

```
clear all, clc
A = [ 7 2 3 -1; 2 8 5 1; 3 5 12 9; -1 1 9 7];
[H, V] = arnoldi(A)
```

§36. TRỰC GIAO HOÁ LANCZOS

Cho ma trận [A] đối xứng. Ta phân tích ma trận thành các ma trận [Q] và [T] sao cho:

$[A] = [Q][T][Q]^T$

với: $[Q][Q]^T = [E]$, nghĩa là [Q] là ma trận trực giao và [T] là ma trận ba đường chéo đối xứng. Thuật toán Lanczos gồm các bước:

- Cho vec tơ $[v_1]$ có $\left\|\left[v_1\right]\right\| = 1$
- Cho $v_0 = 0$, $\beta_0 = 0$. $[V] = [v_1]$
- Lặp :
  - $\left[v_{j+1}\right] = \left[A\right]\left[v_j\right] - \beta_{j-1}\left[v_{j-1}\right]$
  - $\text{alfa} = \left[v_j\right]^T\left[v_{j+1}\right]$
  - $\left[v_{j+1}\right] = \left[v_{j+1}\right] - \alpha\left[v_j\right]$
  - $\left[v_{j+1}\right] = \left[v_{j+1}\right] - \left[V\right]\left[V\right]^T\left[v_{j+1}\right]$
  - $\beta_j = \left\|\left[v_{j+1}\right]\right\|$
  - $\left[v_{j+1}\right] = \left[v_{j+1}\right]/\beta_j$

- $[V] = \left[V, v_{j+1}\right]$

Ta xây dựng hàm ***lanczos()*** để thực hiện thuật toán Lanczos

```
function [T, Q] = lanczos(A);
% thuat toan Lanczos cho ma tran doi xung
n = size(A, 1);
Q(:, 1) = rand(n, 1);
Q(:, 1) = Q(:, 1)./norm(Q(:, 1));
a(1) = Q(:, 1)'*A*Q(:, 1);
Q(:, 2) = A*Q(:, 1) - a(1)*Q(:, 1);
b(1) = norm(Q(:, 2));
Q(:, 2) = 1./b(1)*Q(:,  2);
for i = 2:n-1
   a(i) = Q(:, i)'*A*Q(:, i);
   Q(:, i+1) = A*Q(:, i) - a(i)*Q(:, i) - b(i-1)*Q(:, i-1);
   b(i) = norm(Q(:, i+1));
   Q(:, i+1) = 1./b(i)*Q(:, i+1);
end
a(n) = Q(:, n)'*A*Q(:, n);
T = diag(a) + diag(b, 1) + diag(b, -1);
```

Để phân tích ma trận ta dùng chương trình ***ctlanczos.m***:

```
clear all, clc
A = [4 1 3 -2; 1 -2 4 1; 3 4 1 2; -2 1 2 3];
[T, Q] = lanczos(A);
```

CHƯƠNG 3: HỆ PHƯƠNG TRÌNH ĐẠI SỐ TUYẾN TÍNH

§1. KHÁI NIỆM CHUNG

Trong chương này chúng ta sẽ xét các phương pháp số để giải các phương trình đại số tuyến tính dạng:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \qquad\qquad \cdots\cdots\cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Các phương trình này có thể viết gọn dưới dạng:

[A] [x] = [b]

Trong đó:

$$[A] = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \qquad [b] = \begin{bmatrix} b_1 \\ b_2 \\ \cdots \\ b_n \end{bmatrix} \qquad [x] = \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix}$$

Ta sẽ xét 3 trường hợp:

- ☞ số phương trình bằng số ẩn số nên ma trận [A] là ma trận vuông
- ☞ số phương trình nhỏ hơn số ẩn số
- ☞ số phương trình lớn hơn số ẩn số

§2. NGHIỆM CỦA HỆ PHƯƠNG TRÌNH ĐẠI SỐ TUYẾN TÍNH

1. Trường hợp không suy biến: Khi số phương trình m bằng số ẩn số n, ma trận [A] vuông và ta có:

$$[x] = [A]^{-1}[b] \qquad (1)$$

nếu ma trận A không suy biến, nghĩa là định thức của ma trận khác không. Các lệnh MATLAB để giải hệ là (***ctsys.m***):

```
clc
A = [1 2;3 4];
b = [-1;-1];
x = A^-1*b
%x = inv(A)*b
```

2. Trường hợp số phương trình ít hơn số ẩn(nghiệm cực tiểu chuẩn): Nếu số phương trình m ít hơn số ẩn số n thì nghiệm không duy nhất. Giả sử m hàng của ma trận hệ số [A] là độc lập thì vec tơ n chiều có thể phân tích thành hai thành phần:

$$[x] = [x]^{+} + [x]^{-} \qquad (2)$$

Trong đó một ma trận là ma trận không gian hàng của ma trận [A] và được viết dưới dạng tổ hợp của:

$$[x]^{+} = [A]^{T}[\alpha] \tag{3}$$

và ma trận kia là ma trận không gian không sao cho:

$$[A][x]^{-} = 0 \tag{4}$$

Như vậy:

$$[A]\left([x]^{+} + [x]^{-}\right) = [A][A]^{T}[\alpha] + [A][x]^{-} = [A][A]^{T}[\alpha] = [b] \tag{5}$$

Do $[A][A]^{T}$ là ma trận không suy biến m × m có được bằng cách nhân ma trận m × n với ma trận n × m nên ta có thể giải phương trình đối với [α] để có:

$$[\alpha]^{0} = \left[AA^{T}\right]^{-1}[b] \tag{6}$$

Thay (6) vào (3) ta có:

$$[\alpha]^{0+} = [A]^{T}[\alpha]^{0} = [A]^{T}\left[AA^{T}\right]^{-1}[b] \tag{7}$$

Điều này thoả mãn phương trình [A][x] = [b]. Tuy nhiên nó không là nghiệm duy nhất vì nếu thêm bất kì một vec tơ [x] thoả mãn (4) thì nó sẽ cũng là nghiệm. MATLAB dùng lệnh ***pinv*** để giải hệ (***ctpinv.m***)

```
A = [1 2];
b = 3;
x = pinv(A)*b
```

3. Trường hợp số phương trình nhiều hơn số ẩn(nghiệm sai số bình phương bé nhất): Nếu số phương trình m lớn hơn số ẩn số n thì không tồn tại nghiệm thoả mãn đầy đủ các phương trình. Ta cố gắng tìm vec tơ nghiệm có sai số [e] nhỏ nhất.

$$[e] = [A][x] - [b] \tag{8}$$

Vậy thì bài tiám của ta là cực tiểu hoá hàm:

$$J = 0.5\|e\|^{2} = 0.5\|[A][x] - [b]\|^{2} = 0.5\left[[A][x] - [b]\right]^{T}\left[[A][x] - [b]\right] \tag{9}$$

Ta tìm cực tiểu của J bằng cách cho đạo hàm theo x của (9) bằng không.

$$\frac{\partial}{\partial x}J = [A]^{T}\left[[A][x] - [b]\right] = 0 \qquad [x]^{0} = \left[[A]^{T}[A]\right]^{-1}[A]^{T}[b] \tag{10}$$

Chú ý là ma trận [A] có số hàng lớn hơn số cột cho nên không nghịch đảo được. Nghiệm sai số bình phương bé nhất tìm được nhớ dùng lệnh ***pinv*** hay phép chia trái (***ctover.m***):

```
A = [1; 2];
b = [2.1; 3.9];
x = pinv(A)*b
x = A\b
x = (A'*A)^-1*A'*b
```

Để tiện dùng ta viết hàm ***pttt()*** để giải hệ phương trình trong cả 3 trường hợp trên

```
function x = pttt(A, B)
%Ham nay tim nghiem cua pt Ax = B
[M, N] = size(A);
if size(B,1) ~= M
   error('Kich thuoc A va B trong pttt() khong bang nhau!')
end
if M == N
   x = A^-1*B;
elseif M < N
   x = pinv(A)*B;
else
   x = pinv(A)*B;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctpptt.m***:

```
clear all, clc;
a = [ 1 3 4; 2 5 7; 3 1 2];
b = [8  14  6]';
x = pttt(a, b)
```

§3. CÁC PHƯƠNG PHÁP KHỬ

1. Phương pháp khử Gauss: Chúng ta biết rằng các nghiệm của hệ không đổi nếu ta thay một hàng bằng tổ hợp tuyến tính của các hàng khác. Ta xét một hệ phương trình đại số tuyến tính có ma trận [A] không suy biến với m = n = 3.
Phương trình có dạng:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \end{cases} \qquad (1)$$

☞ Trước hết ta khử x_1 ra khỏi các phương trình, ngoại trừ phương trình đầu tiên, bằng cách nhân phương trình đầu tiên với a_{i1}/a_{11} (i là chỉ số hàng) và trừ đi mỗi phương trình đó:

$$\begin{cases} a_{11}^{(0)}x_1 + a_{12}^{(0)}x_2 + a_{13}^{(0)}x_3 = b_1^{(0)} \\ \qquad\quad a_{22}^{(1)}x_2 + a_{23}^{(1)}x_3 = b_2^{(1)} \\ \qquad\quad a_{32}^{(1)}x_2 + a_{33}^{(1)}x_3 = b_3^{(1)} \end{cases} \qquad (2)$$

Trong đó:

$$a_{ij}^{(0)} = a_{ij} \qquad b_i^{(0)} = b_i \qquad \text{với } i = 1, j = 1, 2, 3$$

$$a_{ij}^{(1)} = a_{ij}^{(0)} - \frac{a_{i1}^{(0)}}{a_{11}^{(0)}} a_{1j}^{(0)} \qquad b_i^{(1)} = b_i^{(0)} - \frac{a_{i1}^{(0)}}{a_{11}^{(0)}} b_1^{(0)} \qquad \text{với } i, j = 2, 3$$

Việc này gọi là lấy trụ tại a_{11} và phần tử a_{11} gọi là trụ.

☞ Tiếp theo ta khử x_2 trong phương trình thứ 3 của (2) bằng cách lấy phương trình thứ 2 nhân với $a_{i2}^{(1)} / a_{22}^{(1)}$ (i = 3) và trừ đi phương trình thứ 3:

$$\begin{cases} a_{11}^{(0)}x_1 + a_{12}^{(0)}x_2 + a_{13}^{(0)}x_3 = b_1^{(0)} \\ \qquad\qquad a_{22}^{(1)}x_2 + a_{23}^{(1)}x_3 = b_2^{(1)} \\ \qquad\qquad\qquad\quad a_{33}^{(2)}x_3 = b_3^{(2)} \end{cases} \tag{3}$$

Trong đó:

$$a_{ij}^{(2)} = a_{ij}^{(1)} - \frac{a_{i2}^{(1)}}{a_{22}^{(1)}} a_{2j}^{(1)} \qquad b_i^{(2)} = b_i^{(1)} - \frac{a_{i2}^{(1)}}{a_{22}^{(1)}} b_2^{(1)} \qquad \text{với } i, j = 3 \tag{4}$$

Quá trình này được gọi là thuật toán khử Gauss tiến và được tổng quát hoá thành:

$$\begin{aligned} a_{ij}^{(k)} &= a_{ij}^{(k-1)} - \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} a_{kj}^{(k-1)} \qquad i, j = k+1, k+2, ..., m \\ b_i^{(k)} &= b_i^{(k-1)} - \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} b_k^{(k-1)} \qquad i = k+1, k+2, ..., m \end{aligned} \tag{5}$$

Để thực hiện thuật toán khử Gauss ta dùng đoạn mã lệnh:

```
for k = 1:n-1
  for i= k+1:n
    if A(i, k) ~= 0
      lambda = A(i, k)/A(k, k);
      A(i, k+1:n) = A(i, k+1:n) - lambda*A(k, k+1:n);
      b(i)= b(i) - lambda*b(k);
    end
  end
end
```

Sau khi có hệ phương trình dạng ta giác ta tìm nghiệm dễ dàng. Từ phương trình thứ 3 của (3) ta có:

$$x_3 = \frac{b_3^{(2)}}{a_{33}^{(2)}} \tag{6a}$$

Thay vào phương trình thứ 2 ta có:

$$x_2 = \frac{b_2^{(1)} - a_{23}^{(1)}x_3}{a_{22}^{(1)}} \tag{6b}$$

và cuối cùng từ phương trình thứ nhất ta có:

$$x_1 = \frac{1}{a_{11}^{(0)}}\left(b_1^{(0)} - \sum_{j=2}^{3} a_{1j}^{(0)} x_j\right) \qquad (6c)$$

Ta cũng có thể tổng quát hoá quá trình tìm nghiệm bằng cách tính lùi và tìm nghiệm bằng:

$$x_i = \frac{1}{a_{ii}^{(i-1)}}\left(b_i^{(i-1)} - \sum_{j=i+1}^{m} a_{ij}^{(i-1)} x_j\right) \qquad i = m, m-1, ..., 1 \qquad (7)$$

và tìm nghiệm bằng đoạn mã lệnh:

```
for k = n:-1:1
    b(k) = (b(k) - A(k, k+1:n)*b(k+1:n))/A(k, k);
end
```

Như vậy phương pháp Gauss gồm hai bước:

- khử theo thuật toán Gauss
- tìm nghiệm của phương trình dạng tam giác

Đoạn mã lệnh để tráo hàng được viết trong hàm ***swaprows()***:

```
function v = swaprows(v ,i ,j)
% Trao doi hang i va hang j cua ma tran  v.
% Cu phap: v = swaprows(v, i, j)
temp = v(i, :);
v(i, :) = v(j, :);
v(j, :) = temp;
```

Ta xây dựng hàm ***gauss()*** để thực hiện thuật toán khử Gauss

```
function x = gauss(A, B)
%Kich thuoc cua ma tran A, B la NA x NA va NA x NB.
%Ham nay dung giai he pt Ax = B bang phuong phap khu Gauss
NA = size(A,2);
[NB1, NB] = size(B);
if NB1 ~= NA
   error('A va B phai co kich thuoc tuong ung');
end
N = NA + NB;
AB = [A(1:NA, 1:NA) B(1:NA, 1:NB)];
epss = eps*ones(NA, 1);
for k = 1:NA
   %Chon tru AB(k, k)
   [akx,kx] = max(abs(AB(k:NA, k))./ ...
```

```
    max(abs([AB(k:NA, k + 1:NA) epss(1:NA - k + 1)]'))');
    if akx < eps
        error('Ma tran suy bien va nghiem khong duy nhat');
    end
    mx = k + kx - 1;
    if kx > 1 % trao hang khi can
        swaprows(AB, k, mx);
    end
% Khu Gauss
    AB(k,k + 1:N) = AB(k,k+1:N)/AB(k,k);
    AB(k, k) = 1;
    for m = k + 1: NA
        AB(m, k+1:N) = AB(m, k+1:N) - AB(m, k)*AB(k, k+1:N); %(2.2.5)
        AB(m, k) = 0;
    end
end
%Tim nghiem
x(NA, :) = AB(NA, NA+1:N);
for m = NA-1: -1:1
    x(m, :) = AB(m, NA + 1:N)-AB(m, m + 1:NA)*x(m + 1:NA, :); %(2.2.7)
end
```

Để giải hệ phương trình ta dùng ***ctgauss.m***

```
clear all, clc
A = [1 1 1;2 -1 -1; 1 1 -1];
b = [2 0 1]';
x = gauss(A, b)
```

2. Phương pháp khử Gauss - Jordan: Xét hệ phương trình AX = B. Khi giải hệ bằng phương pháp Gauss ta đưa nó về dạng ma trận tam giác sau một loạt biến đổi. Phương pháp khử Gauss - Jordan cải tiến cách khử Gauss bằng cách đưa hệ về dạng :

$[E][X] = [B^*]$

và khi đó nghiệm của hệ chính là $[B^*]$. Trong phương pháp Gauss - Jordan mỗi bước tính phải tính nhiều hơn phương pháp Gauss nhưng lại không phải tính nghiệm. Để đưa ma trận [A] về dạng ma trận [E] tại bước thứ i ta phải có $a_{ii} = 1$ và $a_{ij} = 0$. Như vậy tại lần khử thứ i ta biến đổi:

1. $a_{ij} = a_{ij}/a_{ii}$ (j = i + 1, i + 2,..., n)
2. k = 1, 2,..., n

   $a_{kj} = a_{kj} - a_{ij}a_{ki}$ (j = i + 1, i + 2,..., n)

   $b_k = b_k - b_i a_{ki}$

Để giải hệ phương trình bằng phương pháp Gauss - Jordan ta tạo ra hàm ***gaussjordan()***

```
function x = gaussjordan(A, B)
%Kich thuoc cua ma tran A, B la NA x va NA x NB.
%Ham nay dung giai he Ax = B bang thuat toan loai tru Gauss-Jordan
NA = size(A, 2);
[NB1,NB] = size(B);
if NB1 ~= NA
    error('A va B phai co kich thuoc tuong ung');
end
for i = 1:NA
   if A(i, i) == 0 % trao hang neu can
      swaprows(A, i, mx);
   end
   c = A(i, i);
   for j = i:NA
       A(i,j) = A(i, j)/c;
   end
    B(i) = B(i)/c;
       for k = 1:NA
          if k~=i
            c = A(k, i);
            A(k, i:NA) = A(k, i:NA)-A(i, i:NA)*c;
            B(k) = B(k) - B(i)*c;
          end
       end
end
x = B;
```

và dùng chương trình ***ctgaussjordan.m*** giải hệ:

```
clear all, clc
a = [5  3  1;2  -1  1; 1  -1  -1];
b = [9; 2; -1];
x = gaussjordan(a, b)
```

§4. GIẢI HỆ PHƯƠNG TRÌNH BẰNG CÁCH PHÂN TÍCH MA TRẬN

1. Khái niệm chung: Một ma trận không suy biến [A] gọi là phân tích được thành tích hai ma trận [L] và [R] nếu:

[A] = [L] [R]

Việc phân tích này, nếu tồn tại, là không duy nhất. Nếu ma trận [L] có các phần tử nằm trên đường chéo chính bằng 1, ta có phép phân tích Doolittle. Nếu ma trận [R] có các phần tử nằm trên đường chéo chính bằng 1, ta có phép phân tích Crout. Nếu $[R] = [L]^T$ (hay $[L] = [R]^T$) ta có phép phân tích Choleski.

2. Phân tích Doolittle: Ta xét hệ phương trình [A][X] = [B]. Nếu ta phân tích ma trận [A] thành tích hai ma trận [L] và [R] sao cho:

[A] = [L][R]

trong đó [L] là ma trận tam giác trái và [R] là ma trận tam giác phải. Với ma trận bậc 3 [L] và [R] có dạng:

$$[L] = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \qquad [R] = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

Khi đó hệ phương trình được viết lại là:

[L][R][X] = [B]

Ta đặt [R][X] = [Y] và hệ trở thành

[L][Y] = [B]

Do [L] là ma trận tam giác nên ta dễ dàng tìm được [Y]. Sau khi có [Y] ta tiếp tục tìm [X]. Như vậy bài toán đưa về việc phân tích ma trận [A].

Để giải hệ phương trình bằng cách phân tích ma trận theo thuật toán Doolittle ta dùng hàm ***doolittlesol()***:

```
function x = doolittlesol(A, b)
% Giai he AX = B, trong do A = LU
% nghia la A co dang [L\U].
% Cu phap: x = doolittlesol(A, b)
n = size(A, 1);
[l, r] = doolittle(A);
%tim nghiem mt tam giac trai
y(1,:) = b(1)/l(1, 1);
for m = 2:n
    y(m, :) = (b(m)  -l(m, 1:m-1)*y(1:m-1, :))/l(m, m);
end
%tim nghiem mt tam giac phai
x(n, :) = y(n)/r(n, n);
for m = n-1: -1:1
   x(m, :) = (y(m) -r(m, m + 1:n)*x(m + 1:n, :))/r(m, m);
end
```

Áp dụng hàm ***doolittlesol()*** giải hệ phương trình:

$$\begin{bmatrix} 4 & -3 & 6 \\ 8 & -3 & 10 \\ -4 & 12 & -10 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

ta dùng chương trình ***ctdoolitle.m***:

```
a = [4 -3 6; 8 -3 10; -4 12 -10];
b = [1; 0; 0];
x = doolittlesol(a, b)
```

3. Phân tích Crout: Tương tự như thuật toán Doolittle, ta có thể phân tích ma trận [A] theo thuật toán Crout thành tích của ma trận [L] và [R]. Để giải hệ phương trình bằng cách phân tích ma trận theo thuật toán Crout ta dùng hàm ***croutsol()***:

```
function x = croutsol(a, b)
%Ham dung giai he pt AX = B bang thuat toan Crout
% Cu phap: x = croutsol(a, b)
n =size(a,1);
[l,r] = crout(a);

y(1,:) = b(1)/l(1, 1);
for m = 2:n
  y(m,:) = (b(m) - l(m, 1:m-1)*y(1:m-1,:))/l(m, m);
end
x(n, :) = y(n)/r(n, n);
for m = n-1: -1:1
  x(m, :) = (y(m) - r(m, m + 1:n)*x(m + 1:n, :))/r(m, m);
end
```

Khi giải phương trình ta chương trình ***ctcrout.m***:

```
clear all, clc
a = [ 4  8  20;  6  13  16; 20 16 -91];
b = [24; 18; -110];
x = croutsol(a, b)
```

4. Phân tích Choleski: Sau khi phân tích ma trận [A] theo thuật toán Choleski, hệ phương trình [A][X] = [B] trở thành:

$[L][L]^T[X] = [B]$

Trước hết ta tìm nghiệm của hệ phương trình [L][Y] = [B] và sau đó tìm nghiệm [X] từ hệ phương trình $][L]^T[X] = [Y]$. Ta xây dựng hàm ***choleskisol()*** để thực hiện thuật toán này:

```
function x = choleskisol(a, b)
%Giai  he pt bang thuat toan Choleski
%Cu phap: x = choleskisol(a, b)
n =size(a,1);
l = choleski(a);
r = l';
y(1,:) = b(1)/l(1, 1);
for m = 2:n
    y(m,:) = (b(m) - l(m, 1:m-1)*y(1:m-1, :))/l(m, m);
end
x(n, :) = y(n)/r(n, n);
for m = n-1: -1:1
   x(m, :) = (y(m)  -r(m, m + 1:n)*x(m + 1:n, :))/r(m, m);
end
```

Để giải hệ phương trình

$$\begin{bmatrix} 4 & -2 & 2 \\ -2 & 2 & -4 \\ 2 & -4 & 11 \end{bmatrix}\begin{bmatrix} x_1 \\ x_1 \\ x_1 \end{bmatrix} = \begin{bmatrix} 5 \\ -10 \\ 27 \end{bmatrix}$$

ta dùng chương trình ***ctcholeski.m***:

```
clear all, clc
a = [4 -2 2;-2 2 -4;2 -4  11];
b = [6; -10; 27];
x = choleskisol(a, b)
```

5. Phân tích QR: Ta xét hệ phương trình [A][X] = [B]. Phân tích ma trận [A] thành tích của hai ma trận [Q] và [R] sao cho:

[A] = [Q]*[R]

Trong đó [Q] là ma trận trực giao, nghĩa là $[Q]^T[Q] = [E]$, và [R] là ma trận tam giác phải. Như vậy phương trình trở thành:

[Q]*[R]*[X] = [B]

Nhân hai vê của phương trình với $[Q]^T$ ta có:

$[Q]^T[Q]*[R]*[X] = [Q]^T[B]$

hay:

$[R]*[X] = [Q]^T[B]$

Hệ phương trình này dễ dàng tìm nghiệm vỉ [R] là ma trận tam giác. Khi giải hệ phương trình ta dùng chương trình ***ctqrsol.m***:

```
clear all, clc
```

```
A = [ 1 2 3 5; 4 5 6 2; 4 6 8 9; 9 3 6 7];
b = [2 4 6 8]';
[q, r] = qrdecomp(A);
c = transpose(q)*b;
x = r\c
```

§5. CÁC MA TRẬN ĐẶC BIỆT

1. Ma trận đường chéo bậc 3: Ta xét hệ phương trình [A][X] = [B] với [A] là ma trận đường chéo có dạng:

$$[A]=\begin{bmatrix} d_1 & e_1 & 0 & 0 & \cdots & 0 \\ c_1 & d_2 & e_2 & 0 & \cdots & 0 \\ 0 & c_2 & d_3 & e_3 & \cdots & 0 \\ 0 & 0 & c_3 & d_4 & & \vdots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & c_{n-1} & d_n \end{bmatrix}$$

Ta lưu các phần tử khác 0 của [A] dưới dạng vec tơ:

$$[c]=\begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{n-1} \end{bmatrix} \qquad [d]=\begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \\ d_n \end{bmatrix} \qquad [e]=\begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{n-1} \end{bmatrix}$$

để giảm bớt số lượng phần tử cần lưu trữ.

Bây giờ ta phân tích ma trận theo thuật toán Doolittle:

hàng k - $(c_{k-1}/d_{k-1})\times$hàng k-1 $\rightarrow$ hàng k k = 1, 2,…, n

và: $d_k - (c_{k-1}/d_{k-1})\times e_{k-1} \rightarrow d_k$

Để hoàn tất thuật việc phân tích, ta lưu hệ số $\lambda = c_{k-1}/d_{k-1}$ vào vị trí của c_{k-1} trước đó

$c_{k-1}/d_{k-1} \rightarrow c_{k-1}$

Như vậy thuật toán phân tích ma trận là:

```
for k = 2:n
        lambda = c(k-1)/d(k-1);
        d(k) = d(k) - lambda*e(k-1)
        c(k-1) = lambda;
end
```

Sau đó ta tìm nghiệm của phương trình [L][R][X] = [B] bằng cách giải phương trình [L][Y] = [B] và sau đó là phương trình [R][X] = [Y]. Phương trình [L][Y] = [B] có dạng:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ c_1 & 1 & 0 & 0 & \cdots & 0 \\ 0 & c_2 & 1 & 0 & \cdots & 0 \\ 0 & 0 & c_3 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & 0 & 0 & c_{n-1} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ \cdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ \cdots \\ b_n \end{bmatrix}$$

để tìm nghiệm [Y] bằng cách thay thế tiến ta dùng đoạn lệnh:

```
y(1) = b(1);
for k = 2:n
        y(k) = b(k) - c(k-1)*y(k-1);
end
```

Phương trình [R][X] = [Y] có dạng:

$$\begin{bmatrix} d_1 & e_1 & 0 & 0 & \cdots & 0 \\ 0 & d_2 & e_2 & 0 & \cdots & 0 \\ 0 & 0 & d_3 & e_3 & \cdots & 0 \\ 0 & 0 & 0 & d_4 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & d_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \cdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ \cdots \\ y_n \end{bmatrix}$$

để tìm nghiệm [X] bằng cách thay thế lùi ta dùng đoạn lệnh:

```
x(n) = y(n);
for k = n-1:-1:1
        x(k) = (y(k) - e(k)*x(k+1))/d(k);
end
```

Ta xây dựng hàm ***band3()*** để phân tích ma trận dạng đường chéo:

```
function [c, d, e] = band3(c , d, e)
% Phan tich ma tran  A = [c\d\e].
% Cu phap: [c, d, e] = band3(c, d, e)
n = length(d);
for k = 2:n
   lambda = c(k-1)/d(k-1);
   d(k) = d(k) - lambda*e(k-1);
 c(k-1) = lambda;
end
```

Ta viết hàm ***band3sol()*** dùng để giải hệ phương trình có ma trận [A] dạng đường chéo:

```
function x = band3sol(c ,d, e, b)
% Giai he  A*x = b voi A = [c\d\e] la tich LU
% Cu phap: x =band3sol(c, d, e, b)
[c, d, e] = band3(c, d, e);
n = length(d);
for k = 2:n % thay the tien
   b(k) = b(k) - c(k-1)*b(k-1);
end
b(n) = b(n)/d(n); % thay the lui
for k = n-1:-1:1
   b(k) = (b(k) - e(k)*b(k+1))/d(k);
end
x = b;
```

Ta dùng chương trình ***ctband3eq. m*** để giải hệ phương trình:

```
clear all, clc
c = [-1; -2; 3; 3];
d = [6 7 8 7 5]';
e = [2 2 2 -2]';
b = [2; -3; 4; -3; 1];
x = band3sol(c, d, e, b);
```

2. Ma trận đường chéo đối xứng bậc 5: Khi giải phương trình vi phân thường bậc 4 ta thường gặp một hệ phương trình đại số tuyến tính dạng băng đối xứng có bề rộng bằng 5. Ma trận [A] khi đó có dạng:

$$[A]=\begin{bmatrix} d_1 & e_1 & f_1 & 0 & 0 & 0 & \cdots & 0 \\ e_1 & d_2 & e_2 & f_2 & 0 & 0 & \cdots & 0 \\ f_1 & e_2 & d_3 & e_3 & f_3 & 0 & \cdots & 0 \\ 0 & f_2 & e_3 & d_4 & & & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & f_{n-4} & e_{n-3} & d_{n-2} & e_{n-2} & f_{n-2} \\ 0 & \cdots & 0 & 0 & f_{n-3} & e_{n-2} & d_{n-1} & e_{n-1} \\ 0 & \cdots & 0 & 0 & 0 & f_{n-2} & e_{n-1} & d_n \end{bmatrix}$$

và ta lưu ma trận [A] dưới dạng vec tơ:

$$[d]=\begin{bmatrix} d_1 \\ d_1 \\ \vdots \\ d_{n-2} \\ d_{n-1} \\ d_n \end{bmatrix} \quad [e]=\begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{n-2} \\ e_{n-1} \end{bmatrix} \quad [f]=\begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_{n-2} \end{bmatrix}$$

Ta thực hiện thuật toán biến đổi ma trận:

hàng (k + 1) - (e_k/d_k) × hàng k → hàng (k + 1)

hàng (k + 2) - (f_k/d_k) × hàng k → hàng (k + 2)

Các số hạng bị thay đổi theo thuật toán này là:

$d_{k+1} - (e_k/d_k)\, e_k \rightarrow d_{k+1}$

$e_{k+1} - (e_k/d_k)\, f_k \rightarrow e_{k+1}$

$d_{k+2} - (f_k/d_k)\, f_k \rightarrow d_{k+2}$

và lưu trữ lại:

$e_k/d_k \rightarrow e_k \qquad f_k/d_k \rightarrow f_k$

sau khi đã biến đổi ma trận, ta giải hệ phương trình có ma trận tam giác.

Hàm ***band5()*** dùng để phân tích ma trận:

```
function [d, e, f] = band5(d, e, f)
%  A = [f\e\d\e\f].
% Cu phap: [d, e, f] = band5(d, e, f)
n = length(d);
for k = 1:n-2
   lambda = e(k)/d(k);
   d(k+1) = d(k+1) - lambda*e(k);
   e(k+1) = e(k+1) - lambda*f(k);
   e(k) = lambda;
   lambda = f(k)/d(k);
   d(k+2) = d(k+2) - lambda*f(k);
   f(k) = lambda;
end
lambda = e(n-1)/d(n-1);
d(n) = d(n) - lambda*e(n-1);
e(n-1) = lambda;
```

Ta viết hàm ***band5sol()*** để giải hệ phương trình:

```
function x = band5sol(d, e, f, b)
% Giai he A*x = b voi A = [f\e\d\e\f]
```

```
% Cu phap: x = band5sol(d, e, f, b)
[e,d,f ] = band5(e, d, f);
n = length(d);
b(2) = b(2) - e(1)*b(1);
for k = 3:n
   b(k) = b(k) - e(k-1)*b(k-1) - f(k-2)*b(k-2);
end
```

Để giải hệ phương trình

$$\begin{bmatrix} 1 & 1 & 2 & 0 & 0 & 0 \\ 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 3 & 3 & 2 & 2 & 0 \\ 0 & 1 & 2 & 1 & 2 & 1 \\ 0 & 0 & 2 & 2 & 2 & -1 \\ 0 & 0 & 0 & 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \\ 12 \\ 7 \\ 5 \\ 1 \end{bmatrix}$$

ta dùng chương trình ***cban5eq.m***

```
clear all, clc
d = [1  2  3  1  2  1]';
e = [1  3  2  2  -1]';
f = [2  1  2  1]';
b = [4  7  12  7  5  1];
x = band5sol(d, e, f, b)
```

§6. CÁC PHƯƠNG PHÁP LẶP ĐỂ GIẢI HỆ PHƯƠNG TRÌNH ĐẠI SỐ TUYẾN TÍNH

Nói chung có hai phương pháp giải hệ phương trình đại số tuyến tính: phương pháp trực tiếp và phương pháp lặp. Các bài toán kĩ thuật thường đưa về hệ phương trình đại số tuyến tính có ma trận [A] thưa và lớn nên các phương pháp lặp rất thích hợp.

Các phương pháp lặp được chia thành hai loại: phương pháp lặp tĩnh và phương pháp lặp động.

Ta xét hệ phương trình đại số tuyến tính [A][X] = [B]. Ta đưa về dạng lặp:

[X] = [C][X] + [D]

Sau mỗi lần tính ta có số dư:

[R] = [B] - [A][X]

Khi lặp từ phương trình này, các ma trận [C] và [D] không đổi. Vì vậy nên các phương pháp xuất phát từ đây gọi là các phương pháp lặp tĩnh. Các phương pháp này dễ hiểu, dễ lập trình nhưng không hiệu quả.

Các phương pháp này gồm có:

• Phương pháp lặp Jacobi: Phương pháp này tính giá trị của một biến dựa trên giá trị của các biến khác. Nó hội tụ chậm và rất có thể không hội tụ trong một số trường hợp.

• Phương pháp lặp Gauss - Seidel: Nó tương tự như phương pháp lặp Jacobi nhưng khi tính giá trị của biến thứ k ta dùng các giá trị các biến vừa được cập nhật. Phương pháp này hội tụ nhanh hơn phương pháp lặp Jacobi nhưng không nhanh bằng các phương pháp lặp không ổn định.

• Phương pháp lặp có tăng SOR: Phương pháp này đưa ra từ phương pháp Gauss - Seidel bằng cách đưa thêm hệ số ngoại suy ω. Với ω được chọn tối ưu, phương pháp này hội tụ nhanh hơn phương pháp Gaus - Seidel. Khi $\omega = 1$ phương pháp SOR trở thành phương pháp Gauss -
Seidel. Tốc độ hội tụ của phương pháp SOR phụ thuộc vào ω

• Phương pháp lặp có tăng đối xứng SSOR: Phương pháp này không có ưu điểm nào trội hơn SOR.

Các phương pháp lặp không ổn định mới được xây dựng, khó hiểu, nhưng hiệu quả cao. Trong quá trình lặp, việc tính toán bao hàm các thông tin thay đổi sau mỗi bước tính.

Các phương pháp này bao gồm:

• Phương pháp gradient liên hợp CG(Conjugate Gradient): Phương pháp này tạo ra một dãy các vec tơ liên hợp (hay trực giao) là số dư của phép lặp. Chúng cũng là gradient của một hàm bậc 2 mà việc tìm cực tiểu tương đương với việc giải hệ phương trình đại số tuyến tính. Phương pháp CG rất hiệu quả khi ma trận [A] đối xứng, xác định dương ví chỉ đòi hỏi lưu trữ một số ít phần tử. Tốc độ hội tụ của phương pháp này phụ thuộc ***số điều kiện*** của ma trận (số điều kiện của ma trận đo độ nhạy của nghiệm của hệ phương trình đại số tuyến tính với sai số trong số liệu. Nó cho biết độ chính xác của kết quả từ phép nghịch đảo ma trận và nghiệm của hệ phương trình đại số tuyến tính).

• Phương pháp số dư cực tiểu MINRES(Minimum Residual) và phương pháp LQ đối xứng SYMMLQ(Symmetric LQ)

• Phương pháp gradient liên hợp dùng cho hệ thường CGNE(Conjugate Gradient on Normal Equations) và CGNR(Conjugate Gradient on Normal Equations minimizing the Residual): Các phương pháp này dựa trên việc áp dụng phương pháp CG vào một trong hai dạng hệ phương trình đại số tuyến tính.

- CNGR dùng giải hệ dạng $[A]^T[A][X] = [B']$ với $[B'] = [A]^T[B]$
- CGNE dùng giải hệ dạng $[A][A]^T[Y] = [B]$ đối với [Y] và sau đó giải hệ $[X] = [A]^T[Y]$

Khi ma trận [A] không đối xứng, không suy biến thì $[A][A]^T$ và $[A]^T[A]$ đối xứng, xác định dương nên có thể dùng phương pháp CG.

• Phương pháp số dư cực tiểu tổng quát GMRES(Generalized Minimal Residual): Phương pháp GMRES tính toán dãy các vec tơ trực giao và kết hợp

các này bằng bài toán bình phương bé nhất để giải và cập nhật. Tuy nhiên nó đòi hỏi lưu toàn bộ dãy. Do vậy phương án khởi động lại được dùng trong phương pháp này. Phương pháp này tiện dùng khi ma trận hệ số không đối xứng.

- Phương pháp gradient liên hợp kép BiCG(Biconjugate Gradient): Phương pháp này tạo ta hai dãy vec tơ giống như CG, một dựa trên hệ với ma trận [A] và một dựa trên $[A]^T$. Thay vì trực giao hoá mỗi dãy, chúng trực giao tương hỗ hai "trực giao kép". Nó rất hữu ít khi ma trận có ma trận hệ số không đối xứng, không suy biến.
- Phương pháp gần như số dư cực tiểu QMR(Quasi - Minimal Residual): Phương pháp QMR dùng bình phương tối thiểu để giải và cập nhật số dư BiCG. Phương pháp này dùng cho hệ phương trình có ma trận hệ số không đối xứng.
- Phương pháp gradient liên hợp bậc 2 CGS(Conjugate Gradient Squared): Phương pháp CGS là một biến thể của BiCG, dùng cập nhất dãy [A] và $[A]^T$. Phương pháp này có ưu điểm là không cần nhân với ma trận hệ số chuyển vị và được dùng cho hệ phương trình đại số tuyến tính có ma trận hệ số không đối xứng.
- Phương pháp gradient liên hợp kép ổn định BiCGSTAB(Biconjugate Gradient Stabilized): Phương pháp BiCGSTAB cũng là một biến thể của BiCG. Nó được dùng cho hệ phương trình có ma trận hệ số không đối xứng.
- Phương pháp Chebyshev: Phương pháp này tính lặp các đa thức với các hệ số được chọn để cực tiểu hoá chuẩn của số dư theo nghĩa min - max. Ma trận hệ số phải xác định dương. Nó được dùng cho hệ phương trình có ma trận hệ số không đối xứng.

Ta biết rằng tốc độ hội tụ của phép lặp phụ thuộc rất nhiều vào phổ của ma trận(các giá trị riêng của ma trận). Do vậy phép lặp thường đưa thêm một ma trận thứ hai để biến đổi ma trận hệ số thành ma trận có phổ thích hợp. Ma trận biến đổi như vậy gọi là ma trận điều kiện trước(preconditioner). Một preconditioner tốt sẽ cải thiện sự hội tụ của phương pháp lặp. Nhiều trường hợp, nếu không có preconditioner, phép lặp sẽ không hội tụ. Preconditioner đơn giản nhất chính là ma trận đường chéo mà $M_{i,j} = A_{i,j}$ nếu i = j và các phần tử khác bằng zero. Ma trận như vậy gọi là ma trận điều kiện trước Jacobi.

Trong tính toán, tồn tại hai loại ma trận điều kiện trước:

- ma trận [M] xấp xỉ ma trận [A] và làm cho việc giải hệ [M][X] = [B] dễ hơn giải hệ [A][X] = [B]

- ma trận [M] xấp xỉ $[A]^{-1}$ sao cho chỉ cần tính [M][B] là có [X]

Phần lớn các ma trận [M] thuộc loại thứ nhất.

§7. PHƯƠNG PHÁP LẶP JACOBI

Xét hệ phương trình AX = F. Bằng cách nào đó ta đưa hệ phương trình về dạng

X = BX + G

trong đó:

$B = (b_{ij})_{n,n}$

$G = (g_1, g_2, ..., g_n)^T$

Chọn vectơ:

$X = (x_1^{(o)}, x_2^{(o)}, ..., x_n^{(o)})^T$

làm xấp xỉ thứ 0 của nghiệm đúng và xây dựng xấp xỉ

$X^{(m+1)} = BX^{(m)} + G \quad (m = 0,1,....)$

Người ta chứng minh rằng nếu phương trình ban đầu có nghiệm duy nhất và một trong ba chuẩn của ma trận B nhỏ hơn 1 thì dãy xấp xỉ hội tụ về nghiệm duy nhất đó. Cho một ma trận B, chuẩn của ma trận B, kí hiệu $\|B\|$ là một trong 3 số :

$$\|B\|_1 = \max_i \sum_{j=1}^{n} b_{ij}$$

$$\|B\|_2 = \max_j \sum_{j=1}^{n} b_{ij}$$

$$\|B\|_3 = \left(\sum_{i=1}^{n} \sum_{j=1}^{n} b_{ij}^2 \right)^{1/2}$$

(Chuẩn của ma trận quan hệ tới sự hội tụ của phương pháp lặp)

Ta xây dựng hàm ***jacobi()*** để thực hiện thuật toán trên:

```
function x = jacobi(a, b, x0, kmax)
%Tim nghiem cua pt Ax = B bang thuat toan Jacobi.
%Cu phap: x = jacobi(a, b, x0, kmax)
% hay jacobi(a, b, x0, kmax)
if nargin < 4
   tol = 1e-6;
   kmax = 100; % jacobi(a, b, x0)
elseif kmax < 1
   tol = max(kmax, 1e-16);
   kmax = 100; %jacobi(a, b, x0, tol)
else
   tol = 1e-6; %jacobi(a, b, x0, kmax)
end
if nargin < 3
   x0 = zeros(size(b)); end
   na = size(a, 1);
   x = x0;
   At = zeros(na, na);
   for m = 1:na
      for n = 1:na
         if n ~= m
            At(m, n) = -a(m, n)/a(m, m);
```

```
            end
        end
        Bt(m, :) = b(m, :)/a(m, m);
    end
for k = 1: kmax
    x = At*x + Bt;
    if nargout == 0,
        x
    end
    if norm(x - x0)/(norm(x0) + eps) < tol
        break;
    end
    x0 = x;
end
```

Để giải phương trình ta chương trình ***ctjacobi.m***:

```
b = [1 -1]';
a = [3 2;1 2];
x0 = [0 0]';
x = jacobi(a, b, x0, 20)
```

§8. PHƯƠNG PHÁP LẶP GAUSS - SEIDEL

Phương pháp lặp Gauss - Seidel được cải tiến từ phương pháp Jacobi. Nội dung cơ bản của phương pháp là ở chỗ khi tính nghiệm xấp xỉ thứ (k+1) của ẩn x_i ta sử dụng các xấp xỉ thứ (k+1) đã tính của các ẩn $x_1,...,x_{i-1}$. Giả sử đã cho hệ [A][X] = [B] thì ta có nghiệm :

$$x_i = \beta_i + \sum_{j=1}^{n} \alpha_{ij} x_j \qquad i = 1,...,n$$

Lấy xấp xỉ ban đầu tuỳ ý $x_1^{(o)}$, $x_2^{(o)}$,...., $x_n^{(o)}$ và tất nhiên ta cố gắng lấy chúng tương ứng với x_1, x_2 ,..., x_n (càng gần càng tốt). Tiếp theo ta giả sử rằng đã biết xấp xỉ thứ k $x_i^{(k)}$ của nghiệm. Theo Seidel ta sẽ tìm xấp xỉ thứ (k+1) của nghiệm theo các công thức sau :

$$x_1^{(k+1)} = \beta_1 + \sum_{j=1}^{n} \alpha_{ij} x_j^{(k)}$$

$$x_2^{(k+1)} = \beta_1 + \alpha_{21} x_1^{(k+1)} + \sum_{j=2}^{n} \alpha_{ij} x_j^{(k)}$$

.....

$$x_i^{(k+1)} = \beta_i + \sum_{j=1}^{i-1} \alpha_{ij} x_j^{(k+1)} + \sum_{j=i}^{n} \alpha_{ij} x_j^{(k)}$$

.....

$$x_n^{(k+1)} = \beta_n + \sum_{j=1}^{n-1} \alpha_{ij} x_j^{(k+1)} + \alpha_{nn} x_n^{(k)}$$

Thông thường phương pháp Gauss - Seidel hội tụ nhanh hơn phương pháp Jacobi nhưng tính toán phức tạp hơn. Dể dễ hiểu phương pháp này chúng ta xét một ví dụ cụ thể:

Cho hệ phương trình :

$$\begin{cases} 10x_1 + x_2 + x_3 = 12 \\ 2x_1 + 10x_2 + x_3 = 13 \\ 2x_1 + 2x_2 + 10x_3 = 14 \end{cases}$$

nghiệm đúng của hệ là (1 , 1, 1)

Ta đưa về dạng thuận tiện cho phép lặp :

$$\begin{cases} x_1 = 1.2 - 0.1x_2 - 0.1x_3 \\ x_2 = 1.3 - 0.2x_1 - 0.1x_3 \\ x_3 = 1.4 - 0.2x_1 - 0.2x_2 \end{cases}$$

Lấy $x_1^{(o)} = 1.2$; $x_2^{(o)} = 0$; $x_3^{(o)} = 0$

Sử dụng phương pháp lặp Gauss - Seidel ta có:

$$\begin{cases} x_1^{(1)} = 1.2 - 0.1 \times 0 - 0.1 \times 0 = 1.2 \\ x_2^{(1)} = 1.3 - 0.2 \times 1.2 - 0.1 \times 0 = 1.06 \\ x_3^{(1)} = 1.4 - 0.2 \times 1.2 - 0.2 \times 1.06 = 0.948 \end{cases}$$

$$\begin{cases} x_1^{(2)} = 1.2 - 0.1 \times 1.06 - 0.1 \times 0.948 = 0.9992 \\ x_2^{(2)} = 1.3 - 0.2 \times 0.9992 - 0.1 \times 0.948 = 1.00536 \\ x_3^{(2)} = 1.4 - 0.2 \times 0.9992 - 0.2 \times 1.00536 = 0.999098 \end{cases}$$

và cứ thế tiếp tục cho đến khi hội tụ. Ta xây dựng hàm ***gausseidel()*** để thực hiện thuật toán trên:

```
function x = gausseidel(a, b, x0, kmax)
%Tim nghiem cua he  AX  = B bang cach lap Gauss–Seidel.
if nargin < 4
   kmax = 100;
end
if nargin < 3
```

```
    x0 = zeros(size(b));
    kmax = 100;
end
na = size(a,1);
x = x0;
for k = 1: kmax
    x(1, :) = (b(1, :) - a(1, 2:na)*x(2: na, :))/a(1,1);
    for m = 2:na-1
        tmp = b(m, :) - a(m, 1:m-1)*x(1: m - 1, :) - a(m, m + 1:na)*x(m + 1:na,:);
        x(m, :) = tmp/a(m, m);
    end
    x(na, :) = (b(na,:) - a(na,1:na - 1)*x(1:na - 1,:))/a(na, na);
    err = sqrt(x - x0)'*(x - x0);
    if  err < eps
        break;
    end
    x0 = x;
end
if k == kmax
    fprintf('Khong hoi tu sau %d lan lap',kmax);
else
     fprintf('Hoi tu sau %d lan lap',k);
end
```

Để giải phương trình ta chương trình ***ctgausseidel.m***:

```
b = [1 -1]';
a = [3 2;1 2];
x0 = [0 0]';
x = gausseidel(a, b, x0, 20)
```

§9. PHƯƠNG PHÁP LẶP RICHARDSON

Trong các phép lặp nói trên, ma trận [B] không thay đổi. Bây giờ ta xét các phương pháp lặp có [B] thay đổi. Phương pháp đơn giản nhất là phương pháp lặp Richardson. Ta có công thức lặp sau:

$$x^{(k+1)} = x^{(k)} + \alpha P^{-1} r^{(k)}$$

Trong đó α là thông số relaxation và số dư $r^{(k)}$ được tính bằng:

$$r^{(k)} = b - Ax^{(k)}$$

Ma trận lặp lần k là:

$$B_k = E - \alpha_k P^{-1} A$$

Như vậy phép lặp Jacobi cũng như phép lặp Gauss - Seidel là trường hợp riêng của phép lặp Richardson với $\alpha = 1$, P = D hay P = D + L. Người ta đã chứng minh là phép lặp Richardson hội tụ khi:

$$0 < \alpha < \frac{2\mathrm{Re}(\lambda_i)}{|\lambda_i|^2}$$

Ta xây dựng hàm ***richardsoniter()*** thực hiện thuật toán trên:

```
function x =  richardsoniter(a, b, x, maxiter, tol)
d = eig(a);
k = length(d);
alfa1 = abs(2*real(d(1))/(abs(d(1))^2));
for j = 2:k
   alfa = abs(2*real(d(j))/(abs(d(j))^2));
   if alfa < alfa1
      alfa1 = alfa;
   end
end
omega = alfa1/2;
for i = 1:maxiter
   r = b - a*x;
   x = x + omega*r;
   if norm(r) < tol
      break;
   end
end
i
```

Để giải hệ phương trình ta dùng chương trình ***ctrichardsoniter.m***

```
clear all, clc
a = [ 10  1  1;1  10  2; 2  2  10];
b = [12  13  14]';
x = [ 0  0  0]';
maxiter = 50;
tol = 1e-6;
x = richardsoniter(a, b, x, maxiter, tol)
```

§10. PHƯƠNG PHÁP SOR

Giả sử ta dùng phương pháp lặp để giải hệ phương trình tuyến tính [A][X] = [B] và $[X_k]$ là nghiệm gần đúng. Như vậy ta có vec tơ số dư là:

$[R_k] = [B] - [A][X_k]$

Nếu xấp xỉ tốt thì $[R_k] \approx 0$. Một phương pháp dựa trên việc giảm chuẩn của vec tơ dư sẽ tạo ra dãy số $[X_k]$ hội tụ nhanh hơn. Phương pháp SOR(succesive over relaxtion - là phương pháp giải các phương trình trong đó sai số được giảm liên tiếp cho đến khi đạt được sai số mong muốn) đưa vào một tham số ω để tăng tốc độ hội tụ.

Ta khảo sát ma trận [A] bậc n. Ta chia [A] thành 3 phần: phần đường chéo chính [D], phần bên dưới đường chéo chính [L] và phần bên trên đường chéo chính [U].

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix}$$

$$- \begin{bmatrix} 0 & 0 & \cdots & 0 \\ -a_{21} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -a_{n1} & -a_{n2} & \cdots & 0 \end{bmatrix} - \begin{bmatrix} 0 & -a_{12} & \cdots & -a_{1n} \\ 0 & 0 & \cdots & -a_{2n} \\ \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}$$

Khi cho giá trị của tham số ω, thường chọn trong khoảng $0 < \omega < 2$, nghiệm của hệ phương trình tuyến tính, khi cho giá trị ban đầu $[X_0]$ được tính bằng công thức lặp:

$$[X_{k+1}] = M_\omega[X_k] + C_\omega$$

Trong đó:

$$M_\omega = ([D] - \omega[L])^{-1}\{ (1 - \omega)[D] + \omega[U]\}$$

$$C_\omega = \omega ([D] - \omega[L])^{-1}[B]$$

Khai triển các phần tử ta có:

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)} \right)$$

Phương pháp rất hiệu quả khi số phương trình lớn. Nếu ω = 1 ta có phép lặp Gauss - Seidel.

Ta xây dựng hàm ***soriter()*** để thực hiện thuật toán này.

```
function y = soriter(a, b, omega, x0, kmax)
%cu phap y = soriter(a, b, omega, x0, kmax)
% giai he pt bang pp SOR
%vao
%   - a, b la cac  ma tran he so
%   - x0 la nghiem ban dau
%   - kmax so lan lap max
%ra
```

```
%   - x la nghiem
n = size(a,1);
if nargin < 5
   kmax = 100;
end
if nargin < 4
   kmax = 100;
   x0 = zeros(1,n);
end
if nargin < 3
   kmax = 100;
   x0 = zeros(1,n);
   omega = 1;
end
if size(x0, 1) ==1
   x0 = x0';
end
x = x0;
kmax = 100;
for k = 1: kmax
   x(1, :) = (1-omega)*x(1,:) + omega*(b(1, :) - a(1, 2:n)*x(2: n, :))/a(1,1);
   for m = 2:n-1
      tmp = b(m, :) - a(m, 1:m-1)*x(1: m - 1, :) - a(m, m + 1:n)*x(m + 1:n,:);
      x(m, :) = (1 - omega)*x(m,:) + tmp*omega/a(m, m);
   end
   x(n, :) = (1 - omega)*x(n,:) + omega*(b(n,:) - a(n,1:n - 1)*x(1:n - 1,:))/a(n,
n);
   err = sqrt((x - x0)'*(x - x0));
   if err < eps
      break;
   end
   x0 = x;
end
if k == kmax
   fprintf('Khong hoi tu sau %d lan lap',kmax);
else
   fprintf('Hoi tu sau %d lan lap',k);
end
y = x;
```

Để giải hệ phương trình

$$\begin{cases} 10x_1 + x_2 + x_3 = 12 \\ 2x_1 + 10x_2 + x_3 = 13 \\ 2x_1 + 2x_2 + 10x_3 = 14 \end{cases}$$

ta dùng chương trình ***ctsoriter.m***

```
clear all, clc
a = [ 2   1   0   0   0   0   0   0   0;
      1   2   1   0   0   0   0   0   0;
      0   1   2   1   0   0   0   0   0;
      0   0   1   2   1   0   0   0   0;
      0   0   0   1   2   1   0   0   0;
      0   0   0   0   1   2   1   0   0;
      0   0   0   0   0   1   2   1   0;
      0   0   0   0   0   0   1   2   1;
      0   0   0   0   0   0   0   1   2];
b = [1; 2; 3; 4; 5; 4; 3; 2; 1];
x0 = [1  1  1  1  1  1  1  1  1  ];
x = soriter(a, b,1.25, x0, 500)
```

§11. PHƯƠNG PHÁP SSOR

Nếu ma trận hệ số [A] là đối xứng thì phép lặp SSOR kết hợp hai lần tính theo SOR sao cho ma trận kết quả giống với ma trận đối xứng. đặc biệt lần thực hiện SOR đầu tiên là:

$$x_k = ([D] - \omega[L])^{-1}\{\omega[U] + (1 - \omega)[D]\}x_{k-1} + \omega([D] - \omega[L])^{-1}[B]$$

Lần thực hiện SOR thứ hai các ẩn số được cấp nhật theo hướng ngược lại. Như vậy SSOR là lặp SOR thuận và sau đó là SOR ngược.

Dưới dạng ma trận, phép lặp SSOR là:

$$[X_k] = [B_1][B_2][X_{k-1}] + \omega(2 - \omega)([D] - [U])^{-1}[D]([D] - \omega[L])^{-1}[B]$$

Trong đó:

$$[B_1] = ([D] - \omega[U])^{-1}\{\omega[L] - (1 - \omega)[D]\}$$

$$[B_2] = ([D] - \omega[L])^{-1}\{\omega[U] - (1 - \omega)[D]\}$$

$[B_2]$ là ma trận của phép lặp SOR còn $[B_2]$ cũng tương tự nhưng đổi vai trò của [U] và [L]

Ta xây dựng hàm ***ssoriter()*** để thực hiện thuật toán này:

```
function x = ssoriter(a, b, x1, omega, maxiter, tol)
% ham thuc hien thuat toan SSOR
if  size(x1, 1) == 1
   x1 = x1';
end
n = length(a);
```

```
d = zeros(n);
for i = 1:n
   d(i, i) = a(i, i);
end
l = tril(a);
for i = 1:n
   l(i, i) = 0;
end;
u = triu(a);
for i = 1:n
   u(i, i) = 0;
end;
u = -u;
l = - l;
b1 = inv(d - omega*u)*(omega*l + (1 - omega)*d);
b2 = inv(d - omega*l)*(omega*u + (1 - omega)*d);
for k = 1: maxiter
   x = b1*b2*x1 + omega*(2 - omega)*inv(d - omega*u)*d*inv(d -
omega*l)*b;
   if norm(x - x1) <= tol
      break;
   end
   x1 = x;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctssoriter.m***

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
maxiter = 50;
tol = 1e-6;
x1 = [0  0  0]';
omega = 1.2;
x = ssoriter(a, b, x1, omega, maxiter, tol)
```

§12. PHƯƠNG PHÁP ARNOLDI VÀ LANCZOS

Một biến thể của thuật toán Arnoldi là thuật toán do nhà toán học Hungary Lanczos đưa ra. Thuật toán gồm các bước sau:

- cho $[X_0]$
- tính $[R_0] = [B] - [A][X_0]$
- $\beta = \|R_0\|_2$ và $v_1 = \dfrac{[R_0]}{\beta}$

- lặp k = 0, 1, 2,..., maxiter
    - $w = [A]v_j - \beta v_{j-1}$ nếu j = 1 cho $\beta_1 v_0 \equiv 0$
    - $\alpha = w_j^T v_j$
    - $w_j = w_j - \alpha_j v_j$
    - $\beta_{j+1} = \|w_j\|_2$ nếu $\beta_{j+1} = 0$ thoát khỏi vòng lặp
    - $v_{j+1} = \dfrac{w_j}{\beta_{j+1}}$
- $T_m = \text{tridiag}(\beta_j, \alpha_i, \beta_{j+1})$
- $V = [v_1, v_2, ..., v_m]$
- $y = T_m^{-1}(\beta e_1)$
- $x_m = x_0 + V_m y_m$

Ta xây dựng hàm ***lanczos4sys()*** để thực hiện thuật toán trên

```
function x = lanczos4sys(a, b, x0, maxiter, tol)
% hamf giai he phuong trinh bang thuat toan Lanczos
r0 = b - a*x0;
nres0 = norm(r0, 2);
if nres0 ~= 0
  V = r0/nres0;
  Z = V;
  gamma(1) = 0;
  beta(1) = 0;
  k = 1;
  nres = 1;
  while k <= maxiter & nres > tol
    vk = V(:, k);
    zk = Z(:, k);
    if   k == 1,
       vk1 = 0*vk;
       zk1 = 0*zk;
    else
       vk1 = V(:, k-1);
       zk1 = Z(:, k-1);
    end
    alpha(k) = zk'*a*vk;
    tildev = a*vk - alpha(k)*vk - beta(k)*vk1;
    tildez = a'*zk - alpha(k)*zk - gamma(k)*zk1;
    gamma(k+1) = sqrt(abs(tildez'*tildev));
    if gamma(k+1) == 0
```

```
        k = maxiter + 2;
      else
        beta(k+1) = tildez'*tildev/gamma(k+1);
        Z = [Z, tildez/beta(k+1)];
        V = [V,tildev/gamma(k+1)];
      end
      if k ~= maxiter + 2
        if k == 1
          Tk = alpha;
        else
          Tk = diag(alpha) + diag(beta(2:k),1) + diag(gamma(2:k),-1);
        end
        yk = Tk \ (nres0*[1,0*[1:k-1]]');
        xk = x0 + V(:, 1:k)*yk;
        nres = abs(gamma(k+1)*[0*[1:k-1], 1]*yk)*norm(V(:, k+1), 2)/nres0;
        k = k+1;
      end
    end
    else
      x = x0;
    end
    if k == maxiter + 2
       niter = -k;
    else
       niter = k - 1;
    end
```

Để giải hệ phương trình ta dùng chương trình ***ctlanczos.m***

```
clear all, clc
a = [ 1 3 5; 3 2 4; 5 4 6];
b = [ 9  9  15]';
maxiter = 50;
tol = 1e-6;
x0 = [0  0  0]';
x = lanczos4sys(a, b, x0, maxiter, tol)
```

§13. PHƯƠNG PHÁP ĐỘ DỐC LỚN NHẤT

Ta khảo sát bài toán tìm cực trị của hàm

$$f([X]) = 0.5[X]^T[A][X] - [B][X] \qquad (1)$$

với [A] là ma trận đối xứng, xác định dương.

Do f([X]) đạt cực trị khi gradient $\nabla f([X]) = [A][X] - [B] = 0$ nên bài toán tìm cực trị tương đương với việc giải hệ phương trình đại số tuyến tính:

$$[A][X] = [B] \quad (2)$$

Ta biết rằng gradient là hướng hàm tăng nhanh nhất. Như thế muốn đi đến cực tiểu ta cho [X], tính gradient và tìm theo hướng ngược lại cho đến khi hàm không giảm nữa. Phương pháp độ dốc lớn nhất (steepest descent) thực hiện thuật toán lặp, bắt đầu từ $[X_0]$. Tại lần lặp thứ k, nghiệm được hiệu chỉnh bằng:

$$[X_{k+1}] = [X_k] + \alpha_k[R_k] \quad (3)$$

Giá trị của α được xác định bằng:

$$\alpha_k = \frac{[R_k]^T[R_k]}{[R_k]^T[A][R_k]}$$

Như vậy thuật toán steepest descent là:

- cho $[X_0]$
- tính $[R_0] = [B] - [A][X_0]$
- lặp k = 1, 2,...
  - $\alpha_k = \frac{[R_k]^T[R_k]}{[R_k]^T[A][R_k]}$
  - $[X_{k+1}] = [X_k] + \alpha_k[R_k]$
  - $[R_{k+1}] = [B] - [A][X_{k+1}]$

  cho đến khi hội tụ

Thuật toán này có nhược điểm là hội tụ không nhanh.

Ta xây dựng hàm ***steepest()*** để thực hiện thuật toán trên:

```
function x = steepest(a, b, x, maxiter, tol)
% Steepest descent
r = b - a*x;
k = 1;
while k <= maxiter & norm(r)>tol
  ar = a*r;
  alpha = (r'*r)/(r'*ar);
  x = x + alpha*r;
  r = r - alpha*ar;
  k = k + 1;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctsteepest.m***

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
```

x = steepest(a, b, x, maxiter, tol)

§14. PHƯƠNG PHÁP CG

Thuật toán gradient liên hợp được Hestennes và Stiefel trình bày năm 1952. Nó thích hợp để giải các hệ phương trình có ma trận [A] đối xứng, xác định dương. Nó là trường hợp đặc biệt của phương pháp Lanczos khi ma trận hệ số đối xứng, xác định dương.

Phương pháp gradient thực hiện thuật toán lặp, bắt đầu từ $[X_0]$. Tại lần lặp thứ k, nghiệm được hiệu chỉnh bằng:

$$[X_{k+1}] = [X_k] + \alpha_k[S_k] \quad (1)$$

Độ dài của α_k được chọn sao cho $[X_{k+1}]$ cực tiểu $f([X_{k+1}])$ theo hướng tìm $[S_k]$. Như vậy $[X_{k+1}]$ phải thoả mãn:

$$[A]([X_k] + \alpha_k[S_k]) = [B] \quad (2)$$

Số dư của phép lặp là:

$$[R_k] = [B] - [A][X_k] \quad (2)$$

Như vậy (4) trở thành:

$$\alpha_k[A][S_k] = [R_k] \quad (4)$$

Nhân cả hai vế của (4) với $[S_k]^T$ ta có:

$$\alpha_k = \frac{[S_k]^T[R_k]}{[S_k]^T[A][S_k]} \quad (5)$$

Ta chọn $[S_k]$ theo gradient liên hợp:

$$[S_{k+1}] = [R_{k+1}] + \beta_k[S_k] \quad (6)$$

Hằng số β_k sao cho hai hướng tìm liên tiếp liên hợp với nhau, nghĩa là:

$$[S_{k+1}]^T[A][S_k] = 0 \quad (7)$$

Như vậy $\left([R_{k+1}]^T + \beta_k[S_k]\right)[A][S_k] = 0$, nên:

$$\beta_k = -\frac{[R_{k+1}]^T[A][S_k]}{[S_k]^T[A][S_k]} \quad (8)$$

Như vậy thuật toán gradient liên hợp là:

- cho $[X_0]$
- tính $[R_0] = [B] - [A][X_0]$
- $[R_0] = [S_0]$
- lặp k = 0, 1, 2,...
  - $\alpha_k = \dfrac{[S_k]^T[R_k]}{[S_k]^T[A][S_k]}$
  - $[X_{k+1}] = [X_k] + \alpha_k[S_k]$
  - $[R_{k+1}] = [B] - [A][X_{k+1}]$

- $\beta_k = -\dfrac{[R_{k+1}]^T[A][S_k]}{[S_k]^T[A][S_k]}$
- $[S_{k+1}] = [R_{k+1}] + \beta_k[S_k]$

cho đến khi hội tụ

Ta xây dựng hàm ***conjgradient()*** để thực hiện thuật toán trên

```
function  x = conjgradient ( a,  b, x1,  maxiter, tol )
% giai pt AX = B bang pp gradient lien hop
% cu phap
% x = conjgradient ( a,  b, x1, maxiter, tol )
if nargin < 5
   tol = 1e-6;
end
if nargin < 4
   tol = 1e-6;
   maxiter = 50;
end
if size(x1, 1) == 1
   x1 = x1';
end
r1 = b - a*x1;
s1 = r1;
for  k = 1:maxiter
   alfa = (s1'*r1)/(s1'*a*s1);
   x2 = x1 + alfa*s1;
   r2 = b - a*x2;
   if norm(r2) < tol
      break
   end
   beta = -(r2'*a*s1)/(s1'*a*s1);
   s2 = r2 + beta*s1;
   s1 = s2;
   x1 = x2;
end
x = x2
```

Để giải hệ phương trình ta dùng chương trình ***ctconjgradient.m***

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
```

```
maxiter = 50;
tol = 1e-6;
x1 = [0  0  0]';
x = conjgradient(a, b, x1, maxiter, tol);
```

Ta cũng có thể dùng kỹ thuật preconditionning cho phương pháp CG. Thuật toán sẽ là:

- Chọn $[X_0]$
- tính $[R_0] = [B] - [A][X_0]$
- Lặp từ i = 1, 2,..,maxiter
  - $[Z_{i-1}] = [M]^{-1}[R_{i-1}]$
  - $\rho_{i-1} = [R_{i-1}]^T [Z_{i-1}]$
  - Nếu i = 1
    - $[P_1] = [Z_0]$

    không thì:
    - $\beta_{i-1} = \dfrac{\rho_{i-1}}{\rho_{i-2}}$
    - $[P_i] = [Z_{i-1}] + \beta_{i-1}[P_{i-1}]$
  - $[Q_i] = [A][P_i]$
  - $\alpha_i = \dfrac{\rho_{i-1}}{[P_i]^T[Q_i]}$
  - $[X_i] = [X_{i-1}] + \alpha_i[P_i]$
  - $[R_i] = [R_{i-1}] - \alpha_i[Q_i]$

  cho đến khi hội tụ

Ta xây dựng hàm ***pcg()*** để thực hiện thuật toán trên

```
function  y = pcg ( a, b, x, M, maxiter, tol )
% giai pt AX = B bang pp gradient lien hop co preconditionner
% cu phap
% x = conjgradient ( a,  b, x1, M, maxiter, tol )

r = b - a*x;
if nargin < 6
   tol = 1e-6;
end
if nargin < 6
   tol = 1e-6;
   maxiter = 50;
end
```

```
if size(x, 1) == 1
   x = x';
end
for iter = 1 : maxiter
   z = M\r;
   rho = ( r' * z );
   if ( iter  == 1 )
      p = z;
   else
      beta = rho / rho_1;
      p = z + beta * p;
   end
   q = a * p;
   alpha = rho / ( p' * q );
   x = x + alpha * p;
   r = r - alpha*q;
   if  norm(r) < tol
      break;
   end
   rho_1 = rho;
end
y = x;
```

Để giải hệ phương trình ta dùng chương trình ***ctpcg.m***

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
for i = 1:3
   M(i, i) = a(i, i);
end
x = pcg(a, b, x, M, maxiter, tol)
```

§15. PHƯƠNG PHÁP CGNE

Ta xét hệ phương trình

[A][X] = [B]

Khi ma trận [A] không đối xứng và không suy biến thì $[A][A]^T$ sẽ đối xứng và xác định dương nên có thể áp dụng thuật toán CG. Thuật toán CGNE(thuật toán Craig) gồm các bước:

- Chọn $[X_0]$, tính $[R_0] = [B] - [A][X_0]$
- Tính $[S_0] = [A]^T[R_0]$
- $[P_0] = [S_0]$
- Lặp cho đến khi hội tụ
  - $[V_k] = [A][P_k]$
  - $\alpha_k = \dfrac{[S_{k-1}]^T[S_{k-1}]}{[V_{k-1}]^T[V_{k-1}]}$
  - $[X_k] = [X_{k-1}] + \alpha_k[P_{k-1}]$
  - $[R_k] = [R_{k-1}] - \alpha_k[V_k]$
  - $[S_k] = [A]^T[R_k]$
  - $\beta_k = \dfrac{[S_k]^T[S_k]}{[S_{k-1}]^T[S_{k-1}]}$
  - $[P_k] = [S_k] + \beta_k[P_{k-1}]$

Ta xây dựng hàm ***cgne()*** để thực hiện thuật toán trên

```
function  x = cgne(a, b, x0, maxiter, tol) ;
%Ham nay thuc hien thuat toan CGNE
x = x0(:,:);
i = 1;
r = b - a*x;
s = a'*r;
p = s;
delta1 = norm(s)^2;
rnorm = norm(r);
rho = rnorm;
while ((rnorm/rho > tol) & (i < maxiter))
   v = a*p;
   alfa = delta1/norm(v)^2;
   x = x + alfa*p;
   r = r - alfa*v;
   rnorm = norm(r);
   s = a'*r;
   delta2 = norm(s)^2;
   beta = delta2/delta1;
   p = s + beta*p;
   delta1 = delta2;
   i = i + 1;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctcgne.m***

```
clear all, clc
a = [ 1 3 4; 2 5 7; 3 1 2];
b = [8  14  6]';
maxiter = 50;
tol = 1e-6;
x0 = [0  0  0]';
x = cgne(a, b, x0, maxiter, tol)
```

§16. PHƯƠNG PHÁP CGNR

Khi ma trận hệ số [A] không đối xứng ta không dùng được phương pháp CG. Vì vậy ta cần biến đổi hệ phương trình để dùng được phương pháp CG. Xét hệ phương trình:

[A][X] = [B]

Ta đưa hệ về dạng:

$[A]^T[M][A][X] = [A]^T[M][B]$

hay

$$\left[\hat{A}\right]\left[X\right]=\left[\hat{B}\right]$$

với các ma trận $\left[\hat{A}\right]$ và $\left[\hat{B}\right]$ đối xứng nên có thể dùng được phương pháp CG. Thuật toán của phương pháp CGNR là:

- Cho $[X_0]$ tính $\left[r\right]=\left[\hat{B}\right]-\left[\hat{A}\right]\left[X_0\right]$ và v = r
- Lặp cho đến khi hội tụ
  - $\alpha_k = \dfrac{r_k^T r_k}{v_k^T([\hat{A}]v_k)}$
  - $x_{k+1} = x_k - \alpha_k v_k$
  - $r_{k+1} = r_k - \alpha_k\left[\hat{A}\right]v_k$
  - $\beta_k = \dfrac{r_k^T r_k}{r_{k-1}^T r_{k-1}}$
  - $v_{k+1} = r_{k+1} + \beta_k v_k$

Ta xây dựng hàm ***cgnr()*** để thực hiện thuật toán trên

```
function x = cgnr(a, b, x, maxiter, tol)
% dung thuat toan cgnr der giai he phuong trinh
n = size(a,1);
m = ones(n,1);
m = diag(m);
m = 1.2*m;
am = a'*m*a;
```

```
bm = a'*m*b;
r = bm - am*x;
v = r;
delta1 = norm(r)^2;
for k = 1:maxiter
   if norm(r) < tol
      break
   end
   alfa = norm(r)^2/(v'*(am*v));
   x = x + alfa*v;
   r = r - alfa*am*v;
   delta2 = norm(r)^2;
   beta = delta2/delta1;
   v = r + beta*v;
   delta1 = delta2;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctcgnr.m***

```
clear all, clc
a = [ 1 3 4; 2 5 7; 3 1 2];
b = [8  14  6]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
x = cgnr(a, b, x, maxiter, tol)
```

§17. PHƯƠNG PHÁP CGLS

Phương pháp CGLS cũng dùng để giải các hệ phương trình có ma trận hệ số không đối xứng với cách tính sao cho tổng bình phương số dư bé nhất. Do vậy ta phải cho $\|[A][X]-[B]\|_2$ min. Thuật toán CGLS gồm các bước sau:

- Cho $[X_0]$ tính $[r]=[B]-[A][X_0]$, $[d] = [A]^T[r]$ và $\rho = [d]^T[d]$
- Lặp cho đến khi hội tụ

  - $\alpha_k = \dfrac{\rho_{k-1}}{([A][d])^T([A][d])}$
  - $x_{k+1} = x_k - \alpha_k d_k$
  - $r_{k+1} = r_k - \alpha_k [A] d_k$
  - $s_k = [A]^T r_k$
  - $\beta_k = \dfrac{s_k^T s_k}{s_{k-1}^T s_{k-1}}$
  - $d_{k+1} = s_k + \beta_k d_k$

Ta xây dụng hàm ***cgls()*** để thực hiện thuật toán trên.

```
function x = cgls(a, b, x, maxiter, tol)
r = b - a*x;
d = a'*r;
rho1 = d'*d;
for j = 1:maxiter
   ad = a*d;
   alpha = rho1/(ad'*ad);
   x  = x + alpha*d;
   r  = r - alpha*ad;
   s  = a'*r;
   rho2 = s'*s;
   beta = rho2/rho1;
   rho1 = rho2;
   d = s + beta*d;
   if norm(r) < 1e-6
      break;
   end
end
```

Để giải hệ phương trình ta dùng chương trình ***ctcgls.m***

```
clear all, clc
a = [ 1 3 4; 2 5 7; 3 1 2];
b = [8  14  6]';
maxiter = 50;
x = [0  0  0]';
tol = 1e-6;
x = cgls(a, b, x, maxiter, tol)
```

Nếu dùng kỹ thuật preconditionning với ma trận [m] thì thuật toán sẽ là:

- Cho $[X_0]$

tính $[r]=[B]-[A][X_0]$, $[p]=[m]^{-1}[A]^T[r]$, $[s]=[p]$ và $\gamma=\|s\|_2^2$

- Lặp cho đến khi hội tụ
  - $t_k = [s]^{-1}p_k$
  - $q_k = [A]t_k$
  - $\alpha_k = \dfrac{\gamma_k}{\|q_k\|_2^2}$
  - $x_{k+1} = x_k - \alpha_k t_k$
  - $r_{k+1} = r_k - \alpha_k q_k$
  - $s_{k+1} = [m]^{-1}([A]^T r_{k+1})$
  - $\gamma_{k+1} = \dfrac{\gamma_{k+1}}{\gamma_k}$
  - $p_{k+1} = s_{k+1} + \beta_k p_k$

Ta xây dựng hàm ***cglsp()*** để thực hiện thuật toán trên.

```
function x = cglsp(a, b, x, maxiter, tol)
%giai he bg thuat toan CGLS co preconditionning
n = size(a,1);
m = ones(n,1);
m = diag(m);
m = 1.*m;
r = b - a*x;
p = inv(m)*(a'*r);
s = p;
k = 1;
gamma1 = norm(s)^2;
while k <= maxiter & norm(r) > tol
   t = inv(m)*p;
   q = a*t;
   alfa = gamma1/(norm(q)^2);
   x = x + alfa*t;
   r = r - alfa*q;
   s = inv(m)*(a'*r);
   gamma2 = norm(s)^2;
   beta = gamma2/gamma1;
   gamma1 = gamma2;
```

```
    p = s + beta*p;
    k = k + 1;
end
```

§18. PHƯƠNG PHÁP BiCG

Phương pháp gadient liên hợp không thích hợp cho hệ phương trình không đối xứng vì các vec tơ số dư không thể trực giao với một số ít lần lặp. Phương pháp gradient liên hợp kép thay thế dãy vec tơ dư trực giao bằng hai dãy trực giao tương hỗ.

Khi cập nhật số dư ta dùng ma trận $[A]^T$ thay cho ma trận [A]. Như vậy ta có:

$$[R_i] = [R_{i-1}] - \alpha_i [A][P_i] \qquad \left[\tilde{R}_i\right] = \left[\tilde{R}_{i-1}\right] - \alpha_i \left[A\right]^T \left[\tilde{P}_i\right]$$

và hai dãy hướng tìm:

$$[P_i] = [R_{i-1}] + \beta_{i-1} [P_{i-1}] \qquad \left[\tilde{P}_i\right] = \left[\tilde{R}_{i-1}\right] + \beta_{i-1} \left[\tilde{P}_{i-1}\right]$$

Việc chọn:

$$\alpha_i = \frac{\left[\tilde{R}_{i-1}\right]^T \left[R_{i-1}\right]}{\left[\tilde{P}_i\right]^T \left[A\right]\left[P_i\right]} \qquad \beta_i = \frac{\left[\tilde{R}_i\right]^T \left[R_i\right]}{\left[\tilde{R}_{i-1}\right]^T \left[R_{i-1}\right]}$$

bảo đảm quan hệ trực giao kép:

$$\left[\tilde{R}_i\right]^T \left[R_j\right] = \left[\tilde{P}_i\right]^T \left[A\right]\left[P_j\right] = 0 \qquad \text{nếu } i \neq j$$

Ta xây dựng hàm ***biconjgrad()*** để thực hiện thuật toán trên

```
function x = biconjgrad(a, b, x, maxiter, tol)
%ham thuc hien thuat toan gradient lien hop kep
if size(x, 1) ==1
    x = x';
end
r = b - a*x;
rn = r;
for i = 1:maxiter
    z = r;
    zn = rn;
    rho = z'*rn;
    if rho == 0
        error(' Khong dung duoc phuong phap nay!');
        break;
    end
    if i == 1
        p = z;
        pn = zn;
    else
```

```
        beta = rho/rho1;
        p = z + beta*p;
        pn = zn + beta*pn;
    end
    q = a*p;
    qn = a*pn;
    alfa = rho/(pn'*q);
    x = x + alfa*p;
    r = r - alfa*q;
    rn = rn - alfa*qn;
    if norm(r) <= tol
        break
    end
    rho1 = rho;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctbiconjgrad.m***

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
x = biconjgrad(a, b, x, maxiter, tol)
```

§19. PHƯƠNG PHÁP BiCGSTAB

Phương pháp gradient liên hợp kép ổn định được xây dựng để giải các hệ phương trình tuyến tính không đối xứng. Thuật toán gồm có các bước sau:

- cho vec tơ nghiệm ban đầu tính $[X_0]$ ta tính $[R_0] = [B] - [A][X_0]$
- ta chọn $\left[\tilde{R}\right]$. Để đơn giảin ta chọn $\left[\tilde{R}\right] = \left[R_0\right]$
- thực hiện các bước lặp
  - $\rho_{i-1} = \left[\tilde{R}\right]^T \left[R_{i-1}\right]$
  - nếu i = 1 thì $\left[P_1\right] = \left[R_0\right]$
  - $\beta_{i-1} = \dfrac{\rho_{i-1}}{\rho_{i-2}} \dfrac{\alpha_{i-1}}{\omega_{i-1}}$
  - $\left[P_i\right] = \left[R_{i-1}\right] + \beta_{i-1}\left(\left[P_{i-1}\right] + \omega_{i-1}\left[V_{i-1}\right]\right)$
  - nếu có điều kiện trước ta giải hệ $\left[M\right]\left[\hat{P}\right] = \left[P_i\right]$

- $\left[V_i\right]=\left[A\right]\left[\hat{P}\right]$
- $\alpha_i=\dfrac{\rho_{i-1}}{\left[\tilde{R}\right]^T\left[V_i\right]}$
- $[S]=\left[R_{i-1}\right]-\alpha_i\left[V_i\right]$
- kiểm tra chuẩn của [S]. Nếu đủ nhỏ thì $\left[X_i\right]=\left[X_{i-1}\right]+\alpha_i\left[\hat{P}\right]$ và dừng
- giải hệ phương trình $\left[M\right]\left[\hat{S}\right]=\left[S\right]$
- $\left[T\right]=\left[A\right]\left[\hat{S}\right]$
- $\omega_i=\dfrac{[T]^T[S]}{[T]^T[T]}$
- $\left[X_i\right]=\left[X_{i-1}\right]+\alpha_i\left[\hat{P}\right]+\omega_i\left[\hat{S}\right]$
- $\left[R_i\right]=[S]-\omega_i[T]$

cho đến khi hội tụ

Ta xây dựng hàm ***bicgstab()*** để thực hiện thuật toán này

```
function  x  = bicgstab ( a, b, x, maxiter, tol )
% ham dung giai he pt bang pp gradient kep on dinh
if size(x, 1) == 1
   x = x';
end
iter = 0;
r = b - a*x;
err = norm(r);
if ( err < tol )
   fprintf('Hoi tu sau %d lan lap', iter);
   return
end
omega  = 1.0;
rm = r;
for iter = 1 : maxiter
   rho = rm' * r;
   if ( rho == 0.0 )
     fprintf('Khong su dung duoc phuong phap nay');
     break
   end
   if ( 1 < iter )
      beta  = ( rho/rho1 )*( alfa/omega );
      p = r + beta*( p - omega*v );
```

```
    else
        p = r;
    end
    ph = p;
    v = a*ph;
    alfa = rho/( rm'*v );
    s = r - alfa*v;
    if ( norm ( s ) < tol )
        fprintf('Phep lap hoi tu sau %d lan lap', iter);
        x = x + alfa*ph;
        resid = norm(s);
        err = norm(s);
        break;
    end
    sh = s;
    t = a*sh;
    omega = ( t'*s )/( t'*t );
    x = x + alfa*ph + omega*sh;
    r = s - omega*t;
    err = norm(r);
    if (err <= tol)
        fprintf('Phep lap hoi tu sau %d lan lap', iter)
        break
    end
    if ( omega == 0.0 )
      fprintf('Khong dung duoc phuong phap nay');
      break
    end
    rho1 = rho;
end
```

Để giải phương trình ta dùng chương trình ***ctbicgstab.m***

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
x  = bicgstab ( a, b, x, maxiter, tol )
```

§20. PHƯƠNG PHÁP CGS

Phương pháp gradient liên hợp bậc 2 được Sonneveld đưa ra. Nó là một biến thể của phương pháp BiCG. Thuật toán gồm có các bước sau:

- cho vec tơ nghiệm ban đầu tính $[X_0]$ ta tính $[R_0] = [B] - [A][X_0]$
- ta chọn $\left[\tilde{R}\right]$ sao cho $\left[R_0\right]^T\left[\tilde{R}\right] \neq 0$. Để đơn giảin ta chọn $\left[\tilde{R}\right] = \left[R_0\right]$
- thực hiện các bước lặp
  - $\rho_{i-1} = \left[\tilde{R}\right]^T\left[R_{i-1}\right]$
  - nếu i = 1 thì $\left[P_1\right] = \left[U_1\right] = \left[R_0\right]$
  - $\beta_{i-1} = \dfrac{\rho_{i-1}}{\rho_{i-2}}$
  - $\left[U_i\right] = \left[R_{i-1}\right] + \beta_{i-1}\left[Q_{i-1}\right]$
  - $\left[P_i\right] = \left[U_i\right] + \beta_{i-1}\left(\left[Q_{i-1}\right] + \beta_{i-1}\left[P_{i-1}\right]\right)$
  - nếu có điều kiện trước ta giải hệ $[M]\left[\hat{P}\right] = \left[P_i\right]$
  - $\left[\hat{V}\right] = \left[A\right]\left[\hat{P}\right]$
  - $\alpha_i = \dfrac{\rho_{i-1}}{\left[\tilde{R}\right]^T\left[\hat{V}\right]}$
  - $\left[Q_i\right] = \left[U_i\right] - \alpha_i\left[\hat{V}\right]$
  - giải hệ phương trình $\left[M\right]\left[\hat{U}\right] = \left[U_i\right] + \left[Q_i\right]$
  - $\left[X_i\right] = \left[X_{i-1}\right] + \alpha_i\left[\hat{U}\right]$
  - $\left[\hat{Q}\right] = \left[A\right]\left[\hat{U}\right]$
  - $\left[R_i\right] = \left[R_{i-1}\right] - \alpha_i\left[A\right]\left[\hat{Q}\right]$

  cho đến khi hội tụ

Ta xây dựng hàm ***conjgradsq()*** để thực hiện thuật toán trên:

```
function x = conjgradsq(a, b, x, maxiter, tol)
%ham thuc hien thuat toan gradient lien hop bac hai
if size(x, 1) ==1
   x = x';
end
r = b - a*x;
rn = r;
for i = 1:maxiter
   rho = rn'*r;
   if rho == 0
```

```
        error(' Khong dung duoc phuong phap nay!');
        break;
    end
    if i == 1
        u = r;
        p = u;
    else
        beta = rho/rho1;
        u = r + beta*q;
        p = u + beta*(q + beta*p1);
    end
    pm = p;
    vm = a*pm;
    alfa = rho/(rn'*vm);
    q = u - alfa*vm;
    um = u + q;
    x = x + alfa*um;
    qm = a*um;
    r = r - alfa*qm;
    if norm(r) <= tol
        break
    end
    rho1 = rho;
    p1 = p;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctconjgradsq.m***:

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
x = conjgradsq(a, b, x, maxiter, tol)
```

§21. PHƯƠNG PHÁP MINRES

Phương pháp này nhằm cực tiểu hoá số dư [R] = [B] - [A][X]. Phép lặp tìm nghiệm của hệ phương trình cho bởi:

$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$

với $p^{(k)}$ là hướng tìm.

Số dư của phép lặp:

$r^{(k+1)} = r^{(k)} - \alpha_k[A][R^{(k)}] + \alpha_k b_{k-1}[A]p^{(k-1)}$

Các hệ số được chọn để tăng tính trực giao. Nếu [A] đối xứng, ta thấy rằng số dư được cực tiểu hoá và ta có thuật toán MINRES. Thuật toán cụ thể gồm các bước sau:

- Cho $[X_0]$, tính:
  - $[R] = [B] - [A][X_0]$, $\gamma_0 = \|R_0\|$, $v = 0$; $v^{new} = [R_0]/\gamma_0$, $\beta_{new} = 0$
  - $c = 0$, $s = 0$, $c_{new} = 1$, $s_{new} = 0$
  - $p = 0$, $p^{new} = 0$
- Lặp với k = 1, 2,…

  %thuật toán Lanczos tìm T_k
  - $\beta = \beta_{new}$
  - $[v^{old}] = [v]$; $[v] = [v^{new}]$
  - $[v^{new}] = [A][v] - \beta[v^{old}]$
  - $\alpha = [v^{new}]^T[v]$
  - $[v^{new}] = [v^{new}] - \alpha[v]$
  - $\beta_{new} = \left\|\left[v^{new}\right]\right\|$
  - $\left[v^{new}\right] = \dfrac{\left[v^{new}\right]}{\beta^{new}}$

  %dùng phép quay trên cột cuối của T_k
  - $c_{lod} = c$, $s_{old} = s$, $c = c_{new}$, $s = s_{new}$
  - $\rho_1 = s_{lod}\beta$
  - $\rho_2 = c.c_{lod}\beta + s\alpha$
  - $\tilde{\rho}_3 = c\alpha - sc_{old}\beta$

  %loại trừ $T_k(k+1, k)$
  - $\tau = |\tilde{\rho}_3| + |\beta_{new}|$
  - $\nu = \tau\sqrt{\left(\dfrac{\tilde{\rho}_3}{\tau}\right)^2 + \left(\dfrac{\beta_{new}}{\tau}\right)^2}$
  - $c_{new} = \dfrac{\tilde{\rho}_3}{\nu}$
  - $s_{new} = \dfrac{\beta_{new}}{\nu}$
  - $\rho_3 = \nu$

  %Tính P_k
  - $p^{old} = p$, $p = p^{new}$
  - $p^{new} = \dfrac{v - \rho_1 p^{old} - \rho_2 p^{old}}{\rho_3}$

  %tính x
  - $x = x + c_{new}\gamma p^{new}$

% điều kiện ngừng lặp

- $\gamma = -s_{new}\gamma$

cho đến khi $\frac{|\gamma|}{\gamma_0} \le \varepsilon$

Ta xây dựng hàm ***minres()*** để thực hiện thuật toán này.

```
function x = minres(a, b, x, maxiter, tol)
k = 0;
r = b - a*x;
gamma0 = norm(r);
v = 0;
vnew = r/gamma0;
gamma = gamma0;
betanew = 0;
c = 1;
s = 0;
cnew = 1;
snew = 0;
p = 0;
pnew = 0;
for k = 1:maxiter
   beta = betanew;
   vold = v;
   v = vnew;
   vnew = a*v - beta*vold;
   alfa = vnew'*v;
   vnew = vnew - alfa*v;
   betanew = norm(vnew);
   vnew = vnew/betanew;
      cold = c;
   sold = s;
   c = cnew;
   s = snew;
   rho1 = sold*beta;
   rho2 = c*cold*beta + s*alfa;
   rhon3 = c*alfa - s*cold*beta;
      tho = abs(rhon3) + abs(beta);
   nuy = tho*sqrt((rhon3/tho)^2 + (betanew/tho)^2);
   cnew = rhon3/nuy;
   snew = betanew/nuy;
   rho3 = nuy;
   pold = p;
```

```
    p = pnew;
    pnew = (v - rho1*pold - rho2*p)/rho3;
    x = x + cnew*gamma*pnew;
    gamma = -snew*gamma;
    if abs(gamma)/gamma0 < tol
        break;
    end
end
```

Để giải hệ ta dùng chương trình ***ctminres.m***

```
clear all, clc
a  = [1  1  3  5;1  2  2 4;3  2  3  2;5  4  2  4];
b = [10   9   10  15]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0  0]';
x = minres(a, b, x, maxiter, tol)
```

§22. PHƯƠNG PHÁP QMR

Phương pháp gần như cực tiểu hoá số dư (quasi minimal residual - QMR) được Freud và Nachtigal đưa ra. Thuật toán cụ thể của phương pháp gồm các bước:

- Cho x_0, tính $R_0 = B - Ax_0$
- $\tilde{v}^{(1)} = R_0$, giải hệ $M_1 y = \tilde{v}_1$
- $\rho_1 = \|y\|_2$, chọn $\tilde{w}^{(1)}$ ví dụ bằng R_0
- Giải hệ $M_2^T z = \tilde{w}^{(1)}$
- $\gamma_0 = 1$, $\eta_0 = -1$
- Lặp cho đến khi hội tụ
    - nếu $\rho_{(i)} = 0$ hay $\xi_i = 0$ thì không dùng phương pháp này
    - $v^{(i)} = \tilde{v}^{(i)} / \rho_i$, $y = y / \rho_i$
    - $w^{(i)} = \tilde{w}^{(i)} / \xi_i$, $z = z / \xi_i$
    - $\delta_i = z^T y$, nếu $\delta_i = 0$ thì không dùng phương pháp này
    - giải hệ $M_2 \tilde{y} = y$
    - giải hệ $M_1^T \tilde{z} = z$
    - nếu i = 1
        * $p^{(1)} = \tilde{y}$, $q^{(1)} = \tilde{z}$

      không thì
        * $p^{(i)} = \tilde{y} - (\xi_i \delta_i / \varepsilon_{i-1}) p^{(i-1)}$
        * $q^{(i)} = \tilde{z} - (\rho_i \delta_i / \varepsilon_{i-1}) q^{(i-1)}$

- $\tilde{p} = Ap^{(i)}$
- $\varepsilon_i = q^{(i)T}\tilde{p}$, nếu $\varepsilon_i = 0$ thì không dùng phương pháp này
- $\beta_i = \varepsilon_i / \delta_i$, nếu $\beta_i = 0$ thì không dùng phương pháp này

- $\tilde{v}^{(i+1)} = \tilde{p} - \beta_i v^{(i)}$
- giải hệ $M_1 y = \tilde{v}^{(i+1)}$
- $\rho_{i+1} = \|y\|_2$
- $\tilde{w}^{(i+1)} = A^T q^{(i)} - \beta_i w^{(i)}$
- giải hệ $M_2^T z = \tilde{w}^{(i+1)}$
- $\xi_{i+1} = \|z\|_2$
- $\theta_i = \rho_{i+1} / (\gamma_{i-1} |\beta_i|)$
- $\gamma_i = \sqrt{1 + \theta_i^2}$, nếu $\theta_i = 0$ thì không dùng phương pháp này
- $\eta_i = -\eta_{i-1}\rho_i\gamma_i / (\beta_i \gamma_{i-1}^2)$
- nếu i =1 thì
  * $d^{(1)} = \eta_1 p^{(1)}$, $s^{(1)} = \eta_1 \tilde{p}$

  không thì
  * $d^{(i)} = \eta_i p^{(i)} + (\theta_{i-1}\gamma_i)^2 d^{(i-1)}$
  * $s^{(i)} = \eta_i \tilde{p} + (\theta_{i-1}\gamma_i)^2 s^{(i-1)}$
- $x^{(i)} = x^{(i-1)} + d^{(i)}$
- $r^{(i)} = r^{(i-1)} - s^{(i)}$

Ta xây dựng hàm ***qmr()*** để thực hiện thuật toán:

```
function x = qmr( a, x, b, maxiter, tol )
% qmr.m giai he phuong trinh  ax = b theo thuat toan
% QMR co dung preconditioning.
r = b - a*x;
error = norm(r);
if ( error < tol )
   return
end
n = size(a,1);
M = ones(n,1);
M = diag(M);
M = 1.2*M;
[M1,M2] = lu( M );
vn = r;
y = M1 \ vn;
rho = norm(y);
```

```
wn = r;
z = M2'\wn;
xi = norm(z);
gamma =  1.0;
eta = -1.0;
theta =  0.0;
for iter = 1:maxiter,
   if ( rho == 0.0 | xi == 0.0 )
      error('Khong dung duoc phuong phap nay')
      break;
   end
   v = vn/rho;
   y = y/rho;
   w = wn/xi;
   z = z/xi;
   delta = z'*y;
   if ( delta == 0.0 )
      error('Khong dung duoc phuong phap nay')
      break
   end
   yn = M2\y;
   zn = M1\z;
      if ( iter > 1 ),
         p = yn - ( xi*delta/ep )*p;
         q = zn - ( rho*delta/ep )*q;
      else
         p = yn;
         q = zn;
      end
      pn = a*p;
      ep = q'*pn;
      if ( ep == 0.0 )
         error('Khong dung duoc phuong phap nay')
         break
      end
      beta = ep/delta;
      if ( beta == 0.0 )
         error('Khong dung duoc phuong phap nay')
         break
      end
      vn = pn - beta*v;
      y =  M1\vn;
```

```
        rho1 = rho;
        rho = norm( y );
        wn = ( a'*q ) - ( beta*w );
        z =  M2'\wn;
        xi = norm( z );
        gamma1 = gamma;
        theta1 = theta;
        theta = rho / ( gamma1*beta );
        gamma = 1.0 / sqrt( 1.0 + (theta^2) );
        if ( gamma == 0.0 )
           error('Khong dung duoc phuong phap nay')
           break
        end
        eta = -eta*rho1*(gamma^2) / ( beta*(gamma1^2) );
        if ( iter > 1 ),
           d = eta*p + (( theta1*gamma )^2)*d;
           s = eta*pn + (( theta1*gamma )^2)*s;
        else
          d = eta*p;
          s = eta*pn;
        end
        x = x + d;
        r = r - s;
        error = norm(r);
        if ( error <= tol )
           break
        end
    end
```

Để giải hệ phương trình ta dùng chương trình ***ctqmr.m***:

```
clear all, clc
a  = [4 -1  1;-1  4  -2;1  -2  4];
b = [ 12  -1  5]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
x = qmr(a, b, x, maxiter, tol)
```

§23. PHƯƠNG PHÁP GMINRES

Phương pháp này thường dùng để giải hệ phương trình có ma trận hệ số không suy biến, không đối xứng. Phương pháp này mở rộng phương pháp MINRES cho hệ

không đối xứng. Giống như phương pháp MINRES, phương pháp này tạo ra một dãy các vec tơ trực giao có dạng:

$w^{(i)} = Av^{(i)}$

for k = 1,..,i

$w^{(i)} = w^{(i)} - \left(w^{(i)}v^{(k)}\right)v^{(k)}$

end

$v^{(i+1)} = w^{(i)} / \left\|w^{(i)}\right\|$

Các lần lặp theo GMINRES có dạng:

$x^{(i)} = x^{(0)} + y_1v^{(1)} + \cdots + y_iv^{(i)}$

Thuật toán cụ thể gồm các bước sau:

- Cho $x^{(0)}$
- Tính r từ hệ phương trình $Mr = b - Ax^{(0)}$
- Lặp cho đến khi hội tụ
  - $v^{(1)} = r / \|r\|_2$
  - $s = \|r\|_2 e_1$
  - for i = 1,2,..,m
    - ♦ giải hệ $Mw = Av^{(i)}$
    - ♦ for k = 1,..,i
      - ∗ $h_{k,i} = (w, v^{(k)})$
      - ∗ $w = w - h_{k,i}v^{(k)}$

      end
    - ♦ $h_{i+1,i} = \|w\|_2$
    - ♦ $v^{(i+1)} = w/h_{i+1,i}$
    - ♦ dùng biến đổi $J_1,\ldots,J_{i-1}$ cho $(h_1,\ldots,h_{i+1,i})$
    - ♦ cập nhật x, m

  end

Ta xây dựng hàm ***gmres()*** để thực hiện thuật toán trên:

```
function x = gmres( a, b, x, restart, maxiter, tol )
%Giai he phuong trinh bang thuat toan GMINRES
n = size(a, 1);
M = ones(n, 1);
M = diag(M);
M = 1.2*M;
r = M\( b - a*x);
error = norm(r);
if ( error < tol )
   return
end
```

```
[n, n] = size(a);
m = restart;
V(1:n,1:m+1) = zeros(n, m+1);
H(1:m+1, 1:m) = zeros(m+1, m);
cs(1:m) = zeros(m, 1);
sn(1:m) = zeros(m, 1);
e1 = zeros(n, 1);
e1(1) = 1.0;
for iter = 1:maxiter
   r = M\( b - a*x );
   V(:,1) = r/norm( r );
   s = norm(r)*e1;
   for i = 1:m
      w = M\(a*V(:,i));
      for k = 1:i
         H(k, i)= w'*V(:, k);
         w = w - H(k, i)*V(:,k);
      end
      H(i+1, i) = norm(w);
      V(:, i+1) = w / H(i+1, i);
      for k = 1:i-1
         temp     =  cs(k)*H(k, i) + sn(k)*H(k+1, i);
         H(k+1, i) = -sn(k)*H(k, i) + cs(k)*H(k+1, i);
         H(k, i)   = temp;
      end
      [cs(i), sn(i)] = rotmat(H(i, i), H(i+1, i));
      temp   = cs(i)*s(i);
      s(i+1) = -sn(i)*s(i);
      s(i)   = temp;
      H(i,i) = cs(i)*H(i, i) + sn(i)*H(i+1, i);
      H(i+1,i) = 0.0;
      error  = abs(s(i+1));
      if ( error <= tol )
         y = H(1:i, 1:i) \ s(1:i);
         x = x + V(:, 1:i)*y;
         break;
      end
   end
   if ( error <= tol )
      break
   end
   y = H(1:m, 1:m) \ s(1:m);
```

```
    x = x + V(:, 1:m)*y;
    r = M \ ( b - a*x );
    s(i+1) = norm(r);
    error = s(i+1) / bnrm2;
    if ( error <= tol )
       break
    end;
end
```

Để giải hệ phương trình ta dùng chương trình ***ctgmres.m***:

```
clear all, clc
a = [ 1 3 4; 2 5 7; 3 1 2];
b = [8  14  6]';
maxiter = 50;
tol = 1e-6;
x = [0  0  0]';
restart = 5;
x = gmres(a, b, x, restart, maxiter, tol)
```

§24. PHƯƠNG PHÁP FOM

Full Orthogonalisation Method (FOM) là phương pháp trực giao hoá ma trận hệ số [A]. Ta xét hệ phương trình [A][X] = [B] với ma trận [A] không suy biến. Đặt $a_{i,n+1} = - b_i$ và $a_j = (a_{i1}, .., a_{in}, a_{i,\,n+1})$ ta sẽ đi đến hệ:

$$\sum_{j=1}^{n} a_{i,j}x_j + a_{i,n+1} = 0 \qquad (1)$$

Hệ (n + 1) vec tơ gồm $\{a_i\}_{i=1}^{n}$ và a_{n+1} độc lập tuyến tính. Ta áp dụng quá trình trực giao hoá cho dãy $\{a_i\}_{i=1}^{n+1}$ bằng cách đặt $u_1 = a_1$, $v_1 = u_1/\|u_1\|$. Nói chung $u_k = \sum_{i=1}^{k-1} c_{k,i}v_i$ và $v_k = u_k/\|u_k\|$. Công thức tính toán sẽ là:

$$\begin{cases} u_k = a_k - \sum_{i=1}^{k}(a_k, v_i)v_i \\ v_k = u_k/\|u_k\|;\ v_1 = a_1/\|a_1\| \end{cases} \qquad (2)$$

Giả sử vec tơ u_{n+1} có các thành phần $(z_1, z_2,\ldots, z_{n+1})$. Nếu $z_{n+1} = 0$ thì từ điều kiện u_{n+1} trực giao với a_i ta có:

$$\sum_{j=1}^{n} a_{i,j}z_j = 0$$

Do $\|u_{n+1}\|^2 = \sum_{i=1}^{n+1} z_i^2 = \sum_{i=1}^{n} z_i^2 > 0$

Nên phương trình $\sum_{j=1}^{n} a_{i,j} z_j = 0$ có nghiệm không tầm thường. Điều này mâu thuẫn với điều kiện det(A) ≠ 0. Như vậy $z_{n+1} \neq 0$. Từ điều kiện u_{n+1} trực giao với a_i ta có:

$$(u_{n+1}, a_i) = \sum_{j=1}^{n} a_{i,j} z_j + a_{i,n+1} z_{n+1} = 0$$

Chia hai vế cho z_{n+1} ta được

$$\sum_{j=1}^{n} a_{i,j} \left(\frac{z_j}{z_{n+1}} \right) + a_{i,n+1} = 0$$

Đẳng thức này chứng tỏ $x_i = \frac{z_i}{z_{n+1}}$ là nghiệm của (1). Thuật toán FOM cụ thể gồm các bước:

- Cho $[X_0]$, tính $r_0 = [B]$, $\beta = \|r_0\|$, $v_1 = r_0 / \|r_0\|$
- Lặp cho đến khi hội tụ
  - $w_j = [A]v_j$
  - trực giao hoá Gram - Schmidt
  - $h_{j+1,j} = \|w_j\|$
  - nếu $h_{j+1,j} = 0$ thì m = j, kết thúc lặp
- $y_m = H_m^{-1}(\beta e_1)$
- $x_m = x_0 + V_m y_m$

Ta xây dựng hàm ***fom()*** để thực hiện thuật toán trên:

```
function x = fom(a, b, x0, maxiter, tol)
%Giai he pt bang thuat toan FOM
i = 1;
x = x0(:);
r = b - a*x;
rnorm = norm(r);
rho = rnorm;
v (:,i) = r/rho;
while ((rnorm/rho > tol) & (i <= maxiter))
   v(:, i+1) = a*v(:, i);
   h(1:i, i) = v(:, 1:i)'* v(:, i+1);
   v(:, i+1) = v(:, i+1) - v(:, 1:i)*h(1:i, i);
   h(i+1, i) = norm(v(:, i+1));
    v(:, i+1) = v(:, i+1)/h(i+1, i);
```

```
    x = x0 + v(:, 1:i)*(h(1:i, 1:i)\[rho; zeros(i-1,1)]);
    r = b - a*x;
    rnorm = norm(r);
    i = i + 1;
end
i
```

Để giải hệ phương trình ta dùng chương trình ***ctfom.m***:

```
clear all, clc
a = [ 1 3 4; 2 5 7; 3 1 2];
b = [8  14  6]';
maxiter = 50;
tol = 1e-6;
x0 = [0  0  0]';
x = fom(a, b, x0, maxiter, tol)
```

§25. PHƯƠNG PHÁP LSQR

Phương pháp LSQR - Least Squares QR do Paige và Saunder đưa ra vào năm 1982. Phương pháp LSQR tương đương với phương pháp CGLS nhưng cho kết quả tốt hơn đối với các hệ phương trình có ma trận hệ số có điều kiện

xấu. Trước hết ta cần chú ý là bài toán bình phương bé nhất [A][X] = [B] tương đương với hệ phương trình tuyến tính dạng:

$$\begin{pmatrix} [E] & [A] \\ [A]^T & [0] \end{pmatrix}\begin{pmatrix} [R] \\ [X] \end{pmatrix} = \begin{pmatrix} [B] \\ [0] \end{pmatrix} \qquad (1)$$

Tạo ra một cơ sở trực giao với (1) với vec tơ ban đầu:

$$w_1 = \frac{1}{\|[B]\|_2}\begin{pmatrix} [B] \\ [0] \end{pmatrix}$$

ta có vec tơ thứ hai $\frac{1}{\|B\|_2}\begin{pmatrix} [B] \\ [A]^T[B] \end{pmatrix}$. Sau khi trực giao hoá nó với w_1 và chuẩn hoá kết quả ta có vec tơ cơ sở trực giao thứ hai:

$$w_2 = \frac{1}{\|[A]^T[B]\|_2}\begin{pmatrix} [B] \\ [A]^T[B] \end{pmatrix}$$

và tiếp tục.

Thuật toán LSQR để giải hệ phương trình Ax = b gồm các bước sau:

- Cho x_0, tính $\beta_1 = \|b\|$, $u_1 = \|b\|_2/\beta_1$, $v = A^T u_1$, $\alpha = \|v\|_2$, $w_1 = v_1 = v/\alpha$
- $\tilde{\phi}_1 = \beta_1$, $\tilde{\rho}_1 = \alpha_1$
- Lặp cho đến khi hội tụ:
  - $u = Av_i - \alpha_i u_i$, $\beta_1 = \|u\|_2$, $u_{i+1} = u/\beta_{i+1}$
  - $v = A^T u_{i+1} - \beta_{i+1} v_i$, $\alpha_{i+1} = \|v\|_2$, $v_{i+1} = v/\beta_{i+1}$
  - $\rho_i = \sqrt{\tilde{\rho}_i^2 + \beta_{i+1}^2}$
  - $c_i = \tilde{\rho}_i/\rho_i$
  - $s_i = \beta_{i+1}/\rho_i$
  - $\theta_{i+1} = s_i \alpha_{i+1}$
  - $\tilde{\rho}_{i+1} = -c_i \alpha_{i+1}$
  - $\phi_i = c_i \tilde{\phi}_i$
  - $\tilde{\phi}_{i+1} = s_i \tilde{\phi}_i$
  - $x_i = x_{i-1} + (\phi_i/\rho_i) w_i$
  - $w_{i+1} = v_{i+1} - (\theta_{i+1}/\rho_i) w_i$

Ta xây dựng hàm ***lsqr()*** thực hiện thuật toán trên:

```
function x = lsqr(A, b, maxiter)
%Giai he phuong trinh bang phuong phap LSQR.
%    min   || A x - b || .
s = 1;
```

```
tol = 1e-6;
[m,n] = size(A);
X = zeros(n, maxiter);
UV = 0;
eta = zeros(maxiter, 1);
rho = eta;
c2 = -1;
s2 = 0;
xnorm = 0;
z = 0;
% Chuan bi lap LSQR .
v = zeros(n, 1);
x = v;
beta = norm(b);
if (beta==0)
   error('Ve phai phai khac khong')
end
u = b/beta;
if (UV)
   U(:, 1) = u;
end
r = A'*u;
alpha = norm(r);
v = r/alpha;
if (UV)
   V(:, 1) = v;
end
phi_bar = beta;
rho_bar = alpha;
w = v;
for i = 2:maxiter+1
   alpha_old = alpha;
   beta_old = beta;
   % Tinh A*v - alpha*u.
   p = A*v - alpha*u;
   beta = norm(p);
   u = p/beta;
   % Tinh A'*u - beta*v.
   r = A'*u - beta*v;
   alpha = norm(r);
   v = r/alpha;
   % Luu U va V neu can
```

```
    if (UV)
        U(:,i) = u;
        V(:,i) = v;
    end
  rrho = pythag(rho_bar, beta);
  c1 = rho_bar/rrho;
  s1 = beta/rrho;
  theta = s1*alpha;
  rho_bar = -c1*alpha;
  phi = c1*phi_bar;
  phi_bar = s1*phi_bar;
  % Tinh chuan cua nghien va so du;
  delta = s2*rrho;
  gamma_bar = -c2*rrho;
  rhs = phi - delta*z;
  z_bar = rhs/gamma_bar;
  eta(i-1) = pythag(xnorm,z_bar);
  gamma = pythag(gamma_bar,theta);
  c2 = gamma_bar/gamma;
  s2 = theta/gamma;
  z = rhs/gamma;
  xnorm = pythag(xnorm,z);
  rho(i-1) = abs(phi_bar);
  % Cap nhat nghiem
  x = x + (phi/rrho)*w;
  w = v - (theta/rrho)*w;
  if  rho(i-1) < tol
      break;
  end
end
i

function x = pythag(y,z)
%tinh sqrt( y^2 + z^2 ).
rmax = max(abs([y;z]));
if (rmax == 0)
  x = 0;
else
  x = rmax*sqrt((y/rmax)^2 + (z/rmax)^2);
end
```

Để giải hệ phương trình ta dùng chương trình ***ctlsqr.m***:

```
clear all, clc
maxiter = 50;
A = [ 1 3 4; 2 5 7; 3 1 2];
b = [8  14  6]';
x = lsqr(A, b, maxiter)
```

§26. PHƯƠNG PHÁP SYMMLQ

Liên quan đến phương pháp MINRES và CG là thuật toán SYMMLQ do Paige và Saunders đưa ra. Ta xét hệ phương trình [A][X] = [B] với [A] là ma trận đối xứng nhưng không cần xác định dương. Ta chọn nghiệm ban đầu là $\beta_1[v_1]$ = {B], $\beta_1 = \|[B]\|_2$. Tại lần lặp thứ k của phương pháp CG ta có được x_k sao cho $[r_k]$ = [B] - $[A][X_k]$ trực giao. Do $[V_k]$ là cơ sở trực giao nên ta có thể đặt $[X_k] = [V_k][y_k]$ và có:

$$[r_k] = [B] - [A][V_k][y_k] = \beta_1[v_1] - [V_k][T_k][y_k] - \beta_{k+1}([e]^T[y_k])[v_{k+1}] \quad (1)$$

Do $[V_k]^T[r_k] = 0$ nên nhân (1) với $[V_k]^T$ và dùng $[V_k]^T[v_{k+1}] = 0$ và $[V_k]^T v_1 = e_1$ ta có:

$$0 = [V_k]^T[r_k] = \beta_1 e_1 - [T_k][y_k] \quad (2)$$

để giải hệ (2), Paige và Saunders đề nghị thực hiện phép phân tích LQ:

$$[T_k]^T = [L_k][Q_k] \qquad [Q_k]^T[Q_k] = [E]$$

với $[L_k]$ là ma trận tam giác và $[Q_k]$ là ma trận trực giao. Thuật toán SYMMLQ gồm các bước sau:

- Cho x_0
- Tính $x = x_o$, $r = b - Ax$, $\rho = \|r\|$, $v = r/\rho$
- $\beta = 0$, $\tilde{\beta} = 0$, $c = -1$, $s = 0$, $k = \rho$
- $v_{old} = 0$, $w = v$, $g = 0$, $\tilde{g} = 0$
- Lặp khi k < tol:
  - $\tilde{v} = Av - \beta v_{old}$
  - $\alpha = v * \tilde{v}$, $\tilde{v} = \tilde{v} - \alpha v$
  - $\beta = \|\tilde{v}\|$, $v_{old} = v$, $v = \tilde{v}/\beta$
  - $l_1 = s\alpha - c\tilde{\beta}$, $l_2 = s\beta$
  - $\tilde{\alpha} = -s\tilde{\beta} - c\alpha$, $\tilde{\beta} = c\beta$
  - $l_0 = \sqrt{\tilde{\alpha}^2 + \beta^2}$
  - $c = \tilde{\alpha}/l_0$, $s = \beta/l_0$
  - $\tilde{g} = \hat{g} - l_1 g$, $\hat{g} = -l_2 g$, $g = \tilde{g}/l_0$
  - $x = x + (gc)w + (gs)v$
  - $w = sw - cv$

- $k = \sqrt{\tilde{g}^2 + \hat{g}^2}$

Ta xây dựng hàm ***symmlq()*** để thực hiện thuật toán này:

```
function x = symmlq(A, b, x, maxiter, tol)
%Ham thuc hien thua toan SYMMLQ voi A la ma tran doi xung
[m,n] = size(A);
n2b = norm(b);
xmin = x;
imin = 0;
tolb = tol * n2b;
r = b - A * x;
normr = norm(r);
normrmin = normr;
v = r;
vold = r;
u = vold;
v = u;
beta1 = vold' * v;
beta1 = sqrt(beta1);
vv = v / beta1;
wbar = vv;
v = A * vv;
alpha = vv' * v;
v = v - (alpha/beta1) * vold;
numer = vv' * v;
denom = vv' * vv;
v = v - (numer/denom) * vv;
volder = vold;
vold = v;
u = vold;
v = u;
betaold = beta1;
beta = vold' * v;
beta = sqrt(beta);
gammabar = alpha;
deltabar = beta;
gamma = sqrt(gammabar^2 + beta^2);
cs = gammabar / gamma;
sn = beta / gamma;
zeta = beta1 / gamma;
epsilonzeta = 0;
```

```
normrcgcs = abs(beta1 * sn);
if (cs == 0)
    normrcg = Inf;
else
    normrcg = normrcgcs / abs(cs);
end
stag = 0;
for i = 1 : maxiter
   vv = v / beta;
   w = cs * wbar + sn * vv;
   stagtest = zeros(n, 1);
   ind = (x ~= 0);
   stagtest(ind) = w(ind) ./ x(ind);
   stagtest(~ind & (w ~= 0)) = Inf;
   if (zeta == 0) | (abs(zeta)*norm(stagtest, inf) < eps)
      stag = stag + 1;
   else
      stag = 0;
   end
   x = x + zeta * w;
   wbar = sn * wbar - cs * vv;
   v = A * vv;
   v = v - (beta / betaold) * volder;
   alpha = vv' * v;
   v = v - (alpha / beta) * vold;
   volder = vold;
   vold = v;
   u = vold;
   v = u;
   betaold = beta;
   beta = vold' * v;
   if (beta < 0)
     break
   end
   beta = sqrt(beta);
   delta = cs * deltabar + sn * alpha;
   deltazeta = - delta * zeta;
   gammabar = sn * deltabar - cs * alpha;
   epsilon = sn * beta;
   deltabar = - cs * beta;
   gamma = sqrt(gammabar^2 + beta^2);
   csold = cs;
```

```
  snzeta = sn * zeta;
  cs = gammabar / gamma;
  sn = beta / gamma;
  epszdelz = epsilonzeta + deltazeta;
  epsilonzeta = - epsilon * zeta;
  zeta = epszdelz / gamma;
  mrcg = norm((csold*epszdelz/gammabar - snzeta)*vold);
  normr = sqrt(epszdelz^2 + epsilonzeta^2);
  normrcgcs = normrcgcs * abs(sn);
  if (cs == 0)
     normrcg = Inf;
  else
     normrcg = normrcgcs / abs(cs);
  end
  if (normr <= tolb)
     r = b - A * x;
     normr = norm(r);
     if (normr <= tolb)
        break
     end
  end
  if (normrcg <= tolb)
     xcg = x + (epszdelz/gammabar) * wbar;
     r = b - A * xcg;
     normrcg = norm(r);
     if (normrcg <= tolb)
       x = xcg;
       break
    end
  end
  if (stag >= 2)
    break
  end
  if (normr < normrmin)
    normrmin = normr;
    xmin = x;
    imin = i;
  end
end
r = b - A * x;
normr = norm(r);
i
```

Để giải hệ phương trình bằng thuật toán SYMMLQ ta dùng chương trình ***ctsymmlq.m***:

```
clear all, clc
A = [ 1  2  4  1;2  3  1  5;4  1  1  6;1  5  6  5];
b = [ 8  11  12  17]';
maxiter = 50;
x = [0  0  0]';
tol = 1e-12;
x = symmlq(A, b, x, maxiter, tol)
```

§27. PHƯƠNG PHÁP CHEBYSHEV

Tính hội tụ của phương pháp lặp phụ thuộc vào tính chất của phổ - nghĩa là của các giá trị riêng - của ma trận [A]. Để cải thiện tính chất này người ta thường biến đổi hệ phương trình tuyến tính bằng một phép biến đổi tuyến tính thích hợp. Quá trình này được gọi là thử trước(preconditioner). Ví dụ nếu ma trận [M] xấp xỉ ma trận hệ số [A] theo một cách nào đó, hệ được biến đổi

$[M]^{-1}[A][X] = [M]^{-1}[B]$

sẽ có nghiệm như hệ phương trình [A][X] = [B] nhưng tính chất phổ của hệ số của ma trận $[M]^{-1}[A]$ có thể thuận lợi hơn.

Ta xét phương trình với [A] là ma trận đối xứng, xác định dương. Lúc đó phổ của ma trận [A] sẽ nằm trong đoạn $[\lambda_{min}, \lambda_{max}]$ với λ_{min}, λ_{max} là các giá trị riêng lớn nhất và nhỏ nhất của $[M]^{-1}[A]$. Thuật toán tìm nghiệm là:

- cho $[X_0]$, tính $[R_0] = [B] - [A][X_0]$
- chọn tham số α và c sao cho phổ của [A] nằm trên đoạn [d - c, d + c] hay trong ellip có tiêu điểm $d \pm c$ không chứa gốc toạ độ

và tính với n = 1, 2,..., n cho đến khi hội tụ:

$[Z] = [M]^{-1}[R]$

$\alpha = \dfrac{2}{d}$ $\qquad [P] = [Z] \qquad$ khi n = 1

$\beta_n = \left(\dfrac{c\alpha_{n-1}}{2}\right)^2 \qquad \alpha_n = \dfrac{1}{d-\beta_n} \qquad [P_n] = [Z_n] + \beta[P_{n-1}] \qquad$ khi $n \geq 2$

$[X_{n+1}] = [X_n] + \alpha_n[P_n]$

$[R_{n+1}] = [R_n] - \alpha_n[A][P_n]$

Ta xây dựng hàm ***chebyiter()*** để thực hiện thuật toán trên:

```
function  x = chebyiter ( A, x, b, M, maxiter, tol )
%  Cu phap  x = chebyiter ( A, x, b, M, maxiter, tol )
%  Dung pp lap Chebyshev de giai he pt  A*x = b.
%  A(n, n) ma tran doi xung, xac dinh duong
```

```
%  X(n), vec to nghiem ban dau
%  B(n), ve phai
%  M, ma tran preconditioner
%  cho M bang mt don vi neu khong thu truoc
%  X(n), nghiem
if size(x, 1) == 1
   x = x';
end
r = b - A * x;
eigs = eig ( inv ( M ) * A );
eigmax = max ( eigs );
eigmin = min ( eigs );
c = ( eigmax - eigmin ) / 2.0;
d = ( eigmax + eigmin ) / 2.0;
for i = 1 : maxiter
   z =  M \ r;
   if ( i == 1 )
      p = z;
      alfa  = 2.0 / d;
   else
      beta = ( c * alfa / 2.0 )^2;
      alfa = 1.0 / ( d - beta );
      p = z + beta * p;
   end
   x  = x + alfa * p;
   r = r - alfa * A * p;
   err  = norm ( r );
   if ( err <= tol  )
      break
   end
end
```

Ta dùng chương trình ***ctchebyiter.m*** để giải hệ phương trình:

```
clear all, clc;
n = 10;
A = zeros ( n, n );
for i = 1 : n
   A(i, i) = 3.0;
end
for i = 1 : n-1
  A(i, i + 1) = -1;
```

```
end
for i = 1 : n-1
  A(i + 1, i) = -1;
end
x = [1:n ]';
b = A * x;
x = ones ( n, 1 );
M = 2.0 * eye ( n );
maxiter = 50;
tol = 1e-6;
y = chebyiter ( A, x, b, M, maxiter, tol );
fprintf('Nghiem cua he phuong trinh\n');
fprintf('    %f\n', y)
```

§28. PHƯƠNG PHÁP QR

Ta phân tích ma trận hệ số [A] thành:

$[A] = [Q][R]$

Do $[Q]^T[Q] = [E]$

nên:

$[A][X] = [Q][R][X] = [B]$

$[Q]^T[A][X] = [Q]^T[Q][R][X] = [R][X] = [Q]^T[B]$

Do [R] là ma trận tam giác trên nên ta tìm nghiệm dễ dàng. Ta xây dựng hàm ***givens()*** để thực hiện phép quay Givens:

```
function [c, s, r] = givens(x, y);
%  tinh c, s, r sao cho  [c  s] [x] = [r]
%  [-s c] [y] =  [0]
%  voi c*c + s*s = 1;
if (y == 0)
   c = 1;
   s = 0;
   r = x;
else
   if (abs(x) >= abs(y))
      t = y/x;
      r = sqrt(1 + t*t);
      c = 1/r;
      s = t*c;
      r = x*r;
   else
      t = x/y;
      r = sqrt(1 + t*t);
```

```
        s = 1/r;
        c = t*s;
        r = y*r;
    end
end
```

Tiếp theo ta xây dựng hàm ***qrgivens()*** thực hiện việc tìm nghiệm của hệ phương trình bằng thuật toán phân tích QR nhờ phép quay Givens:

```
function x = qrgivens(A, b);
[m, n] = size(A);
tau = zeros(n, 1);
%R = [A(1:n+1, :) b(1:n+1)];
R = [A(1:n, :) b(1:n)];
for j = 2:n
    for i = 1:j-1
        [c, s, r] = givens(R(i, i), R(j, i));
        R(i, i) = r;
        R(j, i) = 0;
        t = c*R(i, i+1:n+1) + s*R(j, i+1:n+1);
        R(j, i+1:n+1) = -s*R(i, i+1:n+1) + c*R(j, i+1:n+1);
        R(i, i+1:n+1) = t;
    end
end
for k = n+2:m,
    a = [A(k, :) b(k)];
    for i = 1:n+1
        [c, s, r] = givens(R(i, i),a(i));
        R(i,i) = r;
        a(i) = 0;
        t = c*R(i, i+1:n+1) + s*a(i+1:n+1);
        a(i+1:n+1) = -s*R(i, i+1:n+1) + c*a(i+1:n+1);
        R(i, i+1:n+1) = t;
    end
end
x = R(1:n, n+1);
   for j = n:-1:2
       x(j) = x(j)/R(j, j);
       x(1:j-1) = x(1:j-1) - R(1:j-1, j)*x(j);
   end
   x(1) = x(1)/R(1, 1);
```

Để giải hệ phương trình ta dùng chương trình ***ctqrgivens.m***:

```
clear all, clc
A = [1 2 -1;2 1 1; 1 1 3];
b = [2 4 5]';
x = qrgivens(A, b)
```

CHƯƠNG 3: NỘI SUY VÀ XẤP XỈ HÀM

§1. NỘI SUY LAGRANGE

Trong thực tế nhiều khi ta cần tính giá trị của hàm y = f(x) tại một giá trị x trong một đoạn [a, b] nào đó mà chỉ biết một số nhất định các giá trị của hàm tại một số điểm cho trước. Các giá trị này được cung cấp qua thực nghiệm hay tính toán. Vì vậy nảy sinh vấn đề toán học là trên đoạn $a \le x \le b$ cho một loạt các điểm x_i (i = 0, 1, 2...) và tại các điểm x_i này giá trị của hàm là $y_i = f(x_i)$ đã biết và ta cần tìm y = f(x) dựa trên các giá trị đã biết đó. Lúc đó ta cần tìm đa thức :

$$P_n(x) = a_o x_n + a_1 x^{n-1} + \ldots + a_{n-1}x + a_n$$

sao cho $P_n(x_i) = f(x_i) = y_i$. Đa thức $P_n(x)$ được gọi là đa thức nội suy của hàm y=f(x). Ta chọn đa thức để nội suy hàm y = f(x) vì đa thức là loại hàm đơn giản, luôn có đạo hàm và nguyên hàm. Việc tính giá trị của nó theo thuật toán Horner cũng đơn giản.

Bây giờ ta xây dựng đa thức nội suy kiểu Lagrange. Gọi L_i là đa thức:

$$L_i = \frac{(x-x_0)\ldots(x-x_{i-1})(x-x_{i+1})\ldots(x-x_n)}{(x_i-x_0)\ldots(x_i-x_{i-1})(x_i-x_{i+1})\ldots(x_i-x_n)}$$

Rõ ràng là $L_i(x)$ là một đa thức bậc n và :

$$L_i(x_j) = \begin{cases} 1 & j = i \\ 0 & j \ne i \end{cases}$$

Ta gọi đa thức này là đa thức Lagrange cơ bản.

Bây giờ ta xét biểu thức :

$$P_n(x) = \sum_{i=0}^{n} f(x_i)L_i(x)$$

Ta thấy $P_n(x)$ là một đa thức bậc n vì các $L_i(x)$ là các đa thức bậc n và thoả mãn điều kiện $P_n(x_i) = f(x_i) = y_i$. Ta gọi nó là đa thức nội suy Lagrange.

Với n = 1 ta có bảng

x	x_0	x_1
y	y_0	y_1

Đa thức nội suy sẽ là :

$$P_1(x) = y_o L_0(x) + y_1 L_1(x_1)$$

$$L_0 = \frac{x-x_1}{x_0-x_1} \qquad L_1 = \frac{x-x_0}{x_1-x_0}$$

nên $$P_1(x) = y_0\frac{x-x_1}{x_0-x_1} + y_1\frac{x-x_0}{x_1-x_0}$$

Như vậy $P_1(x)$ là một đa thức bậc nhất đối với x

Với n = 2 ta có bảng

x	x_0	x_1	x_2

y	y_0	y_1	y_2

Đa thức nội suy sẽ là :

$$P_2(x) = y_oL_0(x) + y_1L_1(x_1) + y_2L_2(x_2)$$

$$L_0 = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}$$

$$L_1 = \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}$$

$$L_2 = \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}$$

Như vậy $P_1(x)$ là một đa thức bậc hai đối với x.

Ta xây dựng hàm ***lagrange()*** để thực hiện việc nội suy hàm theo thuật toán Lagrange:

```
function [l, L] = lagrange(x, y)
%Dua vao : x = [x0 x1 ... xn], y = [y0 y1 ... yn]
%ket qua: l = He so cua da thuc Lagrange bac n
% L = Da thuc Lagrange
n = length(x) - 1; %bac cua da thucl
l = 0;
for m = 1:n + 1
   p = 1;
   for k = 1:n + 1
      if k ~= m
         p = conv(p, [1 -x(k)])/(x(m) - x(k));
      end
   end
   L(m, :) = p; %da thuc Lagrange
   l = l + y(m)*p;
end
```

Cho hàm dưới dạng bảng:

x	-2	-1	1	2
y	-6	0	0	6

và tìm y(2.5) ta dùng chương trình ***ctlagrange.m***:

```
clear all, clc
x = [-2 -1 1 2];
y = [-6 0 0 6];
l = lagrange(x, y);
yx = polyval(l, 2.5)
```

§2. NỘI SUY NEWTON

Bây giờ ta xét một cách khác để xây dựng đa thức nội suy gọi là phương pháp Newton. Trước hết ta đưa vào một khái niệm mới là tỉ hiệu

Giả sử hàm y = y(x) có giá trị cho trong bảng sau:

x	x_0	x_1	x_2	…	x_{n-1}	x_n
y	y_0	y_1	y_2	…	y_{n-1}	y_n

Tỉ hiệu cấp 1 của y tại x_i, x_j là :

$$y[x_i, x_j] = \frac{y_i - y_j}{x_i - x_j}$$

Tỉ hiệu cấp hai của y tại x_i, x_j, x_k là :

$$y[x_i, x_j, x_k] = \frac{y[x_i, x_j] - y[x_j, x_k]}{x_i - x_k}$$

v.v.

Với y(x) = $P_n(x)$ là một đa thức bậc n thì tỉ hiệu cấp 1 tại x, x_0 :

$$P_n[x, x_0] = \frac{P_n(x) - P_n(x_0)}{x - x_0}$$

là một đa thức bậc (n - 1). Tỉ hiệu cấp 2 tại x, x_0, x_1 :

$$P_n[x, x_0, x_1] = \frac{P_n[x, x_0] - P_n[x_0, x_1]}{x - x_1}$$

là một đa thức bậc (n-2) v.v và tới tỉ hiệu cấp (n + 1) thì :

$P_n[x, x_o,.., x_n] = 0$

Từ các định nghĩa tỉ hiệu ta suy ra :

$P_n(x) = P_n(x_0) + (x - x_0)P_n[x, x_o]$

$P_n[x, x_0] = {}_{Pn}[x_0, x_1] + (x - x_1)P_n[x, x_o, x_1]$

$P_n[x, x_o, x_1] = P_n[x_0, x_1, x_2] + (x - x_2)P_n[x, x_o, x_1, x_2]$

.....

$P_n[x, x_o,.., x_{n-1}] = P_n[x_0, x_1,.., x_n] + (x - x_n)P_n[x, x_o,.., x_n]$

Do $P_n[x, x_o,.., x_n] = 0$ nên từ đó ta có :

$$P_n(x) = P_n(x_0) + (x - x_0)P_n[x_o, x_1] + (x - x_0)(x - x_1)P_n[x_0, x_1, x_2] + \ldots + (x - x_0)\ldots(x - x_{n-1})P_n[x_0, \ldots, x_n]$$

Nếu $P_n(x)$ là đa thức nội suy của hàm y = f(x) thì:

$P_n(x_i) = f(x_i) = y_i$ với $i = 0 \div n$

Do đó các tỉ hiệu từ cấp 1 đến cấp n của P_n và của y là trùng nhau và như vậy ta có :

$$P_n(x) = y_0 + (x - x_0)y[x_0, x_1] + (x - x_0)(x - x_1)y[x_0, x_1, x_2] + .. + (x - x_0)(x - x_1)...(x - x_{n-1})y[x_0,..,x_n]$$

Đa thức này gọi là đa thức nội suy Newton tiến xuất phát từ nút x_0 của hàm y = f(x). Ngoài đa thức tiến còn có đa thức nội suy Newton lùi xuất phát từ điểm x_n có dạng như sau :

$$P_n(x) = y_n + (x - x_n)y[x_n, x_{n-1}] + (x - x_n)(x - x_{n-1})y[x_n, x_{n-1}, x_{n-2}] + .. + (x - x_n)(x - x_{n-1})...(x - x_1)y[x_n,.., x_0]$$

Trường hợp các nút cách đều thì $x_i = x_0 + ih$ với i = 0, 1,.., n. Ta gọi sai phân tiến cấp 1 tại i là :

$$\Delta y_i = y_{i+1} - y_i$$

và sai phân tiến cấp hai tại i:

$$\Delta^2 y_i = \Delta(\Delta y_i) = y_{i+2} - 2y_{i+1} + y_i$$

.....

và sai phân tiến cấp n là :

$$\Delta^n y_i = \Delta(\Delta^{n-1} y_i)$$

Khi đó ta có:

$$y[x_0, x_1] = \frac{\Delta y_0}{h}$$

$$y[x_0, x_1, x_2] = \frac{\Delta^2 y_0}{2h^2}$$

.....

$$y[x_0, x_1, x_2, ..., x_n] = \frac{\Delta^n y_0}{n! h^n}$$

Bây giờ đặt $x = x_0 + ht$ trong đa thức Newton tiến ta được:

$$P_n(x_0 + ht) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \cdots + \frac{t(t-1)\cdots(t-n+1)}{n!}\Delta^n y_0$$

thì ta nhận được đa thức Newton tiến xuất phát từ x_0 trong trường hợp nút cách đều. Với n = 1 ta có :

$$P_1(x_0 + ht) = y_0 + \Delta y_0$$

Với n = 2 ta có:

$$P_n(x_0 + ht) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0$$

Một cách tương tự ta có khái niệm các sai phân lùi tại i:

$$\nabla y_i = y_i - y_{i-1}$$

$$\nabla^2 y_i = \nabla(\nabla y_i) = y_i - 2y_{i-1} + y_{i-2}$$

.....

$$\nabla^n y_i = \nabla(\nabla^{n-1} y_i)$$

và đa thức nội suy Newton lùi khi các điểm nội suy cách đều:

$$P_n(x_0 + ht) = y_n + t\nabla y_n + \frac{t(t+1)}{2!}\nabla^2 y_n + \cdots + \frac{t(t+1)\cdots(t+n-1)}{n!}\nabla^n y_n$$

Ta xây dựng hàm ***newton()*** để nội suy:

function [n,DD] = newton(x,y)

```
%Dua vao : x = [x0 x1 ... xN]
% y = [y0 y1 ... yN]
%Lay ra: n = he so cua da thuc Newton bac N
N = length(x) - 1;
DD = zeros(N + 1, N + 1);
DD(1:N + 1, 1) = y';
for k = 2:N + 1
   for m = 1: N + 2 - k
      DD(m,k) = (DD(m + 1, k - 1) - DD(m, k - 1))/(x(m + k - 1) - x(m));
   end
end
a = DD(1, :);
n = a(N+1);
for k = N:-1:1
   n = [n a(k)] - [0 n*x(k)];
end
```

Cho hàm dưới dạng bảng:

x	-2	-1	1	2	4
y	-6	0	0	6	60

Ta dùng chương trình ***ctnewton.m*** để nội suy:

```
clear all, clc
x = [-2 -1 1 2 4];
y = [-6 0 0 6 60];
a = newton(x, y)
yx = polyval(a, 2.5)
```

§3. NỘI SUY AITKEN - NEVILLE

Một dạng khác của đa thức nội suy được xác định bằng thuật toán Aitken - Neville. Giả sử ta có n điểm đã cho của hàm f(x). Như vậy qua hai điểm x_0 và x_1 ta có đa thức nội suy Lagrange của hàm f(x) được viết dưới dạng:

$$P_{01}(x) = \frac{\begin{vmatrix} y_0 & x_0 - x \\ y_1 & x_1 - x \end{vmatrix}}{x_1 - x_0}$$

Đây là một đa thức bậc 1:

$$P_{01}(x) = y_0 \frac{x - x_1}{x_0 - x_1} + y_1 \frac{x - x_0}{x_1 - x_0}$$

Khi $x = x_0$ thì:

$$P_{01}(x_0) = \frac{\begin{vmatrix} y_0 & x_0 - x_0 \\ y_1 & x_1 - x_0 \end{vmatrix}}{x_1 - x_0} = y_0$$

Khi $x = x_1$ thì:

$$P_{01}(x_1) = \frac{\begin{vmatrix} y_0 & x_0 - x_1 \\ y_1 & x_1 - x_1 \end{vmatrix}}{x_1 - x_0} = y_1$$

Đa thức nội suy Lagrange của f(x) qua 3 điểm x_0, x_1, x_2 có dạng:

$$P_{012}(x) = \frac{\begin{vmatrix} P_{01}(x) & x_0 - x \\ P_{12}(x) & x_2 - x \end{vmatrix}}{x_2 - x_0}$$

và là một đa thức bậc 2:

$$P_{012}(x) = y_0 \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + y_1 \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + y_2 \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}$$

Khi $x = x_0$ thì:

$$P_{012}(x_0) = \frac{\begin{vmatrix} y_0 & x_0 - x_0 \\ P_{12}(x) & x_2 - x_0 \end{vmatrix}}{x_2 - x_0} = y_0$$

Khi $x = x_1$ thì:

$$P_{012}(x_1) = \frac{\begin{vmatrix} y_1 & x_0 - x_1 \\ y_1 & x_2 - x_1 \end{vmatrix}}{x_2 - x_0} = y_1$$

Khi $x = x_2$ thì:

$$P_{012}(x_2) = \frac{\begin{vmatrix} P_{01}(x_2) & x_0 - x_2 \\ y_2 & x_2 - x_2 \end{vmatrix}}{x_2 - x_0} = y_2$$

Tổng quát đa thức nội suy Lagrange qua n điểm là:

$$P_{012..n}(x) = \frac{\begin{vmatrix} P_{01..(n-1)}(x) & x_0 - x \\ P_{12..n}(x) & x_n - x \end{vmatrix}}{x_2 - x_0}$$

Như vậy ta có thể dùng phép lặp để xác định lần lượt các đa thức Lagrange. Sơ đồ tính toán như vậy gọi là sơ đồ Neville - Aitken.

Ta xây dựng hàm ***aitkenneville()*** để nội suy:

```
function a = aitkenneville(xData, yData, x)
% Tra ve gia tri noi suy tai x.
```

```
% Cu phap: y = aitkenneville(xData, yData, x)
n = length(xData);
y = yData;
for k = 1:n-1
   y(1:n-k) = ((x - xData(k+1:n)).*y(1:n-k)...
    + (xData(1:n-k) - x).*y(2:n-k+1))...
   ./(xData(1:n-k) - xData(k+1:n));
end
a = y(1);
```

Cho các cặp số (1, 3), (2, 5), (3, 7), (4, 9) và (5, 11), để tìm y tại x = 2.5 ta dùng chương trình ***ctaitkennevile.m***:

```
clear all, clc
x = [1  2  3  4];
y = [3  5  7  9];
yx = aitkenneville(x, y, 2.5)
```

§4. NỘI SUY BẰNG ĐƯỜNG CONG SPLINE BẬC BA

Khi số điểm cho trước dùng khi nội suy tăng, đa thức nội suy có dạng sóng và sai số tăng. Ta xét hàm thực:

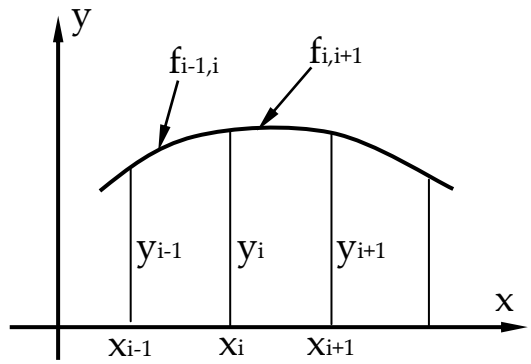
$$f31(x) = \frac{1}{1+8x^2}$$

và nội suy nó bằng thuật toán Newton nhờ chương trình ***cttestintp.m***

```
%Noi suy Newton
x1 = [-1 -0.5 0 0.5 1.0];
y1 = f31(x1);
n1 = newton(x1,y1)
x2 = [-1 -0.75 -0.5 -0.25 0 0.25 0.5 0.75 1.0];
y2 = f31(x2);
n2 = newton(x2,y2)
x3 = [-1 -0.8 -0.6 -0.4 -0.2  0  0.2  0.4  0.6  0.8  1.0];
y3 = f31(x3);
n3 = newton(x3,y3)
xx = [-1:0.02: 1]; %pham vi noi suy
yy = f31(xx); %ham thuc
yy1 = polyval(n1, xx); %ham xap xi qua 5 diem
yy2 = polyval(n2, xx); %ham xap xi qua 9 diem
yy3 = polyval(n3, xx); %ham xap xi qua 11 diem
subplot(221)
plot(xx, yy, 'k-',  xx, yy1, 'b')
```

```
subplot(224)
plot(xx, yy1-yy, 'r', xx, yy2-yy, 'g', xx, yy3-yy,'b') %do thi sai so
subplot(222)
plot(xx,yy,'k-',  xx, yy2, 'b')
subplot(223)
plot(xx, yy, 'k-',  xx, yy3, 'b')
```

và nhận được kết quả.



Để tránh hiện tượng sai số lớn khi số điểm mốc tăng ta dùng nội suy nối trơn(spline). Trên các đoạn nội suy ta thay hàm bằng một đường cong. Các đường cong này được ghép trơn tại các điểm nối. Ta chọn các đường cong này là hàm bậc 3 vì hàm bậc 1 và bậc hai khó bảo đảm điều kiện nối trơn.

Cho một loạt giá trị nội suy $(x_1, y_1),\ldots,(x_i, y_i),\ldots,(x_n, y_n)$. Trên mỗi đoạn ta có một hàm bậc 3. Như vậy giữa nút i và (i +1) ta có hàm $f_{i,i+1}(x)$, nghĩa là ta dùng (n - 1) hàm bậc 3 $f_{1,2}(x), f_{2,3}(x),\ldots, f_{n-1,n}(x)$ để thay thế cho hàm thực. Hàm $f_{i,i+1}(x)$ có dạng:

$$f_{i,i+1}(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \qquad (1)$$

Hàm này thoả mãn:

$$f_{i,i+1}(x_i) = a_i = y_i \qquad (3)$$

$$f_{i,i+1}(x_{i+1}) = d_i h_i^3 + c_i h_i^2 + b_i h_i + a_i = y_{i+1} \qquad (4)$$

$$f'_{i,i+1}(x_i) = b_i \qquad (5)$$

$$f'_{i,i+1}(x_{i+1}) = 3d_i h_i^2 + 2c_i h_i + b_i \qquad (6)$$

$$f''_{i,i+1}(x_i) = 2c_i = y''_i \qquad (7)$$

$$f''_{i,i+1}(x_{i+1}) = 6d_i h_i + 2c_i = y''_{i+1} \qquad (8)$$

Muốn nối trơn ta cần có đạo hàm bậc nhất liên tục và do đó:

$$f''_{i-1,i}(x_i) = f''_{i,i+1}(x_i) = k_i$$

Lúc này các giá trị k chưa biết, ngoại trừ $k_1 = k_n = 0$(ta các các mút là điểm uốn). Điểm xuất phát để tính các hệ số của $f_{i,i+1}(x)$ là biểu thức của $f''_{i,i+1}(x_i)$. Sử dụng nội suy Lagrange cho hai điểm ta có:

$$f''_{i,i+1}(x_i) = k_i L_i(x) + k_{i+1} L_{i+1}(x)$$

Trong đó:

$$L_i(x) = \frac{x - x_{i+1}}{x_i - x_{i+1}} \qquad L_{i+1}(x) = \frac{x - x_i}{x_{i+1} - x_i}$$

Do vậy:

$$f''_{i,i+1}(x_i) = \frac{k_i(x - x_{i+1}) - k_{i+1}(x - x_i)}{x_i - x_{i+1}}$$

Tích phân biểu thức trên hai lần theo x ta có:

$$f_{i,i+1}(x_i) = \frac{k_i(x - x_{i+1})^3 - k_{i+1}(x - x_i)^3}{6(x_i - x_{i+1})} + A(x - x_{i+1}) - B(x - x_i)$$

Trong đó A và B là các hằng số tích phân

Số hạng cuối trong phương trình trên thường được viết là Cx + D.

Đặt C = A - B và D = $-Ax_{i+1} + Bx_i$ để dễ dàng tính toán. Từ điều kiện $f_{i,i+1}(x_i) = y_i$ ta có:

$$\frac{k_i(x_i - x_{i+1})^3}{6(x_i - x_{i+1})} + A(x_i - x_{i+1}) = y_i$$

nên:

$$A = \frac{y_i}{x_i - x_{i+1}} - \frac{k_i(x_i - x_{i+1})}{6}$$

Tương tự, điều kiện $f_{i,i+1}(x_{i+1}) = y_{i+1}$ cho ta:

$$B = \frac{y_{i+1}}{x_i - x_{i+1}} - \frac{k_{i+1}(x_i - x_{i+1})}{6}$$

Kết quả là:

$$\begin{aligned} f_{i,i+1}(x_i) = {} & \frac{k_i}{6}\left[\frac{(x - x_{i+1})^3}{x_i - x_{i+1}} - (x - x_{i+1})(x_i - x_{i+1})\right] \\ & - \frac{k_{i+1}}{6}\left[\frac{(x - x_i)^3}{x_i - x_{i+1}} - (x - x_i)(x_i - x_{i+1})\right] \\ & + \frac{y_i(x - x_{i+1}) - y_{i+1}(x - x_i)}{x_i - x_{i+1}} \end{aligned}$$

Đạo hàm cấp 2 k_i tại các nút bên trong được tính từ điều kiện:

$$f'_{i-1,i}(x_i) = f'_{i,i+1}(x_i)$$

Sau khi biến đổi ta có phương trình:

$$\begin{aligned} & k_{i-1}(x_{i-1} - x_i) + 2k_i(x_{i-1} - x_{i+1}) + k_{i+1}(x_i - x_{i+1}) \\ & = 6\left(\frac{y_{i-1} - y_i}{x_{i-1} - x_i} - \frac{y_i - y_{i+1}}{x_i - x_{i+1}}\right) \end{aligned}$$

Khi các điểm chia cách đều $(x_{i+1} - x_i) = h$ ta có:

$$k_{i-1} + 4k_i + k_{i+1} = \frac{6}{h^2}\left(y_{i-1} - 2y_i + y_{i+1}\right) \qquad i = 2, 3, \ldots, n - 1$$

Ta xây dựng hàm ***cubicspline()*** để nội suy:

```
function y = cubicspline(xData, yData, x)
%Ham nay xap xi bang da thuc bac 3 spline
%Cu phap: [yi,f] = cubicspline(xData, yData, x)
n = length(xData);
```

```
c = zeros(n-1, 1); d = ones(n, 1);
e = zeros(n-1, 1); k = zeros(n, 1);
c(1:n-2) = xData(1:n-2) - xData(2:n-1);
d(2:n-1) = 2*(xData(1:n-2) - xData(3:n));
e(2:n-1) = xData(2:n-1) - xData(3:n);
k(2:n-1) = 6*(yData(1:n-2) - yData(2:n-1))...
./(xData(1:n-2) - xData(2:n-1))...
- 6*(yData(2:n-1) - yData(3:n))...
./(xData(2:n-1) - xData(3:n));
[c, d, e] = band3(c, d e);
k = band3sol(c, d, e, k);
i = findseg(xData, x);
h = xData(i) - xData(i+1);
y = ((x - xData(i+1))^3/h - (x - xData(i+1))*h)*k(i)/6.0...
- ((x - xData(i))^3/h - (x - xData(i))*h)*k(i+1)/6.0...
+ yData(i)*(x - xData(i+1))/h...
- yData(i+1)*(x - xData(i))/h;
```

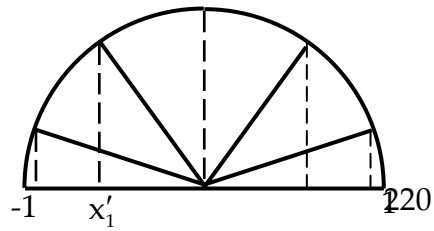
Ta có chương trình ***ctcubicspline.m*** dùng nội suy:

```
clear all, clc
x1 = 0:0.1:5;
y1 = (x1+1).^2;
while 1
  x = input('x = ');
   if isempty(x)
     fprintf('Ket thuc');
     break
   end
   y = cubicspline(xData, yData, x)
   fprintf('\n')
end
```

§5. NỘI SUY BẰNG ĐA THỨC CHEBYSHEV

Khi nội suy bằng đa thức Newton hay Lagrange, nghĩa là thay hàm thực bằng đa thức xấp xỉ, có khoảng cách cách đều thì sai số giữa đa thức nội suy và hàm thực có xu hướng tăng tại hai mút nội suy. Ta thấy rõ điều này khi chạy chương trình ***cttestintp.m***.

Do vậy ta nên chọn các điểm mốc nội suy ở hai mút dày hơn ở giữa. Một trong những cách chọn phân bố các điểm mốc là hình chiếu lên trục x của các điểm



cách đều trên đường tròn tâm tại điểm giữa của đoạn nội suy. Như vậy với đoạn nội suy [-1, 1] ta có:

$$x'_k = \cos\frac{2n+1-2k}{2(n+1)}\pi \qquad k = 1, 2,\ldots,n \tag{1}$$

Với đoạn nội suy [a, b] bất kì:

$$x_k = \frac{b-a}{2}x'_k + \frac{b+a}{2} = \frac{b-a}{2}\cos\frac{2n+1-2k}{2(n+1)}\pi + \frac{a+b}{2} \qquad k = 1, 2,\ldots,n \tag{2}$$

Các nút nội suy này được gọi là các nút Chebyshev. Đa thức nội suy dựa trên các nút Chebyschev gọi là đa thức nội suy Chebyshev.

Ta xét hàm thực:

$$f(x) = \frac{1}{1+8x^2}$$

Ta chọn số nút nội suy lần lượt là 5, 9, 11 và xây dựng các đa thức Newton (hay Lagrange) $c_4(x)$, $c_8(x)$ và $c_{10}(x)$ đi qua các nút này và vẽ đồ thị của hàm thực cũng như sai số khi nội suy bằng chương trình ***ctcomchebynew.m*** với các N khác nhau.

```
x1 = [-1 -0.5 0 0.5 1.0];
y1 = f31(x1);
n1 = newton(x1,y1);
xx = [-1:0.02: 1]; %pham vi noi suy
yy1 = polyval(n1,xx); %ham xap xi qua 5 diem
yy = f31(xx); %ham thuc
subplot(221)
plot(xx,yy,'k-', x, y, 'o', xx, yy1, 'b');
title('Newton')
subplot(223)
plot(xx, yy1-yy, 'r') %do thi sai so
N = 4;
k = [0:N];
x = cos((2*N + 1 - 2*k)*pi/2/(N + 1));
y = f31(x);
c = newton(x, y) %da thuc noi suy dua tren cac nut Chebyshev
xx = [-1:0.02: 1]; %doan noi suy
yy = f31(xx); %do thi ham thuc
yy1 = polyval(c, xx); %do thi ham xap xi
subplot(222)
plot(xx, yy, 'k-',  x, y, 'o',  xx, yy1, 'b')
title('Chebyshev')
subplot(224)
plot(xx, yy1-yy, 'r') %do thi sai so
```

Khi tăng số điểm mốc, nghĩa là tăng bậc của đa thức Chebyschev, sai số giảm. Đa thức Chebyshev bậc n được xác định bằng:

$$T_{n+1}(x') = \cos((n+1)\arccos(x')) \qquad (3)$$

và các nút Chebyshev cho bởi (1) là nghiệm của (3).

Ta có:

$$\begin{aligned} T_{n+1}(x') &= \cos(\arccos(x') + n\arccos(x')) \\ &= \cos(\arccos(x'))\cos(n\arccos(x') - \sin(\arccos(x'))\sin(n\arccos(x')) \\ &= x'T_n(x') + 0.5\left[\cos((n+1)\arccos(x') - \cos((n-1)\arccos(x')\right] \\ &= x'T_n(x') + 0.5T_{n+1}(x') - 0.5T_{n-1}(x') \end{aligned}$$

nên:

$$T_{n+1}(x') = 2xT_n(x') - T_{n-1}(x') \qquad n \geq 1 \qquad (4)$$

và $T_0(x') = 1 \qquad T_1(x') = \cos(\arccos(x') = x' \qquad (5)$

Các đa thức Chebyshev đến bậc 6 là:

$T_0(x) = 1$

$T_1(x') = x'$

$T_2(x') = 2x'^2 - 1$

$T_3(x') = 4x'^3 - 3x'$

$T_4(x') = 8x'^4 - 8'x^2 + 1$

$T_5(x') = 16x'^5 - 20'x^3 + 5x'$

$T_6(x') = 32x'^6 - 48x'^4 + 18x'^2 - 1$

$T_7(x') = 64x'^7 - 112x'^5 + 56x'^3 - 7x'$

Hàm f(x) được xấp xỉ bằng:

$$f(x) = \sum_{m=0}^{N} d_m T_m(x')\Big|_{x'=\frac{2}{b-a}\left(x-\frac{a+b}{2}\right)} \qquad (6)$$

Trong đó:

$$d_0 = \frac{1}{n+1}\sum_{k=0}^{n} f(x_k)T_0(x'_k) = \frac{1}{n+1}\sum_{k=0}^{n} f(x_k) \qquad (7)$$

$$\begin{aligned} d_m &= \frac{2}{n+1}\sum_{k=0}^{n} f(x_k)T_m(x'_k) \\ &= \frac{2}{n+1}\sum_{k=0}^{n} f(x_k)\cos\frac{m(2n+1-2k)}{2(n+1)}\pi \qquad m = 1,2,...,n \end{aligned} \qquad (8)$$

Ta xây dựng hàm ***cheby()*** để tìm đa thức nội suy Chebyshev:

```
function [c, x, y] = cheby(f, N, a, b)
%vao : f = ten ham tren doan [a, b]
%Ra: c = Cac he so cua da thuc Newton bac N
% (x,y) = cac nut Chebyshev
if nargin == 2
   a = -1;
```

```
    b = 1;
end
k = [0: N];
theta = (2*N + 1 - 2*k)*pi/(2*N + 2);
xn = cos(theta); %pt.(1)
x = (b - a)/2*xn +(a + b)/2; %pt.(2)
y = feval(f,x);
d(1) = y*ones(N + 1,1)/(N+1);
for m = 2: N + 1
        cos_mth = cos((m-1)*theta);
    d(m) = y*cos_mth'*2/(N + 1); %pt.(7)
end
xn = [2 -(a + b)]/(b - a); %nghich dao cua t. (2)
T_0 = 1; T_1 = xn; %pt.(5)
c = d(1)*[0 T_0] +d(2)*T_1; %pt.(6)
for m = 3: N + 1
    tmp = T_1;
    T_1 = 2*conv(xn,T_1) -[0 0 T_0]; %pt.(4)
    T_0 = tmp;
    c = [0 c] + d(m)*T_1; %pt.(6)
end
```

Để tìm đa thức Chebyshev dùng xấp xỉ hàm $f(x) = \dfrac{1}{1+8x^2}$ ta dùng chương trình ***ctcheby.m***:

```
clear all, clc
N = 2;
a = -2;
b = 2;
[c, x1, y1] = cheby('f31', N, a, b) %da thuc Chebyshev
%so sanh voi da thuc Lagrange/Newton
k = [0:N];
xn = cos((2*N + 1 - 2*k)*pi/2/(N + 1));%pt.(1):nut Chebyshev
x = ((b-a)*xn +a + b)/2; %pt.(2)
y = f31(x);
n = newton(x, y)
l = lagrange(x, y)
```

§6. XẤP XỈ HÀM BẰNG PHÂN THỨC HỮU TỈ

Xấp xỉ Padé dùng để xấp xỉ hàm f(x) tại x_0 bằng hàm hữu tỉ:

$$P_{m,n}(x-x_0)=\frac{Q_m(x-x_0)}{D_n(x-x_0)}$$

$$=\frac{q_0+q_1(x-x_0)+q_2(x-x_0)^2+\cdots+q_m(x-x_0)^m}{1+d_1(x-x_0)+d_2(x-x_0)^2+\cdots+d_n(x-x_0)^n} \quad (1)$$

với m = n hay m = n + 1

Trong đó $f(x_0)$, $f'(x_0)$,…, $f^{(m+n)}(x_0)$ đã cho

Trước hết ta khai triển Taylor hàm f(x) tại $x = x_0$ đến bậc (m + n).

$$f(x)\approx T_{m+n}(x)=f(x_0)+f'(x_0)(x-x_0)$$

$$+\frac{f''(x_0)}{2!}(x-x_0)^2+\cdots+\frac{f^{(m+n)}(x_0)}{(m+n)!}(x-x_0)^{m+n} \quad (2)$$

$$=a_0+a_1(x-x_0)+a_2(x-x_0)^2+\cdots+a_{m+n}(x-x_0)^{m+n}$$

Để đơn giản ta coi $x_0 = 0$. Ta cần tính các hệ số của $D_n(x)$ và $Q_m(x)$ sao cho:

$$T_{m+n}(x)-\frac{Q_m(x)}{D_n(x)}=0 \text{ hay } T_{m+n}(x)D_n(n) - Q_m(x) = 0$$

nghĩa là:

$$(a_0+a_1x+\cdots+a_{m+n}x^{m+n})(1+d_1x+\cdots+d_nx^n)=(q_0+q_1x+\cdots+q_mx^m) \quad (3)$$

Cân bằng các số hạng cùng bậc ở hai vế ta có:

$$\begin{cases} a_0=q_0 \\ a_1+a_0d_1=q_1 \\ a_2+a_1d_1+a_0d_2=q_2 \\ \cdots \\ a_m+a_{m-1}d_1+a_{m-2}d_2+\cdots+a_{m-n}d_n=q_m \end{cases} \quad (4)$$

$$\begin{cases} a_{m+1}+a_md_1+a_{m-1}d_2+\cdots+a_{m-n+1}d_n=0 \\ a_{m+2}+a_{m+1}d_1+a_md_2+\cdots+a_{m-n+2}d_n=0 \\ \cdots \\ a_{m+n}+a_{m+n-1}d_1+a_{m+n-2}d_2+\cdots+a_md_n=0 \end{cases} \quad (5)$$

Trước hết ta giải (5) để tìm d_i và sau đó thay vào (4) để tìm q_i.

Ta xây dựng hàm ***padeapp()*** để tính xấp xỉ:

```
function [num, den] = padeapp(f, xo, M, N, x0, xf)
%Vao : f = Ham can xap xi trong doan [xo, xf]
%Ra: num = Cac he so cua tu so
% den = Cac he so cua mau so
a(1) = feval(f, xo);
h = .01;
```

```
tmp = 1;
for i = 1:M + N
    tmp = tmp*i*h; %i!h^i
    dix = difapx(i, [-i i])*feval(f, xo + [-i:i]*h)'; %dao ham
    a(i + 1) = dix/tmp; %he so chuoi Taylor
```

```
end
for m = 1:N
n = 1:N;
   A(m, n) = a(M + 1 + m - n);
   b(m) = -a(M + 1 + m);
end
d = A\b'; %pt.(5)
for m = 1: M + 1
   mm = min(m - 1,N);
   q(m) = a(m:-1:m - mm)*[1; d(1:mm)]; %pt.(4)
end
num = q(M + 1:-1:1)/d(N); den = [d(N:-1:1)' 1]/d(N); %giam dan
if nargout == 0 % ve ham thuc, khai trien taylor va ham Pade
   if nargin < 6
      x0 = xo - 1;
      xf = xo + 1;
   end
   x = x0 + [xf - x0]/100*[0:100];
   yt = feval(f, x);
   x1 = x - xo;
   yp = polyval(num,x1)./polyval(den,x1);
   yT = polyval(a(M + N + 1:-1:1),x1);
   clf,  plot(x, yt, 'k', x, yp, 'r',  x, yT, 'b')
end
```

Để xấp xỉ hàm e^x ta dùng chương trình ***ctpadeapp.m***:

```
f1 = inline('exp(x)', 'x');
M = 3;
N = 2; %bac cua Q(x) va D(x)
xo = 0; %tam cua chuoi Taylor
[n,d] = padeapp(f1, xo, M, N) %tinh cac he so cua Q(x)/P(x)
x0 = -3.5;
xf = 0.5; %bien trai va phai cua khoang xap xi
padeapp(f1, xo, M, N, x0, xf) %xem do thi
```

§7. NỘI SUY BẰNG ĐA THỨC HERMIT

Trong một số trường hợp, ta cần tìm hàm đa thức không những đi qua các điểm cho trước mà còn phải thoả mãn điều kiện về đạo hàm tại các điểm đó. Ta gọi đa thức như vậy là đa thức nội suy Hermit. Để đơn giản, ta khảo sát một đa thức bậc 3:

$$h(x) = H_3x^3 + H_2x^2 + H_1x + H_0 \qquad (1)$$

đi qua hai điểm (x_0, y_0), (x_1, y_1) và có các đạo hàm là y'_0, y'_1. Ta tìm các hệ số H_i bằng cách giải hệ phương trình:

$$\begin{cases} h(x_0) = H_3x_0^3 + H_2x_0^2 + H_1x_0 + H_0 = y_0 \\ h(x_1) = H_3x_1^3 + H_2x_1^2 + H_1x_1 + H_0 = y_1 \\ h'(x_0) = 3H_3x_0^2 + 2H_2x_0 + H_1 = y'_0 \\ h'(x_1) = 3H_3x_1^2 + 2H_2x_1 + H_1 = y'_1 \end{cases} \quad (2)$$

Các đạo hàm bậc nhất được tính gần đúng bằng:

$$y'_0 = \frac{h(x_0+\varepsilon) - h(x_0)}{\varepsilon} = \frac{y_2 - y_0}{\varepsilon}$$
$$y'_1 = \frac{h(x_1) - h(x_1-\varepsilon)}{\varepsilon} = \frac{y_1 - y_3}{\varepsilon} \quad (3)$$

Bây giờ ta tìm đa thực nội suy Lagrange hay Newton đi qua 4 điểm:

(x_0, y_0), $(x_2 = x_0 + \varepsilon, y_2 = y_0 + y'_0\varepsilon)$, $(x_3 = x_1 - \varepsilon, y_3 = y_1 - y'_1\varepsilon)$, (x_1, y_1)

Hàm ***hermit()*** tạo nên phương trình (2):

```
function H = hermit(x0, y0, dy0, x1, y1, dy1)
A = [x0^3 x0^2 x0 1; x1^3 x1^2 x1 1;
     3*x0^2 2*x0 1 0; 3*x1^2 2*x1 1 0];
b = [y0 y1 dy0 dy1]'; %Pt.(2)
H = (A\b)';
```

Hàm ***hermits()*** dùng hàm ***hermit()*** để tính các hệ số của đa thức Hermit trên nhiều đoạn và giá trị nội suy:

```
function [H,yi]  = hermits(x, y, dy, xi)
% Tim cac he so cua c da thuc Hermite tren c doan
clc
for n = 1:length(x)-1
   H(n,:) = hermit(0, y(n), dy(n), x(n + 1)-x(n), y(n + 1), dy(n + 1));
end
yi = ppval(mkpp(x, H),xi)
```

Để nội suy ta dùng chương trình ***cthermite.m***:

```
clear all, clc
x = [0  1  2  3];
y = [1  2  4  5];
dy = [0  2  4  6];
[h, y] = hermits(x, y, dy, 1.5)
```

§8. BIẾN ĐỔI FOURIER

1. Biến đổi Fourrier: Tín hiệu thực tế thường bao gồm các thành phần có tần số khác nhau. Chuỗi Fourier và phép biến đổi Fourier là công cụ toán học dùng để phân tích đặc tính tần số của tín hiệu. Có 4 định nghĩa tương tự nhau về chuỗi và phép biến đổi Fourier, gồm: chuỗi Fourier liên tục theo t(CFS), phép biến đổi Fourier liên tục theo t(CFT), chuỗi Fourier gián đoạn theo t(DFS) và phép biến đổi Fourier gián đoạn theo t(DFT). Trong các công cụ này, DFT dễ dàng lập trình trên máy tính nên trong phần này ta sẽ chú ý đến nó.

Giả sử chuỗi số liệu { x[n] = x(nT), n = 0 : M - 1} với T là chu kì lấy mẫu có được bằng cách lấy mẫu một tín hiệu liên tục x(t) T lần trong một giây. N cặp điểm DFT và iDFT được định nghĩa bằng:

DFT: $$X(k) = \sum_{n=0}^{N-1} x[n]e^{-j2\pi nk/N} \quad (1a)$$

iDFT: $$x[n] = \frac{1}{N}\sum_{n=0}^{N-1} X(k)e^{j2\pi nk/N} \quad (1b)$$

Nói chung hệ số DFT của X(k) là một số phức và nó xác định biên độ và pha của thành phần tín hiệu có tần số số $\Omega_k = k\Omega_0$(rad), tương ứng với tần số tương tự $\omega_k = k\omega_0 = k\Omega_0/T = 2\pi k/NT$ (rad/s). Ta gọi $\Omega_0 = 2\pi/N$ và $\omega_0 = 2\pi/NT$ là các tần số cơ bản số và tương tự (tần số phân giải) vì đây là hiệu tần số có thể phân biệt bởi N điểm DFT.

DFT và DFS có cùng bản chất nhưng khác nhau về phạm vi thời gian/tần số. Cụ thể là tín hiệu x[n] và DFT X[k] của nó kéo dài hữu hạn trên phạm vi thời gian/tần số $\{0 \le n \le N-1\}$ và $\{0 \le k \le N-1\}$. Tín hiệu x[n] được phân tích bởi DFS và DFS của nó X(k) là chu kì tín hiệu với chu kì N trên toàn bộ tập số nguyên.

Biến đổi Fourier nhanh FFT là thuật toán hiệu quả để tính DFT và iDFT được xây dựng bằng cách dùng tính chu kì và tính đối xứng cuả nhân tử $e^{i2\pi nk/N}$ để giảm bớt số nhân tử phức từ N^2 thành $(N/2)\log_2 N$)N thể hiện kích thước của DFT. Hàm MATLAB fft() và ifft() thực hiện thuật toán đối với $N = 2^l$ (l là số nguyên không âm). Nếu độ dài M của chuỗi số liệu ban đầu không phải là bội số của 2, có thể mở rộng bằng cách đệm thêm số 0 vào cuối chuỗi và gọi là đệm zero.

Ta xem xét hiệu qủa này bằng cách thực hiện đoạn lệnh trong ***ctcompdftfft.m***.

```
%So sanh phep bien doi Fourier nhanh va roi rac
clear, clf
N = 2^10;
n = [0:N - 1];
x = cos(2*pi*200/N*n)+ 0.5*sin(2*pi*300/N*n);
tic %ngung dong ho
for k = 0:N - 1
   X(k+1) = x*exp(-j*2*pi*k*n/N).';
```

```
end %DFT
k = [0:N - 1];
for n = 0:N - 1
   xr(n + 1) = X*exp(j*2*pi*k*n/N).';
end %IDFT
time_dft = toc
plot(k,abs(X))
pause, hold on
tic
X1 = fft(x); %FFT
xr1 = ifft(X1); %IFFT
time_fft = toc %dua ra thoi gian thuc hien
clf, plot(k,abs(X1),'r') %pho bien do
```

Chạy đoạn lệnh và so sánh thời gian thực hiện 1024 điểm tính DFT/iDFT và FFT/iFFT.

2. Ý nghĩa vật lý của biến đổi Fourrier rời rạc: Để hiểu được ý nghĩa vật lí của FFt ta thực hiện các lệnh trong chương trình ***ctmeanning.m***. Chương trình cho ta phổ biên độ của tín hiệu

$$x(t) = \sin(1.5\pi t) + 0.5\cos(3\pi t) \qquad (2)$$

được lấy mẫu mỗi T s.

Từ các kết quả ta thấy khi T = 0.1 và N = 32 thì $X_a(k)$ lớn tại k = 2 và k= 5. Lúc đó $k\omega_0 = 2\pi k/NT = 2\pi k/3.2 \approx 1.5\pi$ và $3.125\pi \approx 3\pi$.

Khi T = 0.05 và N = 64 thì $X_b(k)$ cũng lớn tại k = 2 và k = 5. Lúc đó $k\omega_0 = 1.25\pi \approx 1.5\pi$ và $3.125\pi \approx 3\pi$.

Khi T = 0.1 và N = 64 thì $X_c(k)$ lớn tại k = 4 ,k = 5, k = 9 và k = 10. Lúc đó $k\omega_0 = 2\pi k/NT = 2\pi k/6.4 \approx 1.25\pi \sim 1.5625\pi$ và $2.8125\pi \sim 3\pi$.

Khi T = 0.1 và N = 64 thì $X_d(k)$ lớn tại k = 5 và k = 10. Lúc đó $k\omega_0 = 1.5625\pi \approx 1.5\pi$ và $3.125\pi \approx 3\pi$.

Tồn tại nhiều phổ DFT khác nhau của cùng một tín hiệu tương tự, tuỳ thuộc vào kích thước DFT, chu kì lấy mẫu, khoảng biến thiên của hàm và đệm zero. So sánh với phổ tại T = 0.1s, phổ tại T = 0.05s có phạm vi tần số tương tự $[0, 2\pi/T_b]$ rộng hơn nhưng có cùng tần số phân giải tương tự là $\omega_0 = \Omega_0/T_b = 2\pi/N_bT_b = \pi/1.6 = 2\pi/N_aT_a$. Phổ khi có đệm zero trơn.

```
clear, clf
w1 = 1.5*pi;
w2 = 3*pi;
N = 32;
n = [0:N - 1];
T = 0.1; %chu ki lay mau
```

```
t = n*T;
xan = sin(w1*t) + 0.5*sin(w2*t);
subplot(421)
stem(t,xan,'.')
k = 0:N - 1;
Xa = fft(xan);
dscrp=norm(xan-real(ifft(Xa)))
subplot(423)
stem(k,abs(Xa),'.')
N = 32;
n = [0:N - 1];
T = 0.1;
t = n*T;
xan = sin(w1*t) + 0.5*sin(w2*t);
subplot(422)
stem(t,xan,'.')
k = 0:N - 1;
Xa = fft(xan);
Dscrp = norm(xan - real(ifft(Xa)))
subplot(424)
stem(k, abs(Xa),'.')
N = 64;
n = [0:N - 1];
T = 0.05;
t = n*T;
xbn = sin(w1*t) + 0.5*sin(w2*t);
subplot(425)
stem(t,xbn,'.')
k = 0:N - 1;
Xb = fft(xbn);
subplot(427)
stem(k,abs(Xb),'.')
N = 64;
n = [0:N-1];
T = 0.1;
t = n*T;
xbn = sin(w1*t) + 0.5*sin(w2*t);
subplot(426)
stem(t, xbn,'.')
k = 0:N - 1;
Xb = fft(xbn);
subplot(428)
```

```
stem(k, abs(Xb),'.')
```

Ta có nhiều phổ DFT cho cùng một tín hiệu tương tự, tuỳ thuộc vào kích thước DFT, chu kì lấy mẫu, khoảng lấy mẫu và đệm zero. So sánh phổ khi giảm chu kì lấy mẫu T từ 0.1s đến 0.05s

3. Nội suy bằng các dùng biến đổi Fourrier rời rạc: Ta dùng DFS/DFT để nội suy dãy x[n] nhận được từ kết quả lấy mẫu tín hiệu ở khoảng cách cách đều. Thủ tục gồm hai bước: lấy N điểm FFT X(k) của x[n] và dùng công thức:

$$\begin{aligned}\hat{x}(t) &= \frac{1}{N}\sum_{|k|<N/2}\tilde{X}(k)e^{j2\pi kt/NT} \\ &= \frac{1}{N}\left\{X(0) + 2\sum_{k=1}^{N/2-1}\mathrm{Real}\left[X(k)e^{j2\pi kt/NT}\right] + X(N/2)\cos(\pi/T)\right\}\end{aligned} \tag{5}$$

Ta xây dựng hàm nội suy ***interpdfs()***:

```
function [xi,Xi] = interpdfs(T, x, Ws, ti)
%T : chu li lay mau
%x : thu tu roi rac hoa
%Ws: tan so dung chuan (1.0 = pi[rad])
%ti: khoang thoi gian noi suy
if nargin < 4
   ti = 5;
end
if nargin < 3 | Ws > 1
   Ws = 1;
end
N = length(x);
if length(ti) == 1
   ti = 0:T/ti:(N-1)*T; %khoang con duoc chia cho ti
end
ks = ceil(Ws*N/2);
Xi = fft(x);
Xi(ks + 2:N - ks) = zeros(1,N - 2*ks - 1); %pho da loc
xi = zeros(1,length(ti));
for k = 2:N/2
   xi = xi+Xi(k)*exp(j*2*pi*(k - 1)*ti/N/T);
end
xi = real(2*xi+Xi(1)+Xi(N/2+1)*cos(pi*ti/T))/N; %pt.(.5)
```

Để nội suy ta dùng chương trình ***ctfourier.m***:

```
clear, clf
```

```
w1 = pi;
w2 = .5*pi; %hai tan so
N = 32;
n = [0:N - 1];
T = 0.1;
t = n*T;
x = sin(w1*t)+0.5*sin(w2*t)+(rand(1,N) - 0.5); %0.2*sin(20*t);
ti = [0:T/5:(N - 1)*T];
subplot(411), plot(t,x,'k.') %so lieu ban dau
title('So lieu ban dau va ket qua noi suy')
[xi,Xi] = interpdfs(T,x,1,ti);
hold on, plot(ti,xi,'r') %tai tao tin hieu
k = [0:N - 1];
subplot(412), stem(k,abs(Xi),'k.') %pho ban dau
title('Pho ban dau')
[xi,Xi] = interpdfs(T,x,1/2,ti);
subplot(413), stem(k,abs(Xi),'r.') %pho da  loc
title('Pho da loc')
subplot(414), plot(t,x,'k.', ti,xi,'r') %tin hieu da loc
title('Tin hieu da loc')
```

§9. XẤP XỈ HÀM BẰNG PHƯƠNG PHÁP BÌNH PHƯƠNG BÉ NHẤT

1. Khái niệm chung: Trong các mục trước ta đã nội suy giá trị của hàm. Bài toán đó là cho một hàm dưới dạng bảng số và phải tìm giá trị của hàm tại một giá trị của đối số không nằm trong bảng.

Trong thực tế, bên cạnh bài toán nội suy ta còn gặp một dạng bài toán khác. Đó là tìm công thức thực nghiệm của một hàm.

Nội dung bài toán là từ một loạt các điểm cho trước (có thể là các giá trị của một phép đo nào đó) ta phải tìm một hàm xấp xỉ các giá trị đã cho. Ta sẽ dùng phương pháp bình phương tối thiểu để giải bài toán.

Giả sử có mẫu quan sát (x_i, y_i) của hàm $y = f(x)$. Ta chọn hàm $f(x)$ có dạng:

$$f(x) = a_0f_0(x) + a_1f_1(x) + a_2f_2(x)... \quad (1)$$

Trong đó các hàm $f_0(x)$, $f_1(x)$, $f_2(x)$ v.v. là (m+1) hàm độc lập tuyến tính mà ta có thể chọn tuỳ ý và các hệ số a_i là tham số chưa biết mà ta phải xác định dựa vào hệ hàm đã chọn và các điểm quan sát. Sai số giữa trị đo được và trị tính theo (1) là :

$$e_i = y_i - f(x_i) \quad (2)$$

Sai số này có thể âm hay dương tuỳ từng giá trị của y_i. Khi dùng phương pháp bình phương bé nhất ta xét bình phương của sai số tại một điểm:

$$e_i^2 = \left[y_i - f(x_i)\right]^2 \quad (3)$$

Với n điểm tổng bình phương của sai số sẽ là :

$$S = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} \left\{ y_i - \left[a_0 f_0(x_i) + a_1 f_1(x_i) + \cdots + a_m f_m(x_i) \right] \right\}^2$$

Rõ ràng S là hàm của các giá trị cần tìm a_i và chúng ta sẽ chọn các a_i sao cho S đạt giá trị min, nghĩa là các đạo hàm $\frac{\partial S}{\partial a_i}$ phải bằng không.

Ta sẽ xét các trường hợp cụ thể.

2. Hàm xấp xỉ có dạng đa thức: Trong trường hợp tổng quát ta chọn hệ hàm xấp xỉ là một đa thức, nghĩa là:

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_m x^m$$

Vậy hàm S là :

$$S = \left(y_i - a_0 + a_1 x + a_2 x^2 + \cdots + a_m x^m \right)^2$$

Theo điều kiện đạo hàm $\frac{\partial S}{\partial a_i} = 0$ ta nhận được hệ phương trình:

$$\begin{cases} a_m \sum_{i=1}^{n} x_i^m + a_{m-1} \sum_{i=1}^{n} x_i^{m-1} + \cdots + n a_0 = \sum_{i=1}^{n} y_i \\ a_m \sum_{i=1}^{n} x_i^{m+1} + a_{m-1} \sum_{i=1}^{n} x_i^{m} + \cdots + a_0 \sum_{i=1}^{n} x_i = \sum_{i=1}^{n} x_i y_i \\ a_m \sum_{i=1}^{n} x_i^{m+2} + a_{m-1} \sum_{i=1}^{n} x_i^{m+1} + \cdots + a_0 \sum_{i=1}^{n} x_i^2 = \sum_{i=1}^{n} x_i^2 y_i \\ a_m \sum_{i=1}^{n} x_i^{m+3} + a_{m-1} \sum_{i=1}^{n} x_i^{m+2} + \cdots + a_0 \sum_{i=1}^{n} x_i^3 = \sum_{i=1}^{n} x_i^3 y_i \\ \cdots \\ a_m \sum_{i=1}^{n} x_i^{2m} + a_{m-1} \sum_{i=1}^{n} x_i^{2m-1} + \cdots + a_0 \sum_{i=1}^{n} x_i^m = \sum_{i=1}^{n} x_i^m y_i \end{cases}$$

Đây là một hệ phương trình tuyến tính. Giải nó ta nhận được các gía trị a_i.

Ta xây dựng hàm ***polynomfit()*** thực hiện thuật toán trên:

```
function x = polyfits(xData, yData, m)
%Dung de tinh he so cua da thuc xap xi
% Cu phap: x = polyfits(xData, yData, m)
m = m+1;
A = zeros(m);
b = zeros(m, 1);
s = zeros(2*m-1, 1);
for i = 1:length(xData)
   temp = yData(i);
   for j = 1:m
```

```
        b(j) = b(j) + temp;
        temp = temp*xData(i);
    end
    temp = 1;
    for j = 1:2*m-1
        s(j) = s(j) + temp;
        temp = temp*xData(i);
    end
end
for i = 1:m
    for j = 1:m
        A(i, j) = s(i+j-1);
    end
end
x = A\b;
% Sap xep lai he so tu so mu cao nhat
x = flipdim(x, 1);
```

Để xấp xỉ một dãy số liệu bằng hàm đa thức ta dùng chương trình ***ctpolynomfit.m***:

```
clear all, clc
xData = [0 1 2 3 4];
yData = [1  8 24  63 124];
x = polyfits(xData, yData, 3);
y = 0:0.1:4;
z = polyval(x', y);
hold on
plot(y, z,'-b', xData, yData, 'ro');
```

3.Hàm dạng Ae^{cx}: Khi các số liệu thể hiện một sự biến đổi đơn điệu ta dùng hàm xấp xỉ là y = Ae^{cx}. Lấy logarit hai vế ta có :

lny = lnA + cxlne

Theo điều kiện đạo hàm $\frac{\partial S}{\partial a_i} = 0$ ta có hệ phương trình :

$$\begin{cases} c\sum_{i=1}^{n} x_i + n\ln A = \sum_{i=1}^{n} \ln y_i \\ c\sum_{i=1}^{n} x_i^2 + \ln A\sum_{i=1}^{n} x_i = \sum_{i=1}^{n} x_i \ln y_i \end{cases}$$

Giải hệ phương trình này ta có các hệ số A và c.

Ta xây dựng hàm ***expfit()*** để xấp xỉ

```
function [c,A] = expfit(x, y)
a = sum(x);
b = size(x,2);
c = sum(log(y));
d = sum(x.^2);
e = sum(x.*log(y));
d1 = a*a - d*b;
d2 = c*a - e*b;
d3 = a*e - c*d;
c = d2/d1;
A = exp(d3/d1);
```

Ta dùng chương trình ***ctexpfit.m*** để xấp xỉ dãy số liệu đã cho

```
clear all, clc
x = [1.2  2.8  4.3    5.4      6.8    7.9];
y = [7.5       16.1   38.9   67      146.6 266.2];
[c, A] = expfit(x, y);
t = 0:0.1:8;
z = A*exp(c*t);
plot(t, z, '-b', x, y, 'ro');
```

4. Hàm dạng Ax^q: Khi các số liệu thể hiện một sự biến đổi đơn điệu ta cũng có thể dùng hàm xấp xỉ là $y = Ax^q$. Lấy logarit hai vế ta có:

$$\ln y = \ln A + q\ln x$$

Theo điều kiện đạo hàm triệt tiêu ta có hệ phương trình :

$$\begin{cases} q\sum_{i=1}^{n}\ln x_i + n\ln A = \sum_{i=1}^{n}\ln y_i \\ q\sum_{i=1}^{n}\ln^2 x_i + \ln A\sum_{i=1}^{n}\ln x_i = \sum_{i=1}^{n}\ln x_i \ln y_i \end{cases}$$

Giải hệ phương trình này ta có các hệ số A và q.

Ta xây dựng hàm ***powerfit()*** để xấp xỉ:

```
function [q, A] = powerfit(x, y)
a = sum(log(x));
b = size(x, 2);
c = sum(log(y));
d = sum(log(x).^2);
e = sum(log(x).*log(y));
d1 = a*a - d*b;
d2 = c*a - e*b;
d3 = a*e - c*d;
q = d2/d1;
```

```
A = exp(d3/d1);
```

Ta dùng chương trình ***ctpowerfit.m*** để xấp xỉ dãy số liệu đã cho:

```
clc
x = [  1       2       3       4      5];
y = [1.5       15.1      52.5    130.5       253];
[q,A] = powerfit(x, y)
t = 0.1:0.1:5;
z = exp(log(A)+q*log(t));
plot(t, z, '-b', x, y, 'ro');
```

5. Hàm lượng giác: Khi quan hệ y = f(x) có dạng tuần hoàn ta dùng hàm xấp xỉ là tổ hợp tuyến tính của các hàm sin và cosin dạng:

$$f(x) = a_0 + \sum_{i=1}^{n} a_i \cos(i\omega x) + \sum_{i=1}^{n} b_i \sin(i\omega x)$$

Để đơn giản trước hết ta xét hàm chỉ có một số hạng sin-cos, nghĩa là :

$$f(x) = a_0 + a_1 \cos\omega x + b_1 \sin\omega x$$

Hàm S sẽ có dạng :

$$S = \sum_{i=1}^{n} \left[y_i - (a_0 + a_1 \cos\omega x + b_1 \sin\omega x) \right]^2$$

Theo điều kiện đạo hàm triệt tiêu ta có hệ phương trình đối với các hệ số dạng:

$$\begin{bmatrix} n & \sum \cos\omega x_i & \sum \sin\omega x_i \\ \sum \cos\omega x_i & \sum \cos^2\omega x_i & \sum \cos\omega x_i \sin\omega x_i \\ \sum \sin\omega x_i & \sum \cos\omega x_i \sin\omega x_i & \sum \sin^2\omega x_i \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ b_1 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum y_i \cos\omega x_i \\ \sum y_i \sin\omega x_i \end{bmatrix}$$

Do:

$$\frac{\sum \sin\omega x_i}{n} = 0 \qquad \frac{\sum \cos\omega x_i}{n} = 0$$

$$\frac{\sum \sin^2\omega x_i}{n} = \frac{1}{2} \qquad \frac{\sum \cos^2\omega x_i}{n} = \frac{1}{2}$$

$$\frac{\sum \cos\omega x_i \sin\omega x_i}{n} = 0$$

nên hệ phương trình có dạng đơn giản :

$$\begin{bmatrix} n & 0 & 0 \\ 0 & n/2 & 0 \\ 0 & 0 & n/2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ b_1 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum y_i \cos\omega x_i \\ \sum y_i \sin\omega x_i \end{bmatrix}$$

Giải hệ ta có :

$$a_0 = \frac{\sum y_i}{n} \qquad a_1 = \frac{2}{n}\sum y_i \cos\omega x_i \qquad b_1 = \frac{2}{n}\sum y_i \sin\omega x_i$$

Trong trường hợp tổng quát, một cách tương tự ta có:

$$a_0 = \frac{\sum y}{n} \qquad a_i = \frac{2}{n}\sum y \cos i\omega x \qquad b_i = \frac{2}{n}\sum y \sin i\omega x$$

Ta xây dựng hàm ***sinfit()*** để xấp xỉ:

```
function [a, b, c, omega] = sinfit(x, y, T)
%T la chu ki
omega = 2*pi/T;
n = size(x,2);
a = sum(y)/n;
b = (2/n)*sum(y.*cos(omega*x));
c = (2/n)*sum(y.*sin(omega*x));
```

Ta dùng chương trình ***ctsinfit.m*** để tính:

```
c ear all, clc
x = [00.15   0.3     0.45   0.6     0.75   0.9     1.05   1.2     1.3];
y = [2.2       1.595 1.031  0.722  0.786  1.2     1.81   2.369  2.678  2.614];
T = 1.5;
[a, b, c, omega] = sinfit(x, y, T)
t = 0.:0.01:1.5;
z = a + b*cos(omega*t) + c*sin(omega*t);
plot(t, z,'-b', x, y, 'ro');
```

6. Hàm hữu tỉ: Khi quan hệ y = f(x) có dạng đường cong bão hoà hay dạng arctan, tan v.v ta dùng hàm xấp xỉ là hàm hữu tỉ dạng đơn giản:

$$y = \frac{ax}{b+x}$$

Lấy nghịch đảo của nó ta có :

$$\frac{1}{y} = \frac{b}{a}\frac{1}{x} + \frac{1}{a}$$

Đặt 1/y = Y, 1/x = X, b/a = B và 1/a = A phương trình trên sẽ có dạng:

$$Y = A + BX$$

và là một đa thức bậc một. Do vậy ta có hệ phương trình đối với các hệ số A và B là:

$$\begin{cases} nA + B\sum_{i=1}^{n}\frac{1}{x_i} = \sum_{i=1}^{n}\frac{1}{y_i} \\ A\sum_{i=1}^{n}\frac{1}{x_i} + B\sum_{i=1}^{n}\frac{1}{x_i^2} = \sum_{i=1}^{n}\frac{1}{x_i y_i} \end{cases}$$

và từ đó tính được a và b.
Ta xây dựng hàm ***racfit()*** để xấp xỉ:

```
function [a, b] = racfit(x, y)
a1 = size(x, 2);
b1 = sum(1./x);
c1 = sum(1./y);
d1 = sum(1./x.^2);
e1 = sum((1./x).*(1./y));
del = a1*d1 - b1*b1;
del1 = c1*d1 - e1*b1;
del2 = a1*e1 - b1*c1;
A = del1/del;
B = del2/del;
a = 1/A;
b = B/A;
```

Để xấp xỉ ta dùng chương trình ***ctracfit.m***:

```
clear all, clc
x = [1  2  3  4  5];
y = [0.3333333      0.5   0.6      0.66666     0.7142857];
[a, b] = racfit(x, y)
t = 0.:0.01:5;
z = a*t./(b+t)
plot(t, z,'-b', x, y, 'ro');
```

CHƯƠNG 5: CÁC PHƯƠNG TRÌNH PHI TUYẾN

§1. KHÁI NIỆM CHUNG

Nếu phương trình đại số hay siêu việt khá phức tạp thì ít khi tìm được nghiệm đúng. Bởi vậy việc tìm nghiệm gần đúng và ước lượng sai số là rất cần thiết.

Ta xét phương trình :

$f(x) = 0$ (1)

với f(x) là hàm cho trước của biến x. Chúng ta cần tìm giá trị gần đúng của nghiệm của phương trình này.

Quá trình giải thường chia làm hai bước: bước sơ bộ và bước kiện toàn nghiệm.

Bước giải sơ bộ có 3 nhiệm vụ: vây nghiệm, tách nghiệm và thu hẹp khoảng chứa nghiệm.

Vây nghiệm là tìm xem các nghiệm của phương trình có thể nằm trên những đoạn nào của trục x. Tách nghiệm là tìm các khoảng chứa nghiệm sao cho trong mỗi khoảng chỉ có đúng một nghiệm. Thu hẹp khoảng chứa nghiệm là làm cho khoảng chứa nghiệm càng nhỏ càng tốt. Sau bước sơ bộ ta có khoảng chứa nghiệm đủ nhỏ. Để xác định khoảng chứa nghiệm ta có thể dùng phương pháp đồ thị. Ngoài ra ta cũng có thể tìm nghiệm bằng phương pháp tìm tăng dần. Ý tưởng của phương pháp này là nếy $f_1(x).f_2(x) < 0$ thì có ít nhất một nghiệm của phương trình trong đoạn $[x_1, x_2]$. Nếu đoạn $[x_1, x_2]$ đủ nhỏ thì trong đạon đó sẽ có một nghiệm duy nhất. Như vậy ta có thể phát hiện ra nghiệm bằng cách tính trị của hàm trên các đoạn Δx và xem chúng có đổi dấu không.

Ta xây dựng hàm ***rootsearch()*** để tìm khoảng chứa nghiệm.

```
function [x1,x2] = rootsearch(func,a,b,dx)
% Tim doan chua nghiem cua ham f(x).
% Cu phap: [x1,x2] = rootsearch(func,a,d,dx)
% func = ham f(x).
% a,b = daon tim.
% dx = khoang tang
% x1,x2 = doan chu nghiem (a,b);
% dat la NaN neu khong thay nghiem

x1 = a; f1 = feval(func,x1);
x2 = a + dx; f2 = feval(func,x2);
while f1*f2 > 0.0
   if x1 >= b
      x1 = NaN; x2 = NaN;
      return
   end
   x1 = x2; f1 = f2;
   x2 = x1 + dx; f2 = feval(func,x2);
```

end

Khi phát hiện thấy khoảng chứa nghiệm, hàm trả về giá trị biên của đoạn. Nếu không có nghiệm, x1 = x2 = NaN. Ta gọi ***rootsearch()*** nhiều lần để phát hiện hết các đoạn chứa nghiệm. Với ví dụ tìm khoảng chứa nghiệm của hàm $f(x) = x^3 - 10x^2 + 5$ ta dùng chương trình ***ctrootsearch.m***

```
clear all, clc
f = inline('x^3 - 10*x^2 + 5');
[x1, x2] = rootsearch(f,2,10,.2)
```

Bước kiện toàn nghiệm tìm các nghiệm gần đúng theo yêu cầu đặt ra.

Có rất nhiều phương pháp xác định nghiệm của (1). Sau đây chúng ta xét từng phương pháp.

§2. PHƯƠNG PHÁP LẶP ĐƠN

Giả sử phương trình (1) được đưa về dạng tương đương:

$$x = g(x) \qquad (2)$$

từ giá trị x_o nào đó gọi là giá trị lặp đầu tiên ta lập dãy xấp xỉ bằng công thức:

$$x_n = g(x_{n-1}) \qquad (3)$$

với n = 1,2,....

Hàm g(x) được gọi là hàm lặp. Nếu dãy $x_n \to \alpha$ khi $n \to \infty$ thì ta nói phép lặp (3) hội tụ.

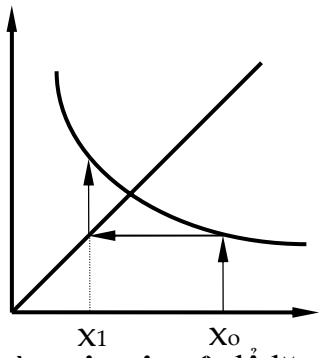
Ta có định lí: Xét phương pháp lặp (3), giả sử:

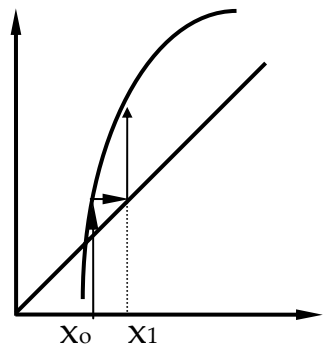
- [a, b] là khoảng chứa nghiệm α của phương trình (1) tức là của (2)
- mọi x_n tính theo (3) đều thuộc [a, b]
- g(x) có đạo hàm thoả mãn :

$$|g'(x)| \le q < 1 \qquad a < x < b \qquad (4)$$

trong đó q là một hằng số thì phương pháp lặp (3) hội tụ

Ta có thể minh hoạ phép lặp trên bằng hình vẽ sau.





Ta xây dựng hàm ***simpiter()*** để lặp

```
function [x, err, xx] = simpiter(g, x0, tolx, maxiter)
```

```
% giai pt x = g(x) tu x0 bang cah lap
%vao : g, x0 = ham va gia tri dau
% tolx = sai so mong muon
% maxiter = so lan lap max
%ra: x = nghiem
% err = sai so |x(k) - x(k - 1)|
% xx = cac gia tri trung gian

if nargin < 4
  maxiter = 100;
end
if nargin < 3
   tolx = 1e-6;
end
xx(1) = x0;
for k = 2:maxiter
   xx(k) = feval(g, xx(k - 1));
   err = abs(xx(k) - xx(k - 1));
   if err < tolx
      break;
   end
end
x = xx(k);
if k == maxiter
   fprintf('Khong hoi tu sau %d lan lap\n', maxiter)
else
    fprintf('Hoi tu sau %d lan lap\n',k)
end
```

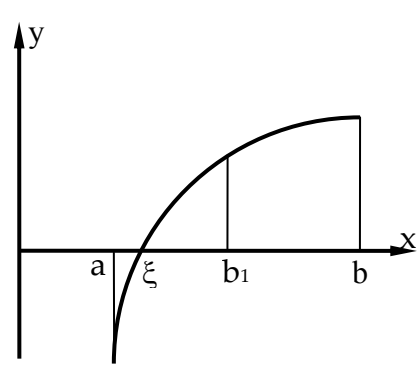
Để tính lại ví dụ trên ta dùng chương trình ***ctsimpiter4_2.m***

```
clear all, clc
f = inline('-0.5*((x - 1).^2 - 3)');
[x, ss, xx] = simpiter(f, 0.5,.00001,200)
```

§3. PHƯƠNG PHÁP CHIA ĐÔI CUNG

Giả sử cho phương trình f(x) = 0 với f(x) liên tục trên đoạn [a, b] và f(a).f(b) < 0. Chia đoạn [a, b] thành 2 phần bởi chính điểm chia (a + b)/2.

1. Nếu f((a+b)/2) = 0 thì ξ = (a+b)/2



2. Nếu f((a + b)/2) ≠ 0 thì chọn [a,(a+b)/2] hay [(a + b)/2, b] mà giá trị hàm hai đầu trái dấu và kí hiệu là $[a_1,b_1]$. Đối với $[a_1, b_1]$ ta lại tiến hành như [a, b].

Ta xây dựng hàm ***bisection()*** thực hiện thuật toán trên

```
function [x,err,xx] = bisection(f, a, b, tolx, maxiter)
%bisection.m de giai pt f(x) = 0 bang phuong phap chia doi cung
%vao: f = ham can tim nghiem
% a/b = bien cua doan can tim nghiem
% tolx = sai so mong muon
% maxiter lan lap max
%ra: x = nghiem
% err = sai so
% xx = cac gia tri trung gian

tol = eps;
fa = feval(f, a);
fb = feval(f, b);
if fa*fb > 0
   error('Nghiem khong o trong doan nay');
end
for k = 1: maxiter
   xx(k) = (a + b)/2;
   fx = feval(f, xx(k));
err = (b - a)/2;
   if abs(fx) < tol | abs(err) < tolx
      break;
   elseif fx*fa > 0
      a = xx(k);
      fa = fx;
   else b = xx(k);
   end
end
x = xx(k);
if k == maxiter
   fprintf('Khong hoi tu sau %d lan lap\n', maxiter),
else
    fprintf('Hoi tu sau %d lan lap\n',k),
end
```

Để tìm nghiệm của hàm f(x) = tg(π -) - x ta dùng chương trình ***ctbisection.m***

```
clear all, clc
f = inline('tan(pi - x) - x');
[x, ss, xx] = bisection(f, 1.6, 3, 1e-4, 50)
```

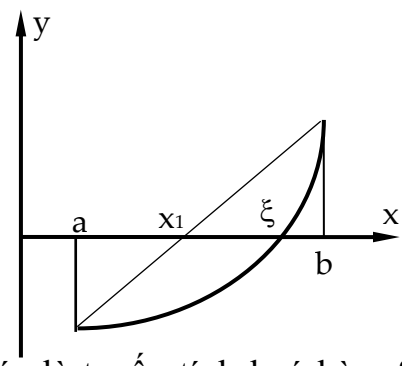
§4. PHƯƠNG PHÁP DÂY CUNG

Giả sử f(x) liên tục trên trên đoạn [a, b] và f(a).f(b) < 0. Cần tìm nghiệm của f(x) = 0. Để xác định ta xem f(a) < 0 và f(b) > 0. Khi đó thay vì chia đôi đoạn [a, b] ta chia [a, b] theo tỉ lệ -f(a)/f(b). Điều đó cho ta nghiệm gần đúng :

$$x_1 = a + h_1$$

Trong đó

$$h_1 = \frac{-f(a)}{-f(a)+f(b)}(b-a)$$



Tiếp theo dùng cách đó với đoạn [a, x_1] hay [x_1, b] mà hai đầu hàm nhận giá trị trái dấu ta được nghiệm gần đúng x_2 v.v.

Về mặt hình học, phương pháp này có nghĩa là kẻ dây cung của đường cong f(x) qua hai điểm A[a, f(a)] và B[b, f(b)] hay nói cách khác là tuyến tính hoá hàm f(x) trong đoạn [a, b].

Thật vậy phương trình dây cung AB có dạng:

$$y = \frac{f(a)-f(b)}{a-b}x + \frac{af(b)-bf(a)}{a-b}$$

Cho x = x_1, y = 0 ta có

$$x_1 = \frac{af(b)-bf(a)}{f(b)-f(a)} \qquad (1)$$

Ta xây dựng hàm ***chord()*** để thực hiện thuật toán trên

```
function [x, err, xx] = chord(f, a, b, tolx, maxiter)
%giai pt f(x) = 0 bg phuong phap day cung.
%vao : f - ham can tim nghiem
% a/b - khoang tim nghiem
% tolx - sai so mong muon cua nghiem
% maxiter lan lap max
%ra: x -  nghiem
% err - sai so
% xx - cac gia tri trung gian
tolfun = eps;
fa = feval(f, a);
fb = feval(f, b);
if fa*fb > 0
   error('Nghiem khong o trong doan nay !');
```

```
end
for k = 1: maxiter
   xx(k) = (a*fb - b*fa)/(fb - fa); %pt.(1)
   fx = feval(f, xx(k));
   err = min(abs(xx(k) - a), abs(b - xx(k)));
   if abs(fx) < tolfun | err<tolx
      break;
   elseif fx*fa > 0
      a = xx(k);
      fa = fx;
   else
      b = xx(k);
      fb = fx;
   end
end
x = xx(k);
if k == maxiter
   fprintf('Khong hoi tu sau  %d lan lap\n', maxiter)
else
    fprintf('Hoi tu sau  %d lan lap\n', k)
end
```

Để tìm nghiệm của hàm f(x) = tg(π -x) - x ta dùng chương trình ***ctchord.m***

```
clear all, clc
f = inline('tan(pi - x) - x');
[x, ss, xx] = falsp(f, 1.7, 3, 1e-4, 50)
```

§5. PHƯƠNG PHÁP NEWTON - RAPHSON

Phương pháp lặp Newton(còn gọi là phương pháp tiếp tuyến)được dùng nhiều vì nó hội tụ nhanh. Tuy nhiên phương pháp này đòi hỏi tính f'(x). Công thức Newton - Raphson được suy từ khai triển Taylor của f(x) lân cận x:

$$f(x_{i+1}) = f(x_i) + f'(x_i)(x_{i+1} - x_i) + O(x_{i+1} - x_i)^2 \qquad (1)$$

Nếu x_{i+1} là nghiệm của phương trình f(x) = 0 thì (1) trở thành:

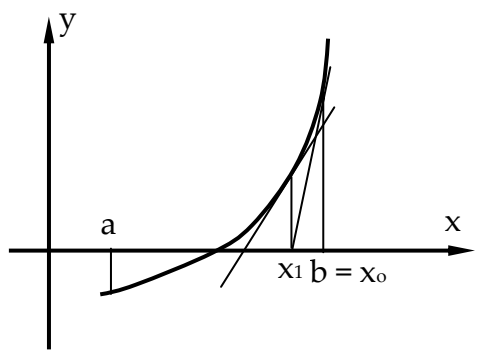
$$0 = f(x_i) + f'(x_i)(x_{i+1} - x_i) + O(x_{i+1} - x_i)^2 \qquad (2)$$

Giả sử rằng x_i gần với x_{i+1}, ta có thể bỏ qua số hạng cuối trong (2) và có công thức Newton - Raphson:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \qquad (3)$$

Nếu x_{i+1} là nghiệm đúng của phương trình thì sai số là $e_i = x - x_i$. Khi nghiệm được tính theo (3) thì sai số là:

$$e_{i+1} = -\frac{f''(x_i)}{2f'(x_i)} e_i^2$$



Minh hoạ hình học của thuật toán Newton - Raphson như hình bên.

Thuật toán được tóm lược như sau:

- cho x_o
- tính $\Delta x = -\frac{f(x)}{f'(x)}$
- cho $x = x + \Delta x$
- lặp lại bước 2 và 3 cho đến khi $|\Delta x| \le \varepsilon$

Ta xây dựng hàm ***newtonraphson()*** để thực hiện thuật toán trên.

```
function [x, fx, xx] = newtonraphson(f, df, x0, tolx, maxiter)
%giai pt f(x) = 0 bang pp Newton-Raphson.
%vao: f = ftn to be given as a string 'f' if defined in an M-file
% df = df(x)/dx (neu khong cho se dung dao ham so.)
% x0 = gia tri ban dau
% tolx = sai so mong muon
% maxiter = so lan lap max
%ra: x = nghiem
% fx = f(x(last)), xx = cac gia tri trung gian
h = 1e-4;
h2 = 2*h;
tolf = eps;
if nargin == 4 & isnumeric(df)
   maxiter = tolx;
   tolx = x0;
   x0 = df;
end
xx(1) = x0;
fx = feval(f,x0);
for k = 1: maxiter
   if ~isnumeric(df)
      dfdx = feval(df, xx(k)); %dao ham cua ham
   else
      dfdx = (feval(f, xx(k) + h)-feval(f, xx(k) - h))/h2; %dao ham so
   end
   dx = -fx/dfdx;
   xx(k+1) = xx(k) + dx; %pt.(3)
   fx = feval(f, xx(k + 1));
   if abs(fx)<tolf | abs(dx) < tolx,
      break;
```

```
    end
end
x = xx(k + 1);
if k == maxiter
    fprintf('Khong hoi tu sau %d lan lap\n', maxiter)
else
     fprintf('Hoi tu sau %d lan lap\n', k)
end
```

Để tính lại nghiệm của hàm cho trong ví dụ trên ta dùng chương trình ***ctnewraph.m***

```
clear all, clc
f = inline('x.^3 - 10*x.^2 + 5');
[x, ss, xx] = newtonraphson(f, 0.7, 1e-4, 50)
```

§6. PHƯƠNG PHÁP CÁT TUYẾN

Phương pháp cát tuyến có thể coi là biến thể của phương pháp Newton - Raphson theo nghĩa đạo hàm được thay bằng xấp xỉ:

$$f'(x) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}} \qquad (1)$$

và tốn ít thời gian tính hơn khi dùng đạo hàm giải tích hay đạo hàm số.

Bằng cách xấp xỉ, biểu thức:

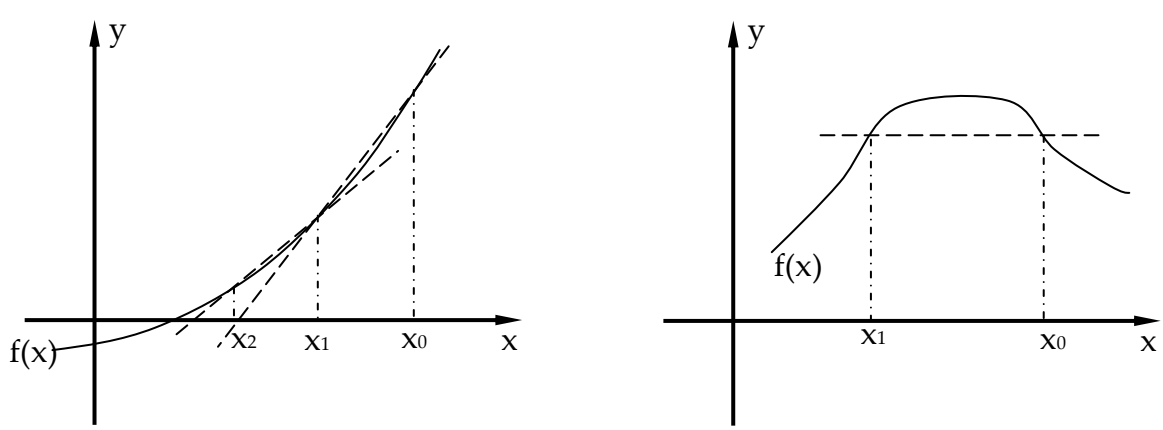
$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

trở thành:

$$x_{k+1} = x_k - f(x_k)\left[\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}\right] = x_k - \frac{f(x_k)}{dfdx_k} \qquad (2)$$

với $\quad dfdx_k = \dfrac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$

Phương trình (2) chính là biểu thức tổng quát của phép lặp. Hai giá trị đầu tiên x_1 và x_2 cần để khởi động phép lặp. Quá trình lặp được minh hoạ bằng hình a



a b

Phép lặp có thể không hội tụ (hình b). Tuy nhiên khi hội tụ, nó hội rất nhanh. Ta xây dựng hàm ***secant()*** để thực hiện thuật toán trên.

```
function [x, fx, xx] = secant(f, x0, x1, tolx, maxiter)
% giai pt f(x) = 0 bg phuong phap day cung
%vao : f - ham can tim nghiem
% x0, x1 - gia tri khoi dong phep lap
% tolx - sai so mong muon
% maxiter - so lan lap max
%ra: x = nghiem
% fx = f(x(last)), xx = cac gia tri trung gian
h = (x1 - x0)/100;
h2 = 2*h;
tolfun = eps;
xx(1) = x0;
fx = feval(f, x0);
for k = 1: maxiter
   if k <= 1
      dfdx = (feval(f,xx(k) + h) - feval(f,xx(k) - h))/h2;
   else
      dfdx = (fx - fx0)/dx;
   end
   dx = -fx/dfdx;
   xx(k + 1) = xx(k) + dx; %pt.(2)
   fx0 = fx;
   fx = feval(f, xx(k+1));
   if abs(fx) < tolfun | abs(dx) < tolx,
      break;
   end
end
x = xx(k + 1);
if k == maxiter
   fprintf('Khong hoi tu sau %d lan lap\n', maxiter)
else
    fprintf('Hoi tu sau %d lan lap\n', k)
end
```

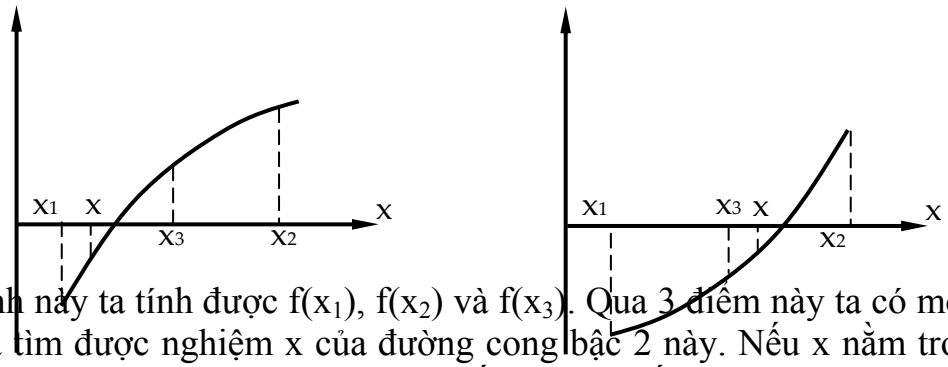
Để tính nghiệm của hàm $f(x) = x^3 - 10x^2 + 5$ ta dùng chương trình ***ctsecant.m***

```
clear all, clc
f = inline('x.^3 - 10*x.^2 + 5');
```

[x, ss, xx] = secant(f, 0.5, 1, 1e-4, 50)

§7. PHƯƠNG PHÁP BRENT

Phương pháp Brent kết hợp phương pháp chia đôi cung và phương pháp nội suy bậc hai để tạo ra một phương pháp tìm nghiệm của phương trình f(x) = 0 rất hiệu quả. Phương pháp này dùng khi đạo hàm của f(x) khó tính hay không thể tính được. Giả sử ta cần tìm nghiệm trong đoạn $[x_1, x_2]$. Quá trình tìm nghiệm bắt đầu bằng việc chia đôi đoạn $[x_1, x_2]$ bằng điểm x_3.



Trong quá trình này ta tính được $f(x_1)$, $f(x_2)$ và $f(x_3)$. Qua 3 điểm này ta có một đường cong bậc 2 và tìm được nghiệm x của đường cong bậc 2 này. Nếu x nằm trong đoạn $[x_1, x_2]$ như hình trên thì giá trị này được chấp nhận. Tiếp theo ta tìm nghiệm trong đoạn $[x_1, x_3]$ hay $[x_3, x_2]$ tuỳ theo vị trí của x.

Đa thức nội suy Lagrange qua 3 điểm là:

$$x(y) = \frac{(f-f_2)(f-f_3)}{(f_1-f_2)(f_1-f_3)}x_1 + \frac{(f-f_1)(f-f_3)}{(f_2-f_1)(f_2-f_3)}x_2 + \frac{(f-f_1)(f-f_2)}{(f_3-f_1)(f_3-f_2)}x_3$$

Cho y = 0 ta có:

$$x = -\frac{f_2f_3x_1(f_2-f_3) + f_1f_3x_2(f_3-f_1) + f_1f_2x_3(f_1-f_2)}{(f_1-f_2)(f_2-f_3)(f_3-f_1)} \qquad (1)$$

Độ thay đổi của nghiệm là:

$$\Delta x = x - x_3 = f_3\frac{x_3(f_1-f_2)(f_2-f_3+f_1) + f_2x_1(f_2-f_3) + f_1x_2(f_3-f_1)}{(f_1-f_2)(f_2-f_3)(f_3-f_1)} \qquad (2)$$

Ta xây dựng hàm ***brent()*** để thực hiện thuật toán trên

```
function root = brent(f, a, b, tol)
% giai pt f(x) = 0 bang thuat toan Brent.
% Cu phap: root = brent(f, a, b, tol)
% vao: f = ham can tim nghiem
% a, b = doan chua nghiem
% tol = sai so cho truoc
if nargin < 4;
   tol = 1.0e6*eps;
end
% lan chia doi dau tien
x1 = a;
```

```
f1 = feval(f,x 1);
if f1 == 0;
   root = x1;
   return;
end
x2 = b;
f2 = feval(f, x2);
if f2 == 0;
   root = x2;
   return;
end
if f1*f2 > 0.0
   error('Nghiem khong nam trong doan nay')
end
x3 = 0.5*(a + b);
% bat dau lap.
for i = 1:30
   f3 = feval(f, x3);
   if abs(f3) < tol
      root = x3;
      return
   end
% xac dinh doan chua nghiem.
   if f1*f3 < 0.0;
      b = x3;
   else
      a = x3;
   end
   if (b - a) < tol*max(abs(b),1.0)
      root = 0.5*(a + b);
    return
   end
% noi suy bac 2.
   denom = (f2 - f1)*(f3 - f1)*(f2 - f3);
   numer = x3*(f1 - f2)*(f2 - f3 + f1)...
         + f2*x1*(f2 - f3) + f1*x2*(f3 - f1);
   if denom == 0;
      dx = b - a;
   else
      dx = f3*numer/denom;
   end
   x = x3 + dx;
%neu lap ra ngoai doan (a,b), dung cach chia doi cung
```

```
    if (b - x)*(x - a) < 0.0
        dx = 0.5*(b - a);
        x = a + dx;
    end
% cho x = x3 & chon x1, x2 moi sao cho  x1 < x3 < x2.
    if x < x3
        x2 = x3;
        f2 = f3;
    else
        x1 = x3;
        f1 = f3;
    end
    x3 = x;
end
root = NaN;
```

Để tìm nghiệm của phương trình x|cos(x)| - 1 = 0 ta dùng chương trình ***ctbrent.m***

```
clear all, clc
f = inline('x.*abs(cos(x)) - 1');
x = brent(f, 0.0, 4,1e-4)
```

§8. PHƯƠNG PHÁP NGOẠI SUY AITKEN

Xét phương pháp lặp:

$$x = f(x) \qquad (1)$$

với f(x) thoả mãn điều kiện hội tụ của phép lặp, nghĩa là với mọi $x \in [a, b]$ ta có:

$$| f'(x) | \le q < 1 \qquad (2)$$

Như vậy :

$$x_{n+1} = f(x_n) \qquad (3)$$

$$x_n = f(x_{n-1}) \qquad (4)$$

Trừ (3) cho (4) và áp dụng định lí Lagrange cho vế phải với $c \in [a, b]$ ta có :

$$x_{n+1} - x_n = f(x_n) - f(x_{n-1}) = (x_n - x_{n-1})f'(c) \qquad (5)$$

Vì phép lặp (1) nên :

$$| x_{n+1} - x_n | \le q | x_n - x_{n-1} | \qquad (6)$$

Do (6) đúng với mọi n nên cho n = 1 , 2 , 3 , . . . ta có :

$$| x_2 - x_1 | \le q | x_1 - xo |$$

$$| x_3 - x_2 | \le q | x_2 - x_1 |$$

. . . . . .

$$| x_{n+1} - x_n | \le q | x_n - x_{n-1} |$$

Điều này có nghĩa là dãy $x_{i+1} - x_i$, một cách gần đúng, là một cấp số nhân. Ta cũng coi rằng dãy x_n - y với y là nghiệm đúng của (1), gần đúng như một cấp số nhân có

công sai q . Như vậy :

$$\frac{x_{n+1} - y}{x_n - y} = q < 1 \qquad (7)$$

hay : $x_{n+1} - y = q(x_n - y)$ (8)

Tương tự ta có :

$$x_{n+2} - y = q(x_{n+1} - y) \qquad (9)$$

Từ (8) và (9) ta có :

$$q = \frac{x_{n+2} - x_{n+1}}{x_{n+1} - x_n} \qquad (10)$$

Thay giá trị của q vừa tính ở (10) vào biểu thức của q ở trên ta có :

$$y = x_n - \frac{(x_n - x_{n+1})^2}{x_n - 2x_{n+1} + x_{n+2}} \qquad (11)$$

Công thức (11) được gọi là công thức ngoại suy Adam. Như vậy theo (11) trước hết ta dùng phương pháp lặp để tính giá trị gần đúng x_{n+2}, x_{n+1}, x_n của nghiệm và sau đó theo (11) tìm được nghiệm với sai số nhỏ hơn.

Khi phương pháp lặp được kết hợp với phương pháp Aitken ta có phương pháp Steffensen. Bắt đầu lặp từ x_0, hai bước lặp được dùng để tính:

$x_1 = f(x_0)$ $\qquad$ $x_2 = f(x_1)$

và sau đó dùng thuật toán Aitken để tinh y_0 theo (11). Để tiếp tục lặp ta cho $x_0=y_0$ và lặp lại bước tính trước.

Ta xây dựng hàm ***aitstef()*** để thực hiện hai thuật toán trên

```
function [x, y] = aitstef(g, x0, tolx, maxiter)
% phuong phap Aitken - Steffenson
% giai pt x = g(x)
xstart = x0;
f0 = 0;
f0old = 1.;
count = 0;
while ((count < maxiter) & (abs(f0 - f0old) > .0000001))
        count = count + 1;
        f0old = f0;
        x1 = feval(g, x0);
   x2 = feval(g, x1);
        f0 = x0 - (x1 - x0)*(x1 - x0) / (x2 - 2.*x1 + x0);
        x0 = x1;
end
x = f0;

fprintf('\n');
fprintf('Phuong phap Aitken-Steffenson');
```

```
x0 = xstart;
count = 0;
f0 = 0;
x2 = 1.;
while ((count < maxiter) & (abs(f0 - x2) > .0000001))
        count = count+1;
        x1 = feval(g, x0);
   x2 = feval(g, x1);
        f0 = x0 - (x1 - x0)*(x1 - x0) / (x2 - 2.*x1 + x0);
        x0 = f0;
end
y = f0;
```

Để tìm nghiệm của phương trình $x = (2 - e^x + x^2)/3 = g(x)$ ta dùng chương trình ***ctaitstef.m***

```
clear all, clc
f = inline('(2.-exp(x)+x.^2)/3');
[x, y] = aitstef(f,0., 1e-4, 50)
```

§9. PHƯƠNG PHÁP MUELLER

Trong phương pháp dây cung khi tìm nghiệm trong đoạn [a, b] ta xấp xỉ hàm bằng một đường thẳng. Tuy nhiên để giảm lượng tính toán và để nghiệm hội tụ nhanh hơn ta có thể dùng phương pháp Muller. Nội dung của phương pháp này là thay hàm trong đoạn [a, b] bằng một đường cong bậc 2 mà ta hoàn toàn có thể tìm nghiệm chính xác của nó.

Gọi các điểm đó có hoành độ lần lượt là $a = x_2$, $b = x_1$ và ta chọn thêm một điểm x_0 nằm trong đoạn $[x_2, x_1]$. Gọi

$h_1 = x_1 - x_0$

$h_2 = x_0 - x_2$

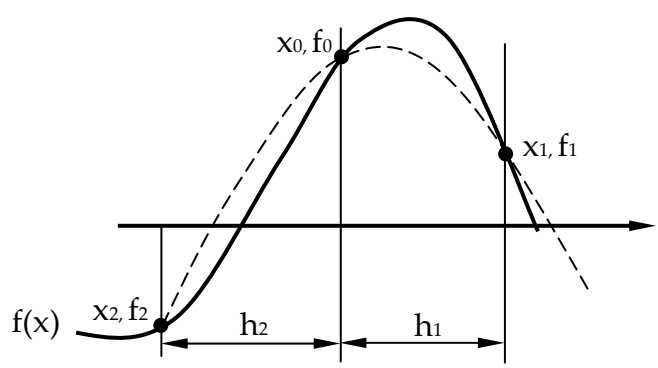
$v = x - x_0$

$f(x_0) = f_0$

$f(x_1) = f_1$

$f(x_2) = f_2$

$\gamma = \dfrac{h_2}{h_1}$



Qua 3 điểm này ta có một đường parabol:

$y = av^2 + bv + c$

Ta tìm các hệ số a, b, c từ các giá trị đã biết v:

$$v = 0 \ (x = x_0) \qquad a(0)^2 + b(0) + c = f_0$$

$$v = h_1 (x = x_1) \qquad ah_1^2 + bh_1 + c = f_1$$

$$v = -h_2 (x = x_2) \qquad ah_2^2 - bh_2 + c = f_2$$

Từ đó ta có :

$$a = \frac{\gamma f_1 - f_0(1+\gamma) + f_2}{\gamma h_1^2 (1+\gamma)}$$

$$b = \frac{f_1 - f_0 - ah_1^2}{h_1}$$

$$c = f_0$$

Sau đó ta tìm nghiệm của phương trình $av^2 + bv + c = 0$ và có :

$$n = x_0 - \frac{2c}{b \pm \sqrt{b^2 - 4ac}}$$

Dấu của mẫu số được chọn sao cho nó có giá trị tuyệt đối lớn nhất, nghĩa là khi $b > 0$, ta lấy dấu +, khi $b < 0$ ta lấy dấu -.

Tiếp đó ta chọn x_0 làm một trong 3 điểm để tính xấp xỉ mới. Các điểm này được chọn gần nhau nhất, nghĩa là nếu nghiệm n ở bên phải x_0 thì ba điểm tính mới là x_0, x_1 và n; nếu n nằm bên trái x_0 thì 3 điểm tính mới là x_0, x_2 và nghiệm. Tiếp tục quá trình tính đến khi đạt độ chính xác yêu cầu thì dừng lại.

Ta xây dựng hàm ***muller()*** để thực hiện thuật toán trên

```
function p = muller(f, a, b, maxiter)
% giai pt f(x) = 0
%vao - f la ham can tim nghiem
%      -  a, b la doan chua nghiem
%      - maxiter so lan lap max
%ra   - p xap xi Muller cua f
%      - y la gia tri y = f(p)
%      - err sai so thuc cua nghiem.
%khoi gan a,b,x0 va cac gia tri tuong ung cua ham
x0 = (a + b)/2.;
P = [x0 a b];
Y = feval(f, P);
delta = 1e-4;
%tinh cac he so cua pt bac hai
for k = 1:maxiter
  h0 = P(1) - P(3);
  h1 = P(2) - P(3);
  e0 = Y(1) - Y(3);
  e1 = Y(2) - Y(3);
  c = Y(3);
```

```
    denom = h1*h0^2 - h0*h1^2;
    a = (e0*h1 - e1*h0)/denom;
    b = (e1*h0^2 - e0*h1^2)/denom;
    %neu nghiem phuc
    if b^2-4*a*c > 0
      disc = sqrt(b^2 - 4*a*c);
    else
      disc = 0;
    end
    %tim nghiem nho nhat
    if b < 0
      disc = -disc;
    end
    z = -2*c/(b + disc);
    p = P(3) + z;
   %sap xep lai cac tri tinh lap
    if abs(p - P(2)) < abs(p  -P(1))
      Q = [P(2) P(1) P(3)];
      P = Q;
      Y = feval(f, P);
    end
    if abs(p-P(3)) < abs(p-P(2))
      R = [P(1) P(3) P(2)];
      P = R;
      Y = feval(f, P);
    end
    %cac tri tinh lan moi
    P(3) = p;
    Y(3) = feval(f, P(3));
    y = Y(3);
    %dieu kien dung lap
    err = abs(z);
    relerr = err/(abs(p) + delta);
    if (err < delta)|(relerr < delta)|(abs(y) < eps)
      break
    end
  end
```

Để giải phương trình sin(x) - 0.5*x = 0 ta dùng chương trình ***ctmuller.m***

```
clear all, clc
format long
f = inline('sin(x) - 0.5*x');
```

```
x = muller(f, 1.8, 2.2, 50)
```

§10. PHƯƠNG PHÁP HALLEY

Phép lặp Newton tìm nghiệm của hàm phương trình x = g(x) là:

$$g(x) = x - \frac{f(x)}{f'(x)} \qquad (1)$$

Tốc độ hội tụ tăng đáng kể khi hàm có nghiệm đơn. Để tăng tốc độ hội tụ, Edmon Halley đưa ra công thức lặp:

$$h(x) = x - \frac{f(x)}{f'(x)}\left\{1 - \frac{f(x)f''(x)}{2\left[f'(x)\right]^2}\right\}^{-1} \qquad (2)$$

Ta xây dựng hàm ***halley1()*** để thực hiện thuật toán trên

```
function x = halley1(f, x, maxiter)
%ham dung de tim nghiem cua pt f(x) = 0
%vao:     -  ham can tim nghiem f
%         -  gia tri dau x0
%         - so lan lap cuc dai maxiter
%ra       - nghiem x
% dung dao ham so
i = 0;
h = 1e-4;
while (i < maxiter)
   f1 = feval(f, x);
   df1 = (feval(f, x + h)-feval(f, x - h))/(2*h);
   ddf1 = (feval(f, x + h)-2*feval(f, x)+feval(f, x - h))/(h*h);
   hx = x - (f1/df1)*1./(1 - (f1*ddf1)/(2*(df1)^2))
   x = hx;
   i = i + 1;
   if (abs(f1) < eps)
      break;
   end
end
```

hay dùng hàm ***halley2()***

```
function x = halley2(f, x, maxiter)
%ham dung de tim nghiem cua pt f(x) = 0
%vao:     -  ham can tim nghiem f
%         -  gia tri dau x
%         - so lan lap cuc dai maxiter
%ra       - nghiem x
% dung dao ham symbolic
```

```
df = diff(f, x);
ddf = diff(f, x, 2);
i = 0;
while (i < maxiter)
   f1 = eval(f);
```

```
    df1 = eval(df);
    ddf1 = eval(ddf);
    hx = x - (f1/df1)*1./(1 - (f1*ddf1)/(2*(df1)^2));
    x = hx;
    i = i + 1;
    if (abs(f1) < eps)
        break;
    end
end
```

Để giải phương trình $f(x) = x^3 - 3x + 2 = 0$ ta dùng chương trình ***cthalley.m***:

```
clc, clear all
%f = inline('x.^3 - 3*x + 2');%khi dung halley1()
%x = halley1(f, -3, 50);
syms x
f = x^3 - 3*x + 2;%khi dung halley2()
x = halley2(f, -3, 50)
```

§11. PHƯƠNG PHÁP CHEBYSHEV

Khi tìm nghiệm của phương trình đại số tuyến tính hay phương trình siêu việt f(x) = 0 ta có thể dùng một hàm có 4 thông số để xấp xỉ hàm f(x)

$$y(x) = p_1 + p_2(x - p_3)^e \qquad (1)$$

Các thông số p_1 và p_3 tạo sự chuyển dịch theo các trục; thông số p xác định biên độ và e cung cấp độ cong của hàm.

Ta khảo sát hàm f(x) trên đoạn [a, b] trong đó f(a).f(b) < 0, nghĩa là trong đoạn [a, b] tồn tại nghiệm của phương trình f(x) = 0. Ta có thêm điều kiện $f'(x).f''(x) \neq 0\ \forall x \in [a, b]$. Gọi $x_i \in [a, b]$ là lần xấp xỉ thứ i của nghiệm thì nghiệm lần thứ i + 1 theo công thức Popovski là:

$$x_{i+1} - x_i = (e-1)\frac{f'}{f''}\left[\left(1 - \frac{e}{e-1}\frac{f.f''}{f'^2}\right)^{\frac{1}{e}} - 1\right] \qquad (2)$$

Khi e = -1

$$x_{i+1} - x_i = \frac{f.f'}{0.5f.f'' - f'^2}$$

và đó là phép lặp Halley

Khi $e \to 1$:

$$x_{i+1} - x_i = -\frac{f}{f'}$$

và đó là phép lặp Newton

Khi e = 0.5

$$x_{i+1} - x_i = \frac{-f\left(\frac{1+0.5f.f''}{f'^2}\right)}{f'} = -\frac{f(x_i)}{f'^3(x_i)} - \frac{f^2(x_i)\times f''(x_i)}{2f'^3(x_i)}$$

và ta có phép lặp Chebyshev.

Ta xây dựmg hàm ***chebyiter()*** để thực hiện thuật toán trên

```
function [x, fx, xx] = chebyiter(f, x0, tol, maxiter)
%giai pt f(x) = 0 bang pp Chebyshev.
%vao: f = ham can tim nghiem
% x0 = gia tri ban dau
% tol = sai so mong muon
% maxiter = so lan lap max
%ra: x = nghiem
% fx , xx = cac gia tri trung gian
if nargin < 4
   maxiter = 200;
end
if nargin < 3
   maxiter = 100;
   tol = 1e-4;
end
h = 1e-4;
h2 = 2*h;
xx(1) = x0;
fx = feval(f, x0);
for k = 1:maxiter
   df =  (feval(f, xx(k) + h) - feval(f, xx(k) - h))/h2; %dao ham so
   d2f = (feval(f, xx(k) + h) - 2*feval(f, xx(k)) + feval(f, xx(k) - h))/h^2;
   dx = - fx/df^3 - 0.5*fx^2*d2f/df^3;
   xx(k+1) = xx(k) + dx;
   fx = feval(f, xx(k+1));
   if abs(fx) < tol | abs(dx) < tol
      break;
   end
end
x = xx(k + 1);
```

Để giải phương trình ta dùng chương trình ***ctchebyiter***

```
clear all, clc
f = inline('x.^3 - 10*x.^2 + 5');
x = chebyiter(f, -3, 1e-4)
```

§12. PHƯƠNG PHÁP NEWTON DÙNG CHO HỆ PHI TUYẾN

Phương pháp Newton có thể được tổng quát hoá để giải hệ phương trình phi tuyến dạng :

$$\begin{cases} f_1(x_1, x_2, x_3, ..., x_n) = 0 \\ f_2(x_1, x_2, x_3, ..., x_n) = 0 \\ \ldots\ldots \\ f_n(x_1, x_2, x_3, ..., x_n) = 0 \end{cases}$$

hay viết gọn hơn dưới dạng :

$F(X) = 0$

Trong đó :

$X = (x_1, x_2, x_3, ..., x_n)$

Với một phương trình một biến, công thức Newton là :

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

hay :

$f'(x_i).\Delta x = -f(x_i)$

với $\Delta x = x_{i+1} - x_i$

Đối với hệ phương trình, công thức lặp là :

$J(X_i)\Delta X = -F(X_i)$

Trong đó $J(X_i)$ là toán tử Jacobi. Nó là một ma trận bậc n (n - tương ứng với số thành phần trong vectơ X) có dạng :

$$J(X_i) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} & \ldots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} & \ldots & \frac{\partial f_2}{\partial x_n} \\ \ldots & & & \ldots & \\ \ldots & & & \ldots & \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \frac{\partial f_n}{\partial x_3} & \ldots & \frac{\partial f_n}{\partial x_n} \end{pmatrix}$$

và $\Delta X = X_{i+1} - X_i$

Phương pháp Newton tuyến tính hoá hệ và như vậy với mỗi bước lặp cần giải một hệ phương trình tuyến tính (mà biến là ΔX_i) xác định bởi công thức lặp cho tới khi vectơ $X(x_1, x_2, x_3, ..., x_n)$ gần với nghiệm.

Ta xây dựng hàm ***new4sys()*** để thực hiện thuật toán này

```
function [P, iter, err] = new4sys(f, jf, P, max1)
%vao    -F la he pt luu trong  M-file f.m
%       -JF la ma tran jacobi luu trong M-file jf.m
```

```
%         -P vec to nghiem ban dau
%         -max1 so lan lap cuc dai
%ra      -P la ve to nghiem
%         -iter so lan lap thuc te
%         -err sai so
Y = f(P);
for k = 1:max1
   J = jf(P);
   Q = P - (J\Y')';
   Z = f(Q);
   err = norm(Q - P);
   relerr = err/(norm(Q) + eps);
   P = Q;
   Y = Z;
   iter = k;
   if (err<eps)|(relerr<eps)|(abs(Y)<eps)
     break
   end
end
```

Để giải hệ phương trình:

$$\begin{cases} x^2 + xy - 2 = 0 \\ 2xy + y^2 - 3 = 0 \end{cases}$$

ta dùng chương trình ***ctnew4sys.m***

```
clear all, clc
format long
p = [.5, .5];
[n ,ll, ss] = new4sys(@f, @jf, p, 50)
```

Nội dung của ***f.m***:

```
function f = f(p)
f =  [(p(1)^2 + p(1)*p(2) - 2), (2*p(1)*p(2) + p(2)^2 - 3)];
```

Nội dung của ***jf.m***:

```
function jf = jf(p)
jf = [(2*p(1) + p(2))    p(1)
      (2*p(1))          (2*p(1) + 2*p(2))];
```

Ta có thể dùng hàm ***new4sys2()*** để thực hiện thuật toán:

```
function root = new4sys2(func, x, maxiter)
% Phuong phap Newton-Raphson de tim nghiem
% cua he pt fi(x1,x2,...,xn) = 0, i = 1, 2,..., n.
% Cu phap: root = new4sys2(func, x, tol)
% vao:
% func = con tro ham, tra ve [f1, f2,..., fn].
% x = vec to ban dau [x1, x2,..., xn].
% tol = sai so mong muon
% ra:
% root - nghiem cua he

if size(x,1) == 1;
   x = x';
end % x phai la vec to
i = 0;
while (i < maxiter)
   [jac, f0] = jacobi(func, x);
   dx = jac\(-f0);
   x = x + dx;
   root = x;
   if max(abs(dx)) < eps
      break;
   else
       i = i + 1;
   end
end
if i == maxiter
  fprintf('Khong hoi tu sau %d lan lap\n', maxiter);
else
  fprintf('Hoi tu sau %d lan lap\n', i);
end
```

Hàm ***jacobi()*** gồm các lệnh:

```
function [jac, f0] = jacobi(func, x)
% Tinh ma tran Jacobi va ham f(x).
h = 1.0e-4;
n = length(x);
jac = zeros(n);
f0 = feval(func, x);
for i =1:n
    temp = x(i);
    x(i) = temp + h;
```

```
    f1 = feval(func, x);
    x(i) = temp;
    jac(:,i) = (f1 - f0)/h;
end
```

Hàm ***t()*** gồm các lệnh:

```
function x = t(p)
x = [(p(1)^2 + p(2)^2 + p(3)^2 - 14)
     (p(1)^2 + 2*p(2)^2 - p(3) - 6
     (p(1) -3*p(2 )^2 + p(3)^2 + 2)];
```

Để giải hệ phương trình ta dùng chương trình ***ctnew4sys2.m***:

```
clear all, clc
format long
p = [1  1  1 ];
r = new4sys2(@t, p, 50)
```

§13. PHƯƠNG PHÁP BROYDEN DÙNG CHO HỆ PHI TUYẾN

1. Phương pháp Broyden: Để giải hệ phương trình phi tuyến tính F([X]) = [0] bằng phương pháp lặp Newton ta cho vec tơ nghiệm ban đầu $[P_0]$ và tạo ra dãy $[P_k]$ hội tụ về nghiệm [P], nghĩa là F([P]) = [0]. Khi này ta cần tính ma trận Jacobi của hệ. Việc tính ma trận Jacobi đòi hỏi tính n^2 đạo hàm riêng.

Đạo hàm của hàm f(x) tại p_k có thể tính gần đúng bằng công thức:

$$f'(p_k) = \frac{f(p_k) - f(p_{k-1})}{p_k - p_{k-1}}$$

nên:

$$f'(p_k)(p_k - p_{k-1}) = f(p_k) - f(p_{k-1})$$

Mở rộng cho hệ phương trình ta có thể viết:

$$J([P_k])([P_k]-[P_{k-1}]) = F([P_k]) - F([P_{k-1}])$$

Phương pháp Broyden bắt đầu bằng việc tính ma trận Jacobi $A_0 = J([P_0])$. Sau đó trong quá trình lặp liên tiếp ta dùng ma trận Jacobi được cập nhật A_k:

$$A_k([P_k] - [P_{k-1}]) = F([P_k]) - F([P_{k-1}])$$

Thuật toán Broyden như vậy bao gồm các bước:

<u>*Bước 0*</u>: Tính ma trận Jacobi ban đầu $A_0 = J([P_0])$.

Sử dụng nó để tính lần lặp đầu tiên theo phương pháp Newton

$$[P_1] = [P_0] - (A_0)^{-1}F([P_0])$$

Với $k \geq 1$, giả sử đã biết $[P_k]$ sử dụng các bước sau tính $[P_{k+1}]$

<u>*Bước 1*</u>: Tính hàm $F([P_k]) = \begin{bmatrix} f_1(p_k, q_k) \\ f_2(p_k, q_k) \end{bmatrix}$

<u>*Bước 2*</u>: Cập nhật ma trận Jacobi bằng cách dùng

$$[S] = [P_k] - [P_{k-1}] \text{ và } [Y_k] = F([P_k]) - F([P_{k-1}])$$

và có:

$$[A_k] = [A_{k-1}] + \frac{1}{[S]^T[S]}\left([Y_k] - [A_{k-1}][S]\right)[S]^T$$

<u>*Bước 3*</u>: Giải hệ phương trình tuyến tính $[A_k]\Delta[P_k] = -F([P_k])$ để tìm $\Delta[P_k]$

<u>*Bước 4*</u>: Lặp tiếp với $[P_{k+1}] = [P_k] + \Delta[P_k]$

Để thực hiện thuật toán này ta xây dựng hàm ***broyden()***

```
function root = broyden(g, p0, maxiter)
if nargin == 2
   maxiter = 50;
end
if size(p0, 1) == 1;
   p0 = p0';
end
[a0, f0]  = jacobi(g, p0);
p1 = p0 - inv(a0)*f0;
i = 0;
while i < maxiter
   s = p1 - p0;
   d = (s')*s;
   f1 = feval(g, p1);
   y = feval(g, p1) - feval(g, p0);
   m = y - a0*s;
   a1 = a0 - (1/d)*(y - a0*s)*s';
   dp = -inv(a1)*f1;
   p2 = p1 + dp;
   err = max(abs(dp));
   if err < eps
      root = p2;
      break;
   end
   p0 = p1;
   p1 = p2;
   f0 = f1;
   a0 = a1;
   i = i + 1;
end
```

Để giải hệ phương trình

$$\begin{cases} 2x^2 - 2y^3 - 4x + 1 = 0 \\ x^4 + 4y^4 + 4y - 4 = 0 \end{cases}$$

ta dùng chương trình ***ctbroyden.m***

```
clear all, clc
format long
p = [.1 .7 ];
root = broyden(@g, p, 50)
```

Ta có thể dùng hàm ***broyden1( )***:

```
function [t, k] = broyden1(fcn1, fcn2, x0, maxits, tol)
%  Tim nghiem cua he pt phi tuyen
%  cu phap [t, k] = broyden1(fcn1, fcn2, x0, maxits, tol)
%  vao
%          - fcn1: ham thu nhat
%          - fcn2: ham thu hai
%          - x0: nghiem ban dau
%          - maxiter: so lan lap max
%          - tol sai so mong muon
% ra
%          - x: nghiem
%          - k: so lan lap

tol = 1e-8;
maxiter = 50;
if size(x0, 1) == 1
   x0 = x0';
end
syms x  y
B = [diff(fcn1, x) diff(fcn1, y);diff(fcn2, x) diff(fcn2, y)];
x = x0(1);
y = x0(2);
h = inline(fcn1);
g = inline(fcn2);
f(1:2,1) = [h(x, y);g(x,  y)];
B = eval(B);
t = [x  y]';
for k = 1:maxiter
   s = B\(-f);
   t = t + s;
   fnew = [h(t(1), t(2));g(t(1), t(2))];
```

```
    u = fnew-f;
    if abs(fnew-f) < tol
        break
    end
    f = fnew;
    B = B + ((u - B*s)*s')/(s'*s);
end
```

và dùng chương trình ***ctbroyden1.m***

```
clc, clear all
syms x  y
f1 = 2*x^2 - 2*y^3 - 4*x + 1;
f2 = x^4 + 4*y^4 + 4*y - 4;
[n, l] = broyden1(f1, f2, [.1  .7 ], 50)
```

Ngoài ra ta có một phiên bản khác là hàm ***broyden2()***:

```
function [t, k] = broyden2(f, x0, maxiter, tol)
%  Tim nghiem cua he pt phi tuyen
%  cu phap [t, k] = broyden2(fcn1, fcn2, x0, maxits, tol)
%  vao
%          - fcn1: ham thu nhat
%          - fcn2: ham thu hai
%          - x0: nghiem ban dau
%          - maxiter: so lan lap max
%          - tol sai so mong muon
% ra
%          - x: nghiem
%          - k: so lan lap

tol = eps;
maxiter = 50;
if size(x0, 1) == 1
    x0 = x0';
end
syms x  y
B = [diff(f(1), x) diff(f(1), y);
       diff(f(2), x) diff(f(2), y)];
x = x0(1);
y = x0(2);
h = inline(f(1));
g = inline(f(2));
```

```
f(1:2,1) = double( [h(x, y);g(x,  y)]);
B = double(eval(B));
t = [x  y]';
for k = 1:maxiter
   s = double(B\(-f));
   t = t + s;
   fnew = [h(t(1),t(2));
           g(t(1),t(2))];
   u = fnew - f;
   if abs(double(u)) < tol
      break
   end
   f = fnew;
   B = B + ((u - B*s)*s')/(s'*s);
end
```

và dùng với chương trình ***ctroyden2.m***

```
clc, clear all
syms x  y
f = [ 2*x^2 - 2*y^3 - 4*x + 1;
      x^4 + 4*y^4 + 4*y - 4];
[n, l] = broyden2(f, [.7  .7 ])
```

2. Phương pháp Broyden cải tiến: Ma trận nghịch đảo tính toán rất phức tạp. Vì vậy ta dùng công thức Sherman - Morison để giảm bớt khối lượng tính toán khi nghịch đảo ma trận.

Nếu $[A]^{-1}$ không suy biến và [U], [V] là 2 vec tơ sao cho $[V]^T[A]^{-1}[U] \neq -1$ thì:

$$\left([A]+[U][V]^T\right)^{-1} = [A]^{-1} - \frac{\left([A]^{-1}[U][V]^T[A]^{-1}\right)}{\left(1+[V]^T[A]^{-1}[U]\right)[A]^{-1}}$$

hay

$$\left([A]+[U][V]^T\right)^{-1} = \left\{[E] - \frac{\left([A]^{-1}[U]\right)[V]^T}{\left(1+[V]^T[A]^{-1}[U]\right)}\right\}[A]^{-1}$$

Để giải hệ phương trình phi tuyến F([X]) = [0] ta cho vec tơ nghiệm ban đầu $[P_0]$ và tạo ra dãy $[P_k]$ hội tụ về [P], nghĩa là F([P]) = [0].

Trước hết ta tính ma trận Jacobi $A_0 = J([P_0])$ và dùng nó tính lần lặp thứ nhất theo phương pháp Newton.

$[P_1] = [P_0] - [A]^{-1}F([P_0])$

Giả sử đã có $[P_k]$ với $k \geq 1$ ta dùng các bước sau để tính $[P_{k+1}]$

Bước 1: Tính hàm $F_k = F([P_k])$

Bước 2: Cập nhật ma trận Jacobi bằng cách dùng

$[V] = [P_k] - [P_{k-1}]$

$$[U] = \frac{1}{[V]^T[V]}\left([F_k] - [F_{k-1}] - [A_{k-1}][V]\right)$$

và có:

$$[A_k] = [A_{k-1}] + [U][V]^T$$

Bước 3: Tính $[B_k] = [A_k]^{-1}$ bằng cách dùng công thức Sherman - Morison

$$[B_k] = [A_k]^{-1} = \left\{[E] - \frac{\left([A_{k-1}]^{-1}[U]\right)[V]^T}{\left(1 + [V]^T[A_{k-1}]^{-1}[U]\right)}\right\}[A_{k-1}]^{-1}$$

Bước 4: Lặp tiếp với $[P_{k+1}] = [P_k] - [B_k]F([P_k])$

Để thực hiện thuật toán trên ta xây dựng hàm ***improvedbroyden()***

```
function [m, ll] = improvedbroyden(g, p0, maxiter);
% cu phap   [n, ll] = improvedboyden(g, x0, maxiter);
% vao:
%       - g la fi chua ham can tim nghiem
%       - p0 la vec to nghiem ban dau
%       - maxiter so lan lap max
%ra:
%       - m la nghiem
 %      - ll so lan lap thuc te
 if size(p0, 1) == 1
    p0 = p0';
 end
 n = length(p0);
 [a0, f0]  = jacobi(g, p0);
 b0 = inv(a0);
 p1 = p0 - b0*g(p0);
 for k = 1: maxiter
   f1 = g(p1);
   v = p1 - p0;
   d = v'*v;
   u = (1/d)*(f1 - f0 - a0*v);
   a1 = a0 + u*v';
   e = eye(n);
   ts = (b0*u)*v';
   ms = 1 + v'*b0*u;
   b1 = (e - ts/ms)*b0;
   p2 = p1 - b1*g(p1);
```

```
    if abs(max(v)) < eps
        break;
    end
    a0 = a1;
    b0 = b1;
    p0 = p1;
    p1 = p2;
    f0 = f1;
  end
  m = p2;
  ll = k;
```

Ta giải hệ phương trình:

$$\begin{cases} 2x^2 - 2y^3 - 4x + 1 = 0 \\ x^4 + 4y^4 + 4y - 4 = 0 \end{cases}$$

bằng cách dùng chương trình ***ctimprovedbroyden.m***

```
clear all, clc
format long
p = [.1 .7 ];
[r, s] = improvedbroyden(@g, p, 50)
```

§14. TÍNH TRỊ SỐ CỦA ĐA THỨC

1. Sơ đồ Horner: Giả sử chúng ta cần tìm giá trị của một đa thức tổng quát dạng:

$$P(x) = a_0x^n + a_1x^{n-1} + a_2x^{n-2} + ... + a_n \qquad (1)$$

tại một trị số x nào đó. Trong (1) các hệ số a_i là các số thực đã cho. Chúng ta viết lại (1) theo thuật toán Horner dưới dạng:

$$P(x_o) = (...((a_0x + a_1)x + a_2x) + ... + a_{n-1})x + a_n \qquad (2)$$

Từ (2) ta có :

$P_0 = a_0$

$P_1 = P_0x + a_1$

$P_2 = P_1x + a_2$

$P_3 = P_2x + a_3$

.....

$P(x) = P_n = P_{n-1}x + a_n$

Tổng quát ta có :

$P_k = P_{k-1}x + a_k$ với k = 1, 2...n ; $P_0 = a_0$

Do chúng ta chỉ quan tâm đến trị số của P_n nên trong các công thức truy hồi về sau chúng ta sẽ bỏ qua chỉ số k của P và viết gọn P := Px + a_k với k = 0...n. Khi ta tính tới k = n thì P chính là giá trị cần tìm của đa thức khi đã cho x. Chúng ta thử các bước tính như sau :

Ban đầu P = 0

Bước 0 k = 0 $P = a_o$
Bước 1 k = 1 $P = a_o x + a_1$
Bước 2 k = 2 $P = (a_o x + a_1)x + a_2$
.....
Bước n-1 k = n - 1 $P = P(x_o) = (...((a_o x + a_1)x+a_2 x)+...+a_{n-1})x$
Bước n k = n $P = P(x_o) = (...((a_o x + a_1)x+a_2 x)+...+a_{n-1})x + a_n$

Ta xây dựng hàm ***horner()*** để tính trị của đa thức tại x:

```
function p = horner(a, x)
% Tinh tri so da thuc
% p = a(1)*x^n + a(2)*x^(n-1) + ... + a(n+1)
% cu phap: p = horner(a,x)
n = length(a) - 1;
p = a(1);
for i = 1:n
   p = p*x + a(i+1);
end
```

Để tính trị số của đa thức $P_3(x) = x^3 + 3x^2 + 2x - 5$ tại x = 1 ta dùng chương trình ***cthorner.m***

```
clear all, clc
a = [1  3  2  -5];
p = horner(a, 1)
```

2. Sơ đồ Horner tổng quát: Giả sử chúng ta có đa thức :

$$P_n(x) = a_0x^n + a_1x^{n-1} + a_2x^{n-2} + ...+ a_n \quad (1)$$

Khai triển Taylor của đa thức tại $x = x_o$ có dạng :

$$P_n(x) = P_n(x_0) + \frac{P'(x_0)}{1!}(x - x_0) + \frac{P''(x_0)}{2!}(x - x_0)^2 + \cdots + \frac{P^{(n)}(x_0)}{2!}(x - x_0)^n \quad (2)$$

Mặt khác chúng ta có thể biến đổi đa thức về dạng :

$$P_n(x) = (x - x_o)P_{n-1}(x) + P_n(x_o) \quad (3)$$

Trong đó $P_{n-1}(x)$ là đa thức bậc n - 1 và có dạng :

$$P_{n-1}(x) = b_o x^{n-1} + b_{o-1}x^{n-2} + b_2x^{n-3} + ...+ b_{n-1} \quad (4)$$

Thuật toán để tìm các hệ số nhận được bằng cách so sánh (1) và (3) :

$b_o = a_o$
$b_i = a_i + b_{i-1}x_o$
$b_n = P_n(x_o)$

So sánh (2) và (3) ta có :

$$(x-x_0)P_{n-1}(x_0)+P_n(x_0)=P_n(x_0)+\frac{P'(x_0)}{1!}(x-x_0)+\frac{P''(x_0)}{2!}(x-x_0)^2$$
$$+\cdots+\frac{P^{(n)}(x_0)}{2!}(x-x_0)^n$$

hay :

$$(x-x_0)P_{n-1}(x)=\frac{P'(x_0)}{1!}(x-x_0)+\frac{P''(x_0)}{2!}(x-x_0)^2+\cdots+\frac{P^{(n)}(x_0)}{2!}(x-x_0)^n$$

và khi chia hai vế cho $(x - x_0)$ ta nhận được :

$$P_{n-1}(x)=\frac{P'(x_0)}{1!}+\frac{P''(x_0)}{2!}(x-x_0)\ +\cdots+\frac{P^{(n)}(x_0)}{2!}(x-x_0)^{n-1} \qquad (5)$$

So sánh (4) và (5) ta nhận được kết quả :

$$P_{n-1}(x_0)=\frac{P'(x_0)}{1!}$$

Trong đó $P_{n-1}(x)$ lại có thể phân tích giống như $P_n(x)$ dạng (3) để tìm ra $P_{n-1}(x_o)$. Quá trình này được tiếp tục cho đến khi ta tìm hết các hệ số của chuỗi Taylor của $P_n(x)$. Tổng quát thuật toán thể hiện ở bảng sau:

$P_n(x)$	a_o	a_1	a_2	a_3	...	a_{n-1}	a_n
$x = x_o$	0	b_ox_o	b_1x_o	b_2x_o	...	$b_{n-2}x_o$	$b_{n-1}x_o$
$P_{n-1}(x)$	b_o	b_1	b_2	b_3	...	b_{n-1}	$b_n = P_n(x_o)$

Ta xây dựng hàm ***genhorner()*** để thực hiện thuật toán trên

```
function b = genhorner(a, x)
% tra ve he so cua da thuc khai trien
% c(1)(x-x0)^n + c(2)(x-x0)^(n-1) + ...+ c(n+1)
m = length(a)
x = 2;
for k = 1:m
   b(1) = a(1);
   for i = 2:m-k+1
      b(i) = b(i - 1)*x + a(i);
   end
   c(m-k+1) = b(m-k+1)
   a = b(1:m-k);
end
```

Để khai triển đa thức $P(x) = x^5 - 2x^4 + x^3 - 5x + 4$ tại $x_0 = 2$ ta dùng chương trình ***ctgenhorner.m***

```
clear all, clc
```

```
a = [1  -2   1   0  -5    4];
c = genhorner(a, 2)
```

§15. PHƯƠNG PHÁP LAGUERRE

Ta xét đa thức bậc n:

$$P_n(x) = a_1x^n + a_2x^{n-1} + \cdots + a_{n+1} \quad (1)$$

Nếu đa thức có nghiệm là r thì ta có:

$$P_n(x) = (x - r)P_{n-1}(x) \quad (2)$$

Trong đó:

$$P_{n-1}(x) = b_1x^{n-1} + b_2x^{n-2} + \cdots + b_n$$

Cân bằng (1) và (2) ta có:

$b_1 = a_1$

$b_2 = a_2 + rb_1$

. . . . . .

$b_n = a_n + rb_{n-1}$

Ta xây dựng hàm ***deflpoly()*** để tính các hệ số của đa thức $P_{n-1}(x)$

```
function b = deflpoly(a, r)
% ha bac da thuc
n = length(a) - 1;
b = zeros(n, 1);
b(1) = a(1);
for i = 2:n
   b(i) = a(i) + r*b(i-1);
end
```

Bây giờ ta xét đa thức $P_n(x)$ có nghiệm đơn x = r và (n-1) nghiệm trùng nhau x = q. Đa thức như vậy sẽ được viết thành:

$$P_n(x) = (x - r)(x - q)^{n-1} \quad (3)$$

Bài toán của ta là cho đa thức (3) dưới dạng:

$$P_n(x) = a_1x^n + a_2x^{n-1} + \cdots + a_{n+1}$$

và cần tìm r(chú ý là q cũng chưa biết).

Đạo hàm (3) theo x ta có:

$$P'_n(x) = (x-q)^{n-1} + (n-1)(x-q)^{n-2}(x-r)$$

$$= P_n(x)\left(\frac{1}{x-r} - \frac{n-1}{x-q}\right)$$

Như vậy:

$$\frac{P'_n(x)}{P_n(x)} = \frac{1}{x-r} - \frac{n-1}{x-q} \quad (4)$$

Từ đó ta có:

$$\frac{P_n''(x)}{P_n(x)} - \left[\frac{P_n'(x)}{P_n(x)}\right]^2 = -\frac{1}{(x-r)^2} - \frac{n-1}{(x-q)^2} \tag{5}$$

Ta đặt:

$$G(x) = \frac{P_n'(x)}{P_n(x)} \qquad H(x) = G^2(x) - \frac{P_n''(x)}{P_n(x)} \tag{6}$$

Như vậy (4) và (5) trở thành:

$$G(x) = \frac{P_n'(x)}{P_n(x)} = \frac{1}{x-r} + \frac{n-1}{x-q} \tag{7}$$

$$H(x) = G^2(x) - \frac{P_n''(x)}{P_n(x)} = \frac{1}{(x-r)^2} + \frac{n-1}{(x-q)^2} \tag{8}$$

Nếu ta giải (7) theo (x - q) và thay kết quả vào (8) ta nhận được phương trình bậc 2 đối với (x - r). Nghiệm của phương trình này là công thức Laguerre:

$$x - r = \frac{n}{G(x) \pm \sqrt{(n-1)\left[nH(x) - G^2(x)\right]}} \tag{9}$$

Như vậy thuật toán để tìm điểm zero của đa thức tổng quát theo Laguerre là:

- cho giá trị ban đầu của nghiệm của đa thức là x
- tính $P_n(x)$, $P_n'(x)$ và $P_n''(x)$
- tính G(x) và H(x) theo (6)
- tính nghiem theo (9). Chọn dấu sao cho mẫu số lớn nhất.
- cho x = r và tiếp tục lặp từ bước 2 cho đến 5 đến khi $|P_n(x)| < \varepsilon$ hay $|x - r| < \varepsilon$.

Ta xây dựng hàm ***laguerre()*** để thực hiện thuật toán trên

```
function x = laguerre(a, tol)
% tim nghiem cua da thuc
x = randn; % cho x ngau nhien
n = length(a) - 1;
for i = 1:30
    [p, dp, ddp] = evalpoly(a, x);
    if abs(p) < tol;
        return;
    end
    g = dp/p;
    h = g*g - ddp/p;
    f = sqrt((n - 1)*(n*h - g*g));
    if abs(g + f) >= abs(g - f)
        dx = n/(g + f);
    else
        dx = n/(g - f);
    end
```

```
    x = x - dx;
    if abs(dx) < tol
        return;
    end
end
error('Lap qua nhieu')
```

Để tìm tất cả các nghiệm của đa thức ta dùng hàm ***polyroots()***

```
function root = polyroots(a, tol)
% Tim cac nghiem cua da thuc% a(1)*x^n + a(2)*x^(n-1) + ... + a(n+1).
% Cu phap: root = polyroots(a, tol).
% tol = sai so
if nargin == 1
    tol = 1.0e-6;
end
n = length(a) - 1;
root = zeros(n, 1);
for i = 1:n
    x = laguerre(a, tol);
    if abs(imag(x)) < tol
        x = real(x);
    end
    root(i) = x;
    a = deflpoly(a, x);
end
```

Để tìm nghiệm của một đa thức theo thuật toán Laguerre ta dùng chương trình ***ctpolyroots.m***

```
clear all, clc
a = [1  3  2  -5];
r = polyroots(a);
```

§16. PHƯƠNG PHÁP LẶP BERNOULLI

Có nhiều phương pháp để tìm nghiệm của một đa thức. Ta xét phương trình:

$a_o x^n + a_1 x^{n-1} + \cdots + a_n = 0$

Nghiệm của phương trình trên thoả mãn định lí:

Nếu $\max\{|a_1|, |a_2|, \ldots, |a_n|\} = A$ *thì các nghiệm của phương trình thoả mãn điều kiện* $|x| < 1 + A/|a_0|$

Phương pháp Bernoulli cho phép tính toán nghiệm lớn nhất α của một đa thức $P_n(x)$ có n nghiệm thực phân biệt. Sau khi tìm được nghiệm lớn nhất α ta chia đa thức

$P_n(x)$ cho $(x - \alpha)$ và nhận được đa thức mới $Q_{n-1}(x)$. Tiếp tục dùng phương pháp Bernoulli để tìm nghiệm lớn nhất của $Q_{n-1}(x)$.

Sau đó lại tiếp tục các bước trên cho đến khi tìm hết các nghiệm của $P_n(x)$.

Chúng ta khảo sát phương trình sai phân φ có dạng như sau :

$$\varphi = a_o y_{k+n} + a_1 y_{k+n-1} + \ldots + a_n y_k = 0 \qquad (1)$$

Đây là một phương trình sai phân tuyến tính hệ số hằng. Khi cho trước các giá trị đầu $y_o, y_1, .. y_{n-1}$ ta tìm được các giá trị $y_n, y_{n+1}, ..$ Chúng được gọi là nghiệm của phương trình sai phân tuyến tính (1).

Đa thức

$$P_n(x) = a_0 x^n + a_1 x^{n-1} + .. + a_{n-1} x + a_n \qquad (2)$$

với cùng một hệ số a_i như (1) được gọi là đa thức đặc tính của phương trình sai phân tuyến tính (1). Nếu (2) có n nghiệm phân biệt $x_1, x_2, .., x_n$ thì (1) có các nghiệm riêng là

$$y_i = x_i^k$$

Nếu y_i là các nghiệm của phương trình sai phân là tuyến tính (1),thì

$$y_k = c_1 x_1^k + c_2 x_2^k + \cdots + c_n x_n^k \qquad (3)$$

với các hệ số c_i bất kì cũng là nghiệm của phương trình sai phân tuyến tính hệ số hằng (1).

Nếu các nghiệm cần sao cho :

$$|x_1| \geq |x_2| \geq \ldots |x_n|$$

Vậy $$y_k = c_1 x_1^k \left[1 + \frac{c_1}{c_2} \left(\frac{x_2}{x_1} \right)^k + \cdots \right]$$

và $$y_{k+1} = c_1 x_1^{k+1} \left[1 + \frac{c_1}{c_2} \left(\frac{x_2}{x_1} \right)^{k+1} + \cdots \right]$$

do đó : $$\frac{y_{k+1}}{y_k} = x_1 \frac{\left[1 + \frac{c_1}{c_2} \left(\frac{x_2}{x_1} \right)^{k+1} + \cdots \right]}{\left[1 + \frac{c_1}{c_2} \left(\frac{x_2}{x_1} \right)^{k} + \cdots \right]}$$

do $x_1 > x_2 > \ldots > x_n$

nên: $$\left(\frac{x_2}{x_1} \right)^k, \left(\frac{x_2}{x_1} \right)^{k+1} \to 0 \text{ khi } k \to \infty$$

vậy: $$\frac{y_{k+1}}{y_k} \to 0 \text{ khi } k \to \infty$$

Nghĩa là :

$$\lim_{k \to \infty} \frac{y_{k+1}}{y_k} = x_1$$

Nếu phương trình vi phân gồm n+1 hệ số, một nghiệm riêng y_k có thể được xác định từ n giá trị y_{k-1}, y_{k-2},...,y_{n-1}. Điều cho phép tính toán bằng cách truy hồi các nghiệm riêng của phương trình vi phân.

Để tính nghiệm lớn nhất của đa thức, ta xuất phát từ các nghiệm riêng $y_1 = 0$, $y_1 = 0$,.., $y_n = 1$ để tính y_{n+1}. Cách tính này được tiếp tục để tính y_{n+2} xuất phát từ $y_1 = 0$, $y_2 = 0$,..,y_{n+1} và tiếp tục cho đến khi y_{k+1}/y_k không biến đổi nữa. Trị số của y_{k+n} được tính theo công thức truy hồi :

$$y_{k+n} = -\frac{1}{a_0}\left(a_1 y_{k+n-1} + \cdots + a_n y_k\right) \qquad (4)$$

Ta xây dựng hàm ***bernoulli()*** thực hiện thuật toán trên

function x = bernoulli(p)
%tim nghiem lon nhat cua P(x)
%theo thuat toan Bernoulli

```
%cu phap x = bernoulli(p)
n = length(p);
maxiter = 200;
y = zeros(maxiter);
y(n-1) = 1;
k = 0;
while (k < maxiter)
   s = 0;
   for i = 1:n-1
      s = s + p(i+1)*y(k+n-i);
   end
   y(k+n) = -s/p(1);
   x = y(k+n)/y(k+n-1);
   err = polyval(p, x);
   if err < 1e-6
      break;
   end
   k = k + 1;
end
```

Để tìm nghiệm của đa thức $P_4(x) = x^4 - 22x^3 + 179x^2 - 638x + 840$ ta dùng ***ctbernoulli.m***

```
clear all, clc
 p = [1  -22  179  -638  840];
 m = length(p);
 fprintf('Nghiem cua da thuc:\n');
 while m > 3
   x = bernoulli(p);
   m = m - 1;
   fprintf('%f\n',x);
   p = deflpoly(p,x);
 end
 x1 = (-p(2) + sqrt(p(2)^2 - 4*p(1)*p(3)))/2*p(1);
 fprintf('%f\n', x1);
 x2 = (-p(2) - sqrt(p(2)^2 - 4*p(1)*p(3)))/2*p(1);
 fprintf('%f\n', x2);
```

§17. PHƯƠNG PHÁP LẶP BERGE - VIETTE

Các nghiệm thực, đơn của một đa thức $P_n(x)$ được tính toán khi sử dụng phương pháp Newton

$$x_{i+1} = x_i - \frac{P_n(x_i)}{P'_n(x_i)} \quad (1)$$

Để bắt đầu tính toán cần chọn một giá trị ban đầu x_o. Chúng ta có thể chọn một giá trị x_o nào đó, ví dụ :

$$x_0 = -\frac{a_n}{a_{n-1}}$$

và tính tiếp các giá trị sau :

$$x_1 = x_0 - \frac{P_n(x_0)}{P'_n(x_0)}$$

$$x_2 = x_1 - \frac{P_n(x_1)}{P'_n(x_1)}$$

Tiếp theo có thể đánh giá $P_n(x_i)$ theo thuật toán Horner :

$$\begin{aligned} P_0 &= a_0 \\ P_1 &= P_0x_i + a_1 \\ P_2 &= P_1x_i + a_2 \\ P_3 &= P_2x_i + a_3 \\ &\ldots\ldots\ldots\ldots\ldots\ldots \\ P(x_i) &= P_n = P_{n-1}x_i + a_n \end{aligned} \quad (2)$$

Mặt khác khi chia đa thức $P_n(x)$ cho một nhị thức $(x - x_i)$ ta được :

$$P_n(x) = (x - x_i)P_{n-1}(x) + b_n \quad (3)$$

với $b_n = P_n(x_i)$. Đa thức $P_{n-1}(x)$ có dạng:

$$P_{n-1}(x) = b_ox^{n-1} + b_1x^{n-2} + p_3x^{n-3} +..+ b_{n-2}x + b_{n-1} \quad (4)$$

Để xác định các hệ số của đa thức (4) ta thay (4) vào (3) và cân bằng các hệ số với đa thức cần tìm nghiệm $P_n(x)$ mà các hệ số ai đã cho:

$$(x - x_i)(b_ox^{n-1} + b_1x^{n-2}+b_3x^{n-3} +..+ b_{n-2}x + b_{n-1}) + b_n$$

$$= a_ox^n + a_1x^{n-1} + a_2x^{n-2} +...+ a^n_{-1}x + a \quad (5)$$

Từ (5) rút ra :

$$\begin{aligned} b_o &= a_o \\ b_1 &= a_1 + b_ox_i \\ b_2 &= a_2 + b_1x_i \\ &\ldots\ldots \\ b_k &= a_k + b_{k-1}x_i \\ &\ldots\ldots\ldots \\ b_n &= a_n + b_{n-1}x_i = P_n(x_i) \end{aligned} \quad (6)$$

Đạo hàm (3) ta được :

$$P'_n(x) = (x - x_i)P'_{n-1}(x) + P_{n-1}(x)$$

và: $$P'_n(x_i) = P_{n-1}(x_i) \quad (7)$$

Như vậy với một giá trị x_i nào đó theo (2) ta tính được $P_n(x_i)$ và kết hợp (6) với (7) tính được $P'_n(x_i)$. Thay các kết quả này vào (1) ta tính được giá trị x_{i+1}. Quá trình được tiếp tục cho đến khi $| x_{i+1} - x_i | < \varepsilon$ hay $P_n(x_{i+1}) \approx 0$ nên $\alpha_1 \approx x_{i+1}$ là một nghiệm của đa thức.

Phép chia $P_n(x)$ cho $(x - \alpha_1)$ cho ta $P_{n-1}(x)$ và một nghiệm mới khác được tìm theo cách trên khi chọn một giá trị x_o mới hay chọn chính $x_o = \alpha_1$. Khi bậc của đa thức giảm xuống còn bằng 2 ta dùng các công thức tìm nghiệm của tam thức để tìm các nghiệm còn lại.

Ta xây dựng hàm ***birgeviette()*** để thực hiện thuật toán trên

```
function x = birgeviette(a, x0)
%tim nghiem theo thuat toan lap Birge-Viette
%vao:   - a da thuc can tim nghiem
%       - x0 gia tri dau
%cu phap x = birgeviete(a, x0, maxiter)
n = length(a) - 1;
k = 0;
x0 = 3.5;
maxiter = 50;
while (k < maxiter)
   p = a(1);
   b(1) = a(1);
   for i = 1:n
      p = p*x0 + a(i+1);
      b(i+1) = p;
   end
   b = b(1, 1:n);
   p1 = horner(b, x0);
   x1 = x0 - p/p1;
   k = k + 1;
   err = horner(a, x1);
   if (abs(err) < eps)
      break;
   end
   x0 = x1;
end
x = x0;
```

Để tìm nghiệm của đa thức $P_4(x) = x^4 - 22x^3 + 179x^2 - 638x + 840$ ta dùng ***ctbirgeviette.m***

```
clear all, clc
 p = [1  -22  179  -638  840];
 m = length(p);
 x0 = 1;
 fprintf('Nghiem cua da thuc:\n');
```

```
while m > 3
   x = birgeviete(p, x0);
   m = m - 1;
   fprintf('%f\n', x);
   p = deflpoly(p, x);
end
x1 = (-p(2) + sqrt(p(2)^2 - 4*p(1)*p(3)))/2*p(1);
fprintf('%f\n', x1);
x2 = (-p(2) - sqrt(p(2)^2 - 4*p(1)*p(3)))/2*p(1);
fprintf('%f\n', x2);
```

§18. PHƯƠNG PHÁP BAIRSTOW

Nguyên tắc của phương pháp Bairstow là trích từ đa thức $P_n(x)$ một tam thức $Q_2(x) = x^2 - rx - s$ mà ta có thể tính nghiệm thực hay nghiệm phức của nó một cách đơn giản bằng các phương pháp đã biết. Việc chia đa thức $P_n(x)$ cho ta thức $Q_2(x)$ đưa tới kết quả:

$$P_n(x) = Q_2(x).P_{n-2}(x) + R_1(x)$$

với: $P_n(x) = a_1x^n + a_2x^{n-1} + a_3x^{n-2} + \cdots + a_{n+1}$

$Q_2(x) = x^2 - rx - s$

$P_{n-2}(x) = b_1x^{n-2} + b_2x^{n-3} + b_3x^{n-4} + \cdots + b_{n-1}$

$R_1(x) = \alpha x + \beta$

Để có được một thương đúng, cần tìm các giá trị s và p sao cho $R_1(x) = 0$ (nghĩa là α và β triệt tiêu). Với r và s đã cho, các hệ số b_i của đa thức $P_{n-2}(x)$ và các hệ số α và β được tính bằng phương pháp truy hồi. Các công thức nhận được khi khai triển biểu thức $P_n(x) = Q_2(x).P_{n-2}(x) + R_1(x)$ và sắp xếp lại các số hạng cùng bậc:

$$a_1x^n + a_2x^{n-1} + a_3x^{n-2} + \ldots + a_{n+1} = (x^2 - rx - s)(b_1x^{n-2} + b_2x^{n-3} + b_3x^{n-4} + \ldots + b_{n-1})$$

Số hạng bậc	Hệ số của $P_n(x)$	Hệ số của $Q_2(x).P_{n-2}(x)$
x^n	a_1	b_1
x^{n-1}	a_2	$b_1 - rb_1$
x^{n-2}	a_3	$b_3 - rb_2 - sb_1$
.....	.....	.....
x^{n-k}	a_k	$b_k - rb_{k-1} - sb_{k-2}$
x	a_n	$\alpha - rb_{n-1} - sb_{n-2}$
x^o	a_{n+1}	$\beta - rb_{n-1}$

Như vậy :

$$b_1 = a_1 \qquad (1)$$
$$b_2 = a_2 + rb_1$$
$$b_3 = a_3 + rb_2 + sb_1$$
$$\ldots\ldots\ldots\ldots$$
$$b_k = a_k + rb_{k-1} + sb_{k-2}$$
$$\alpha = a_n + rb_{n-1} + sb_{n-2}$$

$\beta = a_n - pb_{-2}$

Chúng ta nhận thấy rằng α được tính toán xuất phát từ cùng một công thức truy hồi như các hệ số b_k và tương ứng với hệ số b_n

$b_n = a_{n-1} + rb_{n-1} + sb_{n-2} = \alpha$

Hệ số b_{n+1} là :

$b_{n+1} = a_{n+1} + rb_{n-1} + sb_{n-2} = sb_{n-2} + \beta$

và cuối cùng :

$R_1(x) = \alpha x + \beta = b_n(x - r) + b_{n+1}$

Ngoài ra các hệ số b_i phụ thuộc vào r và s và bây giờ chúng ta cần phải tìm các giá trị đặc biệt r^* và s^* để cho b_n và b_{n+1} triệt tiêu. Khi đó $R_1(x) = 0$ và nghiệm của tam thức $x^2 - r^*x - s^*x$ sẽ là nghiệm của đa thức $P_n(x)$.

Ta biết rằng b_{n-1} và b_n là hàm của s và p :

$b_n = f(r, s)$

$b_{n+1} = g(r, s)$

Việc tìm r^* và s^* đưa đến việc giải hệ phương trình phi tuyến:

$$\begin{cases} f(r,s)=0 \\ g(r,s)=0 \end{cases}$$

Phương trình này có thể giải dễ dàng nhờ phương pháp Newton. Thật vậy với một phương trình phi tuyến ta có công thức lặp:

$x_{i+1} = x_i - f(x_i)/f'(x_i)$

hay $f'(x_i)(x_{i+1} - x_i) = -f(x_i)$

Với một hệ có hai phương trình, công thức lặp trở thành:

$J(X_i)(X_{i+1} - X_i) = -F(X_i)$

với $X_i = \{ r_i, s_i\}^T$ và $X_{i+1} = \{ r_{i+1}, s_{i+1}\}^T$

$$F(X_i) = \begin{bmatrix} f(r_i, s_i) \\ g(r_i, s_i) \end{bmatrix}$$

$$J(X_i) = \begin{pmatrix} \dfrac{\partial f}{\partial r} & \dfrac{\partial f}{\partial s} \\ \dfrac{\partial g}{\partial r} & \dfrac{\partial g}{\partial s} \end{pmatrix}$$

Quan hệ : $J(X_i)\Delta X = -F(X_i)$ với $\Delta X = \{r_{i+1} - r_i, s_{i+1} - s_i\}^T$ tương ứng với một hệ phương trình tuyến tính hai ẩn số $\Delta r = r_{i+1} - r_i$ và $\Delta s = s_{i+1} - s_i$:

$$\begin{cases} \dfrac{\partial f}{\partial r}\Delta r + \dfrac{\partial f}{\partial s}\Delta s = -f(r_i, s_i) \\ \dfrac{\partial g}{\partial r}\Delta r + \dfrac{\partial g}{\partial s}\Delta s = -g(r_i, s_i) \end{cases}$$

Theo công thức Cramer ta có :

$$\Delta r = \frac{-f\dfrac{\partial g}{\partial s} + g\dfrac{\partial f}{\partial s}}{\delta}$$

$$\Delta s = \frac{-g\frac{\partial f}{\partial r} + f\frac{\partial g}{\partial r}}{\delta}$$

$$\delta = \frac{\partial f}{\partial r}\frac{\partial g}{\partial s} - \frac{\partial f}{\partial s}\frac{\partial g}{\partial r}$$

Để dùng được công thức này ta cần tính được các đạo hàm $\frac{\partial f}{\partial r}, \frac{\partial f}{\partial s}, \frac{\partial g}{\partial r}, \frac{\partial g}{\partial s}$. Các đạo hàm này được tính theo công thức truy hồi.

Do $b_1 = a_1$ nên

$$\frac{\partial b_1}{\partial r} = \frac{\partial a_1}{\partial r} = 0 \qquad \frac{\partial b_1}{\partial s} = \frac{\partial a_1}{\partial s} = 0$$

$b_2 = a_2 + rb_1$ nên

$$\frac{\partial b_2}{\partial r} = r\frac{\partial b_1}{\partial r} + b_1 = b_1 \qquad \frac{\partial b_2}{\partial s} = \frac{\partial a_2}{\partial s} + r\frac{\partial b_1}{\partial s} = 0$$

$b_3 = a_3 + rb_2 + sb_1$ nên

$$\frac{\partial b_3}{\partial r} = \frac{\partial a_3}{\partial r} + \frac{\partial (rb_2)}{\partial r} + \frac{\partial (sb_1)}{\partial r}$$

Mặt khác :

$$\frac{\partial a_3}{\partial r} = 0 \qquad \frac{\partial (rb_2)}{\partial r} = r\frac{\partial (b_2)}{\partial r} + b_2 \qquad \frac{\partial (rb_1)}{\partial r} = 0$$

nên: $\frac{\partial b_3}{\partial r} = b_2 + rb_1 \qquad \frac{\partial b_3}{\partial s} = r\frac{\partial b_2}{\partial s} + s\frac{\partial b_1}{\partial s} + b_1 = b_1$

$b_4 = a_4 + rb_3 + sb_2$ nên:

$$\frac{\partial b_4}{\partial r} = b_3 + r\frac{\partial b_3}{\partial r} + s\frac{\partial b_2}{\partial r} \qquad \frac{\partial b_4}{\partial s} = r\frac{\partial b_3}{\partial s} + s\frac{\partial b_2}{\partial s} + b_2$$

. . . . .

$$\frac{\partial b_n}{\partial r} = r\frac{\partial b_{n-1}}{\partial r} + s\frac{\partial b_{n-2}}{\partial r} + b_{n-1} \qquad \frac{\partial b_n}{\partial s} = r\frac{\partial b_{n-1}}{\partial r} + s\frac{\partial b_{n-2}}{\partial r} + b_{n-2}$$

Nếu chúng ta đặt:

$$\frac{\partial b_k}{\partial r} = c_{k-1}$$

thì :

$$c_1 = b_1 \tag{2}$$

$c_2 = b_2 + rc_1$

$c_3 = b_3 + rc_2 + sc_1$

. . . . . .

$c_n = b_n + rc_{n-1} + sc_{n-2}$

Như vậy ta có:

$$\Delta r = \frac{-b_n c_{n-1} + b_{n+1} c_{n-2}}{c_{n-1}^2 - c_n c_{n-2}} \tag{3}$$

$$\Delta s = \frac{-b_{n+1}c_{n-1} + b_n c_n}{c_{n-1}^2 - c_n c_{n-2}} \quad (4)$$

Sau khi phân tích xong $P_n(x)$ ta tiếp tục phân tích $P_{n-2}(x)$ theo phương pháp trên. Các bước tính toán gồm:

- Chọn các giá trị ban đầu bất kì s_0 và p_0
- Tính các giá trị $b_1,..,b_{n+1}$ theo (1)
- Tính các giá trị $c_1,...,c_n$ theo (2)
- Tính Δr_o và Δs_o theo (3) và (4)
- Tính $s_1 = r_0 + \Delta r_o$ và $s_1 = s_o + \Delta s_o$
- Lặp lại bước 1 cho đến khi $r_{i+1} = r_i = r$ và $s_{i+1} = s_i = s$
- Giải phương trình x_2 - rx - s để tìm 2 nghiệm của đa thức
- Bắt đầu quá trình trên cho đa thức $P_{n-2}(x)$

Ta xây dựng hàm ***bairstow()*** để thực hiện thuật toán tìm r, s

```
function [r,s] = bairstow(p, r0, s0, maxiter)
% tim da thuc bac 2 dang x^2 - rx - s
%vao -p la da thuc can tim nghiem
%    -r0, s0 gia tri ban dau
%    -maxiter so lan lap max
%ra  -r, s
%cu phap [r,s] = bairstow(p, r0, s0 ,maxiter)
n = length(p) - 1;
c = zeros(n);
b = zeros(n);
j = 0;
while j < maxiter
   b(1) = p(1);
   b(2) = p(2) + r0*b(1);
   for k = 3:(n+1)
      b(k) = p(k) + r0*b(k-1) + s0*b(k-2);
   end
   c(1) = b(1);
   c(2) = b(2) + r0*c(1);
   for k = 3:(n)
      c(k) = b(k) + r0*c(k-1) + s0*c(k-2);
   end
   d0 = det([c(n-1), c(n-2); c(n), c(n-1)]);
   d1 = det([-b(n), c(n-2); -b(n+1), c(n-1)]);
   d2 = det([c(n-1), -b(n); c(n) -b(n+1)]);
   r = r0 + d1/d0;
```

```
    s = s0 + d2/d0;
    if ((abs(d1/d0))&(abs(d2/d0)))<eps
       break;
    end
    r0 = r;
    s0 = s;
    j = j + 1;
  end
```

Để tìm nghiệm của đa thức $P_4(x) = x^4 - 1.1x^3 + 2.3x^2 - 0.5x + 3.3$ ta dùng chương trình ***ctbairstow.m***

```
clear all, clc
p = [1  -1.1  2.3  0.5  3.3];
m = length(p);
s0 = -1,;
r0 = -1;
fprintf('Nghiem cua da thuc:\n');
while m > 3
   [r, s] = bairstow(p,r0,s0,50);
   m = m - 2;
   x1 = (r + sqrt(r^2+4*s))/2;
   x2 = (r - sqrt(r^2+4*s))/2;
   fprintf('%s\n',num2str(x1));
   fprintf('%s\n',num2str(x2));
   p = deconv(p,[1 -r -s]);
end
if length(p) == 3
   x1 = (-p(2) + sqrt(p(2)^2-4*p(3)))/2;
   x2 = (-p(2) - sqrt(p(2)^2-4*p(3)))/2;
   fprintf('%s\n',num2str(x1));;
   fprintf('%s\n',num2str(x2));;
else
   x1 = -p(2)/p(1);
   fprintf('%f\n',x1);;
end
```

§19. PHƯƠNG PHÁP LOBACHEVSKY - GRAEFFE

Phương pháp này đã được Karl Heinrich Gräffe, Germinal Pierre Dandelin và Nikolai Ivanovich Lobachevsky đưa ra. Nó có một nhược điểm là các kết quả trung gian có trị số rất lớn.

Xét phương trình :

$$P(x) = a_0x^n + a_1x^{n-1} + \cdot\cdot\cdot + a_n = 0 \qquad (1)$$

chỉ có nghiệm thực với giá trị tuyệt đối khác nhau. Ta đánh số các nghiệm đó theo thứ tự giá trị tuyệt đối giảm :

$$|x_1| > |x_2| > \cdots > |x_n| \qquad (2)$$

Dựa vào (1) ta xây dựng một phương trình mới :

$$Q(x) = c_0x^n + c_1x^{n-1} + \cdots + c_n = 0 \qquad (c_0 \neq 0) \qquad (3)$$

có nghiệm là $-x_1^m, -x_2^m, ..., -x_n^m$. Sau đó ta viết lại phương trình trên:

$$Q(x) = c_0(x + x_1^m)(x + x_2^m)\cdots(x + x_n^m) \qquad (4)$$

So sánh (3) và (4) ta có:

$$\begin{cases} x_1^m + x_2^m + \cdots + x_n^m = \dfrac{c_1}{c_0} \\ x_1^m x_2^m + x_2^m x_3^m + \cdots + x_{n-1}^m x_n^m = \dfrac{c_2}{c_0} \\ \cdots \\ x_1^m x_2^m \cdots x_{n-1}^m x_n^m = \dfrac{c_n}{c_0} \end{cases} \qquad (5)$$

Vì có giả thiết (2) nên khi m tăng thì ở vế trái của các đẳng thức (5) số hạng đầu trội lên, lấn át các số hạng sau và với m đủ lớn ta có :

$$\begin{cases} x_1^m \approx \dfrac{c_1}{c_0} \\ x_1^m x_2^m \approx \dfrac{c_2}{c_0} \\ \cdots \\ x_1^m x_2^m \cdots x_{n-1}^m x_n^m \approx \dfrac{c_n}{c_0} \end{cases} \qquad (6)$$

Từ (6) ta có :

$$\begin{cases} x_1^m \approx \dfrac{c_1}{c_0} \\ x_2^m \approx \dfrac{c_2}{c_1} \\ \cdots \\ x_n^m \approx \dfrac{c_n}{c_{n-1}} \end{cases} \qquad (7)$$

Ta suy ra trị tuyệt đối của các nghiệm là :

$$\begin{cases} |x_1| \approx \sqrt[m]{\left|\dfrac{c_1}{c_0}\right|} \\ |x_2| \approx \sqrt[m]{\left|\dfrac{c_2}{c_1}\right|} \\ \dots \\ |x_n| \approx \sqrt[m]{\left|\dfrac{c_n}{c_{n-1}}\right|} \end{cases} \qquad (8)$$

Sau đó ta thay $\pm|x_i|$ vào (1) và tìm được nghiệm. Như vậy vấn đề là phải tìm được phương trình (3) dựa vào phương trình (1). Quá trình biến đổi (1) thành (3) được tiến hành như sau:

Đầu tiên từ (1) ta xây dựng phương trình mới có nghiệm là $-x_1^2$ và quá trình này được gọi là bình phương nghiệm. Vì P(x) có các nghiệm là x_i nên nó có dạng:

$$P(x) = a_0(x - x_1)(x - x_2)\cdots(x - x_n)$$

Do đó :

$$(-1)^n P(-x) = a_0(x + x_1)(x + x_2)\cdots(x + x_n)$$

Ta suy ra :

$$(-1)^n P(x)P(-x) = a_0^2(x^2 - x_1^2)(x^2 - x_2^2)\cdots(x^2 - x_n^2)$$

Thay $x^2 = -y$ ta có :

$$P_1(y) = P(x)P(-x) = a_0^2(y + x_1^2)(y + x_2^2)\cdots(y + x_n^2)$$

Đa thức $P_1(y)$ có nghiệm là $y_i = -x_1^2$. Đa thức này có dạng :

$$P_1(y) = a_0^{(1)}y^n + a_1^{(1)}y^{n-1} + a_2^{(1)}y^{n-2} + \cdots + a_n^{(1)} \qquad (9)$$

Do $\quad P(x) = a_0x^n + a_1x^{n-1} + \cdots + a_n = 0$

nên $\quad P(-x) = (-1)^n[a_0x^n - a_1x^{n-1} + \cdots + (-1)^n a_n]$

$$(-1)^n P(-x) = a_0x^n - a_1x^{n-1} + \cdots + (-1)^n a_n$$

và

$$(-1)^n P(x)P(-x) = a_0^2x^{2n} - (a_1^2 - 2a_0a_2)x^{2n-2} + (a_2^2 - 2a_1a_3 + 2a_0a_4)x^{2n-4} + \cdots + (-1)^n a_n^2$$

Thay $x^2 = -y$ ta có :

$$P_1(y) = a_0^2y^n - (a_1^2 - 2a_0a_2)y^{n-1} + (a_2^2 - 2a_1a_3 + 2a_0a_4)y^{n-2} + \cdots + (-1)^n a_n^2 \qquad (10)$$

So sánh (9) và (10) ta có công thức tính các hệ số của $P_1(y)$ từ các a_k là :

$$\begin{cases} a_0^{(1)} = a_0^2 \\ a_1^{(1)} = a_1^2 - 2a_0a_2 \\ a_2^{(1)} = a_2^2 - 2a_1a_3 + 2a_0a_4 \\ \dots \\ a_k^{(1)} = a_k^2 + 2\sum_{i=1}^{k}(-1)^i a_{k-i}a_{k+i} \\ \dots \\ a_{n-1}^{(1)} = a_{n-1}^2 - 2a_{n-2}a_n \\ a_n^{(1)} = a_n^2 \end{cases} \tag{11}$$

Tiếp tục quá trình bình phương nghiệm đa thức $P_1(y)$ ta được $P_2(y)$ có nghiệm là $y_k = (-x_1^2)^2$ với các hệ số $a_k^{(2)}$ được tính theo $a_k^{(1)}$ tương tự như (11) khi tính $a_k^{(1)}$ theo a_k. Tiến hành bình phương nghiệm (s + 1) lần ta có (3). Lúc đó các hệ số được xác định bằng:

$$\begin{cases} a_0^{(s+1)} = \left[a_0^{(s)}\right]^2 \\ \dots \\ a_k^{(s+1)} = \left[a_k^{(s)}\right]^2 + 2\sum_{i=1}^{k}(-1)^i a_{k-i}^{(s)} - a_{k+i}^{(s)} \\ \dots \\ a_{n-1}^{(s+1)} = \left[a_{n-1}^{(s)}\right]^2 - 2a_{n-2}^{(s)}a_n^{(s)} \\ a_n^{(s+1)} = \left[a_n^{(s)}\right]^2 \end{cases}$$

Vậy khi s đủ lớn ta có :

$$-x_i^{2^s} = -x_i^m \approx \frac{a_i^{(s)}}{a_{i-1}^{(s)}}$$

Ta có $m = 2^6 = 64$. Dùng logarit ta tính ra các nghiệm : $x_1 = -4$, $x_2 = 2$, $x_3 = 1$

Ta xây dựng hàm ***lobachevskygraeffe()*** để thực hiện thuật toán trên

```
function y = lobachevskygraeffe(a, maxiter)
% giai pt bang pp Lochevsky - Graeffe
% a la da thuc can tim nghiem
% maxiter la so lan lap max
c = a;
n = length(a);
m = 1;
while m < maxiter
   b = a;
```

```
    for k = 2:n-1;
        s = 0;
        i = 1;
        while (((k - i) >= 1)&((k + i) <= n))
            s = s + (-1)^i*b(k - i)*b(k + i);
            i = i + 1;
        end
        a(k) = b(k)^2 + 2*s;
    end
    a(1) = a(1)^2;
    a(n) = a(n)^2;
    j = 2^m;
    for i = 1:n-1
        err = 1;
        x(i) = a(i + 1)/a(i);
        if x(i) == 1
            x(i) = 1;
        else
            x(i) = exp((1/j)*log(x(i)));
        end
        err = abs(horner(c, x(i)));
    end
    if err < eps
        break;
    end
    m = m + 1;
end
for i = 1:n-1
    if round(polyval(c, x(i))) ~= 0
        x(i) = -x(i);
    end
end
y = x;
```

Để tìm nghiệm của đa thức $P_4(x) = x^4 + 2x^3 - 25x^2 - 26x^2 + 120$ ta dùng chương trình ***ctlobachevskygraeffe.m***:

```
clc, clear all
a = [1 2 -25 -26 120];
x = lobachevskygraeffe(a, 50)
```

§20. PHƯƠNG PHÁP SCHRODER

Phương pháp lặp Schroder dùng để tìm nghiệm bội, có dạng tương tự như công thức lặp Newton:

$$x_{k+1} = x_k - \frac{mf(x_k)}{f'(x_k)}$$

Trong đó m là bội của nghiêm. Ban đầu ta có thể chưa biết m nên cần phải thử. Ta xây dựng hàm ***schroder()*** để thực hiện thuật toán trên:

```
function [r, iter] = schroder(f1, df1, m, x0, tol)
%Ham tim nghiem boi bang thuat toan Schroder
iter = 0;
d = feval(f1, x0)/feval(df1, x0);
while abs(d) > tol
   x1 = x0 - m*d;
   iter = iter + 1;
   x0 = x1;
   d = feval(f1, x0)/feval(df1, x0);
end
r = x0;
```

Để giải phương trình

$$(e^{-x} - x)^2 = 0$$

Ta dùng chương ***trình ctschroder.m*** với m = 2:

```
clear all, clc
[x, iter] = schroder('f1', 'df1', 2, -2, 1e-4)
```

Trong đó:

```
function y = f1(x)
y = (exp(-x) - x).^2;

function y = df1(x)
y = 2.0*(exp(-x) - x).*(-exp(-x) - 1);
```

CHƯƠNG 6: ĐẠO HÀM VÀ TÍCH PHÂN SỐ

§1. TÍNH ĐẠO HÀM BẬC NHẤT BẰNG PHƯƠNG PHÁP ROMBERG

Đạo hàm theo phương pháp Romberg là một phương pháp ngoại suy để xác định đạo hàm với một độ chính xác cao. Ta xét khai triển Taylor của hàm f(x) tại (x + h) và (x - h):

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x) + \cdots \quad (1)$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x) - \cdots \quad (2)$$

Trừ (1) cho (2) ta có:

$$f(x+h) - f(x-h) = 2hf'(x) + \frac{2h^3}{3!}f'''(x) + \frac{2h^5}{5!}f^{(5)}(x) + \cdots \quad (3)$$

Như vậy rút ra:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{3!}f'''(x) - \frac{h^4}{5!}f^{(5)}(x) - \cdots \quad (4)$$

hay ta có thể viết lại:

$$f'(x) = \frac{1}{2h}\left[f(x+h) - f(x-h)\right] + a_2h^2 + a_4h^4 + a_6h^6 + \cdots \quad (5)$$

trong đó các hệ số ai phụ thuộc f và x.

Ta đặt:

$$\varphi(h) = \frac{1}{2h}\left[f(x+h) - f(x-h)\right] \quad (6)$$

Như vậy từ (5) và (6) ta có:

$$D(1,1) = \varphi(h) = f'(x) - a_2h^2 - a_4h^4 - a_6h^6 - \cdots \quad (7)$$

$$D(2,1) = \varphi\left(\frac{h}{2}\right) = f'(x) - a_2\frac{h^2}{4} - a_4\frac{h^4}{16} - a_6\frac{h^6}{64} - \cdots \quad (8)$$

và tổng quát với $h_i = h/2^{i-1}$ ta có :

$$D(i,1) = \varphi(h_i) = f'(x) - a_2h_i^2 - a_4h_i^4 - a_6h_i^6 - \cdots \quad (9)$$

Ta tạo ra sai phân D(1,1) - 4D(2,1) và có:

$$\varphi(h) - 4\varphi\left(\frac{h}{2}\right) = -3f'(x) - \frac{3}{4}a_4h^4 - \frac{15}{16}a_6h^6 - \cdots \quad (10)$$

Chia hai vế của (10) cho -3 ta nhận được:

$$D(2,2) = \frac{4D(2,1) - D(1,1)}{4} = f'(x) + \frac{1}{4}a_4h^4 + \frac{5}{16}a_6h^6 + \cdots \quad (11)$$

Trong khi D(1, 1) và D(2, 1) sai khác f′(x) phụ thuộc vào h^2 thì D(2, 2) sai khác f′(x) phụ thuộc vào h^4 . Bây giờ ta lại chia đôi bước h và nhận được:

$$D(3,2) = f'(x) + \frac{1}{4}a_4\left(\frac{h}{2}\right)^4 + \frac{5}{16}a_6\left(\frac{h}{2}\right)^6 + \cdots \quad (12)$$

và khử số hạng có h^4 bằng cách tạo ra:

$$D(2,3) - 16D(3,2) = -15f'(x) + \cdots + \frac{15}{64}a_6h^6 \qquad (13)$$

Chia hai vế của (13) cho -15 ta có:

$$D(3,3) = \frac{16D(3,2) - D(2,2)}{15} = f'(x) - \frac{1}{64}a_6h^6 - \cdots \qquad (14)$$

Với lần tính này sai số của đạo hàm chỉ còn phụ thuộc vào h^6. Lại tiếp tục chia đôi bước h và tính D(4, 4) thì sai số phụ thuộc h^8. Sơ đồ tính đạo hàm theo phương pháp Romberg là :

D(1, 1)			
D(2, 1)	D(2, 2)		
D(3, 1)	D(3, 2)	D(3, 3)	
D(4, 1)	D(4, 2)	D(4, 3)	D(4, 4)

. . . . . .

trong đó mỗi giá trị sau là giá trị ngoại suy của giá trị trước đó ở hàng trên .

Với $2 \le j \le i \le n$ ta có:

$$D(i,j) = \frac{4^{j-1}D(i,j-1) - D(i-1,j-1)}{4^{j-1} - 1}$$

và giá trị khởi đầu là:

$$D(i,j) = \varphi(h_i) = \frac{1}{2h_i}\left[f(x+h_i) - f(x-h_i)\right]$$

với $h_i = h/2^{i-1}$.

Chúng ta ngừng lại khi hiệu giữa hai lần ngoại suy đạt độ chính xác yêu cầu. Ta xây dựng hàm ***diffromberg()*** để thực hiên thuật toán trên:

```
function df = diffromberg(f, x, h, maxiter, tol)
%Tinh dao ham bang phuong phap Romberg
D(1, 1) = (feval(f,x+h) - feval(f, x-h))/(2*h);
for i = 1:maxiter
   h = h/2;
  D(i + 1, 1) = (feval(f,x+h) - feval(f, x-h))/(2*h);
   for j = 1:i
      D(i + 1, j + 1) = (4^j*D(i + 1, j) - D(i, j))/(4^j - 1);
   end
   if (abs( D(i + 1, i + 1) - D(i, i) ) < tol)
      df = D(i+1, i+1);
      break;
   elseif ( i == maxiter )
      error( 'Ngoai suy Richardson khong hoi tu' );
   end
end
```

Để tính đạo hàm của hàm cho trước ta dùng chương trình ***ctdiffromberg.m***:

```
clear all, clc
format long;
f = inline('x^2 + 2*x*exp(x) + 1');
x = 2;
h = 0.5;
tol = 1e-6;
maxiter = 10;
df = diffromberg(f, x, h, maxiter, tol)
```

§2. TÍNH ĐẠO HÀM BẬC CAO

Ta xét khai triển Taylor của hàm f(x):

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x) + \cdots \qquad (1)$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x) - \cdots \qquad (2)$$

Từ (1) và (2) ta có:

$$\begin{aligned} D_{c2}^{(2)}(x,h) &= \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} \\ &= f''(x) + \frac{h^2}{12}f^{(4)}(x) + \frac{2h^4}{6!}f^{(6)}(x) + \cdots \end{aligned} \qquad (3)$$

Như vậy nếu ta tính đạo hàm cấp 2 theo (3) thì sai số cỡ h^2. Dùng phương pháp ngoại suy Richadson ta có:

$$\begin{aligned} \frac{2^2 D_{c2}^{(2)}(x,h) - D_{c2}^{(2)}(x,2h)}{2^2 - 1} &= \frac{-f(x+2h) + 16f(x+h) - 30f(x) + 16f(x+h) - f(x-2h)}{12h^2} \\ &= f''(x) - \frac{h^4}{90}f^{(5)}(x) + \cdots \end{aligned}$$

Do vậy:

$$\begin{aligned} D_{c2}^{(2)}(x,h) &= \frac{-f(x+2h) + 16f(x+h) - 30f(x) + 16f(x+h) - f(x-2h)}{12h^2} \\ &= f''(x) - \frac{h^4}{90}f^{(5)}(x) + \cdots \end{aligned} \qquad (4)$$

Nếu đạo hàm cấp được tính theo (4) thì sai số chỉ còn cỡ h^4. Từ (4) ta có:

$$D_{c2}^{(2)}(x,h) = \frac{c_2 f_2 + c_1 f_1 + c_0 f_0 + c_{-1} f_{-1} + c_{-2} f_{-2}}{h^2} \qquad (5)$$

Trong đó:

$f_2 = f(x + 2h)$

$f_1 = f(x + h)$
$f_0 = f(x)$
$f_{-1} = f(x - h)$
$f_{-2} = f(x - 2h)$

Viết rõ các khai triển Taylor của f_2, f_1, f_0, f_{-1}, f_{-2} ta có:

$$D_{c2}^{(2)}(x,h) = \frac{1}{h^2}\left\{\begin{array}{l} c_2\left[f_0 + 2hf_0' + \frac{(2h)^2}{2!}f_0'' + \cdots\right] + c_1\left[f_0 + hf_0' + \frac{h^2}{2!}f_0'' + \cdots\right] \\ +c_0 f_0 + c_{-1}\left[f_0 - hf_0' + \frac{h^2}{2!}f_0'' - \cdots\right] + c_{-2}\left[f_0 - 2hf_0' + \frac{(-2h)^2}{2!}f_0'' - \cdots\right] \end{array}\right\}$$

$$D_{c2}^{(2)}(x,h) = \frac{1}{h^2}\left\{\begin{array}{l} (c_2 + c_1 + c_0 + c_{-1} + c_{-2})f_0 + h(2c_2 + c_1 + c_{-1} - 2c_{-2})f_0' \\ +h^2\left(\frac{2^2}{2}c_2 + \frac{1}{2}c_1 + \frac{1}{2}c_{-1} + \frac{2^2}{2}c_{-2}\right)f_0'' + \cdots \end{array}\right\} \quad (6)$$

Ta phải giải hệ phương trình sau để tìm các hệ số c_i.

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 2 & 1 & 0 & -1 & -2 \\ 2^2/2! & 1/2! & 0 & 1/2! & 2^2/2! \\ 2^2/3! & 1/3! & 0 & -1/3! & -2^3/3! \\ 2^4/4! & 1/4! & 0 & 1/4! & -2^4/4! \end{bmatrix}\begin{bmatrix} c_2 \\ c_1 \\ c_0 \\ c_{-1} \\ c_{-2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad (7)$$

Kết quả ta có $c_2 = -1/12$, $c_1 = 4/3$, $c_0 = -5/2$, $c_{-1} = 4/3$ $c_{-2} = -1/12$. Do vậy:

$$D_{c2}^{(2)}(x,h) = \frac{-f_2 + 16f_1 - 30f_0 + 16f_{-1} - f_{-2}}{12h^2}$$

Tương tự ta có đạo hàm bậc 4 của hàm:

$$D_{c2}^{(4)}(x,h) = \frac{f_2 - 4f_1 + 6f_0 - 4f_{-1} + f_{-2}}{12h^4}$$

Ta xây dựng hàm ***diffn()*** để tính đạo hàm tới bậc 5:

```
function df = diffn(f, n, x)
% Tinh dao ham cap n cua f tai x
if n>5
   error('Ham chi tinh duoc dao ham den bac 5');
   return;
end;
N = 5;
xo = x;
T(1) = feval(f,xo);
```

```
h = 0.005;
tmp = 1;
for i = 1:N
   tmp = tmp*h;
   c = difapx(i,[-i i]); %he so cua dao ham
   dix = c*feval(f,xo + [-i:i]*h)';
   T(i+1) = dix/tmp; %dao ham
end
df = T(n+1);
h = 0.005;
tmp = 1;
for i = 1:N
   tmp = tmp*h;
   c = difapx(i,[-i i]); %he so cua dao ham
   dix = c*feval(f,xo + [-i:i]*h)'; %/h^i; %dao ham
   T(i+1) = dix/tmp; %he so cua chuoi Taylor
end
df = T(n+1);
```

Để tính đạo hàm của hàm ta dùng chương trình ***ctdiffn.m***

```
clear all, clc
f = inline('x.^2 + atan(x)','x');
df = diffn(f, 5, 0)
```

§3. TÍNH ĐẠO HÀM BẰNG PHƯƠNG PHÁP NỘI SUY

Giả sử ta có hàm cho dưới dạng bảng:

x	x_0	x_1	x_0	…	x_n
y	y_0	y_1	y_0	…	y_n

Để tìm đạo hàm của hàm tại một điểm nào đó ta sẽ nội suy hàm rồi sau đó tính đạo hàm của hàm tại điểm đã cho. Ta xây dựng hàm ***diffinterp()*** để thực hiện công việc trên.

```
function df = diffinterp(x, y, n, x0)
%Tinh dao ham cap 1 hai 2 bang phuogphap noi suy
px = lagrange(x, y); % Tim da thuc noi suy Lagrange (x, y)
[p, dp, ddp] = peval(px, x0);
fprintf('Tri so cua ham la:  %f\n',p)
if n ==1
   df = dp;
```

```
else
   df = ddp;
end
fprintf('Dao ham cap %d la: %f\n',n, df);
```

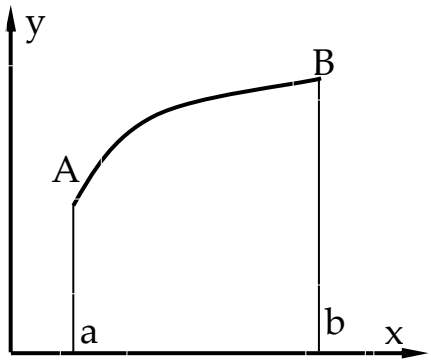
Để tính đạo hàm ta dùng chương trình ***ctdiffinterp.m***:

```
clear, clc
x0 = pi/4;
x = [2:6]*pi/16;
y = sin(x);
x = [1.5  1.9  2.1  2.6  3.2];
y = [1.0628  1.3961  1.5432  1.8423  2.0397];
n = 2;
df = diffinterp(x, y, n, x0);
```

§4. TÍCH PHÂN XÁC ĐỊNH

Mục đích của tính tích phân xác định, còn gọi là cầu phương, là đánh giá định lượng biểu thức:

$$J = \int_a^b f(x)dx$$



trong đó f(x) là hàm liên tục trong khoảng [a,b] và có thể biểu diễn bởi đường cong y = f(x). Như vậy tích phân xác định J là diện tích S_{ABba}, giới hạn bởi đường cong f(x), trục hoành, các đường thẳng x = a và x = b. Tích phân này thường được tính gần đúng bằng công thức:

$$J = \sum_{i=1}^{n} A_i f(x_i)$$

trong đó A_i là trọng số, phụ thuộc phương pháp tính tích phân.

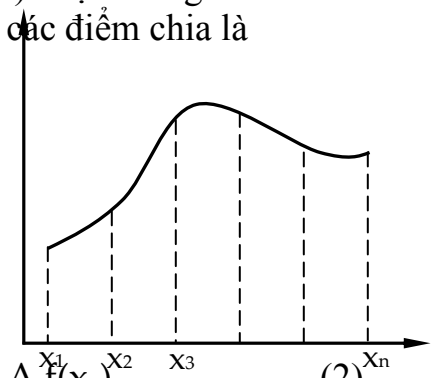
Tất cả các phương pháp tính tích phân được suy ra từ phương pháp nội suy hàm dưới dấu tích phân. Do vậy kết quả sẽ chính xác nếu hàm có thể xấp xỉ bằng đa thức. Các phương pháp tính tích phân xác định bằng phương pháp số được chia thành 2 nhóm: các phương pháp Newton - Cotes và các phương pháp Gauss. Khi dùng các phương pháp Newton - Cotes khoảng lấy tích phân được chia đều như trong phương pháp hình thang hay phương pháp Simpson. Khi dùng các phương pháp Gauss, cácc diểm chia được chọn để đạt độ chính xác cao nhất. Do phương pháp này cần ít lần tính giá trị hàm dươci dấu tích phân nên thích hợp khi hàm f(x) khó tính.

§5. CÁC CÔNG THỨC NEWTON - COTES

1. Khái niệm chung: Ta khảo sát tích phân

$$J=\int_a^b f(x)dx \qquad (1)$$

Ta chia miền lấy tích phân [a, b] thành (n - 1) đoạn bằng nhau có chiều dài mỗi đoạn h = (b - a)/(n - 1) như hình vẽ và kí hiệu các điểm chia là $x_1, x_2,.., x_n$. Sau đó ta xấp xỉ hàm f(x) bằng đa thức bậc (n - 1) đi qua các nút. Đa thức nội suy Lagrange của f(x) có dạng:



$$P_{n-1}(x)=\sum_{i=1}^{n} f(x_i)L_i(x)$$

Như vậy, xấp xỉ tích phân (1) là:

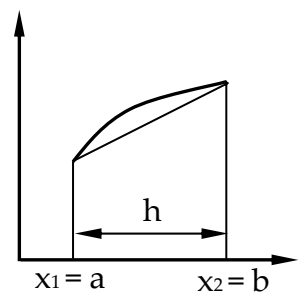
$$J=\int_a^b f(x)dx=\int_a^b P_{n-1}(x)dx=\sum_{i=1}^{n} f(x_i)\int_a^b L_i(x)dx=\sum_{i=1}^{n} A_i f(x_i) \qquad (2)$$

Trong đó:

$$A_i=\int_a^b L_i(x)dx \qquad i=1,2,...,n \qquad (3)$$

Công thức (2) là công thức Newton - Cotes. Với n = 2 ta có công thức hình thang và với n = 3 ta có công thức Simpson.

2. Phương pháp hình thang: Khi n = 2 ta có:



$$L_1(x)=\frac{x-x_2}{x_1-x_2}=-\frac{x-b}{h}$$

$$L_2(x)=\frac{x-x_1}{x_2-x_1}=\frac{x-a}{h}$$

$$A_1=-\frac{1}{h}\int_a^b (x-b)dx=\frac{1}{2h}(b-a)^2=\frac{h}{2}$$

$$A_2=\frac{1}{h}\int_a^b (x-a)dx=\frac{1}{2h}(b-a)^2=\frac{h}{2}$$

Vậy:

$$J=\left[f(a)+f(b)\right]\frac{h}{2}$$

Trong thực tế, phương pháp hình thang được áp dụng trên từng đoạn. Trên mỗi đoạn $[x_i, x_{i+1}]$ ta có:

$$J_i=\left[f(x_i)+f(x_{i+1})\right]\frac{h}{2}$$

và: $$J=\sum_{i=1}^{n} J_i=\left[f(x_1)+2f(x_2)+2f(x_3)+\cdots+2f(x_{n-1})+2f(x_n)\right]\frac{h}{2} \qquad (7)$$

Ta gọi H = b - a. Nếu tích phân trên được tính chỉ bởi k hình thang thì:

k = 1: $$J_1 = \left[f(a) + f(b) \right] \frac{H}{2} \qquad (8)$$

k = 2: $$J_2 = \left[f(a) + 2f\left(a + \frac{H}{2} \right) + f(b) \right] \frac{H}{4} = \frac{1}{2} J_1 + f\left(a + \frac{H}{2} \right) \frac{H}{2}$$

k = 3: $$J_2 = \left[f(a) + 2f\left(a + \frac{H}{4} \right) + 2f\left(a + \frac{H}{2} \right) + 2f\left(a + \frac{3H}{4} \right) + f(b) \right] \frac{H}{8}$$

$$= \frac{1}{2} J_2 + \left[f\left(a + \frac{H}{4} \right) + f\left(a + \frac{3H}{4} \right) \right] \frac{H}{4}$$

Tổng quát, với k > 1 ta có:

$$J_k = \frac{1}{2} J_{k-1} + \frac{H}{2^{k-1}} \sum_{i=1}^{2^{k-1}} f\left[a + \frac{(2i-1)H}{2^{k-1}} \right] \quad k = 2,3,... \qquad (9)$$

Công thức (8) là công thức hình thang lặp. Ta thấy rằng tổng chỉ chứa các nút mới xuất hiện khi số hình thang tăng gấp đôi. Tính dãy J_1, J_2,... bằng (8) và (9) cần cùng một số lần tính như khi dùng (7). Nhưng khi dùng (8) và (9) ta kiểm tra được tính hội tụ và có thể dừng lặp khi đạt độ chính xác cho trước.

Ta xây dựng hàm ***trapezoid()*** để thực hiện thuật toán trên.

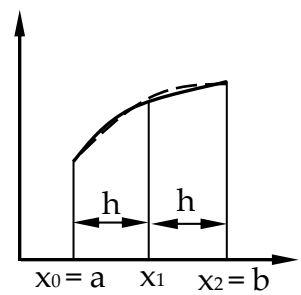
```
function J = trapezoid(f, a, b, maxiter, tol)
% Quy tac hinh thang lap.
% Cu phap: J = trapezoid(f, a, b, k)

fa = feval(f, a);
fb = feval(f, b);
J1 = (fa + fb)*(b - a)/2;
for k = 2:maxiter
   n = 2^(k -2 ); % so diem moi
   h = (b - a)/n ; % khoang chia moi
   x = a + h/2.0; % toa do diem moi thu nhat
   sum = 0.0;
   for i = 1:n
      fx = feval(f, x);
      sum = sum + fx;
      x = x + h;
   end
   J = (J1 + h*sum)/2;
   if abs(J1 - J) < tol
```

```
        break;
    end
    J1 = J;
end
```

Để tính tích phân ta dùng chương trình ***cttrapezoid.m***

```
clear all, clc
f = inline('(x^3+1)*sin(x)','x');
a = 0;
b = 1;
maxiter = 50;
tol = 1e-6;
J = trapezoid(f, a, b, maxiter, tol)
```

3. Phương pháp Simpson: Khi n = 3 ta có công thức Simpson. Qua 3 điểm, hàm f(x) được xấp xỉ bằng một hàm bậc hai(một parabol). Để tính tích phân ta thay hàm f(x) ở vế phải bằng đa thức nội suy Newton tiến bậc 2:



$$P_2 = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 \qquad (10)$$

và ta có :

$$\int_a^b f(x)dx = \int_a^b P_2(x)dx \qquad (11)$$

Đổi biến $x = x_1 + th$ thì $dx = hdt$. Với $x = x_1$ thì $t = 0$ và với $x = x_3$ thì $t = 2$ nên:

$$\begin{aligned}\int_a^b P_2(x)dx &= h\int_0^2\left[y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0\right]dt \\ &= h\left[y_0 t + \frac{t^2}{2}\Delta y_0 + \frac{1}{2}\left(\frac{t^3}{3} - \frac{t^2}{2}\right)\Delta^2 y_0\right]_{t=0}^{t=2} \\ &= h\left[2y_0 + 2\Delta y_0 + \frac{1}{2}\left(\frac{8}{3} - \frac{4}{2}\right)\Delta^2 y_0\right] \\ &= \frac{h}{3}\left(y_0 + 4y_1 + y_2\right) = \frac{h}{3}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]\end{aligned} \qquad (12)$$

Thực tế ta chia đoạn [a, b] thành 2n phần và tính tích phân trên mỗi đoạn con. Cộng các tích phân trên các đoạn con ta có:

$$\int_a^b f(x)dx = \frac{h}{3}\left[y_0 + 4\left(y_1 + y_3 + \cdots + y_{2n-1}\right) + 2\left(y_2 + y_4 + \cdots + y_{2n-2}\right) + y_{2n}\right] \qquad (13)$$

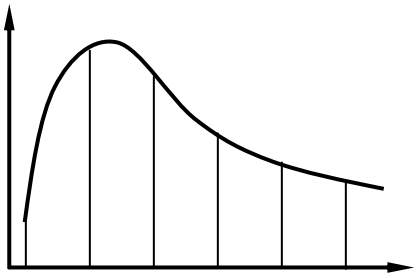
Công thức (13) đòi hỏi n là số chẵn.

Ta xây dựng hàm ***simpson()*** để thực hiện thuật toán trên

```
function s = simpson(f, a, b, n)
%n so khoang chia
%neu  f chua trong mot file dung ki hieu @ de goi
% s = simpson(@f, a, b, n).
%neu f la ham inline
% s = simpson(f, a, b, n).
if mod(n, 2) ~= 0
   n = n + 1
end
h = (b - a)/(2*n);
s1 = 0;
s2 = 0;
for k = 1:n
  x = a + h*(2*k-1);
  s1 = s1+ f(x);
end
for k = 1:(n-1)
  x = a + h*2*k;
  s2 = s2 + f(x);
end
s = h*(f(a) + f(b) + 4*s1 + 2*s2)/3;
clc
```

Để tính tích phân ta dùng chương trình ***ctsimpson.m***:

```
clear all, clc
f = inline('exp(x).*sin(x)','x');
a = 0;
b = 1;
n = 6;
s = simpson(f, a, b, n)
```



3. Phương pháp cầu phương thích nghi: Trong tích phân bằng phương pháp Simpson, các đoạn được chia đều và làm cho sai số không giống nhau trên các đoạn: sai số lớn trên các đoạn hàm biến đổi nhiều và sai số nhỏ trên các đoạn hàm tương đối bằng phẳng. Ngược lại phương pháp cầu phương thích nghi chia các đoạn không đều: ngắn trên các đoạn hàm thay đổi nhiều và dài trên các đoạn thay đổi ít và sẽ có sai số nhỏ khi số đoạn chia nhỏ.

Thuật toán cầu phương thích nghi bắt đầu bằng việc tính tích phân int đối với toàn bộ đoạn [a, b] và tổng tích phân int12 = int1 + int2 trên 2 đoạn bằng nhau. Dựa trên int và int12 ta tính sai số. Nếu chưa đạt độ chính xác, ta chia đôi mỗi đoạn và lặp lại quá trình tính. Ta dùng hàm ***adaptivesimpson()*** để thực hiện thuật toán này:

```
function int = adaptivesimpson(f, a, b, tol)
 mid = (b + a)/2.0;
 int = simpsonapprox (f, a, b);
 int12 = simpsonapprox (f, a, mid) + simpsonapprox (f, mid, b);
 if( abs(int - int12) < 15.0*tol )
    int = int12;
 else
   leftint = adaptivesimpson (f, a, mid, tol/2);
   rightint  = adaptivesimpson (f, mid, b, tol/2);
   int = leftint + rightint;
 end

function int = simpsonapprox (f, a, b)
h = (b - a)/2.0;
int = h*( feval(f, a) + 4.0*feval(f, (a + h)) + feval(f, b) )/3.0;
```

Để tính tích phân ta dùng chương trình **ctadaptive.m**:

```
clc, clear all
f = inline('sqrt(x).*cos(x)');
a = 0;
b = 1;
tol = 1e-5;
J = adaptivesimpson(f, a, b, tol)
```

§6. TÍCH PHÂN ROMBERG

Tích phân Romberg kết hợp quy tắc tích phân hình thang với phương pháp ngoại suy Richardson. Trước hết ta đưa vào khái niệm:

$R_{i,1} = J_i$

Trong đó J_i là giá trị xấp xỉ của $\int_a^b f(x)dx$ có được bằng cách tính theo quy tắc lặp hình thang lần thứ i.

Tích phân Romberg bắt đầu từ $R_{1,1} = J_1$ (một hình thang) và $R_{2,1} = J_2$ (hai hìn thang). Sau đó tính $R_{2,2}$ bằng cách ngoại suy:

$$R_{2,2} = \frac{2^2 R_{2,1} - R_{1,1}}{2^2 - 1} = \frac{4}{3}R_{2,1} - \frac{1}{3}R_{1,1} \qquad (1)$$

Để tiện dùng ta lưu các kết quả vào mảng dạng:

$$\begin{bmatrix} R_{1,1} & \\ R_{2,1} & R_{2,2} \end{bmatrix}$$

Bước tiếp theo là tính $R_{3,1} = J_3$ (bốn hình thang) và lặp lại ngoại suy Richadson ta có:

$$R_{3,2} = \frac{2^2 R_{3,1} - R_{2,1}}{2^2 - 1} = \frac{4}{3} R_{3,1} - \frac{1}{3} R_{2,1} \qquad (2)$$

và: $$R_{3,3} = \frac{2^4 R_{3,2} - R_{2,2}}{2^4 - 1} = \frac{16}{15} R_{3,2} - \frac{1}{15} R_{2,2} \qquad (3)$$

Các phần tử của R bây giờ gồm:

$$\begin{bmatrix} R_{1,1} & & \\ R_{2,1} & R_{2,2} & \\ R_{3,1} & R_{3,2} & R_{3,3} \end{bmatrix}$$

Công thức tổng quát dùng trong sơ đồ này là:

$$R_{i,j} = \frac{4^{j-1} R_{i,j-1} - R_{i-1,j-1}}{4^{j-1} - 1} \quad i > 1, j = 2, 3, ... \qquad (4)$$

Ta xây dựng hàm ***romberg()*** để thực hiện thuật toán trên:

```
function J = romberg(f, a, b, maxiter, tol)
m = 1;
h = b-a;
err = 1;
j = 0;
R = zeros(4, 4);
R(1,1) = h*(f(a) + f(b))/2;
while((err > tol) & (j < maxiter))|(j < 4)
  j = j + 1;
  h = h/2;
  s = 0;
  for p =1:m
    x = a + h*(2*p-1);
    s = s + f(x);
  end
  R(j+1, 1) = R(j, 1)/2 + h*s;
  m = 2*m;
  for k = 1:j
    R(j+1, k+1) = R(j+1, k) + (R(j+1, k) - R(j,k))/(4^k-1);
  end
  err = abs(R(j, j)-R(j+1, k+1));
end
```

```
J = R(j+1, j+1);
```

Để tính tích phân ta dùng chương trình ***ctromberg.m***:

```
clear all, clc
f = inline('exp(x).*sin(x)','x');
a = 0;
b = 1;
maxiter = 20;
tol = 1e-6;
J = romberg(f, a, b, maxiter, tol)
```

§7. TÍCH PHÂN BOOL

Ta khảo sát hàm y = f(x) trên đoạn $[x_0, x_4]$, với:

$x_1 = x_0 + h$, $x_2 = x_0 + 2h$, $x_3 = x_0 + 3h$, $x_4 = x_0 + 4h$

Theo Bool, tích phân:

$$J = \int_{x_0}^{x_4} f(x)dx = \frac{2h}{45}\sum_{k=1}^{m} 7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)$$

Xét tích phân:

$$J = \int_{a}^{b} f(x)dx$$

Ta chia đoạn [a, b] thành 4m đoạn con đều nhau có độ rộng $h = \frac{b-a}{4m}$ bởi các điểm chia $x_k = x_0 + hk = a + hk$, k = 0, 1,..., 4m. Công thức Bool cho 4m đoạn con là:

$$J = \int_{a}^{b} f(x)dx = \frac{2h}{45}\sum_{k=1}^{m} 7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)$$

Ta xây dựng hàm ***intbool()*** để thực hiện thuật toán này

```
function tp = intbool(f, a, b, m)
%Tinh tich phan bang phuong phap Bool
a = 0;
b = 2;
m = 2;
h = (b - a)/(4*m);
for k = 1:4*m
   x(k) = a + k*h;
end
tp = 0;
j = 1;
for k = 1:m
```

```
    tp = tp + (7*feval(f, a) + 32*feval(f, x(j)) +...
        12*feval(f, x(j+1)) + 32*feval(f, x(j+2)) + 7*feval(f, x(j+3)));
    a = x(4*k);
    j = 4*k + 1;
end
tp = tp*h*2/45;
```

Để tính tích phân của một hàm ta dùng chương trình ***ctintbool.m***:

```
clear all, clc
format long
f = inline('x.*sin(x)');
a = 0;
b = 2;
m = 2;
J = intbool(f, a, b, m)
```

§8. CÔNG THỨC TÍCH PHÂN FILON

Giả sử cần tính tích phân:

$$J = \int_a^b f(x)\cos(\omega x)dx$$

Lúc đó ta có thể dùng công thức tích phân Filon:

$$\int_{x_0}^{x_n} f(x)\cos(tx)dx$$

$$= h\left\{\alpha(th)\left[f_{2n}\sin(tx_{2n})\text{-}f_{2n}\sin(tx_{2n})\right] + \beta(th)C_{2n} + \gamma(th)C_{2n-1} + \frac{2}{45}th^4 S'_{2n}\right\}$$

Trong đó:

$a = x_0$, $b = x_n$, $t = \omega$

$$C_{2n} = \sum_{i=0}^{n} f_{2i}\cos(tx_{2i}) - 0.5\left[f_{2n}\cos(tx_{2n}) + f_0\cos(tx_0)\right]$$

$$C_{2n-1} = \sum_{i=0}^{n} f_{2i-1}\cos(tx_{2i\text{-}1})$$

$$S'_{2n-1} = \sum_{i=1}^{n} f'''_{2i-1}\sin(tx_{2i-1})$$

$$\alpha(\theta) = \frac{1}{\theta} + \frac{\sin 2\theta}{2\theta^2} - \frac{\sin^2\theta}{\theta^3}$$

$$\beta(\theta) = 2\left[\frac{1+\cos^2\theta}{\theta^2} - \frac{\sin 2\theta}{\theta^3}\right]$$

$$\gamma(\theta) = 4\left(\frac{\sin\theta}{\theta^3} - \frac{\cos\theta}{\theta^2}\right)$$

Ta xây dựng hàm ***filon()*** để thực hiện các công thức trên:

```
function int = filon(f, a, b, t, m, key)
% ham  filon tinh gan dung tich phan
%    ∫_a^b f(x)cos(tx)dx     neu key = 1,
% hay
%    ∫_a^b f(x)sin(tx)dx     neu  key = 2,
% dung m diem theo quy tac Filon (m le).
if (any(size(a) ~= [1 1]))
   error ('Thong so a  phai la so.') ;
end
if (any(size(b) ~= [1 1]))
   error (Thong so b nhap vao phai la so.') ;
end
if (any(size(t) ~= [1 1]))
   error ('Thong so t  phai la so.') ;
end
if (any(size(m) ~= [1 1]))
   error ('Thong so m  phai la so.') ;
end
if (any([(fix(m) ~= m) (rem(m, 2) == 0)]))
   error ('Thong so m  phai la so le.') ;
end
if (m < 3)
   error ('Thong so m  phai lon hon 3.') ;
end
if (all([(key ~= 1) (key ~= 2)]))
   error ('Thong so  key  phai la 1 hoac 2.') ;
end
n = m - 1 ;
h = (b - a)/n;
th = t*h ;
thh = th*th ;
if (abs(th) >= 0.1)
   s = sin(th) ;
   c = cos(th) ;
   alfa = (1.0 + s*(c - 2.0*s/th)/th)/th ;
```

```
    beta = 2.0*(1.0 + c*c - 2.0*s*c/th)/thh ;
    gamma = 4.0*(s/th - c)/thh ;
  else
    alfa = th*thh*(2.0/45.0 + thh*(-2.0/315.0 + 2.0*thh/4725.0)) ;
    beta = 2.0/3.0 + thh*(2.0/15.0 + thh*(4.0/105.0 + 2.0*thh/567.0)) ;
    gamma = 4.0/3.0 + thh*(-2.0/15.0 + thh*(1.0/210.0 - thh/11340.0)) ;
  end
  args = [a  b];
  fbounds = feval(f, args) ;
  s1 = sin(a*t) ;
  s2 = sin(b*t) ;
  c1 = cos(a*t) ;
  c2 = cos(b*t) ;
  if (key == 1)
    sum = s2*fbounds(2) - s1*fbounds(1) ;
    sum0 = 0.5*(c1*fbounds(1) + c2*fbounds(2)) ;
    if (n > 2)
      args = (a + (2:2:n-2)*h)' ;
      sum0 = sum0 + cos(t*args)'*feval(f, args) ;
    end
    args = (a + (1:2:n-1)*h)' ;
    sum1 = cos(t*args)'*feval(f, args) ;
  else
    sum = c1*fbounds(1) - c2*fbounds(2) ;
    %sum = -(c1*fbounds(1) - c2*fbounds(2)) ;
    sum0 = 0.5*(s1*fbounds(1) + s2*fbounds(2)) ;
    %if (n == 2)
    if (n > 2)
      args = (a + (2:2:n-2)*h)' ;
      sum0 = sum0 + sin(t*args)'*feval(f, args) ;
    end
    args = (a + (1:2:n-1)*h)' ;
    sum1 = sin(t*args)'*feval(f, args) ;
  end
  int = h*(alfa*sum + beta*sum0 + gamma*sum1) ;
```

Khi tính tích phân ta dùng chương trình ***ctintfilon.m***:

```
  clear all, clc
  a = 0;
  b = 2;
  key = 2;
```

```
t = 3;
m = 51;
f = inline('(x.^3+1).*sin(x)');
J = filon(f, a, b, t, key)
```

§9. QUY TẮC HARDY

Để tính tích phân $J = \int_a^b f(x)dx$ ta có thể dùng công thức Hardy:

$$\int_{x_1}^{x_7} f(x)dx = 0.01h\left(28f_1 + 162f_2 + 220f_4 + 162f_6 + 28f_7\right)$$

Để tăng độ chính xác ta dùng phương pháp chia đoạn [a, b] thành m đoạn và trên mỗi đoạn ta dùng công thức Hardy. Ta xây dựng hàm ***inthardy()*** để thực hiện công thức trên:

```
function tp = inthardy(f, a, b, m)
%Tinh tich phan bang phuong phap Hardy
h = (b - a)/(6*m);
for k = 1:6*m
   x(k) = a + k*h;
end
tp = 0;
j = 1;
for k = 1:m
   tp = tp + (28*feval(f, a) + 162*feval(f, x(j)) +...
         220*feval(f, x(j+2)) + 162*feval(f, x(j+4)) + 28*feval(f, x(j+5)));
   a = x(6*k);
   j = 6*k + 1;
end
tp = tp*h*0.01;
```

Để tính tích phân ta dùng chương trình ***ctinthardy.m***:

```
clear all, clc
format long
f = inline('exp(x).*sin(x)','x');
a = 0;
b = 2;
m = 20;
J = inthardy(f, a, b, m)
```

§10. QUY TẮC DURANT

Để tính tích phân $J = \int_a^b f(x)dx$ ta có thể dùng công thức Durant:

$$\int_{x_1}^{x_n} f(x)dx = h\left(\frac{2}{5}f_1 + \frac{11}{10}f_2 + f_3 + \cdots + f_{n-2} + \frac{11}{10}f_{n-1} + \frac{2}{5}f_n\right)$$

Ta xây dựng hàm ***intdurant()*** để thực hiện công thức trên:

```
function tp = intdurant(f, a, b, n)
%Tinh tich phan bang phuong phap Durant
h = (b - a)/(n);
for k = 1:n
   x(k) = a + k*h;
end
tp = 0;
for i = 2:n-2
   tp = tp+ feval(f, x(i));
end
tp = tp + 0.4*feval(f, a) + 1.1*feval(f, x(1)) +...
      1.1*feval(f, x(n-1)) + 0.4*feval(f, x(n));
tp = h*tp;
```

Để tính tích phân ta dùng chương trình ***ctintdurant.m***:

```
clear all, clc
format long
f = inline('1./(1 + x.^2)');
a = 0;
b = 2;
n = 50;
J = intdurant(f, a, b, n)
```

§11. QUY TẮC SHOVELTON

Để tính tích phân $J = \int_a^b f(x)dx$ ta có thể dùng công thức Shovelton:

$$\int_{x_1}^{x_{11}} f(x)dx = \frac{5}{126}h\left[8\left(f_1 + f_{11}\right) + 35\left(f_2 + f_4 + f_8 + f_{10}\right) + 15\left(f_3 + f_5 + f_7 + f_9\right) + 36f_6\right]$$

Để tăng độ chính xác ta dùng phương pháp chia đoạn [a, b] thành m đoạn và trên mỗi đoạn ta dùng công thức Shovelton. Ta xây dựng hàm ***intshovelton()*** để thực hiện công thức trên:

```
function tp = intshovelton(f, a, b, m)
%Tinh tich phan bang phuong phap Shovelton
h = (b - a)/(10*m);
for k = 1:10*m
   x(k) = a + k*h;
end
tp = 0;
j = 1;
for k = 1:m
   tp = tp + 8*(feval(f, a) + feval(f, x(j+9))) +...
   35*(feval(f, x(j)) + feval(f, x(j+2)) + feval(f, x(j+6)) + feval(f, x(j+8))) +...
   15*(feval(f, x(j+1)) + feval(f, x(j+3)) + feval(f, x(j+5)) + feval(f, x(j+7)))
+...
   36*feval(f, x(j+4));
   a = x(10*k);
   j = 10*k + 1;
end
tp = tp*h*5/126;
```

Để tính tích phân ta dùng chương trình ***ctshovelton.m***:

```
clear all, clc
format long
f = inline('1./(1 + x.^2)');
a = 0;
b = 2;
m = 20;
J = intshovelton(f, a, b, m)
```

§12. QUY TẮC WEDDLE

Để tính tích phân $J = \int_a^b f(x)dx$ ta có thể dùng công thức Weddle:

$$\int_{x_1}^{x_7} f(x)dx = 0.3h\left(f_1 + 5f_2 + f_3 + 6f_4 + f_5 + 5f_6 + f_7\right)$$

Để tăng độ chính xác ta dùng phương pháp chia đoạn [a, b] thành m đoạn và trên mỗi đoạn ta dùng công thức Weddle. Ta xây dựng hàm ***intweddle()*** để thực hiện công thức trên:

```
function tp = intweddle(f, a, b, m)
%Tinh tich phan bang phuong phap Weddle
h = (b - a)/(6*m);
```

```
for k = 1:6*m
   x(k) = a + k*h;
end
tp = 0;
j = 1;
for k = 1:m
   tp = tp + feval(f, a) + 5*feval(f, x(j)) +...
        feval(f, x(j+1)) + 6*feval(f, x(j+2)) +...
        feval(f, x(j+3)) + 5*feval(f, x(j+4)) + feval(f, x(j+5));
   a = x(6*k);
   j = 6*k + 1;
end
tp = tp*h*0.3;
```

Để tính tích phân ta dùng chương trình ***ctweddle.m***:

```
format long
f = inline('exp(x).*sin(x)','x');
a = 0;
b = 2;
m = 20;
J = intweddle(f, a, b, m)
```

§13. CẦU PHƯƠNG GAUSS

1. Các công thức tích phân Gauss: Trong phần này chúng ta sẽ xét một số phương pháp cầu phương Gauss:

- Tích phân Gauss - Legendre dùng xấp xỉ:

$$\int_a^b f(t)dt$$

- Tích phân Gauss - Hermite dùng xấp xỉ:

$$\int_{-\infty}^{\infty} e^{-t^2} f(t)dt$$

- Tích phân Gauss - Laguerre dùng xấp xỉ:

$$\int_{-\infty}^{\infty} e^{-t} f(t)dt$$

- Tích phân Gauss - Chebyshev 1 dùng xấp xỉ:

$$\int_{-1}^{1} \frac{1}{\sqrt{1-t^2}} f(t)dt$$

- Tích phân Gauss - Chebyshev 2 dùng xấp xỉ:

$$\int_{-1}^{1}\sqrt{1-t^2}f(t)dt$$

2. Tích phân Gauss - Legendre: Nếu hàm dưới dấu tích phân f(t) là đa thức bậc nhỏ hơn hay bằng 3 (bằng 2n - 1) thì tích phân:

$$\int_{a}^{b}f(t)dt \qquad (1)$$

có thể tính chính xác bởi 2(n) điểm bằng cách dùng công thức:

$$J[t_1, t_2] = w_1f(t_1) + w_2f(t_2) \qquad (2)$$

với w_1 và w_2 là các trọng số và t_1, t_2 là các nút.

$$f(t) = 1 \qquad w_1f(t_1)+w_2f(t_2)=w_1+w\equiv\int_{-1}^{+1}1dt=2 \qquad (3a)$$

$$f(t) = t \qquad w_1f(t_1)+w_2f(t_2)=w_1t_1+w_2t_2\equiv\int_{-1}^{+1}tdt=0 \qquad (3b)$$

$$f(t) = t^2 \qquad w_1f(t_1)+w_2f(t_2)=w_1t_1^2+w_2t_2^2\equiv\int_{-1}^{+1}t^2dt=\frac{2}{3} \qquad (3c)$$

$$f(t) = t^3 \qquad w_1f(t_1)+w_2f(t_2)=w_1t_1^3+w_2t_2^3\equiv\int_{-1}^{+1}t^3dt=0 \qquad (3d)$$

Nhân (3b) với t_1^2 và trừ kết quả cho (3d) ta có:

$$w_2(t_2^3-t_1^2t_2)=0 \quad \text{nên} \quad t_2 = -t_1$$

Thay $t_2 = -t_1$ vào (3b) ta có:

$$(w_1 - w_2)t_1 = 0 \quad \text{nên} \quad w_1 = w_2$$

Thay $w_1 = w_2$ vào (3a) ta có:

$$w_1 + w_2 = 2 \quad \text{nên} \quad w_1 = w_2 = 1$$

Thay $w_1 = w_2 = 1$ vào (3c) ta có:

$$t_1^2+(-t_1)^2=\frac{2}{3} \quad \text{nên} \quad t_1=-t_2=-\frac{1}{\sqrt{3}}$$

Như vậy (2) trở thành:

$$J[t_1,t_2]=f\left(-\frac{1}{\sqrt{3}}\right)+f\left(\frac{1}{\sqrt{3}}\right) \qquad (4)$$

nghĩa là công thức xấp xỉ này cho ta kết quả chính xác của (1) khi $n \le 3$.

Khi đa thức dưới dấu tích phân có bậc nhỏ hơn (2n - 1) thì công thức tích phân Gauss - Legendre:

$$J_{GL}[t_1,t_2,\ldots,t_n]=\sum_{i=1}^{n}w_if(t_i) \qquad (5)$$

cho tích phân chính xác của đa thức. Các điểm nút (n nút) là nghiệm của đa thức Legendre bậc n:

$$L_n(t) = \sum_{i=0}^{n/2} (-1)^i \frac{(2n-2i)!}{2^n i!(n-i)!(n-2i)!} t^{n-2i} \tag{6a}$$

hay tính theo công thức lặp:

$$L_n(t) = \frac{(2n-1)tL_{n-1}(t) - (n-1)L_{n-2}(t)}{n} \tag{6b}$$

Để tạo ra đa thức Legendre ta dùng hàm ***legendre()***:

```
function p = legendre(n)
%tao ra da thuc Legendre
p0 = 1;
p1 = [1  0];
if n <= 0
   p = p0;
elseif n == 1
   p = p1;
end
for k = 2:n
   p = ((2*k - 1)*[p1  0] - (k - 1)*[0  0  p0])/k;
   p0 = p1;
   p1 = p;
end
```

Cho n điểm nút, ta có thể tính các trọng số tương ứng của công thức tích phân Gauss - Legendre n điểm bằng cách giải hệ phương trình tuyến tính:

$$\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ t_1 & t_2 & t_i & \cdots & t_n \\ t_1^2 & t_2^2 & t_i^2 & \cdots & t_n^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ t_1^{n-1} & t_2^{n-1} & t_i^{n-1} & \cdots & t_n^{n-1} \end{bmatrix} \begin{bmatrix} w_{n,1} \\ w_{n,1} \\ w_{n,i} \\ \vdots \\ w_{n,n} \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ \left[1-(-1)^i\right]/n \\ \vdots \\ \left[1-(-1)^n\right]/n \end{bmatrix} \tag{7}$$

trong đó phần tử thứ i của vế phải RHS(i) là:

$$\mathrm{RHS}(i) = \int_{-1}^{+1} t^{i-1} dt = \frac{1}{n} t^i \Big|_{-1}^{+1} = \frac{1-(-1)^i}{i} \tag{8}$$

Thủ tục tìm n điểm nút và các trọng số $w_{n,i}$ của công thức tích phân Gauss - Legendre được viết trong hàm ***gausslegendre()***:

```
function [t, w] = gausslegendre(n)
if n < 0
   fprintf('\nBac cua da thuc Legendre phai la so duong\n');
   return;
else
   t = roots(legendre(n))'; %tao thanh mot hang cua ma tran;
   A(1,:) = ones(1, n);
   b(1) = 2;
   for i = 2:n % Pt.(7)
      A(i, :) = A(i - 1, :).*t;
      if mod(n, 2) == 0
         b(i) = 0;
      else b(i) = 2/n; % Pt.(8)
      end
   end
   w = b/A';
end
```

Khi tính tích phân trên đoạn [a, b] ta phải biến đổi nó về đoạn [-1, 1] bằng cách đặt:

$$x = \frac{(b-a)t + a + b}{2} \qquad dx = \frac{b-a}{2} dt \tag{9}$$

Như vậy ta có thể viết lại công thức tích phân Gauss - Legendre trên đoạn [a, b] là:

$$J = \int_a^b f(x)dx = \frac{b-a}{2} \int_{-1}^{+1} f[x(t)]dt$$

và xấp xỉ bằng công thức:

$$J[x_1, x_2, \ldots, x_n] = \frac{b-a}{2}\sum_{i=1}^{n} w_i f(x_i) \text{ với } x_i = \frac{(b-a)t_i + a + b}{2} \quad (10)$$

Việc chuyển đổi này được thực hiện bằng hàm ***intglegendre()***:

```
function J = intglegendre(f, a, b, n)
[t, w] = gausslegendre(n);
x = ((b - a)*t + a + b)/2;%Pt. (9)
fx = feval(f, x);
J = w*fx'*(b - a)/2;% Pt. (10)
```

Để tính tích phân ta dùng chương trình ***ctgausslegendre.m***:

```
clc, clear all
% tinh tich phan ham f(x) tren doan [a, b]
% n <= 25;
f = inline('exp(x).*sin(x)','x');
a = 0;
b = 1;
n = 20;
J = intglegendre(f, a, b, n)
```

3. Tích phân Gauss - Hermite: Công thức tích phân Gauss - Hermite có dạng:

$$J_{GH}[t_1, t_2, \ldots, t_n] = \sum_{i=1}^{n} w_i f(t_i) \quad (11)$$

dùng để tính tích phân của hàm mũ e^{-t^2} nhân với f(t) là đa thức bậc nhỏ hơn (2n - 1) trên đoạn [-∞, ∞]:

$$J = \int_{-\infty}^{+\infty} e^{-t^2} f(t)dt \quad (12)$$

Các điểm nút là nghiệm của đa thức Hermite:

$$H_n = \sum_{i=0}^{n/2} \frac{(-1)^i}{i!} n(n-1)\cdots(n-2i+1)(2t)^{n-2i} \quad (13a)$$

hay:

$$H_n(t) = 2tH_{n-1}(t) - H'_{n-1}(t) \quad (13b)$$

Để tạo ra các hệ số của đa thức Hermite theo (13b) ta xây dựng hàm ***hermite()***:

```
function p = hermite(n)
%Hn + 1(x) = 2xHn(x)-Hn'(x)
if n <= 0
   p = 1;
else
   p = [2  0];
```

```
    for n = 2:n
        p = 2*[p 0] - [0 0 polyder(p)]; %Pt.(13b)
    end
end
```

Cho n điểm nút ta tìm các trọng số của công thức tích phân Gauss - Hermite bằng cách giải hệ phương trình (7) với vế phải là:

$$\begin{aligned}\mathrm{RHS}(1) &= \int_{-\infty}^{+\infty} e^{-t^2}dt = \sqrt{\int_{-\infty}^{+\infty} e^{-x^2}dx \int_{-\infty}^{+\infty} e^{-y^2}dy} \\ &= \sqrt{\int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} e^{-(x^2+y^2)}dxdy} = \sqrt{\int_{-\infty}^{+\infty} e^{-r^2}2\pi r dr} = \sqrt{-\pi e^{-r^2}\Big|_0^{\pi}} = \sqrt{\pi}\end{aligned} \tag{14a}$$

$$\begin{aligned}&\mathrm{RHS}(n) = 0 \text{ nếu n chẵn} \\ &\mathrm{RHS}(n) = \int_{-\infty}^{+\infty} e^{-t^2}t^{n-1}dt = \int_{-\infty}^{+\infty}(-2t)e^{-t^2}\frac{1}{-2}t^{n-2}dt \\ &= -\frac{1}{2}e^{-t^2}t^{n-2}\Big|_{-\infty}^{+\infty} + \frac{1}{2}(n-2)\int_{-\infty}^{+\infty} e^{-t^2}t^{n-3}dt = \frac{1}{2}(n-2)\mathrm{RHS}(n-2)\end{aligned} \tag{14b}$$

Thủ tục tìm n nút và các trọng số tương ứng của công thức tích phân Gauss - Hermite được thực hiện trong hàm ***gausshermite()***:

```
function [t, w] = gausshermite(n)
if n < 0
    error('Bac cua da thuc Hermite phai la so duong');
end
t = roots(hermite(n))';
A(1,:) = ones(1, n);
b(1) = sqrt(pi);
for n = 2:n
    A(n,:) = A(n - 1,:).*t; %Pt.(7)
    if mod(n,2) == 1
        b(n) = (n - 2)/2*b(n - 2); %Pt.(14)
    else
        b(n) = 0;
    end
end
w = b/A';
```

Ta xây dựng hàm ***intghermite()*** để tính tích phân:

```
function J = intglegendre(f, n)
```

```
[t, w] = gausshermite(n);
fx = feval(f, t);
J = w*fx';% Pt. (11)
```

Để tính tích phân ta dùng chương trình ***ctgausshermite.m***:

```
clc, clear all
f = inline('(x^3+1)*sin(x)','x');
n = 15;
J = intghermite(@f1, n)
```

4. Tích phân Gauss - Laguerre: Khi dùng công thức tích phân Gauss - Laguerre (5) trở thành:

$$J_{GLa}\left[t_1, t_2, \ldots, t_n\right] = \sum_{i=1}^{n} w_i f(t_i) \tag{16}$$

và cho ta kết quả của tích phân:

$$J = \int_{-\infty}^{+\infty} e^{-t} f(t)dt \tag{17}$$

Các điểm nút là nghiệm của đa thức Laguerre bậc n:

$$L_n = \sum_{0}^{n} \frac{(-1)^i}{i!} \frac{n^i}{(n-i)!i!} t^i \tag{18a}$$

hay:

$$(n+1)L_{n+1}(t) = (2n+1-t)L_n(t) - nL_{n-1}(t) \tag{18b}$$

Để tạo ra đa thức Laguerre bậc n ta dùng hàm ***laguerre()***:

```
function p = laguerre(n)
%Ln + 1(x) = (2n + 1 - x)Ln(x) - nLn-1(x)
p0 = 1;
p1 = [-1  1];
if n <= 0
   p = p0;
elseif n == 1
   p = p1;
end
for k = 2:n
   p = (-[p1  0] + (2*k - 1)*[0 p1]  - (k - 1)*[0  0  p0])/k;
   p0 = p1;
   p1 = p;
end
```

Cho n nút t_i ta có thể tìm các trọng số tương ứng bằng cách giải hệ phương trình đại số tuyến tính (7) với vế phải là:

$$RHS(1) = \int_0^\infty e^{-t}dt = -e^{-t}\Big|_0^\infty = 1 \tag{19a}$$

$$\begin{aligned} RHS(n) &= \int_0^\infty e^{-t}t^{n-1}dt = -e^{-t}t^{n-1}\Big|_0^\infty + (n-1)\int_0^\infty e^{-t}t^{n-2}dt \\ &= (n-1)RHS(n-1) \end{aligned} \tag{19b}$$

Để tính toạ độ các nút và trọng số tương ứng ta dùng hàm ***gausslaguerre()***:

```
function [t, w] = gausslaguerre(n)
if n < 0
   error('Bac cua da thuc Laguerre phai la so duong');
end
t = roots(laguerre(n))';
A(1, :) = ones(1, n);
b(1) = 1;
for i = 2:n
   A(i, :) = A(i - 1,:).*t; %Pt.(7)
   b(i) = (i - 1)*b(i - 1); %Pt.(15)
end
w = b/A';
```

Hàm ***intglaguerre()*** *dùng để tính tích phân*:

```
function J = intglaguerre(f, n)
[t, w] = gausslaguerre(n);
fx = feval(f, t);
J = w*fx';% Pt. (10)
```

Để tính tích phân ta dùng chương trình ***ctgausslaguerre.m***:

```
clear all, clc
format long
f = inline('(x.^2).*cos(x)','x');
n = 10;
J = intglaguerre(f, n)
```

5. Tích phân Gauss - Chebyshev: Công thức tính tích phân Gauss - Chebyshev 1 có dạng:

$$J_{GC1}[t_1, t_2, \ldots, t_n] = \sum_{i=1}^{n} w_i f(t_i) \tag{20}$$

Công thức (20) cho ta tính tích phân:

$$J = \int_{-1}^{+1} \frac{1}{\sqrt{1-t^2}} f(t)dt \qquad (21)$$

Các điểm nút là nghiệm của đa thức Chebyshev bậc n:

$$t_i = \cos\frac{(2i-1)\pi}{2n} \quad i = 1, 2,..., n \qquad (22)$$

Các trọng số tương ứng là:

$$w_{in} = \frac{\pi}{i} \quad i = 1, 2,..., n \qquad (23)$$

Ta xây dựng hàm ***gausscheb1()*** để tính các toạ độ các nút và trọng số tương ứng:

```
function [t, w] = gausscheb1(n)
if n < 0
   error('Bac cua da thuc Chebyshev phai la so duong');
end
for i = 1:n
   t(i) = cos((2*i - 1)*pi/(2*n));
   w(i) = pi/n;
end
```

Hàm ***intgcheb1()*** dùng để tính tích phân theo công thức Gauss - Chebyshev 1:

```
function J = intgcheb1(f, n)
[t, w] = gausscheb1(n);
fx = feval(f, t);
fx = sqrt(1 - t.^2).*fx;
J = w*fx';% Pt. (21)
```

Để tính một tích phân nào đó ta dùng chương trình ***ctgausscheb1.m***:

```
clear all, clc
format long
f = inline('1./(1 + x.^2)');
n = 16;
J = intgcheb1(f, n)
```

Công thức tính tích phân Gauss - Chebyshev 2 có dạng:

$$J_{GC1}[t_1, t_2, \ldots, t_n] = \sum_{i=1}^{n} w_i f(t_i) \qquad (24)$$

Công thức (24) cho ta tính tích phân:

$$J = \int_{-1}^{+1} \sqrt{1-t^2} f(t)dt \qquad (25)$$

Các điểm nút là nghiệm của đa thức Chebyshev bậc n:

$$t_i = \cos\left(\frac{i\pi}{n+1}\right) \qquad i = 1, 2,..., n \tag{26a}$$

Các trọng số tương ứng là:

$$w_{in} = \frac{\pi}{n+1}\sin^2\left(\frac{i\pi}{n+1}\right) \qquad i = 1, 2,..., n \tag{26b}$$

Ta xây dựng hàm ***gausscheb2()*** để tính các toạ độ các nút và trọng số tương ứng:

```
function [t, w] = gausscheb2(n)
if n < 0
   error('Bac cua da thuc Chebyshev phai la so duong');
end
for i = 1:n
   t(i) = cos(i*pi/(n + 1));
   w(i) = (pi/(n + 1))*(sin(i*pi/(n + 1))).^2;
end
```

Hàm ***intgcheb2()*** dùng để tính tích phân theo công thức Gauss - Chebyshev 2:

```
function J = intgcheb2(f, n)
[t, w] = gausscheb2(n);
fx = feval(f, t);
fx = (1./sqrt(1 - t.^2)).*fx;
J = w*fx';% )
```

Để tính một tích phân nào đó ta dùng chương trình ***ctgausscheb2.m***:

```
clear all, clc
format long
f = inline('1./(1 + x.^2)');
n = 16;
J = intgcheb2(f, n)
```

6. Tích phân Gauss với điểm kì dị logarit: Ta có công thức cầu phương:

$$\int_0^0 f(x)\ln(x)dx = -\sum_{i=1}^{n} A_i f(x_i)$$

Trong đó với n = 6 ta có:

x_i	0.0216334	0.129583	0.314020	0.538657	0.756916	0.922669
A_i	0.238764	0.308287	0.245317	0.142009	0.0554546	0.010169

Ta xây dựng hàm ***intlogarit()*** để thực hiện tích phân này:

```
function tp = intlogarit(f)
n = 6;
x = [0.0216344; 0.129583; 0.314020; 0.538657; 0.756916; 0.922669];
A = [0.238764; 0.308287; 0.245317; 0.142009; 0.0554546; 0.010169];
tp = 0;
for i = 1:n
   t = feval(f, x(i));
   tp = tp + A(i)*t;
end
tp = - tp;
```

Để tính tích phân ta dùng chương trình ***ctintlogarit.m***:

```
clear all, clc
format long
f = inline('(x.^3+1).*sin(x)','x');
J = intlogarit(f)
```

§14. TÍCH PHÂN GAUSS - LOBATTO

Trong tích phân này các điểm cận trên và dưới đều là nút của cầu phương. Công thức cầu phương Gauss - Lobatto có dạng:

$$\int_{-1}^{1} f(x)dx = w_1 f(-1) + w_2 f(1) + \sum_{i=2}^{n-1} w_i f(x_i)$$

Ngoài hai điểm nút x = 1 và x = -1, các điểm nút khác có trọng số được xác định bằng:

$$w_i = \frac{2}{n(n-1)\left[P_{n-1}(x_i)\right]^{2'}}$$

Ta xây dựng hàm ***gausslobatto()*** để tính các nút và các trọng số tương ứng:

```
function [x, w ] = gausslobatto(N)
% Tinh cac nut va trong so Legendre-Gauss-Lobatto
N1 = N + 1;
% Dau tien dung cac nut Chebyshev-Gauss-Lobatto
x = cos(pi*(0:N)/N)';
% Tinh ma tran Legendre - Vandermonde
P = zeros(N1, N1);
xold = 2;
while max(abs(x-xold))>eps
   xold = x;
```

```
    P(:, 1) = 1;
    P(:, 2) = x;
    for k = 2:N
        P(:,k+1) = ( (2*k-1)*x.*P(:, k)  -(k-1)*P(:, k-1) )/k;
    end
    x = xold - ( x.*P(:, N1)-P(:, N) )./( N1*P(:, N1) );
end
w = 2./(N*N1*P(:, N1).^2);
```

Thông thường, ta cần tính tích phân trên đoạn [a, b] nên cần đổi biến. Tích phân trên [a, b] của hàm f(x) được tính nhớ hàm ***intgausslobatto()***:

```
function J = intgausslobatto(f, n, a, b)
[t, w] = gausslobatto(n);
x = ((b - a)*t + a + b)/2;%Pt. (9)
fx = feval(f, x);
J = w'*fx*(b - a)/2;% Pt. (10)
```

Để tính tích phân của hàm ta dùng chương trình chương trình ***ctintgausslobatto.m***:

```
clear all, clc
% tinh tich phan ham f(x) tren doan [a, b]
% n <= 25;
f = inline('(x.^3+1).*sin(x)','x');
a = 0;
b = 1;
n = 6;
J = intgausslobatto(f, n, a, b)
```

§15. TÍCH PHÂN GAUSS - KRONROD

Ta xét tích phân:

$$J = \int_a^b f(x)dx$$

Theo công thức cầu phương Gaus - Krỏnod ta có:

$$J = \int_a^b f(x)dx = \sum_{i=1}^{n} w_i f(x_i)$$

Trong đó các nút là x_i và các trọng số tương ứng là w_i. Các giá trị nay khi n = 41 được lưu trong hàm ***kronrod***:

```
function[x, w] = kronrod
%cac nut va trong so tuong ung trong tich phan Gauss - Kronrod
```

x(1) = - 0.9988590315882777;
x(2) = - 0.9931285991850949;
x(3) = - 0.9815078774502503;
x(4) = - 0.9639719272779138;
x(5) = - 0.9408226338317548;
x(6) = - 0.9122344282513259;
x(7) = - 0.8782768112522820;
x(8) = - 0.8391169718222188;
x(9) = - 0.7950414288375512;
x(10) = - 0.7463319064601508;
x(11) = - 0.6932376563347514;
x(12) = - 0.6360536807265150;
x(13) = - 0.5751404468197103;
x(14) = - 0.5108670019508271;
x(15) = - 0.4435931752387251;
x(16) = - 0.3737060887154196;
x(17) = - 0.3016278681149130;
x(18) = - 0.2277858511416451;
x(19) = - 0.1526054652409227;
x(20) = - 0.7652652113349733E-01;
x(21) = 0.0;
x(22) = 0.7652652113349733E-01;
x(23) = 0.1526054652409227;
x(24) = 0.2277858511416451;
x(25) = 0.3016278681149130;
x(26) = 0.3737060887154196;
x(27) = 0.4435931752387251;
x(28) = 0.5108670019508271;
x(29) = 0.5751404468197103;
x(30) = 0.6360536807265150;
x(31) = 0.6932376563347514;
x(32) = 0.7463319064601508;
x(33) = 0.7950414288375512;
x(34) = 0.8391169718222188;
x(35) = 0.8782768112522820;
x(36) = 0.9122344282513259;
x(37) = 0.9408226338317548;
x(38) = 0.9639719272779138;
x(39) = 0.9815078774502503;
x(40) = 0.9931285991850949;
x(41) = 0.9988590315882777;

w(1) = 0.3073583718520532E-02;
w(2) = 0.8600269855642942E-02;
w(3) = 0.1462616925697125E-01;
w(4) = 0.2038837346126652E-01;
w(5) = 0.2588213360495116E-01;
w(6) = 0.3128730677703280E-01;
w(7) = 0.3660016975820080E-01;
w(8) = 0.4166887332797369E-01;
w(9) = 0.4643482186749767E-01;
w(10) = 0.5094457392372869E-01;
w(11) = 0.5519510534828599E-01;
w(12) = 0.5911140088063957E-01;
w(13) = 0.6265323755478117E-01;
w(14) = 0.6583459713361842E-01;
w(15) = 0.6864867292852162E-01;
w(16) = 0.7105442355344407E-01;
w(17) = 0.7303069033278667E-01;
w(18) = 0.7458287540049919E-01;
w(19) = 0.7570449768455667E-01;
w(20) = 0.7637786767208074E-01;
w(21) = 0.7660071191799966E-01;
w(22) = 0.7637786767208074E-01;
w(23) = 0.7570449768455667E-01;
w(24) = 0.7458287540049919E-01;
w(25) = 0.7303069033278667E-01;
w(26) = 0.7105442355344407E-01;
w(27) = 0.6864867292852162E-01;
w(28) = 0.6583459713361842E-01;
w(29) = 0.6265323755478117E-01;
w(30) = 0.5911140088063957E-01;
w(31) = 0.5519510534828599E-01;
w(32) = 0.5094457392372869E-01;
w(33) = 0.4643482186749767E-01;
w(34) = 0.4166887332797369E-01;
w(35) = 0.3660016975820080E-01;
w(36) = 0.3128730677703280E-01;
w(37) = 0.2588213360495116E-01;
w(38) = 0.2038837346126652E-01;
w(39) = 0.1462616925697125E-01;
w(40) = 0.8600269855642942E-02;
w(41) = 0.3073583718520532E-02;

Ta dùng hàm ***intgkronrod()*** để chuyển đoạn lấy tích phân từ [-1 1] sang đoạn [a, b]:

```
function J = intgkronrod(f, a, b)
[t, w] = kronrod;
x = ((b - a)*t + a + b)/2;)
fx = feval(f, x);
J = w*fx'*(b - a)/2;
```

Để tính tích phân ta dùng chương trình ***ctgausskronrod.m***:

```
clc, clear all
% tinh tich phan ham f(x) tren doan [a, b]
f = @f2;
a = 0;
b = 1;
J = intgkronrod(f, a, b)
```

§16. TÍCH PHÂN GAUSS - JACOBI

Tích phân Gauss – Jacobi, còn gọi là tích phân Mehler, dùng để tính tích phân dạng:

Ta xét tích phân:

$$J = \int_a^b (1-x)^{\alpha}(1+x)^{\beta} f(x)dx$$

Theo công thức cầu phương Gaus - Jacobi ta có:

$$J = \int_a^b f(x)dx = \sum_{i=1}^{n} w_i f(x_i)$$

Trong đó x_i làn nghiệm của đa thức Jacobi. Các công thức tính đa thức Jacobi là:

$$P_0^{(\alpha,\beta)}(x) = 1$$

$$P_1^{(\alpha,\beta)}(x) = 0.5\left[2(\alpha+1) + (\alpha+\beta+2)(x-1)\right]$$

$$P_n^{(\alpha,\beta)}(x) = \frac{(b_{2n} + b_{3n}x)P_{n-1}^{(\alpha,\beta)}(x) - b_{4n}P_{n-2}^{(\alpha,\beta)}(x)}{b_{1n}}$$

Với: $b_{1n} = 2i(i + \alpha + \beta)(2i + \alpha + \beta - 2)$

$b_{2n} = (2i + \alpha + \beta - 1)(\alpha^2 - \beta^2)$

$b_{3n} = (2i + \alpha + \beta - 2)(2i + \alpha + \beta - 1)(2i + \alpha + \beta)$

$b_{4n} = 2(i + \alpha - 1)(i + \beta - 1)(2i + \alpha + \beta)$

Các trọng số w_i được xác định bằng:

$$w_i = \frac{\Gamma(i+\alpha+1)\Gamma(i+\beta+1)}{\Gamma(i+\alpha+\beta+1)} \frac{2^{2i+\alpha+\beta+1} n!}{(1-x_i^2)\left[V_n'(x_i)\right]^2}$$

với:

$$V_n = P_n^{(\alpha,\beta)}(x)\frac{2^n n!}{(-1)^n}$$

Ta xây dựng hàm ***gaussjacobi()*** để tìm x_i và w_i:

```
function [x, w] = gaussjacobi(n, alfa, beta)
%tinh cac trong so va hoanh do trong tich phan Gauss-Jacobi
p = [0.5*(alfa + beta + 2)   0.5*(alfa - beta)];
a  = 1;
b = p;
for i = 2:n+1
   b1 = 2*i*(i + alfa + beta)*(2*i + alfa + beta -2);
   b2 = (2*i + alfa + beta -1)*(alfa^2 - beta^2)/b1;
   b3 = ((2*i + alfa + beta -2)*(2*i + alfa + beta -1 )*(2*i + alfa + beta))/b1;
   b4= (2*(i + alfa -1)*(i + beta - 1)*(2*i + alfa + beta))/b1;
   s = [b3 b2];
   if i == n+1
      pn1 = conv(s, b) - [0 0 b4*a];
      break;
   else
      p = conv(s, b) - [0 0 b4*a];
   end
   a  = b;
   b = p;
end
x = roots(p);
w = zeros(n, 1);
dv = polyder(p);
if mod(n, 2) == 1
   sign = -1;
else
   sign = 1;
end
dv = dv*(2^n)*factorial(n)/sign;
pn1 = -pn1*(2^(n+1))*factorial(n+1)/sign;
for i = 1:n
   num = (2*n + alfa + beta +...

2)*gamma(n+alfa+1)*gamma(n+beta+1)*(2^(2*n+alfa+beta+1))*factorial(n
);
   den = (n + alfa + beta + 1)*gamma(n+alfa+beta+1)*polyval(dv,...
   x(i))*polyval(pn1, x(i));
   w(i) = num/den;
end
```

Tiếp theo ta xây dựng hàm ***intgaussjacobi()*** để tính tích phân:

```
function J = intgaussjacobi(f, n, alf, bta)
[t, w] = gaussjacobi(n, alf, bta);
fx = feval(f, t);
J = w'*fx;
```

Để tính tích phân ta dùng chương trình ***ctgaussjacobi.m***:

```
clear al, clc
f = inline('exp(x).*sin(x)','x');
n = 6;%n <= 40
alfa = 1;
beta = 0;
J = intgaussjacobi(f, n, alfa, beta)
```

§17. TÍCH PHÂN RADAU

Cầu phương Radau dùng để tính tích phân:

$$J = \int_{-1}^{1} f(x)dx \qquad (1)$$

Theo công thức cầu phương Radau ta có:

$$J = \int_{-1}^{1} f(x)dx = w_1 f(-1) + \sum_{i=2}^{n} w_i f(x_i) \qquad (2)$$

Điểm (-1) là một một nút trong số các nút của cầu phương. Các hoành độ còn lại là nghiệm của đa thức:

$$\frac{P_{n-1}(x) - P_n(x)}{1+x} \qquad (3)$$

Trong đó P(x) là đa thức Legendre. Các trọng số tương ứng được tính theo:

$$w_i = \frac{1-x_i}{n^2\left[P_{n-1}(x_i)\right]^2} \qquad (4)$$

và điểm cuối có:

$$w_1 = \frac{2}{n^2}$$

Ta xây dựng hàm ***radau()*** để tính các hoành độ x_i và trọng số w_i:

```
function [x , w] = radau(n)
%Tinh cac hoanh do va trong so trong cau phuong Radau
tol = 1e-8;
%danh gia ban dau cac hoanh do la ca nut Chebyshev-Gauss-Radau
x(1:n,1) = - cos(2.0*pi*(0:n-1)/(2*n-1))';
```

```
p = zeros ( n, n+1 );
xold(1:n,1) = 2.0;
while (tol < max(abs(x(1:n,1)-xold(1:n,1))))
   xold = x;
   p(1,1:n+1) = (-1.0).^(0:n);
   p(2:n,1) = 1.0;
   p(2:n,2) = x(2:n,1);
   for j = 2:n
     p(2:n,j+1) = ((2*j - 1)*x(2:n,1).*p(2:n,j)+ ( -j+1)*p(2:n,j-1))/j;
   end
   x(2:n,1) = xold(2:n,1) - ((1.0 - xold(2:n,1))/n)...
     .*(p(2:n,n)+p(2:n,n+1))./(p(2:n,n) - p(2:n,n+1));
end
w = zeros(n,1);
w(1) = 2/n^2;
```

Tiếp theo ta xây dựng hàm ***intradau()***, để tính tích phân. Trong hàm ta đổi cận lấy tích phân trong khoảng [-1, 1] thành tích phân trong khoảng:

```
function J = intradau(f, n, a, b)
[t, w] = radau(n);
x = ((b - a)*t + a + b)/2;
fx = feval(f, x);
J = w'*fx*(b - a)/2;
```

Để tính tích phân ta dùng chương trình ***ctradau.m***:

```
clear al, clc
f = inline('x.*sin(x)');
n = 6;%n <= 40
a = 1;
b = 3;
J = intradau(f, n, a, b)
```

§18. TÍCH PHÂN CHEBYSHEV - RADAU

Cầu phương Chebyshev – Radau dùng tính tích phân:

$$J = \int_{-1}^{1} f(x)dx \qquad (1)$$

Theo công thức cầu phương Chebyshev - Radau ta có:

$$J = \int_{-1}^{1} f(x)dx = \sum_{i=2}^{n} w_i \left[f(x_i) - f(-x_i) \right] \qquad (2)$$

Các hoành độ x_i và trọng số tương ứng w_i cho trong bảng:

x_i	0.3549416	0.6433097	0.7783202	0.9481574
w_i	0.1223363	0.1223363	0.1223363	0.1223363

Ta xây dựng hàm ***chebradau()*** để chứa các giá trị x và w:

```
function [x, w] = chebradau
x(1) = 0.3549416;
x(2) = 0.6433097;
x(3) = 0.7783202;
x(4) = 0.9481574;

w(1) = 0.1223363;
w(2) = 0.1223363;
w(3) = 0.1223363;
w(4) = 0.1223363;
```

và hàm ***intchebradau()*** để tính tích phân:

```
function J = intchebradau(f, a, b)
[t, w] = chebradau;
fx1 = feval(f, t);
fx2 = feval(f, -t);
J = (w*fx1' - w*fx2');
```

Để tính tích phân của một hàm cụ thể ta dùng chương trình ***ctchebradau.m***:

```
clear al, clc
f = inline('exp(x).*sin(x)','x');
J = intchebradau(f)
```

§19. TÍCH PHÂN GAUSS – RADAU

Công thức cầu phương Gauss - Radau có dạng:

$$\int_{-1}^{1} f(x)dx = w_1 f(-1) + w_2 f(1) + \sum_{i=2}^{n-1} w_i f(x_i)$$

Ngoài hai điểm nút x = 1, các điểm nút khác là nghiệm của đa thức $P_n(x) + P_{n+1}(x)$, với P(x) là đa thức Legendre. Các trọng số được xác định bằng:

$$w_1 = \frac{2}{(n+1)^2}$$

$$w_i = \frac{1 - x_i}{(n+1)\left[P_{n+1}(x_i)\right]^2}$$

Ta xây dựng hàm ***gaussradau()*** để tính các hoành độ và trọng số của cầu phương:

```
function [x, w] = gaussradau(N)
% tinh cac nut va trong so cua cau phuong Gauss - Radau
N1 = N + 1;
% dung cac nut Chebyshev-Gauss-Radau lam xap xi dau tien
x = -cos(2*pi*(0:N)/(2*N+1))';
P = zeros(N1, N1+1);
xold = 2;
free = 2:N1;
while max(abs(x-xold)) > eps
   xold = x;
   P(1, :) = (-1).^(0:N1);
   P(free, 1) =1;
   P(free, 2) = x(free);
   for k = 2:N1
      P(free, k+1) = ( (2*k - 1)*x(free).*P(free, k) - (k - 1)*P(free, k-1) )/k;
   end
x(free) = xold(free) - ((1-xold(free))/N1).*(P(free, N1) + P(free,
N1+1))..../(P(free, N1) - P(free, N1+1));
end P = P(1:N1, 1:N1);
w = zeros(N1, 1);
w(1) =2/N1^2;
w(free) = (1-x(free))./(N1*P(free, N1)).^2;
```

Ta dùng hàm ***intgaussradau()*** để tính tích phân của hàm f(x) trên đoạn [a, b]:

```
function J = intgaussradau(f, n, a, b)
[t, w] = gaussradau(n);
x = ((b - a)*t + a + b)/2;
fx = feval(f, x);
J = w'*fx*(b - a)/2;
```

Để tính tích phân của hàm ta dùng chương trình ***ctgaussradau.m***:

```
clear all, clc
f = inline('exp(x).*sin(x)','x');
n = 6;
a = 1;
b = 3;
```

J = intgaussradau(f, n, a, b)

§20. ĐA THỨC NỘI SUY VÀ TÍCH PHÂN SỐ

Khi hàm được cho dưới dạng bảng số, để tính tính phân của hàm ta thực hiện các bước sau:

- Tìm đa thức nội suy, có thể là đa thức Lagrange, đa thức Newton...
- Tìm tích phân của đa thức nội suy

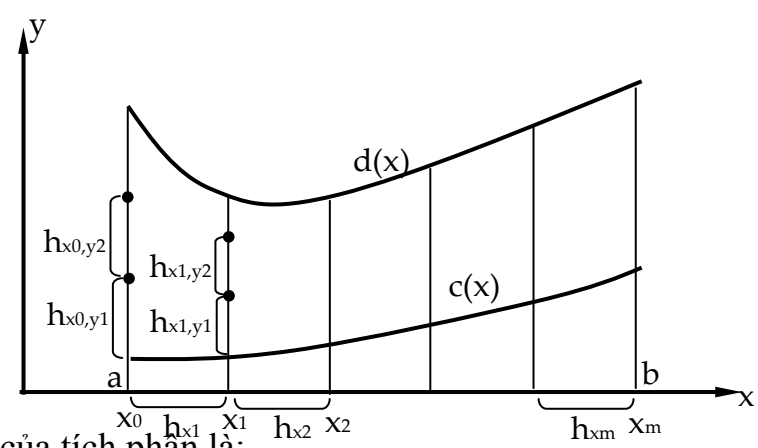
Sau đây chúng ta sẽ xây dựng chương trình ***ctinterp.m*** để tính tích phân dùng đa thức nội suy Lagrange:

```
clear all, clc
x = [1.0000   1.2000   1.4000   1.6000   1.8000   2.0000];
y = [1.9221   1.9756   1.6517   0.8501  -0.4984  -2.4199];
l = lagrange(x, y);
n = length(l);
p = conv(l, [1 0]);
n = length(p);
for i = 1:n-1
   q(i) = p(i)/(n-i);
end
q(n) = p(n);
tp = polyval(q, x(length(x))) - polyval(q, x(1));
```

§21. TÍCH PHÂN KÉP

1. Khái niệm chung: Ta khảo sát tích phân của hàm z = f(x, y) trên miền $R = \{(x, y \mid a \le x \le b, c(x) \le y \le d(x)\}$ như hình vẽ. Ta cần tính tích phân:

$$J = \iint_R f(x,y)dxdy = \int_a^b \left\{ \int_{c(x)}^{d(x)} f(x,y)dy \right\} dx$$



Công thức gần đúng của tích phân là:

$$J[a,b,c(x),d(x)] = \sum_{i=1}^{m} w_i \sum_{j=1}^{n} v_j f(x_i, y_{i,j})$$

với các trọng số w_i, v_j tuỳ thuộc vào cáh tính tích phân hàm một biến.

Ta xây dựng hàm ***int2simpson()*** để tính tích phân kép bằng công thức Simpson.

```
function J = int2simpson(f, a, b, c, d, m, n)
% tich phan kep cua ham f(x,yen mien R = {(x,y)|a <= x <= b, c(x) <= y <=
d(x)}
% dung quy tac Simpson
if ceil(m) ~= floor(m) %be rong co dinh cua cac doan tren x
   hx = m;
   m = ceil((b - a)/hx);
end
if mod(m, 2) ~= 0
   m = m + 1;
end
hx = (b - a)/m;
i = 1:m+1;
x = a + (i - 1)*hx;
if isnumeric(c)
   cx(i) = c; %neu c la hang so
else
   cx(i) = feval(c,x(i)); %khi c la ham cua c x
end
if isnumeric(d)
   dx(i) = d; %neu d la hang so
else
   dx(i) = feval(d,x(i)); %khi d la ham cua x
end
if ceil(n) ~= floor(n) %be rong co dinh theo y
   hy = n;
   nx(i) = ceil((dx(i)- cx(i))/hy);
   ind = find(mod(nx(i),2) ~= 0);
   nx(ind) = nx(ind) + 1;
else %so khoang co dinh
   if mod(n, 2) ~= 0
      n = n + 1;
   end
   nx(i) = n;
end
for i = 1:m + 1
   sx(i) = simpsonfxy(f,x(i),cx(i),dx(i),nx(i));
```

```
end
kodd = 2:2:m;
keven = 3:2:m - 1; %the set of odd/even indices
J = hx/3*(sx(1) + sx(m + 1) + 4*sum(sx(kodd)) + 2*sum(sx(keven)));

function Jf = simpsonfxy(f, x, c, d, n)
%tich phan mot bien cua f(x,y) voi Ry = {c <= y <= d}
if nargin < 5
   n = 100;
end
if abs(d - c)< eps | n <= 0
   Jf = 0;
   return;
end
if mod(n, 2) ~= 0
   n = n + 1;
end
h = (d - c)/n;
y = c + [0:n]*h;
fxy = feval(f,x,y);
fxy(find(fxy == inf)) = realmax;
fxy(find(fxy == -inf)) = -realmax;
kodd = 2:2:n;
keven = 3:2:n - 1;
Jf = h/3*(fxy(1) + fxy(n + 1) + 4*sum(fxy(kodd)) + 2*sum(fxy(keven)));
```

Để tính thể tích của hình cầu ta dùng chương trình ***ctint2simp.m***:

```
clear all, clc
%Tinh the tich hinh cau
x = [-1:0.05:1];
y = [0:0.05:1];
[X,Y] = meshgrid(x,y);
f = inline('sqrt(max(1 - x.*x - y.*y,0))','x','y');
Z = f(X,Y);
mesh(x,y,Z);
a = -1;
b = 1;
c = 0;
d = inline('sqrt(max(1 - x.*x,0))','x');
Vs1 = int2simpson(f, a, b, c, d, 100, 100)% so diem cho truoc
error1 = Vs1 - pi/3
Vs2 = int2simpson(f, a, b, c, d, 0.01, 0.01) %be rong cac doan cho truoc
```

error2 = Vs2 - pi/3

CHƯƠNG 7: CÁC PHƯƠNG TRÌNH VI PHÂN THƯỜNG

§1. BÀI TOÁN CAUCHY

Một phương trình vi phân cấp 1 có thể viết dưới dạng giải được $y' = f(x, y)$ mà ta có thể tìm được hàm y từ đạo hàm của nó. Tồn tại vô số nghiệm thoả mãn phương trình trên. Mỗi nghiệm phụ thuộc vào một hằng số tuỳ ý. Khi cho trước giá trị ban đầu của y là y_o tại giá trị đầu x_o ta nhận được một nghiệm riêng của phương trình. Bài toán Cauchy (hay bài toán có điều kiện đầu) tóm lại như sau: cho x sao cho $b \geq x \geq a$, tìm y(x) thoả mãn điều kiện:

$$\begin{cases} y'(x) = f(x, y) \\ y(a) = \alpha \end{cases} \qquad (1)$$

Người ta chứng minh rằng bài toán này có một nghiệm duy nhất nếu f thoả mãn điều kiện Lipschitz:

$$\left| f(x, y_1) - f(x, y_2) \right| \leq L \left| y_1 - y_2 \right|$$

với L là một hằng số dương.

Người ta cũng chứng minh rằng nếu f'_y (đạo hàm của f theo y) là liên tục và bị chặn thì f thoả mãn điều kiện Lipschitz.

Một cách tổng quát hơn, người ta định nghĩa hệ phương trình bậc 1:

$$y'_1 = f_1(x, y_1, y_2, ..., y_n)$$
$$y'_2 = f_2(x, y_1, y_2, ..., y_n)$$
$$....$$
$$y'_n = f_n(x, y_1, y_2, ..., y_n)$$

Ta phải tìm nghiệm $y_1, y_2, ..., y_n$ sao cho:

$$\begin{cases} Y'(x) = f(x, X) \\ Y(a) = \alpha \end{cases}$$

với:

$$Y' = \begin{pmatrix} y'_1 \\ y'_2 \\ .. \\ .. \\ y'_n \end{pmatrix} \qquad F = \begin{pmatrix} f_1 \\ f_2 \\ .. \\ .. \\ f_n \end{pmatrix} \qquad Y = \begin{pmatrix} y_1 \\ y_2 \\ .. \\ .. \\ y_n \end{pmatrix}$$

Nếu phương trình vi phân có bậc cao hơn (n), nghiệm sẽ phụ thuộc vào n hằng số tuỳ ý. Để nhận được một nghiệm riêng, ta phải cho n điều kiện đầu. Bài toán sẽ có giá trị đầu nếu với giá trị x_o đã cho ta cho $y(x_o)$, $y'(x_o)$, $y''(x_o)$,....

Một phương trình vi phân bậc n có thể đưa về thành một hệ phương trình vi phân cấp 1. Ví dụ nếu ta có phương trình vi phân cấp 2:

$$\begin{cases} y'' = f(x, y, y') \\ y(a) = \alpha, y'(a) = \beta \end{cases}$$

Khi đặt u = y và v = y′ ta nhận được hệ phương trình vi phân cấp 1:

$$\begin{cases} u' = v \\ v' = g(x, u, v) \end{cases}$$

với điều kiện đầu: u(a) = α và v(a) = β

Các phương pháp giải phương trình vi phân được trình bày trong chương này là các phương pháp rời rạc: đoạn [a, b] được chia thành n đoạn nhỏ bằng nhau được gọi là các "bước" tích phân h = (b - a) / n.

§2. PHƯƠNG PHÁP EULER

Giả sử ta có phương trình vi phân:

$$\begin{cases} y'(x) = f(x, y) \\ y(a) = \alpha \end{cases} \qquad (1)$$

và cần tìm nghiệm của nó. Ta chia đoạn $[x_o, x]$ thành n phần bởi các điểm chia:

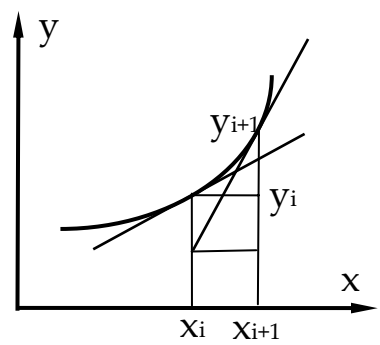
$$x_o < x_1 < x_2 < ... < x_n = x$$

Theo công thức khai triển Taylor một hàm lân cận x_i ta có:

$$y(x_{i+1}) = y(x_i) + (x_{i+1} - x_i)y'(x_i) + \frac{(x_{i+1} - x_i)^2}{2} y''(x_i) + \frac{(x_{i+1} - x_i)^3}{6} y'''(x_i) + \cdots$$

Nếu $(x_{i+1} - x_i)$ khá bé thì ta có thể bỏ qua các số hạng $(x_{i+1} - x_i)^2$ và các số hạng bậc cao

$$y(x_{i+1}) = y(x_i) + (x_{i+1} - x_i)y'(x_i)$$



Trường hợp các mốc cách đều:

$$(x_{i-1} - x_i) = h = (x - x_o)/ n$$

thì ta nhận được công thức Euler đơn giản:

$$y_{i+1} = y_i + hf(x_i, y_i) \qquad (2)$$

Về mặt hình học ta thấy (1) cho kết quả càng chính xác nếu bước h càng nhỏ.

Ta xây dựng hàm ***euler()*** để thực hiện thuật toán trên:

```
function [X, Y] = euler(fxy, xo, xf, yo, n)
%   %Giai phuong trinh  y'(x) = f(x,y(x)) hay y' = f(x)
if n < 2
   n = 2;
end
h = (xf - xo)/n;
X = zeros(n+1, 1);
M = max(size(yo));% so phuong trinh (so cot cua ma tran Y)
Y = zeros(n+1, M);
%dat dieu kien dau
x = xo;
```

```
X(1) = x;
y = yo;
Y(1,:) = y';
for i = 1:n
    if nargin(fxy) > 1
        k1 = h*feval(fxy, x, y);
    else
        k1 = h*feval(fxy, x);
    end
   y = y + k1;
   x = x + h;
   X(i+1) = x;
   Y(i+1, :) = y';
end

function dy = f1(t, y)
dy = zeros(3, 1);
dy(1) = y(2) * y(3);
dy(2) = -y(1) * y(3);
dy(3) = -0.51 * y(1) * y(2);
```

Để giải phương trình cho bởi hàm f1(x, y) ta dùng chương trình ***cteuler.m***:

```
clear all, clc
a = 0;
b = 1;
y = @f1;
ya = [0 1 1]';
m = 200;
[x, y] = euler(y, a, b, ya, m)
plot(x, y);
```

§3. PHƯƠNG PHÁP HEUN

Phương pháp Heun còn được gọi là phương pháp hình thang hay phương pháp. Cho phương trình:

y’ = f(t, y)

Ta có:

$$y\Big|_{t_k}^{t_{k+1}} = y(t_{k+1}) - y(t_k) = \int_{t_k}^{t_{k+1}} f(t,y)dt$$

hay:

$$y(t_{k+1}) = y(t_k) + \int_{t_k}^{t_{k+1}} f(t,y)dt \qquad \text{với } y(t_0) = y_0$$

Nếu ta sử dụng quy tắc tích phân hình thang thì ta có:

$$y_{k+1} = y_k + \frac{h}{2}\left[f(t_k, y_k) - f(t_{k+1}, y_{k+1})\right]$$

Vế phải (RHS) của phương trình này có y_{k+1} là gái trị chưa biết tại thời điểm t_k. Để giải quyết vấn đề này ta thay y_{k+1} ở RHS bằng công thức xấp xỉ:

$$y_{k+1} \cong y_k + hf(t_k, y_k)$$

Như vậy:

$$y_{k+1} = y_k + \frac{h}{2}\left\{f(t_k, y_k) + f\left[(t_{k+1}, y_k + hf(t_k, y_k)\right]\right\}$$

Đây chính là công thức Heun.

Ta xây dựng hàm ***heun()*** để thực hiện thuật toán trên:

```
function [X, Y] = heun(fxy, xo, xf, yo, n)
%Giai phuong trinh  y'(x) = f(x,y(x)) hay y' = f(x)
%dung thuat toan Heun voi n buoc tinh
if n < 2
   n = 2;
end
h = (xf - xo)/n;
X = zeros(n+1, 1);
M = max(size(yo));% so phuong trinh (so cot cua ma tran Y)
Y = zeros(n+1, M);
%dat dieu kien dau
x = xo;
X(1) = x;
y = yo;
Y(1,:) = y';
for i = 1:n
   if nargin(fxy) > 1
      f1 = feval(fxy, x, y);
      f2 = feval(fxy, x+h, y+h*f1);
   else
      f1 = feval(fxy, x);
      f2 = feval(fxy, x+h);
   end
   y = y + h*(f1 + f2)/2;
   x = x + h;
   X(i+1) = x;
   Y(i+1, :) = y.';
```

```
end
```

Để giải phương trình ta dùng chương trình ***ctheun.m***:

```
clear all, clc
a = 0;
b = 1;
y = inline('2*x + y');
ya = 0.5;
n = 10;%so lan tinh chi n = 10
[x, y] = heun(y, a, b, ya, n)
plot(x, y);
```

§4. PHƯƠNG PHÁP RUNGE - KUTTA

Mặc dù phương pháp Heun tốt hơn phương pháp Euler nhưng nó vẫn chưa đủ độ chính xác đối với các bài toán thực tế.

Xét bài toán Cauchy (1). Giả sử ta đã tìm được giá trị gần đúng y_i của $y(x_i)$ và muốn tính y_{i+1} của $y(x_{i+1})$. Trước hết ta viết công thức Taylor:

$$y(x_{i+1}) = y(x_i) + hy'(x_i) + \frac{h^2}{2}y''(x_i) + \cdots + \frac{h^m}{m!}y^{(m)}(x_i) + \frac{h^{m+1}}{m!}y^{(m+1)}(c) \quad (11)$$

với $c \in (x_i, x_{i+1})$ và:

$$y'(x_i) = f[x_i, y(x_i)]$$

$$y^{(k)}(x_i) = \frac{d^{k-1}}{dx^{k-1}}f[x_i, y(x_i)]$$

Ta viết lại (11) dưới dạng:

$$y_{i+1} - y_i = hy'(x_i) + \frac{h^2}{2}y''(x_i) + \cdots + \frac{h^m}{m!}y^{(m)}(x_i) + \frac{h^{m+1}}{m!}y^{(m+1)}(c) \qquad (12)$$

Ta đã kéo dài khai triển Taylor để kết quả chính xác hơn. Để tính y'_i, y''_i v.v. ta có thể dùng phương pháp Runge-Kutta bằng cách đặt:

$$y_{i+1} - y_i = r_1k_1^{(i)} + r_2k_2^{(i)} + r_3k_3^{(i)} + r_4k_4^{(i)} \qquad (13)$$

trong đó:

$$\begin{cases} k_1^{(i)} = hf(x_i, y_i) \\ k_2^{(i)} = hf(x_i + ah, y_i + \alpha k_1^{(i)}) \\ k_3^{(i)} = hf(x_i + bh, y_i + \beta k_1^{(i)} + \gamma k_2^{(i)}) \\ ..... \end{cases} \qquad (14)$$

và ta cần xác định các hệ số a, b,..; α, β, γ,...; r_1, r_2,.. sao cho vế phải của (13) khác với vế phải của (12) một vô cùng bé cấp cao nhất có thể có đối với h.

Khi dùng công thức Runge-Kutta bậc hai ta có:

$$\begin{cases} k_1^{(i)} = hf(x_i, y_i) \\ k_2^{(i)} = hf(x_i + ah, y_i + \alpha k_1^{(i)}) \end{cases} \quad (15)$$

và $\quad y_{i+1} - y_i = r_1 k_1^{(i)} + r_2 k_2^{(i)} \quad (16)$

Ta có:

$$y'(x) = f[x,y(x)]$$
$$y''(x) = f'_x[x, y(x)] + f'_y[x, y(x)]$$

.....

Do đó vế phải của (12) là:

$$hf(x_i, y_i) + \frac{h^2}{2}\left[f'_x(x_i, y_i) + f'_y(x_i, y_i)\right]y'(x) + \cdots \quad (17)$$

Mặt khác theo (15) và theo công thức Taylor ta có:

$$k_1^{(i)} = hf(x_i, y_i) = hy'_i$$
$$k_2^{(i)} = h[f(x_i, y_i) + ahf'_x(x_i, y_i) + \alpha k_1^{(i)} f'_y(x_i, y_i) + \cdots]$$

Do đó vế phải của (16) là:

$$h(r_1 + r_2)f(x_i, y_i) + h^2[ar_2 f'_x(x_i, y_i) + \alpha r_2 y'_i f'_y(x_i, y_i)] + \cdots \quad (18)$$

Bây giờ cho (17) và (18) khác nhau một vô cùng bé cấp $O(h^3)$ ta tìm được các hệ số chưa biết khi cân bằng các số hạng chứa h và chứa h^2:

$r_1 + r_2 = 1$

$a.r_1 = 1/ 2$

$\alpha.r_2 = 1$

Như vậy: $\alpha = a$, $r_1 = (2a - 1)/ 2a$, $r_2 = 1/ 2a$ với a được chọn bất kì.

Nếu a = 1 / 2 thì $r_1 = 0$ và $r_2 = 1$. Lúc này ta nhận được công thức Euler. Nếu a=1 thì $r_1 = 1 / 2$ và $r_2 = 1/2$. Lúc này ta nhận được công thức Euler cải tiến.

Một cách tương tự chúng ta nhận được công thức Runge - Kutta bậc 4. Công thức này hay được dùng trong tính toán thực tế :

$k_1 = h.f(x_i, y_i)$

$k_2 = h.f(x_i+h/ 2, y_i + k_1/ 2)$

$k_3 = h.f(x_i+h/ 2, y_i + k_2/ 2)$

$k_4 = h.f(x_i+h, y_i + k_3)$

$yi_{+1} = yi + (k_1 + 2k_2 + 2k_3 + k_4) / 6$

Ta xây dựng hàm ***rungekutta()*** để thực hiện công thức Runge - Kutta bậc 4:

```
function [x, y] = rungekutta(f, a, b, y0, n)
%Phuong phap Runge-Kutta de giai phuong trinh y'(x) = f(x,y(x)) hay y' =
%f(x)
if nargin < 4 | n <= 0
   n = 100;
end
if nargin < 3
```

```
    y0 = 0;
end
y(1,:) = y0(:)'; %
h = (b - a)/n;
x = a + [0:n]'*h;
if nargin(f) >1
    for k = 1:n
        f1 = h*feval(f, x(k), y(k, :));
        f1 = f1(:)';
        f2 = h*feval(f, x(k) + h/2, y(k, :) + f1/2);
        f2 = f2(:)';
        f3 = h*feval(f, x(k) + h/2, y(k, :) + f2/2);
        f3 = f3(:)';
        f4 = h*feval(f, x(k) + h, y(k, :) + f3);
        f4 = f4(:)';
        y(k+1, :) = y(k, :) + (f1 + 2*(f2 + f3) + f4)/6;
    end
else
    for k = 1:n
        f1 = h*feval(f, x(k));
        f1 = f1(:)';
        f2 = h*feval(f, x(k) + h/2);
        f2 = f2(:)';
        f3 = h*feval(f, x(k) + h/2);
        f3 = f3(:)';
        f4 = h*feval(f, x(k) + h);
        f4 = f4(:)';
        y(k+1, :) = y(k, :) + (f1 + 2*(f2 + f3) + f4)/6;
    end
end
```

Để giải phương trình ta dùng chương trình ***ctrungekutta.m***:

```
clear all, clc
a = 0;
b = 1;
y = inline('x + y');
ya = 0.5;
n = 10;%so lan tinh chi n = 10
[x, y] = rungekutta(y, a, b, ya, n)
plot(x, y);
```

§5. PHƯƠNG PHÁP RUNGE - KUTTA THÍCH NGHI

Vấn đề xác định bước tính h là rất quan trọng. Nếu muốn có độ chính xác cao thì bước tính h phải nhỏ. Tuy nhiên khi h nhỏ, ta lại tốn thời gian tính toán. Hơn nữa bước hằng số sẽ không thích hợp trên toàn bộ miền tìm nghiệm. Ví dụ nếu đường cong nghiệm ban đầu thay đổi nhanh rồi sau đó gần như không đổi thì ta phải dùng h nhỏ ở đoạn đầu và h lớn ở đoạn sau. đây là chỗ mà các phương pháp thích nghi chiếm ưu thế. Chúng đánh giá sai số làm tròn tại mỗi lần tích phân và tự động hiệu chỉnh độ lớn của h để sai số nằm trong giới hạn cho phép.

Phương pháp Runge - Kutta thích nghi còn gọi là phương pháp tích phân kết hợp. Các công thức này đi thành cặp: một công thức tích phân bậc m và một công thức tích phân bậc m+1. Ý tưởng là dùng hai công thức này cải thiện nghiệm trong đoạn [x, x+h]. Gọi kết quả là $y_m(x+h)$ và $y_{m+1}(x+h)$ ta có sai số đối với công thức bậc m là:

$$E(h) = y_{m+1}(x+h) - y_m(x+h) \qquad (1)$$

Chúng ta dùng công thức kết hợp bậc 4 và 5 mà đạo hàm được tính bằng công thức Fehlenberg. Do vậy công thức Runge - Kutta thích nghi còn được gọi là công thức Runge - Kutta - Fehlenberg:

$$K_1 = hF(x, y)$$

$$K_i = hF\left(x + A_i h, y + \sum_{j=0}^{i-1} B_{i,j} K_j\right) \qquad i = 1, 2,..,6 \qquad (2)$$

$$y_5(x+h) = y(x) + \sum_{i=1}^{6} C_i K_i \text{ (công thức bậc 5)} \qquad (3)$$

$$y_4(x+h) = y(x) + \sum_{i=1}^{6} D_i K_i \text{ (công thức bậc 4)} \qquad (4)$$

Các hệ số xuất hiện trong các công thức này không duy nhất. Bảng sau cho các hệ số tính theo Cash và Karp:

i	A_i	$B_{i,j}$					C_i	D_i
1	*	*	*	*	*	*	$\frac{37}{378}$	$\frac{2825}{27648}$
2	$\frac{1}{5}$	$\frac{1}{5}$	*	*	*	*	0	0
3	$\frac{3}{10}$	$\frac{3}{40}$	$\frac{9}{40}$	*	*	*	$\frac{250}{621}$	$\frac{18575}{48384}$
4	$\frac{3}{5}$	$\frac{3}{10}$	$-\frac{9}{10}$	$\frac{6}{5}$	*	*	$\frac{125}{594}$	$\frac{13525}{55296}$
5	1	$-\frac{11}{54}$	$\frac{5}{2}$	$-\frac{70}{27}$	$\frac{35}{27}$	*	0	$\frac{277}{14336}$

6	$\frac{7}{8}$	$\frac{1631}{55296}$	$\frac{175}{512}$	$\frac{575}{13824}$	$\frac{44275}{110592}$	$\frac{253}{4096}$	$\frac{512}{1771}$	$\frac{1}{4}$

Sai số sẽ là:

$$E(h) = y_5(x+h) - y_4(x+h) = \sum_{i=1}^{6}(C_i - D_i)K_i \qquad (5)$$

Chú ý là E(h) là một vec tơ, thành phần $E_i(h)$ biểu diễn sai số của biến y_i. Sai số e(h) ta cần kiểm soát là:

$$e(h) = \max_i |E(h)| \qquad (6)$$

Ta cũng có thể kiểm soát sai số trung bình bình phương:

$$e(h) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} E_i^2(h)} \qquad (7)$$

với n là số phương trình bậc 1.

Việc kiểm soát sai số đạt được bằng cách thay đổi h sao cho sai số tại mỗi bước tính ậiphỉ cỡ sai số mong muốn ε. Sai số khi thực hiên thuật táon Runge - Kutta bậc bốn là $O(h^5)$ nên:

$$\frac{e(h_1)}{e(h_2)} \approx \left(\frac{h_1}{h_2}\right)^5 \qquad (8)$$

Giả sử là ta đã tính nghiệm tại bước tính với h_1 và có sai số là $e(h_1)$. Tại bước tính với h_2 ta muốn có $e(h_2) = \varepsilon$ thì:

$$h_2 = h_1\left[\frac{\varepsilon}{e(h_1)}\right]^{1/5} \qquad (9)$$

Để dự phòng, ta lấy:

$$h_2 = 0.9h_1\left[\frac{\varepsilon}{e(h_1)}\right]^{1/5} \qquad (10)$$

Ta xây dựng hàm ***adaptrk()*** để thực hiện thuật toán này:

```
function [xsol, ysol] = adaptrk(f, xo, x1, y, n)
% Tich phan Runge-Kutta bac 5 dung giap phuong trinh y' = f(x, y) hay y' =
%f(x).
% xo, x1 - doan tim nghiem.
% y  gia tri dau, n dung tim h ban dau
h = (x1 - xo)/n;
if size(y, 1) > 1 ;
   y = y'; % y phai la vec to hang
end
eTol = 1.0e-9;
```

```
n = length(y);
A = [0 1/5 3/10 3/5 1 7/8];
B = [ 0 0 0 0 0
      1/5 0 0 0 0
      3/40 9/40 0 0 0
      3/10 -9/10 6/5 0 0
      -11/54 5/2 -70/27 35/27 0
      1631/55296 175/512 575/13824 44275/110592 253/4096];
C = [37/378 0 250/621 125/594 0 512/1771];
D = [2825/27648 0 18575/48384 13525/55296 277/14336 1/4];
%nghiem ban dau
xsol = zeros(2, 1);
ysol = zeros(2, n);
xsol(1) = xo;
ysol(1,:) = y;
stopper = 0;
k = 1;
for p = 2:5000
% Tinh K tu (2)
   K = zeros(6, n);
   if nargin(f) > 1
      K(1, :) = h*feval(f, xo, y);
   else
       K(1, :) = h*feval(f, xo);
   end
   for i = 2:6
      BK = zeros(1, n);
      for j = 1:i-1
         BK = BK + B(i, j)*K(j, :);
      end
      if nargin(f) > 1
         K(i, :) = h*feval(f, xo + A(i)*h, y + BK);
      else
         K(i, :) = h*feval(f, xo + A(i)*h);
      end
   end
% tinh su thay doi cua y theo (3) & (4)
   dy = zeros(1, n);
   E = zeros(1, n);
   for i = 1:6
      dy = dy + C(i)*K(i,:);
      E = E + (C(i) - D(i))*K(i,:);
```

```
    end
    e = sqrt(sum(E.*E)/n);
% neu sai so dat den gia tri cho phep, chap nhan ket qua
% kiem tr dieu kien ket thuc
    if e <= eTol
        y = y + dy;
        xo = xo + h;
        k = k + 1;
        xsol(k) = xo;
        ysol(k,:) = y;
        if stopper == 1;
            break
        end
    end
% tinh lai h theo (10)
    if e ~= 0;
        hnext = 0.9*h*(eTol/e)^0.2;
    else;
        hnext=h;
    end
    if (h > 0) == (xo + hnext >= x1 )
        hnext = x1 - xo;
        stopper = 1;
    end
    h = hnext;
end
```

Để tìm nghiệm của phương trình vi phân ta dùng chương trình ***ctadaptrk.m***:

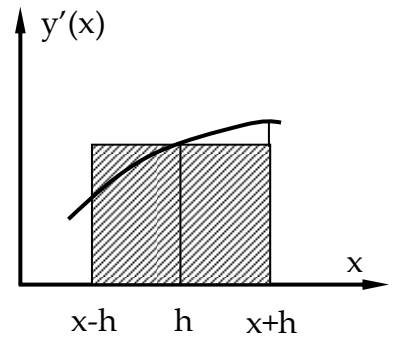
```
clear all, clc
a = 0;
b = 1;
y = inline('x + y');
ya = 0.5;
n = 10;%so lan tinh chi n = 10
%y = @f4;
[u, v] = adaptrk(y, a, b, ya, n)
plot(u, v)
```

§6. PHƯƠNG PHÁP BURLIRSCH - STÖR

1. Phương pháp điểm giữa: Công thức điểm giữa của tích phân số của $y' = f(x, y)$ là:

$$y(x+h) = y(x-h) + 2hf[x, y(x)] \qquad (1)$$

Đây là công thức bậc 2, giống như công thức Euler. Ta xem xét phương pháp này vì đây là cơ sở của phương pháp Burlisch - Stör dùng tìm nghiệm có độ chính xác cao. Hình bên minh hoạ công thức điểm giữa đối với phương trình đơn dạng $y' = f(x,y)$. Sự thay đổi y trên hai phía được xác định bằng:



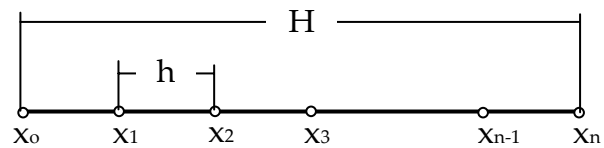
$$y(x+h) - y(x-h) = \int_{x-h}^{x+h} y'(x)dx$$

và bằng diện tích bên dưới đường cong. Xấp xỉ điểm giữa của diện tích này là diện tích của hình chữ nhật có gạch chéo.

Bây giờ ta xét ưu điểm của phương pháp điểm giữa khi tìm nghiệm của phương trình $y' = f(x,y)$ từ $x = x_0$ đến $x = x_0 + H$ với công thức điểm giữa.

Ta chia đoạn tích phân thành n đoạn nhỏ có độ dài mỗi đoạn là $h = H/n$ như hình bên và tính:

$$\begin{aligned} y_1 &= y_0 + hf_0 \\ y_2 &= y_0 + 2hf_1 \\ y_3 &= y_1 + 2hf_2 \qquad (2) \\ &\vdots \\ y_n &= y_{n-2} + 2hf_{n-1} \end{aligned}$$



Ta đã dùng khái niệm $y_i = y(x_i)$ và $f_i = f(x_i, y_i)$. Phương trình đầu tiên trong (2) dùng công thức Euler để thay cho phương pháp điểm giữa. Các phương trình khác là các công thức điểm giữa. Kết quả cuối cùng là trung bình cộng của y_n trong (2) và ta có:

$$y(x_o + H) = 0.5[y_n + (y_{n-1} + hf_n)] \qquad (3)$$

2. Ngoại suy Richardson: Ta có thể thấy sai số trong (3) là:

$$E = c_1h^2 + c_2h^4 + c_3h^6 + \cdots$$

Để giảm bớt sai số ta dùng phương pháp ngoại suy Richardson. Cụ thể ta tính $y(x_o+H)$ với một giá trị nào đó của h và rồi lặp lại quá trình tính với h/2. Gọi kết quả là g(h) và $g(h_1)$ ta có ngoại suy Richardson:

$$y(x_o + H) = \frac{4g(h_1) - g(h)}{3}$$

Ta xây dựng hàm ***midpoint()*** để kết hợp phương pháp điểm giữa và phương pháp ngoại suy Richardson. Đầu tiên phương pháp điểm giữa được dùng cho 2 tích phân. Số bước tính được tăng gấp đôi trong các lần lặp sau, mỗi lần lặp đều dùng ngoại suy Richardson. Chương trình dừng khi sai số nhỏ hơn sai số cho phép.

```
function y = midpoint(f, x, x1, y, tol)
% Phuong phap diem giua dung cho phuong trinh y' = f(x,y) hay y' = f(x).
if size(y, 1) > 1 ;
```

```
    y = y';
end % y phai la vec to hang
if nargin < 5
    tol = 1.0e-6;
end
kmax = 51;
n = length(y);
r = zeros(kmax, n);
% Bat dau bang 2 buoc tich phan
nsteps = 2;
r(1, 1:n) = mid(f, x, x1, y, nsteps);
rold = r(1, 1:n);
for k = 2:kmax
    % Tang gap doi so buoc va tinh chinh ket qua
    % ngoai suy Richardson
    nsteps = 2*k;
    r(k, 1:n) = mid(f, x, x1, y, nsteps);
    r = richardson(r, k, n);
   % kiem tra hoi tu.
    dr = r(1, 1:n) - rold;
    e = sqrt(dot(dr, dr)/n);
    if e < tol; y = r(1, 1:n);
        return;
    end
    rold = r(1, 1:n);
end
error('Phuong phap diem giua khong hoi tu')

function y = mid(f, x, xf, y, nsteps)
% Cong thuc diem giua
h = (xf - x)/nsteps;
y0 = y;
if nargin(f) > 1
    y1 = y0 + h*feval(f, x, y0);
else
    y1 = y0 + h*feval(f, x);
end
for i = 1:nsteps-1
    x = x + h;
    if nargin(f) > 1
        y2 = y0 + 2.0*h*feval(f, x, y1);
    else
```

```
        y2 = y0 + 2.0*h*feval(f, x);
    end
    y0 = y1;
    y1 = y2;
end
if nargin(f) > 1
    y = 0.5*(y1 + y0 + h*feval(f, x, y2));
else
     y = 0.5*(y1 + y0 + h*feval(f, x));
end

function r = richardson(r, k, n)
% Richardson extrapolation.
for j = k-1:-1:1
    c =(k/(k-1))^(2*(k-j));
    r(j, 1:n) =(c*r(j+1, 1:n) - r(j, 1:n))/(c - 1.0);
end
```

3. Thuật toán Burlisch - Stör: Phương pháp điểm giữa có nhược điểm là nghiệm nằm tại điểm giữa của khoảng tìm nghiệm không được tinh chỉnh bằng phương pháp ngoại suy Richardson. Khuyết điểm này được khác phục trong phương pháp Burlisch - Stör. Ý tưởng của phương pháp này là áp dụng phương pháp điểm giữa trên từng đoạn. Ta xây dựng hàm ***burlischstoer()*** để thực hiện thuật toán này:

```
function [xout, yout] = burlischstoer(f, x, x1, y, H, tol)
% Phuong phap Bulirsch-Stoer giai phuong trinh y' = F(x, y) hay y’ = f(x).
%[x, x1] la khoang tim nghiem.
% H = do tang sau moi lan tinh
if size(y, 1) > 1
    y = y';
end % y phai la vec to hang
if nargin < 6
    tol = 1.0e-8;
end
n = length(y);
xout = zeros(2, 1);
yout = zeros(2, n);
xout(1) = x;
yout(1, :) = y;
k = 1;
while x < x1
    k = k + 1;
```

```
    H = min(H, x1 - x);
    y = midpoint(f, x, x + H, y, tol);
    x = x + H;
    xout(k) = x;
    yout(k, :) = y;
end
```

Để giải phương trình ta dùng chương trình ***ctburlischstoer.m***:

```
clear all, clc
a = 0;
b = 1;
y = @f3;
ya = 1;
H = .1;
[u, v] = burlischstoer(y, a, b, ya, H)
plot(u, v)
```

§7. PHƯƠNG PHÁP CHUỖI TAYLOR

Phương pháp chuỗi Taylor đơn giản về ý tưởng và có độ chính xác cao. Cơ sở của phương pháp này là cắt chuỗi Taylor của y theo x:

$$y(x+h) \approx y(x) + y'(x)h + \frac{1}{2!}y''(x)h^2 + \frac{1}{3!}y''(x)h^3 + \cdots + \frac{1}{m!}y^{(m)}(x)h^m \quad (1)$$

Do phương trình (1) dự đoán trước y tại (x + h) từ các thông tin có tại x, nó cũng là công thức tích phân. Số hạng cuối trong (1) là bậc của tích phân. Như vậy (1) là tích phân bậc m. Sai số là:

$$E = \frac{1}{(m+1)!}y^{(m+1)}(\xi)h^{m+1} \qquad x < \xi < x+h$$

Dùng xấp xỉ đạo hàm:

$$y^{(m+1)}(\xi) \approx \frac{y^{(m)}(x+h) - y^{(m)}(x)}{h}$$

ta có:

$$E = \frac{h^m}{(m+1)!}\left[y^{(m)}(x+h) - y^{(m)}(x)\right] \quad (2)$$

Ta xây dựng hàm ***taylor()*** để giải bài toán trên:

```
function [xout, yout] = taylor(deriv, x, y, x1, h)
% Tich phan chuoi Taylor bac 4.
% x, y = cac gia tri dau; i la vec to hang.
% x1 = gia tri cuoi cua x

if size(y,1) > 1;
```

```
    y = y';
end
xout = zeros(2, 1);
yout = zeros(2, length(y));
xout(1) = x;
yout(1, :) = y;
k = 1;
while x < x1
    h = min(h,x1 - x);
    d = feval(deriv,x,y); % Dao ham cua [y]
    hh = 1;
    for j = 1:4 % tao chuoi Taylor
        hh = hh*h/j;
        y = y + d(j, :)*hh;
    end
    x = x + h;
    k = k + 1;
    xout(k) = x;
    yout(k,:) = y;
end
```

Ta dùng chương trình ***cttaylor.m*** để giải phương trình:

```
clear all, clc
    y = @f5;
    a = 0;
    b = 2;
    ya = [0 1];
    h = 0.2;
    [x, y] = taylor(y, a, ya, b, h)
    plot(x, y);
```

§8. PHƯƠNG PHÁP DỰ ĐOÁN - HIỆU CHỈNH

1. Phương pháp Adam - Bashfort - Moulton: Năm 1855, nhà toán học người Anh Adams đề xuất một phương pháp đa bước giải bài toán Cauchy theo yêu cầu của ông Bashforth, một chuyên gia kỹ thuật pháo binh Anh. Kết quả của Adams sau này bị quên lãng. Mãi đến đầu thế kỷ 20, nhà toán học Nauy khi tính quỹ đạo của hạt điện tích rời xa mặt trời với vận tốc lớn đã phát minh lại công thức Adams. Sau này viện sỹ Krylov đã hoàn thiện phương pháp Adams. Phương pháp Adams - Bashfort - Moulton (ABM) gồm hai bước. Bước dầu tiên là xấp xỉ f(x, y) bằng một đa thức(ví dụ đa thức Lagrange) bậc 4 qua 4 điểm:

$$\left\{\left(t_{k-3},f_{k-3}\right),\left(t_{k-2},f_{k-2}\right),\left(t_{k-1},f_{k-1}\right),\left(t_k,f_k\right)\right\}$$

và thay thế đa thức này vào phương trình vi phân để có được giá trị dự báo y_{k+1}:

$$p_{k+1} = y_k + \int_0^h l_3(t)dt = y_k + \frac{h}{24}\left(-9f_{k-3} + 37f_{k-2} - 59f_{k-1} + 55f_k\right) \tag{1a}$$

Bước thứ hai là lặp lại công việc với 4 điểm được cập nhật:

$$\left\{(t_{k-2}, f_{k-2}), (t_{k-1}, f_{k-1}), (t_k, f_k), (t_{k+1}, f_{k+1})\right\}$$

và nhận giá trị hiệu chỉnh:

$$f_{k+1} = f(t_{k+1}, p_{k+1})$$

$$c_k = y_k + \int_0^h l_3'(t)dt = y_k + \frac{h}{24}\left(f_{k-2} - 5f_{k-1} + 19f_k + 9f_{k+1}\right) \tag{1b}$$

Ta viết khai triển Taylor của y_{k+1} lân cận t_k và của y_k lân cận t_{k+1}:

$$y_{k+1} = y_k + hf_k + \frac{h^2}{2}f_k' + \frac{h^3}{3!}f_k'' + \cdots \tag{2a}$$

$$y_k = y_{k+1} - hf_{k+1} + \frac{h^2}{2}f_{k+1}' - \frac{h^3}{3!}f_{k+1}'' + \cdots$$

$$y_{k+1} = y_k + hf_{k+1} - \frac{h^2}{2}f_{k+1}' + \frac{h^3}{3!}f_{k+1}'' + \cdots \tag{2b}$$

và thay thế các đạo hàm bậc 1, 2, 3 bằng các xấp xỉ

$$\begin{aligned}
y_{k+1} &= y_k + hf_k + \frac{h^2}{2}\left(\frac{-\frac{1}{3}f_{k-3} + \frac{3}{2}f_{k-2} - 3f_{k-1} + \frac{11}{6}f_k}{h} + \frac{1}{4}h^3 f_k^{(4)} + \cdots\right) \\
&\quad + \frac{h^3}{3!}\left(\frac{-f_{k-3} + 4f_{k-2} - 5f_{k-1} + 2f_k}{h^2} + \frac{11}{12}h^2 f_k^{(4)} + \cdots\right) \\
&\quad + \frac{h^4}{4!}\left(\frac{-f_{k-3} + 3f_{k-2} - 3f_{k-1} + f_k}{h^3} + \frac{3}{2}hf_k^{(4)} + \cdots\right) + \frac{h^5}{120}f_k^{(4)} + \cdots \\
&= y_k + \frac{h}{24}\left(-9f_{k-3} + 37f_{k-2} - 59f_{k-1} + 55f_k\right) + \frac{251}{720}h^5 f_k^{(4)} + \cdots \\
&\approx p_{k+1} + \frac{251}{720}h^5 f_k^{(4)}
\end{aligned} \tag{3a}$$

$$\begin{aligned}
y_{k+1} &= y_k + hf_{k+1} - \frac{h^2}{2}\left(\frac{-\frac{1}{3}f_{k-2} + \frac{3}{2}f_{k-1} - 3f_k + \frac{11}{6}f_{k+1}}{h} + \frac{1}{4}h^3 f_{k+1}^{(4)} + \cdots\right) \\
&\quad + \frac{h^3}{3!}\left(\frac{-f_{k-2} + 4f_{k-1} - 5f_k + 2f_{k+1}}{h^2} + \frac{11}{12}h^2 f_{k+1}^{(4)} + \cdots\right) \\
&\quad - \frac{h^4}{4!}\left(\frac{-f_{k-2} + 3f_{k-1} - 3f_k + f_{k+1}}{h^3} + \frac{3}{2}hf_{k+1}^{(4)} + \cdots\right) + \frac{h^5}{120}f_{k+1}^{(4)} + \cdots
\end{aligned}$$

$$= y_k + \frac{h}{24}\left(f_{k-2} - 5f_{k-1} + 19f_k + 9f_{k+1}\right) - \frac{19}{720}h^5 f_{k+1}^{(4)} + \cdots$$

$$\approx c_{k+1} - \frac{19}{720}h^5 f_{k+1}^{(4)} \qquad (3b)$$

Từ các phương trình này và giả sử rằng $f_{k+1}^{(4)} \cong f_k^{(4)} \cong K$ ta có thể viết các sai số dự đoán/hiệu chỉnh:

$$E_{P,k+1} = y_{k+1} - p_{k+1} \approx \frac{251}{720}h^5 f_k^{(4)} \cong \frac{251}{720}Kh^5 \qquad (4a)$$

$$E_{C,k+1} = y_{k+1} - c_{k+1} \approx -\frac{19}{720}h^5 f_{k+1}^{(4)} \cong -\frac{19}{720}Kh^5 \qquad (4b)$$

Do K chưa biết nên ta phải tìm nó. Ta có;

$$E_{P,k+1} - E_{C,k+1} = c_{k+1} - p_{k+1} \cong \frac{270}{720}Kh^5 \equiv \frac{270}{251}E_{P,k+1} \equiv -\frac{270}{19}E_{C,k+1} \qquad (5)$$

Do vậy ta có các công thức dùng để đánh giá sai số:

$$E_{P,k+1} = y_{k+1} - p_{k+1} \cong \frac{251}{720}\left(c_{k+1} - p_{k+1}\right) \qquad (6a)$$

$$E_{C,k+1} = y_{k+1} - c_{k+1} \cong -\frac{19}{270}\left(c_{k+1} - p_{k+1}\right) \qquad (6b)$$

Tóm lại, thuật toán Adams - Bashforth - Moulton gồm:

Dự đoán: $$p_{k+1} = y_k + \frac{h}{24}\left(-9f_{k-3} + 37f_{k-2} - 59f_{k-1} + 55f_k\right) \qquad (7a)$$

Biến đổi: $$m_{k+1} = p_{k+1} + \frac{251}{720}\left(c_k - p_k\right) \qquad (7b)$$

Hiệu chỉnh: $$c_k = y_k + \frac{h}{24}\left[f_{k-2} - 5f_{k-1} + 19f_k + 9f\left(t_{k+1}, m_{k+1}\right)\right] \qquad (7c)$$

$$y_{k+1} = c_{k+1} - \frac{19}{270}\left(c_{k+1} - p_{k+1}\right) \qquad (7d)$$

Ta xây dựng hàm ***odeabm()*** để thực hiện thuật toán này:

```
function [t, y] = odeabm(f, to, tf, y0, n)
%Phuong phap Adams-Bashforth-Moulton
%de giai pt y'(t) = f(t,y(t)) hay y'(t)= f(t)
if (nargin < 5) | (n < 0)
   n = 10;
end
h = (tf - to)/n;
t1 = to + 3*h;
[t, y] = rungekutta(f, to, t1, y0, 3); %khoi gan bang pp Runge-Kutta
t = [t(1:3)'  t(4):h:tf]';
for k = 1:4
```

```
    if nargin(f) > 1
        F(k,:) = feval(f, t(k), y(k, :));
    else
        F(k,:) = feval(f, t(k));
    end
end
p = y(4, :);
c = y(4, :);
KC22 = 251/270;
KC12 = 19/270;
h24 = h/24;
h241 = h24*[-9 37 -59 55];
h242 = h24*[1 -5 19 9];
for k = 4:n
    p1 = y(k, :) + h241*F; %Pt.(7a)
    m1 = p1 + KC22*(c - p); %Pt.(7b)
    if nargin(f) > 1
        c1 = y(k, :) + h242*[F(2:4, :); feval(f, t(k+1), m1)']; %Pt.(7c)
    else
        c1 = y(k, :) + h242*[F(2:4, :); feval(f, t(k+1))']; %Pt.(7c)
    end
    y(k + 1, :) = c1 - KC12*(c1 - p1); %Pt.(7d)
    p = p1;
    c = c1; %cap nhat cac gia tri du doan/hieu chinh
    if nargin(f) > 1
        F = [F(2:4, :);  feval(f, t(k + 1), y(k + 1,:))'];
    else
        F = [F(2:4, :);  feval(f, t(k + 1))'];
    end
end
```

Để giải phương trình ta dùng chương trình ***ctodeabm.m***:

```
clear all, clc
a = 0;
b = 1;
y = @f1;
ya = [0 1 1]';
n = 10;
[t, y] = odeabm(y, a, b, ya, n)
plot(t, y)
```

2. Phương pháp Hamming: Thuật toán Hamming cũng như thuật toán Adams - Bashforth - Moulton nhưng các công thức dự báo/hiệu chỉnh là:

Dự đoán: $$p_{k+1} = y_{k-3} + \frac{4h}{3}\left(2f_{k-2} - f_{k-1} + 2f_k\right) \quad (8a)$$

Biến đổi: $$m_{k+1} = p_{k+1} + \frac{112}{121}\left(c_k - p_k\right) \quad (8b)$$

Hiệu chỉnh: $$c_k = 0.125\left\{9y_k - y_{k-2} + 3h\left[-f_{k-1} + 2f_k + f\left(t_{k+1}, m_{k+1}\right)\right]\right\} \quad (8c)$$

$$y_{k+1} = c_{k+1} - \frac{9}{121}\left(c_{k+1} - p_{k+1}\right) \quad (dd)$$

Ta xây dựng hàm ***hamming()*** để thực hiện thuật toán này:

```
function [t, y] = hamming(f, to, tf, y0, n)
% Phuong phap Hamming de giai phuong trinh y'(t) = f(t,y(t)) hay y'=f(t)
if (nargin < 5) | (n <= 0)
   n = 100;
end
h = (tf - to)/n;
ts = to + 3*h;
[t, y] = rungekutta(f, to, ts,y0, 3); %Khoi gan bang Runge-Kutta
t = [t(1:3)'  t(4):h:tf];
for k = 2:4
   if nargin(f) > 1
      F(k - 1,:) = feval(f, t(k), y(k,:));
   else
      F(k - 1,:) = feval(f, t(k));
   end
end
p = y(4, :);
c = y(4, :);
h34 = h/3*4;
KC1 = 112/121;
KC2 = 9/121;
h312 = 3*h*[-1 2 1];
for k = 4:n
   p1 = y(k - 3, :) + h34*(2*(F(1, :) + F(3, :)) - F(2, :)); %Pt.(8a)
   m1 = p1 + KC1*(c - p); %Pt.(8b)
   if nargin(f) > 1
      c1 = (-y(k - 2, :) + 9*y(k, :) + h312*[F(2:3, :);
     feval(f, t(k + 1), m1)'])/8; %Pt.(8c)
   else
      c1 = (-y(k - 2, :) + 9*y(k, :) + h312*[F(2:3, :);
      feval(f, t(k + 1))'])/8; Pt.(8c)
   end
   y(k+1,:) = c1 - KC2*(c1 - p1); %Pt.(8d)
   p = p1; c = c1; %cap nhat cac gia tri du bao/hieu chinh
   if nargin(f) > 1
      F = [F(2:3, :); feval(f, t(k + 1),y(k + 1,:))'];
   else
      F = [F(2:3, :); feval(f, t(k + 1))'];
   end
end
```

Để giải phương trình ta dùng chương trình ***cthamming.m***:

```
clear all, clc
a = 0;
b = 1;
y = @f1;
ya = [0 1 1]';
n = 10;
tic
[t, y] = hamming(y, a, b, ya, n);
toc
plot(t, y)
```

§9. PHƯƠNG PHÁP MILNE

Quá trình tính toán nghiệm được thực hiện qua ba bước:

- Tính gần đúng y_{m+1} theo công thức (dự đoán):

$$y_{m+1}^{(1)} = y_{m-3} + \frac{4h}{3}\left(2y'_{m-2} - y'_{m-1} + 2y'_m\right) \quad (1)$$

- Dùng $y_{m+1}^{(1)}$ để tính:

$$y'_{m+1} = f(x_{m+1}, y_{m+1}^{(1)}) \quad (2)$$

- Dùng y'_{m+1} vừa tính được để tính gầm đúng thứ 2 của y_{m+1}(hiệu chỉnh)

$$y_{m+1}^{(2)} = y_{m-1} + \frac{h}{3}\left(y'_{m-1} + 4y'_m + y'_{m+1}\right) \quad (3)$$

Ta xây dựng hàm ***milne()*** để thực hiện thuật toán trên:

```
function [t, y] = milne(f, to, tf, y0, n)
h = (tf - to)/n;
y(1, :) = y0';
ts = to + 3*h;
[t, y] = rungekutta(f, to, ts, y0, 3);
t = [t(1:3)'  t(4):h:tf]';
for i = 2:4
   if nargin(f) > 1
      F(i - 1, :) = feval(f, t(i), y(i, :));
   else
      F(i - 1, :) = feval(f, t(i));
   end
end
for i = 4:n
   p = y(i - 3, :) + (4*h/3)*(2*F(1, :) - F(2, :) + 2*F(3, :)); %Pt.(1)
   if nargin(f) > 1
      F(4, :) = f(t(i+1), p);%Pt.(2)
```

```
    else
        F(4, :) = f(t(i+1));
    end
    y(i+1, :) = y(i-1, :) + (h/3)*(F(2, :) + 4*F(3, :) + F(4, :));%Pt.(3)
    F(1, :) = F(2, :);
    F(2, :) = F(3, :);
    if nargin(f) > 1
        F(3, :) = f(t(i+1), y(i+1, :));
    else
        F(3, :) = f(t(i+1));
    end
end
```

Để giải phương trình ta dùng chương trình ***ctmilne.m***:

```
clear all, clc
a = 0;
b = 1;
y  = @f2;
ya = 1;
n = 10;
[t, y] = milne(y, a, b, ya, n);
plot(t, y)
```

§10. BÀI TOÁN GIÁ TRỊ BIÊN

1. Khái niệm chung: Ta xét bài toán tìm nghiệm của phương trình:

$$y'' = f(x, y, y') \quad y(a) = \alpha, y(b) = \beta$$

Đây là bài toán tìm nghiệm của phương trình vi phân khi biết điều kiện biên và được gọi là bài toán giá trị biên hai điểm. Trong bài toán giá trị đầu ta có thể bắt đầu tìm nghiệm ở điểm có các giá trị đầu đã cho và tiếp tục cho các thời điểm sau. Kỹ thuật này không áp dụng được cho bài toán giá trị biên vì không có đủ điều kiện đầu ở các biên để có nghiệm duy nhất. Một cách để khác phục khó khăn này là cho các giá trị còn thiếu. Nghiệm tìm được dĩ nhiên sẽ không thoả mãn điều kiện ở các biên. Tuy nhiên ta có thể căn cứ vào đó để thay đổi điều kiện đầu trước khi tích phân lại. Phương pháp này cũng giống như bắn bia. Trước hết ta bắn rồi xem có trúng đích hay không, hiệu chỉnh và bắn lại. Do vậy phương pháp này gọi là phương pháp bắn.

Một phương pháp khác để giải bài toán giá trị biên là phương pháp sai phân hữu hạn trong các đạo hàm được thay bằng các xấp xỉ bằng sai phân hữu hạn tại các nút lưới cách đều. Như vậy ta sẽ nhận được hệ phương trình đại số đối với các sai phân.

Cả hai phương pháp này có một vấn đề chung: chúng làm tăng số phương trình phi tuyến nếu phương trình vi phân là phi tuyến. Các phương trình này được giải bằng

phương pháp lặp nên rất tốn thời gian tính toán. Vì vạy việc giải bài toán biên phi tuyến rất khó. Ngoài ra, đối với phương pháp lặp, việc chọn giá trị đầu rất quan trọng. Nó quyết định tính hội tụ của phương pháp lặp.

2. Phương pháp shooting: Ta xét bài toán biên là phương trình vi phân cấp 2 với điều kiện đầu tại x = a và x = b. Ta xét phương trình:

$$y'' = f(x, y, y') \quad y(a) = \alpha, y(b) = \beta \qquad (1)$$

Ta tìm cách đưa bài toán về dạng bài toán giá trị đầu:

$$y'' = f(x, y, y') \quad y(a) = \alpha, y'(a) = u \qquad (2)$$

Chìa khoá thành công là tìm ra giá trị đúng u. ta có thể thực hiện việc này bằng phương pháp "thử và sai": cho một giá trị u và giải bài toán giá trị đầu bằng cách đi từ a đến b. Nếu nghiệm giống với điều kiện biên mô tả y(b) = β thì ta đã có nghiệm của bài toán. Nếu không ta phải hiệu chỉnh u và làm lại. Tuy nhiên làm như vậy chưa hay. Do nghiệm của bài toán giá trị đầu phụ thuộc u nên giá trị biên tính được y(b) là hàm của u, nghĩa là:

$$y(b) = \theta(u)$$

Do đó u là nghiệm của phương trình:

$$r(u) = \theta(u) - \beta = 0 \qquad (3)$$

Trong đó θ(u) gọi là số dự biên(hiệu số giữa giá trị tính được và giá trị biên cho trước). Phương trình (3) có thể gải bằng các phương pháp tìm nghiệm trong chương trước. Tuy nhiên phương pháp chia đôi cung đòi hỏi tính toán lâu còn phương pháp Newton - Raphson đòi hỏi tìm đạo hàm dθ/dt. Do vậy cúng ta sẽ dùng phương pháp Brent. Tóm lại thuật oán giải bài toán giá trị biên gồm các bước:

- Mô tả giá trị u_1 và u_2 vây nghiệm u của (3)
- Dùng phương pháp Brent tìm nghiệm u của (3). Chú ý là mỗi bước lặp đòi hỏi tính θ(u) bằng cách giải phương trình vi phân như là bài toán điều kiện đầu.
- Khi đã có u, giải phương trình vi phân lần nữa để tìm nghiệm

Ta xây dựng hàm ***bvp2shoot()*** để giải phương trình bậc 2:

```
function [t, x] = bvp2shoot(f, t0, tf, x0, xf, n, tol, kmax)
%Giai phuong trinh: [x1, x2]' = f(t, x1, x2) voi x1(t0) = x0, x1(tf) = xf
if nargin < 8
   kmax = 10;
end
if nargin < 7
   tol = 1e-8;
end
if nargin < 6
   n = 100;
end
dx0(1) = (xf - x0)/(tf - t0); % cho gia tri dau cua x'(t0)
y0 = [x0 dx0(1)]';
```

```
[t, x] = rungekutta(f, t0, tf , y0, n); % khoi gan bg RK4
e(1) = x(end,1) - xf;
dx0(2) = dx0(1) - 0.1*sign(e(1));
for k = 2: kmax-1
   y1 = [x0 dx0(k)]';
   [t, x] = rungekutta(f, t0, tf, y1, n);
   %sai so giua gia tri cuoi va dich
   e(k) = x(end, 1) - xf; % x(tf)- xf
   ddx = dx0(k) - dx0(k - 1);
   if abs(e(k))< tol | abs(ddx)< tol
      break;
   end
   deddx = (e(k) - e(k - 1))/ddx;
   dx0(k + 1) = dx0(k) - e(k)/deddx;
end
```

Để giải phương trình:

$y'' = 2y^2 + 4xyy'$ với điều kiện biên: y(0) = 0.25, y(1) = 1/3

Đặt: $y' = y_1$, $y'' = y_2$ ta đưa phương trình về hệ phương trình vi phân cấp 1:

$$\begin{cases} y_1 = y_2 \\ y_2 = 2y_1 + 4xy_2y_1 \end{cases}$$

và biểu diễn nó bằng hàm ***f7()***:

```
function dx = f7(t, x) %Eq.(6.6.5)
dx(1) = x(2);
dx(2) = (2*x(1) + 4*t*x(2))*x(1);
```

Để giải phương trình ta dùng chương trình ***ctbvp2shoot.m***:

```
clear all, clc
t0 = 0;
tf = 1;
x0 = 1/4;
xf = 1/3; % thoi gian dau/cuoi va cac vi tri
n = 100;
tol = 1e-8;
kmax = 10;
y = @f7;
[t, x] = bvp2shoot(y, t0, tf, x0, xf, n, tol, kmax);
xo = 1./(4 - t.*t);
err = norm(x(:,1) - xo)/(n + 1)
```

```
plot(t,x(:, 1))
```

3. Phương pháp sai phân hữu hạn: Ta xét phương trình:

$$y'' = f(x, y, y') \quad y(a) = \alpha, y(b) = \beta$$

Ý tưởng của phương pháp này là chia đoạn [a, b] thành n đoạn nhỏ có bước h và xấp xỉ các đạo hàm bằng các sai phân:

$$y' = \frac{y_{i+1} - y_i}{2h}$$

$$y'' = \frac{y_{i-1} - 2y_i + y_{i+1}}{h^2}$$

Như vậy:

$$y'' = \frac{y_{i-1} - 2y_i + y_{i+1}}{h^2} = f\left(x_i, y_i, \frac{y_{i+1} - y_i}{2h}\right) \quad i = 1, 2, 3,...$$

Phương trình bnày sec đưa đến hệ phương trình đối với x_i, y_i. Ta xây dựng hàm ***bvp2fdf()***:

```
function [t, x] = bvp2fdf(a1, a0, u, t0, tf, x0, xf, n)
% Giai pt : x'' + a1*x' + a0*x = u with x(t0) = x0, x(tf) = xf
% bang pp sai phan huu han
h = (tf - t0)/n;
h2 = 2*h*h;
t = t0 + [0:n]'*h;
if ~isnumeric(a1)
   a1 = a1(t(2: n));
else
   length(a1) == 1
   a1 = a1*ones(n - 1, 1);
end
if ~isnumeric(a0)
   a0 = a0(t(2:n));
else length(a0) == 1
   a0 = a0*ones(n - 1, 1);
end
if ~isnumeric(u)
   u = u(t(2:n));
elseif length(u) == 1
   u = u*ones(n-1,1);
else
   u = u(:);
end
A = zeros(n - 1, n - 1);
```

```
b = h2*u;
ha = h*a1(1);
A(1, 1:2) = [-4 + h2*a0(1) 2 + ha];
b(1) = b(1) + (ha - 2)*x0;
for m = 2:n - 2
   ha = h*a1(m);
   A(m, m - 1:m + 1) = [2-ha   -4+h2*a0(m)   2+ha];
end
ha = h*a1(n - 1);
A(n - 1, n - 2:n - 1) = [2-ha   -4+h2*a0(n - 1)];
b(n - 1) = b(n-1) - (ha+2)*xf;
x = [x0 trid(A,b)' xf]';

function x = trid(A, b)
% giai he pt trdiagonal
n = size(A, 2);
for m = 2:n
   tmp = A(m, m - 1)/A(m - 1, m - 1);
   A(m, m) = A(m, m) -A(m - 1, m)*tmp;
   A(m, m - 1) = 0;
   b(m, :) = b(m, :) -b(m - 1, :)*tmp;
end
x(n, :) = b(n, :)/A(n, n);
for m = n - 1: -1: 1
   x(m, :) = (b(m, :) -A(m, m + 1)*x(m + 1))/A(m, m);
end
```

Để giải phương trình:

$$y'' + \frac{2}{x}y' - \frac{2}{x^2} = 0$$

với y(1) = 5 và y(2) = 3 ta dùng chương trình ***ctbvp2fdf.m***:

```
clear, clc
x0 = 1;%toa do bien dau
y0 = 5; %gia tri bien dau
xf = 2; %toa do bien cuoi
yf = 3; %gia tri bien cuoi
n = 100;%so buoc tinh
a1 = inline('2./x', 'x');
a0 = inline('-2./x./x','x');
u = 0;%ve phai cua phupng trinh
[x, y] = bvp2fdf(a1, a0, u, x0, xf, y0, yf, n);
```

```
plot(x, y)
```

§11. PHƯƠNG PHÁP LẶP PICARD

Việc giải bài toán Cauchy:

$$y' = f(t, y),\ y(t_o) = y_o \qquad (1)$$

hoàn toàn tương đương với việc tìm nghiệm y(t) của phương trình tích phân:

$$y(t) = y_o + \int_{t_o}^{t_1} f[z, y(z)]dz \qquad (2)$$

Nội dung của phương trình Picard là thay cho việc tìm nghiệm đúng của phương trình (2) ta tìm nghiệm gần đúng theo công thức:

$$y_{n+1} = y_o + \int_{t_o}^{t_1} f[z, y(z)]dz \qquad (3)$$

Trong đó để đơn giản ta chọn nghiệm gần đúng đầu tiên là:

$y_o(x) = y_o$

Ta xây dựng hàm ***picard()*** để thực hiện thuật toán trên:

```
function g = picard(f, y0, maxiter)
syms x y
g = subs(f, y0);
g = int(g, x);
for i = 1:maxiter
   g = subs(f, g);
   g = int(g, x);
end
g = sort(g);
```

Để giải phương trình ta dùng chương trình ***ctpicard.m***:

```
clear all, clc
syms x y
%f = 1 + y^2;%Phuong trinh y' = 1+y^2
%y0 = 0;%dieu kien dau
f = y - x;
y0 = 2;
g = picard(f, y0, 4)
```

§12. PHƯƠNG PHÁP GILL

Phương pháp Gill cũng tương tự như phương pháp Runge - Kutta, giá trị nghiệm tại y_{n+1} được tính bằng:

$$y_{n+1} = y_n + \frac{1}{6}\left[k_1 + (2-\sqrt{2})k_2 + (2+\sqrt{2})k_4 + k_4\right]$$

Với:

$$k_1 = hf(x_n, y_n)$$

$$k_2 = hf\left[x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1\right]$$

$$k_3 = hf\left[x_n + \frac{1}{2}h, y_n + \frac{1}{2}\left(-1+\sqrt{2}\right)k_1 + \left(1 - \frac{\sqrt{2}}{2}\right)k_2\right]$$

$$k_4 = hf\left[x_n + h, y_n - \frac{\sqrt{2}}{2}k_2 + \left(1 + \frac{\sqrt{2}}{2}\right)k_3\right]$$

Ta xây dựng hàm gill() để thực hiên thuật toán trên:

```
function [x, y] = gill(f, a, b, y0, n)
%Phuong phap Gill de giai phuong trinh y'(x) = f(x,y(x))
if nargin < 4 | n <= 0
   n = 100;
end
if nargin < 3
   y0 = 0;
end
y(1,:) = y0(:)';
h = (b - a)/n;
x = a + [0:n]'*h;
if nargin(f) >1
   for k = 1:n
      k1 = h*feval(f, x(k), y(k, :));
      k1 = k1(:)';
      k2 = h*feval(f, x(k)+h/2, y(k, :)+k1/2);
      k2 = k2(:)';
      k3 = h*feval(f, x(k)+h/2, y(k, :)+(sqrt(2)-1)*k1/2+(1-sqrt(2)/2)*k2);
      k3 = k3(:)';
      k4 = h*feval(f, x(k)+h, y(k, :) -sqrt(2)/2*k2+(1+sqrt(2)/2)*k3);
      k4 = k4(:)';
      y(k+1, :) = y(k, :) + (k1 + (2 - sqrt(2))*k2 + (2+sqrt(2))*k3 + k4)/6;
   end
else
   for k = 1:n
      f1 = h*feval(f, x(k));
      f1 = f1(:)';
      f2 = h*feval(f, x(k) + h/2);
      f2 = f2(:)';
      f3 = h*feval(f, x(k) + h/2);
```

```
        f3 = f3(:)';
        f4 = h*feval(f, x(k) + h);
        f4 = f4(:)';
        y(k+1, :) = y(k, :) + (f1 + (2 - sqrt(2))*f2 + (2+sqrt(2))*f3 + f4)/6;
    end
end
```

Để giải phương trình ta dùng chương trình ***ctgill.m***:

```
clear all, clc
a = 0;
b = 1;
y = inline('x + y');
ya = 0.5;
n = 10;%so lan tinh chi n = 10
[t, u] = gill(y, a, b, ya, n);
plot(t, u);
[l, v] = rungekutta(y, a, b, ya, n);
hold on
plot(l, v, '.r')
```

§13. PHƯƠNG PHÁP RUNGE – KUTTA – FEHLBERG

Một phương pháp để bảo đảm độ chính xác của nghiệm của phương trình vi phân là giả bài toán hai lần với bươc tính là h và 0.5h rồi so sánh kết quả tại các nút ứng với h. Tuy nhiên điều này đòi hỏi tăng số lần tính và phải tính lại khi giá trị tại các nút khác nhau nhiều. Dùng phương pháp Runge – Kutta – Fehlberg có thể tránh được khó khăn này. Nó có thủ tục để xác định liệu kích thước bước tính h đã thích hợp chưa. Tại mỗi bươc tính, hai xấp xỉ khác nhau của nghiệm được tính và so sánh với nhau. Nếu chúng sai khác trong phạm vi sai số cho trước thì nghiệm được chấp nhận. Nếu sai khác còn lơn hơn sai số cho phép thì bước h được giảm và ta lại tìm nghiệm với h mới. Mỗi bước tính theo phương pháp Runge – Kutta – Fehlberg đòi hỏi 6 giá trị:

$$k_1 = hf(t_i, y_i)$$

$$k_2 = hf\left(t_i + \frac{1}{4}h, y_i + \frac{1}{4}k_1\right)$$

$$k_3 = hf\left(t_i + \frac{3}{8}h, y_i + \frac{3}{32}k_1 + \frac{9}{32}k_2\right)$$

$$k_4 = hf\left(t_i + \frac{12}{13}h, y_i + \frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3\right)$$

$$k_5 = hf\left(t_i + h, y_i + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4\right)$$

$$k_5 = hf\left(t_i + \frac{1}{2}h, y_i - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5\right)$$

Xấp xỉ nghiệm theo phương pháp Runge – Kutta bậc 4:

$$y_{i+1} = y_i + \frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5$$

Và nghiệm tốt hơn dùng phương pháp Runge – Kutta bậc 5:

$$z_{i+1} = y_i + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6$$

Bước tính tối ưu được xác định bằng sh với s là:

$$s = \left(\frac{\varepsilon h}{2\left|z_{i+1} - y_{i+1}\right|}\right)^{\frac{1}{4}} = 0.840896\left(\frac{\varepsilon h}{\left|z_{i+1} - y_{i+1}\right|}\right)^{\frac{1}{4}}$$

Ta xây dựng hàm ***rkf()*** để thực hiện thuật toán trên:

```
function [t, y] = rkf ( f, t0, tf, x0, parms )
% tim nghiem cua phuong trinh  y'(t) = f( t, y ), y(t0) = x0
% dung phuong phap Runge-Kutta-Fehlberg bac 4, bac 5
neqn = length(x0);
hmin = parms(1);
hmax = parms(2);
tol  = parms(3);
t(1) = t0;
y(1:neqn, 1) = x0';
count = 0;
h = hmax;
i = 2;
while( t0 < tf )
   if nargin(f) > 1
      k1 = h*feval(f, t0, x0);
      k2 = h*feval(f, t0 + h/4, x0 + k1/4);
      k3 = h*feval(f, t0 + 3*h/8, x0 + 3*k1/32 + 9*k2/32);
      k4 = h*feval(f, t0 + 12*h/13, x0 + 1932*k1/2197 -...
            7200*k2/2197 + 7296*k3/2197);
      k5 = h*feval(f, t0 + h, x0 + 439*k1/216 - 8*k2 +...
            3680*k3/513 - 845*k4/4104);
      k6 = h*feval(f, t0 + h/2, x0 - 8*k1/27 + 2*k2 -...
            3544*k3/2565 + 1859*k4/4104 - 11*k5/40);
   else
      k1 = h * feval(f, t0);
      k2 = h * feval(f, t0 + h/4);
      k3 = h * feval(f, t0 + 3*h/8);
      k4 = h * feval(f, t0 + 12*h/13);
```

```
        k5 = h * feval(f, t0 + h);
        k6 = h * feval(f, t0 + h/2);
      end
      r = max(abs(k1/360 - 128*k3/4275 - 2197*k4/75240 +...
          k5/50 + 2*k6/55)/h);
      q = 0.84*(tol/r)^(1/4);
      if ( r < tol )
        x0 = x0 + 16*k1/135 + 6656*k3/12825 + 28561*k4/56430 -...
              9*k5/50 + 2*k6/55;
         t0 = t0 + h;
         t(i) = t0;
         y(1:neqn, i) = x0';
         i = i + 1;
     end;
      h = min(max(q, 0.1), 4.0)*h;
     if (h > hmax)
        h = hmax;
      end;
      if (t0 + h > tf)
         h = tf - t0;
     elseif (h < hmin)
        disp('Can giam kich thuoc buoc tinh');
        return;
     end;
end;
```

Để giải phương trình ta dùng chương trình ***ctrkf.m***:

```
clear all, clc
a = 0;
b = 1;
y  = @f2;
ya = 0.5;
p = [1e-5 1e-3 1e-8];%[hmin  hmax tol]
[t, y] = rkf(y, a, b, ya, p)
plot(t, y);
```

CHƯƠNG 8: TỐI ƯU HOÁ

§1. KHÁI NIỆM CHUNG VỀ TỐI ƯU HOÁ

Tối ưu hoá là thuật ngữ thường được dùng để cực tiểu hoá hay cực đại hoá một hàm. Thông thường ta chỉ cần tìm cực tiểu một hàm là đủ. Việc tìm cực đại của f(x) thực hiện một cách đơn giản bằng cách tìm cực tiểu của hàm $-f(x)$. Hàm f là hàm giá trị hay hàm đối tượng, cần được giữ cực tiểu. Biến x là biến có thể hiệu chỉnh tự do.

Các thuật toán cực tiểu hoá là các thủ thuật lặp đòi hỏi một giá trị ban đầu của biến x. Nếu f(x) có nhiều cực tiểu địa phương, việc chọn giá trị đầu sẽ xác định cực tiểu nào được tính. Ta không có cách nào bảo đảm là tìm được cực tiểu toàn cục.

Các biến có thể bị ràng buộc bằng các đẳng thức hay bất đẳng thức. Phần lớn các phương pháp là tìm cực tiểu không ràng buộc, nghĩa là không có hạn chế nào đối với biến x. Các bài toán này bao gồm tìm cực tiểu của hàm, tìm điểm tĩnh - điểm có gradient triệt tiêu. Các bài toán tìm cực tiểu có ràng buộc khó hơn và thuật toán khá phức tạp.

Trong chương này chúng ta sẽ lần lượt xét các thuật toán tìm cực tiểu không ràng buộc và có ràng buộc.

§2. PHƯƠNG PHÁP TIẾT DIỆN VÀNG

Ta xét bài toán tìm cực tiểu của hàm một biến f(x). Điểm cực tiểu được xác định theo điều kiện df/dx = 0. Do có thể có nhiều điểm cực tiểu nên ta phải vây điểm cực tiểu(xác định lân cận chứa điểm cực tiểu) trước. Thủ thuật vây điểm cực tiểu khá đơn giản: cho điểm đầu x_0 và tính giá trị của hàm đang đi xuống tại các điểm tiếp theo x_1, x_2, x_3,... cho đến tại x_n hàm tăng lại thì dừng. Điểm cực tiểu bị vây trong khoảng (x_{n-2}, x_n). Khoảng (x_{i+1}, x_i) không nên chọn là hằng số vì như vậy cần nhiều bước tính. Hợp lí nhất là nên tăng kích thước bước tính để đạt được cực tiểu nhanh hơn, ngay cả khi cực tiểu bị vây trong một đoạn khá rộng. Ta chọn kích thước tăng theo dạng hằng số: $h_{i+1} = ch_i$ với $c > 1$. ta xây dựng hàm ***goldbracket()*** để vây điểm cực tiểu của hàm:

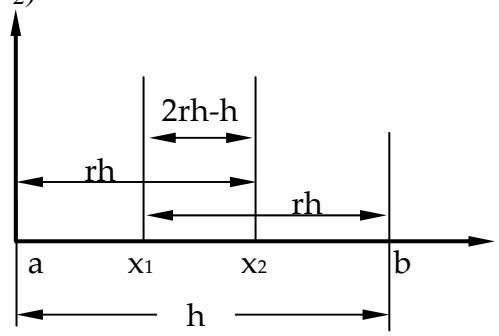
```
function [a, b] = goldbracket(func, x1, h)
% vay diem cuc tieu cua f(x).
c = 1.618033989;
f1 = feval(func, x1);
x2 = x1 + h;
f2 = feval(func,x2);
if f2 > f1
   h = -h;
   x2 = x1 + h;
   f2 = feval(func, x2);
   if f2 > f1
      a = x2;
      b = x1 - h;
```

```
        return
    end
end
for i = 1:100
    h = c*h;
    x3 = x2 + h;
    f3 = feval(func, x3);
    if f3 > f2
        a = x1;
        b = x3;
        return
    end
    x1 = x2;
    f1 = f2;
    x2 = x3;
    f2 = f3;
end
error('Goldbracket khong tim thay diem cuc tieu')
```

Tiết diện vàng là một biến thể của phương pháp chia đôi dùng khi tìm nghiệm của phương trình f(x) = 0. Giả sử điểm cực tiểu bị vây trong khoảng (a, b) có độ dài h. Để thu nhỏ khoảng (a, b) ta tính giá trị của hàm tại $x_1 = b - rh$ và $x_2 = a + rh$ như hình vẽ. Nếu $f_1 = f(x_1)$ lớn hơn $f_2 = f(x_2)$ như hình a thì cực tiểu nằm trong khoảng (x_1, b) nếu ngược lại cực tiểu nằm trong khoảng (a, x_2).

Giả sử $f_1 > f_2$, ta đặt $a = x_1$ và và $x_1 = x_2$ và có khoảng (a, b) mới như hình b. Để thực hiện bước thu gọn tiếp theo ta lại tính giá trị của hàm tại $x_2 = a + rh'$ và lặp lại quá trình. Quá trình làm việc chỉ nếu hình a và hình b tương tự, nghĩa là hằng số r không đổi khi xác định x_1 và x_2 ở cả hai hình. Từ hình a ta thấy:

$$x_2 - x_1 = 2rh - h$$



a

Cùng một khoảng cách đó từ hình b ta có:

$$x_1 - a = h' - rh'$$

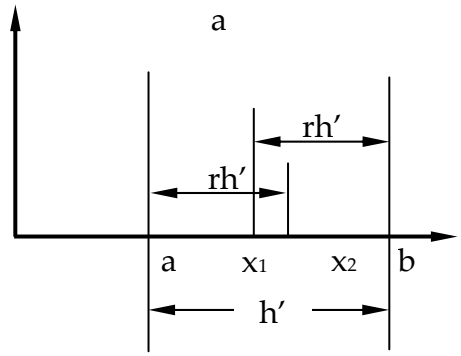
Cân bằng các khoảng này ta được:

$$2rh - h = h' - rh'$$

Thay h' = rh và khử h:

$$2r - 1 = r(1 - r)$$

Giải phương trình này ta nhận được tỉ lệ vàng:



b

$$r = \frac{\sqrt{5}-1}{2} = 0.618033989...$$

Chú ý là mỗi lần thu gọn khoảng chứa điểm cực tiểu thì khoảng (a, b) giảm tỉ lệ với r. Điều này làm số lần tính lớn hơn phương pháp chia đôi. Tuy nhiên phương pháp tỉ lệ vàng chỉ đòi hỏi tính giá trị hàm một lần trong khi phương pháp chia đôi cần tính giá trị hàm 2 lần. Số lần tính xác định bằng:

$$|b-a|r^n = \varepsilon$$

hay: $$n = \frac{\ln\frac{\varepsilon}{|b-a|}}{\ln} = -2.078087\, n\frac{\varepsilon}{|b-a|}$$

$$h = b - a = 0.382$$

Ta xây dựng hàm ***golden()*** để thực hiện thuật toán này:

```
function [xmin, ymin] = golden(f, a, b, delta, epsilon)
% a va b la doan tim cuc tieu
% delta sai so cua x
% epsilon sai so cua y
r1 = (sqrt(5) - 1)/2;
r2 = r1^2;
h = b - a;
fa = f(a);
fb = f(b);
x1 = a + r2*h;
x2 = a + r1*h;
f1 = f(x1);
f2 = f(x2);
k = 1;
while (abs(fb-fa) > epsilon)|(h > delta)
  k = k + 1;
  if (f1 < f2)
    b = x2;
    fb = f2;
    x2 = x1;
    f2 = f1;
    h = b - a;
    x1 = a + r2*h;
    f1 = f(x1);
  else
    a = x1;
    fa = f1;
    x1 = x2;
```

```
      f1 = f2;
      h = b - a;
      x2 = a + r1*h;
      f2 = f(x2);
    end
  end
  dp = abs(b - a);
  dy = abs(fb - fa);
  p = a;
  yp = fa;
  if (fb < fa)
    p = b;
    yp = fb;
  end
  xmin = p;
  ymin = yp;
```

Để tìm cực tiểu của hàm ta dùng chương trình ***ctgolden.m***:

```
clear all, clc
f = inline('1.6*x^3 + 3*x^2 -2*x');
x = 0;
delta = 1e-8;
epsilon = 1e-10;
[a, b] = goldbracket(f, x, 0.2);
[xmin, ymin] = golden(f, a, b, delta, epsilon)
```

§3. PHƯƠNG PHÁP XẤP XỈ BẬC HAI

Ý tưởng của phương pháp này là:

- xấp xỉ hàm đối tượng f(x) bằng một hàm bậc 2 $p_2(x)$ qua 3 điểm cho trước
- cập nhật 3 điểm này bằng cách thay một trong 3 điểm bằng cực tiểu của hàm $p_2(x)$

Qua 3 điểm:

$$\{[(x_0, f(x_0)], [(x_1, f(x_1)], [(x_2, f(x_2)]\} \quad x_0 < x_1 < x_2$$

ta tìm được đa thức nội suy $p_2(x)$ và điểm có đạo hàm bằng zero:

$$x = x_3 = \frac{f_0(x_1^2 - x_2^2) + f_1(x_2^2 - x_0^2) + f_2(x_0^2 - x_1^2)}{2\left[f_0(x_1 - x_2) + f_1(x_2 - x_0) + f_2(x_0 - x_1)\right]} \qquad (1)$$

Đặc biệt, nếu các điểm tìm được trước đây phân bố đều với khoảng cách h(nghĩa là $x_2 - x_1 = x_1 - x_0 = h$) thì (1) trở thành:

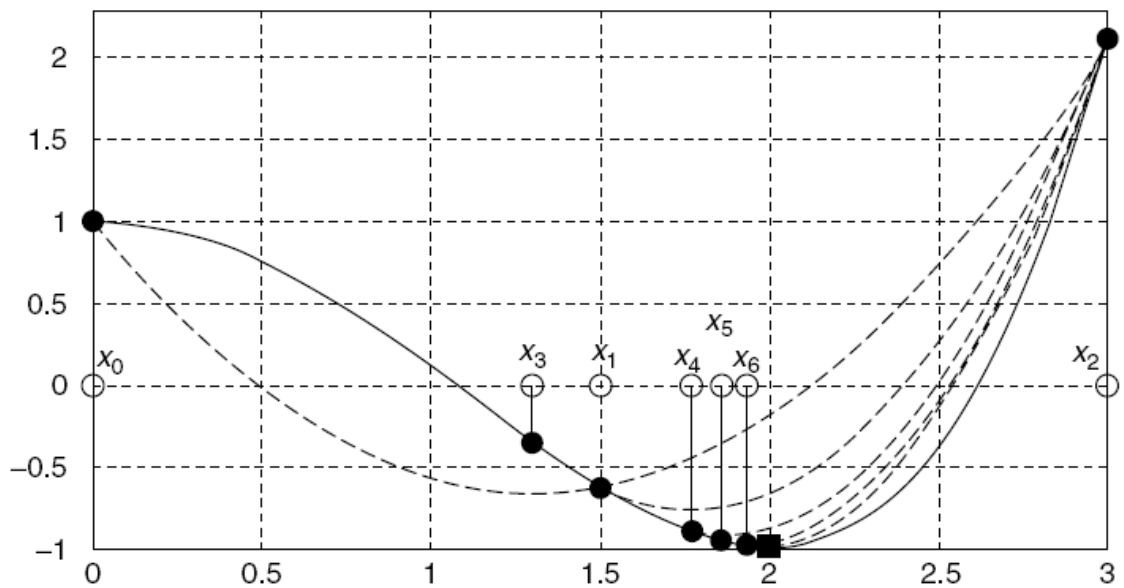
$$x_3 = \frac{f_0(x_1^2 - x_2^2) + f_1(x_2^2 - x_0^2) + f_2(x_0^2 - x_1^2)}{2\left[f_0(x_1 - x_2) + f_1(x_2 - x_0) + f_2(x_0 - x_1)\right]} = x_0 + h\frac{3f_0 - 4f_1 + f_2}{2(-f_0 + 2f_1 - f_2)} \quad (2)$$

Ta cập nhật 3 điểm theo cách này cho đến khi $|x_2 - x_0| \approx 0$ hay $|f(x_2) - f(x_0)| \approx 0$ và cực tiểu là x_3. Quy tắc cập nhật 3 điểm là:

- Trong trường hợp $x_0 < x_3 < x_1$ ta dùng (x_0, x_3, x_1) hay (x_3, x_1, x_2) làm 3 điểm mới tuỳ theo $f(x_3) < f(x_1)$ hay không

- Trong trường hợp $x_1 < x_3 < x_2$ ta dùng (x_1, x_3, x_2) hay (x_0, x_1, x_3) làm 3 điểm mới tuỳ theo $f(x_3) \le f(x_1)$ hay không.

Quá trình tìm cực tiểu được mô tả trên hình sau:



Ta xây dựng hàm ***optquad()*** để thực hiện thuật toán này.

```
function [xo, fo] = optquad(f, x0, tolx, tolfun, maxiter)
%Tim cuc tieu cua f(x) bang phuong phap xap xi bac 2
if length(x0) > 2
   x012 = x0(1:3);
else
   if length(x0) == 2
      a = x0(1);
      b = x0(2);
   else
      a = x0 - 10;
      b = x0 + 10;
   end
   x012 = [a (a + b)/2  b];
end
```

```
f012 = f(x012);
[xo, fo] = optquad0(f, x012, f012, tolx, tolfun, maxiter);

function [xo, fo] = optquad0(f, x012, f012, tolx, tolfun, k)
x0 = x012(1);
x1 = x012(2);
x2 = x012(3);
f0 = f012(1);
f1 = f012(2);
f2 = f012(3);
nd = [f0 - f2 f1 - f0 f2 - f1]*[x1*x1 x2*x2 x0*x0; x1 x2 x0]';
x3 = nd(1)/2/nd(2);
f3 = feval(f, x3); %Pt.(1)
if k <= 0 | abs(x3 - x1) < tolx | abs(f3 - f1) < tolfun
   xo = x3;
   fo = f3;
   if k == 0
      fprintf('Day co the xem la diem cuc tieu')
   end
else
   if x3 < x1
      if f3 < f1
         x012 = [x0 x3 x1];
         f012 = [f0 f3 f1];
      else
         x012 = [x3 x1 x2];
         f012 = [f3 f1 f2];
      end
   else
      if f3 <= f1
         x012 = [x1 x3 x2];
         f012 = [f1 f3 f2];
      else
         x012 = [x0 x1 x3];
         f012 = [f0 f1 f3];
      end
   end
   [xo, fo] = optquad0(f, x012, f012, tolx, tolfun, k - 1);
end
```

Để tìm điểm cực tiểu ta dùng chương trình ***ctoptquad.m***:

```
clear all, clc
%f = inline('(x.^2 - 4).^2/8 - 1');
a = 0;
b = 3;
delta = 1e-8;
epsilon = 1e-10;
maxiter = 100;
[xmin, ymin] = optquad(f, [a  b], delta, epsilon, maxiter)
```

§4. PHƯƠNG PHÁP NELDER - MEAD

Phương pháp Nelder - Mead có thể dùng để tìm cực tiểu của hàm nhiều biến mà phương pháp tiết diện vàng hay phương pháp xấp xỉ bậc 2 không dùng được. Thuật toán Nelder - Mead gồm các bước như sau:

Bước 1: Cho 3 điểm đầu tiên a, b, c với f(a) < f(b) < f(c)

Bước 2: Nếu 3 điểm và các giá trị tương ứng của hàm đủ gần nhau thì ta coi a là điểm cực tiểu và kết thúc quá trình tính

Bước 3: Nếu không ta coi điểm cực tiểu nằm đối diện với điểm c trên đường ab(xem hình vẽ) và lấy:

e = m + 2(m - c)

với

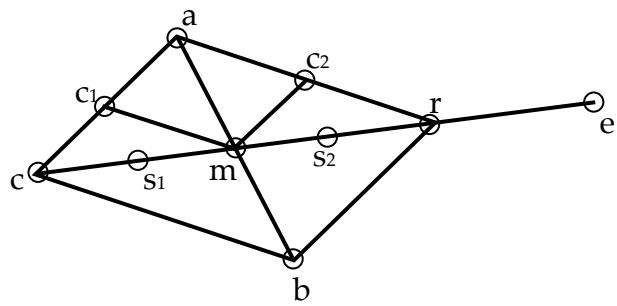
m = (a + b)/2

và nếu f(e) < f(b) thì lấy:

r = (m + e)/2 = 2m - c

và nếu f(r) < f(c) thì lấy r làm giá trị mới của c; nếu f(r) ≥ f(b) thì lấy:

s = (c + m)/2



m = (a + b)/2 r = m + (m - c)

e = m + 2(m - c) s_1 = (c + m)/2

s_2 = (m + r)/2 c_1 = (c + a)/2

c_2 = (m + a)/2

và nếu f(s) < f(c) thì lấy s làm giá trị mới của c, nếu không bỏ các điểm b, c và dùng m và c_1 = (a + c)/2 làm điểm b và c mới và cho rằng cực tiểu nằm quanh điểm a.

Bước 4: Trở về bước 1

Ta xây dựng hàm ***neldermead()*** để thực hiện thuật toán này:

```
function [xo, fo] = neldermead(f, x0, tolx, tolfun, maxiter)
n = length(x0);
if n == 1 %truong hop ham 1 bien
   [xo,fo] = optquad(f, x0, tolx, tolfun);
   return
end
S = eye(n);
for i = 1:n
   i1 = i + 1;
   if i1 > n
```

```
        i1 = 1;
    end
    abc = [x0; x0 + S(i,:); x0 + S(i1,:)];
    fabc = [feval(f, abc(1, :)); feval(f,abc(2, :)); feval(f,abc(3, :))];
    [x0, fo] = neldermead0(f, abc, fabc, tolx, tolfun, maxiter);
    if n < 3,
        break;
    end
end
xo = x0;

function [xo, fo] = neldermead0(f, abc, fabc, tolx, tolfun, k)
[fabc, I] = sort(fabc);
a = abc(I(1), :);
b = abc(I(2), :);
c = abc(I(3), :);
fa = fabc(1);
fb = fabc(2);
fc = fabc(3);
fba = fb - fa;
fcb = fc - fb;
if k <= 0 | abs(fba) + abs(fcb) < tolfun | abs(b - a) + abs(c - b) < tolx
    xo = a;
    fo = fa;
    if k == 0
        fprintf('Xem day la diem cuc tieu')
    end
else
    m = (a + b)/2;
    e = 3*m - 2*c;
    fe = feval(f, e);
    if fe < fb
        c = e;
        fc = fe;
    else
        r = (m + e)/2;
        fr = feval(f, r);
        if fr < fc
            c = r;
            fc = fr;
        end
        if fr >= fb
```

```
        s = (c + m)/2;
        fs = feval(f, s);
        if fs < fc
           c = s;
           fc = fs;
        else
           b = m;
           c = (a + c)/2;
           fb = feval(f, b);
           fc = feval(f, c);
        end
     end
  end
  [xo, fo] = neldermead0(f, [a;b;c], [fa fb fc], tolx, tolfun, k - 1);
end
```

Để tìm cực tiểu của hàm $z = f(x, y) = x_1^2 - x_1x_2 - 4x_1 + x_2^2 - x_2$ lân cận [0 0] ta dùng chương trình ***ctneldermead.m***:

```
clear all, clc
f = inline('x(1)*x(1) - x(1)*x(2) - 4*x(1) + x(2) *x(2) - x(2)');
x0 = [0  0];
b = 1;
delta = 1e-8;
epsilon = 1e-10;
maxiter = 100;
[xmin, ymin] = neldermead(f, x0, delta, epsilon, maxiter)
```

§5. PHƯƠNG PHÁP ĐỘ DỐC LỚN NHẤT

Phương pháp này tìm điểm cực tiểu của hàm n biến theo hướng gradient âm:

$$-g([x]) = -\nabla f([x]) = -\left[\frac{\partial f(x)}{\partial x_1} \quad \frac{\partial f(x)}{\partial x_2} \quad \cdots \quad \frac{\partial f(x)}{\partial x_n}\right]$$

với kích thước bước tính α_k tại lần lặp thứ k được hiệu chỉnh sao cho giá trị hàm cực tiểu theo hướng tìm. Thuật toán gồm các bước sau:

- Tại lần lặp thứ k = 0, tìm giá trị hàm $f(x_0)$ với điểm khởi đầu x_0
- Tại lần lặp thứ k, tìm α_k theo hướng -g(x)

$$\alpha_{k-1} = f\left(x_{k-1} - \alpha g_{k-1} / \|g_{k-1}\|\right) \qquad (1)$$

- Tính giá trị x_k:

$$x_k = x_{k-1} - \alpha_{k-1} g_{k-1} / \|g_{k-1}\| \qquad (2)$$

• Nếu $x_k \approx x_{k-1}$ và $f(x_k) \approx f(x_{k-1})$ thì coi là cực tiểu, nếu không thì quay về bước 2.

```
function [xo, fo] = steepest(f, x0, tolx, tolfun, alpha0,maxiter)
if nargin < 6
   maxiter = 100;
end
if nargin < 5
   alpha0 = 10; %kich thuoc khoi gan
end
if nargin < 4
   tolfun = 1e-8;
end %|f(x)| < tolfun mong muon
if nargin < 3
   tolx = 1e-6;
end %|x(k)- x(k - 1)| < tolx mong muon
x = x0;
fx0 = feval(f, x0);
fx = fx0;
alpha = alpha0;
kmax1 = 25;
warning = 0;
for k = 1:maxiter
   g = grad(f, x);
   g = g/norm(g); %gradient la vec to hang
   alpha = alpha*2; %de thu di theo huong gradient am
   fx1 = feval(f, x - alpha*2*g);
   for k1 = 1:kmax1 %tim buoc toi uu
      fx2 = fx1;
    fx1 = feval(f, x - alpha*g);
      if fx0 > fx1+ tolfun & fx1 < fx2 - tolfun %fx0 > fx1 < fx2
         den = 4*fx1 - 2*fx0 - 2*fx2;
         num = den - fx0 + fx2; %
         alpha = alpha*num/den;Pt.(1)
         x = x - alpha*g;
         fx = feval(f,x); %Pt.(2)
         break;
      else
         alpha = alpha/2;
      end
   end
   if k1 >= kmax1
```

```
        warning = warning + 1; %kg tim duoc buoc toi uu
    else
        warning = 0;
    end
    if warning >= 2|(norm(x - x0) < tolx&abs(fx - fx0) < tolfun)
        break;
    end
    x0 = x;
    fx0 = fx;
end
xo = x; fo = fx;
if k ==maxiter
    fprintf('Day la ket qua tot nhat sau  %d lan lap', maxiter)
end

function g = grad(func, x)
% Tinh gradient cua ham f(x).
h = 1.0e-6;
n = length(x);
g = zeros(1, n);
f0 = feval(func, x);
for i = 1:n
    temp = x(i);
    x(i) = temp + h;
    f1 = feval(func, x);
    x(i) = temp;
    g(1, i) = (f1 - f0)/h;
end
```

Để tìm cực tiểu của hàm ta dùng chương trình ***ctsteepest.m***:

```
clear all, clc
f = inline('x(1)*x(1) - x(1)*x(2) - 4*x(1) + x(2) *x(2) - x(2)');
x0 = [0.5  0.5];
tolx = 1e-4;
tolfun = 1e-9;
alpha0 = 1;
maxiter = 100;
[xo, fo] = steepest(f, x0, tolx, tolfun, alpha0, maxiter)
```

§6. PHƯƠNG PHÁP NEWTON

Việc tìm điểm cực tiểu của hàm f(x) tương đương với việc xác định x để cho gradient g(x) của hàm f(x) bằng zero. Nghiệm của g(x) = 0 có thể tìm được bằng cách dùng phương pháp Newton cho hệ phương trình phi tuyến. Hàm ***newtons(x)*** dùng để tìm nghiệm của phương trình g(x) = 0 là:

```
function [x, fx, xx] = newtons(f, x0, tolx, maxiter)
h = 1e-4;
tolfun = eps;
EPS = 1e-6;
fx = feval(f, x0);
nf = length(fx);
nx = length(x0);
if nf ~= nx
   error('Kich thuoc cua g va x0 khong tuong thich!');
end
if nargin < 4
   maxiter = 100;
end
if nargin < 3
   tolx = EPS;
end
xx(1, :) = x0(:).';
for k = 1: maxiter
   dx = -jacob(f, xx(k, :), h)\fx(:);%-[dfdx]^-1*fx
   xx(k + 1, :) = xx(k, :) + dx.';
   fx = feval(f, xx(k + 1, :));
   fxn = norm(fx);
   if fxn < tolfun | norm(dx) < tolx
      break;
   end
end
x = xx(k + 1, :);
if k == maxiter
   fprintf('Ket qua tot nhat sau %dlan lap\n', maxiter)
end

function g = jacob(f, x, h) %Jacobian cua f(x)
if nargin < 3
   h = 1e-4;
end
h2 = 2*h;
n = length(x);
```

```
x = x(:).';
I = eye(n);
for n = 1:n
   g(:, n) = (feval(f, x + I(n, :)*h) - feval(f, x - I(n,:)*h))'/h2;
end
```

Để tìm cực tiểu của hàm bằng phương pháp Newtons ta dùng chương trình ***ctnewtons.m***:

```
clear all, clc
f = inline('x(1).^2 - x(1)*x(2) - 4*x(1) + x(2).^2 - x(2)');
g = inline('[(2*x(1) - x(2) -4)  ( 2*x(2) - x(1) - 1)]');
x0 = [0.1  0.1];
tolx = 1e-4;
maxiter = 100;
[xo, fo] = newtons(g, x0, tolx, maxiter)
```

§7. PHƯƠNG PHÁP GRADIENT LIÊN HỢP

1. Khái niệm chung: Một trong các phương pháp giải bài tón tìm cực tiểu của hàm nhiều biến là tìm cực tiểu theo một biến liên tiếp để đến gần điểm cực tiểu. Chiến thuật chung là:

- chọn điểm $[x_0]$
- lặp với i = 1, 2, 3,...
  - chọn vec tơ $[v_i]$
  - cực tiểu hoá f([x]) dọc theo đường $[x_{i-1}]$ theo hướng $[v_i]$. Coi $[x_i]$ là cực tiểu. Nếu $\left\|[x_i]-[x_{i-1}]\right\| < \varepsilon$ thì kết thúc lặp
- kết thúc

2. Gradient liên hợp: Ta khảo sát hàm bậc 2:

$$f([x]) = c - \sum_i b_i x_i + \frac{1}{2}\sum_i\sum_j A_{i,j}x_i x_j = c - [b]^T[x] + \frac{1}{2}[x]^T[A][x] \qquad (1)$$

đạo hàm của hàm theo x_i cho ta:

$$\frac{\partial f}{\partial x_i} = -b_i + \sum_j A_{i,j}x_j$$

Viết dưới dạng vec tơ ta có:

$$\nabla f = -[b] + [A][x] \qquad (2)$$

với ∇f là gradient của f.

Bây giờ ta khảo sát sự thay đổi gradient khi ta di chuyển từ $[x_0]$ theo hướng của vec tơ [u] dọc theo đường:

$[x] = [x_0] + s[u]$

với s là khoảng cách di chuyển. Thay biểu thức này vào (2) ta có gradient dọc theo [u]:

$$\nabla f|_{[x_0]+s[u]} = -[b] + [A]([x_0] + s[u]) = \nabla f|_{[x_0]} + s[A][u]$$

Như vậy sự thay đổi gradient là s[A][u]. Nếu ta di chuyển theo hướng vuông góc với vec tơ [v], nghĩa là

$$[v]^T[u] = 0 \text{ hay } [v]^T[A][u] = 0 \qquad (3)$$

thì hướng của [u] và [v] là liên hợp. Điều này cho thấy khi ta muốn cực tiểu hoá f(x) theo hướng [v], ta cần di chuyển theo hướng [u] để không làm hỏng cực tiểu trước đó. Với hàm bậc hai n biến độc lập ta có thể xây dựng n hướng liên hợp. Các phương pháp gradient liên hợp khác nhau dùng các kỹ thuật khác nhau để tìm hướng liên hợp.

3. Phương pháp Powell: Phương pháp Powell là phương pháp bậc zero, chỉ đòi hỏi tính f([x]). Thuật toán gồm các bước:

- chọn điểm $[x_0]$
- chọn vec tơ $[v_i]$, thường lấy $[v_i] = [e_i]$ với $[e_i]$ là vec tơ đơn vị theo hướng $[x_i]$
- vòng tròn
  - lặp với i = 1, 2,...
    - ∗ tìm cực tiểu của f([x]) dọc theo đường đi qua $[x_{i-1}]$ theo hướng $[v_i]$; coi $[x_i]$ là cực tiểu
  - kết thúc lặp
  - $[v_{n+1}] = [x_0] - [x_n]$; tìm cực tiểu của f([x]) dọc theo đường đi qua$[x_0]$ theo hướng $[v_{n+1}]$; coi $[x_{n+1}]$ là cực tiểu
  - nếu $\|[x_{n+1}] - [x_n]\| < \varepsilon$ thì kết thúc vòng lặp
  - lặp
    - ∗ $[v_{i+1}] = [v]$
- kết thúc vòng tròn

Ta xây dựng hàm powell() để thực hiện thuật toán trên:

```
function [xmin, fmin, ncyc] = powell(h, tol)
% Phuong phap Powell de tim cuc tieu cua ham f(x1,x2,...,xn).
global x func v
if nargin < 2;
   tol = 1.0e-6;
end
if nargin < 1
   h = 0.1;
end
if size(x,2) > 1
   x = x';
end
n = length(x);
df = zeros(n, 1);
```

```
u = eye(n);
   xold = x;
   fold = feval(func, xold);
   for i = 1:n
      v = u(1:n, i);
      [a, b] = goldbracket(@fline, 0.0, h);
      [s, fmin] = golden(@fline, a, b);
      df(i) = fold - fmin;
      fold = fmin;
      x = x + s*v;
   end
   v = x - xold;
[a, b] = goldbracket(@fline, 0.0, h);
   [s,fMin] = golden(@fline, a, b);
   x = x + s*v;
   if sqrt(dot(x-xold, x-xold)/n) < tol
      xmin = x;
      ncyc = j;
      return
   end
   imax = 1;
   dfmax = df(1);
   for i = 2:n
      if df(i) > dfmax
         imax = i;
         dfmax = df(i);
      end
   end
   for i = imax:n-1
      u(1:n, i) = u(1:n, i+1);
   end
   u(1:n, n) = v;
   end
error('Phuong phap Powell khong hoi tu')

function z = fiine(s) % f theo huong cua v
global x func v
z = feval(func, x+s*v);

function y = f(x)
y = 100.0*(x(2) - x(1).^2).^2 + (1.0 - x(1)).^2;
```

Để tìm điểm cực tiểu ta dùng chương trình ***ctpowell.m***:

```
clear all, clc
global x  func
func = @f;
x = [-1.0; 1.0];
[xmin, fmin, numcycles] = powell
fminsearch(func, x)
```

3. Phương pháp Fletcher - Reeves: Phương pháp Powell cần n đường cực tiểu hoá. Ta có thể chỉ cần dùng một đường với phương pháp bậc 1. Phương pháp này có 2 phương án: thuật toán Fletcher - Reeves(FR) và thuật toán Polak - Ribiere(PR). Thuật toán tóm tắt như sau:

- cho $[x_0]$, tính $f([x_0])$
- khởi gán $x(n) = x_k$; tính $[g_0] = -\nabla f([x_0])$; $s(k) = - g(x_k)$
- lặp k = 0, 1, 2,...
  - $[x_{k+1}] = [x_k] + \alpha_k[s_k]$
  - $\beta_k = \dfrac{\left([g_{k+1}]^T - [g_k]^T\right)[g_{k+1}]}{[g_k]^T[g_k]}$(FR) hay $\beta_k = \dfrac{[g_{k+1}]^T[g_{k+1}]}{[g_k]^T[g_k]}$(PR)
  - $[s_{k+1}] = -[g_{k+1}] + \beta_k[s_k]$
  - cập nhật $[x_k]$

  cho đến khi hội tụ

Ta xây dựng hàm ***conjugate()*** để thực hiện thuật toán trên:

```
function [xo, fo] = conjugate(f, x0, tolx, tolfun, alpha0, maxiter, KC)
%KC = 1:  Phuong phap gradient lien hop Polak–Ribiere
%KC = 2: Phuong phap gradient lien hop Fletcher–Reeves
if nargin < 7
   KC = 0;
end
if nargin < 6
   maxiter = 100;
end
if nargin < 5
   alpha0 = 10;
end
if nargin < 4
   tolfun = 1e-8;
end
if nargin < 3
```

```
    tolx = 1e-6;
end
n = length(x0);
nmax1 = 20;
warning = 0;
h = 1e-4;
x = x0;
 fx = feval(f, x0);
 fx0 = fx;
for k = 1: maxiter
    xk0 = x;
    fk0 = fx;
    alpha = alpha0;
    g = grad(f, x);
    s = -g;
    for n = 1:n
        alpha = alpha0;
        fx1 = feval(f, x + alpha*2*s);
        for n1 = 1:nmax1
            fx2 = fx1;
            fx1 = feval(f, x+alpha*s);
            if (fx0 > fx1 + tolfun) & ( fx1 < fx2 - tolfun) %fx0 > fx1 < fx2
                den = 4*fx1 - 2*fx0 - 2*fx2;
                num = den - fx0 + fx2;
                alpha = alpha*num/den;
                x = x + alpha*s;
                fx = feval(f, x);
                break;
            elseif n1 == nmax1/2
                alpha = -alpha0;
                fx1 = feval(f, x + alpha*2*s);
            else
                alpha = alpha/2;
            end
        end
        x0 = x;
        fx0 = fx;
        if n < n
            g1 = grad(f, x, h);
            if KC <= 1
                s = - g1 +(g1 - g)*g1'/(g*g'+ 1e-5)*s;
            else
```

```
            s = -g1 + g1*g1'/(g*g'+ 1e-5)*s;
         end
         g = g1;
      end
      if n1 >= nmax1
         warning = warning + 1; %kg tim duoc kich thuoc toi uu
      else
         warning = 0;
      end
   end
   if (warning >= 2)|((norm(x - xk0)<tolx) & (abs(fx - fk0)< tolfun))
      break;
   end
end
xo = x;
fo = fx;
if k == maxiter
   fprintf('’Gia tri tot nhat sau %d lan lap', maxiter)
end
```

§8. PHƯƠNG PHÁP MÔ PHỎNG QUÁ TRÌNH Ủ

Tất cả các phương pháp tìm điểm cực tiểu mà ta xét đến nay chỉ làm việc có hiệu quả khi điểm ban đầu được lực chọn đủ gần với điểm cực tiểu. Điểm cực tiểu tìm được là một trong nhiều điểm cực tiểu có thể có và ta không chắc là tìm được điểm cực tiểu toàn cục. Vấn đề là làm sao lặp lại thủ tục để tìm được tất cả các điểm cực tiểu từ các điểm đầu khác nhau và đưa ra điểm cực tiểu toàn cục. Đây là một bài toán khó vì không có một phương pháp nào để xác định được các điểm đầu thích hợp để tìm được tất cả các điểm cực tiểu. Một sự lựa chọn thú vị dựa trên sự tương tự giữa sự ủ và cực tiểu hoá. Ủ là quá trình gia nhiệt khối kim loại lên đến nhiệt độ cao hơn nhiệt độ nóng chảy và sau đó hạ nhiệt độ từ từ để các nguyên tử bị kích thích mạnh có thể trở về trạng thái năng lượng thấp, tạo thành một tinh thể duy nhất có cấu trúc hình chữ nhật. Làm nguội nhanh sẽ có thể tạo ra sự không đồng nhất và làm biến dạng cấu trúc tinh thể giống như khi tìm cực tiểu toàn cục quá nhanh. Phương pháp mô phỏng quá trình ủ (simulated annealing - SA) có thể thực hiện bằng cách dùng phân bố xác suất Boltzmann của mức năng lượng E tại nhiệt độ T, được mô tả bằng:

$$p(E) = \alpha e^{-\frac{E}{KT}} \qquad (1)$$

Chú ý là ở nhiệt độ cao, đường cong phân bố xác suất phẳng trong một phạm vi E rộng, ngụ ý là hệ thống có thể ở trạng thái năng lượng cao cũng ngang bằng ở trạng thái năng lượng thấp. Trong khi đó ở nhiệt độ thấp đường cong phân bố xác suất cao ở nhiệt độ thấp và thấp ở nhiệt độ cao, ngụ ý là hệ thống có khả năng ở mức năng

lượng thấp nhiều hơn nhưng vẫn có một cơ hội nhỏ ở trạng thái năng lượng cao để nó có thể thoát khỏi trạng thái năng lượng cực tiểu địa phương.

Ý tưởng của thuật toán SA gồm các bước sau:

- Cho giá trị ban đầu [x_o], biên dưới [l], biên trên [u], số lần lặp cực đai, k_{max}, hệ số tắt q > 0(tắt nhanh hay chậm) và sai số tương đối ε_f của dao động giá trị của hàm.
- Cho [x] = [x_o]; [x^o] = [x]; [f^o] = f([x])
- Lặp từ k = 1 đến k_{max}
  - ∗ tạo ra vec tơ ngẫu nhiên N×1: [U] = [-1, +1]
  - ∗ biến đổi vec tơ [U] bằng luật μ nghịch đảo với $\mu = 10^{100\left(\frac{k}{k_{max}}\right)^q}$ để tạo ra [Δx] và lấy [x_1] = [x] + [Δx]
  - ∗ nếu [Δf] = f([x_1]) - f([x]) < 0
    - [x] = [x_1] và nếu f([x]) < [f^o] thì [x] = [x^o] và [f^o] = f([x_o])
  - ∗ không thì:
    - tạo số ngẫu nhiên z trong đoạn [0, 1] và cho [x] = [x_1] chỉ trong trường hợp z < p
- Với [x^o] gần với điểm cực tiểu mà ta đang tìm ta có thể dùng nó như giá trị đầu và dùng các quy tắc tìm cực tiểu địa phương khác để tìm điểm cực tiểu của hàm f([x])

Dựa trên thuật toán này ta xây dựng hàm ***simannealing()***. Hàm này có hai phần có số bước lặp thay đổi khi nhiệt độ giảm. Một phần có kích thước của bước tính [Δx] tạo ra bởi vec tơ ngẫu nhiên [y] có các giá trị nằm trong đoạn [-1, 1] với biến [x] có cùng kích thước và nhân μ^{-1}([y]) với hiệu ([u] -[l]). Quy tắc μ nghịch đảo:

$$g_\mu^{-1} = \frac{(1+\mu)^{|y|}}{\mu}\text{sign}(y) \quad |y| \le 1 \tag{2}$$

thực hiện trong hàm ***invmu()*** với μ tăng theo quy luật:

$$\mu = 10^{100\left(\frac{k}{k_{max}}\right)^q} \tag{3}$$

```
function x = muinv(y, mu) % luat mu nghich dao Pt.(2)
x = (((1 + mu).^abs(y) - 1)/mu).*sign(y);
```

Phần còn lại là xác suất của việc dùng bước [Δx]. Tương tự (1) ta có:

$$p = e^{-\left(\frac{k}{k_{max}}\right)^q \frac{\Delta f}{|f([x])|\varepsilon_f}} \quad \Delta f > 0 \tag{4}$$

Ta xây dựng hàm ***simannealing()*** để thực hiên thuật toán trên:

```
function [xo, fo] = simannealing(f, x0, l, u, kmax, q, tolfun)
% Phuong phap SA de tim cuc tieu cua ham f(x)  l <= x <= u
```

```
n = length(x0);
x = x0;
fx = feval(f, x);
xo = x;
fo = fx;
if nargin < 7
   tolfun = 1e-10;
end
if nargin < 6
   q = 1;
end %he so tat
if nargin < 5
   kmax = 100;
end %so lan lap max
for k = 0:kmax
   Ti = (k/kmax)^q;
   mu = 10^(Ti*100); % Pt.(3)
   dx = muinv(2*rand(size(x))- 1, mu).*(u - l);
   x1 = x + dx;
   x1 = (x1 < l).*l +(l <= x1).*(x1 <= u).*x1 +(u < x1).*u;
   fx1 = feval(f, x1);
   df = fx1 -  fx;
   if (df <0)|(rand < exp(-Ti*df/(abs(f(x)+eps))/tolfun))%Pt.(4)
      x = x1;
      fx = fx1;
   end
   if fx < fo
      xo = x;
      fo = fx1;
   end
end
```

Để tìm cực tiểu của hàm ta dùng chương trình ***ctsimannealing.m***:

```
clear, clc
f = inline('x(1)^4 - 16*x(1)^2 - 5*x(1) + x(2)^4 - 16*x(2)^2 - 5*x(2)','x');
l = [-5  -5];
u = [5  5]; %bien duoi/tren
%x0 = [-0.5   -1];
x0 = [0   0];
kmax = 500;
q = 1;
```

```
tolfun = 1e-10;
[xmin, fmin] = simannealing(f, x0, l, u, kmax, q, tolfun)
[xmin1, fmin1] = neldermead(f, x0, 1e-5, 1e-8, kmax)
[xmin2, fmin2] = fminsearch(f, x0)
```

Trong chương trình trên, ta dùng thêm các thuật toán khác để so sánh. Kết quả là thuật toán SA có thể tìm được cực tiểu toàn cục. Tuy nhiên không phải khi nào thuật toán cũng thành công. Sự thành công của thuật toán này phụ thuộc giá trị đầu và may mắn, trong khi các thuật toán khác sự thành công chỉ phụ thuộc giá trị đầu.

§9. THUẬT TOÁN DI TRUYỀN

Thuật toán di truyền (genetic algorithm - GA) là một kỹ thuật tìm ngẫu nhiên có định hướng, mô phỏng sự chọn lọc tự nhiên để có các cá thể sống sót thích nghi nhất. Cũng như thuật toán SA, GA cho phép tìm được cực tiểu toàn cục ngay cả khi hàm đối tượng có nhiều cực trị gồm các điểm uốn, các cực tiểu địa phương, các cực đại địa phương. Thuật toán di truyền lai gồm các bước: khởi gán, chọn lọc, vượt qua, đột biến. Thuật toán gồm các bước sau:

- Cho giá trị đầu $[x_o] = [x_{o1}, x_{o2},...,x_{oN}]$ (N là kích thước của biến), biên dưới $[l] = [l_1,...,l_N]$, biên trên $[u] = [u_1,...,u_N]$, kích thước của quần thể N_p, vec tơ $N_b = [N_{b1},..., N_{bN}]$ gồm số bit các gán cho mỗi thể hiện của mỗi biến x_i, xác suất sống sót P_c, xác suất đột biến P_m, tỉ lệ học $\eta(0\leq \eta \leq 1$, thể

hiện học nhanh hay chậm), số lần lặp cực đại k_{max}. Chú ý là kích thước của $[x_o]$, [u], [l] đều là N.

• Khởi tạo ngẫu nhiên số cá thể ban đầu của quần thể.

Cho $[x^o] = [x_o]$, $f^o = f([x_o])$ và xây dựng theo cách ngẫu nhiên mảng các cá thể ban đầu $[X_1]$ chứa N_p trạng thái(trong vùng cho phép báo bởi biên trên [u] và biên dưới [l]) bao gồm cả trạng thái ban đầu $[x_o]$ bằng ccáh đặt:

$$[X_1(1)] = [x_o],\ [X_1(k)] = [l] + rand.\times([u] - [l])\ k = 2,..,N_p \qquad (1)$$

Sau đó mã hoá mỗi số của mảng quần thể này thnàh một chuỗi nhị phân bằng:

$$P_1\left(n, 1+\sum_{i=1}^{m-1} N_{bi} : \sum_{i=1}^{m} N_{bi}\right) = \text{biểu diễn nhị phân của } X_1(n,m) \text{ với } N_{bm} \text{ bit}$$

$$= \left(2^{N_{bm}} - 1\right)\frac{X_1(n,m) - l(m)}{u(m) - l(m)} \qquad (2)$$

với $n = 1,...,N_p$ và $m = 1,...,N$

sao cho toàn thể mảng quần thể trở thành mảng mà mỗi hàng là một nhiễn sắc thể được biểu diễn bằng chuỗi nhị phân $\sum_{i=1}^{N} N_{bi}$ bit.

• Lặp từ k = 1 đến k_{max}:

∗ giải mã mỗi số của mảng thành số thập phân bằng:

$$X_k(n,m) = \text{biểu diễn thập phân của } P_1\left(n, 1+\sum_{i=1}^{m-1} N_{bi} : \sum_{i=1}^{m} N_{bi}\right) \text{ với } N_{bm} \text{ bit}$$

$$= P_k(n,.)\frac{u(m) - l(m)}{\left(2^{N_{bm}} - 1\right)} + l(m) \qquad (3)$$

$n = 1,...,N$; $m = 1,...,N$

và tính giá trị f(n) của hàm đối với mỗi hàng $X_k(n, :) = [x(n)]$ tương ứng với mỗi nhiễm sắc thể và tìm cực tiểu $f_{min} = f(n_b)$ tương ứng với $X_k(n, :) = [x(n_b)]$

∗ nếu $f_{min} = f(n_b) < f^o$ thì đặt $f^o = f(n_b)$ và $[x^o] = [x(n_b)]$

∗ biến đổi giá trị của hàm thành giá trị thích hợp bằng:

$$f_1(n) = Max_{n=1}^{N_p}\{f(n)\} - f(n) \qquad (4)$$

∗ nếu $Max_{n=1}^{N_p}\{f(n)\} \approx 0$, kết thúc quá trình và $[x^o]$là giá trị tốt nhất. Nếu không, để tạo nhiều nhiễn sắc thể hơn quanh điểm tốt nhất $[x(n_b)]$ cho thế hệ sau, ta dùng quy tắc chọn lọc:

$$[x(n)] = [x(n)] + \eta\frac{f_1(n_b) - f_1(n)}{f_1(n_b)}\{[x(n_b)] - [x(n)]\} \qquad (5)$$

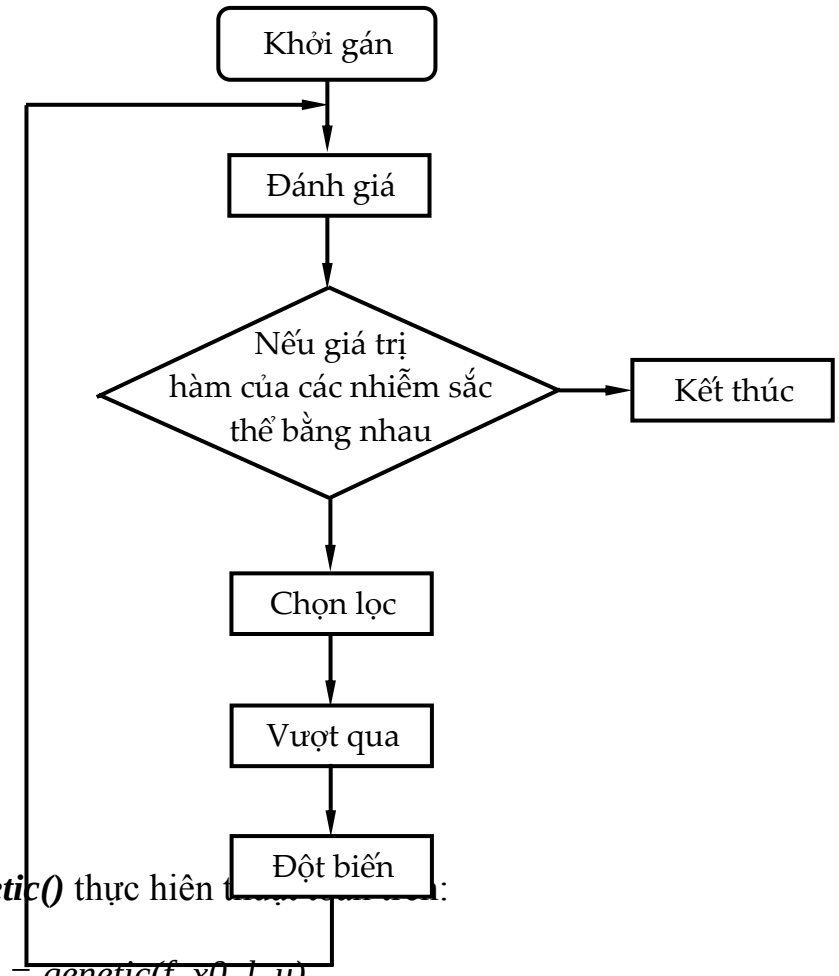
để có được quần thể mới $[X_{k+1}]$ có $X_{k+1}(n, :) = [x(n)]$ và mã hoá nó để xây dựng mảng P_{k+1} mới theo (2)

∗ xáo trộn chỉ số hàng của mảng P_{k+1}

∗ với xác suất tồn tại P_c, thay đổi phần cuối bắt đầu từ vài bit ngẫu nhiên của các số trong 2 cặp nhiễm sắc thể ngẫu nhiên(hàng cả P_{k+1})với các nhiễm sắc thể khác để có ma trận P'_{k+1}

∗ với xác suất đột biến P_m, nghịch đảo một bít ngẫu nhiên của mỗi hàng biểu diễn bởi nhiễm sắc thể (hàng của P'_{k+1}) để tạo ra mảng P_{k+1}

Lưu đồ thuật toán như sau:



Ta xây dựng hàm ***genetic()*** thực hiện thuật toán trên:

```
function [xo, fo] = genetic(f, x0, l, u)
% Thuat toan Genetic Algorithm tim cuc tieu cua ham f(x) tg doan l <= x <= u
N = length(x0);
kmax = 100; %so lan lap(the he)
eta = 1;%ti le hoc(0 < eta < 1)
Pm = 0.01; %xac suat dot bien
Pc = 0.5; end %xac suat ton tai
Nb = 8*ones(1, N);
Np = 10;  %so nhiem sac the
%khoi gan
NNb = sum(Nb);
xo = x0(:)';
l = l(:)';
```

```
u = u(:)';
fo = feval(f, xo);
X(1, :) = xo;
for n = 2:Np
   X(n, :) = l + rand(size(x0)).*(u - l); %Pt.(1)
end
P = genencode(X, Nb, l, u); %Pt.(2)
for k = 1:kmax
   X = gendecode(P, Nb, l, u); %Pt.(3)
   for n = 1:Np
      fX(n) = feval(f, X(n,:));
   end
   [fxb, nb] = min(fX);
   if fxb < fo
      fo = fxb;
      xo = X(nb, :);
   end
   fX1 = max(fX) - fX; %Pt.(4)
   fXm = fX1(nb);
   if fXm < eps
      return;
   end %ket thuc neu tat ca cac nhiem sac the nhu nhau
  %Chon loc the h tiep theo
   for n = 1:Np
      X(n, :) = X(n, :) + eta*(fXm - fX1(n))/fXm*(X(nb, :) - X(n, :)); %Pt.(5)
   end
   P = genencode(X,Nb,l,u);
   is = shuffle([1:Np]);
   for n = 1:2:Np - 1
      if rand < Pc
         P(is(n:n + 1), :) = crossover(P(is(n:n + 1), :), Nb);
      end
   end
   %Dot bien
   P = mutation(P, Nb, Pm);
end

function X = gendecode(P, Nb, l, u)
% giai ma
Np = size(P, 1);
N = length(Nb);
for n = 1:Np
```

```
    b2 = 0;
    for m = 1:N
        b1 = b2 + 1;
        b2 = b1 + Nb(m) - 1; %Pt.(3)
        X(n, m) = bin2dec(P(n,b1:b2))*(u(m) - l(m))/(2^Nb(m) - 1) + l(m);
    end
end

function P = genencode(X, Nb, l, u)
Np = size(X,1);
N = length(Nb);
for n = 1:Np
    b2 = 0;
    for m = 1:N
        b1 = b2 + 1;
        b2 = b2 + Nb(m);
        Xnm = (2^Nb(m)- 1)*(X(n, m) - l(m))/(u(m) - l(m)); %Pt.(2)
        P(n, b1:b2) = dec2bin(Xnm, Nb(m));
    end
end
function chrms = crossover(chrms2, Nb)
Nbb = length(Nb);
b2 = 0;
for m = 1:Nbb
    b1 = b2 + 1;
    bi = b1 + mod(floor(rand*Nb(m)), Nb(m));
    b2 = b2 + Nb(m);
    tmp = chrms2(1, bi:b2);
    chrms(1, bi:b2) = chrms(2, bi:b2);
    chrms(2, bi:b2) = tmp;
end

function P = mutation(P, Nb, Pm)
Nbb = length(Nb);
for n = 1:size(P,1)
    b2 = 0;
    for m = 1:Nbb
        if rand < Pm
            b1 = b2 + 1;
            bi = b1 + mod(floor(rand*Nb(m)),Nb(m));
            b2 = b2 + Nb(m);
            P(n,bi) = ~P(n,bi);
```

```
        end
    end
end

function is = shuffle(is)
N = length(is);
for n = N:-1:2
    in = ceil(rand*(n - 1));
    tmp = is(in);
    is(in) = is(n);
    is(n) = tmp;
end
```

Để tìm cực tiểu của hàm ta dùng chương trình ***ctgenetic.m***:

```
clear all, clc
f = inline('x(1).^2 + 2*x(2).^2');
l = [-5  -5 ];
u = [5  5]; %bien duoi/tren
x0 = [0   0];
[xmin, fmin] = genetic(f, x0, l, u)
```

§10. THUẬT TOÁN FIBONACCI

Trong thuật toán tỉ lệ vàng, hai lần tính giá trị của hàm được thực hiện tại lần lặp đầu tiên và sau đó chỉ tính giá trị hàm một lần trong các lần lặp tiếp theo. Giá trị của r là hằng số trong mỗi đoạn con và việc tìm điểm cực tiểu kết thúc tại đoạn con thứ k có $|a_k - b_k| < \delta$ hay $|f(b_k - f(a_k)| < \varepsilon$. Phương pháp tìm theo thuật toán Fibonacci khác phương pháp tỉ lệ vàng ở chỗ r không phải là hằng số trên mỗi đoạn con. Ngoài ra số đoạn con (số bước lặp) được xác định trước. Thuật toán tìm Fibonacci dựa trên dãy số Fibonacci được xác định bằng phương trình:

$f_o = 0$

$f_1 = 1$

$f_n = f_{n-1} + f_{n-2}$ với n = 2,3,...

Như vậy dãy số Fibonacci là: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55,...

Giả sử ta có hàm f(x) có cực tiểu trên đoạn [a, b]. Như ở trong phương pháp tỉ lệ vàng, $0.5 < r_o < 1$ được chọn sao cho cả hai điểm bên trong c_o và d_o sẽ được dùng trong đoạn con tiếp theo và như vậy chỉ cần tính giá trị của hàm một lần. Nếu $f(c_o) < f(d_o)$ thì cực tiểu nằm trong đoạn $[a_o, d_o]$ và ta thay $a_1 = a_o$ và $b_1 = d_o$ và tiếp tục tìm trong khoảng mới $[a_1, b_1] = [a_o, d_o]$. Nếu $f(c_o) > f(d_o)$ thì cực tiểu nằm trong đoạn $[c_o, b_o]$ và ta thay $a_1 = c_o$ và $b_1 = b_o$ và tiếp tục tìm trong khoảng mới $[a_1, b_1] = [c_o, b_o]$ như hình vẽ.

Nếu $f(c_o) < f(d_o)$ và chỉ muốn tính giá trị của hàm một lần trong đoạn $[a_o, b_o]$ ta sẽ chọn r_1 ($0.5 < r_1 < 1$) trong đoạn con $[a_1, b_1] = [a_o, b_o]$. Ta đã kí hiệu $b_1 = d_o$ và do $c_o \in [a_o, d_o]$ nên ta có:

$$d_o - c_o = b_1 - d_1 \qquad (1)$$

Tỉ số r_o được chọn sao cho $d_o - a_o = r_o(b_o - a_o)$ và $c_o - a_o = (1 - r_o)(b_o - a_o)$ và thay thế:

$$d_o - c_o = (d_o - a_o) - (c_o - a_o)$$

$$d_o - c_o = r_o(b_o - a_o) - (1 - r_o)(b_o - a_o)$$

$$d_o - c_o = (2r_o - 1)(b_o - a_o) \qquad (2)$$

và r_1 được chọn sao cho:

$$b_1 - d_1 = (1 - r_1)(b_1 - a_1) \qquad (3)$$

Thay (2) và (3) vào (1) ta có:

$$(2r_o - 1)(b_o - a_o) = (1 - r_1)(b_1 - a_1) \qquad (4)$$

Như vậy đoạn [a, b] bị co ngắn bằng hệ số r_o và $(b_1 - a_1) = r_o(b_o - a_o)$ và:

$$(2r_o - 1)(b_o - a_o) = (1 - r_1)r_o(b_o - a_o) \qquad (5)$$

Rút gọn ta có:

$$(2r_o - 1) = (1 - r_1)r_o \qquad (6)$$

Từ (6) ta tính được r_1:

$$r_1 = \frac{1 - r_o}{r_o} \qquad (7)$$

Trong (7), thay $r_o = \frac{f_{n-1}}{f_n}$ ta có:

$$r_1 = \frac{1 - \frac{f_{n-1}}{f_n}}{\frac{f_{n-1}}{f_n}} = \frac{f_n - f_{n-1}}{f_{n-1}} = \frac{f_{n-2}}{f_{n-1}}$$

Ta rút ra rằng thuật toán tìm Fibonacci có thể bắt đầu bằng:

$$r_o = \frac{f_{n-1}}{f_n} \qquad r_1 = \frac{f_{n-2}}{f_{n-1}}$$

và:

$$r_k = \frac{f_{n-1-k}}{f_{n-k}}, \; k = 1, 2, ..., n - 3$$

Bước cuối cùng là:

$$r_{n-3} = \frac{f_2}{f_3} = \frac{1}{2}$$

Thuật toán tìm Fibonacci gồm (n - 2) lần tính. Đoạn con thứ (k+1) có được bằng cách giảm độ dài của đoạn thứ k bằng hệ số $r_k = \frac{f_{n-1-k}}{f_{n-k}}$. Sau (n - 2) lần tính, độ dài của bước cuối cùng là:

$$\frac{f_{n-1}}{f_n}\frac{f_{n-2}}{f_{n-1}}\frac{f_{n-3}}{f_{n-2}}\cdots\frac{f_3}{f_4}\frac{f_3}{f_3}(b_o - a_o) = \frac{f_2}{f_n}(b_o - a_o) = \frac{1}{f_n}(b_o - a_o)$$

Nếu sai số cho trước là ε, nghĩa là $\frac{1}{f_n}(b_o - a_o) < \varepsilon$ và cần dùng n lần lặp với n là số nguyên nhỏ nhất sao cho:

$$f_n > \frac{b_o - a_o}{\varepsilon} \tag{8}$$

Các điểm bên trong c_k và d_k được xác định bằng:

$$c_k = a_k + \left(1 + \frac{f_{n-1-k}}{f_{n-k}}\right)(b_k - a_k) \tag{9}$$

$$d_k = a_k + 1 + \frac{f_{n-1-k}}{f_{n-k}}(b_k - a_k) \tag{10}$$

Ta xây dựng hàm ***fibonacci()*** để thực hiện thuật toán trên:

```
function [x, y] = fibonacci(f, a, b, n)
% Phuong phap Fibonacci de tim cuc tieu cua
% ham f trong (a, b) voi n buoc tinh
fn2 = 1;
fn1 = 1;
fn = fn1 + fn2;
for i = 3:n
   fn2 = fn1;
   fn1 = fn;
   fn = fn1 + fn2;
end
l = (b - a)*fn2/fn;
x1 = a + l;
x2 = b - l;
f1 = feval(f, x1);
f2 = feval(f,x2);
fibn = fn;
ll1 = b - a;
for j = 3:n
   llj = ll1*fn2/fibn;
   fn = fn1;
   fn1 = fn2;
```

```
    fn2 = fn - fn1;
    if f2 > f1
      b = x2;
      l = (b - a)*fn2/fn;
      x2 = x1;
      x1 = a + l;
      f2 = f1;
      f1 = feval(f, x1);
    else
      a = x1;
      l = (b - a)*fn2/fn;
      x1 = x2;
      x2 = b - l;
      f1 = f2;
      f2 = feval(f, x2);
    end
  end
  x = x1; y = f1;
```

Để tìm cực tiểu của hàm trong đoạn (a, b) ta dùng chương trình ***ctfibonacci.m*:**

```
clear all, clc
f = inline('1.6*x^2 - 3*x + 2');
a = -0.;
b = 1;
n = 50;
[x, y] = fibonacci(f, a, b, n)
```

CHƯƠNG 9: PHƯƠNG TRÌNH VI PHÂN ĐẠO HÀM RIÊNG

§1. KHÁI NIỆM CHUNG

Phương trình vi phân đạo hàm riêng(PDE) là một lớp các phương trình vi phân có số biến độc lập lớn hơn 1. Trong chương này ta sẽ khảo sát các phương trình vi phân đạo hàm riêng cấp 2 với hai biến độc lập x và y, có dạng tổng quát:

$$A(x,y)\frac{\partial^2 u}{\partial x^2}+B(x,y)\frac{\partial^2 u}{\partial x\partial y}+C(x,y)\frac{\partial^2 u}{\partial y^2}=f\left(x,y,u,\frac{\partial u}{\partial x},\frac{\partial u}{\partial y}\right) \qquad (1)$$

với $x_o \le x \le x_f$, $y_o \le y \le y_f$ và các điều kiện biên:

$$u(x,y_o)=b_{yo}(x) \qquad u(x,y_f)=b_{yf}(x)$$
$$u(x_o,y)=b_{xo}(y) \qquad u(x_f,y)=b_{xf}(y) \qquad (2)$$

Các PDE được phân thành 3 loại:

PDE elliptic: $B^2-4AC<0$

PDE parabolic: $B^2-4AC=0$

PDE hyperbolic: $B^2-4AC>0$

Các phương trình này gắn một cách tương ứng với trạng thái cân bằng, trạng thái truyền nhiệt, hệ thống dao động

§2. PHƯƠNG TRÌNH ELLIPTIC

Ta xét phương trình Helmholz:

$$\nabla^2 u(x,y)+g(x,y)=\frac{\partial^2 u(x,y)}{\partial x^2}+\frac{\partial^2 u(x,y)}{\partial y^2}+g(x,y)u(x,y)=f(x,y) \qquad (1)$$

trên miền $D=\{(x,y): x_o \le x \le x_f, y_o \le y \le y_f\}$ với điều kiện biên dạng:

$$u(x,y_o)=b_{yo}(x) \qquad u(x,y_f)=b_{yf}(x)$$
$$u(x_o,y)=b_{xo}(y) \qquad u(x_f,y)=b_{xf}(y) \qquad (2)$$

Phương trình (1) được gọi là phương trình Poisson nếu g(x, y) = 0 và gọi là phương trình Laplace nếu g(x, y) = 0 và f(x, y) = 0. Để dùng phương pháp sai phân ta chia miền thành M_x đoạn, mỗi đoạn dài $\Delta x = (x_f - x_o)/M_x$ dọc theo trục x và thành M_y đoạn, mỗi đoạn dài $\Delta y = (y_f - y_o)/M_y$ dọc theo trục y và thay đạo hàm bậc 2 bằng xấp xỉ 3 điểm:

$$\left.\frac{\partial^2 u(x,y)}{\partial x^2}\right|_{x_j,y_i} \cong \frac{u_{i,j+1}-2u_{i,j}+u_{i,j-1}}{\Delta x^2} \text{ với } x_j = x_o + j\Delta x,\ y_j = y_o + j\Delta y \qquad (3.a)$$

$$\left.\frac{\partial^2 u(x,y)}{\partial y^2}\right|_{x_j,y_i} \cong \frac{u_{i+1,j}-2u_{i,j}+u_{i-1,j}}{\Delta x^2} \text{ với } u_{i,j} = u(x_j, y_i) \qquad (3.b)$$

Như vậy tại mỗi điểm bên trong (x_j, x_i) với $1 \le i \le M_y - 1$ và $1 \le j \le M_x$ - ậit nhận được phương trình sai phân:

$$\frac{u_{i,j+1}-2u_{i,j}+u_{i,j-1}}{\Delta x^2}+\frac{u_{i+1,j}-2u_{i,j}+u_{i-1,j}}{\Delta y^2}=g_{i,j}u_{i,j}=f_{i,j} \qquad (4)$$

Trong đó:

$u_{i,j} = u(x_j, y_i) f_{i,j} = f(x_j, y_i)$ $\qquad g_{i,j} = g(x_j, y_i)$

Các phương trình này sắp xếp lại theo cách nào đó thành hệ phương trình với (M_y - 1)(M_x - 1) biến $\left\{u_{1,1},u_{1,2},...,u_{1,M_x-1},u_{2,1},...,u_{2,M_x-1},...,u_{M_y-1,2},...,u_{M_y-1,M_x-1}\right\}$. Để dễ dàng ta viết lại phương trình và điều kiện biên dưới dạng:

$$u_{i,j}=r_y(u_{i,j+1}+u_{i,j-1})+r_x(u_{i+1,j}+u_{i-1,j})+r_{xy}(g_{i,j}u_{i,j}-f_{i,j}) \qquad (5a)$$

$$u_{i,o}=b_{xo}(y_i) \quad u_{i,M_x}=b_{xf}(y_i) \quad u_{o,j}=b_{yo}(x_j) \quad u_{M_y,j}=b_{yf}(x_j) \qquad (5b)$$

Trong đó:

$$r_y=\frac{\Delta y^2}{2(\Delta x^2+\Delta y^2)} \qquad r_x=\frac{\Delta x^2}{2(\Delta x^2+\Delta y^2)} \qquad r_{xy}=\frac{\Delta x^2\Delta y^2}{2(\Delta x^2+\Delta y^2)} \qquad (6)$$

Bây giờ ta khảo sát tiếp các dạng điều kiên biên. Các bài toán PDE có 2 loại điều kiện biên: điều kiên biên Neumann và điều kiên biên Dirichlet. Điều kiện biên Neumann mô tả bằng:

$$\left.\frac{\partial u(x,y)}{\partial x}\right|_{x=x_o}=b'_{x_o}(y) \qquad (7)$$

Thay đạo hàm bậc 1 ở biên trái ($x = x_o$) bằng xấp xỉ 3 điểm:

$$\frac{u_{i,1}-u_{i,-1}}{2\Delta x}=b'_{x_o}(y_i) \qquad u_{i,-1}\approx u_{i,1}-2b'_{x_o}(y_i)\Delta x \quad i = 1, 2,..., M_y\text{-}1 \qquad (8)$$

Thay thế ràng buộc này vào (5a) ở các điểm biên ta có:

$$\begin{aligned}u_{i,0}&=r_y(u_{i,1}+u_{i,-1})+r_x(u_{i+1,0}+u_{i-1,0})+r_{xy}(g_{i,0}u_{i,0}-f_{i,0})\\&=r_y\left[u_{i,1}+u_{i,1}-2b'_{x_0}(y_i)\Delta x\right]+r_x(u_{i+1,0}+u_{i-1,0})+r_{xy}(g_{i,0}u_{i,0}-f_{i,0})\\&=2r_yu_{i,1}+r_x(u_{i+1,0}+u_{i-1,0})+r_{xy}\left[g_{i,0}u_{i,0}-f_{i,0}-2b'_{x_0}(y_i)\Delta x\right] \qquad (9)\end{aligned}$$

Nếu điều kiên biên trên biên dưới ($y = y_o$) cũng là kiểu Neumann ta sẽ viết các phương trình tương tự với $j = 1, 2,...,M_x$-1:

$$u_{0,j}=2r_xu_{1,j}+r_y(u_{0,j+1}+u_{0,j-1})+r_{xy}\left[g_{0,j}u_{0,j}-f_{0,j}-2b'_{y_0}(x_j)\Delta y\right] \qquad (10)$$

và bổ sung cho góc dưới trái(x_o, y_o):

$$u_{0,0}=2(r_yu_{0,1}+r_xu_{1,0})+r_{xy}\left[g_{0,0}u_{0,0}-f_{0,0}-2\frac{b'_{x_0}(y_0)}{\Delta x}+2\frac{b'_{y_0}(x_0)}{\Delta y}\right] \qquad (11)$$

Điều kiện biên Dirichlet cho giá trị hàm trên biên nên có thể thay trực tiếp vào phương trình. Ta có thể lấy giá trị trung bình của các giá trị biên làm giá trị đầu của $u_{i,j}$. Ta xây dựng hàm ***poisson()*** để thực hiện thuật toán này:

```
function [u, x, y] = poisson(f, g, bx0, bxf, by0, byf, D, Mx, My, tol, maxiter)
% giai a(u_xx + u_yy + g(x,y)u = f(x,y)
```

```
% tren mien D = [x0, xf, y0, yf] = {(x,y) |x0 <= x <= xf, y0 <= y <= yf}
% voi dieu kien bien:
% u(x0,y) = bx0(y), u(xf,y) = bxf(y)
% u(x,y0) = by0(x), u(x,yf) = byf(x)
% Mx - so doan con tren truc x
% My - so doan con tren truc y
% tol : sai so cho phep
% maxiter: so lan lap
x0 = D(1);
xf = D(2);
y0 = D(3);
yf = D(4);
dx = (xf - x0)/Mx;
x = x0 + [0:Mx]*dx;
dy = (yf - y0)/My;
y = y0 + [0:My]'*dy;
Mx1 = Mx + 1;
My1 = My + 1;
%dieu kien bien
for m = 1:My1
   u(m, [1 Mx1]) = [bx0(y(m)) bxf(y(m))];
end
for n = 1:Mx1
   u([1 My1], n) = [by0(x(n)); byf(x(n))];
end
sumbv = sum(sum([u(2:My, [1 Mx1]) u([1 My1], 2:Mx)']));
u(2:My,  2:Mx) = sumbv/(2*(Mx + My - 2));
for i = 1:My
   for j = 1:Mx
      F(i, j) = f(x(j), y(i));
      G(i, j) = g(x(j), y(i));
   end
end
dx2 = dx*dx;
dy2 = dy*dy;
dxy2 = 2*(dx2 + dy2);
rx = dx2/dxy2;
ry = dy2/dxy2;
rxy = rx*dy2;
for itr = 1:maxiter
   for j = 2:Mx
      for i = 2:My
```

```
            u(i, j) = ry*(u(i, j + 1)+u(i,j - 1)) + rx*(u(i + 1,j)+u(i - 1,j))...
                  + rxy*(G(i,j)*u(i,j)- F(i,j)); %Pt.(5a)
        end
    end
    if itr > 1 & max(max(abs(u - u0))) < tol
        break;
    end
    u0 = u;
end
```

Ta giải phương trình Laplace:

$$\nabla^2 u(x,y) = \frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2} = 0 \qquad \text{(vd1)}$$

trong miền $0 \le x \le 4$, $0 \le y \le 4$ với điều kiện biên:

$$u(0,y) = e^y - \cos y \qquad u(4,y) = e^x \cos 4 - e^4 \cos y \qquad \text{(vd2)}$$

$$u(x,0) = \cos x - e^x \qquad u(x,4) = e^4 \cos x - e^x \cos 4 \qquad \text{(vd3)}$$

Ta muốn nhận được u(x, y), mô tả phân bố nhiệt độ trên một tấm vuông mỗi cạnh dài 4 đơn vị. Ta dùng chương trình ***ctpoisson.m*** gọi hàm ***poisson()*** để giải bài toán này.

```
clear all, clc
f = inline('0','x','y');
g = inline('0','x','y');
x0 = 0;
xf = 4;
Mx = 20;
y0 = 0;
yf = 4;
My = 20;
bx0 = inline('exp(y) - cos(y)','y'); %(vd.2a)
bxf = inline('exp(y)*cos(4) - exp(4)*cos(y)','y'); %(vd.2b)
by0 = inline('cos(x) - exp(x)','x'); %(vd.3a)
byf = inline('exp(4)*cos(x) - exp(x)*cos(4)','x'); %(vd.3b)
D = [x0 xf y0 yf];
maxiter = 500;
tol = 1e-4;
[U, x, y] = poisson(f, g, bx0, bxf, by0, byf, D, Mx, My, tol, maxiter);
clf
mesh(x, y, U)
axis([0 4 0 4 -100 100])
```

§3. PHƯƠNG TRÌNH PARABOLIC

1. Dạng phương trình: Một phương trình vi phân đạo hàm riêng dạng parabolic là phương trình mô tả sự phân bố nhiệt độ ở điểm x tại thời điểm t của một thanh:

$$A\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t} \qquad (1)$$

Để phương trình có thể giải được ta phải cho điều kiện biên u(0, t) = $b_0(t)$, $u(x_f, t) = b_{x_f}(t)$ và điều kiện đầu u(x, 0) = $i_0(x)$

2. Phương pháp Euler tiến tường minh: Để áp dụng phương pháp sai phân hữu hạn, ta chia miên không gian [0, x_f] thành M đoạn, mỗi đoạn dài $\Delta x = x_f / M$ và chia thời gian T thành N phần, mỗi phần là Δt = T/N. Sau đó ta thay đạo hàm bậc 2 ở vế trái và đạo hàm bậc ở vế phải của (1) bằng các xấp xỉ 3 điểm và nhận được:

$$A\frac{u_{i+1}^k - 2u_i^k + u_{i-1}^k}{\Delta x^2} = \frac{u_i^{k+1} - u_i^k}{\Delta t} \qquad (2)$$

Công thức này có thể gói gọn vào thuật toán sau, gọi là thuật toán Eulẻ tiến tường minh:

$$u_i^{k+1} = r(u_{i+1}^k + u_{i-1}^k) + (1-2r)u_i^k \qquad r = A\frac{\Delta t}{\Delta x^2} \qquad (3)$$

i = 1, 2,...,M-1

Để tìm điều kiện ổn định của thuât toán, ta thay nghiệm thử:

$$u_i^k = \lambda^k e^{\frac{ji\pi}{P}} \qquad (4)$$

với P là số nguyên khác zero vào phương trình (3) và có:

$$\lambda = r\left(e^{\frac{j\pi}{P}} + e^{-\frac{j\pi}{P}}\right) + (1-2r) = 1 - 2r\left[1 - \cos\frac{\pi}{P}\right] \qquad (5)$$

Do ta phải có |λ|≤ 1 với bài toán không có nguồn nên điều kiện ổn định là:

$$r = A\frac{\Delta t}{\Delta x^2} \le \frac{1}{2} \qquad (6)$$

Ta xây dựng hàm ***fwdeuler()*** để thực hiện thuật toán trên

```
function [u, x, t] = fwdeuler(a, xf, T, it0, bx0, bxf, M, N)
%giai au_xx = u_t voi  0 <= x <= xf, 0 <= t <= T
% dieu kien dau: u(x,0) = it0(x)
ieu kien bien: u(0,t) = bx0(t), u(xf,t) = bxf(t)
% M - so doan con theo x
% N - so diem theo t
dx = xf/M;
x = [0:M]'*dx;
dt = T/N;
t = [0:N]*dt;
for i = 1:M + 1
   u(i,1) = it0(x(i));
```

```
end
for n = 1:N + 1
   u([1 M + 1], n) = [bx0(t(n)); bxf(t(n))];
end
r = a*dt/dx/dx
r1 = 1 - 2*r;
for k = 1:N
   for i = 2:M
      u(i, k+1) = r*(u(i + 1, k) + u(i-1, k)) + r1*u(i, k); %Pt.(3)
   end
end
```

3. Phương pháp Euler lùi ẩn: Ta khảo sát một thuật toán khác gọi là thuật toán Euler lùi, ẩn sinh ra do thay thế lùi xấp xỉ đạo hàm đối với đạo hàm bậc 1 trên vế phải của (1):

$$A\frac{u_{i+1}^k - 2u_i^k + u_{i-1}^k}{\Delta x^2} = \frac{u_i^k - u_i^{k-1}}{\Delta t} \tag{7}$$

$$-ru_{i-1}^k + (1+2r)u_i^k - ru_{i+1}^k = u_i^{k-1} \quad r = A\frac{\Delta t}{\Delta x^2} \tag{8}$$

Nếu các giá trị u_0^k và u_M^k ở cả hai đầu đã cho trước từ điều kiện biên kiểu Dirichlet nên phương trình (8) đưa tới hệ phương trình:

$$\begin{bmatrix} 1+2r & -r & 0 & 0 & 0 & 0 \\ -r & 1+2r & -r & 0 & 0 & 0 \\ 0 & -r & 1+2r & -r & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1+2r & -r \\ 0 & 0 & 0 & \cdots & -r & 1+2r \end{bmatrix} \begin{bmatrix} u_1^k \\ u_2^k \\ u_3^k \\ \vdots \\ u_{M-2}^k \\ u_{M-1}^k \end{bmatrix} = \begin{bmatrix} u_1^{k-1} + ru_0^k \\ u_2^{k-1} \\ u_3^{k-1} \\ \vdots \\ u_{M-2}^{k-1} \\ u_{M-1}^{k-1} + ru_M^k \end{bmatrix} \tag{9}$$

Điều kiện biên Neumann $\left.\frac{\partial u}{\partial x}\right|_{x=0} = b_0'(t)$ được đưa vào phương trình bằng cách xấp xỉ:

$$\frac{u_1^k - u_{-1}^k}{2\Delta x} = b_0'(k) \tag{10}$$

và ghép nó với phương trình có ẩn u_0^k:

$$-ru_{-1}^k + (1+2r)u_0^k - ru_1^k = u_0^{k-1} \tag{11}$$

để có được phương trình:

$$(1+2r)u_0^k - 2ru_1^k = u_0^{k-1} - 2rb_0'(k)\Delta x \tag{12}$$

Kết quả ta có được hệ phương trình:

$$\begin{bmatrix} 1+2r & -r & 0 & 0 & \cdots & 0 & 0 \\ -r & 1+2r & -r & 0 & \cdots & 0 & 0 \\ 0 & -r & 1+2r & -r & \cdots & 0 & 0 \\ 0 & 0 & -r & 1+2r & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & -r & 0 \\ 0 & 0 & 0 & 0 & \cdots & 1+2r & -r \\ 0 & 0 & 0 & 0 & \cdots & -r & 1+2r \end{bmatrix} \begin{bmatrix} u_0^k \\ u_1^k \\ u_2^k \\ u_3^k \\ \vdots \\ u_0^k \\ u_0^k \end{bmatrix} = \begin{bmatrix} u_0^k - 2rb_0'(k)\Delta x \\ u_1^{k-1} \\ u_2^{k-1} \\ u_3^{k-1} \\ \vdots \\ u_{M-2}^{k-1} \\ u_{M-1}^{k-1} + ru_M^k \end{bmatrix} \quad (13$$

Điểu kiện ổn định của nghiệm là:

$$-re^{-\frac{j\pi}{P}} + (1+2r) - re^{\frac{j\pi}{P}} = \frac{1}{\lambda}$$

hay: $$\lambda = \frac{1}{1+2r\left[1-\cos\frac{\pi}{P}\right]} \qquad |\lambda| \le 1 \qquad (14)$$

Ta xây dựng hàm ***backeuler()*** để thực hiện thuật toán này:

```
function [u, x, t] = backeuler(a, xf, T, it0, bx0, bxf, M, N)
%Giai au_xx = u_t voi 0 <= x <= xf, 0 <= t <= T
% Dieu kien dau: u(x,0) = it0(x)
% ieu kien bien: u(0,t) = bx0(t), u(xf,t) = bxf(t)
% M - so khoang con tren truc x
% N - so khoang theo t
dx = xf/M;
x = [0:M]'*dx;
dt = T/N;
t = [0:N]*dt;
for i = 1:M + 1
   u(i, 1) = it0(x(i));
end
for n = 1:N + 1
   u([1 M + 1], n) = [bx0(t(n)); bxf(t(n))];
end
r = a*dt/dx/dx;
r2 = 1 + 2*r;
for i = 1:M - 1
   A(i, i) = r2; %Pt.(9)
   if i > 1
      A(i - 1, i) = -r;
      A(i, i - 1) = -r; end
```

```
end
for k = 2:N + 1
   b = [r*u(1, k); zeros(M - 3, 1); r*u(M + 1, k)] + u(2:M, k - 1); %Pt.(9)
   u(2:M, k) = trid(A, b);
end
```

4. Phương pháp Crank - Nicholson: Trong (7), xấp xỉ đạo hàm ở vế trái lấy ở thời điểm k, trong khi xấp xỉ đạo hàm ở vế phải. Để cải thiện, ta lấy đạo hàm ở vế trái là trong bình của xấp xỉ đạo hàm tại hai điểm là k và k+1 và có:

$$\frac{A}{2}\left(\frac{u_{i+1}^{k+1}-2u_i^{k+1}+u_{i-1}^{k+1}}{\Delta x^2}+\frac{u_{i+1}^{k}-2u_i^{k}+u_{i-1}^{k}}{\Delta x^2}\right)=\frac{u_i^{k+1}-u_i^k}{\Delta t} \qquad (15)$$

và nhận được phương pháp Crank - Nicholson:

$$-ru_{i+1}^{k+1}+(1+2r)u_i^{k+1}-ru_{i-1}^{k+1}=ru_{i+1}^{k}+(1-2r)u_i^{k}+ru_{i-1}^{k} \qquad r=A\frac{\Delta t}{\Delta x^2} \qquad (16)$$

Với điều kiện biên Dirichlet tại x_0 và điều kiện biên Neumann tại x_M ta có hệ phương trình:

$$\begin{bmatrix} 2(1+r) & -r & 0 & 0 & 0 & 0 \\ -r & 2(1+r) & -r & 0 & 0 & 0 \\ 0 & -r & 2(1+r) & -r & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 2(1+r) & -r \\ 0 & 0 & 0 & \cdots & -r & 2(1+r) \end{bmatrix}\begin{bmatrix} u_1^{k+1} \\ u_2^{k+1} \\ u_3^{k+1} \\ \vdots \\ u_{M-1}^{k+1} \\ u_M^{k+1} \end{bmatrix}$$

$$=\begin{bmatrix} 2(1-r) & r & 0 & 0 & 0 & 0 \\ r & 2(1-r) & r & 0 & 0 & 0 \\ 0 & r & 2(1-r) & r & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 2(1-r) & r \\ 0 & 0 & 0 & \cdots & r & 2(1-r) \end{bmatrix}\begin{bmatrix} u_1^{k} \\ u_2^{k} \\ u_3^{k} \\ \vdots \\ u_{M-1}^{k} \\ u_M^{k} \end{bmatrix}$$

$$+\begin{bmatrix} r(u_0^{k+1}+u_0^k) \\ 0 \\ 0 \\ \vdots \\ 0 \\ 2r\left[b'_M(k+1)+b'_M(k)\right] \end{bmatrix} \qquad (17)$$

Điều kiện ổn định được xác định bằng:

$$2\lambda\left[1+r\left(1-\cos\frac{\pi}{P}\right)\right]=2\left[1-r\left(1-\cos\frac{\pi}{P}\right)\right]$$

hay: $$\lambda=\frac{1-r\left[1-\cos\frac{\pi}{P}\right]}{1+r\left[1-\cos\frac{\pi}{P}\right]} \qquad |\lambda|\le 1 \qquad (18)$$

Ta xây dựng hàm ***cranknicholson()*** để thực hiện thuật toán trên:

```
function [u, x, t] = cranknicholson(a, xf, T, it0, bx0, bxf, M, N)
%Giai au_xx = u_t voi 0 <= x <= xf, 0 <= t <= T
% Dieu kien dau: u(x,0) = it0(x)
% Dieu kien bien: u(0, t) = bx0(t), u(xf, t) = bxf(t)
% M - so khoang con tren truc x
% N - so khoang theo t
dx = xf/M;
x = [0:M]'*dx;
dt = T/N;
t = [0:N]*dt;
for i = 1:M + 1
   u(i, 1) = it0(x(i));
end
for n = 1:N + 1
   u([1 M + 1], n) = [bx0(t(n)); bxf(t(n))];
end
r = a*dt/dx/dx;
r1 = 2*(1 - r);
r2 = 2*(1 + r);
for i = 1:M - 1
   A(i, i) = r2; %Pt.(17)
   if i > 1
      A(i - 1, i) = -r;
      A(i, i - 1) = -r;
   end
end
for k = 2:N + 1
   b = [r*u(1, k); zeros(M - 3, 1); r*u(M + 1, k)] ...
   + r*(u(1:M - 1, k - 1) + u(3:M + 1, k - 1)) + r1*u(2:M, k - 1);
   u(2:M, k) = trid(A,b); %Pt.(17)
end
```

Để giải phương trình:

$$\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t} \quad 0 \le x \le 1,\ 0 \le t \le 0.1 \qquad \text{(vd1)}$$

với điều kiện đầu:

$$u(x, 0) = \sin\pi x \qquad u(0, t) = 0 \qquad u(1, t) = 0 \qquad \text{(vd2)}$$

Như vậy với $\Delta x = x_f/M = 1/20$ và $\Delta t = T/N = 1/100$ ta có:

$$r = A\frac{\Delta t}{\Delta x^2} = 1.\frac{0.001}{0.05^2} = 0.4 \qquad \text{(vd3)}$$

Ta dùng chương trình ***ctheat.m*** để tìm nghiệm của (vd1):

```
clear all, clc
a = 1; %cac thong so cua (vd1)
it0 = inline('sin(pi*x)','x'); %dieu kien dau
bx0 = inline('0');
bxf = inline('0');%dieu kien bien
xf = 1;
M = 25;
T = 0.1;
N = 100;
[u1, x, t] = fwdeuler(a, xf, T, it0, bx0, bxf, M, N);
figure(1), clf, mesh(t, x, u1)
[u2, x, t] = backeuler(a, xf, T, it0, bx0, bxf, M, N);
figure(2), clf, mesh(t, x, u2)
[u3, x, t] = cranknicholson(a, xf, T , it0, bx0, bxf, M, N);
figure(3), clf, mesh(t, x, u3)
```

4. PDE parabolic 2 chiều: Ta xét bài toán phương trình vi phân đạo hàm riêng parabolic hai chiều mô tả sự phân bố nhiệt độ u(x, y, t):

$$A\left[\frac{\partial^2 u(x,y,t)}{\partial x^2} + \frac{\partial^2 u(x,y,t)}{\partial y^2}\right] = \frac{\partial u(x,y,t)}{\partial t} \qquad (19)$$

Để phương trình có thể giải được ta cần cho điều kiện biên:

$$u(x_0,y,t) = b_{x_0}(y,t) \qquad u(x_f,y,t) = b_{x_f}(y,t)$$

$$u(x,y_0,t) = b_{y_0}(x,t) \qquad u(x,y_f,t) = b_{y_f}(x,t)$$

và điều kiện đầu $u(x, y, 0) = i_0(x, y)$

Ta thay đạo hàm bậc 1 theo t ở vế phải bằng sai phân 3 điểm tại điểm giữa $(t_{k+1} + t_k)/2$ như phương pháp Crank - Nicholson. Ta cũng thay thế một trong các đạo hàm bậc hai u_{xx} và u_{yy} bằng xấp xỉ 3 điểm tại thời điểm t_k và đạo hàm kia tại t_{k+1} và có:

$$A\left(\frac{u^k_{i,j+1} - 2u^k_{i,j} + u^k_{i,j-1}}{\Delta x^2} - \frac{u^k_{i,j+1} - 2u^k_{i,j} + u^k_{i,j-1}}{\Delta x^2}\right) = \frac{u^{k+1}_{i,j} - u^k_{i,j}}{\Delta t} \qquad (20)$$

Ta viết phương trình tại thời điểm tiếp theo t_{k+1}:

$$A\left(\frac{u_{i,j+1}^{k+1}-2u_{i,j}^{k+1}+u_{i,j-1}^{k+1}}{\Delta x^2}-\frac{u_{i,j+1}^{k}-2u_{i,j}^{k}+u_{i,j-1}^{k}}{\Delta x^2}\right)=\frac{u_{i,j}^{k+2}-u_{i,j}^{k+1}}{\Delta t} \qquad (21)$$

Công thức này, được Peaceman và Rachford đưa ra, là phương pháp ẩn và tạo nên hệ phương trình:

$$-r_y\left(u_{i-1,j}^{k+1}+u_{i+1,j}^{k+1}\right)+(1+2r_y)u_{i,j}^{k+1}=r_x\left(u_{i,j-1}^{k}-u_{i,j+1}^{k}\right)+(1-2r_x)u_{i,j}^{k} \qquad (22a)$$

với $0 \le j \le M_x - 1$

$$-r_x\left(u_{i,j-1}^{k+2}+u_{i,j+1}^{k+2}\right)+(1+2r_x)u_{i,j}^{k+2}=r_y\left(u_{i-1,j}^{k+1}-u_{i+1,j}^{k+1}\right)+(1-2r_y)u_{i,j}^{k+1} \qquad (22b)$$

với $0 \le i \le M_y - 1$

và: $r_x = A\dfrac{\Delta t}{\Delta x^2}$ $\qquad r_y = A\dfrac{\Delta t}{\Delta y^2}$

$\Delta x = \dfrac{x_f - x_0}{M_x}$ $\qquad \Delta y = \dfrac{y_f - y_0}{M_y}$ $\qquad \Delta t = \dfrac{T}{N}$

Ta xây dựng hàm ***heat2D()*** để thực hiện thuật toán này:

```
function [u, x, y, t] = heat2D(a, D, T, ixy0, bxyt, Mx, My, N)
% Giai au_t = c(u_xx + u_yy) voi D(1) <= x <= D(2), D(3) <= y <= D(4), 0
<= t %<= T
% Dieu kien dau: u(x, y, 0) = ixy0(x, y)
% Dieu kien bien: u(x, y, t) = bxyt(x, y, t) voi (x, y)cB
% Mx/My - cac doan co doc theo truc x/y
% N - cac khoang thoi gian
dx = (D(2) - D(1))/Mx;
x = D(1) + [0:Mx]*dx;
dy = (D(4) - D(3))/My;
y = D(3) + [0:My]'*dy;
dt = T/N;
t = [0:N]*dt;
%Khoi gan
for j = 1:Mx + 1
   for i = 1:My + 1
      u(i, j) = ixy0(x(j), y(i));
   end
end
rx = a*dt/(dx*dx);
rx1 = 1 + 2*rx;
rx2 = 1 - 2*rx;
ry = a*dt/(dy*dy);
ry1 = 1 + 2*ry;
```

```
ry2 = 1 - 2*ry;
for j = 1:Mx - 1 %Pt.(22a)
   Ay(j, j) = ry1;
   if j > 1
      Ay(j - 1, j) = -ry;
      Ay(j, j-1) = -ry;
   end
end
for i = 1:My - 1 %Pt.(22b)
   Ax(i,i) = rx1;
   if i > 1
      Ax(i - 1, i) = -rx;
      Ax(i, i - 1) = -rx;
   end
end
for k = 1:N
   u_1 = u;
   t = k*dt;
   for i = 1:My + 1 %Dieu kien bien
      u(i, 1) = feval(bxyt, x(1), y(i), t);
      u(i, Mx+1) = feval(bxyt, x(Mx+1), y(i), t);
   end
   for j = 1:Mx + 1
      u(1, j) = feval(bxyt, x(j), y(1), t);
      u(My+1, j) = feval(bxyt, x(j), y(My + 1), t);
   end
   if mod(k, 2) == 0
      for i = 2:My
         jj = 2:Mx;
         bx = [ry*u(i, 1) zeros(1, Mx - 3) ry*u(i, My + 1)] ...
         +rx*(u_1(i-1,jj)+ u_1(i + 1,jj)) + rx2*u_1(i,jj);
         u(i, jj) = trid(Ay, bx')'; %Pt.(22a)
      end
   else
      for j = 2:Mx
         ii = 2:My;
         by = [rx*u(1, j); zeros(My-3,1); rx*u(Mx + 1,j)] ...
         + ry*(u_1(ii, j-1) + u_1(ii, j + 1)) + ry2*u_1(ii, j);
         u(ii, j) = trid(Ax, by); %Pt.(22b)
      end
   end
end
```

Ta xét phương trình:

$$10^{-4}\left[\frac{\partial^2 u(x,y,t)}{\partial x^2}+\frac{\partial^2 u(x,y,t)}{\partial y^2}\right]=\frac{\partial u(x,y,t)}{\partial t} \qquad \text{(vd1)}$$

trong miền: $0 \le x \le 4$, $0 \le y \le 4$ và trong khoảng thơig gian $0 \le t \le 5000$

Điều kiện đầu:

$$u(x, y, 0) = 0 \qquad \text{(vd2a)}$$

và điều kiện biên:

$e^y\cos x - e^x\cos y$ tại $x = 0$, $x = 4$; $y = 0$, $y = 4$ (vd2b)

Chương trình chương trình ***ctheat2D.m*** dùng để giải phương trình là:

```
clear, clc, clf
a = 1e-4;
it0 = inline('0','x','y'); %(vd2a)
bxyt = inline('exp(y)*cos(x)-exp(x)*cos(y)','x','y','t'); %(vd.2b)
D = [0 4 0 4];
T = 5000;
Mx = 40;
My = 40;
N = 50;
[u, x, y, t] = heat2D(a, D, T, it0, bxyt, Mx, My, N);
mesh(x, y, u)
```

§4. PHƯƠNG TRÌNH HYPERBOLIC

1. Dạng phương trình: Phương trình truyền sóng một chiều là PDE dạng hyperbolic:

$$A\frac{\partial^2 u(x,t)}{\partial x^2}=\frac{\partial^2 u(x,t)}{\partial t^2} \qquad (1)$$

$0 \le x \le x_f$, $0 \le t \le T$

Điều kiện biên:

$$u(0, t) = b_0(t),\ u(x_f,t)=b_{x_f}(t)$$

và điều biên:

$$u(x, 0) = i_0(x),\ \left.\frac{\partial u}{\partial t}\right|_{t=0}=i_0'(x)$$

phải được cho trước để phương trình có thể giải được

2. Phương pháp sai phân tường minh: Tương tự như khi giải PDE dạng parabolic, ta thay đạo hàm bậc hai ở hai vế của (1) bằng sai phân 3 điểm:

$$A\frac{u_{i+1}^k-2u_i^k+u_{i-1}^k}{\Delta x^2}=\frac{u_i^{k+1}-2u_i^k+u_i^{k-1}}{\Delta t^2} \qquad (2)$$

$$\Delta x=\frac{x_f}{M} \qquad \Delta t=\frac{T}{N}$$

và có được phương pháp sai phân tường minh:

$$u_i^{k+1} = r\left(u_{i+1}^k + u_{i-1}^k\right) + 2(1-r)u_i^k - u_i^{k-1} \qquad (3)$$

với: $r = A\dfrac{\Delta t^2}{\Delta x^2}$

Vì $u_i^{-1} = u(x_i, -\Delta t)$ không cho trước nên ta không thể dùng trực tiếp u_i^1 từ (3) với k = 0:

$$u_i^1 = r\left(u_{i+1}^0 + u_{i-1}^0\right) + 2(1-r)u_i^0 - u_i^{-1} \qquad (4)$$

Như vậy, ta xấp xỉ điều kiện đầu về đạo hàm bằng sai phân:

$$\frac{u_i^1 - u_i^{-1}}{2\Delta t} = i_0'(x_i) \qquad (5)$$

và rút ra u_i^{-1} để đưa vào (3):

$$u_i^1 = r\left(u_{i+1}^0 + u_{i-1}^0\right) + 2(1-r)u_i^0 - \left[u_i^1 - 2i_0'(x_i)\Delta t\right]$$

$$u_i^1 = \frac{1}{2}r\left(u_{i+1}^0 + u_{i-1}^0\right) + (1-r)u_i^0 + i_0'(x_i)\Delta t \qquad (6)$$

Ta dùng (6) cùng với điều kiện đầu để có u_i^1 và rồi thay vào (3). Chú ý là:

- $r \le 1$ để bảo đảm ổn định
- độ chính xác của nghiêm tăng khi r tăng để cho Δx giảm

Hợp lí nhất là lấy r = 1. Điều kiện ổn định có thể nhận được bằng cách thay (4) vào (3):

$$\lambda = 2r\cos\frac{\pi}{P} + 2(1-r) - \frac{1}{\lambda}$$

hay: $\lambda^2 + 2\left[r\left(1 - \cos\dfrac{\pi}{P}\right) - 1\right]\lambda + 1 = 0$

Như vậy:

$$r \le \frac{1}{1 - \left(\cos\dfrac{\pi}{P}\right)'} \qquad r = A\frac{\Delta t^2}{\Delta x^2} \le 1$$

Ta xây dựng hàm ***wave()*** để thực hiện thuật toán trên:

```
function [u, x, t] = wave(a, xf, T, it0, i1t0, bx0, bxf, M, N)
% giai au_xx = u_tt voi  0<=x<=xf, 0<=t<=T
% dieu kien dau: u(x,0) = it0(x), u_t(x, 0) = i1t0(x)
% dieu kien bien: u(0, t)= bx0(t), u(xf, t) = bxf(t)
% M - khoang chia theo x
% N - khoang  chia teo t
dx = xf/M;
x = [0:M]'*dx;
```

```
dt = T/N;
t = [0:N]*dt;
for i = 1:M + 1
   u(i,1) = it0(x(i));
end
for k = 1:N + 1
   u([1 M + 1],k) = [bx0(t(k)); bxf(t(k))];
end
r = a*(dt/dx)^2;
r1 = r/2;
r2 = 2*(1 - r);
u(2:M, 2) = r1*u(1:M - 1, 1) + (1 - r)*u(2:M, 1) + r1*u(3:M + 1, 1) ...
          + dt*i1t0(x(2:M)); %Pt.(6)
for k = 3:N + 1
   u(2:M, k) = r*u(1:M - 1, k - 1) + r2*u(2:M, k-1) + r*u(3:M + 1, k - 1)...
             - u(2:M,k - 2); %Pt.(3)
end
```

Ta xét phương trình:

$$\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial^2 u(x,t)}{\partial t^2} \qquad \text{(vd1)}$$

$0 \le x \le 2,\ 0 \le t \le 2$

Điều kiện đầu và điều kiện biên:

$$u(x, 0) = x(1 - x) \qquad \left.\frac{\partial u}{\partial t}\right|_{t=0} = 0 \qquad \text{(vd2a)}$$

$$u(0,t) = 0 \qquad u(1, t) = 0 \qquad \text{(vd2b)}$$

Ta dùng chương trình ***ctwave.m*** để giải phương trình này:

```
clear all, clc
a = 1;
it0 = inline('x.*(1-x)','x');
i1t0 = inline('0'); %(vd2a)
bx0t = inline('0');
bxft = inline('0'); %(vd2b)
xf = 1;
M = 20;
T = 2;
N = 50;
[u,x,t] = wave(a, xf, T, it0, i1t0, bx0t, bxft, M, N);
figure(1), clf
mesh(t, x, u)
```

```
figure(2), clf
for n = 1:N %hinh dong
   plot(x, u(:, n)), axis([0 xf -0.3 0.3]), pause(0.2)
end
```

4. PDE hyperbolic 2 chiều: Phương trình truyền sóng hai chiều là PDE dạng hyperbolic:

$$A\left[\frac{\partial^2 u(x,y,t)}{\partial x^2}+\frac{\partial^2 u(x,y,t)}{\partial y^2}\right]=\frac{\partial^2 u(x,y,t)}{\partial t^2} \tag{8}$$

$0 \le x \le x_f$, $0 \le y \le y_f$, $0 \le t \le T$

Điều kiện biên:

$$u(0,y,t)=b_{x_0}(y,t) \qquad u(x_f,y,t)=b_{x_f}(y,t)$$

$$u(x,0,t)=b_{y_0}(x,t) \qquad u(x,y_f,t)=b_{y_f}(x,t)$$

và điều biên:

$$u(x,y,0)=i_0(x,y) \qquad \left.\frac{\partial u(x,y)}{\partial t}\right|_{t=0}=i_0'(x,y)$$

Tương tự như hàm một biến, ta thay đạo hàm bậc 2 bằng xấp xỉ 3 điểm:

$$A\left(\frac{u_{i,j+1}^k-2u_{i,j}^k+u_{i,j-1}^k}{\Delta x^2}+\frac{u_{i+1,j}^k-2u_{i,j}^k+u_{i-1,j}^k}{\Delta y^2}\right)=\frac{u_{i,j}^{k+1}-2u_{i,j}^k+u_{i,j}^{k-1}}{\Delta t^2} \tag{9}$$

$$\Delta x=\frac{x_f}{M_x} \qquad \Delta y=\frac{y_f}{M_y} \qquad \Delta t=\frac{T}{N}$$

và nhận đi đến phương pháp tường minh:

$$u_{i,j}^{k+1}=r_x\left(u_{i,j+1}^k+u_{i,j-1}^k\right)+2(1-r_x-r_y)u_{i,j}^k+r_y\left(u_{i+1,j}^k+u_{i-1,j}^k\right)-u_{i,j}^{k-1} \tag{10}$$

với:

$$r_x=A\frac{\Delta t^2}{\Delta x^2} \qquad r_y=A\frac{\Delta t^2}{\Delta y^2}$$

Vì $u_{i,j}^{-1}=u(x_i,y_i,-\Delta t)$ không cho trước nên ta không thể dùng trực tiếp $u_{i,j}^1$ từ (10) với k = 0:

$$u_{i,j}^1=r_x\left(u_{i,j+1}^0+u_{i,j-1}^0\right)+2(1-r_x-r_y)u_{i,j}^0+r_y\left(u_{i+1,j}^0+u_{i-1,j}^0\right)-u_{i,j}^{-1} \tag{11}$$

Như vậy, ta xấp xỉ điều kiện đầu về đạo hàm bằng sai phân:

$$\frac{u_{i,j}^1-u_{i,j}^{-1}}{2\Delta t}=i_0'(x_j,y_i) \tag{12}$$

và rút ra $u_{i,j}^{-1}$ để đưa vào (11):

$$u_{i,j}^{1} = \frac{1}{2}\left[r_x\left(u_{i,j+1}^{0} + u_{i,j-1}^{0}\right) + r_y\left(u_{i+1,j}^{0} + u_{i-1,j}^{0}\right)\right] + 2(1 - r_x - r_y)u_{i,j}^{0} + i_0'(x_j, y_i)\Delta t \quad (13)$$

Điều kiện ổn định:

$$r = \frac{4A\Delta t^2}{\Delta x^2 + \Delta y^2} \le 1$$

Ta xây dựng hàm ***wave2D()*** để thực hiện thuật toán trên:

```
function [u,x,y,t] = wave2D(a, D, T, it0, i1t0, bxyt, Mx, My, N)
% giai a(u_xx + u_yy) = u_tt voi D(1) <= x <= D(2), D(3) <= y <= D(4), 0
<= t %<= T
% dieu kien dau: u(x,y,0) = it0(x,y), u_t(x,y,0) = i1t0(x,y)
% dieu kien bien: u(x,y,t) = bxyt(x,y,t) voi (x,y) tren bien
% Mx/My - so khoang chia tren truc x/y
% N - so khoang chai theo t
dx = (D(2) - D(1))/Mx;
x = D(1) + [0:Mx]*dx;
dy = (D(4) - D(3))/My;
y = D(3) + [0:My]'*dy;
dt = T/N; t = [0:N]*dt;
%khoi gan
u = zeros(My+1, Mx + 1);
ut = zeros(My + 1, Mx + 1);
for j = 2:Mx
   for i = 2:My
      u(i, j) = it0(x(j), y(i));
      ut(i, j) = i1t0(x(j), y(i));
   end
end
adt2 = a*dt*dt;
rx = adt2/(dx*dx);
ry = adt2/(dy*dy);
rxy1 = 1- rx - ry;
rxy2 = rxy1*2;
u_1 = u;
for k = 0:N
   t = k*dt;
   for i = 1:My + 1 %dieu kien bien
      u(i, [1 Mx + 1]) = [bxyt(x(1), y(i),t)  bxyt(x(Mx + 1), y(i),t)];
   end
   for j = 1:Mx + 1
```

```
        u([1 My + 1], j) = [bxyt(x(j),y(1),t); bxyt(x(j),y(My + 1),t)];
    end
    if k == 0
        for i = 2:My
            for j = 2:Mx %Pt.(13)
                u(i, j) = 0.5*(rx*(u_1(i, j - 1) + u_1(i, j + 1))...
                        + ry*(u_1(i - 1,j)+u_1(i + 1,j))) + rxy1*u(i,j) + dt*ut(i,j);
            end
        end
    else
        for i = 2:My
            for j = 2:Mx
                u(i, j) = rx*(u_1(i, j - 1)+ u_1(i, j + 1))...
                        + ry*(u_1(i - 1, j) + u_1(i + 1, j)) + rxy2*u(i, j) - u_2(i, j);
            end
        end
    end
    u_2 = u_1;
    u_1 = u;
    mesh(x, y, u), axis([0 2 0 2 -.1 .1])
    pause(0.1);
end
```

Ta xét phương trình:

$$\frac{1}{4}\left[\frac{\partial^2 u(x,y,t)}{\partial x^2}+\frac{\partial^2 u(x,y,t)}{\partial y^2}\right]=\frac{\partial^2 u(x,t)}{\partial t^2} \qquad \text{(vd1)}$$

$0 \le x \le 2,\ 0 \le y \le 2,\ 0 \le t \le 2$

Điều kiện đầu và điều kiện biên:

$$u(0, y, t) = 0 \quad u(2, y, t) = 0 \quad u(x, 0,t) = 0 \quad u(x, 2,t) = 0 \qquad \text{(vd2)}$$

$$u(x,y,0)=0.1\sin\frac{\pi x}{2}\sin\frac{\pi y}{2} \qquad \left.\frac{\partial u}{\partial t}\right|_{t=0}=0 \qquad \text{(vd3)}$$

Ta dùng chương trình ***ctwave2D.m*** để giải phương trình này:

```
clear all, clc
it0 = inline('0.1*sin(pi*x)*sin(pi*y/2)','x','y'); %(vd3)
i1t0 = inline('0','x','y');
bxyt = inline('0','x','y','t'); %(vd2)
a = .25;
D = [0 2 0 2];
T = 2;
Mx = 40;
```

My = 40;
N = 40;
[u, x, y, t] = wave2D(a, D, T, it0, i1t0, bxyt, Mx, My, N);

§5. PHƯƠNG PHÁP PHẦN TỬ HỮU HẠN (FEM) ĐỂ GIẢI PDE

Phương pháp FEM dùng để tìm nghiệm số của PDE với điều kiện biên. Ta xét một PDE dạng elliptic:

$$\frac{\partial^2 u(x,y)}{\partial x^2}+\frac{\partial^2 u(x,y)}{\partial y^2}+g(x,y)u(x,y)=f(x,y) \qquad (1)$$

trong miền D bao bởi biên B và trên biên có các điều kiện:

$$u(x, y) = b(x, y) \quad \text{trên B} \qquad (2)$$

Các bước dùng FEM để giải phương trình gồm:

☞ Chia miền D thành N_s miền con $\{S_1, S_2,..., S_{Ns}\}$ có dạng hình tam giác

☞ Mô tả vị trí của N_n nút và đánh số chúng bắt đầu từ các nút trên biên: n = 1, 2,..., N_b và các nút bên trong: n = N_b + 1, N_b + 2,...,N_n

☞ Xác định các hàm nội suy, hình dạng và cơ sở:

$$\phi_n(x,y)=\left\{\phi_{n,s} \quad s=1,...,N_s\right\}\forall(x,y)\in D \qquad (3a)$$

$$\phi_{n,s}(x,y)=p_{n,s}(1)+p_{n,s}(2)x+p_{n,s}(3)y$$

cho mỗi miền S_s (3b)

đối với tất cảc các miền con s = 1:N_s và các nút n = 1:N_n sao cho ϕ_n bằng 1 chỉ ở nút n và bằng zero tại các nút khác. Lúc đó nghiệm xấp xỉ của PDE là tổ hợp tuyến tính của các hàm cơ sở $\phi_n(x, y)$:

$$\begin{aligned} u(x,y) &= [c]^T[\varphi(x,y)]=\sum_{i=1}^{N_n} c_n\phi_n(x,y) \\ &= \sum_{i=1}^{N_b} c_n\phi_n(x,y)+\sum_{i=N_b+1}^{N_n} c_n\phi_n(x,y) \\ &= [c_1]^T[\varphi_1]+[c_2]^T[\varphi_2] \end{aligned} \qquad (4)$$

Trong đó:

$$[\varphi_1]=\begin{bmatrix}\phi_1 & \phi_2 & \cdots & \phi_{N_b}\end{bmatrix}^T \qquad [c_1]=\begin{bmatrix}c_1 & c_2 & \cdots & c_{N_b}\end{bmatrix}^T \qquad (5a)$$

$$[\varphi_2]=\begin{bmatrix}\phi_{N_b+1} & \phi_{N_b+2} & \cdots & \phi_{N_n}\end{bmatrix}^T \qquad [c_2]=\begin{bmatrix}c_{N_b+1} & c_{N_b+2} & \cdots & c_{N_n}\end{bmatrix}^T \qquad (5b)$$

Với mỗi miền con , nghiệm này có thể viết dưới dạng:

$$\phi_s(x,y)=\sum_{i=1}^{N_n} c_n\phi_{n,s}(x,y)=\sum_{i=1}^{N_n} c_n\left[p_{n,s}(1)+p_{n,s}(x)+p_{n,s}(3)y\right] \qquad (6)$$

☞ Đặt các giá trị của hệ số nút biên trong $[c_1]$ bằng các giá trị biên tương ứng với điều kiện biên

☞ Xác định trị số của hệ số nút bên trong trong $[c_2]$ bằng cách giải hệ phương trình:

$$[A_2][c_2] = [d] \qquad (7)$$

trong đó:

$$[A_1] = \sum_{s=1}^{N_s} \left\{ \left(\frac{\partial}{\partial x}[\varphi_{2,s}] \right) \left(\frac{\partial}{\partial x}[\varphi_{1,s}] \right)^T + \left(\frac{\partial}{\partial y}[\varphi_{2,s}] \right) \left(\frac{\partial}{\partial y}[\varphi_{1,s}] \right)^T \right\} \Delta S_s$$

$$- \sum_{s=1}^{N_s} \left\{ g(x_s, y_s)[\varphi_{2,s}][\varphi_{1,s}]^T \right\} \Delta S_s \qquad (8)$$

$$[\varphi_{1,s}] = [\phi_{1,s} \quad \phi_{2,s} \quad \cdots \quad \phi_{N_b,s}]^T$$

$$\frac{\partial}{\partial x}[\varphi_{1,s}] = [p_{1,s}(2) \quad p_{2,s}(2) \quad \cdots \quad p_{N_b,s}(2)]^T$$

$$\frac{\partial}{\partial y}[\varphi_{1,s}] = [p_{1,s}(3) \quad p_{2,s}(3) \quad \cdots \quad p_{N_b,s}(3)]^T$$

$$[A_2] = \sum_{s=1}^{N_s} \left\{ \left(\frac{\partial}{\partial x}[\varphi_{2,s}] \right) \left(\frac{\partial}{\partial x}[\varphi_{2,s}] \right)^T + \left(\frac{\partial}{\partial y}[\varphi_{2,s}] \right) \left(\frac{\partial}{\partial y}[\varphi_{2,s}] \right)^T \right\} \Delta S_s$$

$$- \sum_{s=1}^{N_s} \left\{ g(x_s, y_s)[\varphi_{2,s}][\varphi_{2,s}]^T \right\} \Delta S_s \qquad (9)$$

$$[\varphi_{2,s}] = [\phi_{N_b+1,s} \quad \phi_{N_b+2,s} \quad \cdots \quad \phi_{N_n,s}]^T$$

$$\frac{\partial}{\partial x}[\varphi_{2,s}] = [p_{N_b+1,s}(2) \quad p_{N_b+2,s}(2) \quad \cdots \quad p_{N_n,s}(2)]^T$$

$$\frac{\partial}{\partial y}[\varphi_{2,s}] = [p_{N_b+1,s}(3) \quad p_{N_b+2,s}(3) \quad \cdots \quad p_{N_n,s}(3)]^T$$

$$[d] = -[A_1][c_1] - \sum_{s=1}^{N_s} f(x_s, y_s)\varphi_{2,s}\Delta S \qquad (10)$$

(x_s, y_s) là trong tâm của miền con S_s

FEM dựa trên nguyên tắc là nghiệm của (1) có thể nhận được bằng cách cực tiểu hoá hàm:

$$J = \iint_R \left\{ \left[\frac{\partial}{\partial x} u(x,y) \right]^2 + \left[\frac{\partial}{\partial y} u(x,y) \right]^2 \right.$$

$$\left. - g(x,y)u^2(x,y) + 2f(x,y)u(x,y) \right\} dxdy \qquad (11)$$

Với $u(x, y) = [c]^T[\varphi(x, y)]$ ta có:

$$J = \iint_R \left\{ [c]^T \frac{\partial}{\partial x}[\varphi] \frac{\partial}{\partial x}[\varphi][c] + [c]^T \frac{\partial}{\partial y}[\varphi] \frac{\partial}{\partial y}[\varphi]^T [c] \right.$$

$$-g(x,y)[c]^T[\varphi][\varphi]^T[c]+2f(x,y)[c]^T[\varphi]\Big\}dxdy \qquad (12)$$

Điều kiện để hàm này cực tiểu theo [c] là:

$$\frac{d}{d[c_2]}J=\iint_R\Big\{\frac{\partial}{\partial x}[\varphi_2]\frac{\partial}{\partial x}[\varphi]^T[c]+\frac{\partial}{\partial y}[\varphi]^T[c]$$

$$-g(x,y)[\varphi_2][\varphi]^T[c]+2f(x,y)[\varphi_2]\Big\}dxdy=0 \qquad (13)$$

$$\approx[A_1][c_1]+[A_2][c_2]+\sum_{s=1}^{N_s}f(x_s,y_s)\left[\varphi_{2,s}\right]\Delta S=0 \qquad (14)$$

Để xây dựng hàm cơ sở $\phi_{n,s}(x,y)$ thứ s đối với mỗi nút n = 1, 2,..., N_n và mỗi miền con s = 1, 2,...,N_s ta xây dựng hàm ***fembasisftn()***:

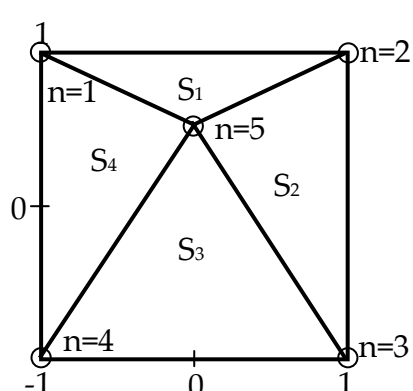
```
function p = fembasisftn(N, S)
%p(i,s,1:3): cac he so cua moi ham co so phi(i)
% cua mien tam giac(mien con) thu s
%N(n, 1:2) : x & y toa do cua nut thu n
%S(s, 1:3) :nut thu s cua mien con tam giac thu s
Nn = size(N, 1); % tong so nut
Ns = size(S, 1); % tong so cac nut cua mien con tam giac
for n = 1:Nn
   for s = 1:Ns
      for i = 1:3
         A(i, 1:3) = [1 N(S(s, i), 1:2)];
         b(i) = (S(s, i) == n); %ham co so thu n bang 1 chi o nut thu n
      end
      pnt = A\b';
      for i = 1:3\
         p(n, s, i) = pnt(i);
      end
   end
end
```

Để xác định các vec tơ hệ số [c] của nghiệm (4) nhờ (7) và các đa thức nghiệm $\phi_s(x,y)$ nhờ (6) đối với mỗi miền con s = 1, 2,..., N_s ta xây dựng hàm ***femcoef()***:

```
function [U, c] = femcoef(f, g, p, c, N, S, Ni)
%p(i,s,1:3): cac he so cua ham co so phi(i) cua mien con thu n
%c = [ .1 1 . 0 0 .] voi cac gia tri bien va 0 voi cac nut ben trong
%N(n, 1:2) : x & y toaj do cua nut thu n
%S(s,1:3) : nut thu s cua mien con thu s
%Ni : so nut ben trong
```

```
%U(s, 1:3) : cac he so cua  p1 + p2(s)x + p3(s)y cua moi mien con
Nn = size(N, 1); % tong so nut bang Nb + Ni
Ns = size(S, 1); % tong so mien con
d = zeros(Ni, 1);
Nb = Nn - Ni;
for i = Nb+1:Nn
   for n = 1:N_n
      for s = 1:Ns
         xy = (N(S(s, 1),:) + N(S(s, 2), :) + N(S(s, 3), :))/3; %trong tam
         %phi(i,x)*phi(n,x) + phi(i,y)*phi(n,y) - g(x,y)*phi(i)*phi(n)
         pvctr = [p([i n], s, 1) p([i n], s, 2) p([i n], s, 3)];
         tmpg(s) = sum(p(i, s, 2:3).*p(n, s, 2:3))...
                  -g(xy(1), xy(2))*pvctr(1, :)*[1 xy]'*pvctr(2, :)*[1 xy]';
         dS(s) = det([N(S(s, 1), :) 1; N(S(s, 2), :) 1;N(S(s, 3), :) 1])/2;
         %dien tich mien con
         if n == 1
            tmpf(s) = -f(xy(1), xy(2))*pvctr(1,:)*[1 xy]';
         end
      end
      A12(i - Nb, n) = tmpg*abs(dS)'; %Pt. (8),(9)
   end
   d(i-Nb) = tmpf*abs(dS)'; %Pt.(10)
end
d = d - A12(1:Ni, 1:Nb)*c(1:Nb)'; %Pt(10)
c(Nb + 1:Nn) = A12(1:Ni, Nb+1:Nn)\d; %Pt.(7)
for s = 1:Ns
   for j = 1:3
      U(s, j) = c*p(:, s, j); %Pt.(6)
   end
end
```

Trước khi dùng FEM để giải một PDE ta xem thử hàm cơ sở(hàm hình dạng) $\phi_n(x, y)$ đối với mỗi nút n = 1, 2,..., N_n được định nghĩa đối với tát cả các miền con hình tam giác sao cho ϕ_n bằng 1 chỉ tại nút n và bằng 0 tại các nút khác được tạo bởi hàm ***fembasisfth()*** hoạt động như thế nào. Ta sẽ vẽ hàm hình dạng của miền được chia thành 4 tam giác như hình sau:



Toạ độ của nút:

```
N = [ -1  1;
       1  1;
       1 -1;
      -1 -1;
       0  0.5];
```

Số nút mỗi miền con

```
S = [1  2  5;
     2  3  5;
     3  4  5;
     1  4  5];
```

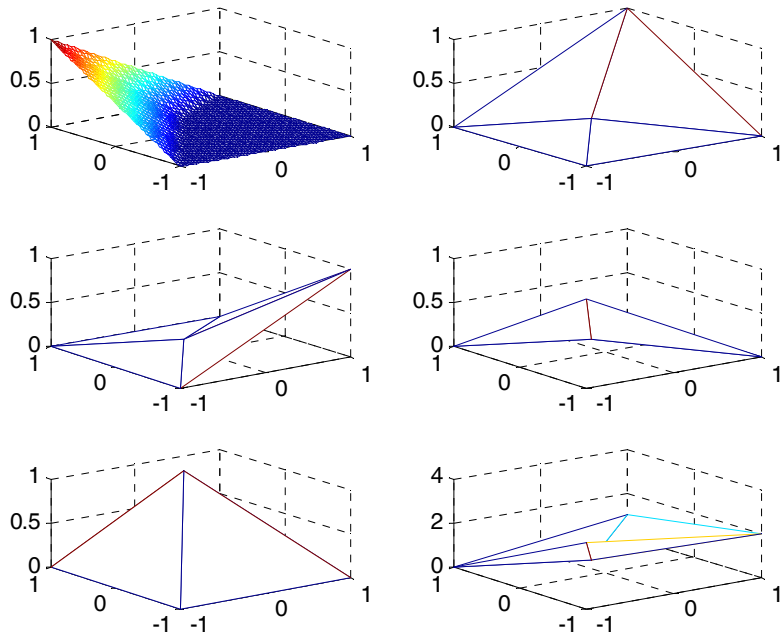
theo hai cách. Trước hết ta tạo các hàm cơ sở bằng cách dùng hàm ***fembasisftn()*** và vẽ một hàm trong số đó bằng cách dùng lệnh MATLAB ***mesh()*** như hình a. Thứ hai, không tạo ra hàm cơ sở, ta dùng lệnh MATLAB ***trimesh()*** để vẽ các hàm hình dạng cho các nút n = 2, 3, 4 và 5 như hình b-e. Hình f là đồ thị của tổ hợp tuyến tính của các hàm cơ sở:

$$u(x,y) = [c]^T [\varphi(x,y)] = \sum_{n=1}^{N_n} c_n \phi_n(x,y) \tag{15}$$

có trị số c_n tại mỗi nút n. Ta chạy chương trình ***ctshowbasic.m***:

```
clear all, clc
N = [-1 1;1 1;1 -1;-1 -1;0.2 0.5]; %danh sach cac nut tren hinh 1
Nn = size(N,1); % so nut
S = [1 2 5; 2 3 5; 3 4 5; 1 4 5]; %danh sach ca mien con tren hinh 1
Ns = size(S,1); % so mien con
figure(1), clf
for s = 1:Ns
   nodes = [S(s, :) S(s, 1)];
   for i = 1:3
      plot([N(nodes(i), 1) N(nodes(i + 1), 1)], ...
      [N(nodes(i), 2)  N(nodes(i+1),2)])
   hold on
   end
end
ins = [1 2 3 4 5]; %danh sach cac nut ma cac ham co so duoc ve
for itr = 1:5
   in = ins(itr);
   if itr == 1
      for i = 1:length(xi)
         for j = 1:length(yi)
            Z(j, i) = 0;
            for s = 1:Ns
               if inpolygon(xi(i), yi(j), N(S(s, :), 1), N(S(s, :), 2)) > 0
                  Z(j, i) = p(in, s, 1) + p(in, s, 2)*xi(i) + p(in, s, 3)*yi(j);
                  break;
               end
            end
         end
      end
      subplot(321), mesh(xi, yi, Z) %ham co so cua nut 1
   else
      c1 = zeros(size(c));
      c1(in) = 1;
```

```
        subplot(320 + itr)
        trimesh(S,N(:,1),N(:,2),c1) %ham co so cua cac nut 2-5
    end
end
c = [0 1 2 3 0]; %cac gia tri tai cac nut
subplot(326)
trimesh(S, N(:, 1),N(:, 2), c) %ham tong hop o hinh f
```



```
c = [0 1 2 3 0];
p = fembasisftn(N, S);
x0 = -1;
xf = 1;
y0 = -1;
yf = 1; %cac mien
figure(2), clf
Mx = 50;
My = 50;
dx = (xf - x0)/Mx;
dy = (yf - y0)/My;
xi = x0 + [0:Mx]*dx;
yi = y0 + [0:My]*dy;
```

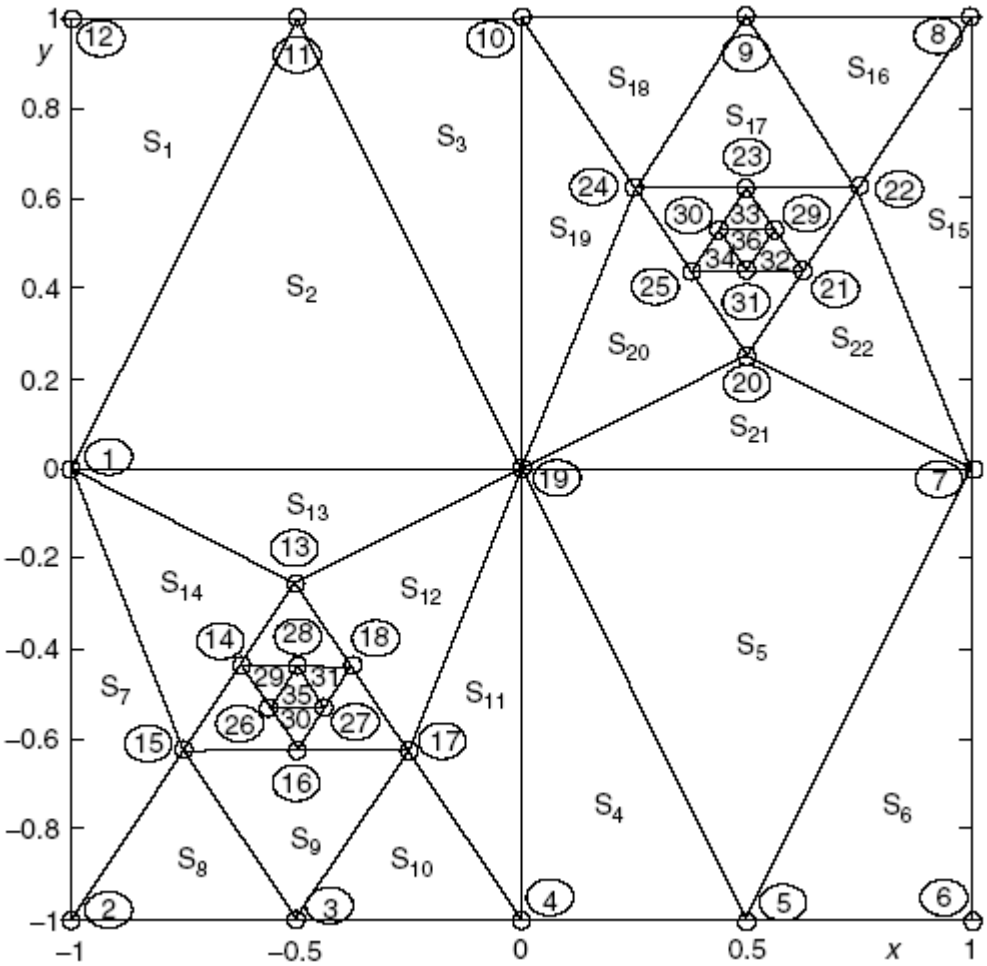
Ví dụ: Giải phương trình Laplace:

$$\nabla^2 u(x,y) = \frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2} = f(x,y) \qquad (1)$$

trên miền $-1 \le x \le 1;\ -1 \le y \le 1$ với:

$$f(x,y)=\begin{cases} -1 & \text{với } (x, y) = (0.5, 0.5) \\ +1 & \text{với } (x, y) = (-0.5, -0.5) \\ 0 & \text{các chỗ khác} \end{cases} \qquad (2)$$

và điều kiện biên là u(x, y) = 0 tại mọi điểm trên biên.



Để giải bài toán này bằng FEM, ta xác định 12 điểm trên biên và 19 điểm bên trong, đánh số chúng và chia miền chữ nhất thành 36 miên con hình tam giác như hình vẽ trên. Tiếp theo ta xây dựng chương trình ***ctlaplace.m*** để giải bài toán

```
clear all, clc
N = [-1 0;-1 -1;-1/2 -1;0 -1;1/2 -1; 1 -1;1 0;1 1;1/2 1; 0 1;
     -1/2 1;-1 1; -1/2 -1/4; -5/8 -7/16;-3/4 -5/8;-1/2 -5/8;
     -1/4 -5/8;-3/8 -7/16; 0 0; 1/2 1/4;5/8 7/16;3/4 5/8;
      1/2 5/8;1/4 5/8;3/8 7/16;-9/16 -17/32;-7/16 -17/32;
     -1/2 -7/16;9/16 17/32;7/16 17/32;1/2 7/16]; %nut
Nb = 12; %so nut tren bien
```

```
S = [1 11 12;1 11 19;10 11 19;4 5 19;5 7 19; 5 6 7;1 2 15; 2 3 15;
     3 15 17;3 4 17;4 17 19;13 17 19;1 13 19;1 13 15;7 8 22;8 9 22;
     9 22 24;9 10 24; 10 19 24; 19 20 24;7 19 20; 7 20 22;13 14 18;
     14 15 16;16 17 18;20 21 25;21 22 23;23 24 25;14 26 28;
     16 26 27;18 27 28; 21 29 31;23 29 30;25 30 31;
     26 27 28; 29 30 31]; %mien con tam giac
fexemp = '(norm([x y] + [0.5 0.5])<0.01) - (norm([x y] - [0.5 0.5]) < 0.01)';
f = inline(fexemp,'x','y'); %(Pt.2)
g = inline('0','x','y');
Nn = size(N, 1); %tong so nut
Ni = Nn - Nb; %so nut ben trong
c = zeros(1, Nn); %gia tri tren bien
p = fembasisftn(N, S);
[U, c] = femcoef(f, g, p, c, N, S, Ni);
%do thi luoi tam giac
figure(1);
clf;
trimesh(S, N(:, 1), N(:, 2), c);
%do thi luoi chu nhat
Ns = size(S, 1); %tong so mien con tam giac
x0 = -1;
xf = 1;
y0 = -1;
yf = 1;
Mx = 16;
dx = (xf - x0)/Mx;
xi = x0 + [0:Mx]*dx;
My = 16;
dy = (yf - y0)/My;
yi = y0 + [0:My]*dy;
for i = 1:length(xi)
   for j = 1:length(yi)
      for s = 1:Ns
         if inpolygon(xi(i), yi(j), N(S(s,:), 1), N(S(s,:),2)) > 0
            Z(i, j) = U(s,:)*[1 xi(i) yi(j)]'; %Pt.(4.5b)
            break;
         end
      end
   end
end
figure(2);
clf;
```

```
mesh(xi, yi, Z)
%de so sanh
bx0 = inline('0');
bxf = inline('0');
by0 = inline('0');
byf = inline('0');
D = [x0 xf y0 yf];
[U, x, y] = poisson(f, g, bx0, bxf, by0, byf, D, Mx, My, 1e-6, 50);
figure(3)
clf;
mesh(x,y,U)
```

§6. GUI CỦA MATLAB ĐỂ GIẢI PDE

1. Các phương trình có thể giải được bằng PDETOOL: Công cụ PDETOOL của MATLAB có thể dùng để giải các loại phương trình sau:

a. Phương trình elliptic: Ta sẽ giải phương trình elliptic

$$-\nabla(c\nabla u) + au = f \tag{1}$$

với điều kiện bên:

$$\begin{aligned} &hu = r && \text{Dirichlet} \\ &\vec{n}c\nabla u + qu = g && \text{Neumann} \end{aligned} \tag{2}$$

trên biên $\partial\Omega$, trong đó $\vec{n}$ là vec tơ pháp tuyến.

Trong trường hợp u là đại lượng vô hướng, phương trình (1) trở thành:

$$-c\left[\frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2}\right] + au(x,y) = f(x,y) \tag{3}$$

và nếu điều kiện biên đối với phân biên bên trái là điều kiện biên Neumann dạng $\left.\frac{\partial u(x,y)}{\partial x}\right|_{x=x_0} = b'_{x_0}(y)$ thì (2) có thể viết thành:

$$\begin{aligned} &-\vec{e}_x c\left[\frac{\partial u(x,y)}{\partial x}\vec{e}_x + \frac{\partial u(x,y)}{\partial y}\vec{e}_y\right] + qu(x,y) \\ &= -c\frac{\partial u(x,y)}{\partial x} + qu(x,y) = g(x,y) \end{aligned} \tag{4}$$

vì vec tơ pháp tuyến của biên phải là $\vec{n} = \vec{e}_x$

b. Phương trình parabolic: Ta xé giải phương trình:

$$-\nabla(c\nabla u) + au + d\frac{\partial u}{\partial t} = f \tag{5}$$

trên miền Ω và trong khoảng thời gian $0 \le t \le T$, với điều kiện bên giống (2) và điều kiện đầu $u(t_0)$

c. Phương trình hyperbolic:

$$-\nabla(c\nabla u) + au + d\frac{\partial^2 u}{\partial t^2} = f \qquad (6)$$

trên miền Ω và trong khoảng thời gian $0 \le t \le T$, với điều kiện bên giống (2) và điều kiện đầu $u(t_0)$, $u'(t_0)$

d. Phương trình giá trị riêng:

$$-\nabla(c\nabla u) + au = \lambda\frac{\partial u}{\partial t} \qquad (7)$$

trên miền Ω với một giá trị riêng chưa biết λ và điều kiện biên tương tự (2).

Công cụ PDETOOL cũng có thể dùng để giải hệ phương trình dạng:

$$\begin{cases} -\nabla(c_{11}\nabla u_1) - \nabla(c_{12}\nabla u_2) + a_{11}u_1 + a_{12}u_2 = f_1 \\ -\nabla(c_{21}\nabla u_1) - \nabla(c_{22}\nabla u_2) + a_{21}u_1 + a_{22}u_2 = f_2 \end{cases} \qquad (8)$$

trên miền Ω với điều kiện biên Dirichlet:

$$\begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix}\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} \qquad (9)$$

hay điều kiện Neumann tổng quát:

$$\begin{cases} \vec{n}(c_{11}\nabla u_1) + \vec{n}(c_{12}\nabla u_2) + q_{11}u_1 + q_{12}u_2 = g_1 \\ \vec{n}(c_{21}\nabla u_1) + \vec{n}(c_{22}\nabla u_2) + q_{21}u_1 + q_{22}u_2 = g_2 \end{cases} \qquad (10)$$

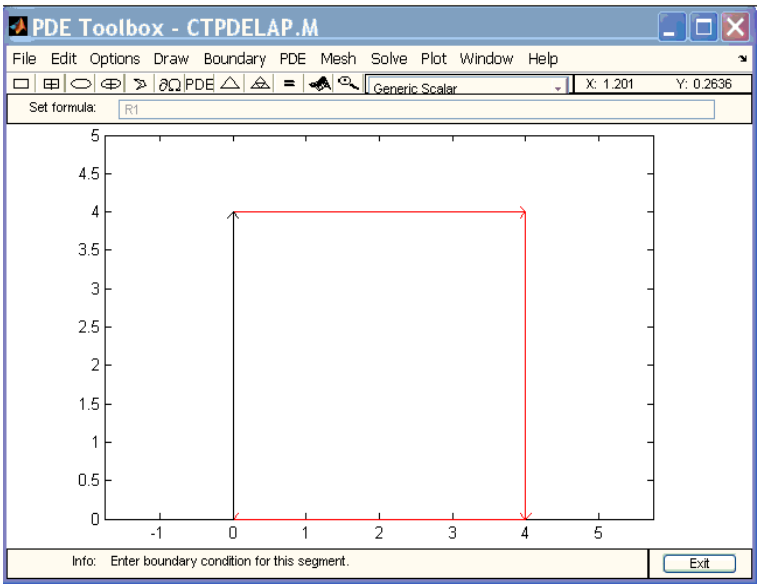
hay điều kiện biên hỗn hợp:

$$c = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \quad a = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad f = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix} \quad u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$h = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix} \quad r = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} \quad q = \begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{bmatrix} \quad g = \begin{bmatrix} g_1 \\ g_2 \end{bmatrix}$$

2. Sử dụng PDETOOL: PDETOOL giải phương trình vi phân đạo hàm riêng bằng cách dùng phương pháp FEM. Để giải phương trình ta theo các bước sau:

☞ Nhập lệnh ***pdetool*** vào cửa sổ lệnh MATLAB. Cửa sổ PDE toolbox xuất hiện. Ta có thể bật/tắt tuỳ chọn Grid bằng cách bấm vào ***Grid*** trên menu ***Option***. Ta cũng có thể hiệu chỉnh phạm vi trục x và y bằng cách chọn ***Axes Limit*** trong nemu ***Option***



Nếu muốn cho các hình gắn vào lưới, ta chọn ***Snap*** trong menu ***Option***. Nếu muốn tỉ lệ xích của trục x và t bằng nhau để hình tròn nhìn không giống hình ellip ta chọn ***Axes Equal*** trong menu ***Option***.

☞ Để vẽ miền Ω ta dùng menu ***Draw*** hay các icon trên thanh công cụ ngay phía dưới các menu.

☞ Để đặt điều kiện biên ta dùng menu ***Boundary*** hay icon ∂Ω. Ta bấm lên từng đoạn biên để đặt điều kiện cho nó.

☞ Tiếp theo ta tạo lưới bằng cách dùng menu ***Mesh*** hay icon Δ. Để tinh chỉnh lưới ta bấm vào ***Refine Mesh*** hay icon

☞ Tiếp theo ta mô tả dạng phương trình và các thông số của nó bằng cách dùng menu ***PDE***. Muốn thế, ta mở menu ***PDE*** hay chọn icon PDE và chọn ***PDE Specification*** và cho các tham số của phương trình.

☞ Để giải phương trình ta dùng menu ***Solve*** hay chọn icon = . Ta chọn menu con Parameters để nhập điều kiện đầu và khoảng thời gian tìm nghiệm

☞ Nếu muốn vẽ kết quả, ta dùng menu ***Plot***

3. Một số ví dụ:

a. Ví dụ 1: Giải phương trình Laplace:

$$\nabla^2 u(x,y) = \frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2} = 0 \qquad \text{(vd1.1)}$$

trong miền $0 \le x \le 4$, $0 \le y \le 4$ với các điều kiện biên:

$u(0, y) = e^y - \cos y$ $\qquad$ $u(4, y) = e^y\cos 4 - e^4\cos y$ $\qquad$ (vd1.2)

$u(x, 0) = \cos x - e^x$ $\qquad$ $u(x, 4) = e^4\cos x - e^x\cos 4$ $\qquad$ (vd1.3)

Để giải phương trình ta thực hiện các bước sau:

➢ Mở công cụ PDETOOL. Vào menu ***Option*** | ***Axes Limit*** để hiệu chỉnh lại phạm vi giá trị của x và y là [0 5] rồi chọn ***Apply*** và ***Close***. Chọn ***Option*** | ***Axes Equal***

➢ Bấm vào icon để vẽ hình vuông. Khi vẽ xong, nếu chưa đúng kích thước ta bấm đúp vào đối tượng bây giờ có tên là R1 để hiệu chỉnh lại thành Left: 0, Bottom: 0, Height: 4, Width: 4.

➢ Bấm vào icon ∂Ω thì đường biên của đối tượng có màu đỏ. Trên mỗi đoạn biên ta cho điều kiện biên theo (vd1.2) và (vd1.3). Để ghi điều kiện biên cho đoạn nào ta bấm đúp chuột lên đoạn đó. Điều kiện biên đã cho là điều kiện biên Dirrichlet. Trên biên trái, ta ghi điều kiện biên:

$h = 1, r = \exp(y) - \cos(y)$

trên biên phải:

$h = 1, r = e^y\cos 4 - e^4\cos y$

trên biên dưới:

$h = 1, r = \cos x - e^x$

và trên biên trên:

$h = 1, r = e^4\cos x - e^x\cos 4$

➢ Bấm đúp chuột vào icon PDE và chọn phương trình dạng elliptic và các thông số theo (vd1.1): c = 1, a = 0, f = 0

➢ Bấm đúp chuột vào icon để tạo lưới và sau đó tinh chỉnh nó.

➢ Bấm đúp chuột vào icon = để giải phương trình.

➢ Vào menu ***Plot*** | ***Parameters*** để chọn cách vẽ và sau đó vẽ ra kết quả

b. Ví dụ 2: Giải phương trình parabolic:

$$10^{-4}\left[\frac{\partial^2 u(x,y,t)}{\partial x^2}+\frac{\partial^2 u(x,y,t)}{\partial y^2}\right]=\frac{\partial u(x,y,t)}{\partial t} \qquad \text{(vd2.1)}$$

trong miền $0 \le x \le 4$, $0 \le y \le 4$ và $0 \le t \le 5000$ với các điều kiện đầu và điều biên:

$u(x, y, 0) = 0$ (vd2.2a)

$u(x, y, t) = e^y\cos x - e^x\cos y$ với $x = 0, x = 4, y = 0, y = 4$ (vd2.2b)

Để giải phương trình ta theo các bước sau:

➢ Mở công cụ PDETOOL. Vào menu ***Option*** | ***Axes Limit*** để hiệu chỉnh lại phạm vi giá trị của x và y là [0 4] rồi chọn ***Apply*** và ***Close***. Chọn ***Option*** | ***Axes Equal***

➢ Bấm vào icon để vẽ hình vuông. Khi vẽ xong, nếu chưa đúng kích thước ta bấm đúp vào đối tượng bây giờ có tên là R1 để hiệu chỉnh lại thành Left: 0, Bottom: 0, Height: 4, Width: 4.

➢ Bấm vào icon ∂Ω thì đường biên của đối tượng có màu đỏ. Trên mỗi đoạn biên ta cho điều kiện biên theo (vd2.2b). Để ghi điều kiện biên cho đoạn nào ta bấm đúp chuột lên đoạn đó. Điều kiện biên đã cho là điều kiện biên Dirrichlet. Trên biên trái, ta ghi điều kiện biên:

$h = 1, r = \exp(y) - \cos(y)$

trên biên phải:

$h = 1, r = e^y\cos 4 - e^4\cos y$

trên biên dưới:

$h = 1, r = \cos x - e^x$

và trên biên trên:

$h = 1, r = e^4\cos x - e^x\cos 4$

➢ Bấm đúp chuột vào icon PDE và chọn phương trình dạng parabolic và các thông số theo (vd2.1): c = 1e-4, a = 0, f = 0, d = 1. Trong menu ***Solve*** | ***Parameters*** ta ghi ***Time***: 0:100:5000, u(t0) = 0 (điều kiện đầu).

➢ Bấm đúp chuột vào icon để tạo lưới và sau đó tinh chỉnh nó.

➢ Bấm đúp chuột vào icon = để giải phương trình.

➢ Vào menu ***Plot*** | ***Parameters*** để chọn cách vẽ và sau đó vẽ ra kết quả

c. Ví dụ 3: Giải phương trình hyperbolic:

$$\frac{1}{4}\left[\frac{\partial^2 u(x,y,t)}{\partial x^2}+\frac{\partial^2 u(x,y,t)}{\partial y^2}\right]=\frac{\partial^2 u(x,y,t)}{\partial t^2} \qquad \text{(vd3.1)}$$

trong miền $0 \le x \le 2$, $0 \le y \le 2$ và $0 \le t \le 2$ với các điều kiện biên zero và điều kiện đầu:

$u(0, y, t) = 0$ $u(2, y, t) = 0$ $u(x, 0, t) = 0$ $u(0, 2, t) = 0$ (vd3.2)

$u(x, y, 0) = 0.1\sin(\pi x)\sin(\pi y/2)$ $\partial u/\partial t(x, y, 0) = 0$ với $t = 0$ (vd3.3)

Để giải phương trình ta theo các bước sau:

➢ Mở công cụ PDETOOL. Vào menu ***Option*** | ***Axes Limit*** để hiệu chỉnh lại phạm vi giá trị của x và y là [0 2] rồi chọn ***Apply*** và ***Close***. Chọn ***Option*** | ***Axes Equal***

➢ Bấm vào icon để vẽ hình vuông. Khi vẽ xong, nếu chưa đúng kích thước ta bấm đúp vào đối tượng bây giờ có tên là R1 để hiệu chỉnh lại thành Left: 0, Bottom: 0, Height: 2, Width: 2.

➢ Bấm vào icon $\partial\Omega$ thì đường biên của đối tượng có màu đỏ. Trên mỗi đoạn biên ta cho điều kiện biên theo (vd3.2). Để ghi điều kiện biên cho đoạn nào ta bấm đúp chuột lên đoạn đó. Điều kiện biên đã cho là điều kiện biên Dirrichlet. Trên biên trái, ta ghi điều kiện biên:

$h = 1, r = 0$

trên biên phải:

$h = 1, r = 0$

trên biên dưới:

$h = 1, r = 0$

và trên biên trên:

$h = 1, r = 0$

➢ Bấm đúp chuột vào icon PDE và chọn phương trình dạng parabolic và các thông số theo (vd2.1): c = 1/4, a = 0, f = 0, d = 1. Trong menu ***Solve*** | ***Parameters*** ta ghi ***Time***: 0: 0.1: 2, u(t0) = 0.1*sin(pi*x).*sin(pi*y/2).

➢ Bấm đúp chuột vào icon để tạo lưới và sau đó tinh chỉnh nó.

➢ Bấm đúp chuột vào icon = để giải phương trình.

➢ Vào menu ***Plot*** | ***Parameters*** để chọn cách vẽ và sau đó vẽ ra kết quả

CHƯƠNG 10: CÁC CÔNG CỤ KHÁC CỦA MATLAB

§1. SIMULINK

1. Khởi động Sinulink: Để khởi động Simulink ta theo các bước sau:

- khởi động MATLAB
- click vào icon của Simulink trên MATLAB toolbar hay đánh lệnh Simulink trong cửa sổ MATLAB.

Lúc này trên màn hình xuất hiện cửa sổ Simulink Library Browser, trong đó có các thư viện các khối của Simulink.

2. Tạo một mô hình mới: Để tạo một mô hình mới, click vào icon trên cửa sổ Simulink Library Browser hay chọn menu File | New | Model trên cửa sổ MATLAB.

3. Thay đổi một mô hình đã có: Ta có thể click vào icon trên cửa sổ Simulink Library Browser hay chọn Open trên cửa sổ MATLAB. File chứa mô hình sẽ mở và ta có thể thay đối các thông số cũng như bản thân mô hình .

4. Chọn một đối tượng: Để chọn một đối tượng, click lên nó. Khi này đối tượng sẽ có một hình chữ nhật có các góc là các hạt bao quanh.

5. Chọn nhiều đối tượng: Ta có thể chọn nhiều đối tượng cùng lúc bằng cách dùng phím Shift và chuột hay vẽ một đường bao quanh các đối tượng đó bằng cách bấm chuột kéo thành hình chữ nhật và thả khi hình chữ nhật đó đã bao lấy các đối tượng cần chọn.

6. Chọn tất cả các đối tượng: Để chọn tất cả các đối tượng trong cửa sổ ta chọn menu Edit | Select All.

7. Các khối: Khối là các phần tử mà Simulink dùng để tạo mô hình. Ta có thể mô hình hoá bất kì một hệ thống động học nào bằng cách tạo mối liên hệ giữa các khối theo cách thích hợp. Khi tạo một mô hình ta cần thấy rằng các khối của Simulink có 2 loại cơ bản: khối nhìn thấy và khối không nhìn thấy. Các khối không nhìn thấy được đóng vai trò quan trọng trong việc mô phỏng một hệ thống. Nếu ta thêm hay loại bỏ một khối không nhìn thấy được ta đã thay đổi thuộc tính của mô hình. Các khối nhìn thấy được, ngược lại, không đóng vai trò quan trọng trong mô hình hoá. Chúng chỉ giúp ta xây dựng mô hình một cách trực quan bằng đồ hoạ. Một vài khối của Simulink có thể là thấy được trong một số trường hợp và lại không thấy được trong một số trường hợp khác. Các khối như vậy được gọi là các khối nhìn thấy có điều kiện.

8. Copy các khối từ một cửa sổ sang một cửa sổ khác: Khi ta xây dựng một mô hình ta thường phải copy các khối từ thư viện khối của Simulink sang cửa sổ mô hình. Để làm việc này ta theo các bước sau:

- mở cửa sổ thư viện khối
- kéo khối ta muốn dùng từ cửa sổ thư viện vào cửa sổ mô hình và thả

Ta có thể copy các khối bằng cách dùng lệnh ***Copy*** & ***Paste*** trong menu ***Edit*** qua các bước sau :

- chọn khối ta muốn copy
- chọn ***Copy*** từ menu ***Edit***
- làm cho cửa sổ cần copy tới hoạt động
- chọn ***Paste*** từ menu ***Edit***

Simulink gán một tên cho mỗi bản copy. Nếu nó là khối đầu tiên trong mô hình thì tên của nó giống như trong thư viện Simulink. Nếu nó là bản thứ 2 hay thứ 3 thì sau nó sẽ có chỉ số 1 hay 2 v.v. Trên cửa sổ mô hình có lưới. Để hiển thị lưới này từ cửa sổ MATLAB đánh vào :

set_param('<model name>','showgrid','on')

Để thay đổi khoảng cách ô lưới đánh lệnh:

set_param('<model name>','gridspacing',<number of pixels>)

Ví dụ: để thay đổi ô lưới thành 20 pixels, đánh lệnh:

set_param('<model name>','gridspacing',20)

Để nhân bản một khối ta giữ phím Ctrl và kéo khối tới một vị trí khác và thả.

9. Mô tả thông số của khối: Để mô tả thông số của khối ta dùng hộp thoại ***Block Properties***. Để hiển thị hộp thoại này ta chọn khối và chọn ***Block Properties*** từ menu ***Edit***. Ta có thể nhắp đúp chuột lên khối để hiên thị hộp thoại này. Hộp thoại ***Block Properties*** gồm :

- ***Description***: Mô tả ngắn gọn về mục đích của khối.
- ***Priority***: thực hiện quyền ưu tiên của khối so với các khối khác trong mô hình .
- ***Tag***: trường văn bản được lưu cùng với khối
- ***Open function***: các hàm MATLAB được gọi khi mở khối này
- ***Attributes format string***: Thông số này sẽ mô tả thông số nào được hiển thị dưới icon của khối.

10. Deleting Blocks: Muốn xoá một hay nhiều khối ta chọn khối đó và nhấn phím ***Del***.

11. Thay đổi hướng của khối: Ta có thể xoay hướng của khối bằng vào menu ***Format*** rồi :

- chọn ***Flip Block*** để quay khối 180°.
- chọn ***Rotate Block*** để quay khối 90°.

12. Định lại kích thước của khối: Để thay đổi kích thước của khối ta đưa con trỏ chuột vào một góc của khối rồi bấm và kéo cho đến kích thước mong muốn rồi thả.

13. Xử lí tên khối: Mỗi khối có tên, phải là duy nhất và phải chứa ít nhất một kí tự. Mặc định tên khối nằm dưới khối. Với tên khối ta có thể thực hiện các thao tác sau đây:

- Thay đổi tên khối bằng cách bấm chuột vào tên đã có và nhập lại tên mới. Nếu muốn thay đổi font chữ dùng cho tên khối hãy chọn khối và vào menu ***Format*** và chọn ***Font***.
- Thay đổi vị trí đặt tên khối từ dưới lên trên hay ngược lại bằng cách kéo tên khối tới vị trí mong muốn.
- Không cho hiển thị tên khối bằng cách vào menu Format và chọn Hide Names hay Show Names

14. Hiển thị các thông số bên dưới khối: Ta có thể bắt Simulink hiển thị một hay nhiều thông số bên dưới khối. Để làm điều này ta nhập vào một dòng vào trường ***Attributes format string*** ở hộp thoại ***Block Properties***.

15. Cắt các khối: Để cắt khối khỏi sơ đồ ta bấm phím ***Shift*** và kéo khối đến vị trí mới.

16. Nhập và xuất các vec tơ: Hầu hết các khối chấp nhận đại lượng đầu vào là vec tơ hay vô hướng và biến đổi thành đại lượng đầu ra là vec tơ hay vô hướng. Ta có thể xác định đầu vào nào nhận đại lượng vec tơ bằng cách chọn mục ***Wide Vector Lines*** từ menu ***Format***. Khi tuỳ chọn này được chọn, các đường nhận vec tơ được vẽ đậm hơn các đường mang số liệu vô hướng. Nếu ta thây đổi mô hình sau khi chọn ***Wide Vector Lines*** ta phải cập nhật hình vẽ bằng cách chọn ***Update Diagram*** từ menu ***Edit***. Khởi động lại Simulink cũng cập nhật sơ đồ.

17. Mở rộng vô hướng các đầu vào và các thông số: Mở rộng vô hướng là biến đổi đại lượng vô hướng thành vec tơ với số phần tử không thay đổi. Simulink áp dụng mở rộng vô hướng cho các đại lượng vào và thông số đối với hầu hết các khối.

- Mở rộng đầu vào: khi dùng khối với nhiều đầu vào ta có thể trộn lẫn các đại lượng vec tơ và đại lượng vô hướng .Khi này các đầu vào vô hướng được mở rộng thành vec tơ với số phần tử như của đầu vào vec tơ,các phần tử đều có trị số như nhau
- Mở rộng thông số: ta có thể đặc tả các thông số đối với khối được vec tơ hoá thành đại lượng vec tơ hay đại lượng vô hướng. Khi ta đặc tả các thông số vec tơ, mỗi một phần tử thông số được kết hợp với phần tử tương ứng trong vec tơ đầu vào. Khi ta đặc tả các thông số vec tơ, Simulink áp dụng mở rông vô hướng để biến đổi chúng thành vec tơ có kích thước phù hợp.

18. Gán độ ưu tiên cho khối: Ta có thể gán độ ưu tiên cho khối không nhìn thấy trong mô hình. Khối có độ ưu tiên cao hơn được đánh giá trước khối có độ ưu tiên nhỏ hơn. Ta có thể gán độ ưu tiên bằng cách dùng lệnh tương tác hay dùng chương trình. Để dùng chương trình ta dùng lệnh:

set_param(b,'Priority','n')

Trong đó b là khối và n là một số nguyên, số càng thấp, độ ưu tiên càng cao.
Để gán độ ưu tiên bằng lệnh ta nhập độ ưu tiên vào trường ***Priority*** trong hộp thoại ***Block Priorities*** của khối.

19. Sử dụng Drop Shadows: Ta có thể thêm Drop Shadow vào khối đã chọn bằng cách chọn ***Show Drop Shadow*** từ menu ***Format***

20. Tạo một thư viện: Để tạo một thư viện, chọn ***Library*** từ menu con ***New*** của menu ***File***. Simulink sẽ hiển thị một cửa sổ mới, có tên là ***Library : untitled***.
21. Thay đổi một thư viện đã có: Khi ta mở một thư viện, nó tự động khoá và ta không thể thay đổi các thành phần của nó được. Muốn mở khoá ta chọn ***Unlock*** từ menu ***Edit***.

22. Copy một khối từ thư viện vào mô hình: Ta có thể copy một khối từ thư viện vào mô hình bằng copy hay paste hay kéo nó và thả vào cửa sổ mô hình .

23. Vẽ đường nối giữa các khối: Để nối cổng ra của một khối với cổng vào của một khối khác ta làm như sau:

- đặt con trỏ chuột lên cổng ra của khối đầu tiên, con trỏ có dạng dấu +
- nhấn và giữ chuột
- kéo con trỏ chuột tới cổng vào của khối thứ hai
- thả chuột

Để vẽ đường gấp khúc,nhấn phím ***Shift*** khi vẽ.

24. Vẽ đường nhánh: Đường nhánh là đường nối từ một đường đã có và mang tín hiệu của nó tới cổng vào của một khối.
Để thêm đường nhánh ta làm như sau:

- đưa con trỏ chuột tới đường cần phân nhánh
- nhấn phím chuột đồng thời nhấn phím ***Ctrl***
- kéo con trỏ chuột tới cổng vào tiếp theo và thả chuột va phím ***Ctrl***.

Tuy nhiên ta có thể dùng phím phải chuột thay vì dùng phím ***Ctrl*** và phím trái chuột.

25. Chèn khối vào một đường: Ta có thể chèn một khối vào một đường bằng cách kéo và thả khối đó lên đường nối. Khối mà ta chèn vào chỉ có một đầu vào và một đầu ra.

26. Nhãn của tín hiệu: Ta có thể gán nhãn cho tín hiệu để ghi chú cho mô hình. Nhãn có thể nằm trên hay dưới đường nối nằm ngang, bên phải hay bên trái đường nối thẳng đứng.

27. Sử dụng nhãn tín hiệu: Để tạo nhãn tín hiệu, bấm đúp chuột lên đường nối và ghi nhãn. Để di chuyển nhãn, sửa một nhãn, click lên nhãn rồi đánh nhãn mới sau khi xóa nhãn cũ

28. Ghi chú: Ghi chú là đoạn văn bản cung cấp thông tin về mô hình. Ta có thể thêm ghi chú vào bất kì trông nào của mô hình. Để tạo một ghi chú, nhấn đúp chuột vào vùng trống của mô hình. Khi này trên màn hình xuất hiện một hình chữ nhật có con nháy ở trong. Ta có thể đánh văn bản ghi chú vào khung này. Khi muốn di chuyển phần ghi chú đến một vị trí khác, ta bấm chuột vào đó và kéo đến vị trí mới rồi thả chuột. Để sửa một ghi chú, bấm chuột vào nó để hiển thị khung văn bản và bắt đầu sửa.

29. Các kiểu dữ liệu: Simulink chấp nhận các kiểu dữ liệu sau :

double	số thực với độ chính xác gấp đôi
single	số thực với độ chính xác đơn
int8	số nguyên có dấu 8 bit
uint8	số nguyên không dấu 8 bit
int16	số nguyên có dấu 16 bit
uint16	số nguyên khg dấu 16 bit
int32	số nguyên có dấu 32-bit
uint32	số nguyên không dấu 32-bit

30. Các kiểu dữ liệu của các khối: Các khối đều chấp nhận kiểu dữ liệu double.

31. Mô tả các kiểu dữ liệu dùng cho tham số khối: Khi nhập vào tham số của một khối, kiểu dữ liệu của nó được người dùng mô tả bằng lệnh ***type(value)*** với ***type*** là tên của kiểu dữ liệu và ***value*** là giá trị của tham số.

Ví dụ: single(1.0)	dữ liệu là số thực có trị là 1
int8(2)	dữ liệu là số nguyên có trị là 2
int32(3+2i)	dữ liệu là số phức, phần thực và phần ảo là số nguyên 32 bit

32.Tạo tín hiệu có kiểu dữ liệu được mô tả: Ta có thể đem vào mô hình một tín hiệu có kiểu dữ liệu được mô tả bằng một trong các phương pháp sau đây:

- nạp tín hiệu có kiểu dữ liệu mong muốn từ MATLAB
- tạo một khối hằng và đặt thông số của nó có kiểu dữ liệu mong muốn.
- sử dụng khối biến đổi kiểu dữ liệu

33. Hiển thị các kiểu dữ liệu của cổng: Để hiển thị kiểu dữ liệu của cổng trong mô hình,t a chọn ***Port Data Types*** từ menu ***Format***.

34. Tín hiệu phức: Mặc định, các giá trị của tín hiệu Simulink là số thực. Tuy nhiên các mô hình có thể tạo và xử lí các tín hiệu là số phức. Ta có thể đưa một tín hiệu là số phức vào mô hình bằng một trong các phương pháp sau:

- nạp tín hiệu phức từ MATLAB
- tạo một khối hằng trong mô hình và cho nó giá trị phức.
- tạo một tín hiệu thực tương ứng với phần thực và phần ảo của tín hiệu phức và kết hợp các phần này thành tín hiệu phức bằng cách sử dụng khối biến đổi tín hiệu thực-ảo thành tín hiệu phức.

Ta có thể xử lí tín hiệu phức nhờ các khối chấp nhận tín hiệu phức. Phần lớn các khối của Simulink chấp nhận tín hiệu vào là số phức.

35. Tạo một hệ thống con bằng cách thêm khối hệ thống con: Để tạo một khối hệ thống con trước khi thêm các khối trong nó ta phải thêm khối hệ thống con vào mô hình rồi thêm các khối tạo nên hệ thống con này vào khối hệ thống con bằng cách sau:

- copy khối hệ thống con từ thư viện Signal & System vào mô hình
- mở khối hệ thống con bằng cách click đúp lên nó
- trong cửa sổ khối con rỗng, tạo hệ thống con. Sử dụng các khối inport để biểu diễn đầu vào và các khối outport để biểu diễn đầu ra.

36. Tạo hệ thống con bằng cách nhóm các khối đã có: Nếu mô hình của ta đã có một số khối mà ta muốn nhóm thành khối hệ thống con thì ta có thể nhóm các khối này thành khối hệ thống con bằng sau:

- bao các khối và đường nối giữa chúng bằng một đường đứt nét(bấm chuột và kéo từ góc này đến góc kia của các khối) rồi thả chuột
- chọn ***Create Subsystem*** từ menu ***Edit***

37. Gán nhãn cho các cổng của hệ thống con: Simulink gán nhãn cho các cổng của hệ thống con. Nhãn là tên của các khối inport và outport nối khối hệ thống con với các khối bên ngoài qua các cổng này. Ta có thể dấu các nhãn này bằng cách chọn khối hệ thống con rồi chọn ***Hide Port Labels*** từ menu ***Format***. Ta cũng có thể dấu một hay nhiều nhãn bằng cách chọn các khối inport hay outport thích hợp trong khối hệ thống con và chọn ***Hide Name*** từ menu ***Format***

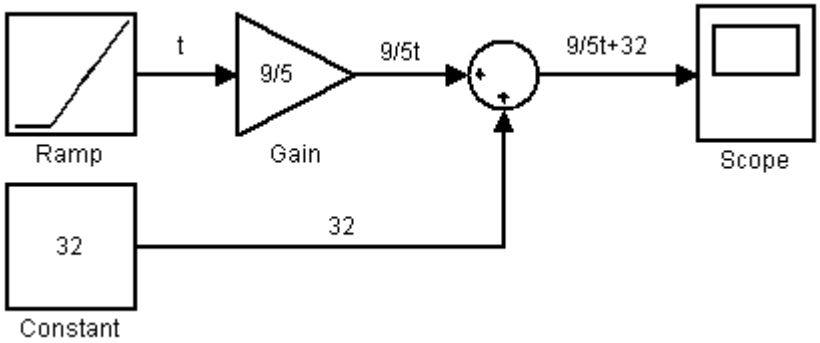
38. Mô phỏng một phương trình: Phương trình dùng để biến đổi độ Celcius thành độ Fahrenheit là :

$$TF = (9/5)TC + 32$$

Trước hết ta khảo sát các khối cần để tạo mô hình:

- khối ***ramp*** trong thư viện ***Sources*** để input tín hiệu nhiệt độ
- khối ***Constant*** trong thư viện ***Sources*** để tạo hằng số 32
- khối ***Gain*** trong thư viện ***Math*** để tạo ra hệ số 9/5
- khối ***Sum*** trong thư viện ***Math*** để cộng hai đại lượng
- khối ***Scope*** trong thư viện ***Sinks*** để hiển thị kết quả.

Tiếp đó ta đưa các khối vào cửa sổ mô hình, gán các giá trị thông số cho ***Gain*** và ***Constant*** bằng cách nhấp đúp lên chúng để mở khối. Sau đó ta nối các khối. Khối ***Ramp*** đưa nhiệt độ Celcius và mô hình. Mở khối này và thay đổi giá trị khởi gán ***Initial output*** về 0. Khối ***Gain*** nhân nhiệt độ này với hệ số 9/5. Khối ***Sum*** cộng giá trị 32 với kết quả và đưa ra nhiệt độ Fahrenheit. Khối ***Scope*** để xem kết quả. Sơ đồ mô phỏng như sau. Bây giờ ***Start*** từ menu ***Simulation*** để chạy simulation. Simulation chạy 10 giây,tương ứng với nhiệt độ Celcius biến đổi từ 0 đến 10°.

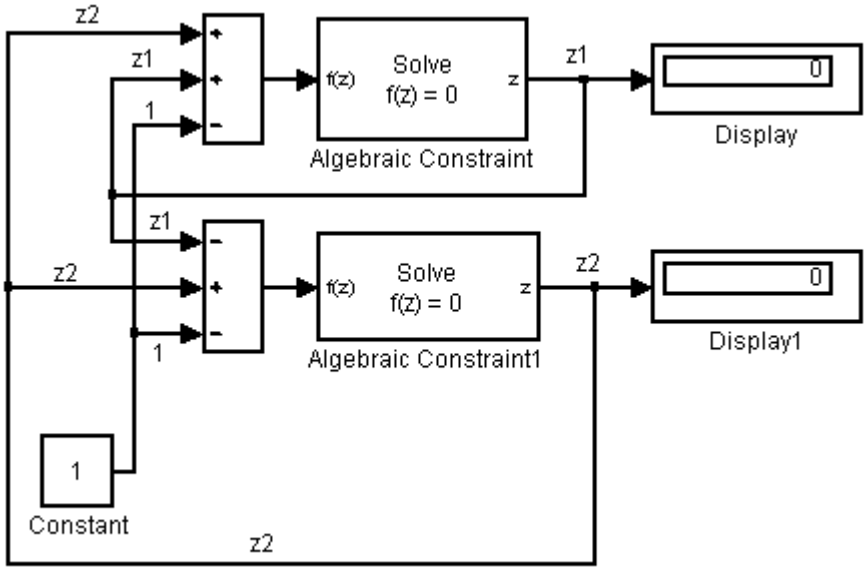


39. Mô phỏng một hệ phương trình tuyến tính: Ta xét hệ phương trình tuyến tính có hai ẩn:

$$\begin{cases} z_1 + z_2 = 1 \\ -z_1 + z_2 = 1 \end{cases}$$

Để mô phỏng ta dùng các khối:

- hai khối ***Algebric Constraint*** trong thư viện ***Math*** để giải phương trình
- hai khối ***Sum*** trong thư viện ***Math*** để tạo phép tính
- hai khối ***Display*** trong thư viện ***Sink*** để hiện thị giá trị nghiệm
- khối ***Constant*** trong thư viện ***Sources*** để tạo giá trị 1



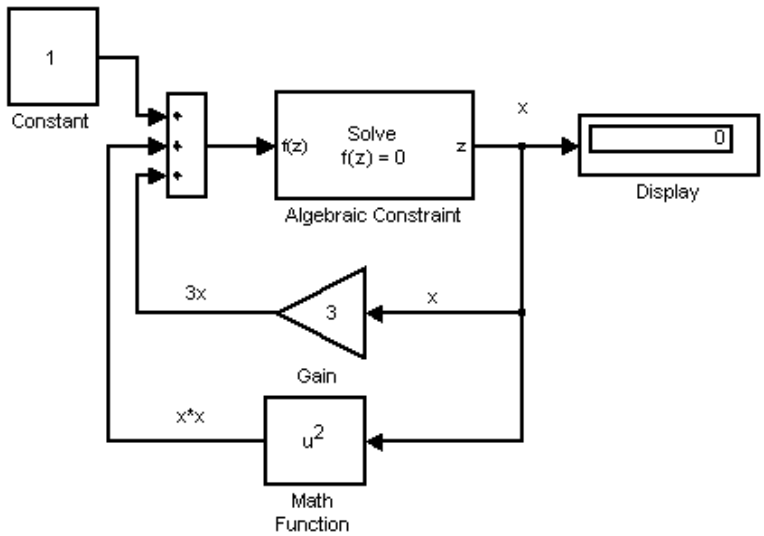
40. Mô phỏng một phương trình bậc cao: Ta xét phương trình :

$$x^2 + 3x + 1 = 0$$

Để mô phỏng ta dùng các khối:

- khối ***Algebric Constraint*** trong thư viện ***Math*** để giải phương trình
- khối ***Display*** trong thư viện ***Sink*** để hiển thị trị số của nghiệm
- khối ***Constant*** trong thư viện ***Sources*** để tạo giá trị 1
- khối ***Sum*** trong thư viện ***Math*** để tạo phép cộng
- khối ***Math*** Function trong thư viện ***Math*** để tạo hàm x^2
- khối ***Gain*** trong thư viện ***Math*** để tạo hệ số 3

Sơ đồ mô phỏng như sau



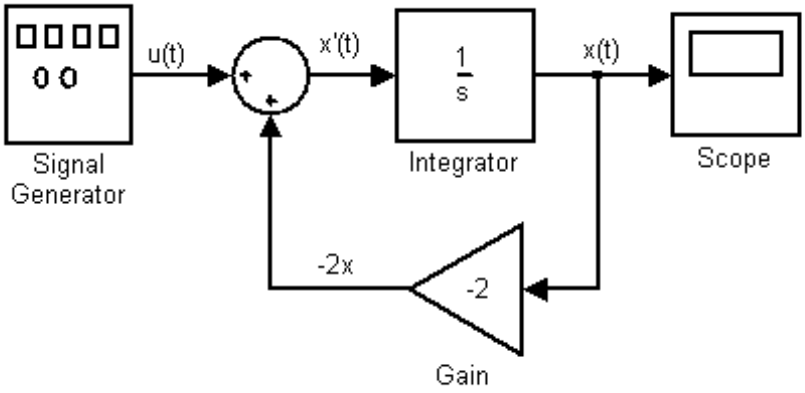
41. Mô phỏng hệ thống liên tục đơn giản: Ta mô hình hoá hệ mô tả bởi phương trình vi phân

$$x'(t) = -2x(t) + u(t)$$

với u(t) là một sóng hình chữ nhật có biên độ bằng 1 và tần số 1 rad/s. Để mô phỏng hệ ta dùng các khối:

- khối ***Gain*** trong thư viện ***Math*** để tạo hệ số 2
- khối ***Sum*** trong thư viện ***Math*** để tạo phép tính
- khối ***Scope*** trong thư viện ***Sink*** để xem kết quả
- khối ***Signal Generator*** trong thư viện ***Sources*** để tạo nguồn
- khối ***Integrator*** trong thư viện ***Continuous*** để tích phân

Sơ đồ mô phỏng như sau:



42. Mô phỏng hệ phương trình vi phân bậc cao: Ta xét hệ mô tả bởi phương trình vi phân bậc hai sau:

$$\frac{d^2x}{dt^2} + 3\frac{dx}{dt} + 2x(t) = 4u(t)$$

Trong đó u(t) là hàm bước nhảy,x′(0) = 0 và x(0) = 0. Biến đổi Laplace của hệ cho ta:

$$p^2X(p) + 3pX(p) + 2X(p) = 4U(p)$$

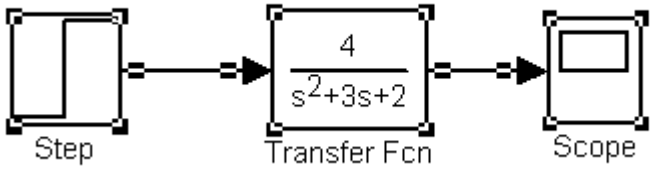
Hàm truyền của hệ là:

$$T(p) = \frac{4}{p^2 + 3p + 2}$$

Ta mô phỏng hệ bằng các phần tử:

- khối ***Step*** trong thư viện ***Sources*** để tạo hàm bước nhảy u(t)
- khối ***Transfer Fcn*** trong thư viện ***Continuous*** để tạo hàm truyền
- khối ***Scope*** trong thư viện ***Sink*** để xem kết quả

Sơ đồ mô phỏng như sau:



43. Mô phỏng hệ có điều kiện đầu khác không:

a. Phương trình vi phân cấp 1: Ta xét hệ mô tả bởi phương trình :

$$\frac{dx}{dt} + x(t) = 0$$

Điều kiện đầu của hệ là x(0) = 1. Ta cần tìm x(t) trong đoạn $0 \le t \le 10$s. Do điều kiện đầu khác không nên ta biến đổi phương trình về dạng không gian- trạng thái.

$$\begin{cases} \dfrac{dx}{dt} = Ax + B \\ y = Cx + Du \end{cases}$$

Trong đó x là biến trạng thái,u là tín hiệu vào,y là tín hiệu ra.

Chọn y(t) = x(t) ta có :

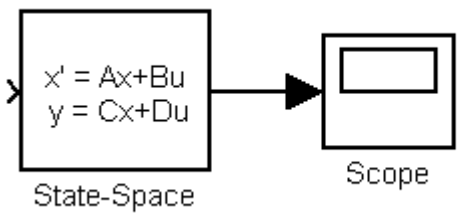
$$\frac{dx}{dt} = -x(t)$$

$$y(t) = x(t)$$

Như vậy A = -1 ; C = 1 ; u(t) = 0 ; B = 0 và D = 0. Sơ đồ mô phỏng gồm các phần tử:

- khối ***State-Space*** trong thư viện ***Continuous***
- khối ***Scope*** trong thư viện ***Sink***

Sơ đồ mô phỏng như sau:



b. Phương trình vi phân cấp cao: Ta xét hệ mô tả bởi phương trình:

$$\frac{d^2x}{dt^2} + 3\frac{dx}{dt} + 2x(t) = 4u(t)$$

Trong đó u(t) là hàm đơn vị, x(0) = 1 và x′(0) = -2.

Ta cũng dùng hệ không gian-trạng thái. Ta đặt $x_1 = x$, $x_2 = \frac{dx_1}{dt}$. Như vậy điều kiện đầu là: $x_1(0) = 1$ và $x_2(0) = -2$. Ngoài ra $\frac{dx_2}{dt} = \frac{d^2x_1}{dt} = \frac{d^2x}{dt}$

$$\frac{dx_2}{dt} + 3x_2(t) + 2x_1(t) = 4u(t)$$

Phương trình cấp hai được đưa về hai phương trình cấp 1:

$$\begin{cases} \frac{dx_1}{dt} = x_2(t) \\ \frac{dx_2}{dt} = -3x_2(t) - 2x_1(t) + 4u(t) \\ \frac{dx_2}{dt} \end{cases}$$

Viết dưới dạng ma trận ta có:

$$\begin{bmatrix} \frac{dx_1}{dt} \\ \frac{dx_2}{dt} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 4 \end{bmatrix} u(t)$$

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$$

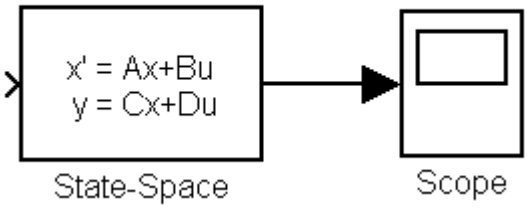
Từ hệ này ta suy ra các ma trận của hệ không gian-trạng thái là:

$$A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} \; B = \begin{bmatrix} 0 \\ 4 \end{bmatrix} \; C = \begin{bmatrix} 1 & 0 \end{bmatrix} \; D = 0$$

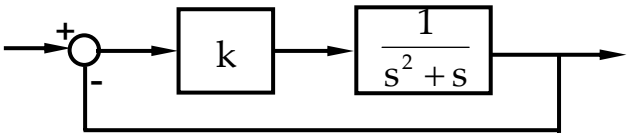
Sơ đồ mô phỏng gồm các khối sau:

- khối ***State-Space*** trong thư viện ***Continuous***
- khối ***Scope*** trong thư viện ***Sink***

Sơ đồ mô phỏng như sau



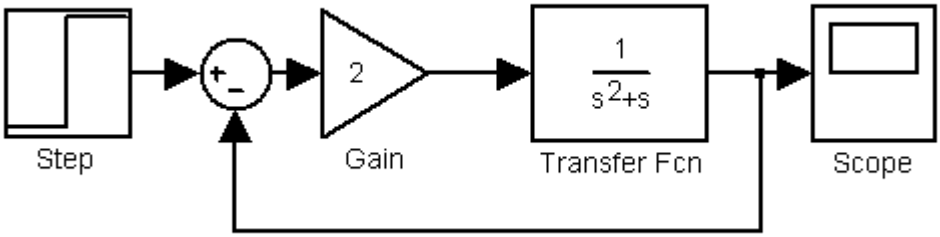
44. Mô phỏng hệ cho bởi sơ đồ khối:Xét một hệ có cấu trúc sơ đồ khối như sau:



Ta mô phỏng hệ bằng các phần tử:

- khối ***Step*** trong thư viện ***Sources***
- khối ***Gain*** trong thư viện ***Math***
- khối ***Transfer Fcn*** trong thư viện ***Continuous***

Sơ đố mô phỏng như sau



45. Mô hình hoá hệ phi tuyến:

a. Hệ cho bởi phương trình vi phân cấp cao: Ta xét phương trình Val der Pol:

$$y'' - (1 - y^2)y' + y = 0$$

Điều kiện đầu y(0) = 2 và y′(0) = 0

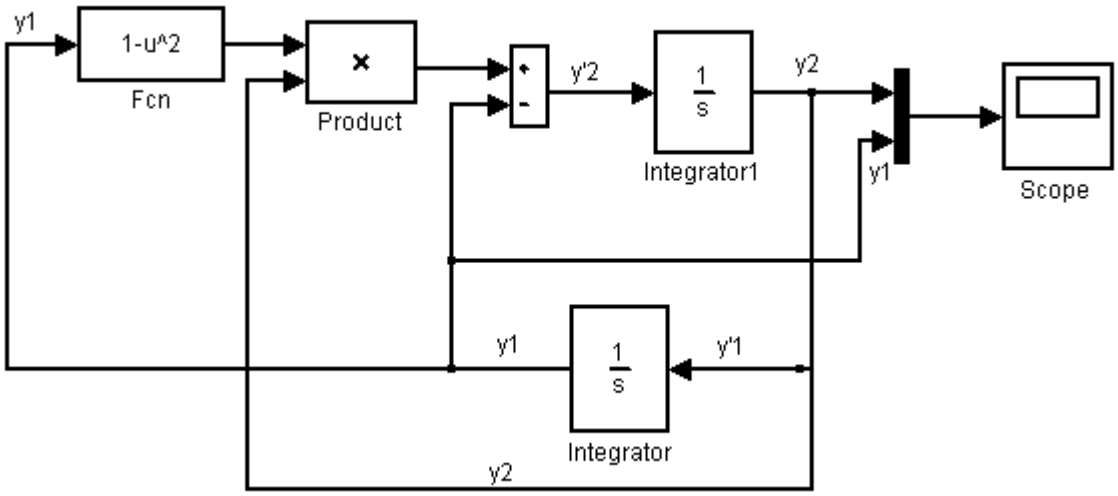
Ta đặt $y = y_1$ và $y' = y_2$ và có được hệ phương trình vi phân cấp 1:

$$\begin{cases} y_1' = y_2 \\ y_2' = (1 - y_1^2)\ y_2 - y_1 \end{cases}$$

Hệ phương trình được mô phỏng bằng các phần tử sau:

- khối hàm ***Fcn*** trong thư viện ***Functions & Tables*** để tạo hàm
- khối ***Product*** trong thư viện ***Math*** để tạo phép nhân
- hai khối ***Integrator*** trong thư viện ***Continous***
- khối ***Sum*** trong thư viện ***Math***
- khối ***Mux*** trong thư viện ***Signal & Systems*** để trộn tín hiệu
- khối ***Scope*** trong thư viện ***Sink*** để xem kết quả.

Sơ đồ mô phỏng như sau:



b. Hệ mô tả bằng hệ phương trình vi phân: Ta xét hệ mô tả bằng hệ phương trình vi phân sau:

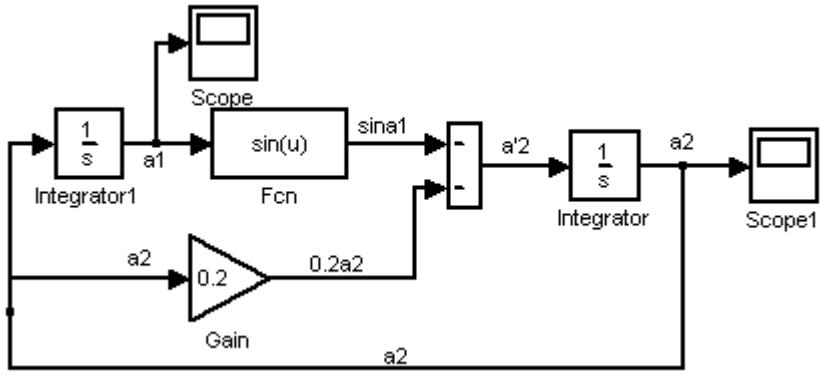
$$\begin{cases} a_1' = a_2 \\ a_2' = -\sin(a_1) - 0.2a_2 \end{cases}$$

với điều kiện đầu là $a_1(0) = a_2(0) = 1.3$

Ta mô phỏng hệ bằng các phần tử:

- hai khối ***Integrator*** trong thư viện ***Continous***
- khối ***Fcn*** trong thư viện ***Functions & Tables***
- khối ***Gain*** trong thư viện ***Math***
- hai khối ***Scope*** trong thư viện ***Sink***
- khối ***Sum*** trong thư viện ***Math***

Sơ đồ mô phỏng như sau:



46. Lưu mô hình: Ta có thể lưu mô hình bằng cách chọn ***Save*** hay ***Save as*** từ menu ***File***.Ta dùng ***Save*** khi mở mô hình cũ, sửa và lưu lại. ***Save as*** dùng khi mô hình có ten là untitled nghĩa là chưa được đặt tên. Simulink sẽ lưu mô hình bằng một file có tên và phần mở rộng là .mdl.

47. In sơ đồ khối: Ta có thể in sơ đồ khối bằng cách chọn ***Print*** từ menu ***File***. Khi này hộp thoại Print sẽ xuất hiện. Nó cho phép ta :

- chỉ in hệ thống hiện hành
- in hệ thống hiện hành và các hệ thống dưới nó trong phân lớp mô hình
- in hệ thống hiện hành và các hệ thống trên nó trong phân lớp mô hình
- in tất cả các hệ thống trong mô hình
- in mỗi mô hình một khung overlay

48. Duyệt qua mô hình: Cửa sổ ***Model Browser*** cho phép ta :

- duyệt qua mô hình có phân lớp
- mở các hệ thống trong các mô hình
- xác định nội dung các khối trong một mô hình

Để hiển thị ***Model Browser***, chọn nó từ menu ***View***. Cửa sổ xuất hiện được chia làm 2 phần. Phía trái là Browser. Cấu trúc cây của mô hình hiển thị ở bên phải. Mỗi dấu + tương ứng với một hệ thống con.

§2. SYMBOLIC MATLAB TOOLBOX

1. Khái niệm chung: ***Symbolic Math Toolboxes*** kết hợp tính toán bằng chữ vào môi trường MATLAB. Các toolbox này bổ sung các tiện ích số và đồ thị với các kiểu tính toán toán học khác nhau.

Tiện ích	Nội dung
Calculus	đạo hàm, tích phân, giới hạn, tổng và chuỗi Taylor
Linear Algebra	nghịch đảo, định thức,giá trị riêng, phân tích và dạng chính tắc của ma trận.
Simplification	phương pháp rút gọn các biểu thức đại số
Solution of Equations	giải bằng chữ và bằng số các phương trình đại số và vi phân
Variable-Precision Arithmetic	đánh giá độ chính xác của các biểu thức đại số
Transform	biến đổi Laplace, Fourrier và z
Special Mathematical Function	các hàm toán học đặc biệt của các ứng dụng toán học kinh điển

Động lực tính toán nằm dưới các toolbox là nhân Maple, một hệ thống tính toán được phát triển đầu tiên ở trường đại học Waterloo, Canada và sau đó tại Eidgenroessiche Technische Hochschule Zurich, Thuỵ sĩ. Maple được thương mại hoá và hỗ trợ của công ty Waterloo Maple.

2. Khởi động TOOLBOX:

a. Các đối tượng chữ: Trong phần này chúng ta sẽ xem xét cách tạo và dùng các đối tượng chữ. Chúng ta cũng sẽ xem xét các biến chữ mặc định. ***Symbolic Math Toolbox*** định nghĩa một kiểu dữ liệu MATLAB mới gọi là đối tượng chữ hay sym. Bên trong, một đối tượng chữ là một cấu trúc số liệu mà nó lưu biểu diễn chuỗi các kí tự. ***Symbolic Math Toolbox*** dùng các đối tượng chữ để biểu diễn các biến chữ, các biểu thức chữ, các ma trận chữ.

b. Tạo các biến và các biểu thức chữ: Lệnh ***sym*** cho phép ta xây dựng các biến và các biểu thức chữ. Ví dụ lệnh:

```
x = sym('x')
a = sym('alpha')
```

tạo ra các biến chữ là x và a với x là x và a là alpha.

Giả sử ta muốn ta muốn dùng biến chữ để biểu diễn tỉ lệ vàng $\rho = \frac{1+\sqrt{5}}{2}$. Ta dùng lệnh:

```
rho = sym('(1 + sqrt(5))/2')
```

Bây giờ ta có thể thực hiên các phép toán khác nhau với rho. Ví dụ:

```
f = rho^2 - rho - 1
f =
        (1/2+1/2*5^(1/2))^2-3/2-1/2*5^(1/2)
```

Ta rút gọn biểu thức:

```
simplify(f)
ans =
        0
```

Bây giờ giả sử ta muốn giải phương trình bậc 2 $f = ax^2 + bx + c$. Phát biểu:

```
f = sym('a*x^2 + b*x + c')
```

gán biểu thức chữ $ax^2 + bx + c$ cho biến f. Tuy nhiên trong trường hợp này ***Symbolic Math Toolbox*** không tạo ra các biến tương ứng với các số hạng a, b, c và x trong biểu thức. Để thực hiện các phép toán bằng chữ(ví dụ tích phân, đạo hàm, thay thế v.v) trên f ta phải tạo các biến một cách rõ ràng, nghĩa là cần viết:

```
a = sym('a')
b = sym('b')
c = sym('c')
x = sym('x')
```

hay đơn giản là:

```
syms a b c x
```

Nói chung là ta có thể dùng ***sym*** hay ***syms*** để tạo các biến chữ nhưng nên dùng ***syms*** để tiết kiệm thời gian.

2. Biến đổi giữa số và chữ:

a. Tạo các biến thực và phức: Lệnh ***sym*** cho phép ta mô tả các thuộc tính toán học của các biến chữ bằng cách dùng tuỳ chọn ***real***. Phát biểu:

```
x = sym('x','real');
y = sym('y','real');
```

hay hiệu quả hơn:

```
syms x y real
z = x + i*y
```

tạo ra biến chữ x và y có thuộc tính là số thực. Đặc biệt:

```
f = x^2 + y^2
```

thực sự là số không âm. Như vậy z là biến phức và các lệnh:

```
conj(x)
conj(z)
expand(z*conj(z))
```

cho kết quả:

return the complex conjugates of the variables

```
x
x - i*y
x^2 + y^2
```

Lệnh ***conj*** là toán tử tạo số phức liên hợp. Để xóa thuộc tính ***real*** của x ta dùng lệnh:

```
syms x unreal
```

hay:

```
x = sym('x','unreal')
```

Lệnh ***clear x*** không xoá thuộc tính số ***real*** của x.

b. Tạo các hàm trừu tượng: Nếu ta muốn tạo một hàm trừ tượng(nghĩa là một hàm không xác định) f(x) cần dùng lệnh:

```
f = sym('f(x)')
```

Khi này f hoạt động như là f(x) và có thể xử lí bằng các lệnh toolbox. Ví dụ để tính vi phân bậc 1 ta viết:

```
df = (subs(f,'x','x+h') – f)/'h'
```

hay

```
syms x h
df = (subs(f,x,x+h)–f)/h
```

trả về:

```
df =
        (f(x+h)-f(x))/h
```

ứng dụng này của hàm ***sym*** sẽ rất hữu ích trong biến đổi Fourrier, Laplace và z.

c. Dùng sym để truy cập các hàm của Maple: Ta có thể truy cập hàm giai thừa k! của ***Maple*** khi dùng ***sym***.

```
kfac = sym('k!')
```

Để tính 6! hay k! ta viết:

```
syms k n
subs(kfac,k,6)
ans =
        720
subs(kfac,k,n)
```

ans =

n!

hay nếu tính 12! ta cũng có thể viết:

prod(1:12)

d. Ví dụ tạo ma trận chữ: Một ma trận vòng là ma trận mà hàng sau có được bằng cách dịch các phần tử của hàng trước đi 1 lần.Ta tạo một ma trận vòng A bằng các phần tử a, b và c:

syms a b c
A = [a b c; b c a; c a b]
kết quả:
A =

[*a, b, c*]
[*b, c, a*]
[*c, a, b*]

Do A là ma trận vòng tổng mỗi hàng và cột như nhau:

sum(A(1,:))
ans =

a+b+c
sum(A(1,:)) = = sum(A(:,2))
ans =

1

Bây giờ ta thay A(2, 3) bằng beta và b bằng alpha:

syms alpha beta
A(2,3) = beta;
A = subs(A,b,alpha)
A =

[*a, alpha, c*]
[*alpha, c, beta*]
[*c, a, alpha*]

Từ ví dụ này ta thấy dùng các đối tượng chữ cũng tượng tự như dùng số trong MATLAB.

e. Biến chữ mặc định: Khi dùng các hàm toán học,việc chọn các biến độc lập thường rất rõ ràng. Ví dụ xem bảng sau:

Hàm toán học	Lệnh MATLAB
f = xn	f = x^n
g = sin(at+b)	g = sin(a*t+b)
h = Jv(z)	h = besselj(nu,z)

Nếu ta tìm đạo hàm của các hàm này nhưng không mô tả biến độc lập (nghĩa là đạo hàm theo biến nào) thì kết quả là:

f' = nxn^{-1}

g' = acos(at + b)

h' =J v (z)(v/z)-Jv_{+1}(z).

Như vậy các biến độc lập là x, t và z. MATLAB hiểu các biến độc lập là các chữ thường và nằm ở cuối bảng chữ cái như x, y, z. Khi không thấy các chữ cái này, MATLAB sẽ tìm chữ gần nhất và coi đó là biến độc lập. Các biến khác như n, a, b và v được coi là hằng hay thông số. Tuy nhiên ta có thể lấy đạo hàm của f theo n bằng cách viết rõ biến độc lập ra. Ta dùng các lệnh sau để tạo ra các hàm:

```
syms a b n nu t x z
f = x^n;
g = sin(a*t + b);
```

Để đạo hàm hàm f ta viết:

```
diff(f);
ans =
        x^n*n/x
```

Trong ví dụ trên x là biến độc lập. Nếu muốn tính đạo hàm của f theo n ta cần viết:

```
diff(f,n)
ans =
        x^n*log(x)
```

4. Tạo các hàm toán học bằng chữ:

a. Dùng các biểu thức chữ: Các lệnh:

```
syms x y z
r = sqrt(x^2 + y^2 + z^2)
t = atan(y/x)
f = sin(x*y)/(x*y)
```

tạo ra các biểu thức chữ r, t và f. Ta có thể dùng các lệnh ***diff***, ***int***, ***subs*** hay các lệnh ***Symbolic Math Toolbox*** khác để xử lí các biểu thức như vậy.

b. Tạo các M-file: M-file cho phép ta dùng các hàm tổng quát hơn. Ví dụ ta muốn tạo ra hàm sinc = sin(x)/x ta sẽ viết một M-file có nội dung như sau:

```
function z = sinc(x)
if isequal(x, sym(0))
   z = 1;
else
   z = sin(x)/x;
end
```

Ta có thể mở rộng các ví dụ như vậy cho các hàm và biến khác nhau.

5. Tính toán:

a. Đạo hàm: Ta tạo biểu thức chữ:

```
syms a x
f = sin(a*x)
```

Vậy thì:

```
df = diff(f)
```

tính đạo hàm của hàm f(x) theo x. Kết quả là:

```
df =
        cos(a*x)*a
```

Để tính đạo hàm của f theo a ta viết:

```
dfa = diff(f,a)
```

kết quả:

```
dfa=
        cos(a*x)*x
```

Hàm toán học	Lệnh MATLAB
f = xn f' = nxn^{-1}	f = x^n diff(f) hay diff(f, x)
g = sin(at+b) g' = acos(at+b)	g = sin(a*t+b) diff(g) hay diff(g, t)

Để tính đạo hàm bậc 2 của f theo x và a ta viết:

```
diff(f,2)
ans =
        - sin(a*x)*a^2
diff(f,x,2)
ans =
        - sin(a*x)*x^2
```

Hàm ***diff*** có thể dùng đối số là ma trận. Trong trường hợp này đạo hàm được thực hiện trên từng phần tử. Ví dụ:

```
syms a x
A = [cos(a*x),sin(a*x);-sin(a*x),cos(a*x)]
```

kết quả:

```
A =
[ cos(a*x),   sin(a*x)]
[-sin(a*x),   cos(a*x)]
```

lệnh :

```
dy = diff(A)
```

cho kết quả:

```
dy =
[ -sin(a*x)*a,        cos(a*x)*a]
[ -cos(a*x)*a,        -sin(a*x)*a]
```

Ta khảo sát biến đổi từ toạ độ Euclid(x, y, z) sang toạ độ cầu (r, λ, φ) thực hiện bằng các công thức:

$x = r\cos\lambda\cos\varphi$

$y = r\cos\lambda\sin\varphi$

$z = r\sin\lambda$

Để tính ma trận Jacobi J của phép biến đổi này ta dùng hàm ***jacobian***. Định nghĩa toán học của J là:

$$J = \frac{\partial(x, y, z)}{\partial(r, \lambda, \varphi)}$$

Để dễ viết ta dùng kí tự l thay cho λ và f thay cho φ. Các lệnh

```
syms r l f
x = r*cos(l)*cos(f);
```

```
y = r*cos(l)*sin(f);
z = r*sin(l);
J = jacobian([x; y; z], [r l f])
```

cho ta kết quả:

```
J =
    [ cos(l)*cos(f),    –r*sin(l)*cos(f),    –r*cos(l)*sin(f) ]
    [ cos(l)*sin(f),    –r*sin(l)*sin(f),    r*cos(l)*cos(f)]
    [ sin(l),             r*cos(l),                    0]
```

và lệnh:

```
detJ = simple(det(J))
```

cho:

```
detJ =
    –cos(l)*r^2
```

Chú ý là đối số thứ nhất của hàm ***jacobian*** phải là vec tơ cột và đối số thứ hai là vec tơ hàng. Hơn nữa do định thức của ma trận Jacobian là biểu thức lượng giác khá phức tạp nên ta dùng lệnh ***simple*** để thay thế và rút gọn.
Bảng sau tổng hợp hàm ***diff*** và hàm ***jacobian***

Toán tử toán học	Lệnh MATLAB
f = exp(ax + b)	syms a b x f = exp(a*x + b)
$\frac{df}{dx}$	diff(x) hay diff(f,x)
$\frac{df}{da}$	diff(f,a)
$\frac{d^2f}{d^2a}$	diff(f,a,2)
$r = u^2 + v^2$ t = arctan(v/u)	syms r t u v r = u^2 + v^2 t = atan(v/u)
$J = \frac{\partial(r,t)}{\partial(u,v)}$	J = jacobian([r ; t],[u , v])

b. Giới hạn: Đạo hàm của một hàm là giới hạn sau đây nếu nó tồn tại :

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Symbolic Math Toolbox cho phép giới hạn của một hàm một cách trực tiếp hơn. Lệnh:

```
syms h n x
dc = limit( (cos(x+h) – cos(x))/h, h, 0 )
```

cho kết quả:

```
dc =
        –sin(x)
```

và:

```
limit( (1 + x/n)^n,n,inf )
```

cho:

```
ans =
exp(x)
```

minh hoạ 2 trong số các giới hạn quan trọng của toán học: đạo hàm(trong trường hợp cosx) và hàm mũ. Trong khi nhiều giới hạn :

$$\lim_{x \to a} f(x)$$

là "hai phía"(nghĩa là kết quả như nhau cho dù x tiến tới bên phải hay bên trái của a) lại có những hàm giới hạn phải và trái khác nhau. Do đó 3 giới hạn:

$$\lim_{x \to 0} \frac{1}{x} \ , \ \lim_{x \to -0} \frac{1}{x} \ , \ \lim_{x \to +0} \frac{1}{x}$$

cho 3 kết quả khác nhau: không xác định , -∞ và +∞

Trong trường hợp không tồn tại giới hạn Symbolic Math Toolbox trả về kết quả NaN. Ví dụ:

limit(1/x, x, 0)

cho:

ans =

NaN

Lệnh:

limit(1/x, x, 0, 'left')

cho:

ans =

–inf

Lệnh:

limit(1/x,x,0,'right')

cho:

ans =

inf

Như vậy limit(f) tương đương với limit(f,x,0). Bảng sau cho các giới hạn:

Hàm toán học	Lệnh MATLAB
$\lim_{x \to 0} f(x)$	limit(f)
$\lim_{x \to a} f(x)$	limit(f, x, a) hay limit(f, a)
$\lim_{x \to -a} f(x)$	limit(f, x, a, 'left')

$\lim_{x \to +a} f(x)$	limit(f, x, a, 'right')

c. Tích phân: Nếu f là một biểu thức chữ thì ***int(f)*** tìm một biểu thức khác F sao cho diff(F) = f. Như vậy ***int(f)*** cho ta tích phân bất định của f. Tương tự như đạo hàm ***int(f, v)*** lấy tích phân theo biến độc lập v. Ta có bảng sau:

Hàm toán học	Lệnh MATLAB
$\int x^n dx = \frac{x^{n+1}}{n+1}$	int(x^n) hay int(x^n, x)
$\int_0^{\frac{\pi}{2}} \sin(2x)dx = 1$	int(sin(2*x), 0, pi/2) hay int(sin(2*x), x, 0, pi/2)
g = cos(at+b) $\int g(t)dt = \frac{1}{a}\sin(at+b)$	g = cos(a*t + b) int(g) hay int(g, t)

Khi MATLAB không tìm được tích phân nó viết lại lệnh đã nhập vào. Ví dụ:

```
syms x
f = exp(–(k*x)^2);
int(f, x);
ezplot(f)
```

tính tích phân bất định của hàm. Để tính tích phân xác định ta viết:

```
syms x
f = exp(–(k*x)^2);
a = int(f, x, 0, 1);
a = double(a)
```

d. Tính tổng: Ta có thể tính tổng biểu thức chữ khi chúng tồn tại bằng cách dùng lệnh ***symcum***. Ví dụ chuỗi:

$$1 + \frac{1}{2^2} + \frac{1}{3^2} + \cdots$$

cho tổng là $\pi^2/6$ còn chuỗi :

$$1 + x^2 + x^3 + \ldots$$

cho tổng là 1/(1-x). Các tổng được tính như sau:

```
syms x k
s1 = symsum(1/k^2, 1, inf)
```

s2 = symsum(x^k, k, 0, inf)

e. Chuỗi Taylor: Cho hàm f(x). Phát biểu:

T = taylor(f, 8)

cho kết quả:

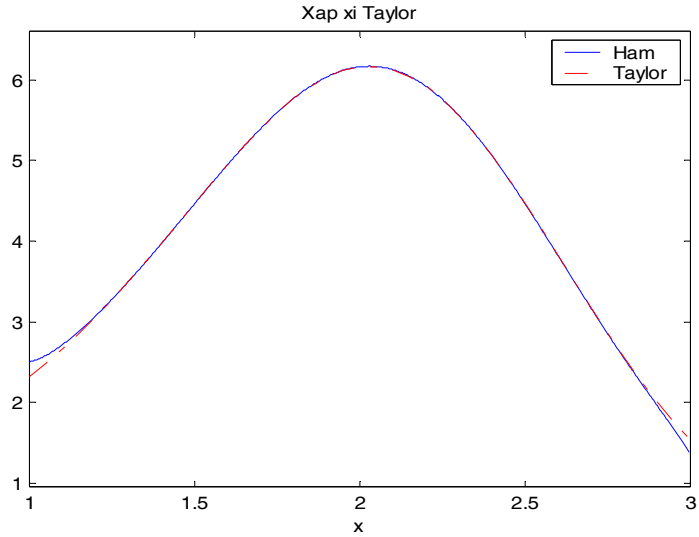
T =
*1/9+2/81*x^2+5/1458*x^4+49/131220*x^6*

là khai triển Taylor của f(x) lân cận x = 0(khai triển MacLaurin) có chứa 8 số hạng khác 0. Phát biểu:

```
syms x
g = exp(x*sin(x))
t = taylor(g,12,2)
```

tạo ra khai triển Taylor của f(x) tại x = 2 và chứa đến 12 số hạng khác 0. Ta vẽ các hàm này lên cùng một đồ thị để thấy được khả năng xấp xỉ của chuỗi Taylor với hàm thực g:

```
xd = 1:0.05:3;
yd = subs(g,x,xd);
ezplot(t, [1,3]);
hold on;
plot(xd, yd, 'r-.')
title('Xap xi Taylor ');
legend('Ham','Taylor')
```



Tiếp đó 1 ức toán học dễ đọc.

6. Rút gọn biêu thức: Ta xét 3 biêu thức khác nhau:

syms x
f = x^3-*6*x^2+11*x-6*
g = (x-1)(x-2)*(x-3)*
h = x(x*(x-6)+11)-6*

Thực hiện các lệnh *pretty(f), pretty(g), pretty(h)* ta nhận được:

$f = x^3 - 6x^2 + 11x - 6$
$g = (x - 1)(x - 2)(x - 3)$
$h = x(x(x - 6) + 11) - 6$

Cả 3 biểu thức này là các dạng biểu diễn toán học khác nhau của cùng một hàm toán học - đó là đa thức bậc 3 theo x. Mỗi một dạng thích hợp với một dạng tính toán. Dạng thứ nhất f là dạng chung nhất thường được dùng biểu diễn đa thức. Nó đơn giản là một tổ hợp tuyến tính của các số mũ của x. Dạng thứ 2, hàm g, là dạng phân tích thành thừa số. Nó biểu diễn nghiệm của đa thức. Tuy nhiên không phai đa thức nào cũng có nghiệm, nghĩa là có thể phân tích thành thừa số. Dạng thứ 2 là dạng Horner của đa thức. Nó rất tiện dùng để tính trị số của đa thức tại một giá trị nào đó của x.

Symbolic Math Toolbox cung cấp một số hàm dùng để biến đổi các biểu thức đại số và lượng giác thành các biểu thức đơn giản hơn. Chúng gồm: ***collect***, ***expand***, ***horner***, ***factor***, ***simplify***, và ***simple***.

a.collect: Phát biểu:

collect(f)

xem f như một đa thức gồm các biến chữ x và gộp tất cả các hệ cùng bậc của x. Đối số thứ 2 của chỉ rõ biến định gộp nếu có nhiều iến trong biểu thưc. Sau đây là một số ví dụ:

f	collect(f)
(x - 1)(x - 2)(x - 3)	x^3 - 6*x^2 + 11*x - 6
x*(x*(x - 6) + 11) - 6	x^3 - 6*x^2 + 11*x - 6
(1 + x)*t + x*t	2*x*t + t

b.expand: Phát biểu:

expand(f)

khai triển biểu thức. Sau đây là một số ví dụ:

f	expand(f)
a*(x + y)	a*x + a*y
(x - 1)*(x - 2)*(x - 3)	x^3 - 6*x^2 + 11*x - 6
x*(x*(x - 6) + 11) - 6	x^3 - 6*x^2 + 11*x - 6
exp(a + b)	exp(a) + exp(b)
cos(x + y)	cos(x)*cos(y) - sin(x)*sin(y)

cos(3*acos(x))	4*x^3 - 3*x

c.horner: Phát biểu:

horner(f)

biến đổi một đa thức thành dạng Horner hay biểu diễn lồng nhau. Ví dụ:

f	horner(f)
x^3 - 6*x^2 + 11*x - 6	-6 + (11 + (-6 + x)*x)*x
1.1 + 2.2*x + 3.3*x^2	11/10 + (11/5 + 33/10*x)*x

d.factor: Nếu f là đa thức hệ số hữu tỉ, phát biểu:

factor(f)

biểu diễn f như là tích của các đa thức có bậc thấp hơn với hệ số hữu tỷ. Ví dụ:

f	factor(f)
x^3 - 6*x^2 + 11*x - 6	(x-1)*(x-2)*(x-3)
x^3 - 6*x^2 + 11*x - 5	x^3 - 6*x^2 + 11*x - 5
x^6 + 1	(x^2 + 1)*(x^4 - x^2 + 1)

Đây là một ví dụ khác về phân tích đa thức x^n +1 thành thừa số:

```
syms x;
n = 1:9;
x = x(ones(size(n)));
p = x.^n + 1;
f = factor(p);
[p; f].'
```

trả về ma trận với các đa thức ở cột thứ nhất và các thừa số ở cột thứ 2:

```
[    x+1,                                  x+1 ]
[ x^2+1,                                       x^2+1 ]
[ x^3+1,                          (x+1)*(x^2-x+1) ]
[ x^4+1,                                     x^4+1 ]
[ x^5+1,                 (x+1)*(x^4-x^3+x^2-x+1)]
[ x^6+1,                      (x^2+1)*(x^4-x^2+1) ]
[ x^7+1,     (x+1)*(1-x+x^2-x^3+x^4-x^5+x^6) ]
[ x^8+1,                                     x^8+1 ]
[ x^9+1,            (x+1)*(x^2-x+1)*(x^6-x^3+1) ]
```

Hàm ***factor*** có thể phân tích các đối tượng chữ có chứa số nguyên thành thừa số. Ví dụ:

```
one = '1'
```

```
for n = 1:11
   N(n ,:) = sym(one(1, ones(1, n)));
end
[N factor(N)]
```

cho kết quả:

```
[              1,                          1 ]
[             11,                       (11) ]
[            111,                    (3)*(37) ]
[           1111,                 (11)*(101) ]
[          11111,                 (41)*(271) ]
[         111111,      3)*(7)*(11)*(13)*(37) ]
[        1111111,              (239)*(4649) ]
[       11111111,     (11)*(73)*(101)*(137) ]
[      111111111,       (3)^2*(37)*(333667) ]
[     1111111111,     (11)*(41)*(271)*(9091)]
[    11111111111,          (513239)*(21649) ]
```

e. simplify: Hàm ***simplify*** là một hàm mạnh, dùng rút gọn các biểu thức. Sau đây là một số ví dụ:

f	simplify(f)
x*(x*(x - 6) + 11) - 6	x^3 - 6*x^2 + 11*x - 6
(1 - x^2)/(1 - x)	x + 1
(1/a^3 + 6/a^2 + 12/a + 8)^(1/3)	((2*a + 1)^3/a^3)^(1/3)
syms x y positive log(x*y)	log(x) + log(y)
exp(x) * exp(y)	exp(x + y)
cos(x)^2 + sin(x)^2	1

f .simple: Hàm ***simple*** đưa ra dạng ngắn nhất có thể có của một biểu thức.Hàm này có nhiều dạng,mỗi dạng trả về kết quả khác nhau. Dạng:

simple(f)

hiển thị dạng ngắn nhất. Ví dụ:

```
syms x
simple(cos(x)^2 + sin(x)^2)
```

Trong một số trường hợp, ta áp dụng ***simple*** 2 lần để nhận được hiệu quả rút gọn cao hơn. Ví dụ:

```
syms a
f = (1/a^3+6/a^2+12/a+8)^(1/3);
```

simple(simple(f))

cho ta:

1/a+2

Trong khi lệnh:

syms a
simple(f)

cho ta:

*(2*a+1)/a*

Hàm ***simple*** đặc biệt có hiệu quả trên các biểu thức lượng giác. Sau đây là một số ví dụ:

f	simple(f)
cos(x)^2 + sin(x)^2	1
2*cos(x)^2 - sin(x)^2	3*cos(x)^2 - 1
cos(x)^2 - sin(x)^2	cos(2*x)
cos(x) + (-sin(x)^2)^(1/2)	cos(x) + i*sin(x)
cos(x) + i*sin(x)	exp(i*x)
cos(3*acos(x))	4*x^3 - 3*x

7. Thay số: Ta xét ví dụ giải phương trình bậc hai $ax^2 + bx + c = 0$. Các lệnh thực hiện nhiệm vụ này là:

syms a b c x
*s = solve(a*x^2 + b*x + c);*

Bây giờ ta muốn tính cụ thể giá trị của x với a = 1, b = 2, c = 4 thì dùng các lệnh:

a = 1;
b = 2;
c = 4;
x = subs(s)

Lệnh ***subs*** có thể kết hợp với lệnh ***double*** để tính trị số của một biểu thức chữ. Giả sử ta có:

syms t
*M = (1 - t^2)*exp(-1/2*t^2);*
*P = (1 - t^2)*sech(t);*

và muốn xem trên đồ thị P và M khác nhau như thế nào. Ta dùng các lệnh:

```
ezplot(M);
hold on;
ezplot(P)
```

Tuy nhiên ta vẫn khó hình dung được sự sai khác giữa hai đường cong. Vì vậy tốt hơn chúng ta kết hợp ***subs, double*** lại trong chương trình ***ctcompsubs.m***:

```
T = -6:0.05:6;
MT = double(subs(M, t, T));
PT = double(subs(P, t, T));
plot(T, MT, 'b', T, PT, 'r-.')
title(' ')
legend('M' ,'P')
xlabel('t');
grid
```

để tạo ra đồ thị nhiều màu.

8. Giải phương trình:

a. Giải các phương trình đại số: Nếu S là biểu thức chữ thì:

```
solve(S)
```

tìm giá trị của biến kí tự trong S để S = 0. Ví dụ:

```
syms a b c x
S = a*x^2 + b*x + c;
solve(S)
```

cho ta:

```
ans =
        [ 1/2/a*(-b+(b^2-4*a*c)^(1/2))]
        [ 1/2/a*(-b-(b^2-4*a*c)^(1/2))]
```

Đây là vec tơ chữ mà các phần tử của nó là 2 nghiệm của phương trình.

Nếu ta muốn tìm nghiệm với một biến được mô tả, ta phải chỉ rõ biến như một thông số phụ. Ví dụ nếu ta muốn giải S theo b thì phải viết:

```
b = solve(S,b)
```

và nhận được kết quả:

```
b =
        -(a*x^2+c)/x
```

Chú ý rằng ví dụ này giả thiết phương trình có dạng f(x) = 0. Nếu ta muốn giải phương trình có dạng f(x) = q(x) ta phải sử dụng chuỗi. Đặc biệt lệnh:

```
s = solve('cos(2*x)+sin(x)=1')
```

cho 4 nghiệm:

```
s =
        [        0]
        [       pi]
        [ 1/6*pi]
        [ 5/6*pi]
```

Phương trình x^3-2*x^2 = x-1 giúp ta hiểu cách giải phương trình. Đánh vào lệnh:

```
s = solve('x^3–2*x^2 = x–1')
```

cho ta kết quả:

```
s =
        [ 1/6*(28+84*i*3^(1/2))^(1/3)+14/3/(28+84*i*3^(1/2))^(1/3)+2/3]
        [ -1/12*(28+84*i*3^(1/2))^(1/3)-7/3/(28+84*i*3^(1/2))^(1/3)
        +2/3+1/2*i*3^(1/2)*(1/6*(28+84*i*3^(1/2))^(1/3)
        -14/3/(28+84*i*3^(1/2))^(1/3))]
        [-1/12*(28+84*i*3^(1/2))^(1/3)-7/3/(28+84*i*3^(1/2))^(1/3)
        +2/3-1/2*i*3^(1/2)*(1/6*(28+84*i*3^(1/2))^(1/3)
        -14/3/(28+84*i*3^(1/2))^(1/3))]
```

Ta tính giá trị số của nghiệm:

```
double(s)
ans =
  2.24697960371747 + 0.00000000000000i
 -0.80193773580484 + 0.00000000000000i
  0.55495813208737 - 0.00000000000000i
```

Nó cho thấy tất cả các nghiệm của phương trình là số thực. Điều này không đúng. Dùng lệnh vpa để xác định độ chính xác:

```
vpa(s, 10)
```

tạo ra:

```
ans =
        [  2.246979604+.1e-9*i]
        [ -.8019377357+.3e-9*i]
        [  .5549581323-.5e-9*i]
```

Điều này nghĩa là phần ảo của s rất nhỏ nhưng khác 0. Ta xem một ví dụ khác:

```
syms x
s = solve(tan(x)+sin(x)–2);
```

Kết quả là một vec tơ 4×1. Như trên, ta dùng lệnh ***double***:

```
X = double(s)
X =
        0.88628729156094
        -1.89793604072796
        2.07662070137841
        2.07662070137841
```

b. Hệ phương trình đại số: Bây giờ ta xét hệ phương trình. Giả sử ta có hệ phương trình:

$$\begin{cases} x^2y^2 = 0 \\ x - \dfrac{y}{2} = \alpha \end{cases}$$

và ta cần tìm x và y. Trước hết ta tạo ra các đối tượng cần thiết:

```
syms x y alpha
```

Có nhiều cách để biểu diễn nghiệm. Một trong các cách đó là viết:

```
[x, y] = solve(x^2*y^2, x – (y/2) – alpha)
```

và có được kết quả:

```
x =
        [       0]
        [       0]
        [ alpha]
        [ alpha]
y =
        [ -2*alpha]
        [ -2*alpha]
        [           0]
        [           0]
```

Sau đó viết vec tơ nghiệm:

```
v = [x, y]
```

cho ta:

```
v =
        [       0, -2*alpha]
        [       0, -2*alpha]
        [   alpha,       0]
        [   alpha,       0]
```

Ta xét tiếp phương trình:

```
eqs1 = 'x^2*y^2=1, x–1/2*y–alpha'
[x, y] =  solve(eqs1)
```

tạo ra các nghiệm:

```
x =
        [ 1/2*alpha+1/2*(alpha^2+2)^(1/2)]
        [ 1/2*alpha-1/2*(alpha^2+2)^(1/2)]
        [ 1/2*alpha+1/2*(alpha^2-2)^(1/2)]
        [ 1/2*alpha-1/2*(alpha^2-2)^(1/2)]

y =
        [ -alpha+(alpha^2+2)^(1/2)]
        [ -alpha-(alpha^2+2)^(1/2)]
        [ -alpha+(alpha^2-2)^(1/2)]
        [ -alpha-(alpha^2-2)^(1/2)]
```

Cách gán các nghiệm như trên chỉ thích hợp với hệ có ít phương trình. Với hệ có nhiều phương trình, ***solve*** tạo ra một cấu trúc mà các trường của nó là các nghiệm. Ta khảo sát hệ phương trình:

$$\begin{cases} u^2 + v^2 = a^2 \\ u + v = 1 \\ a^2 - 2a = 3 \end{cases}$$

Lệnh:

```
S = solve('u^2–v^2 = a^2','u + v = 1','a^2–2*a = 3')
```

Cho kết quả:

```
S =
        a: [2x1 sym]
        u: [2x1 sym]
        v: [2x1 sym]
```

Các nghiệm là các trường của S. Đó là:

```
S.a
```

Tạo ra:

```
ans =
```

[-1]
[3]

Tương tự ta tìm được nghiệm u và v. Cấu trúc S bây giờ có thể được xử lí bằng trường và chỉ số để truy cập đến các phần riêng biệt của nghiệm. Ví dụ nếu ta muốn kiểm tra nghiệm thứ 2, ta có thể dùng phát biểu sau:

s2 = [S.a(2), S.u(2), S.v(2)]

để trích thành phần tứ 2 của mỗi trường.

s2 =
[3, 5, -4]

Phát biểu:

M = [S.a, S.u, S.v]

Tạo ra ma trận nghiệm M:

M =
[-1, 1, 0]
[3, 5, -4]

mà mỗi hàng là một nghiệm của hệ.

Nếu hệ phương trình là tuyến tính ta có thể dùng ma trận để giải hệ. Ví dụ:

```
clear u v x y
syms u v x y
S = solve(x+2*y–u, 4*x+5*y–v);
sol = [S.x;S.y]
```

và:

```
A = [1 2; 4 5];
b = [u; v];
z = A\b
```

cho:

```
sol =
        [ -5/3*u+2/3*v]
        [ 4/3*u-1/3*v]

z =
        [-5/3*u+2/3*v]
        [ 4/3*u-1/3*v]
```

Như vậy ta có cùng một nghiệm cho dù phương pháp giải khác nhau.

c. Giải phương trình vi phân: Hàm ***dsolve*** tính nghiệm bằng chữ của phương trình vi phân thường. Các phương trình được mô tả bằng các biểu thức chữ chứa các chữ cái D để chỉ các đạo hàm. Kí hiệu D2, D3,. . ., Dn tương ứng với đạo hàm cấp

1,cấp 2,..,cấp n. Như vậy D2y trong ***Symbolic Math Toolbox*** là $\frac{d^2y}{dx^2}$. Biến phụ thuộc là biến được xử lí bởi D và biến độc lập mặc định là t. Như vậy tên các biến kí tự không được có D. Có thể dùng biến độc lập khác bằng cách chỉ ra nó như là thông số cuối cùng trong lệnh ***dsolve.*** Điều kiện đầu có thể mô tả như là một phương trình phụ. Nếu điều kiện đầu không có, nghiệm sẽ chứa các hằng số tích phân C1, C2 v.v. Cú pháp của ***dsolve*** được mô tả trong bảng sau:

Cú pháp	Phạm vi
y = dsolve(‘Dyt = y0*y’)	Một phương trình, một nghiệm
[u,v] = dsolve('Du = v', 'Dv = u')	Hai phương trình, hai nghiệm
S = dsolve('Df = g',' Dg = h','Dh = –f') S.f, S.g, S.h	Ba phương trình, ra là cấu trúc nghiệm

Ví dụ 1: Ta dùng lệnh:

dsolve('Dy = 1 + y^2')

và có kết quả:

ans =

tan(t-C1)

Để mô tả điều kiện đầu, ta dùng:

y = dsolve('Dy = 1+y^2','y(0) = 1')

và có:

y =

*tan(t + 1/4*pi)*

Chú ý là y ở trong vùng làm việc của MATLAB nhưng biến độc lập t thì không. Như vậy lệnh diff(y, t) gây ra lỗi. Để đặt t vào vùng làm việc của MATLAB phải dùng ***syms t***

Ví dụ 2: Các phương trình phi tuyến có thể có nhiều nghiệm, thậm chí ngay cả khi đã cho điều kiện đầu.

x = dsolve('(Dx)^2 + x^2 = 1','x(0) = 0')

cho kết quả:

x =

[-sin(t)]

[sin(t)]

Ví dụ 3: Đây là một phương trình bậc 2 với 2 điều kiện đầu. Lệnh:

*y = simplify(dsolve('D2y = cos(2*x) – y','y(0) = 1','Dy(0) = 0', 'x'))*

tạo ra:

y =

*-2/3*cos(x)^2 + 1/3 + 4/3*cos(x)*

Để giải phương trình:

$$\frac{d^3u}{dx^3} = u$$

$$u(0) = 1, u'(0) = 1, u''(0) = \pi$$

ta dùng các lệnh sau:

u = dsolve('D3u = u','u(0) = 1','Du(0) = –1','D2u(0) = pi','x')

d. Hệ phương trình vi phân: Hàm ***dsolve*** có thể xử lí hệ phương trình vi phân, có hay không có điều kiện đầu. Ví dụ ta có hệ phương trình:

y'=3f + 4g

g' = -4f + 3g

Để giải hệ ta dùng lệnh:

*S = dsolve('Df = 3*f + 4*g', 'Dg = –4*f + 3*g')*

Nghiệm được tính và trả về dưới dạng cấu trúc S:

S =

f: [1x1 sym]

g: [1x1 sym]

Ta có thể xác định giá trị của f và g bằng lệnh:

f = S.f

f =

*exp(3*t)*(cos(4*t)*C1 + sin(4*t)*C2)*

g = S.g

g =

*-exp(3*t)*(sin(4*t)*C1 - cos(4*t)*C2)*

Nếu ta cho cả điều kiện đầu thì viết:

*[f, g] = dsolve('Df = 3*f + 4*g, Dg = –4*f + 3*g', 'f(0) = 0, g(0) = 1')*

f =

*exp(3*t)*sin(4*t)*

g =

*exp(3*t)*cos(4*t)*

Bảng sau mô tả một vài ví dụ và cú pháp của ***Symbolic Math Toolbox***.

Phương trình vi phân	Lệnh MATLAB
$\frac{dy}{dt} + 4y(t) = e^{-t}$ $y(0) = 1$	y = dsolve('Dy + 4*y = exp(-t)','y(0) = 1')
$\frac{d^2y}{dx^2} + 4y(x) = e^{-2x}$ $y(0) = 0, y(\pi) = 0$	y = dsolve('D2y + 4*y = exp(–2*x)', 'y(0) = 0', 'y(pi) = 0', 'x')
$\frac{d^2y}{dx^2} = xy(x)$ $y(0) = 0, y(3) = \frac{1}{\pi}K_{\frac{1}{3}}(2\sqrt{3})$ (phương trình Airy)	y = dsolve('D2y = x*y','y(0) = 0', 'y(3) = besselk(1/3, 2*sqrt(3))/pi', 'x')

9. Biến đổi Fourier và Fourier ngược:

a. Biến đổi Fourier: Biến đổi Fourier dùng để biến đổi phương trình vi phân thành phương trình đại số. Cú pháp:

F = fourier(f)
F = fourier(f, v)
F = fourier(f, v, u)

Ta có thể xem các biến đổi Fourier trong bảng sau:

Biến đổi Fourier	Lệnh MATLAB
$f(x) = e^{-x^2}$ $F[f](w) = \int_{-\infty}^{\infty} f(x)e^{-iwx}dx = \sqrt{\pi}e^{-w^2/4}$	f = exp(-x^2) fourier(f) cho: pi^(1/2)*exp(-1/4*w^2)
$g(w) = e^{-\lvert w\rvert}$ $F[g](t) = \int_{-\infty}^{\infty} g(w)e^{-iwt}dt = \frac{2}{1+t^2}$	g = exp(-abs(w)) fourier(g) cho 2/(1+t^2)
$f(x) = xe^{-\lvert x\rvert}$ $F[f](u) = \int_{-\infty}^{\infty} f(x)e^{-ixu}dx = -\frac{4i}{(1+u^2)^{2u}}$	f = x*exp(-abs(x)) f = x*exp(-abs(x)) cho -4*i/(1+u^2)^2*u

b. Biến đổi Fourier ngược: Khi biết hàm ảnh Fourier dùng biến đổi Fourier ngược ta tìm được hàm gốc. Cú pháp:

f = ifourier(F)
f = ifourier(F, u)
f = ifourier(F, v, u)

Biến đổi Fourier ngược	Lệnh MATLAB
$f(w) = e^{\frac{w^2}{4a^2}}$ $F^{-1}[f](x) = \int_{-\infty}^{\infty} f(w)e^{iwx}dw = \frac{a}{\sqrt{\pi}}e^{-(ax)^2}$ $g(x) = e^{-\lvert x\rvert}$ $F^{-1}[g](t) = \int_{-\infty}^{\infty} g(x)e^{itx}dx = \frac{\pi}{1+t^2}$	syms a real f = exp(-w^2/(4*a^2)) F = ifourier(f) F = simple(F) cho ha*exp(-x^2*a^2)/pi^(1/2) g = exp(-abs(x)) ifourier(g) cho 1/(1+t^2)/pi
$f(w) = 2e^{-\lvert w\rvert} - 1$ $F^{-1}[f](t) = \int_{-\infty}^{\infty} f(w)e^{iwt}dw =$ $\frac{2 - \pi\delta(t)(1-t^2)}{\pi(1+t)}$	f = 2*exp(-abs(w)) - 1 simple(ifourier(f,t)) cho (2-pi*Dirac(t)-pi*Dirac(t)*t^2)/ (pi+pi*t^2)

10. Biến đổi Laplace và Laplace ngược:

a. Biến đổi Laplace: Biến đổi Laplace dùng biến đổi phương trình vi phân thành phương trình đại số. Cú pháp:

laplace(F)
laplace(F, t)
laplace(F, w, z)

Biến đổi Laplace	Lệnh MATLAB
$f(t) = t^4$ $L[f] = \int_0^{\infty} F(t)e^{-st}dt = \frac{24}{s^5}$	f = t^4 laplace(f) cho 24/s^5
$g(s) = \frac{1}{\sqrt{s}}$	g = 1/sqrt(s) laplace(g) cho 1/(s^(1/2))*pi^(1/2)

$L[g](t) = \int_0^\infty g(s)e^{-st}ds = \frac{\pi}{\sqrt{s}}$	
$f(t) = e^{-at}$ $L[f](x) = \int_0^\infty f(t)e^{-tx}dt = \frac{1}{x+a}$	f = exp(-a*t) laplace(f) cho 1/(x+a)

b. Biến đổi Laplace ngược: Khi có ảnh của hàm,ta có thể tìm lại hàm gốc bằng biến đổi Laplace ngược. Cú pháp:

F = ilaplace(L)
F = ilaplace(L, y)
F = ilaplace(L, y, x)

Biến đổi Laplace ngược	Lệnh MATLAB
$f(s) = \frac{1}{s^2}$ $L^{-1}[f] = \frac{1}{2\pi i}\int_{c-i\infty}^{c+i\infty} f(s)e^{st}ds = t$	f = 1/s^2 ilaplace(f) cho t
$g(t) = \frac{1}{t-a}$ $L^{-1}[g] = \frac{1}{2\pi i}\int_{c-i\infty}^{c+i\infty} g(t)e^{xt}dt = xe^{ax}$	g = 1/(t - a) ilaplace(g) cho x*exp(a*x)
$f(u) = \frac{1}{u^2-a^2}$ $L^{-1}[f] = \frac{1}{2\pi i}\int_{c-i\infty}^{c+i\infty} g(u)e^{xu}du = \frac{1}{2ae^{ax}} - \frac{1}{2ae^{-ax}}$	f = 1/(u^2-a^2) ilaplace(f) cho 1/(2*a*exp(a*x)) - 1/(2*a*exp(-a*x))

§3. POWER SYSTEM BLOCKSET

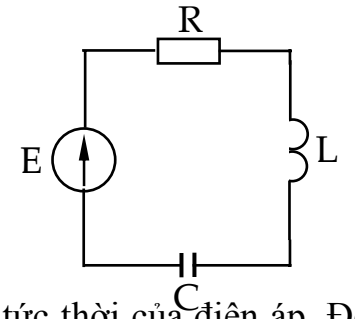
1. Khái niệm chung: ***Power System Blockset*** được thiết kế để cung cấp cho chúng ta công cụ hiệu quả và tiện lợi để mô phỏng nhanh và dễ các mạch điện, các hệ thống điện. Thư viện của nó chứa các phần tử cơ bản của mạch điện như máy biến áp, đường dây, các máy điện và các thiết bị điện tử công suất. Giao diện đồ hoạ cung cấp các thành phần của hệ thống điện. Các thành phần này dược lưu trong thư viện powerlib. Để mở thư viện này từ cửa sổ MATLAB ta đánh lệnh ***powerlib***. Khi này MATLAB mở một cửa sổ chứa các khối hệ thống con khác nhau. Các hệ thống con này bao gồm:

Electrical Sources
Elements
Power Electronics
Machines

Connectors
Measuremets
Extras
Demos

Ta có thể mở các hệ thống con này để tạo ra các cửa sổ chứa các khối mà ta cần copy vào mô hình. Mỗi một thành phần được biểu diễn bằng một icon đặc biệt.

2. Mô hình hoá một mạch điện đơn giản: Power System Blockset cho phép ta xây dựng và mô phỏng một mạch điện chứa các phần tử tuyến tính cũng như phi tuyến. Ta xét một mạch điện như hình vẽ:



$e = \sqrt{2}.220\sin(314\pi + 10°)$ V

$R = 10\Omega$

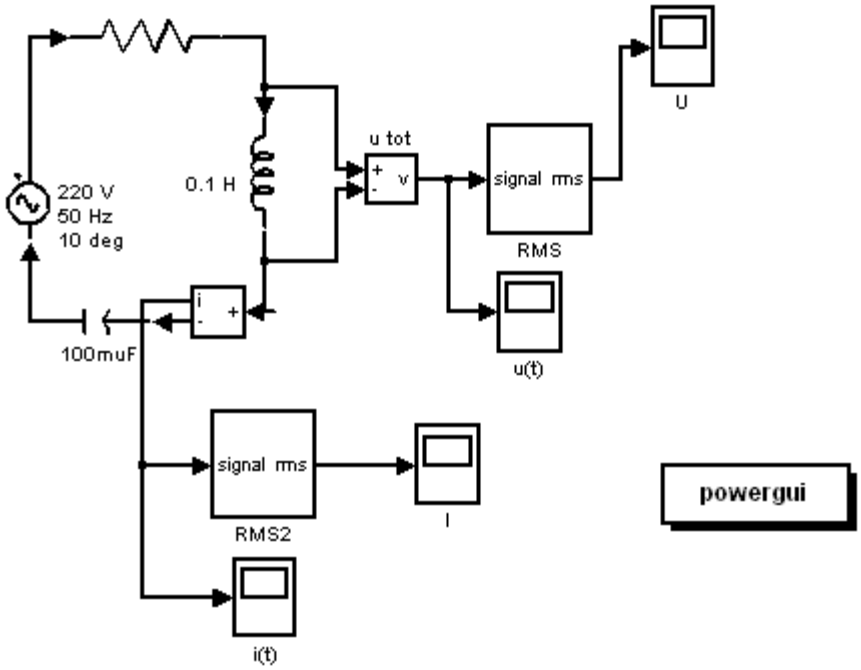
$L = 0.1$ H

$C = 100\mu F$

Để mô phỏng mạch điện này ta dùng các khối: nguồn, điện trở, điện kháng, điện dung và dụng cụ đo. Để đo điện áp ta dùng khối Vmet. Nó cho trị số tức thời của điện áp. Để thấy đươc giá trị hiệu dụng ta dùng khối RMS. Các bước thực hiện như sau:

- Từ menu ***File*** của cửa sổ powerlib chọn ***New*** rồi chọn ***Model*** sẽ chứa mạch điện và gọi là ***ctcircuit.mdl***
- Mở thư viện ***Electrical Sources*** để copy ***AC Voltage Source Block*** vào cửa sổ ***ctcircuit.mdl***
- Mở hộp thoại ***AC Voltage Source Block*** bằng cách nhấp đúp lên nó để nhập vào biên độ, phase và tần số theo các giá trị đã cho trong sơ đồ. Chú ý là biên độ là giá trị max của điện áp.
- Do khối điện trở không có nên copy khối ***Series RLC Branch*** và đặt giá trị điện trở như đã cho và đặt L là vô cùng và C là zero.
- Thực hiện tương tự với phần tử L và C.
- Lấy khối đo điện áp trong hệ thống con ***Measurement***
- Để xem điện áp, dùng khối ***Scope*** của Simulink chuẩn. Mở Simulink và copy khối Scope vào mô hình ***ctcircuit.mdl***. Nếu khối ***Scope*** được nối trực tiếp với đầu ra của thiết bị đo điện áp nó sẽ hiển thị điện áp theo V.
- Để hoàn thành mạch điện, ta cần nối các phần tử với nhau

Sơ đồ mô phỏng(lưu trong ***ctcircuit.mdl***) như sau:



Bây giờ ta có thể bắt đầu mô phỏng từ menu simulation. ta vào menu này, chọn các thông số cho qua trình mô phỏng và bấm nút start.

Để dễ dàng cho việc phân tích trạng thái xác lập của mạch điện chúng ta, thư viện powerlib cung cấp giao diện đồ hoạ(GUI). Copy khối giao diện ***Powergui*** vào cửa sổ ***ctcircuit.mdl*** và nhấn đúp vào icon để mở nó. Mỗi dụng cụ đo đại lượng ra được xác định bằng mỗi chuỗi tương ứng với tên của nó. Các biến trạng thái được hiển thị tương ứng với các giá trị xác lập của dòng điện và điện áp. Tên các biến chứa tên các khối, bắt đầu bằng tiếp đầu ngữ Il- hay Uc_. Dấu quy ước được sử dụng với dòng điện và điện áp và các biến trạng thái đươc xác định bằng hướng của các khối:

- dòng điện điện cảm chạy theo hướng mũi tên tương ứng với dấu dương
- điện áp trên tụ C bằng điện áp ra trừ đi điện áp vào

Chọn menu ***Tool | Steady - State Voltages and Currents*** để xem các trị số xác lập của dòng điện và điện áp.

Bây giờ chọn menu ***Tool | Initial Value of State Variables*** để hiển thị các giá trị khởi đầu của các biến trạng thái. Các giá trị khởi đầu này được đặt để bắt đầu simulation ở trạng thái xác lập.

Tiếp theo ta tính các biểu diễn của không gian trạng thái của mô hình ***ctcircuit*** bằng hàm ***power2sys***. Nhập dòng lệnh sau đây vào cửa sổ MATLAB:

[A, B, C, D, x0, states, inputs, outputs] = power2sys('ctcircuit');

Hàm ***power2sys*** trả về mô hình không gian trạng thái của mạch trong 4 ma trận A, B, C, D, x0 là vec tơ các điều kiện đầu mà ta vừa hiển thị với ***Powergui***. Tên của các biến trạng thái, các đại lượng vào và các đại lượng ra được trả về trong 3 ma trận chuỗi.

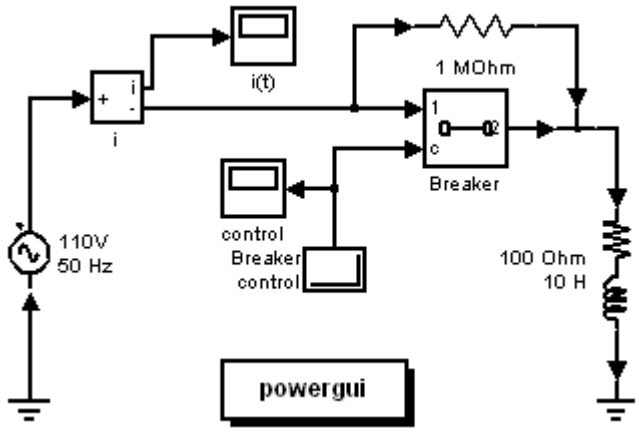
Một khi mô hình trạng thái đã biết, nó có thể phân tích được trong vùng tần số. Ví dụ các mode của mạch này có thể tìm từ các giá trị riêng của ma trận A(dùng lệnh MATLAB eig(A)):

```
eig(A)
ans =
  1.0e+002 *
  -0.5000 + 3.1225i
  -0.5000 - 3.1225i
```

Hệ thống này có dao động tắt dần vì phần thực âm. Nếu ta dùng ***Control System Toolbox***, ta có thể vẽ đồ thị Bode. Các lệnh MATLAB(lưu trong ***ctcircuitm.m***) như sau:

```
freq = 0:1500;
w = 2*pi*freq;
[bien, pha, w] = bode(A, B, C, D);
semilogy(w, mag1(:, 2));
semilogy(w, mag1(:, 2));
```

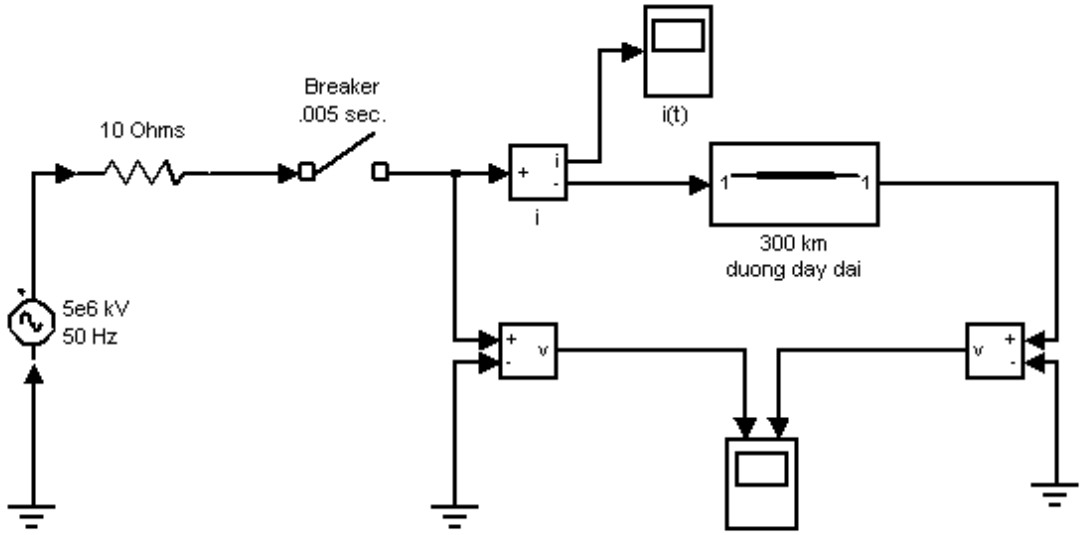
3. Mô hình hoá quá trình quá độ: Một trong những phạm vi ứng dụng của ***Power System Blockset*** là simulation quá trình quá độ trong các mạch điện. Điều này có thể làm được cả với cầu dao cơ khí và mạch điện tử. Ta xét quá trình quá độ khi đóng một mạch RL vào nguồn điện xoay chiều. Sơ đồ mô phỏng (lưu trong ***cttransient.mdl***) như sau:



Trước quá trình quá độ, cầu dao(được mô phỏng bằng phần tử ***breaker***) ở trạng thái mở. Sau khoảng thời gian 1.5 chu kì, cầu dao đóng, nối mạch RL vào nguồn e = $\sqrt{2}\sin 314t$.

4. Mô hình hoá đường dây dài: Đường dây dài là đường dây có thông số rải. Nó được mô phỏng bằng khối ***Distributed Parameter Line***. Nó được xây dựng trên cơ sở

xét quá trình truyền sóng trên đường dây. Ta xét một đường dây dài 1000 km có mô hình (lưu trong ***ctlongline.mdl***)như sau:



Khi sử dụng mô hình ta phải khai báo điện trở, điện dung và điện cảm của đường dây trên một đơn vị dài, số pha và chiều dài của đường dây.

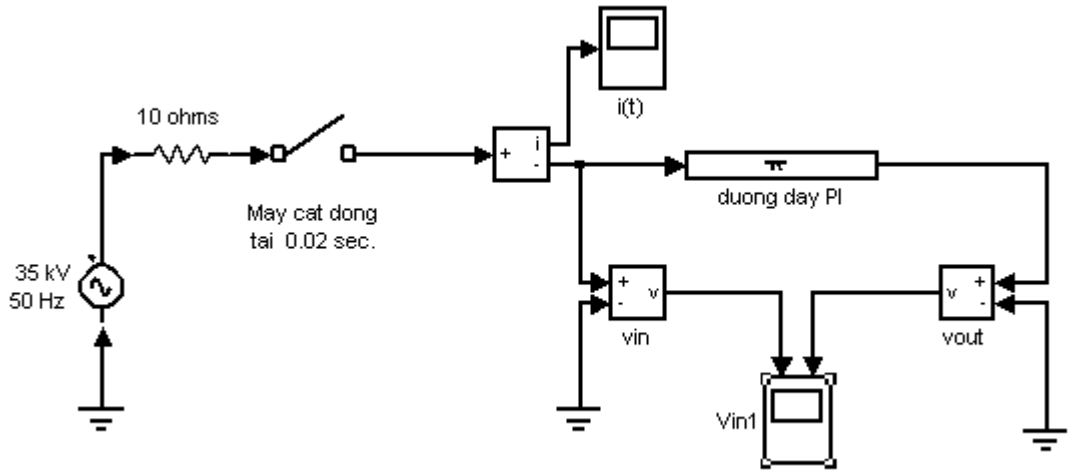
5. Mô hình hoá đường dây bằng các đoạn hình π: Mục đích của mô hình này là thực hiện đường dây 1 pha với thông số được tập trung trên từng đoạn. Khối ***PI Section Line*** thực hiện đường dây truyền tải một pha với thông số tập trung trên từng đoạn π. Đối với đường dây truyền tải, điện trở, điện cảm và điện dung phân bố đều trên suốt chiều dài. Một mô hình xấp xỉ đường dây thông số phân bố có được bằng cách nối nhiều đoạn pi giống nhau. Không giống như đường dây thông số rải có số trạng thái là vô hạn, mô hình tuyến tính các đoạn π có số hữu hạn các trạng thái cho phép mô hình không gian-trạng thái được dùng để rút ra đáp ứng tần số. Số đoạn được dùng phụ thuộc vào tần số được biểu diễn. Xấp xỉ tốt nhất thực hiện theo phương trình:

$$f_{max} = \frac{Nv}{8l}$$

Trong đó:

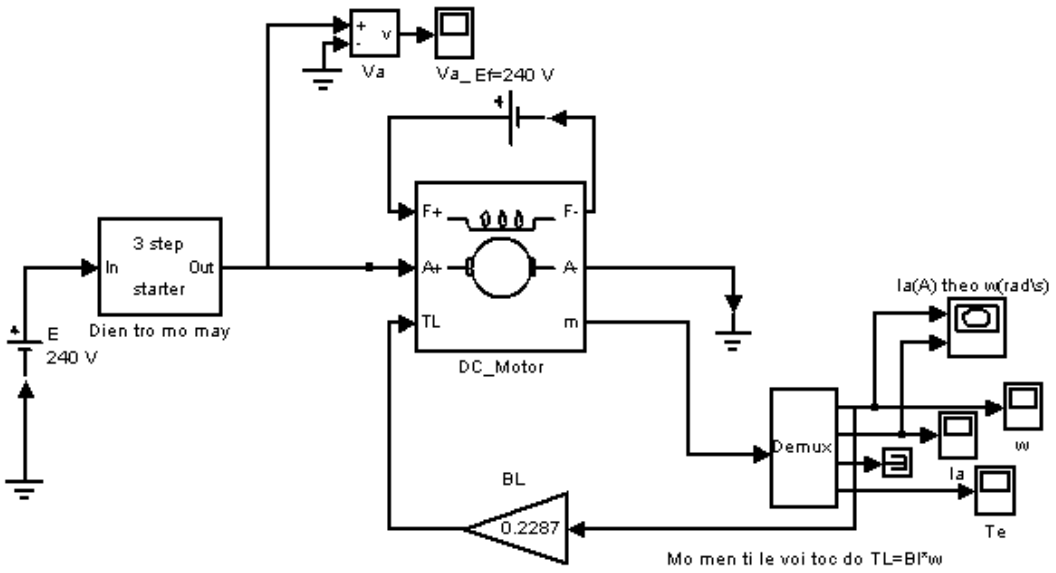
- N : số đoạn pi
- v : tốc độ truyền sóng(km/s = 1/√L(H/km)C(F/km)
- l : chiều dài đường dây(km)

Ta xét đường dây trên không dài 100 km có tốc độ truyền sóng 300000 km/s, tần số lớn nhất biểu diễn được khi dùng 1 đoạn π là 375Hz. Mô hình đơn giản này đủ dùng trong hệ thống truyền tải năng lượng. Ta xây dựng mô hình (lưu trong ***ctpiline7_7.mdl***)như sau:

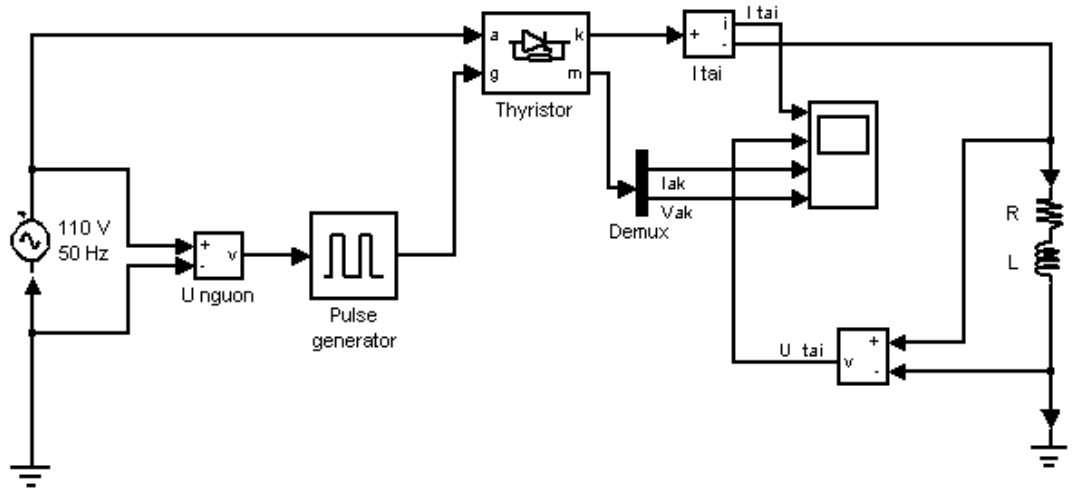


Ta nhập điện trở, điện cảm và điện dung trên một đơn vị dài vào 3 ô đầu tiên của hộp thoại. Nhập độ dài và số đoạn pi mong muốn vào 2 ô cuối.

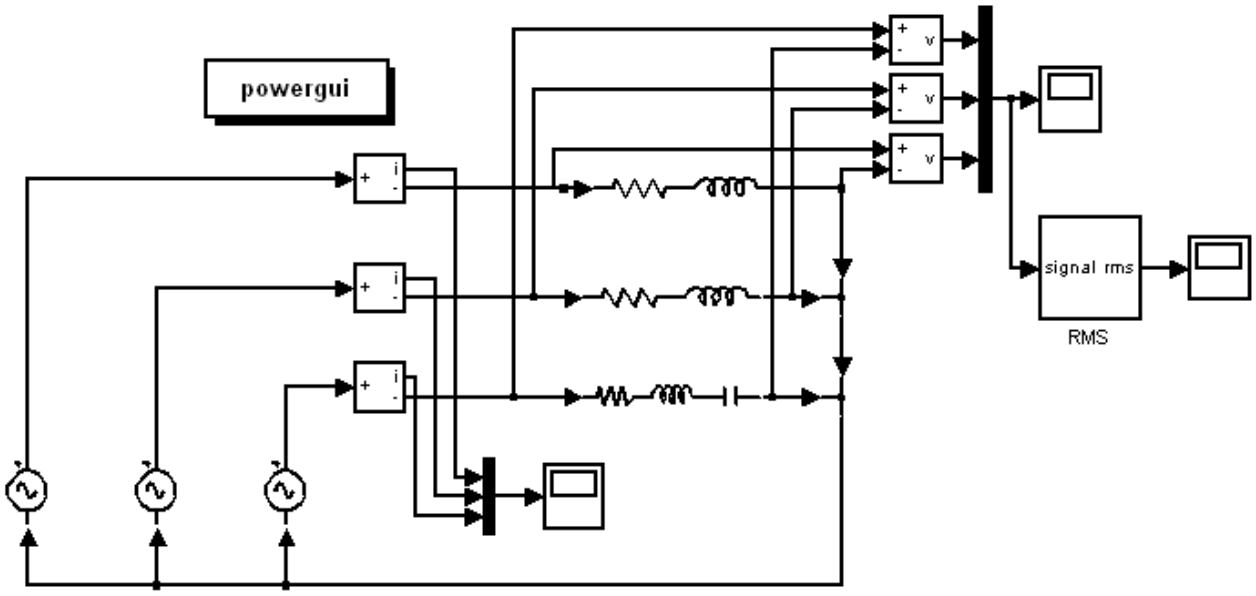
6. Mô hình hoá máy điện: Các máy điện nằm trong thư viện ***Machines***. Các máy điện được mô phỏng dựa trên các phương trình cơ bản của nó và được chia thành 2 dạng: máy điện trong hệ đơn vị tương đối và máy điện trong hệ đơn vị SI. ta xét quá trình mở máy bằng điện trở một động cơ điện một chiều. Sơ đồ mô phỏng (lưu trong ***ctdcmachine.mdl***) như sau:



7. Giới thiệu về điện tử công suất: ***Power System Blockset*** được thiết kế để simulation các thiết bị điện tử công suất. Chúng ta khảo sát một mạch điện có thyristor cung cấp cho một mạch RL. Sơ đồ mô phỏng (lưu trong ***ctthyristor.mdl***) như sau:

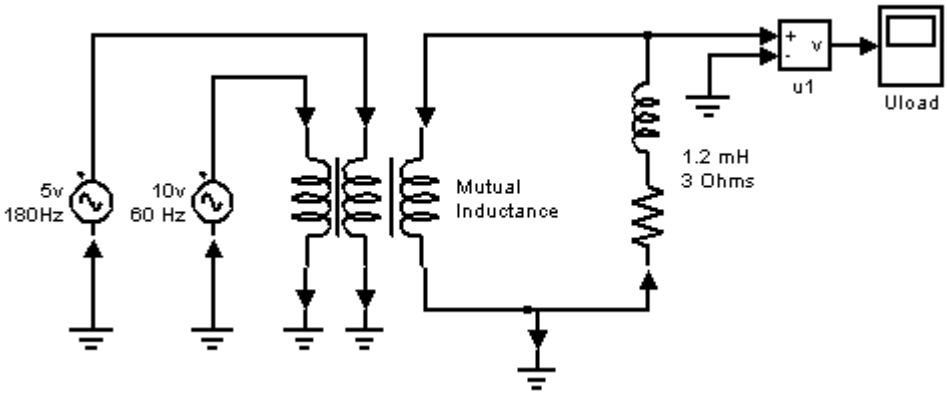


8. Mô hình hoá mạch điện 3 pha: Ta mô hình hoá một mạch điện 3 pha có nguồn đối xứng nhưng tải không đối xứng. Sơ đồ mô phỏng (lưu trong ***ctthreephases.mdl***) như sau:



Điện áp các nguồn có trị hiệu dụng là 231 V. Tải pha thứ nhất là R = 1Ω, L = 1H, pha thứ hai R = 15Ω, L = 2H và pha thứ 3 là R = 10Ω, L = 1H và C = 1μF.

9. Mô hình điện kháng hỗ cảm: Phần tử điện kháng hỗ cảm thực hiện mối liên hệ từ giữa 2 hay 3 dây quấn. Khối ***Mutual Inductance*** thực hiện liên hệ từ giữa 3 dây quấn riêng biệt. Ta mô tả điện trở và điện cảm của từng dây quấn trên mục vào thứ nhất của hộp thoại và điện trở, điện cảm hỗ cảm trên mục vào cuối cùng. Mô hình điện như sau (lưu trong ***ctmutualinduc.mdl***):

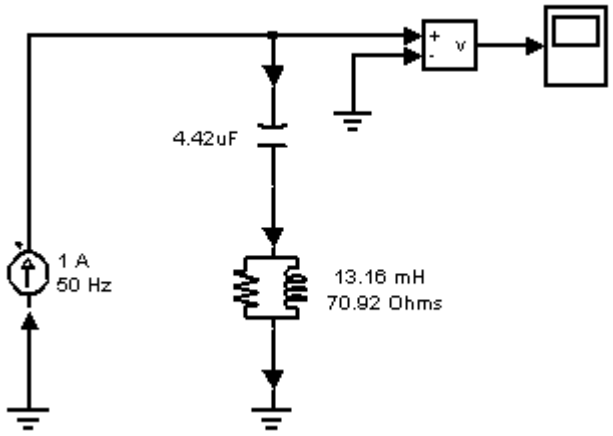


Nếu mục vào của dây quấn thứ 3 bị bỏ trống, nghĩa là chỉ có hỗ cảm giữa 2 dây quấn. Các đầu vào của khối Mutual Inductance là cùng cực tính tại một thời điểm.
Do simulation nên cần:

$Rs > 0$, $Rs > Rm$, $Lm \neq 0$, $Ls \neq Lm$

Điện trở của dây quấn phải dương và lớn hơn điện trở hỗ cảm. Điện cảm hỗ cảm phải khác 0 nhưng điện trở hỗ cảm có thể bằng 0. Dây quấn có thể thả nối, nghĩa là không nối với tổng trở hay phần còn lại của mạch.

10. Mô hình nhánh RLC nối song song: Phần tử này thực hiện nhánh RLC nối song song. Khối ***Parallel RLC Branch*** thực hiện điện trở, điện cảm và điện dung nối song song. Để bỏ một phần tử R,L hay C ta phải đặt các thông số tương ứng là Inf, Inf và 0. Ta có thể dùng giá trị âm cho các thông số. Để có đáp ứng tần của bộ lọc tần số bậc 7 ở 660Hz ta dùng mạch như trong file ***ctpararlc.mdl***.



Tổng trở của mạch:

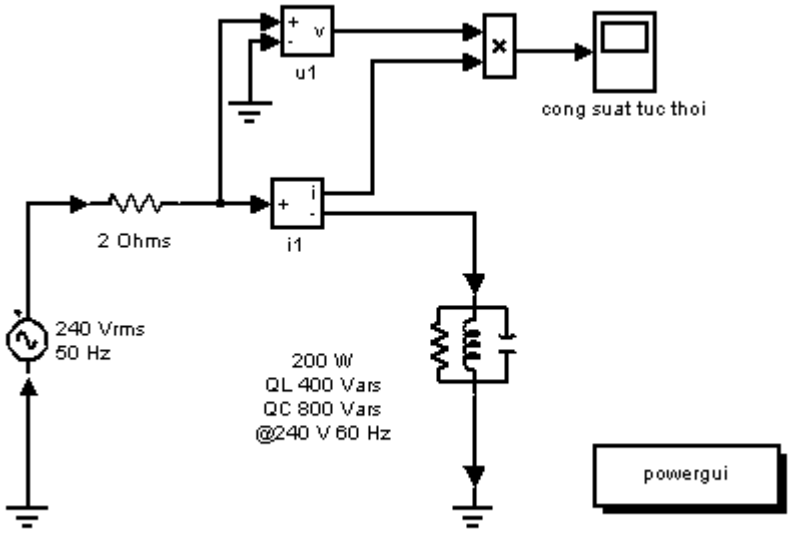
$$Z(s) = \frac{V(s)}{I(s)} = \frac{RLCs^2 + Ls + R}{LCs^2 + RCs}$$

Để có đáp ứng tần của tổng trở ta phải xác định mô hình không gian-trạng thái (ma trận A B C D) của hệ thống (lưu trong ***ctpararlcm.m***)

```
[A, B, C, D] = power2sys('ctpararlc');
freq = logspace(1, 4, 500);
w = 2*pi*freq;
[Z,phaseZ] = bode(A, B, C, D, 1, w);
subplot(2, 1, 1)
loglog(freq, Z)
grid
title('Bo loc song hai bac 11')
xlabel('Tan so, Hz')
ylabel('Tong tro Z')
subplot(2, 1, 2)
semilogx(freq, phaseZ)
xlabel('Tan so, Hz')
ylabel('Pha Z')
grid
```

11. Mô hình tải RLC nôi song song: Phần tử này thực hiện tải RLC nối song song. Khối ***Parallel RLC Load*** thực hiện tải tuyến tính như tổ hợp nối song song của các phần tử R, L và C. Để xác định tham số ta nhập điện áp định mức và tần số định mức

vào 2 mục đầu tiên. Nhập công suất tác dụng, công suất phản kháng trên cuộn dây và công suất phản kháng trên tụ điện vào 3 mục cuối. Các công suất phản kháng phải dương. Tại tần số đã mô tả, tải sẽ có tổng trở hằng và công suất tỉ lệ với bình phương điện áp đặt vào. Ta tìm các giá trị xác lập của điện áp và dòng điện tải trong mạch trong file ***ctloadrclp.mdl***.



12. Mô hình nhánh RLC nối nối tiếp: Phần tử này thực hiện nhánh RLC nối nối tiếp. Khối ***Series RLC Branch*** thực hiện điện trở, điện cảm và điện dung nối nối tiếp. Để loại trừ R, L hay C ta cho chúng bằng 0, 0 hay Inf. Các giá trị này có thể đặt là số âm. Ta xét môt mô hình như trong file ***ctserierlc.mdl***. Tổng trở của nhánh là:

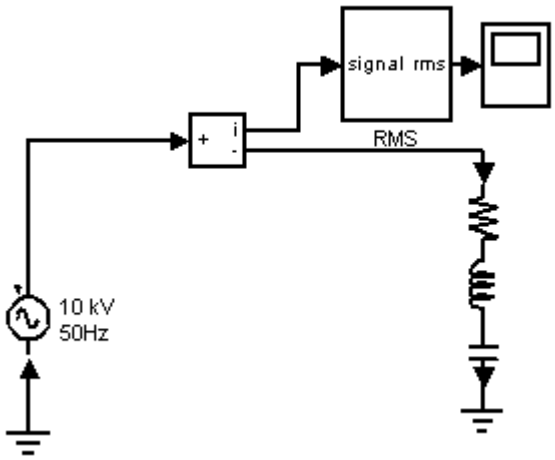
$$Z(s) = \frac{V(s)}{I(s)} = \frac{LCs^2 + RCs + 1}{Cs}$$

Để nhận được đáp ứng tần số của tổng trở ta phải xây dựng mô hình không gian - trạng thái của hệ thống:

```
[A,B,C,D] = power2sys('ctserierlc');
freq = logspace(1, 4, 500);
w = 2*pi*freq;
[Y, phaseY] = bode(A, B, C, D, 1, w);
Z = 1./Y;
phaseZ = -phaseY;
subplot(2, 1, 1)
loglog(freq, Z)
grid
title('Bo loc song bac 5')
xlabel('Tan so, Hz')
ylabel('Tong tro  Z')
subplot(2,1,2)
semilogx(freq,phaseZ)
```

xlabel('Tan so, Hz')
ylabel('Pha Z')
grid

12. Mô hình tải RLC nối nối tiếp: Phần tử này thực hiện tải RLC nối nối tiếp tuyến tính.



Khối ***Series RLC Load*** thực hiên tải RLC nối nối tiếp tuyến tính. Ta nhập giá trị điện áp và tần số định mức vào 2 ô đầu của hộp thoại. Nhập công suất tác dụng,công suất phản kháng trên điện cảm và công suất tác dụng trên điện dung vào 3 ô cuối.Các công suất phản kháng phải có trị số dương. Tại tần số đã mô tả, tải có tổng trở xác định hằng và công suất của nó tỉ lệ với bình phương điện áp đặt vào. Ta tìm giá trị xác lập của điện áp và dòng điện của tải trong file ***ctloadrlcs.mdl***.

§4. ỨNG DỤNG MATLAB TRONG ĐIỀU KHIỂN TỰ ĐỘNG

1. Các dạng mô hình hệ thống: Để xây dựng mô hình của hệ thống, MATLAB cung cấp một số lệnh. Mô hình hệ thống mô tả bằng hàm truyền được xây dựng nhờ lệnh ***tf(ts,ms)*** với ts là đa thức tử số và ms là đa thức mẫu số. Hàm ***zpk(z, p, k)*** với z là vec tơ điểm không, p là vec tơ điểm cực và k là hệ số khuyếch đại tạo nên mô hình điểm không-điểm cực. Hàm ***ss(a, b, c, d)*** với a, b, c, d là các ma trận tạo nên mô hình không gian - trạng thái.

Ví dụ: Ta tạo ra một số mô hình nhờ các lệnh MATLAB sau(lưu trong ***ctspacestate.m***):

```
clc
ts = [1 2];
ms = [1 5 4];
sys1 = tf(ts, ms)
sys2 = zpk([-6 1 1], [-5 1], 3)
sys3 = ss([1 2; 3 4], [1 1; 0 1], [0 1; 1 2; 3 1], 0)
```

Kết quả là:

```
Transfer function:
    s + 2
-------------
s^2 + 5 s + 4
Zero/pole/gain:
3 (s+6) (s-1)^2
---------------
  (s+5) (s-1)
a =
                x1        x2
        x1       1         2
        x2       3         4
b =
                u1        u2
        x1       1         1
        x2       0         1
 c =
                x1        x2
        y1       0         1
        y2       1         2
        y3       3         1
d =
                u1        u2
        y1       0         0
        y2       0         0
        y3       0         0
Continuous-time model.
```

2. Điểm cực và điểm zero của hàm truyền: Để biến đổi hệ thống cho bởi hàm truyền thành hệ cho bởi điểm cực, điểm zero và hệ số khuếch đại dùng hàm ***tf2zp***. Ta cũng có thể dùng hàm ***pole(sys)*** để tìm điểm cực của hệ thống sys và dung hàm ***zero(sys)*** để tìm điểm không của hệ thống sys

Ta dùng các lệnh MATLAB sau(lưu trong ***ctzp2tf.m***):

```
z = [-6; -5; 0];
k = 1;
p = [-3+4*i; -3-4*i; -2; -1];
[ts,ms] = zp2tf(z, p ,k)
```

Kết quả là:

```
ts =
     0    1   11   30    0
ms =
     1    9   45   87   50
```

Để thấy được sự phân bố điểm không và điểm cực của hệ thống trên mặt phẳng phức ta dùng hàm ***pzmap***. Trục của đồ thi được chia lưới bằng lệnh sgrid. Các điểm không biểu thị bằng vòng tròn và điểm cực biểu thị bằng dấu ×. Ta xét các lệnh MATLAB sau(lưu trong ***ctpzmap.m***):

```
clc
sys = zpk([-6 1 1],[-5 1],3)
axis equal
pzmap(sys)
sgrid
```

3. Khai triển hàm truyền thành tổng các phân thức đơn giản: Cho hàm truyền, ta có thể khai triển nó thành tổng các phân thức đơn giản bằng lệnh ***residue***. Hàm ***residue*** cho vec tơ cột các phần dư r, vec tơ cột các điểm cực p và phần nguyên k. Ngược lại, có r, p, k ta có thể tìm hàm truyền bằng các lệnh MATLAB sau(lưu trong ***ctresidue1.m***):

```
r = [0.0-0.25*i; 0+0.25*i; -2];
p = [0+2*i;0-2*i;-1];
k = 2;
[ts, ms] = residue(r, p, k)
```

Kết quả là:

```
ts =
   2    0    9    1
ms =
   1    1    4    4
```

4. Biến đổi hàm truyền thành không gian - trạng thái: Cho phương trình vi phân:

$$a_n \frac{d^n y}{dx^n} + a_{n-1} \frac{d^{n-1} y}{dx^{n-1}} + \cdots + a_1 \frac{d\,y}{dx} + a_0 y = u(t)$$

Đặt $x_1 = y$; $x_2 = y'$; $x_3 = y''$ v.v ta có hệ phương trình trạng thái:

$$\mathbf{x}' = \mathbf{Ax} + \mathbf{Bu}$$
$$\mathbf{y} = \mathbf{Cx} + \mathbf{Du}$$

gọi là phương trình không gian - trạng thái

Nếu một hệ điều khiển tự động cho bởi hàm truyền ta có thể biến đổi về không gian - trạng thái bằng lệnh ***tf2ss***.

Ví dụ: Cho hàm truyền :

$$H(s) = \frac{s^2 + 7s + 2}{s^3 + 9s^2 + 26s + 24}$$

Ta biến hệ về dạng không gian-trạng thái bằng các lệnh MATLAB sau(lưu trong ***cttf2ss.m***):

```
ts = [1 7 2];
ms = [1 9 26 24];
[a,b,c,d ] = tf2ss(ts, ms)
```

Kết quả là:

```
a =
   -9   -26   -24
    1     0     0
    0     1     0
b =
    1
    0
    0
c =
    1     7     2
d =
    0
```

5. Biến đổi không gian - trạng thái thành hàm truyền: Để biến đổi hệ cho dưới dạng không gian - trạng thái thành hàm truyền ta dùng lệnh ***ss2tf***. Ta xét các lệnh sau(lưu trong ***ctss2tf.m***)

```
a = [0 1 0; 0 0 1; -1 -2 -3];
b = [10; 0; 0];
c = [1 0 0];
d = [0];
[ts,ms] = ss2tf(a, b, c, d, 1)
```

Kết quả là:

```
ts =
                0  10.00  30.00  20.00

ms =
   1.00   3.00   2.00   1.00
```

Như vậy hàm truyền là:

$$G(s) = \frac{10(s^2 + 3s + 2)}{s^3 + 3s^2 + 2s + 1}$$

6. Nghiệm của phương trình trạng thái: Để tìm nghiệm của phương trình trạng thái ta dùng lệnh **lsim**.

Ví dụ: Cho phương trình trạng thái của một hệ tuyến tính

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -6 & -11 & -6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} u(t)$$

$$y = [1 \;\; 1 \;\; 0]\, \mathbf{x}$$

Cho điều kiện đầu **x**(0) = [1 0.5 -0.5]. Tìm **x**(t), y(t) với u(t) là hàm đơn vị. Ta dùng các lệnh MATLAB sau(lưu trong ***ctlsim.m***):

```
a = [0 1 0; 0 0 1; -6 -11 -6];
b = [1; 1; 1];
c = [1 1 0];
d = 0;
x0 = [1 0.5 -0.5];
t = 0:0.05:4;
u = ones(1, length(t));
[y,x] = lsim(a, b, c, d, u, t, x0);
plot(t, x, t, y)
```

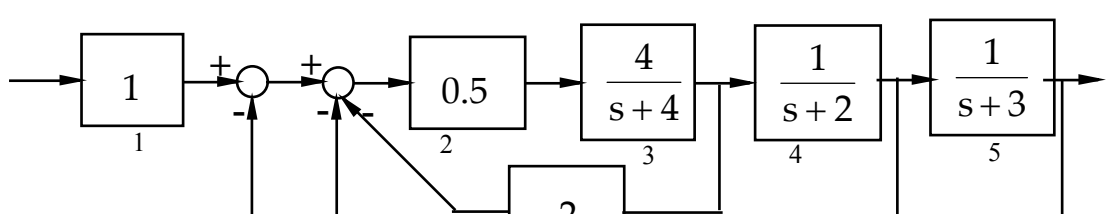
Do điều kiện đầu nên nghiệm y xuất phát từ 1.5

Khi u(t) là sin2πt ta tính đáp ứng như sau(lưu trong ***ctlsim1.m***):

```
a = [0 1 0;0 0 1;-6 -11 -6];
b = [1;1;1];
c = [1 1 0];
d = 0;
x0 = [1 0.5 -0.5];
t = 0:0.05:4;
u = sin(2*pi*t);
[y, x] = lsim(a, b, c, d, u, t, x0);
plot(t, x, t, y)
```

7. Biến đổi sơ đồ khối: Một sơ đồ khối điều khiển thường rất phức tạp. Vì vậy ta thường phải biến đổi nó về dạng đơn giản bằng lệnh **connect**.

Ví dụ: Xét sơ đồ khối sau:



Xác định phương trình trạng thái và hàm truyền của toán bộ sơ đồ:
Gọi ni và di là tử số và mẫu số của hàm truyền của khối thứ i. Ta có các lệnh(lưu trong ***ctconnect.m***):

```
n1 = 1;d1 = 1;
n2 = .5;d2 = 1;
n3 = 4;d3  =[1 4];
n4 = 1;d4 = [1 2];
n5  =1;d5 = [1 3];
n6  =2;d6 = 1;
n7 = 5;d7 = 1;
n8 = 1;d8 = 1;
nblocks = 8;
blkbuild;
q = [1 0 0 0 0
     2 1 -6 -7 -8
     3 2 0 0 0
     4 3 0 0 0
     5 4 0 0 0
     6 3 0 0 0
     7 4 0 0 0
     8 5 0 0 0];
iu = [1];
iy = [5];
[A, B, C, D] = connect(a, b, c, d, q, iu, iy)
```

Kết quả là:

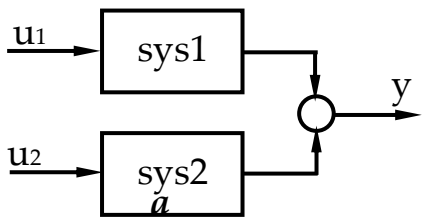
```
A =
  -8.0 -2.5  -0.5
   4.0  -2.0    0
     0    1.0   -3.0
B =
     0.5
      0
      0
C =
```

```
    0    0    1
D =
    0
[ts, ms] = ss2tf(A, B, C, D, 1)
ts =
              0   0   0   2.0
ms =
  1.0 13.0  56.0  80.0
```

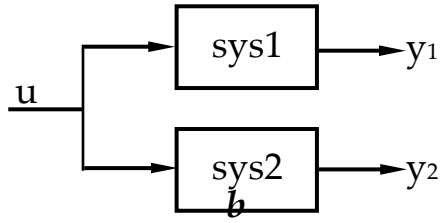
Hàm truyền của hệ là:

$$\frac{C(s)}{R(s)} = \frac{1}{s^3 + 13s^2 + 56s + 80}$$

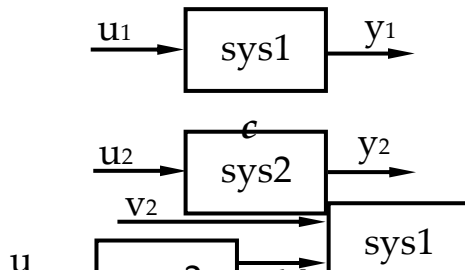
8. Ghép nối các sơ đồ khối: Để ghép nối tạo nên một hệ thống từ nhiều hệ thống con ta có thể sử dụng một số khả năng như sau:



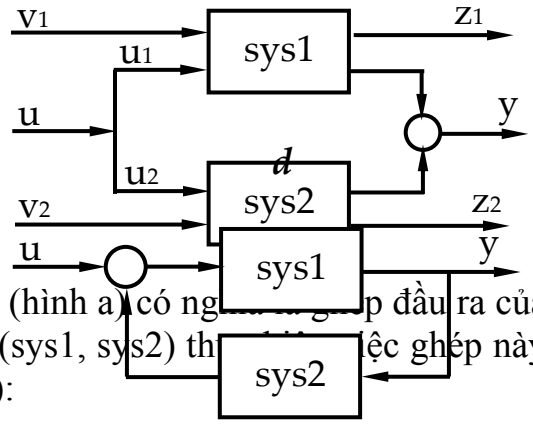
a



b



c



d

e

f

a. Ghép theo hàng: Ghép theo hàng (hình a) có nghĩa là ghép đầu ra của các hệ thống con có đầu vào khác nhau. Hàm sys(sys1, sys2) th[illegible]ệc ghép này. Ta có các lệnh MATLAB sau(lưu trong ***ctrow.m***):

```
clc
sys1 = tf(1,[1 0])
sys2 = ss(1,2,3,4)
sys = [sys1,sys2]
```

b. Ghép theo cột: Ghép theo cột(hình b) có nghĩa là ghép đầu ra của hệ thống con có chung đầu vào. Ta có các lệnh MATLAB sau(lưu trong ***ctcolumn.m***):

```
clc
```

```
sys1 = tf(1, [1 0])
sys2 = ss(1, 2, 3, 4)
sys = [sys1; sys2]
```

c. ***Ghép theo đường chéo***: Khi ghép theo đường chéo(hình c), ta có hệ thống mới bảo đảm cách ly các hệ thống con ban đầu. Để ghép ta dùng lệnh ***append***. Các lệnh MATLAB(lưu trong ***ctdiag.m***) như sau:

```
clc
sys1 = tf(1, [1 0])
sys2 = ss(1, 2, 3, 4)
sys = append(sys1, sys2)
```

d. Ghép song song: Ta dùng cách ghép như trên hình d. Hàm ***parallel*** dùng để ghép song song các hệ thống con. Các lệnh MATLAB (lưu trong ***ctparallel.m***) như sau:

```
clc
sys1 = tf(1, [1 0])
sys2 = ss(1, 2, 3, 4)
sys = parallel(sys1, sys2)
```

e. Ghép tuần tự: Ta dùng cách ghép như trên hình e. Hàm ***series*** dùng để ghép tuần tự các hệ thống con. Các lệnh MATLAB(lưu trong ***ctseries.m***) như sau:

```
clc
sys1 = tf(1,[1 0])
sys2 = ss(1,2,3,4)
sys = series(sys1, sys2)
```

f. Ghép có phản hồi: Ta dùng cách ghép như hình f. Hàm ***feedback*** dùng để ghép có phản hồi các hệ thống con. Các lệnh MATLAB (lưu trong ***ctfeedback.m***) như sau:

```
clc
sys1 = tf(1, [1 0])
sys2 = ss(1, 2, 3, 4)
sys = feedback(sys1, sys2)
```

g. Sử dụng hàm connect: Hàm connect tạo ra mô hình không gian-trạng thái từ các hệ thống con. Cú pháp của hàm:

sysc = connect(sys,Q,inputs,outputs)

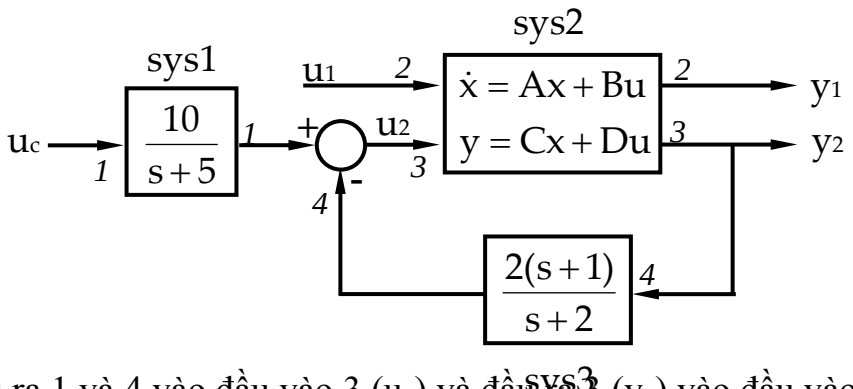
Một hệ thống thường được cho dưới dạng các khối. Ngay cả khi sơ đồ không phức tạp, việc tìm được mô hình không gian-trạng thái của hệ thống khá khó. Để tìm được mô hình không gian-trạng thái, trước hết ta dùng hàm ***append***:

sys = append(sys1, sys2,..., sysN)

để mô tả mỗi hệ thống con sysj hệ thống dạng đường chéo. Tiếp đến dùng lệnh:

sysc = connect(sys, Q, inputs, outputs)

để nối các hệ thống con và rút ra mô hình không gian - trạng thái sysc của toàn bộ hệ thống. Ma trận Q chỉ ra cách nối các hệ thống con trên sơ đồ. Mỗi đầu vào của sys có một hàng, trong đó phần tử đầu tiên của mỗi hàng là số đầu vào. các phần tử tiếp theo của mỗi hàng mô tả đầu vào của hệ thống được lấy từ đâu. Ví dụ đầu vào 7 lấy từ đầu ra 2, 15 và 6 trong đó đầu vào của 15 âm thì hàng tương ứng của Q là [7 2 -15 6]. Hàng nào không đủ phần tử thì thêm số 0. Ta tìm mô hình không gian trạng - thái của sơ đồ sau:



Ta cần nối đầu ra 1 và 4 vào đầu vào 3 (u_2) và đầu ra 3 (y_2) vào đầu vào 4 nên ma trận Q là:

```
Q = [3   1  -4
     4   3   0];
```

Sơ đồ có 2 đầu vào từ các hệ thống khác là u_c và u_1 (đầu vào 1 và 2 của sys) và 2 đầu ra đưa đến các hệ thống khác là y_1 và y_2 (đầu ra 2 và 3 của sys). Như vậy ma trận inputs và outputs là:

```
inputs = [1 2];
outputs = [2 3];
```

Các lệnh MATLAB thực hiện việc biến đổi sơ đồ (lưu trong ***ctconnectsys.m***) như sau:

```
clc
A = [ -9.0201  17.7791
     -1.6943  3.2138 ];
B = [ -.5112  .5362
     -.002  -1.8470];
```

```
C = [ -3.2897  2.4544
     -13.5009  18.0745];
D = [-.5476  -.1410
    -.6459  .2958 ];
sys1 = tf(10,[1 5],'inputname','uc')
sys2 = ss(A,B,C,D,'inputname',{'u1' 'u2'},...
                'outputname',{'y1' 'y2'})
sys3 = zpk(-1,-2,2)
sys = append(sys1,sys2,sys3)
Q = [3 1 -4
    4 3  0];
inputs = [1 2];
outputs = [2 3];
sysc = connect(sys,Q,inputs,outputs)
```

9. Đáp ứng của hệ thống bậc hai: Dạng chuẩn của hàm truyền của hệ thống bậc hai là:

$$G(s) = \frac{1}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

Trong đó ω_n là tần số tự nhiên và ζ là hệ số tắt của hệ thống. Để tạo ra hàm truyền này khi biết ω_n và ζ ta dùng lệnh .

Ví dụ: Tìm hàm truyền và ma trận trạng thái của hệ thống bậc hai biết ω_n = 2.4 rad/s và ζ = 0.4. Các lệnh MATLAB (lưu trong ***ctord2.m***) như sau:

```
[ts, ms] = ord2(2.4, 0.4)
 [a, b, c, d] = ord2(2.4, 0.4)
```

Đáp ứng thực tế của hệ là một dao động tắt dần có dạng:

$$c(t) = 1 - \frac{1}{\beta} e^{\zeta\omega_n t} \sin(\beta\omega_n t + \theta)$$

Trong đó $\beta = \sqrt{1-\zeta^2}$ và $\theta = \tan^{-1}(\beta/\zeta)$

Ta gọi tr là thời gian để dáp ứng đạt từ 10% giá trị cuối đến 90% giá trị cuối; thời gian đạt đến đỉnh là tp; độ nhanh đo bằng tr và tp; thời gian tắt là ts. Thời gian đạt đến định được xác định bằng cách cho đạo hàm của c(t) bằng 0.

$$t_p = \frac{\pi}{\omega\sqrt{1-\zeta^2}} \qquad (4.1)$$

Giá trị đỉnh (percent overshoot-p.o)khi kích thích là bước nhảy là:

$$p.o = e^{\zeta\pi\sqrt{1-\zeta^2}} \times 100 \qquad (4.2)$$

Đáp ứng với kích thích bước nhảy tìm được nhờ hàm ***step*** còn đáp ứng với kích thích xung tìm được nhờ hàm ***impulse***

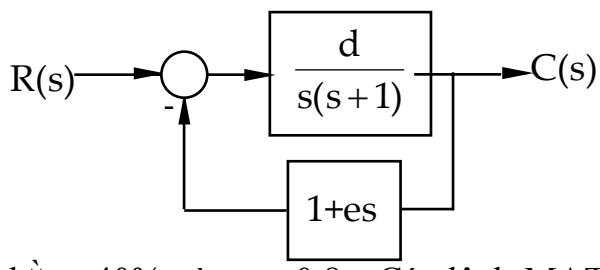
Ví dụ 1: Tìm đáp ứng của khâu bậc hai có hàm truyền :

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

khi $\omega_n = 5$ và $\zeta = 0.6$. Các lệnh MATLAB (lưu trong ***ctstep.m***) như sau:

```
clc
ts = 25;
ms = [1 6 25];
sys = tf(ts ,ms)
t = 0:0.02:2;
c = step(sys, t);
plot(t, c)
xlabel('t(s)');
ylabel('c(t)');
```

Ví dụ 2: Cho hệ có sơ đồ như hình vẽ:



Tìm d và e để p.o bằng 40% và tp = 0.8s. Các lệnh MATLAB (lưu trong ***ctstep1.m***) như sau:

```
clc
po = 40;
z = log(100/po)/sqrt(pi^2+(log(100/po))^2)%theo (4-2)
zn = 0.27999799333504
tp = 0.8;
wn = pi/(tp*sqrt(1-z^2))% theo (4-1)
ts = wn^2;
ms = [1 2*z*wn  wn^2];
sys = tf(ts, ms);
t = 0:0.02:4;
c = step(sys, t);
plot(t,c)
```

Từ sơ đồ khối ta có:

$$\frac{C(s)}{R(s)} = \frac{d}{s^2 + (de+1)s + d}$$

Phương trình đặc tính là:

$$s^2 + (de + 1)s + d = s^2 + 2\omega_n\zeta s + \omega_n^2$$

Với ω_n^2 = wn = 0.28 và z = ζ = 4.0906 ta có d = 16.733 và e = 0.077

Khi có một hàm truyền ta có thể xác định hệ số tắt ζ và tần số tự nhiên ω_n bằng lệnh ***damp***.

Ví dụ 3: Cho hệ có hàm truyền:

$$H(s) = \frac{2s^2 + 5s + 1}{s^2 + 2s + 3}$$

Tìm hệ số tắt ζ và tần số tự nhiên ω_n. Các lệnh MATLAB (lưu trong ***ctdamp.m***) như sau:

```
h = tf([2 5 1], [1 2 3]);
damp(h)
```

Kết quả là:

Eigenvalue	*Damping*	*Freq. (rad/s)*
-1.00e+000 + 1.41e+000i	*5.77e-001*	*1.73e+000*
-1.00e+000 - 1.41e+000i	*5.77e-001*	*1.73e+000*

10. Đáp ứng trong miền thời gian của hệ thống:

a. Đáp giá trị ban đầu: Đáp ứng giá trị ban đầu mô tả phản ứng của hệ khi không có kích thích dầu vào nhưng tồn tại các giá trị ban đầu của vec tơ trạng thái $\mathbf{x}_0$. Phản ứng đó được gọi là chuyển động tự do của hệ. Đáp ứng này được xác định bằng hàm ***initial***. Ta có các lệnh MATLAB tìm đáp ứng ban đầu của một hệ thống (lưu trong ***ctinitial.m***)như sau:

```
clc
a = [-0.5572   -0.7814;0.7814  0];
c = [1.9691  6.4493];
x0 = [1 ; 0]
sys = ss(a, [], c, []);
initial(sys, x0)
```

b. Đáp ứng xung Dirac: Ta tìm đáp ứng của hệ thống với xung nhờ hàm ***impulse***. Các lệnh MATLAB (lưu trong ***ctimpulse.m***)như sau:

```
clc
a = [-0.5572 -0.7814; 0.7814  0];
b = [1 -1; 0 2];
c = [1.9691  6.4493];
sys = ss(a, b, c, 0);
```

```
impulse(sys)
```

Hình bên trái là đáp ứng của kênh thứ nhất và hình bên phải là đáp ứng của kênh thứ 2.

c. Đáp ứng đối với hàm bước nhảy: Để tìm đáp ứng của hệ thống đối với hàm bước nhảy ta dùng hàm ***step***. Các lệnh MATLAB (lưu trong ***ctstep2.m***) như sau:

```
clc
a = [-0.5572   -0.7814;0.7814  0];
b = [1 -1;0 2];
c = [1.9691  6.4493];
sys = ss(a, b, c, 0);
step(sys)
```

d. Đáp ứng với tín hiệu bất kỳ: Để tìm đáp ứng của hệ thống đối với hàm bất kì ta dùng hàm ***lsim***. Các lệnh MATLAB (lưu trong ***ctlsim.m***) như sau:

```
clc
[u, t] = gensig('square', 4, 10, 0.1);
H = [tf([2 5 1], [1 2 3]) ; tf([1 -1], [1 1 5])]
lsim(H, u, t)
```

Ta dùng hàm gensig để tạo một xung hình vuông, trong 4 chu kỳ và lấy mẫu sau 0.1s trong 10 chu kỳ.

11. Đáp ứng trong miền tần số của hệ thống: Cho một hàm truyền của một hệ thống,thay s bằng jω ta có hàm truyền đạt tần số của hệ thống đó. Độ rộng băng của hệ thống ω_B là tần số mà tại đó biên độ của g giảm đi 1/√2. Tần số ứng với giá trị max của G(ω) gọi là ω_r và có trị số là:

$$\omega_r = \omega_n\sqrt{1-2\zeta^2}$$

Để vẽ đặc tính tần biên-pha của một hệ thống ta dùng lệnh ***freqs***.

Ví dụ: Cho hàm truyền của một hệ thống là:

$$G(s) = \frac{4}{s^2+2s+4}$$

Tìm đặc tính tần biên-pha của hệ thống bằng các lệnh MATLAB(lưu trong ***ctfreqs.m***):

```
w = 0:0.01:3;
ms = [1 2 4];
ts = [4];
```

```
freqs(ts, ms, w);
```

Ta cũng có thể tạo đồ thị như sau(lưu trong ***ctfreqplot.m***):

```
ts = [4];
ms = [1 2 4];
w = 0:0.01:3;
g = freqs(ts, ms, w);
mag = abs(g);
pha = angle(g);
subplot(2, 1, 1);
loglog(w, mag);
grid on;
subplot(2,1,2);
semilogx(w, pha);
grid on
```

Ngược lại khi có đặc tính tần biên - pha ta có thể tìm lại được hàm truyền bằng lệnh ***invfreqs***.

Ví dụ: Tìm hàm truyền của hệ thống(lưu trong ***ctinvfreqz.m***):

```
ts = [1 2 3 2 1 4];
ms = [1 2 3 2 3];
[h, w] = freqz(b, a, 64);
[tsm, msm] = invfreqz(h, w, 4, 5)
```

Ta cũng có thể xây dựng đặc tính tần thực-ảo

Ví dụ: Cho hàm truyền :

$$G(s) = \frac{10}{s^3 + 4.5s^2 + 9s + 10}$$

Tìm đặc tính tần thực - ảo của hệ bằng các lệnh MATLAB (lưu trong ***ctfreqsplot.m***):

```
ts = [10];
ms = [1 4.5 9 10];
w = [1:0.01:3];
h = freqs(ts, ms, w);
t = real(h);
a = imag(h);
subplot(2, 1, 1);
plot(w, t)
subplot(2, 1, 2);
plot(w, a)
```

Để vẽ đồ thị Bode của hệ thống ta dùng hàm ***bode***. Đồ thị thứ nhất nhất là đặc tính biên-tần logarit, được chia theo dB. Đồ thị thứ hai là đặc tính pha- tần logarit chia theo độ.

Các dạng của lệnh ***bode*** gồm:

bode(sys)
bode(sys,w)
[bien, pha, w] = bode(sys)

Để vẽ đồ thị Bode của một hệ thống ta dùng các lệnh MATLAB(lưu trong ***ctbode.m***) như sau:

```
clc
g = tf([1 0.1 7.5], [1 0.12 9 0 0]);
figure(1)
bode(g)
figure(2)
bode(g, {0.1 , 100})
gd = c2d(g, 0.5)
figure(3)
bode(g, 'r', gd, 'b--')
```

Hàm ***margin*** cho biết dự trữ ổn định của hệ thống. Dự trữ biên gm là hệ số khuyếch đại F_r mà nếu ta thêm vào hàm truyền đạt của hệ hở thì hệ kín vừa đạt được giới hạn ổn định. Dự trữ pha pm được định nghĩa là khoảng cách góc pha φ_r tới -180°. Hàm cho biết gm tại tần số đảo pha wcg và pm tại tần số cắt pha wcp. Hàm ***allmargin*** có tác dụng rộng hơn hàm ***margin***. Các kết quả trả về của ***allmargin*** gồm:

GMFrequency: giá trị tần số mà tại đó đồ thị pha cắt đường thẳng nằm ngang -180°

GainMargin: dự trữ biên - giá trị đảo của biên độ tại tần số GMFrequency

PMFrequency: giá trị tần số mà tại đó đồ thị biên cắt đường thẳng nằm ngang 0 dB(ứng với hệ số khuyếch đại 1)

PhaseMargin: dự trữ pha - khoảng cách góc (> 0) từ vị trí PMFrequency đến -180°.

DelayMargin: dự trữ thời gian trễ - giá trị thời gian trễ mà nếu vượt quá, hệ thống sẽ mất ổn định.

DMFrequency: giá trị tần số ứng với DelayMargin.

Stable: =1 khi mach vòng kín ổn định; bằng 0 trong các trường hợp khác.

Các đại lượng này có thể đọc được từ đồ thị tạo bởi ***margin***. Để xác định dự trữ ổn định của một hệ thống cụ thể ta dùng các lệnh MATLAB(lưu trong ***ctmatgin6_32.m***) như sau:

```
clc
sys = zpk([], [-1 -1 -1], 4)
margin(sys)
allmargin(sys)
```

Kết quả hệ thống ổn định. Nó có DelayMargin = 0.3s. Bây giờ ta gán cho sys một khoảng thời gian trễ là stabil.DelayMargin + 0.01, nghĩa là vượt quá thời gian trễ ổn định 0.01s. Kết quả tính toan mới của ***allmargin*** sẽ thông báo tính không ổn định của hệ thống. Các lệnh MATLAB (lưu trong ***ctnewstabil6_33.m***) như sau:

```
clc
sys = zpk([], [-1 -1 -1], 4)
margin(sys)
stabil = allmargin(sys)
sys.ioDelay = stabil.DelayMargin + 0.01;
newstabil = allmargin(sys)
```

Một khả năng khác để mô tả đặc tính tần số là đồ thị Nyquist. Nó biểu diễn các giá trị thực và ảo thuộc hàm truyền đạt phức của mạch vòng hở $F_0(j\omega)$ trong dải tần số $\omega = 0 \div \infty$ trên hệ toạ độ phức. Đường cong do các điểm tạo thành được gọi là quỹ đạo biên - pha $F_0(j\omega)$. Trên cơ sở tiêu chuẩn ổn định Nyquist ta có thể rút ra kết luận về tính ổn định của hệ kín(có phản hồi đơn vị âm) từ đồ thị Nyquist. Để vẽ đồ thị Nyquist ta dùng hàm ***Nyquist***. Ta có các lệnh MATLAB(lưu trong ***ctnyquist6_34.m***) như sau:

```
clc
H = tf([2 5 1], [1 2 3])
nyquist(H)
```

12. Tính ổn định: Tiêu chuẩn ổn định nói rằng hệ sẽ ổn định nếu các nghiệm của phương trình đặc tính có phần thực âm. Phương trình đặc tính là đa thức mẫu số của hàm truyền. Do vậy chỉ cần tính nghiệm của đa thức đặc tính bằng lệnh ***roots*** là ta có thể xác dịnh hệ ổn định hay không.

Ví dụ: Xét tính ổn định của hệ có phương trình đặc tính là:

$$s^4 + 10\,s^3 + 35s^2 + 50s + 24$$

Các lệnh MATLAB là:

```
a = [1 10 35 50 24];
roots(a)
ans =
  -4.0000
  -3.0000
  -2.0000
  -1.0000
```

Như vậy hệ ổn định.

13. Độ nhạy: Độ nhạy của hệ thống được đo bằng tỉ số phần trăm sự thay đổi của hàm truyền theo sự thay đổi phần trăm của thông số b. Ví dụ độ nhạy của hàm truyền T(s) theo b được xác định bằng:

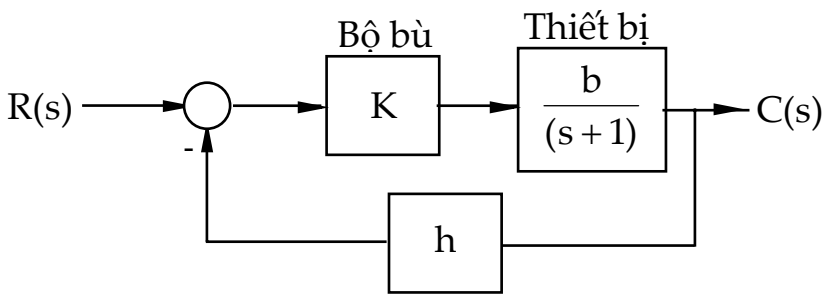
$$S_b^T = \frac{\Delta T(s)/T(s)}{\Delta b/b} = \frac{\Delta T(s)}{\Delta b}\frac{T(s)}{b}$$

Khi Δb gần đến 0 ta có:

$$S_b^T = \frac{\partial T(s)}{\partial b}\frac{b}{T(s)}$$

Độ nhạy tĩnh là giá trị của S khi t→0. Độ nhạy động được tính bằng cách thay s bằng jω và vẽ đường S theo ω. Biên độ của S(jω) đo sai số của hệ thống.

Ví dụ: Khảo sát hệ điều khiển như hình vẽ sau:



Trong đó b có trị định mức là 4 và h có trị định mức là 0,5. Tìm độ nhạy T(s) theo b, vẽ modul hàm độ nhạy theo ω với hai giá trị bù là K = 2 và K = 0.5. Tìm độ nhạy T(s) theo h, vẽ modul của hàm độ nhạy theo h với K = 2 và K = 0.5.

Hàm truyền của hệ thống là:

$$(Ts) = \frac{Kb}{s^2 + 1 + Kbh}$$

Với b = 4 và h = 0.5 ta có $\omega_B = 1 + 2K$.

Độ nhạy của T(s) theo b khi b = 4 và h = 0.5 là:

$$S_b^T = \frac{\partial T(s)}{\partial b}\frac{b}{T(s)} = \frac{s+1}{s+1+Kbh} = \frac{s+1}{s+1+2K}$$

$$S_h^T = \frac{\partial T(s)}{\partial b}\frac{h}{T(s)} = \frac{-Kbh}{s+1+Kbh} = \frac{-2K}{s+1+2K}$$

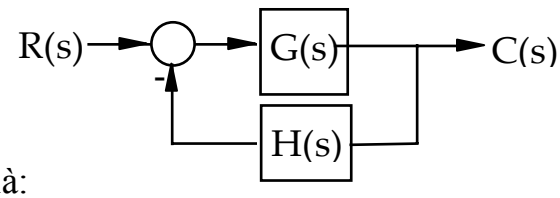
Các lệnh MATLAB (lưu trong ***ctsensibility.m***) như sau:

```
k1 = 1;
k2 = 0.5;
ts = [1 1];
ms1 = [1 1+2*k1];
ms2 = [1 1+2*k2];
w = 0:0.01:15;
stb1 = abs(freqs(ts, ms1, w));
```

```
stb2 = abs(freqs(ts, ms2, w));
subplot(2, 1, 1);
plot(w, stb1, w, stb2);
title('Do nhay cua T theo b');
ts1 = -2*k1;
ts2 = -2*k2;
stb1 = abs(freqs(ts1, ms1, w));
stb2 = abs(freqs(ts2, ms2, w));
subplot(212);
plot(w, stb1, w, stb2);
title('Do nhay cua T theo h');
```

Độ nhạy của hệ thống theo b giảm khi hệ số khuếch đại của vòng hở K tăng trong khi độ nhạy theo h tăng khi K tăng. Rõ ràng là độ nhạy theo b tăng nhanh bên ngoài ω_B.

14. Sai số xác lập: Khảo sát hệ như hình vẽ:



Hàm truyền của hệ kín là:

$$\frac{C(s)}{R(s)} = \frac{G(s)}{1 + H(s)G(s)}$$

Sai số của hệ kín là:

$$E(s) = R(s) - H(s)C(s) = \frac{R(s)}{1 + H(s)G(s)}$$

Sử dụng định lí giá trị cuối ta có:

$$e_{ss} = \lim_{s \to \infty} \frac{sR(s)}{1 + G(s)H(s)}$$

Đầu vào bước nhảy đơn vị:

$$e_{ss} = \frac{1}{1 + \lim_{s \to \infty} G(s)H(s)} = \frac{1}{1 + K_p}$$

Đầu vào tăng tuyến tính đơn vị:

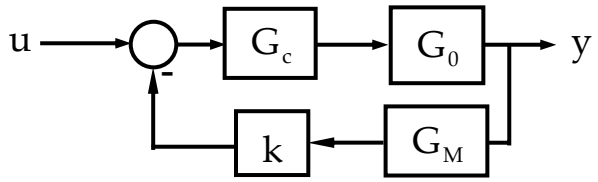
$$e_{ss} = \frac{1}{1 + \lim_{s \to \infty} sG(s)H(s)} = \frac{1}{K_v}$$

Đầu vào parabol đơn vị:

$$e_{ss} = \frac{1}{1 + \lim_{s \to \infty} s^2G(s)H(s)} = \frac{1}{K_a}$$

Ta có thể dùng Symbolic Math để tính các giới hạn trên.

15. Phân tích và thiết kế quỹ đạo nghiệm: Phương pháp kinh điển để tham số hoá khâu điều khiển của vòng điều hỉnh là phương pháp quỹ đạo nghiệm. Quỹ đạo nghiệm là quỹ đạo điểm cực, hợp thành bởi các điểu cực của hệ thống, phụ thuộc vào hệ số khuyếch đại phản hồi k va được biểu diễ trên mặt phẳng phức với phần thưc $Re(\lambda) = \sigma$ trên trục hoành x và phần ảo $Im(\lambda) = \omega$ trên trục tung y. Để vẽ được quỹ đạo nghiệm của hệ thống ta dung hàm ***rlocus***. Ta xét hệ thống sau:



Cú pháp của rlocus là

rlocus(sys[,k])

[r, k] = rlocus(sys)

r = rlocus(sys, k)

Mô hình sys trong lệnh trên là hàm truyền đạt của hệ thống hở $G_oG_cG_M$ được xác định bằng lệnh MATLAB:

*sys = sysM*sysO*sysC*

mà chưa có hệ số khuyếch đại phản hồi k, là tham số tuỳ chọn sẽ được khai báo riêng. Điều đó có nghĩa là sys được ghép nối bởi các mô hình riêng lẻ. Khi gọi ***rlocus(sys[, k])*** mà không yêu trả biến về ta nhận được đồ thị quỹ đạo nghiệm của sys. Nếu ta không khai báo các hệ số khuyêch đại trong vec tơ tham số tuỳ chọn k, MATLAB sẽ tự động quyết định giá trị thích hợp. Sau khi dùng ***rlocus*** vẽ quỹ đạo điểm cực ta tìm các giá trị liên quan đến điểm cực bất kì năm tên quỹ đạo bằng cách nhấp chuột vào một điểm trên quỹ đạo. Lúc đó lệnh ***rlocusfind*** được thực hiện. Ta dùng các lệnh MATLAB sau (lưu trong ***ctrlocus.m***)để vẽ quỹ đạo nghiệm của một hệ thống:

```
clc
sys = zpk([],[-0.1 -1-j -1+j ], 1)
rlocus(sys)
[r, k] = rlocus(sys)
sgrid
```

Để trực quan ta có thể dùng công cụ thiết kế bằng cách nhập lệnh ***sisotool*** vào cửa sổ lệnh MATLAB.

LÝ LỊCH

I. LÝ LỊCH KHOA HỌC

Họ và tên: Trần Văn Chính Giới tính: nam
Ngày, tháng, năm sinh: 19 - 08 - 1954
Dân tộc: Kinh
Quốc tịch: Việt Nam
Tốt nghiệp đại học năm: 1977 tại Trường Đại học Bách khoa Hà Nội
Chuyên ngành: Máy điện
Nơi sinh: Lam Kiều, Can Lộc, Hà Tĩnh.
Học vị: Tiến sĩ kỹ thuật Học hàm: Phó giáo sư
Lĩnh vực nghiên cứu: Tối ưu hoá máy điện
Địa chỉ lien hệ: 54 Nguyễn Lương Bằng, Đà Nẵng
Điện thoại: (0511)842458
E-mail: tranvanchinh54@yahoo.com

II. CÁC CÔNG TRÌNH KHOA HỌC

1. Các đề tài khoa học:

TT	Tên đề tài, mã số	Kết quả nghiên cứu	Năm	Ghi chú
1	B94-15-05 Xây dựng chương trình đào tạo giai đoạn 2 cho khoa Điện	Khá	1996	
2	B96-15-09. Tính toán trường điện từ trong máy điện không đồng bộ	Tốt	1998	
3	B01-15-05 Đánh giá chất lượng máy điện không đồng bộ	Tốt	2002	

2. Các bài báo:

TT	Tên bài báo	Nơi đăng	Năm	Ghi chú
1	Parametergewinnung durch Untersuchung der thermischen Bestaendigkeit von Lackdraetn	10.Internationale Fachtagung	1988	
2	Ueberwachung des Isolationspegel eines	10.Internationale	1988	

	Asynchronmotor durch Ableitsstrom-Registierung	Fachtagung		
3	Ermitteilung der hoechstzulaessigen Temperaturen fuer Wickeldraehten	10.Internationale Fachtagung	1988	
4	Về tuổi thọ cơ học của sơn cách điện ở các dây dẫn có tráng sơn cách điện	Tập san KHKT Đại học Bách Khoa Đà Nẵng 12/89	1989	
5	Xác định hư hỏng nhờ phân tích dòng điện	Tập san KHKT Đại học Bách Khoa Đà Nẵng 1/95	1995	
6	Góp phần tính phân bố dòng áp trên cuộn dây máy biến áp trong quá trình quá độ	Tập san KHKT Đại học Bách Khoa Đà Nẵng 2/95	1995	
7	Tính toán từ trường trong máy điện không đồng bộ	Tạp chí Khoa học và Công nghệ ĐHĐN số 6,8/99	1999	
8	Tính toán phân bố dòng điện trong thanh dẫn động cơ không đồng bộ trong quá trình mở máy	Tạp chí Khoa học và Công nghệ ĐHĐN số 7/2000	2000	
9	Đánh giá chất lượng của máy điện không đồng bộ	Tạp chí Khoa học và Công nghệ 4 trường ĐH số	2002	

		34+35/2002		
10	Khảo sát sự thay đổi điện trở cách điện của máy điện theo nhiệt độ	Tạp chí Khoa học và Công nghệ 4 trường ĐH số 36+372002	2002	
11	Khảo sát các thông số của động cơ điện không đồng bộ khi thực hiện điều khiển bằng biến tần	Tạp chí Khoa học và Công nghệ 4 trường ĐH số 40+41/2003	2003	
12	Khảo sát các quá trình nhiệt trong cuộn dây máy biến áp bằng phương pháp phần tử hữu hạn	Tạp chí Khoa học Công nghệ Đại học Đà Nẵng sô 1/2003	2003	
13	Khảo sát ảnh hưởng của phụ tải đến quá trình mở máy động cơ không đồng bộ	Tạp chí Khoa học và Công nghệ 4 trường ĐH số 44+45/2003	2003	
14	Tính toán chuyển đổi máy biến áp có cấp điện áp 6,10 và 15/0.4 kV thành 22/0.4 kV	Tạp chí Khoa học Công nghệ Đại học Đà Nẵng sô 3/2003	2003	

3. Sách, giáo trình đã xuất bản:

TT	Tên sách	Nhà xuất bản	Năm	Ghi chú
1	Máy điện	Giáo dục	1999	
2	Tin học ứng dụng trong tính toán	Giáo dục	2001	